Transformers

transformers-docs-zh

涵盖 108 篇教程 · 任务指南 · 开发者指南 · 量化方法 · 性能优化 · 概念指南

内容基于 🤗 Hugging Face Transformers 官方文档的中文学习笔记与演示示例

来源：https://github.com/Airmomo/transformers-docs-zh

目 录

前言 8

第一部分 导言 8

安装 Transformers（Windows & macOS） 8

使用 pip 安装 8

创建和激活虚拟环境 9

安装 🤗 Transformers 9

安装深度学习库 9

Transformers 快速上手 13

Pipeline 13

Auto Class 17

自定义模型构建 21

Trainer - PyTorch 优化训练循环 22

使用 Tensorflow 训练 24

接下来做什么 25

第二部分 教程 25

使用 pipelines 进行推理 25

使用 AutoClass 加载预训练实例 35

使用 AutoClass 加载预训练实例 36

预处理数据 39

微调预训练模型 49

使用脚本进行训练 58

使用 Accelerate 进行分布式训练 65

使用 PEFT 加载和训练 Adapters 68
加载 PEFT adapter 模型 70
如何分享模型 73
Transformers Agents 快速上手 79
使用 LLMs 进行生成 84
Agents and Tools 介绍和指南 90
第三部分 任务指南 100
自然语言处理 100
文本分类 100
标记分类（实体分类） 106
因果语言模型（CLM） 115
遮蔽语言模型（MLM） 121
文本翻译 128
文本摘要（文本总结） 134
问题解答任务（问答任务） 138
多项选择任务 145
音频处理 150
音频分类 150
自动语音识别（ASR） 155
计算机视觉 161
图像分类 161
图像分割 170
视频分类 183
目标检测 192

零样本目标检测 204
零样本图像分类 209
单目深度估计（单图像深度估计） 211
以图生图（图像增强、图像修复等图像处理任务） 215
图像特征提取 218
图像掩码生成 220
关键点检测（图像特征点检测） 225
知识蒸馏在计算机视觉中的应用 227
多模态 231
图像描述生成 231
文本转语音（TTS） 237
图像视觉多模态理解模型（Image-text-to-text） 247
视频视觉多模态理解模型（Video-text-to-text） 252
文档问答（DQA） 255
视觉问答（VQA） 265
生成策略 271
（自定义）文本生成策略 271
使用缓存优化生成的最佳实践 283
提示技术 293
使用 IDEFICS 大型多模态模型解决图像-文本任务 293
LLM 提示指南 302
第四部分 开发者指南 311
使用 Tokenizers 中的分词器 311
使用多语言模型运行推理 312

创建自定义架构（模型架构） 316
创建自定义模型 324
聊天模版（Chat Templates） 331
Trainer 类（完整实现 PyTorch 训练与评估循环） 351
将模型导出为 ONNX 格式 365
将模型导出至 TFLite 369
导出到 TorchScript 370
基准测试 374
Transformers Notebooks 记事本示例合集 378
Transformers Community 社区与资源 407
Transformers 故障排除 419
在 Transformers 中加载 GGUF 文件 422
在 Transformers 中加载 Tiktoken 文件 424
模块化 Transformers 425
如何修改 Transformers 模型 430
第五部分 量化方法 434
量化技术概述 434
量化模型（8 位与 4 位量化） 437
GPTQ —— 广义剪枝训练量化 443
AWQ —— 激活感知权重量化 446
AQLM —— 加性量化语言模型 450
Optimum-quanto —— PyTorch 多功能量化工具包 452
EETQ —— 高效的 Tensor 量化方法 453
HQQ —— 半二次量化 454

FBGEMM FP8（权重 8 位、激活 8 位的模型量化） 455
Optimum 456
TorchAO —— 用于 PyTorch 的架构优化库 456
BitNet —— 新型的神经网络架构 459
压缩张量 461
添加新的量化方法 466
第六部分 性能优化与可扩展性 468
性能优化概述 468
LLM 推理优化 469
创建大型模型实例（如何加载大型模型） 477
调试（解决训练过程中的问题） 481
TensorFlow 模型中的 XLA 集成 486
使用 torch.compile() 优化推理速度 490
高效训练方法 501
在单个 GPU 上进行高效训练的方法和工具 501
在多个 GPU 上进行高效训练的方法和工具 513
FSDP —— 完全分片数据并行方法 528
DeepSpeed —— 分布式高效训练 PyTorch 模型优化库 531
在单个 CPU 上高效训练 547
在多个 CPU 上高效训练 549
使用 TensorFlow 在 TPU 上进行训练 554
使用 PyTorch 在 Apple 芯片上训练模型 559
使用自定义的硬件进行训练 560
使用 Trainer API 进行超参数搜索 563

优化推理方法（优化生成方法） 566
通过 CPU 优化推理 566
通过多个 GPU 优化推理 569
通过 GPU 优化推理 570
第七部分 概念指南 577
哲学理念 577
专业术语 579
Transformers 库能做什么 592
Transformers 库解决了哪些问题 600
Transformer 模型家族 609
Tokenizer —— 文本分词器概述 614
Attention —— 注意力机制 620
Padding & Truncation —— 填充与截断 621
BERTology —— 基于 BERT 进行的相关研究 624
Perplexity —— 困惑度 625
使用 pipelines 作为 Web 服务器提供推理服务 628
模型训练剖析 631
优化 LLM 部署的有效技术 636

前言

本书为 transformers-docs-zh 中文学习笔记集，共收录 108 篇 Jupyter Notebook（导言 2 篇、教程 11 篇、任务指南 32 篇、开发者指南 17 篇、量化方法 14 篇、性能优化与可扩展性 19 篇、概念指南 13 篇）。

内容以 🤗 Hugging Face Transformers 官方文档为蓝本翻译整理，在官方示例基础上补充了运行中遇到的问题与解决方案，并对重要函数与参数做了更详细的解释。本书保留每篇笔记的说明文字、完整代码单元与必要的运行输出/插图，离线可直接阅读。

说明：仓库中以 Git LFS 方式存放的插图已尽量还原嵌入；少数引用自 Hugging Face 在线图床的图片以及演示用视频、音频等媒体资源因离线环境无法获取，在对应位置以文字说明标注，可在联网后对照原仓库查看。

第一部分 导言

安装 Transformers（Windows & macOS）

这里主要说明如何在 MacOS 或 Windows 系统中，创建和激活 python 虚拟环境，并使用 pip 安装 🤗 Transformers 和深度学习等必要的库。

在实测过程中发现 3.10 和 3.11 由于一些示例使用的库不支持高于 3.9 版本的 Python，会导致运行示例时发生错误（比如某个函数调用错误）。

所以这里推荐安装 Python 3.9 版本，如果你已经安装了 3.9 版本的 Python 但在运行示例时仍出现错误，可以尝试下载和我相同的 Python 版本（3.9.6）

使用 pip 安装

需要使用最新版本的 pip。

```
pip3 install --upgrade pip
```

创建和激活虚拟环境

1. 首先，在项目目录中创建虚拟环境

```
python -m venv .env
```

2. 激活虚拟环境

在 MacOs 系统中激活虚拟环境

```
source .env/bin/activate
```

在 Windows 系统中激活虚拟环境

```
.env\Scripts\activate
```

安装 🤗 Transformers

3. 激活后安装 🤗 Transformers 等必要的库

```
pip3 install transformers datasets evaluate accelerate tf-keras
```

4. 运行以下命令以检查 🤗 Transformers 是否已被正确安装。

该命令将下载一个预训练的情感分析模型，输出结果将是一个包含情感分析结果的字典。

2024.09.25: 我使用 Mac OS 系统在 Python 虚拟环境中无法正常运行示例，在非虚拟环境中却可以正常运行。如果你也存在相同的问题，可以尝试在非虚拟环境中安装 Transformers 库再运行示例，如有解决办法还请麻烦告知我一下。

```
python -c "from transformers import pipeline; print(pipeline('sentiment-analysis',
device=0)('we love you'))"
```

安装深度学习库

Pytorch

MacOS（M1/ARM）

环境要求：

- MacOS 版本： MacOS 10.15 (Catalina) 或更高版本。
- Python 版本：Python 3.8 - 3.11。

```
这里推荐安装 Python 3.9 ，因为在即将发布的 PyTorch 版本中，对 Python 3.8 的支持将被弃用。

pip3 install torch torchvision torchaudio
```

测试下你的版本是否支持 Apple Silicon GPU 加速?

```
import torch
# print(f"PyTorch version: {torch.__version__}")

# Check PyTorch has access to MPS (Metal Performance Shader, Apple's GPU architecture)
print(f"Is MPS (Metal Performance Shader) built? {torch.backends.mps.is_built()}")
print(f"Is MPS available? {torch.backends.mps.is_available()}")

# Set the device
device = "mps" if torch.backends.mps.is_available() else "cpu"
print(f"Using device: {device}")
```

如果运行后和下面的输出一致，说明支持 Apple Silicon GPU 加速：

```
Is MPS (Metal Performance Shader) built? True
Is MPS available? True
Using device: mps
```

Windows

环境要求：

- Windows 版本：建议使用 Windows 10 或更高版本。
- Python 版本：Python 3.8 - 3.11。

这里推荐安装 Python 3.9 ，因为在即将发布的 PyTorch 版本中，对 Python 3.8 的支持将被弃用。

如果你的 Windows 系统存在 NVIDIA GPU，在安装 PyTorch 时可以选择支持 CUDA 计算的版本，但这不是必需的。

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

若没有支持 CUDA 的系统或不需要 CUDA，则可以选择安装仅 CPU 支持的版本。

```
pip3 install torch torchvision torchaudio
```

验证安装

为了确保 PyTorch 安装正确，我们可以通过运行 PyTorch 示例代码来验证安装。

这里我们使用 PyTorch 构造一个随机初始化的张量。

```
import torch
x = torch.rand(5, 3)
print(x)
```

TensorFlow 2.0

MacOS（M1/ARM）

环境要求：

- MacOS 版本： MacOS 10.12.6 (Sierra) 或更高版本（不支持 GPU）。
- Python 版本：Python 3.9 – 3.12。

目前 TensorFlow 对 MacOS 操作系统尚无官方 GPU 支持。

```
pip3 install tensorflow
```

Windows

环境要求：

- Windows 版本：建议使用 Windows 10 或更高版本。

注意：Windows 10 19044 或更高版本上的 WSL 2 支持带 GPU 访问的 TensorFlow。这对应于 Windows 10 版本 21H2，即 2021 年 11 月更新。

- Python 版本：Python 3.8 - 3.11。

这里推荐安装 Python 3.9 ，因为在即将发布的 PyTorch 版本中，对 Python 3.8 的支持将被弃用。

5. 安装 WSL 2

现在，可以使用单个命令安装运行 WSL 所需的一切内容。

在管理员模式下打开 PowerShell 或 Windows 命令提示符，方法是右键单击并选择“以管理员身份运行”，输入 `wsl --install` 命令，然后重启计算机。

注意：仅当根本没有安装 WSL 时，上述命令才有效。

若要卸载 WSL，请参阅 卸载旧版 WSL 或 注销或卸载 Linux 发行版。

```
wsl --install
```

此命令将启用运行 WSL 并安装 Linux 的 Ubuntu 发行版所需的功能。

6. 安装 NVIDIA GPU 驱动程序（如果没有安装过）。

如果你只在 CPU 上运行 TensorFlow，则可以跳过这一步骤。

你可以使用以下命令来验证它是否已安装。

```
nvidia-smi
```

7. 安装 Tensorflow

- 如果你想在 GPU 上运行 Tensorflow

```
pip3 install tensorflow[and-cuda]
```

使用 TensorFlow 库可以列出当前系统上可用的 GPU 设备。输出结果取决于你的系统配置，是否有可用的 GPU 设备以及 TensorFlow 是否能够识别这些设备。

- 如果有可用的 GPU 设备，则可能会输出：

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

- 如果没有可用的 GPU 设备，则会输出空列表：

```
[]
python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

- 如果你只在 CPU 上运行 TensorFlow

注意：

从 **TensorFlow 2.10** 开始，x86/x64 处理器的 Windows CPU 构建由第三方构建、维护、测试和发布：英特尔。安装 Windows 原生***tensorflow** 或 **tensorflow-cpu** 包将安装英特尔的 **tensorflow-intel** 包。这些软件包按原样提供。

Tensorflow 将尽合理的努力来维护此 pip 包的可用性和完整性。如果第三方未能发布 pip 软件包，则可能会出现延迟。

```
pip3 install tensorflow
```

验证安装

为了确保 TensorFlow 安装正确，可以通过运行 TensorFlow 示例代码来验证安装。

这段代码使用了 TensorFlow 库来生成一个随机正态分布的矩阵，并计算其所有元素的和。

```
python3 -c "import tensorflow as tf; print(tf.reduce_sum(tf.random.normal([1000, 1000])))"
```

Transformers 快速上手

Pipeline

使用 pipeline() 是利用预训练模型进行推理的最简单的方式。你能够将 pipeline() 开箱即用地用于跨不同模态的多种任务。来看看它支持的任务列表：

任务	描述	模态	Pipeline 标识符
文本分类	为给定的文本序列指定标签	自然语言处理	`pipeline(task="sentiment-analysis")`
文本生成	在给定提示的情况下生成文本	自然语言处理	`pipeline(task="text-generation")`
总结	生成文本或文档序列的摘要	自然语言处理	`pipeline(task="summarization")`
图像分类	为图像指定标签	计算机视觉	`pipeline(task="image-classification")`
图像分割	为图像的每个像素分配标签（支持语义、全景和实例分割）	计算机视觉	`pipeline(task="image-segmentation")`
对象检测	预测图像中对象的边界框和类别	计算机视觉	`pipeline(task="object-detection")`
音频分类	为某些音频数据分配标签	音频	`pipeline(task="audio-classification")`
自动语音识别	将语音转录为文本	音频	`pipeline(task="automatic-speech-recognition")`
视觉问题回答	回答一个关于图像的问	Multimodal 多式	`pipeline(task="vqa")`

	题，给出一个图像和一个问题		
文档问答	回答一个关于文档的问题，给出一个文档和一个问题	Multimodal 多式	`pipeline(task="document-question-answering")`
图像字幕	为给定图像生成字幕	Multimodal 多式	`pipeline(task="image-to-text")`

你可以将 pipeline()用于任何一种上面提到的任务，如果想知道支持的任务的完整列表，可以查阅 pipeline API。

首先，创建一个 pipeline() 实例并且指定你想要将它用于的任务(`Task`)。在这篇教程中，我们将 pipeline() 用在一个情感分析任务(`task="sentiment-analysis"`)上作为示例：

```
try:
    import tf_keras
except ImportError:
    raise ImportError(f"'tf_keras' could not be resolved. Please install it by 'pip install tf-keras'.")

from transformers import pipeline

# pipeline() 会下载并缓存一个用于情感分析的预训练模型和分词器。
classifier = pipeline("sentiment-analysis")

# 在目标文本上使用 classifier
classifier("We are very happy to show you the 🤗 Transformers library.")
```

也可以把多个输入放入一个列表然后传给 pipeline()，它将会返回一个字典列表：

```
results = classifier(["We are very happy to show you the 🤗 Transformers library.", "We hope you don't hate it."])
for result in results:
    print(f"label: {result['label']}, with score: {round(result['score'], 4)}")
```

pipeline() 也可以遍历整个数据集。在下面这个示例中，我们选择自动语音识别作为任务，加载 MInDS-14 音频数据集：

```
try:
    import soundfile
except ImportError:
    raise ImportError(f"'soundfile' could not be resolved. Please install it by 'pip install soundfile'.")
```

```
speech_recognizer = pipeline("automatic-speech-recognition", model="facebook/wav2vec2-base-960h")

from datasets import load_dataset, Audio
dataset = load_dataset("PolyAI/minds14", name="en-US", split="train", trust_remote_code=True)

# 你需要确保数据集中音频的采样率与 facebook/wav2vec2-base-960h 训练用到的音频的采样率一致
dataset = dataset.cast_column("audio",
Audio(sampling_rate=speech_recognizer.feature_extractor.sampling_rate))
```

当调用 "audio" 列时, 音频文件将会自动加载并重采样。 从前四个样本中提取原始波形数组，将它作为列表传给 pipeline：

```
try:
    import librosa
except ImportError:
    raise ImportError(f"'librosa' could not be resolved. Please install it by 'pip install librosa'.")


result = speech_recognizer(dataset[:4]["audio"])
print([d["text"] for d in result])
```

需要注意的是，对于输入很大的大型数据集（如语音或视觉），则需要传递一个生成器而不是用一个列表来加载内存中的所有输入。

在 pipeline 中使用另一个模型和分词器

在上文中提到，若 pipeline() 在指定任务后未指定模型和分词器，则会下载并缓存一个用于情感分析的预训练模型和分词器。

因为 pipeline() 可以容纳 Hub 中的任何模型，从而我们可以轻松地让 pipeline() 适用于其他用例。

例如，如果你想要一个能够处理法语文本的模型，可以先使用 Hub 上的 `Tags` 筛选合适的模型。

在这里，我们选择了一个多语言 BERT 模型，该模型经过微调，可用于法语文本的情感分析：

```
# 多语言 BERT 模型
model_name = "nlptown/bert-base-multilingual-uncased-sentiment"
```

在 transformers 库中，加载预训练模型主要有两种方式：

- **使用 pipeline() 加载。**

  特点

8. 高阶封装：pipeline 函数提供了一个高阶封装，使得加载和使用预训练模型变得非常简单。
9. 易用性：适合快速实验和原型开发，因为它自动处理了很多细节，如模型加载、分词器配置等。
10. 多功能：支持多种任务类型（如文本分类、问答、命名实体识别等），**只需指定任务名称**即可。
11. 默认配置：**使用默认的模型和分词器配置**，无需手动调整。

适用场景

12. 快速验证模型效果。
13. 简单的应用程序开发。
14. 不需要自定义模型或分词器配置的情况。

- **直接加载模型和分词器。**

特点

15. 灵活性：允许更细粒度的控制和自定义，如修改分词器的配置、调整模型的超参数等。
16. 可扩展性：适合**需要自定义模型或分词器**的情况，例如添加新的特殊标记、调整分词策略等。
17. 显式控制：加载模型和分词器的过程是显式的，便于调试和理解。

适用场景

18. 需要自定义模型或分词器配置。
19. 进行更深入的模型研究和调优。
20. 开发复杂的应用程序，需要更精细的控制。

这里使用 `AutoModelForSequenceClassification` 和 `AutoTokenizer` 直接加载预训练模型及其关联的分词器：

```
# Pytorch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# TensorFlow
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification
```

```
model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

在 pipeline() 中指定模型和分词器 `tokenizer`，现在你可以将分类器 `classifier` 来处理法语文本：

```
classifier = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer)
# 在后台，`AutoModelForSequenceClassification`和`AutoTokenizer`类一起工作，为上面使用的 pipeline() 提供动力。

classifier("Nous sommes très heureux de vous présenter la bibliothèque 🤗 Transformers.")
```

如果你没有找到适合你的模型，就需要在你的数据上微调一个预训练模型了。

查看微调教程来学习怎样进行微调。

Auto Class

`AutoClass` 是一种快捷方式，可以从预训练模型的名称或路径中自动检索其架构，你只需为任务及其关联的预处理类选择适当的 `AutoClass`。

让我们回到上一节的示例，看看如何使用 AutoClass 复制 pipeline() 的结果。

AutoTokenizer

分词器负责预处理文本，将文本转换为用于输入模型的数字数组。有多个用来管理分词过程的规则，包括如何拆分单词和在什么样的级别上拆分单词。

注意：实例化的分词器要与模型的名称相同，以确保和模型训练时使用相同的分词规则。

```
from transformers import AutoTokenizer

model_name = "nlptown/bert-base-multilingual-uncased-sentiment"
# 使用 AutoTokenizer 加载令牌化器，实例化的分词器要与模型的名称相同
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

将文本传入分词器 tokenizer:

```
encoding = tokenizer("We are very happy to show you the 🤗 Transformers library.")
print(encoding)
```

Tokenizer 会返回一个包含三个重要对象的字典：

- `input_ids`：这是一个整数序列，每个整数代表词汇表中的一个特定 token。**模型使用这个序列作为输入来理解和处理文本数据。**
- `token_type_ids`：这是一个与 input_ids 相同长度的序列，它表示输入序列中每个 token 的类型。这个字段主要用于**区分不同类型的输入序列**，尤其是在处理由多个文本序列组成的输入时，例如问答系统中的问题和答案，或者文本分类任务中的文本和标签。
- `attention_mask`：这个序列表示输入序列中哪些 token 是实际的文本 token，哪些是填充（padding）token。在处理批量数据时，**由于不同序列的长度可能不同，通常需要使用填充 token 来使所有序列长度一致。**`attention_mask` 中的 `1` 表示实际的文本 `token`，而 `0` 表示填充 `token`，这样模型就可以忽略填充 token。

分词器也可以接受列表作为输入，并填充和截断文本，返回具有统一长度的批次：

```
# Pytorch
pt_batch = tokenizer(
    ["We are very happy to show you the 🤗 Transformers library.", "We hope you don't hate it."],
    padding=True,
    truncation=True,
    max_length=512,
    return_tensors="pt",
)

# TensorFlow
tf_batch = tokenizer(
    ["We are very happy to show you the 🤗 Transformers library.", "We hope you don't hate it."],
    padding=True,
    truncation=True,
    max_length=512,
    return_tensors="tf"
)
```

查阅预处理数据教程来获得有关分词的更详细的信息，以及如何使用 `AutoFeatureExtractor` 和 `AutoProcessor` 来处理图像，音频，还有多模式输入。

AutoModel

🤗 Transformers 提供了一种简单统一的方式来加载预训练的实例. 这表示你可以像加载 AutoTokenizer 一样加载 AutoModel。

需要注意的是，**AutoModel 是一个通用的接口，它可以加载各种类型的预训练模型，但可能需要额外的配置才能用于特定任务。**

而 `AutoModelForSequenceClassification` 是专门为文本或序列的分类任务定制的接口，已自动为你配置好了一个适用于序列分类任务的模型。

```
# Pytorch
from transformers import AutoModelForSequenceClassification

model_name = "nlptown/bert-base-multilingual-uncased-sentiment"
pt_model = AutoModelForSequenceClassification.from_pretrained(model_name)

# TensorFlow
from transformers import TFAutoModelForSequenceClassification

model_name = "nlptown/bert-base-multilingual-uncased-sentiment"
tf_model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

通过 任务摘要 查找 AutoModel 支持的任务.

现在可以把之前预处理好的输入批次直接传递模型。你需要通过 ** 来解包字典:

```
# Pytorch
pt_outputs = pt_model(**pt_batch)

# TensorFlow
tf_outputs = tf_model(**tf_batch)
```

logits

- 定义：logits 是模型输出的原始分数，通常是通过一个线性层（全连接层）得到的。这些分数在没有经过任何激活函数处理之前，表示模型对每个类别的原始预测值。
- 作用：logits 本身并不直接表示概率，而是需要通过激活函数（如 softmax）转换为概率。

激活结果

- 定义：激活结果通常是指将 logits 通过激活函数（如 softmax、sigmoid 等）处理后的输出。激活函数的作用是将 logits 转换为某种形式的概率或决策边界。
- 作用：激活函数可以使模型的输出更易于解释和使用。例如，softmax 函数可以将 logits 转换为多类分类任务中的概率分布。

模型在 `pt_outputs.logits` 输出最终的激活结果. 这里在 logits 上应用 `softmax` 函数来查询概率:

- `dim = -1`：表示在最后一个维度上进行 softmax 操作，通常在多类分类任务中，最后一个维度是类别维度。

- 在多类分类任务中，softmax 函数将 logits 转换为概率分布，能够帮助我们了解模型对每个类别的预测置信度，从而做出更准确的决策。

```
# Pytorch
from torch import nn

pt_predictions = nn.functional.softmax(pt_outputs.logits, dim=-1)
print(pt_predictions)
```

`pt_predictions` 是一个概率分布，每个元素表示对应类别的概率，每个类别的概率值表示模型对该类别的置信度。

```
# TensorFlow
import tensorflow as tf

tf_predictions = tf.nn.softmax(tf_outputs.logits, dim=-1)
print(tf_predictions)
```

所有 🤗 Transformers 模型（PyTorch 或 TensorFlow）在最终的激活函数（比如 softmax）之前输出张量， 因为最终的激活函数常常与损失(loss)融合。

模型的输出是特殊的数据类，所以它们的属性可以在 IDE 中被自动补全。模型的输出就像一个元组或字典（你可以通过整数、切片或字符串来索引它），为 None 的属性会被忽略。

保存模型

当你的模型微调完成(预训练完成)，可以使用 `PreTrainedModel.save_pretrained()` 把模型和模型的分词器保存下来：

```
# Pytorch
pt_save_directory = "./pt_save_pretrained"
tokenizer.save_pretrained(pt_save_directory)
pt_model.save_pretrained(pt_save_directory)

# TensorFlow
tf_save_directory = "./tf_save_pretrained"
tokenizer.save_pretrained(tf_save_directory)
tf_model.save_pretrained(tf_save_directory)
```

再次使用这个模型时，可以使用 `PreTrainedModel.from_pretrained()` 加载它：

```
# Pytorch
pt_model = AutoModelForSequenceClassification.from_pretrained("./pt_save_pretrained")

# TensorFlow
tf_model = TFAutoModelForSequenceClassification.from_pretrained("./tf_save_pretrained")
```

😭 Transformers 有一个特别酷的功能，保存一个模型后，在加载时可以选择地将它加载为 PyTorch 或 TensorFlow 模型。`from_pt` 或 `from_tf` 参数可以将模型从一个框架转换为另一个框架：

```
# TensorFlow To Pytorch
from transformers import AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(tf_save_directory)
pt_model = AutoModelForSequenceClassification.from_pretrained(tf_save_directory, from_tf=True)
# from_tf 将 TensorFlow 模型加载为 PyTorch 模型

# Pytorch To TensorFlow
from transformers import TFAutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(pt_save_directory)
tf_model = TFAutoModelForSequenceClassification.from_pretrained(pt_save_directory, from_pt=True)
```

自定义模型构建

你可以修改模型的配置类来改变模型的构建方式。配置模型的属性，比如隐藏层或者注意力头的数量。当你根据自定义的配置类初始化模型时，模型的属性都是随机初始化的，你需要先训练模型才能得到有意义的结果。

自定义模型构建通过导入 `AutoConfig` 来加载你想修改的预训练模型。在 `AutoConfig.from_pretrained()` 中，你能够指定想要修改的属性，比如注意力头的数量：

```
from transformers import AutoConfig

my_config = AutoConfig.from_pretrained("distilbert/distilbert-base-uncased", n_heads=12)
```

使用 `AutoModel.from_config()` 根据你的自定义配置创建一个模型:

```
# Pytorch
from transformers import AutoModel

my_model = AutoModel.from_config(my_config)
# TensorFlow
from transformers import TFAutoModel

my_model = TFAutoModel.from_config(my_config)
```

Trainer - PyTorch 优化训练循环

所有的模型都是标准的

`[torch.nn.Module](https://pytorch.org/docs/stable/nn.html#torch.nn.Module)`，所以你可以在任何典型的训练模型中使用它们。

当你编写自己的训练循环时，😭 Transformers 为 PyTorch 提供了一个 `Trainer` 类，它包含了基础的训练循环并且为诸如分布式训练，混合精度等特性增加了额外的功能。

根据于你的任务, 你可以传递以下的参数给 `Trainer`:

21. 使用 `PreTrainedModel` 或者 `torch.nn.Module` 来开始：

```
# Pytorch
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("distilbert/distilbert-base-uncased")
```

22. **`TrainingArguments` 含有你可以修改的模型超参数**，比如学习率，批次大小和训练时的迭代次数。如果你没有指定训练参数，那么它会使用默认值：

```
# Pytorch
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="path/to/save/folder/",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
)
```

23. 加载一个预处理类，比如分词器，特征提取器或者处理器：

```
# Pytorch
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

24. 加载一个数据集：

```
from datasets import load_dataset

dataset = load_dataset("rotten_tomatoes")  # doctest: +IGNORE_RESULT
```

25. 创建一个给数据集用于分词的函数，并且使用 `map` 将分词器应用到整个数据集：

```
def tokenize_dataset(dataset):
    return tokenizer(dataset["text"])

dataset = dataset.map(tokenize_dataset, batched=True)
```

26. 创建一个数据集预处理工具 `DataCollatorWithPadding`

- `DataCollatorWithPadding` 是一个高效的数据预处理工具，能够自动处理批次数据的填充和掩码生成，极大地简化了 NLP 任务中的数据预处理步骤。通过使用这个类，可以确保模型输入的一致性，提高训练效率。

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

现在把所有的类传给 `Trainer`：

```
# Pytorch
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
)
# 一切准备就绪后，调用 train() 进行训练
trainer.train()
```

对于像翻译或摘要这些使用序列到序列模型的任务，用 `Seq2SeqTrainer` 和 `Seq2SeqTrainingArguments` 来替代。

自定义训练循环

你可以通过子类化 `Trainer` 并重写其中的方法来自定义训练循环。这样你就可以**自定义像损失函数，优化器和调度器等这样的特性**。查阅 Trainer 参考手册了解哪些方法能够被子类化。

另一个自定义训练循环的方式是通过回调(Callbacks)。你可以**使用回调来与其他库集成**，查看训练循环来报告进度或提前结束训练。

注意：回调并不会修改训练循环。如果想自定义损失函数等，就需要子类化 Trainer 了。

使用 Tensorflow 训练

所有模型都是标准的 tf.keras.Model，所以你可以**通过 `Keras API` 实现在 Tensorflow 中训练。**

🤗 Transformers 提供了 `prepare_tf_dataset()` 方法来轻松地将数据集加载为 `tf.data.Dataset`，这样你就可以使用 Keras 的 `compile` 和 `fit` 方法马上开始训练。

27. 使用 `TFPreTrainedModel` 或者 `tf.keras.Model` 来开始：

```
# TensorFlow
from transformers import TFAutoModelForSequenceClassification

model = TFAutoModelForSequenceClassification.from_pretrained("distilbert/distilbert-base-uncased")
```

28. 加载一个数据集

```
from datasets import load_dataset

dataset = load_dataset("rotten_tomatoes")
```

29. 加载一个预处理类，比如分词器，特征提取器或者处理器：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

30. 创建一个给数据集用于分词的函数，并且使用 `map` 将分词器应用到整个数据集：

```
def tokenize_dataset(dataset):
    return tokenizer(dataset["text"])

dataset = dataset.map(tokenize_dataset, batched=True)
```

31. 将数据集和分词器传给 `prepare_tf_dataset()`对数据集进行预处理。如果你需要的话，也可以在这里改变批次大小和是否打乱数据集

```
# TensorFlow
tf_dataset = model.prepare_tf_dataset(
    dataset, batch_size=16, shuffle=True, tokenizer=tokenizer
)
```

32. 一切准备就绪后，调用 `compile` 和 `fit` 开始训练：

```
# TensorFlow
from transformers.keras.optimizers import Adam

model.compile(optimizer=Adam(3e-5))
model.fit(dataset)
```

接下来做什么

现在你已经完成了 🤗 Transformers 的快速上手教程，来看看我们的指南并且学习如何做一些更具体的事情，比如写一个自定义模型，为某个任务微调一个模型以及如何使用脚本来训练模型。

如果你有兴趣了解更多 🤗 Transformers 的核心章节，那就喝杯咖啡然后来看看我们的概念指南吧！

第二部分 教程

使用 pipelines 进行推理

`pipeline()` 让使用 🤗 `Hugging Face Hub` 上的任何模型进行任何语言、计算机视觉、语音以及多模态任务的推理都变得非常简单。

即使你对特定的模态没有经验，或者不熟悉模型的源码，你仍然可以使用 pipeline()进行推理！

本教程说明：

33. 如何使用 pipeline() 进行推理。
34. 如何使用特定的 tokenizer(分词器) 或 model(模型)。
35. 如何使用 pipeline() 进行音频、视觉和多模态任务的推理。

请查看 pipeline()文档以获取已支持的任务和可用参数的完整列表。

Pipeline 使用

虽然每种任务都有一个关联的 pipeline，但我们可以使用通用的 `pipeline()` 方法，其中包含所有特定任务的 pipelines，**能够根据任务类型自动加载一个默认模型和一个能够进行任务推理的预处理类。**

让我们以使用 pipeline() 进行自动语音识别（ASR）或语音转文本为例。

36. 首先，创建一个 pipeline() 并指定推理任务：

```
from transformers import pipeline

transcriber = pipeline(task="automatic-speech-recognition", device=0) # 任务：自动语音识别（ASR）
```

37. 将你的输入传递给 pipeline()。对于语音识别，通常是输入一个音频文件：

```
transcriber("../../resources/speech/mlk.flac")
```

38. 如果你在 MAC OS 中运行，可能会遇到错误信息 "ffmpeg was not found but is required to load audio files from filename"。这表示你的环境中缺少 `ffmpeg`，这是一个用于处理音频和视频文件的强大工具。在 macOS 上，你可以使用 Homebrew 来安装 ffmpeg。

```
brew install ffmpeg
```

如果结果不符合你的预期，还可以在 Hub 上查看一些最受欢迎的自动语音识别模型，看看是否可以获得更好的转录结果。

让我们尝试来自 OpenAI 的 Whisper large-v2 模型。Whisperb 比 Wav2Vec2 晚 2 年发布，使用接近 10 倍的数据进行了训练。因此，它在大多数下游基准测试上击败了 Wav2Vec2。 它还具有预测标点和大小写的附加优势，而 Wav2Vec2 则无法实现这些功能。

让我们在这里尝试一下，看看它的表现如何：

```
transcriber = pipeline(task="automatic-speech-recognition", model="openai/whisper-large-v2")

transcriber("../../resources/speech/mlk.flac")
```

现在这个结果看起来更准确了！

要进行深入的 Wav2Vec2 与 Whisper 比较，请参阅音频变换器课程。

建议在 Hub 上查看不同语言的模型，以及专业领域的模型等。可以在 Hub 上直接查看并比较模型的结果，以确定是否适合或处理边缘情况是否比其他模型更好。

如果没有找到适用于你的用例的模型，可以通过训练来获得适合自己的模型！

如果你有多个输入，可以将输入作为列表传递：

```
transcriber(
    [
        "../../resources/speech/mlk.flac",
        "../../resources/speech/1.flac",
    ]
)
```

从一个模型切换到另一个模型非常琐碎的过程，但使用 Pipelines 能够非常迅速的切换模型，所以它非常适合用于测试。

但是，通过一些方法还可以将 Pipelines 用于大型工作负载而不仅仅是测试。详细请查看以下指南，深入探讨如何迭代整个数据集或在 Web 服务器中使用 Pipelines：

39. 在数据集上使用流水线

40. 在 Web 服务器中使用流水线

参数

`pipeline()` 支持了许多参数；有些是适用于特定任务的，有些则是适用于所有的 pipeline。通常情况下，你可以在任何地方指定对应的参数：

```
transcriber = pipeline(model="openai/whisper-large-v2", my_parameter=1)

out = transcriber(...)  # 这将使用 `my_parameter=1`。
out = transcriber(..., my_parameter=2)  # 这将覆盖并使用 `my_parameter=2`。
out = transcriber(...)  # 这将返回到使用 `my_parameter=1`。
```

让我们查看其中的三个重要参数：

设备

如果你设置 `device=n`，pipeline 会自动将模型加载到指定的设备上（包括存储模型的权重）。支持 PyTorch 或 Tensorflow。

```
transcriber = pipeline(model="openai/whisper-large-v2", device=0)
```

如果你使用的是 PyTorch，且觉得模型对于单个 GPU 来说过于庞大，则可以设置 `device_map="auto"` 使其自动确定如何加载和存储模型的权重。使用 `device_map` 参数需要安装 🤗 `Accelerate` 软件包：

```
pip install --upgrade accelerate
```

以下代码会自动在各个设备上加载和存储模型权重：

```
transcriber = pipeline(model="openai/whisper-large-v2", device_map="auto")
```

请注意，如果设置了 `device_map="auto"`，在实例化 pipeline 时则不需要设置 `device=n` 参数，否则可能会遇到一些意外的状况！

批量大小

默认情况下，pipelines 不会进行批量推理，因为批处理不一定更快，在某些情况下可能会更慢，原因可查阅详细解释。

想在用例中使用批量推理，如下：

```
import os

audio_filenames = [os.path.join('../../resources/speech', f"{i}.flac") for i in range(1, 5)]

transcriber = pipeline(model="openai/whisper-large-v2", device=0, batch_size=2)
texts = transcriber(audio_filenames)
```

以上代码会在提供的 4 个音频文件上运行 pipeline，设置 `batch_size=2` 表示 pipeline 会将它们以每 2 个为一组的批次传递给模型（若模型加载在 GPU 上，此时批处理可能更有作用），不需要编写额外的代码。

输出的结果会始终与没有批处理时收到的结果相一致，因为批量推理只是一种帮助你更快地运行 pipeline 的方式，你可以将它理解为是 pipeline 的带宽大小。

同时，使用 pipeline() 能够帮助我们减轻工作量，降低批处理的复杂性。因为对于某些 pipeline，需要将单个项目（如长音频文件）分成多个部分后才能供模型处理。**pipeline() 会在需要时自动执行 chunk batching（分块处理）。**

任务特定参数

transformers 为所有任务的 pipeline 都提供了特定的参数，这些参数使得 pipeline 更具灵活性，能够帮助你更好地完成工作。

例如，`transformers.AutomaticSpeechRecognitionPipeline.call()` 方法就具有一个 `return_timestamps` 参数，能够在模型推断出文本的同时输出各个句子发音的时间戳，对于需要添加字幕的视频很有帮助：

```
transcriber = pipeline(model="openai/whisper-large-v2", return_timestamps=True)

transcriber("https://huggingface.co/datasets/Narsil/asr_dummy/resolve/main/mlk.flac")
```

每个任务都有许多可用的参数，因此请查看每个任务的 API 参考。

例如，在为整部电影或长达一小时的视频添加字幕时，需要模型处理非常长的音频文件，但这通常是模型无法单独处理的，可以通过 `AutomaticSpeechRecognitionPipeline` 设置 `chunk_length_s` 参数来控制音频处理的分块长度，这非常有帮助：

```
transcriber = pipeline(model="openai/whisper-large-v2", chunk_length_s=30,
return_timestamps=True)

transcriber("https://huggingface.co/datasets/sanchit-
gandhi/librispeech_long/resolve/main/audio.wav")
```

如果你找不到任何一个有帮助的参数，欢迎提出请求！

在数据集上使用 pipelines

pipelines 也可以对大型数据集进行推理。

`yield` 在 Python 中是一个特殊的标识符，用于将函数转换为生成器（迭代器）。生成器可以一次生成并返回一个值，而不是一次性返回所有值，从而节省内存。

建议使用生成器来完成这一任务，这是最简单的方法：

```
def data():
    for i in range(1000):
        yield f"My example {i}"


pipe = pipeline(model="openai-community/gpt2", device=0)
generated_characters = 0
for out in pipe(data()):
    generated_characters += len(out[0]["generated_text"])
```

pipelines 会自动将生成器 `data()` 识别输入为可迭代对象，并在 GPU 上处理数据的同时开始获取数据（在底层使用 DataLoader）。这一点非常重要，因为你不必为整个数据集分配内存，可以尽可能快地将数据传送到 GPU。

由于批处理可以加速处理，因此在这里尝试调整 `batch_size` 参数可能会很有用。

迭代数据集的最简单方法就是从🤗 `(Datasets)[https://github.com/huggingface/datasets/]` 中加载数据集：

```
# KeyDataset 是一个实用工具，它支持从 datasets 库中的数据集中提取特定的字段（或键）传递给 pipelines。
from transformers.pipelines.pt_utils import KeyDataset
from datasets import load_dataset

pipe = pipeline(model="hf-internal-testing/tiny-random-wav2vec2", device=0)
dataset = load_dataset("hf-internal-testing/librispeech_asr_dummy", "clean",
split="validation[:10]")

for out in pipe(KeyDataset(dataset, "audio")): # KeyDataset(dataset, "audio") 表示从 dataset 中
提取键为 "audio" 的数据。
    print(out)
```

在 Web 服务器上使用 pipelines

Using pipelines for a webserver

视觉任务 pipeline()

对于不同视觉任务的 pipeline() 调用方式都非常相似，指定视觉任务模型并将图像传递给分类器。

输入的图像可以是链接、本地路径或 base64 编码的图像。例如，下面显示的是哪种品种的猫?



pipeline-cat-chonk.png

```
from transformers import pipeline

vision_classifier = pipeline(model="google/vit-base-patch16-224", device=0)
preds = vision_classifier(
    images="../../resources/images/pipeline-cat-chonk.png"
)
preds = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in preds]
preds
```

输出结果解释

模型的输出结果 `preds` 是一个列表，每个元素包含两个键：`score` 和 `label`。

- score：表示模型预测该类别对应的置信分数，范围通常在 0 到 1 之间，值越接近 1 表示模型越确信该类别。
- label：表示预测的类别标签。

自然语言处理任务 pipeline()

对于不同 NLP 任务的 pipeline() 调用方式都非常相似，主要区别在于任务类型和模型的指定。

```
from transformers import pipeline

# `facebook/bart-large-mnli`是一个`zero-shot-classification`模型，能够在没有针对特定类别进行专门训练的情况下，对文本进行灵活的分类。
classifier = pipeline(model="facebook/bart-large-mnli", device=0)
classifier(
    "I have a problem with my iphone that needs to be resolved asap!!",
    candidate_labels=["urgent", "not urgent", "phone", "tablet", "computer"],# 它能够对文本内容进行分类，你可以自由地定义任何标签作为候选类别。
)
# 模型的输出通常会包含每个候选标签的置信分数，表示文本属于该标签的可能性。
```

多模态 pipeline()

pipeline() 支持多模态任务的调用。

对于视觉问题回答任务（VQA），pipeline() 结合了文本和图像，使得可以处理任何图像和任何关于该图像的问题。

例如，你想提问关于该图像的问题：

!invoice image:

要运行下面的示例，除了安装🤗 Transformers 之外，还需要安装 `pytesseract` 。

Mac OS

```
brew install tesseract
```

Windows

Tesseract installer for Windows

Python

```
pip install pytesseract
```

```
from transformers import pipeline

vqa = pipeline(model="impira/layoutlm-document-qa", device=0)
output = vqa(
    image="../../resources/images/invoice.png",
    question="What is the invoice number?",
)
output[0]["score"] = round(output[0]["score"], 3)
output
```

在大模型上使用🤗 accelerate 和 pipeline：

你可以轻松地使用 🤗 `accelerate` 在大模型上运行 pipeline!

首先确保你已经使用 `pip install accelerate` 安装了 accelerate。

首先使用 `device_map="auto"` 加载你的模型！我们将在示例中使用模型 `facebook/opt-1.3b`。

device_map="auto"的作用

- 自动设备分配:
    41. 当你设置 device_map="auto"时，transformers 库会自动检测你当前的硬件配置（包括 CPU 和 GPU 的数量及内存大小）。
    42. 根据检测到的硬件配置，它会智能地将模型的不同层分配到不同的设备上。例如，如果有一个大型模型和一个 GPU，它可能会将模型的一部分放在 GPU 上，另一部分放在 CPU 上，以避免内存溢出。
- 优化内存使用:

43. 通过智能分配，可以最大限度地利用可用的硬件资源，避免单个设备（如单个 GPU）因内存不足而无法加载整个模型。
44. 这对于处理像 `facebook/opt-1.3b` 这样的大型模型尤为重要，因为这些模型可能需要大量的内存。

- 提高计算效率：

自动设备分配不仅优化了内存使用，还可以提高计算效率。例如，**将计算密集的部分放在 GPU 上，而将其他部分放在 CPU 上，可以充分利用不同设备的计算能力。**

```
# pip install accelerate
try:
    import accelerate
except ImportError:
    raise ImportError("'accelerate' could not be resolved. Please install it by 'pip install accelerate'.")

import torch
from transformers import pipeline

pipe = pipeline(model="facebook/opt-1.3b", torch_dtype=torch.bfloat16, device_map="auto")
output = pipe("This is a cool example!", do_sample=True, top_p=0.95)
```

如果安装 `bitsandbytes` 可以通过添加参数 `load_in_8bit=True` 以 8 位精度加载模型。

`load_in_8bit=True` 是一个用于模型加载的参数，它指示 transformers 库**以 8 位精度加载模型**（而不是标准的 16 位或 32 位精度）。这个参数通常与 `bitsandbytes` 库结合使用，以实现模型的量化，从而减少内存占用和提高推理速度。虽然 8 位量化通常会保留模型的绝大部分性能，但在某些情况下可能会略微降低模型的准确率。

```
# pip install accelerate
try:
    import accelerate
except ImportError:
    raise ImportError("'accelerate' could not be resolved. Please install it by 'pip install accelerate'.")
# pip install bitsandbytes
try:
    import bitsandbytes
except ImportError:
    raise ImportError("'bitsandbytes' could not be resolved. Please install it by 'pip install bitsandbytes'.")

import torch
from transformers import pipeline
```

```
pipe = pipeline(model="facebook/opt-1.3b", device_map="auto", model_kwargs={"load_in_8bit": True})

# 使用随机抽样
output_sample = pipe("This is a cool example!", do_sample=True, top_p=0.95)
print("With do_sample=True:", output_sample[0]['generated_text'])

# 不使用随机抽样
output_greedy = pipe("This is a cool example!", do_sample=False)
print("With do_sample=False:", output_greedy[0]['generated_text'])
```

do_sample 随机抽样

`do_sample` 参数决定了模型在生成下一个 token 时是选择概率最高的 token，还是从概率分布中随机抽取一个 token。

- 当 `do_sample=True` 时，模型会在每个时间步根据概率分布随机选择下一个 token。这种方法可以增加生成文本的多样性和创造性，使其看起来更自然、更不重复。
- 当 `do_sample=False` 时，模型始终选择概率最高的 token 作为下一个 token。这种方法生成的文本通常更保守、更可预测，但可能会显得单调、缺乏多样性。

参数组合

`do_sample` 通常与 top_k 和 top_p（也称为 nucleus sampling）等参数结合使用，以进一步控制抽样过程。

- `top_k`：与 `do_sample=True` 结合使用时，表示从概率最高的 k 个 token 中进行随机抽样。
- `top_p`：与 `do_sample=True` 结合使用时，表示从累积概率超过 p 的 token 集合中进行随机抽样。

```
在 Hugging Face 的文档中，"checkpoint" 是一个泛指，代表你选择的预训练模型。在代码中，你通过替换 "model" 参数的值来选择不同的模型。
```

请注意，你可以将 `checkpoint` 替换为任何支持大模型加载的 Hugging Face 模型，比如 `BLOOM`。只需将模型 `facebook/opt-1.3b` 替换为 `bigscience/bloom-1b1` 即可。

```
import torch
from transformers import pipeline

# 替换模型名称为 BLOOM 模型
model_name = "bigscience/bloom-1b1"
```

```
# 创建文本生成 pipeline
pipe = pipeline(
    model=model_name,
    device_map="auto",
    model_kwargs={"load_in_8bit": True}
)

# 生成文本
output = pipe(
    "This is a cool example!",
    do_sample=True,
    top_p=0.95
)

# 打印生成的文本
print(output[0]['generated_text'])
```

使用 AutoClass 加载预训练实例

请记住，在本章的内容中，“架构”指的是模型的结构，而 "checkpoint" 是给定架构的权重（也就是预训练模型的权重，也可指代预训练模型本身）。

例如，“BERT” 是一种架构，而 “google-bert/bert-base-uncased” 是一个 checkpoint。“模型”则是一个通用术语，可以指代架构或 checkpoint。

在 🤗 `Transformers` 的示例代码中，你通过替换 "model" 参数的值来选择不同的模型。

在这个教程中，将会学习如何：

45. 加载预训练的分词器（tokenizer）

46. 加载预训练的图像处理器(image processor)

47. 加载预训练的特征提取器(feature extractor)

48. 加载预训练的处理器(processor)

49. 加载预训练的模型。

使用 AutoClass 加载预训练实例

由于存在许多不同的 Transformer 架构，因此为 checkpoint 创建一个可用的架构可能会具有挑战性。

通过 `AutoClass` 可以自动推断并从给定的 checkpoint 加载正确的架构，这也是体现 🥹 Transformers 易于使用、简单且灵活的核心部分。

如果你的代码适用于一个 checkpoint，它将适用于另一个 checkpoint，只要它们是为了相似的任务进行训练的，即使它们的架构有所不同。

`from_pretrained()` 方法允许你快速加载任何架构的预训练模型，不需要花费时间和精力从头开始训练模型。

AutoTokenizer

几乎所有的 NLP 任务都以 `tokenizer` 开始。Tokenizer 能够将你的输入转换为模型可以理解和处理的格式(即模型正确的输入格式)。

使用 `AutoTokenizer.from_pretrained()` 加载 tokenizer：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
```

然后按照如下方式对输入进行分词：

```
sequence = "In a hole in the ground there lived a hobbit."
print(tokenizer(sequence))
```

AutoImageProcessor

对于视觉任务，`ImageProcessor` 能够将图像处理成正确的输入格式。

这里使用 `AutoImageProcessor.from_pretrained()`加载 Image Processor：

```
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
```

AutoFeatureExtractor

对于音频任务, `Feature Extractor` 将音频信号处理成正确的输入格式。

这里使用 `AutoFeatureExtractor.from_pretrained()`加载 Feature Extractor：

```
from transformers import AutoFeatureExtractor

feature_extractor = AutoFeatureExtractor.from_pretrained(
    "ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition"
)
```

AutoProcessor

对于多模态任务只需要一个 `Processor` 就能够将两种类型的预处理工具结合起来。

例如，LayoutLMV2 模型需要一个 image processor 来处理图像和一个 tokenizer 来处理文本；Processor 会将两者结合起来。

使用 `AutoProcessor.from_pretrained()`加载 Processor：

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("microsoft/layoutlmv2-base-uncased")
```

AutoModel

`AutoModelFor*` 类能够加载给定任务类型的预训练模型（支持的任务列表）。

例如，使用 `AutoModelForSequenceClassification.from_pretrained()` 加载用于序列分类的模型：

```
# PyTorch
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("distilbert/distilbert-base-uncased")
```

可以使用相同的 checkpoint 来为不同的任务加载模型架构：

```
# PyTorch
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained("distilbert/distilbert-base-uncased")
```

对于 PyTorch 模型，`from_pretrained()`方法在加载预训练模型时，通常使用 `torch.load()`，它内部使用的是不安全的 `pickle`， pickle 加载不可信的数据可能存在安全风险（如代码执行漏洞）。

永远不要加载那些来自不可信来源或可能被篡改的模型。建议对于托管在 Hugging Face Hub 上的公共模型，因为每次提交都会进行恶意软件扫描，在一定程度上缓解了安全风险。

建议使用使用 GPG 进行签名提交验证等最佳实践来进一步确保安全性，可参阅 Hub 文档以了解最佳实践。

TensorFlow 和 Flax 在加载 checkpoints 不使用 pickle ，不存在由 pickle 引起的安全问题。

在 PyTorch 中可以通过 `from_pretrained()` 方法的 `from_tf` 和 `from_flax` 参数来加载 TensorFlow 和 Flax 的 checkpoints，进而绕过 torch.load() 使用 pickle 的安全风险。

实际操作示例

通过加载 TensorFlow 和 Flax 的 checkpoints，可以避免使用 torch.load()及其 pickle 的安全风险。

假设你想要加载一个在 Hugging Face Hub 上托管的 TensorFlow 模型到 PyTorch 中，可以这样做：

```
from transformers import AutoModel

# 加载 TensorFlow 模型，并将 TensorFlow 模型转换为 PyTorch 模型
pt_model = AutoModel.from_pretrained("bert-base-uncased", from_tf=True)
```

类似地，如果你要加载一个 Flax 模型到 PyTorch 中：

```
from transformers import AutoModel

# 加载 Flax 模型，并将 Flax 模型转换为 PyTorch 模型
pt_model = AutoModel.from_pretrained("bert-base-uncased", from_flax=True)
```

一般来说，我们不建议使用通用的 `AutoModel` 来加载预训练模型，而是使用 `AutoTokenizer` 类和 `AutoModelFor*`类来加载预训练的模型实例，因为这样可以确保加载的是正确的架构。

在下一个教程中，学习如何使用新加载的 tokenizer, image processor, feature extractor 和 processor 对数据集进行预处理以进行微调。

预处理数据

在数据集上训练模型之前，数据需要被预处理为模型期望的输入格式。无论数据是文本、图像还是音频，它们都需要被转换并组合成批量的张量。

🤗 Transformers 提供了一组预处理类来帮助准备数据以供模型使用。在本教程中，你将了解以下内容：

50. 对于文本，使用分词器(Tokenizer)能够将文本转换为一系列标记 `tokens`，并创建 tokens 的数字表示，将它们组合成张量。
51. 对于语音和音频，使用特征提取器(Feature extractor)能够从音频波形中提取顺序特征并将其转换为张量。
52. 对于图像，使用图像处理器(`ImageProcessor`)将图像转换为张量。
53. 对于多模态输入，使用处理器(Processor)，其结合了 Tokenizer 和 ImageProcessor 或 Processor。

`AutoProcessor` 能够有效的自动选择适用于模型输入的预处理工具，无论你使用的是 `Tokenizer`、`ImageProcessor`、`Feature extractor` 还是 `Processor`。

在开始之前，请安装🤗 `Datasets`，可以加载一些数据集来进行实验：

```
pip install datasets
```

自然语言处理（文本预处理）

处理文本数据的主要工具是分词器(`Tokenizer`)，模型所需的任何附加输入都由 Tokenizer 进行添加。

Tokenizer 会根据一组规则将文本拆分为 tokens，然后将这些 tokens 转换为数字，然后转换为张量，成为模型的输入。

如果你选择使用预训练模型，则需要注意使用与模型关联的预训练的 Tokenizer。这是为了确保文本的拆分方式与预训练的语料库相同，并在预训练期间使用相同的对应关系标记-索引（通常称为词汇表 `vocab`）。

开始使用`AutoTokenizer.from_pretrained()`方法加载一个预训练的 Tokenizer，并下载预训练模型的词汇表 vocab：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-cased")
```

然后文本输入给 Tokenizer：

```
encoded_input = tokenizer("Do not meddle in the affairs of wizards, for they are subtle and quick to anger.")
print(encoded_input)
```

Tokenizer 会返回一个包含三个重要对象的字典：

- `input_ids`：这是一个整数序列，每个整数代表词汇表中的一个特定 token。**模型使用这个序列作为输入来理解和处理文本数据。**
- `token_type_ids`：这是一个与 input_ids 相同长度的序列，它表示输入序列中每个 token 的类型。这个字段主要用于**区分不同类型的输入序列**，尤其是在处理由多个文本序列组成的输入时，例如问答系统中的问题和答案，或者文本分类任务中的文本和标签。
- `attention_mask`：这个序列表示输入序列中哪些 token 是实际的文本 token，哪些是填充（padding）token。在处理批量数据时，**由于不同序列的长度可能不同，通常需要使用填充 token 来使所有序列长度一致。**`attention_mask` 中的 `1` 表示实际的文本 `token`，而 `0` 表示填充 `token`，这样模型就可以忽略填充 token。

通过`tokenizer.decode()`可以解码 `input_ids` 得到输入的文本：

```
tokenizer.decode(encoded_input["input_ids"])
```

从解码得到的文本可知，tokenizer 在句子中添加了两个特殊 token - `[CLS]` 和 `[SEP]`（分类器和分隔符）。

注意！并不是所有模型都需要添加特殊 token，但如果需要，tokenizer 会自动给你添加。

如果有多个句子需要进行预处理，可以将它们作为列表传递给 tokenizer：

```
batch_sentences = [
    "But what about second breakfast?",
    "Don't think he knows about second breakfast, Pip.",
    "What about elevensies?",
]
encoded_inputs = tokenizer(batch_sentences)
print(encoded_inputs)
```

填充

输入句子的长度并不总是相同的，这是一个问题，因为模型输入的张量需要具有统一的形状。

填充是一种解决办法，通过在较短的句子中添加一个特殊的 `padding token`，以确保张量是矩形的。

设置参数 `padding=True`，将批次中较短的序列填充到与最长的序列相同的长度，一般用 0 进行填充：

```
batch_sentences = [
    "But what about second breakfast?",
    "Don't think he knows about second breakfast, Pip.",
    "What about elevensies?",
]
encoded_input = tokenizer(batch_sentences, padding=True)
print(encoded_input)
```

截断

另一方面，有时候一个序列可能对模型来说太长了。在这种情况下，则需要将序列截断为更短的长度。

设置参数 `truncation=True`，将过长的序列截断为模型接受的最大长度，也可以通过 max_length 参数设置截断的最大长度：

```
batch_sentences = [
    "But what about second breakfast?",
    "Don't think he knows about second breakfast, Pip.",
    "What about elevensies?",
]
encoded_input = tokenizer(batch_sentences, padding=True, truncation=True)
print(encoded_input)
```

查看填充和截断概念指南，了解更多有关填充和截断参数的信息。

构建张量

最后，`tokenizer` 会返回实际输入到模型的张量。

设置参数 `return_tensors="pt"`（对于 PyTorch ）或 `return_tensors="tf"`（对于 TensorFlow ）：

```
# PyTorch
batch_sentences = [
    "But what about second breakfast?",
    "Don't think he knows about second breakfast, Pip.",
    "What about elevensies?",
]
encoded_input = tokenizer(batch_sentences, padding=True, truncation=True, return_tensors="pt")
print(encoded_input)

# TensorFlow
batch_sentences = [
    "But what about second breakfast?",
    "Don't think he knows about second breakfast, Pip.",
    "What about elevensies?",
]
encoded_input = tokenizer(batch_sentences, padding=True, truncation=True, return_tensors="tf")
print(encoded_input)
```

音频

对于音频任务，需要使用 `feature extractor` 来预处理数据集以供模型使用。

`Feature extractor` 能够从原始的音频数据中提取出特征，并将它们转换为张量。

加载 MInDS-14 数据集（有关如何加载数据集的更多详细信息，请参阅 🤗Datasets 教程）以了解如何在音频数据集中使用 feature extractor：

```
from datasets import load_dataset, Audio

dataset = load_dataset("PolyAI/minds14", name="en-US", split="train")

# 访问 audio 列的第一个元素以查看输入。调用 audio 列会自动加载和重新采样音频文件：
dataset[0]["audio"]
```

dataset[0]["audio"]返回的结果包含了三个对象：

- `array` 是一个包含音频样本振幅值的数组，会在必要时重新采为一位数组（1D array），每个元素对应一个音频样本的振幅，采样率决定了每秒钟采集的样本数。
- `path` 是音频文件的位置。
- `sampling_rate` 是音频的采样率，单位是赫兹（Hz）。常见的采样率有 16000 Hz、22050 Hz、44100 Hz 等。

假设有一个采样率为 16000 Hz 的音频信号，持续时间为 1 秒，那么这个音频信号可以用一个包含 16000 个元素的 1D array 来表示。

接下来，加载一个 `feature extractor` 以对输入进行标准化和填充。

填充文本的理念同样适用于预处理音频数据。当填充文本数据时，会为较短的序列填充 0。在 `feature extractor` 中，会为较短的数组填充 0 表示静音。

使用 `AutoFeatureExtractor.from_pretrained()` 加载 feature extractor：

```
from transformers import AutoFeatureExtractor

feature_extractor = AutoFeatureExtractor.from_pretrained("facebook/wav2vec2-base")
```

将音频样本的数组 `dataset[0]["audio"]["array"]`传递给 feature extractor。：

```
audio_input = [dataset[0]["audio"]["array"]]
feature_extractor(audio_input, sampling_rate=16000) # 建议在 feature extractor 中设置 sampling_rate 参数为音频样本的采样率，以更好地调试可能发生的静音错误
```

就像 `tokenizer` 一样，可以应用填充或截断来处理批次中不同长度的序列。请查看这两个音频样本的序列长度：

```
dataset[0]["audio"]["array"].shape

dataset[1]["audio"]["array"].shape
```

创建一个函数来预处理数据集，以使音频样本具有相同的长度。通过指定样本最大的长度 `max_length`，`feature extractor` 将填充或截断序列以使其匹配：

```
def preprocess_function(examples):
    audio_arrays = [x["array"] for x in examples["audio"]]
    inputs = feature_extractor(
        audio_arrays,
        sampling_rate=16000,
        padding=True,
        max_length=100000,
        truncation=True,
    )
    return inputs
```

将预处理函数 `preprocess_function` 应用于数据集中的前几个示例：

```
processed_dataset = preprocess_function(dataset[:5])
```

现在样本的长度是相同的，并且与指定的最大长度相匹配。现在可以将经过处理的数据集输入模型了！

```
processed_dataset["input_values"][0].shape

processed_dataset["input_values"][1].shape
```

计算机视觉（图像预处理）

对于计算机视觉任务，需要使用 `image processor` 来预处理数据集以供模型使用。

图像预处理包括多个步骤，能够将图像转换为模型期望的输入格式。这些步骤包括调整图像大小、标准化、颜色通道校正以及将图像转换为张量等操作。

图像预处理通常遵循某种形式的图像增强。图像预处理和图像增强都会改变图像数据，但它们有不同的目的：

- `图像增强`可以帮助防止过拟合并增加模型的鲁棒性。你可以在数据增强方面充分发挥创造性，如调整亮度和颜色、裁剪、旋转、调整大小、缩放等，但要注意不要改变图像的含义。
- `图像预处理`确保图像与模型预期的输入格式匹配。在微调计算机视觉模型时，必须保证对样本图像进行与模型训练时相同的预处理。**对于图像预处理，请使用与模型相关联的 `ImageProcessor`。**

加载 food101 数据集（有关如何加载数据集的更多详细信息，请参阅🤗Datasets 教程）以了解如何在计算机视觉数据集中使用图像处理器：

因为图像数据集相当大，请使用 🤗 Datasets 的 `split` 参数加载少量的训练样本.

```
from datasets import load_dataset

dataset = load_dataset("food101", split="train[:100]")
```

使用🤗 Datasets 的 `[Image](https://huggingface.co/docs/datasets/package_reference/main_classes?highlight=image#datasets.Image)` 功能查看图像：

```
dataset[0]["image"]
```



vision-preprocess-tutorial

使用 `AutoImageProcessor.from_pretrained()` 加载 image processor：

```
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
```

首先，让我们进行图像增强。你可以使用任何你喜欢的图像增强库。

如果你有兴趣使用其他的图像增强库，可以参考一下 Albumentations 或 Kornia notebooks 中的示例。

在本教程中，我们将使用 `torchvision` 的 transforms 库来进行图像增强。

54. 在这里，我们使用 Compose 将 RandomResizedCrop 和 ColorJitter 变换连接在一起。**请注意，对于调整大小，我们可以从 image_processor 中获取图像尺寸要求。**对于一些模型，需要定义精确的高度和宽度，而对于其他模型则只需定义 `shortest_edge`。

```
try:
    from torchvision.transforms import RandomResizedCrop, ColorJitter, Compose
except ImportError:
    raise ImportError("'torchvision' could not be resolved. Please install it by 'pip install
torchvision'.")

size = (
    image_processor.size["shortest_edge"]
    if "shortest_edge" in image_processor.size
    else (image_processor.size["height"], image_processor.size["width"]) # 如果不包含
shortest_edge 键，则 size 变量被设置为图像的原始高度和宽度。
)

_transforms = Compose([RandomResizedCrop(size), ColorJitter(brightness=0.5, hue=0.5)])
```

`shortest_edge`：指的是图像的高度和宽度中的较小值，以确保处理后的图像具有一致的最短边长。例如，对于一个尺寸为 800x600 的图像，即 shortest_edge=600。

`image_processor.size`：是一个字典，包含了图像处理的相关尺寸信息。可能包含 `shortest_edge`、`height` 和 `width` 等健。

`RandomResizedCrop(size)`：

- 这个变换会随机裁剪图像并调整到指定的 size。
- 如果 size 是一个整数，则表示裁剪后的图像的最短边长。
- 如果 size 是一个元组 (height, width)，则表示裁剪后的图像的目标尺寸。

55. 模型接受 `pixel_values` 作为输入。`ImageProcessor` 可以进行图像标准化，并生成张量。创建一个函数，将图像增强和图像预处理的步骤组合起来处理批量图像，并生成 `pixel_values`：

```
def transforms(examples):
    images = [_transforms(img.convert("RGB")) for img in examples["image"]]
    examples["pixel_values"] = image_processor(images, do_resize=False,
return_tensors="pt")["pixel_values"]
    return examples
```

默认情况下 `ImageProcessor` 会调整图像的大小。因为我们已经在图像增强转换中调整了图像的大小（`_transforms` 中已经包含了尺寸调整的步骤（如 `RandomResizedCrop`）），所以不需要 `image_processor` 再次调整尺寸，通过设置参数 `do_resize=False` 可以保持图像的原始尺寸。

如果想要将图像标准化步骤为图像增强的一部分，请使用 `image_processor.image_mean` 和 `image_processor.image_std`。

56. 然后使用 🤗 Datasets 的 set_transform 在运行时对图片应用 `transforms` 进行变换：

```
dataset.set_transform(transforms)

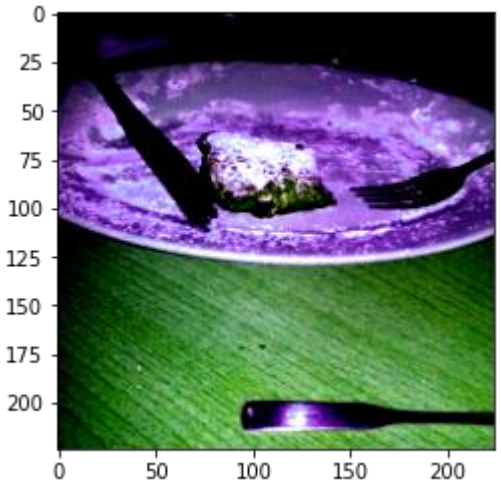

# 当你访问图像时，可以看到`image processor`已添加了`pixel_values（像素值）`。
dataset[0].keys()
```

57. 现在可以将经过处理的数据集输入给模型了！

```
import numpy as np
import matplotlib.pyplot as plt

img = dataset[0]["pixel_values"]
plt.imshow(img.permute(1, 2, 0))
```

这是在应用变换后的图像样子。图像已被随机裁剪，并使其颜色属性发生了变化。



preprocessed_image

对于诸如目标检测、语义分割、实例分割和全景分割等任务，`ImageProcessor` 都提供了训练后处理方法。**这些方法能够将模型的原始输出转换为有意义的预测结果，如边界框或分割地图。**

填充

在某些情况下，例如，在微调 DETR 时，模型在训练阶段应用了尺度增强，这可能导致批处理中的图像大小不同。

我们可以定义一个函数 `collate_fn`，用于在数据加载过程中对每个批次（batch）的数据进行整理和预处理。

在处理图像和文本数据时，`collate_fn` 通常设定为将一个批次中的多个样本整理成一个统一的格式，以便于模型训练。

```
def collate_fn(batch):
    # 1. 提取像素值：从每个样本中提取 pixel_values，这是一个包含图像数据的列表。
    pixel_values = [item["pixel_values"] for item in batch]
    # 2. 填充操作：使用 DetrImageProcessor.pad() 方法对图像数据进行填充，确保所有图像具有相同的尺寸，
返回 PyTorch 张量。
    encoding = image_processor.pad(pixel_values, return_tensors="pt")
    # 3. 提取标签：从每个样本中提取 labels，这是一个包含标签的列表。
    labels = [item["labels"] for item in batch]
    # 4. 创建批次字典：将填充后的像素值、像素掩码和标签整理成一个字典，便于后续的模型输入。
    batch = {}
    batch["pixel_values"] = encoding["pixel_values"]
    batch["pixel_mask"] = encoding["pixel_mask"]
    batch["labels"] = labels
    return batch
```

为什么需要 `collate_fn`

- 批处理一致性：在深度学习中，模型通常以批次为单位进行训练，通过 collate_fn 确保每个批次的数据格式是一致的。
- 数据填充：不同样本的图像尺寸可能不同，通过 collate_fn 中的填充操作可以使它们具有相同的尺寸，便于批处理。
- 掩码生成：掩码可以帮助模型区分实际数据和填充数据，避免模型在训练过程中受到填充数据的干扰。

多模态

对于涉及多模态输入的任务，你需要 processor 来为模型准备数据集。

`Processor` 将两个处理对象组合在一起（例如 tokenizer 和 feature extractor）。

加载 LJ Speech 数据集，有关如何加载数据集的更多详细信息，请参阅🤗 Datasets 教程，以了解如何使用 processor 进行自动语音识别（ASR）：

```
from datasets import load_dataset

lj_speech = load_dataset("lj_speech", split="train")
```

对于 `ASR`（自动语音识别），主要关注 `Audio` 和 `Text`，因此可以删除其他列：

```
lj_speech = lj_speech.map(remove_columns=["file", "id", "normalized_text"])
```

查看 `Audio` 和 `Text` 列：

```
lj_speech[0]["audio"]

lj_speech[0]["text"]
```

注意，你应始终重新采样音频数据集的采样率，与用于预训练模型数据集的采样率保持一致。

```
lj_speech = lj_speech.cast_column("audio", Audio(sampling_rate=16_000))
```

使用 `AutoProcessor.from_pretrained()`加载一个 Processor：

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("facebook/wav2vec2-base-960h")
```

58. 定义一个函数 `prepare_dataset`，用于将包含在 `array` 中的音频数据处理为 `input_values`，并将 `text` 标记为 `labels`。这些将是输入模型的数据：

```
def prepare_dataset(example):
    audio = example["audio"]
    example.update(processor(audio=audio["array"], text=example["text"], sampling_rate=16000))
    return example
```

59. 将 `prepare_dataset` 应用于一个示例：

```
prepare_dataset(lj_speech[0])
```

`Processor` 现在已经添加了 `input_values` 和 `labels`，并且采样率也正确地降低为 16kHz。现在可以将处理后的数据集输入给模型了！

微调预训练模型

🤗 Transformers 提供了涉及各种任务的成千上万的预训练模型。当你使用预训练模型时，你需要在与任务相关的数据集上训练该模型。这种操作被称为`微调`，是一种非常强大的训练技术。

在本教程中，将选择深度学习框架来微调一个预训练模型：

- 使用 🤗 Transformers 的 `Trainer` 来微调预训练模型。
- 在 TensorFlow 中使用 `Keras` 来微调预训练模型。
- 在原生 PyTorch 中微调预训练模型。

准备数据集

在微调预训练模型之前，需要下载一个数据集为训练做好准备。

首先，加载 Yelp 评论数据集：

```
from datasets import load_dataset

dataset = load_dataset("yelp_review_full")
dataset["train"][100]
```

对于文本类型的数据，需要一个 `tokenizer` 来处理文本，包括填充和截断操作以处理长度可变的序列。

如果想要一次性处理你的数据集，可以定义一个预处理函数，再使用 🤗 Datasets 的 `map` 方法将预处理函数应用于整个数据集：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-cased")


def tokenize_function(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True)


tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

还可以在完整的数据集中提取一个较小的子集来进行微调，以减少训练所需的时间：

```
small_train_dataset = tokenized_datasets["train"].shuffle(seed=42).select(range(1000))
small_eval_dataset = tokenized_datasets["test"].shuffle(seed=42).select(range(1000))
```

训练

在接下来，可以应该根据你训练所用的框架来选择对应的教程章节。

使用 PyTorch Trainer 进行训练

🤗 Transformers 提供了一个专为训练 🤗 Transformers 模型而优化的 `Trainer` 类，使你无需手动编写自己的训练循环步骤就可以开始训练模型。

`Trainer API` 还包含了各种训练选项和功能，如日志记录、梯度累积和混合精度。

首先，加载你的模型并指定期望的标签数量。根据 `Yelp Review` 数据集卡片，可以知道有五个标签：

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-cased",
num_labels=5)
```

你会看到一个警告，提到一些预训练权重未被使用，以及一些权重被随机初始化。

不用担心，这是完全正常的！`BERT` 模型预训练的 `head` 会被废弃，替换为一个随机初始化的分类 `head`。在你的序列分类任务上微调这个新模型的 `head`，将预训练模型的知识转移给它。

训练超参数

接下来，创建一个 `TrainingArguments` 类，其中包含所有你可以调整的超参数以及用于激活不同训练选项的标志。

在本教程中，你可以从默认的训练超参数开始，随时可以尝试不同的设置以找到最佳的设置。

指定保存训练检查点的位置：

```
from transformers import TrainingArguments

training_args = TrainingArguments(output_dir="test_trainer")
```

评估

`Trainer` 在训练过程中不会自动评估模型的性能。你需要向 Trainer 传递一个函数来计算和展示模型的性能指标。

🤗 Evaluate 库提供了一个 accuracy 函数，你可以使用 `evaluate.load` 函数加载它（有关更多信息，请参阅此快速入门）：

```
import numpy as np
import evaluate

metric = evaluate.load("accuracy")
```

在 `metric` 上调用 `compute` 来计算预测的准确性。在将预测传递给 `compute` 之前，还需要将预测转换为 `logits`（请记住，所有 🤗 Transformers 模型都返回 logits）：

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```

如果你想要在微调过程中监视评估指标，则需要在训练参数中设置 `eval_strategy` 参数，以在每个 `epoch` 结束时展示评估指标：

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(output_dir="test_trainer", eval_strategy="epoch")
```

训练器

创建一个包含模型、训练参数、训练和测试数据集以及评估函数的 `Trainer` 对象：

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=small_eval_dataset,
    compute_metrics=compute_metrics,
)
```

然后调用 `train()`来微调模型：

```
trainer.train()
```

使用 keras 训练 TensorFlow 模型

在 TensorFlow 中，可以使用 `Keras API` 训练 😭 Transformers 模型！

加载用于 Keras 的数据

当使用 `Keras API` 训练 😭 Transformers 模型时，你需要先将数据集转换为 `Keras` 可理解的格式。

如果你的数据集很小，可以将整个数据集都转换为 `NumPy 数组`并传递给 `Keras`。

首先，需要加载一个数据集。这里会使用 `[GLUE benchmark](https://huggingface.co/datasets/nyu-mll/glue)` 中的 `CoLA` 数据集，因为它是一个简单的二元文本分类任务。

```
from datasets import load_dataset

dataset = load_dataset("glue", "cola")
dataset = dataset["train"]  # 现在只使用训练数据集
```

接下来，加载一个 `tokenizer` 并将数据标记为 `NumPy 数组`。

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-cased")
tokenized_data = tokenizer(dataset["sentence"], return_tensors="np", padding=True)
# 分词器返回一个 BatchEncoding，这里我们将其转换为 Keras 的字典
tokenized_data = dict(tokenized_data)

labels = np.array(dataset["label"])  # 标签已经是一个由 0 和 1 组成的数组，因此我们可以直接将其转换为 NumPy 数组，不需要进行分词处理。
```

最后，加载 compile(编译) 和 fit（训练） 模型：

当你使用 `compile()` 编译模型时，你可以指定损失函数以覆盖默认配置， 也可以不指定，因为 Transformers 模型会自动选择适合其任务和模型架构的损失函数。

```
from transformers import TFAutoModelForSequenceClassification
from tensorflow.keras.optimizers import Adam

# 加载并编译我们的模型
model = TFAutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-cased")
# 指定较低的学习率(optimizer)，通常更适合微调模型
model.compile(optimizer=Adam(3e-5))  # 没有指定损失函数
# 训练模型
model.fit(tokenized_data, labels)
```

这种方法对于较小的数据集效果很好，但对于较大的数据集，可能就会出现问题。为什么呢?

因为分词后的数组和标签必须被完全加载到内存中，由于 NumPy 无法处理“不规则的数组”，所以每个分词后的样本长度都必须被填充到与数据集中最长样本相同的长度。这会导致数组变得更大，而这些 `padding tokens` 也会减慢训练的速度!

将数据加载为 tf.data.Dataset

为了避免训练速度减慢，可以将数据加载为 `tf.data.Dataset`。你可以自定义 `tf.data` 流水线，也可以选择以下两种方便的方法来实现这一点：

- prepare_tf_dataset()：在大多数情况下推荐使用该方法，因为它是模型上的一个方法，可以通过检查模型来自动确定哪些列可用作模型的输入，并丢弃其他列以创建一个更简单、性能更好的数据集。

- `to_tf_dataset`：这个方法更低级，在你想要完全控制数据集的创建方式时，可以通过指定要包含的确切 `columns` 和 `label_cols` 来实现。

在使用 `prepare_tf_dataset()` 之前，还需要将 `tokenizer` 的输出作为列数据添加到数据集，如下代码示例：

```
def tokenize_dataset(data):
    # 返回的键将被添加到数据集中作为列。
    return tokenizer(data["text"])


dataset = dataset.map(tokenize_dataset)
```

注意！Hugging Face 的数据集存储默认在硬盘上，因此这不会增加内存使用！一旦列被添加，你可以从数据集中流式地传输批次数据，并为每个批次添加 `padding tokens`，这与一次性为整个数据集添加 padding tokens 相比，大大减少了 padding tokens 的数量，避免减慢训练的速度。

```
tf_dataset = model.prepare_tf_dataset(dataset["train"], batch_size=16, shuffle=True,
tokenizer=tokenizer)
```

在上面的代码示例中，你需要将 `tokenizer` 传递给 `prepare_tf_dataset()`，以便它可以在加载批次时正确地填充数据集。

- 如果数据集中的所有样本都具有相同的长度而且不需要填充，则可以跳过此参数。
- 如果需要执行比填充样本更复杂的操作（例如，用于掩码语言模型的 tokens 替换），则可以使用 `collate_fn` 参数，而不是传递一个自定义函数来将样本列表转换为批次并应用任何所需的预处理操作。

在创建了 `tf.data.Dataset` 后，你可以像以前一样编译和训练模型：

```
# 指定较低的学习率(optimizer)，通常更适合微调模型
model.compile(optimizer=Adam(3e-5))  # 没有指定损失函数
# 训练模型
model.fit(tf_dataset)
```

在原生 PyTorch 中训练

Trainer 负责训练循环，允许你在一行代码中微调模型。

对于喜欢编写自定义的训练循环的用户，也可以在原生 PyTorch 中微调 🤗 Transformers 模型。

现在，你可能需要重新启动 notebook，或执行以下代码以释放一些内存：

```
del model
del trainer
torch.cuda.empty_cache()
```

接下来，手动处理 tokenized_dataset 以准备进行训练：

60. 移除 text 列，因为模型不接受原始文本作为输入：

```
tokenized_datasets = tokenized_datasets.remove_columns(["text"])
```

61. 将 label 列重命名为 labels，因为模型期望参数的名称是 labels：

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
```

62. 设置数据集返回的格式是 PyTorch 张量 而不是 lists：

```
tokenized_datasets.set_format("torch")
```

接着，创建一个先前展示的数据集的较小子集，以加速微调过程

```
small_train_dataset = tokenized_datasets["train"].shuffle(seed=42).select(range(1000))
small_eval_dataset = tokenized_datasets["test"].shuffle(seed=42).select(range(1000))
```

预处理数据 - DataLoader

为你的训练和测试数据集创建一个 DataLoader 类，以便在迭代时可以处理数据批次

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(small_train_dataset, shuffle=True, batch_size=8)
eval_dataloader = DataLoader(small_eval_dataset, batch_size=8)
```

加载你的模型，并指定期望的标签数量：

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-cased",
num_labels=5)
```

优化器（Optimizer）和学习率调度器（Learning Rate Scheduler)

- Optimizer : 优化器是用于调整模型参数以最小化损失函数的算法，常见的算法有 SGD、Adam、RMSprop 等。在训练过程中，优化器根据损失函数的梯度来更新模型的权重和偏置，从而逐步改进模型的性能。

- Learning Rate Scheduler : 学习率调度器用于在训练过程中动态调整学习率，常见的有 StepLR、ExponentialLR、ReduceLROnPlateau 等。适当调整学习率可以帮助模型更快地收敛，并避免过拟合。

创建一个 `optimizer` 和 `learning rate scheduler` 以进行模型微调。让我们使用 PyTorch 中的 `[AdamW](https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html)` 优化器：

```
from torch.optim import AdamW

optimizer = AdamW(model.parameters(), lr=5e-5)
```

创建来自 `Trainer` 的默认 `learning rate scheduler`：

```
from transformers import get_scheduler

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    name="linear", optimizer=optimizer, num_warmup_steps=0,
num_training_steps=num_training_steps
)
```

最后，指定 `device` 以使用 GPU（如果有的话）。否则，使用 CPU 进行训练可能需要几个小时。

`Metal Performance Shaders (MPS)`：PyTorch 通过 MPS 后端实现对 Apple GPU 的加速。MPS 是 Apple 提供的一套用于高性能图形和计算任务的框架，但 PyTorch 对 MPS 的支持仍处于实验阶段，可靠性未得到保证。

`Compute Unified Device Architecture (CUDA)`：通过 CUDA，PyTorch 能够在 NVIDIA 的 GPU 上进行高效的并行计算，特别是在深度学习和高性能计算任务中。**确保你的 PyTorch 版本与安装的 CUDA 版本兼容。**

下面是在不同操作系统中使用不同 GPU 加速模型训练的示例：

```
# NVIDIA GPU - CUDA
import torch

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
model.to(device)
```

```
# Apple GPU - MPS
import torch

device = torch.device("mps") if torch.backends.mps.is_available() else torch.device("cpu")
model.to(device)
```

如果没有 GPU，可以通过 notebook 平台如 Colaboratory 或 SageMaker StudioLab 来免费获得云端 GPU 使用。

训练循环

为了跟踪训练进度，使用 tqdm 库来添加一个进度条，显示训练步数的进展：

```
from tqdm.auto import tqdm

progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

评估

就像你在 Trainer 中添加了一个评估函数一样，当你编写自定义的训练循环时，也需要做同样的事情。但与在每个 `epoch` 结束时计算和展示指标不同，这一次将使用 `add_batch` 累积所有批次，在最后计算指标。

```
import evaluate

metric = evaluate.load("accuracy")
model.eval()
for batch in eval_dataloader:
    batch = {k: v.to(device) for k, v in batch.items()}
    with torch.no_grad():
        outputs = model(**batch)

    logits = outputs.logits
    predictions = torch.argmax(logits, dim=-1)
    metric.add_batch(predictions=predictions, references=batch["labels"])

metric.compute()
```

附加资源

更多微调例子可参考如下链接：

🤗 Transformers 示例 包含用于在 PyTorch 和 TensorFlow 中训练常见自然语言处理任务的脚本。

🤗 Transformers 笔记 包含针对特定任务在 PyTorch 和 TensorFlow 中微调模型的各种 notebook。

使用脚本进行训练

除了 🤗 Transformers notebooks，还有示例脚本演示了如何使用 PyTorch、TensorFlow 或 JAX/Flax 训练模型以解决特定任务。

你还可以在这些示例中找到我们在研究项目和遗留示例中使用过的脚本，这些脚本主要是由社区贡献的。这些脚本已不再被积极维护，可能与🤗 Transformers 库的最新版本不兼容，需要使用指定版本的🤗 Transformers 。

示例脚本可能无法在初始配置下直接解决每个问题，你可能需要根据要解决的问题调整脚本。为了帮助你，大多数脚本都完全暴露了数据预处理的方式，允许你根据需求对其进行编辑。

对于你想在示例脚本中实现的任何功能，都可以在论坛或 issue 上讨论，然后再提交 Pull Request。我们欢迎修复错误，但不太可能会合并那些帮助添加了更多功能的 Pull Request，因为这会降低可读性。

本指南将向你展示如何在 PyTorch 和 TensorFlow 中运行摘要训练脚本的示例。除非有特别说明，否则所有示例都可以在两个框架中工作。

设置

要成功运行示例脚本的最新版本，你必须在新的虚拟环境中，选择从源代码安装 🤗 Transformers：

```
git clone https://github.com/huggingface/transformers
cd transformers
pip install .
```

然后切换到你 clone 的 🤗 Transformers 库到特定的版本，例如 v3.5.1：

```
git checkout tags/v3.5.1
```

在安装了正确的🤗 Transformers 库版本之后，进入你选择的版本的 `example` 文件夹并安装示例要求的环境：

```
pip install -r requirements.txt
```

运行脚本

Pytorch

示例脚本从🤗 Datasets 库下载并预处理数据集。然后，脚本通过 Trainer 使用支持摘要任务的架构对数据集进行微调。

以下示例展示了如何在 CNN/DailyMail 数据集上微调 T5-small。

由于 T5 模型是通过样本内容的前缀来区分不同的任务（例如翻译、问答、摘要等），因此在示例中需要设置一个额外的参数 `--source_prefix "summarize: "`，通过这个前缀提示 T5 这是一个摘要任务。

```
python examples/pytorch/summarization/run_summarization.py \
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --overwrite_output_dir \
    --predict_with_generate
```

TensorFlow

示例脚本从🤗 Datasets 库下载并预处理数据集。然后，脚本通过 Keras 使用支持摘要任务的架构对数据集进行微调。

以下示例展示了如何在 CNN/DailyMail 数据集上微调 T5-small。

由于 T5 模型的训练方式，它也需要设置一个额外的参数 `source_prefix`，这个提示让 T5 知道这是一个摘要任务。

```
python examples/tensorflow/summarization/run_summarization.py  \
    --model_name_or_path google-t5/t5-small \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization  \
    --per_device_train_batch_size 8 \
    --per_device_eval_batch_size 16 \
    --num_train_epochs 3 \
    --do_train \
    --do_eval
```

分布式训练和混合精度

Trainer 支持分布式训练和混合精度，这意味着你也可以在脚本中使用它。要启用这两个功能，可以做如下设置：

- 添加参数 `fp16`，启用混合精度。
- 添加参数 `nproc_per_node`，设置使用的 GPU 数量。

```
torchrun \
    --nproc_per_node 8 pytorch/summarization/run_summarization.py \
    --fp16 \
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --overwrite_output_dir \
    --predict_with_generate
```

TensorFlow 脚本使用 `MirroredStrategy` 进行分布式训练，所以无需在训练脚本中添加任何其他参数。如果可用，TensorFlow 脚本将默认使用多个 GPU。

在 TPU 上运行脚本

PyTorch

张量处理单元 `TPUs` 是专门设计用于加速性能的。PyTorch 使用 `XLA` 深度学习编译器来支持 TPU（更多细节请参见这里）。

要使用 TPU，需要启动 `xla_spawn.py` 脚本并使用参数 `num_cores` 设置要使用的 TPU 核心数量。

```
python xla_spawn.py --num_cores 8 \
    summarization/run_summarization.py \
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --overwrite_output_dir \
    --predict_with_generate
```

TensorFlow

张量处理单元（TPUs）是专门设计用于加速性能的。TensorFlow 脚本使用 `TPUStrategy` 在 TPU 上进行训练。

要使用 TPU，需要将要使用的 TPU 资源的名称传递给 `tpu` 参数。

```
python run_summarization.py  \
    --tpu name_of_tpu_resource \
    --model_name_or_path google-t5/t5-small \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --output_dir /tmp/tst-summarization  \
    --per_device_train_batch_size 8 \
    --per_device_eval_batch_size 16 \
    --num_train_epochs 3 \
    --do_train \
    --do_eval
```

基于🤗 Accelerate 运行脚本

🤗 Accelerate 是一个仅支持 PyTorch 的库，它会自动检测可用的硬件资源（包括 CPU、GPU 和 TPU）并相应地配置训练环境，对外提供了一个统一的训练接口，让开发者可以在不同硬件上以相同的方式启动训练过程，同时保持对 PyTorch 训练循环的完全可见性。

通过 Accelerate，你可以用更少的代码实现复杂的分布式训练设置。如果你还没有安装 🤗 Accelerate，请确保你已经安装了它：

注意：由于 Accelerate 正在快速发展，因此必须安装 git 版本的 accelerate 来运行脚本。

```
pip install git+https://github.com/huggingface/accelerate
```

你需要使用 `run_summarization_no_trainer.py` 脚本，而不是 `run_summarization.py` 脚本。

🤗 Accelerate 支持的脚本需要在文件夹中有一个 `task_no_trainer.py` 文件。首先运行以下命令来创建和保存配置文件：

```
accelerate config
```

检测你的设置以确保配置正确：

```
accelerate test
```

现在你可以开始训练模型了：

```
accelerate launch run_summarization_no_trainer.py \
    --model_name_or_path google-t5/t5-small \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir ~/tmp/tst-summarization
```

使用自定义数据集

摘要脚本支持自定义数据集，只要它们是 CSV 或 JSON Line 文件。当你使用自己的数据集时，需要指定一些额外的参数：

- `train_file` 和 `validation_file` 分别是指定你的训练和验证文件的路径。
- `text_column` 是指定你的数据集中包含输入文本的列名。具体来说，它指定了数据集中哪一列的文本数据将被用作模型的输入，即指定需要进行摘要的原始文本。

- `summary_column` 是指定你的数据集中包含目标摘要文本的列名。具体来说，它指定了数据集中哪一列的文本数据被用作模型的输出目标，即你期望模型生成的摘要文本。

使用了自定义数据集的摘要脚本看起来是这样的：

```
python examples/pytorch/summarization/run_summarization.py \
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --train_file path_to_csv_or_jsonlines_file \
    --validation_file path_to_csv_or_jsonlines_file \
    --text_column text_column_name \
    --summary_column summary_column_name \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --overwrite_output_dir \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --predict_with_generate
```

测试脚本

通常，在提交整个数据集之前，最好先在较少的数据集上运行脚本，以确保一切按预期工作。因为处理完整的数据集可能需要花费几个小时。可以添加以下三个参数来截断数据集：

- max_train_samples：控制训练数据集的最大样本数。
- max_eval_samples：控制验证数据集的最大样本数。
- max_predict_samples：控制预测数据集的最大样本数。

通过合理地设置这些参数，你可以在处理大型数据集之前，先在较小的数据集上快速验证和调试你的脚本，确保一切按预期工作，从而节省时间和资源。

```
python examples/pytorch/summarization/run_summarization.py \
    --model_name_or_path google-t5/t5-small \
    --max_train_samples 50 \
    --max_eval_samples 50 \
    --max_predict_samples 50 \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
```

```
    --overwrite_output_dir \
    --predict_with_generate
```

并非所有示例脚本都支持 `max_predict_samples` 参数。如果你不确定你的脚本是否支持此参数，可以使用 -h 参数先进行检查：

```
examples/pytorch/summarization/run_summarization.py -h
```

从 checkpoint 恢复训练

另一个方式是从之前的 checkpoint 恢复训练。确保在训练中断时，可以从之前停止的地方继续进行，而无需重新开始训练。

有两种方法可以从 checkpoint 恢复训练：

63. 第一种方法是设置参数 `output_dir`，从存储在 output_dir 中的最新的 checkpoint 恢复训练。

    在这种情况下，你应该删除 `overwrite_output_dir`：

```
python examples/pytorch/summarization/run_summarization.py
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --output_dir previous_output_dir \
    --predict_with_generate
```

64. 第二种方法设置参数 `resume_from_checkpoint` 从指定的 checkpoint 文件夹恢复训练。

```
python examples/pytorch/summarization/run_summarization.py
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --overwrite_output_dir \
    --resume_from_checkpoint path_to_specific_checkpoint \
    --predict_with_generate
```

分享模型

所有的脚本都可以将你的最终模型上传到 Model Hub。在开始之前，请确保你已登录 Hugging Face：

```
huggingface-cli login
```

在脚本中添加参数 `push_to_hub`。这个参数会默认创建一个带有你的 Hugging Face 用户名 和 output_dir 中指定的文件夹的名称的仓库。

想要给你的仓库指定名称，可以添加参数 `push_to_hub_model_id` 来设置。该仓库将在你的命名空间下列出。

以下示例展示了如何上传具有指定的仓库名称的模型：

```
python examples/pytorch/summarization/run_summarization.py
    --model_name_or_path google-t5/t5-small \
    --do_train \
    --do_eval \
    --dataset_name cnn_dailymail \
    --dataset_config "3.0.0" \
    --source_prefix "summarize: " \
    --push_to_hub \
    --push_to_hub_model_id finetuned-t5-cnn_dailymail \
    --output_dir /tmp/tst-summarization \
    --per_device_train_batch_size=4 \
    --per_device_eval_batch_size=4 \
    --overwrite_output_dir \
    --predict_with_generate
```

使用 Accelerate 进行分布式训练

随着模型变得越来越大，并行性已经成为在有限硬件上训练更大模型和加速训练速度的策略，增加了数个数量级。

在 Hugging Face 中我们创建了🤗 加速库 Accelerate，以帮助用户在任何类型的分布式设置上轻松训练🤗 Transformers 模型，无论是在一台机器上的多个 GPU 还是在多个机器上的多个 GPU。

在本教程中，将了解如何自定义原生 PyTorch 训练循环，以启用在分布式环境中的训练。

设置

通过安装 🤗 Accelerate 开始:

```
pip install accelerate
```

然后导入并创建 `Accelerator` 对象。Accelerator 将自动检测分布式设置的类型，并初始化所有必要的训练组件。你不需要显式地设置将模型放在哪个设备上。

```
from accelerate import Accelerator

accelerator = Accelerator()
```

准备加速

下一步是将所有相关的训练对象传递给 `prepare` 方法。这包括训练和评估的 DataLoader、一个模型和一个优化器:

- `train_dataloader` 用于训练阶段，加载数据并分批提供给模型进行训练。
- `eval_dataloader` 用于评估阶段，加载数据并分批提供给模型进行性能评估。

accelerator.prepare 的作用：

在 Hugging Face Accelerate 库中，accelerator.prepare 方法用于将 DataLoader、模型和优化器等对象包装起来，以便在分布式训练环境中正确地管理它们。具体作用包括：

- 自动分发：将数据、模型和优化器自动分发到不同的 GPU 或 TPU 上。
- 同步：在分布式训练中，确保各个进程之间的数据同步。
- 优化：提供性能优化，如自动混合精度训练。

```
train_dataloader, eval_dataloader, model, optimizer = accelerator.prepare(
    train_dataloader, eval_dataloader, model, optimizer
)
```

反向传播

最后将训练循环中的典型 loss.backward（）替换为 RoundAccelerate 的 backward 方法：

```
for epoch in range(num_epochs):
    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss
```

```
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

如以下的代码所见，你只需要添加四行额外的代码到你的训练循环中即可启用分布式训练！

```
from accelerate import Accelerator
from transformers import AdamW, AutoModelForSequenceClassification, get_scheduler

accelerator = Accelerator()

model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
optimizer = AdamW(model.parameters(), lr=3e-5)

# - device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
# - model.to(device)

# Add patch #
train_dataloader, eval_dataloader, model, optimizer = accelerator.prepare(
    train_dataloader, eval_dataloader, model, optimizer
)
# Add patch #

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps
)

progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
# -     batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
# -     loss.backward()
        accelerator.backward(loss) # Add

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

训练

在添加了相关代码行后，可以在脚本或 notebook（如 Colaboratory）中启动训练。

用脚本训练

如果在脚本中运行训练，请运行以下命令以创建和保存配置文件:

```
accelerate config
```

然后使用以下命令启动训练:

```
accelerate launch train.py
```

用 notebook 训练

使用 Colaboratory 的 TPU 支持在 notebook 中运行 Accelerate。它将所有负责训练的代码包装在一个函数中，并将其传递给 notebook_launcher:

```
from accelerate import notebook_launcher

notebook_launcher(training_function)
```

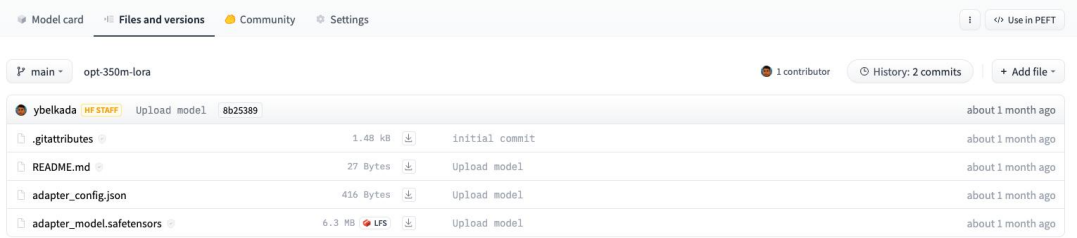
有关 🤗 Accelerate 及其丰富功能等更多信息，可参阅文档。

使用 PEFT 加载和训练 Adapters

PEFT（Parameter-Efficient Fine-Tuning）是一种高效微调预训练模型的方法。具体来说，在微调过程中，预训练模型的大部分参数都被冻结，通过添加少量可训练的参数（如 adapters、low-rank matrices 等），调整它们以适应特定任务，而无需对整个预训练模型进行微调。

`Adapters` 是一组轻量级的、可训练的参数，通常被添加到预训练模型的特定层中。由于 adapters 是独立的参数模块，通常比完整的模型小一个数量级，所以使其具有方便共享、存储和加载的特点。

如图所示，与完整尺寸的模型权重（约为 700MB）相比，存储在 Hub 上的 OPTForCausalLM 模型的 adapter 权重仅约为 6MB:



PEFT-hub-screenshot

通过使用 PEFT 和 adapters，可以在保持预训练模型大部分参数不变的情况下，高效地微调模型以适应特定任务，从而节省资源并提高性能。如果你对学习更多关于 🤗 PEFT 库 感兴趣，请查看文档。

设置

peft 库是一个实现了参数高效微调（PEFT）方法的库，支持 LoRA（Low-Rank Adaptation）、Adapter Tuning、Prefix Tuning 等多种方法。

peft 库的核心组件：

- 配置类：如 `LoraConfig`、`AdapterConfig` 等配置类，用于定义和配置特定的 PEFT adapter，它将所有 Adapter 相关的配置参数都封装在一个独立的对象中，使得可以轻松地调整 Adapter 的配置，而不需要修改模型的其他部分。
- 模型接口：提供了 `add_adapter`、`freeze_model` 等方法，用于在模型中添加和配置 adapter。
- 训练工具：提供了训练和评估工具，帮助开发者高效地进行模型微调和评估。

首先安装 🤗 peft：

```
pip install peft
```

如果你想尝试全新的特性，需要源代码安装这个库：

```
pip install git+https://github.com/huggingface/peft.git
```

支持的 PEFT 模型

Transformers 原生支持一些 PEFT 库的方法，这意味着你可以加载在本地存储或在 Hub 上的 adapter 权重，只需要使用几行代码就可以轻松地运行或训练它们。以下是受支持的方法：

- Low Rank Adapters

- IA3
- AdaLoRA

如果你想使用其他 PEFT 库的方法，例如提示学习、提示微调，或者关于通用的 🤗 PEFT 库，可以参阅文档。

加载 PEFT adapter 模型

想从 huggingface 的 Transformers 库中加载并使用 PEFT adapter 模型，**请确保 Hub 仓库或本地目录包含一个 `adapter_config.json` 文件和 `adapter` 权重。**

确认存在后，可以使用 `AutoModelFor` 类加载 `PEFT adapter` 模型。例如，想要给一个因果语言建模加载一个 PEFT adapter 模型：

- 指定 PEFT 模型的 id
- 将其传递给 AutoModelForCausalLM 类

```
from transformers import AutoModelForCausalLM, AutoTokenizer

peft_model_id = "ybelkada/opt-350m-lora"
model = AutoModelForCausalLM.from_pretrained(peft_model_id)
```

你可以使用 AutoModelFor 类或基础模型类（如 OPTForCausalLM 或 LlamaForCausalLM ）来加载一个 PEFT adapter。

你也可以通过 `load_adapter` 方法来加载 PEFT adapter。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "facebook/opt-350m"
peft_model_id = "ybelkada/opt-350m-lora"

model = AutoModelForCausalLM.from_pretrained(model_id)
model.load_adapter(peft_model_id)
```

基于 8bit 或 4bit 进行加载

`bitsandbytes` 集成支持 `8bit` 和 `4bit` 精度数据类型，这对于加载大模型非常有用，因为它可以节省内存（请参阅 bitsandbytes 指南以了解更多信息）。

想要有效地将模型分配到你的硬件，请在 `from_pretrained()` 中添加参数 `load_in_8bit` 或 `load_in_4bit`，并设置 `device_map="auto"`：

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

peft_model_id = "ybelkada/opt-350m-lora"
model = AutoModelForCausalLM.from_pretrained(peft_model_id,
quantization_config=BitsAndBytesConfig(load_in_8bit=True))
```

添加新的 adapter

你可以使用 `peft.PeftModel.add_adapter` 方法为一个已有 adapter 的模型添加一个新的 adapter，只要新的 adapter 的类型与当前 adapter 相同即可。

例如，如果你想添加一个 LoRA adapter 到模型上：

```
from transformers import AutoModelForCausalLM, OPTForCausalLM, AutoTokenizer
from peft import LoraConfig

model_id = "facebook/opt-350m"
model = AutoModelForCausalLM.from_pretrained(model_id)

lora_config = LoraConfig(
    target_modules=["q_proj", "k_proj"],
    init_lora_weights=False
)

model.add_adapter(lora_config, adapter_name="adapter_1")
```

添加一个新的 adapter：

```
# 使用相同配置连接新的 adapter，并取别名为 adapter_2
model.add_adapter(lora_config, adapter_name="adapter_2")
```

现在你可以使用 `peft.PeftModel.set_adapter` 来设置要使用的 adapter。

```
# 使用 adapter_1
model.set_adapter("adapter_1")
output = model.generate(**inputs)
print(tokenizer.decode(output_disabled[0], skip_special_tokens=True))

# 使用 adapter_2
model.set_adapter("adapter_2")
output_enabled = model.generate(**inputs)
print(tokenizer.decode(output_enabled[0], skip_special_tokens=True))
```

启用和禁用 adapters

一旦你将 adapter 添加到模型中，你随时可以选择启用或禁用 adapter 模块。

若想要启用 adapter 模块：

```
from transformers import AutoModelForCausalLM, OPTForCausalLM, AutoTokenizer
from peft import PeftConfig

model_id = "facebook/opt-350m"
adapter_model_id = "ybelkada/opt-350m-lora"
tokenizer = AutoTokenizer.from_pretrained(model_id)
text = "Hello"
inputs = tokenizer(text, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(model_id)
peft_config = PeftConfig.from_pretrained(adapter_model_id)

# to initiate with random weights
peft_config.init_lora_weights = False

model.add_adapter(peft_config)
model.enable_adapters()
output = model.generate(**inputs)
```

若想要禁用 adapter 模块：

```
model.disable_adapters()
output = model.generate(**inputs)
```

训练一个 PEFT adapter

PEFT 适配器受 Trainer 类支持，因此你可以选择特定的用例来训练 adapter。

只需要添加几行代码即可。例如，要训练一个 LoRA adapter：

65. 创建一个 LoraConfig 或其他**适配器配置类**的实例，并设置所需的超参数。

这里使用 PEFT 库中的 LoraConfig 类来配置 LoRA adapter，其中需要配置的超参数如下：

- lora_alpha：控制低秩矩阵的大小，通常称为 "rank" 或 "r"。在这个例子中，lora_alpha=16 指定了低秩矩阵的大小。
- lora_dropout：用于指定 dropout 的概率，lora_dropout=0.1 表示 dropout 的概率为 10%。

- r：表示低秩矩阵的秩。在这个例子中，r=64 指定了低秩矩阵的秩为 64。
- bias：用于指定是否添加偏差项。在这个例子中，bias="none" 表示不添加偏差项。
- task_type：用于指定任务的类型。在这个例子中，task_type="CAUSAL_LM" 表示任务是因果语言模型。

```
from peft import LoraConfig

peft_config = LoraConfig(
    lora_alpha=16,
    lora_dropout=0.1,
    r=64,
    bias="none",
    task_type="CAUSAL_LM",
)
```

66. 将 adapter 添加到模型中。

```
model.add_adapter(peft_config)
```

67. 现在可以将添加了 adapter 的模型传递给 Trainer 进行训练了！

```
trainer = Trainer(..., model=model)
trainer.train()
```

保存训练好的 adapter 并重新加载它：

```
model.save_pretrained(save_dir)
model = AutoModelForCausalLM.from_pretrained(save_dir)
```

如何分享模型

在 Hugging Face，我们相信公开分享知识和资源，能实现人工智能的普及化，让每个人都能受益。我们鼓励你将你的模型与社区分享，以帮助他人节省时间和精力。

要与社区共享模型，你需要在 huggingface.co 上拥有一个帐户。你还可以加入现有的组织或创建一个新的组织。

在本教程中，你将学习两种在 Model Hub 上共享训练好的或微调的模型的方法：

- 通过编程将文件推送到 Hub。
- 使用 Web 界面将文件拖放到 Hub。

仓库功能

Model Hub 上的每个仓库都像是一个典型的 GitHub 仓库。我们的仓库提供版本控制、提交历史记录以及可视化差异的能力。

`Model Hub` 的内置版本控制基于 `git` 和 `git-lfs`。换句话说，你可以将一个模型视为一个仓库，从而实现更好的访问控制和可扩展性。版本控制允许使用修订方法来固定特定版本的模型，可以使用提交哈希值、标签或分支来标记。

因此，你可以通过 revision 参数加载特定的模型版本：

```
model = AutoModel.from_pretrained(
    "julien-c/EsperBERTo-small", revision="v2.0.1"  # tag name, or branch name, or commit hash
)
```

文件也可以轻松地在仓库中编辑，你可以查看提交历史记录以及差异：



vis_diff

设置

在将模型共享到 Hub 之前，你需要拥有 Hugging Face 的凭证。如果你有访问终端的权限，请在安装🤗 Transformers 的虚拟环境中运行以下命令。这将在你的 Hugging Face 缓存文件夹（默认为 `~/.cache/`）中存储你的 `access token`：

```
huggingface-cli login
```

如果你正在使用像 `Jupyter` 或 `Colaboratory` 这样的 notebook，请确保你已安装了 huggingface_hub 库。该库允许你以编程方式与 Hub 进行交互。

```
pip install huggingface_hub
```

然后使用 notebook_login 登录到 Hub，并按照这里的链接生成一个 token 进行登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

转换模型适用于所有框架

为确保你的模型可以被使用不同框架的人使用，我们建议你将 PyTorch 和 TensorFlow checkpoints 都转换并上传。如果你跳过此步骤，用户仍然可以从其他框架加载你的模型，但速度会变慢，因为🤗 Transformers 需要实时转换 checkpoints。

为另一个框架转换 checkpoints 很容易。确保你已安装 PyTorch 和 TensorFlow（请参阅此处的安装说明），然后在其他框架中找到适合你任务的特定模型。

PyTorch

指定 `from_tf=True` 将 checkpoint 从 TensorFlow 转换为 PyTorch。

```
pt_model = DistilBertForSequenceClassification.from_pretrained("path/to/awesome-name-you-picked", from_tf=True)
pt_model.save_pretrained("path/to/awesome-name-you-picked")
```

TensorFlow

指定 `from_pt=True` 将 checkpoint 从 PyTorch 转换为 TensorFlow 。

```
tf_model = TFDistilBertForSequenceClassification.from_pretrained("path/to/awesome-name-you-picked", from_pt=True)
```

然后，你可以使用新的 checkpoint 保存你的新 TensorFlow 模型：

```
tf_model.save_pretrained("path/to/awesome-name-you-picked")
```

JAX

如果模型在 Flax 中可用，你还可以将 PyTorch checkpoint 转换为 Flax：

```
flax_model = FlaxDistilBertForSequenceClassification.from_pretrained(
    "path/to/awesome-name-you-picked", from_pt=True
)
```

在训练过程中推送模型

PyTorch

将模型分享到 Hub 就像添加一个额外的参数或回调函数一样简单。请记住，在微调教程中，`TrainingArguments` 类是你指定超参数和附加训练选项的地方。其中一项训练选项包括直接将模型推送到 `Hub` 的能力。在你的 `TrainingArguments` 中设置 `push_to_hub=True`：

```
training_args = TrainingArguments(output_dir="my-awesome-model", push_to_hub=True)
```

像往常一样将你的训练参数传递给 `Trainer`：

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=small_eval_dataset,
    compute_metrics=compute_metrics,
)
```

在你微调完模型后，在 `Trainer` 上调用 `push_to_hub()`将训练好的模型推送到 Hub。😊

Transformers 甚至会自动将训练超参数、训练结果和框架版本添加到你的模型名片中！

```
trainer.push_to_hub()
```

TensorFlow

使用 PushToHubCallback 将模型分享到 Hub。在 PushToHubCallback 函数中，添加以下内容：

- 一个用于存储模型的输出目录。
- 一个 tokenizer。
- `hub_model_id`，即你的 Hub 用户名和模型名称。

```
from transformers import PushToHubCallback

push_to_hub_callback = PushToHubCallback(
    output_dir="./your_model_save_path", tokenizer=tokenizer, hub_model_id="your-username/my-awesome-model"
)
```

将回调函数添加到 fit 中，然后😊 Transformers 会将训练好的模型推送到 Hub：

使用 push_to_hub 功能

你可以直接在你的模型上调用 `push_to_hub` 来将其上传到 Hub。

在 `push_to_hub` 中指定你的模型名称：

```
pt_model.push_to_hub("my-awesome-model")
```

这会在你的用户名下创建一个名为 `my-awesome-model` 的仓库。用户现在可以使用 `from_pretrained` 函数加载你的模型：

```
from transformers import AutoModel

model = AutoModel.from_pretrained("your_username/my-awesome-model")
```

如果你属于一个组织，并希望将你的模型推送到组织名称下，只需将其添加到 `repo_id` 中：

```
pt_model.push_to_hub("my-awesome-org/my-awesome-model")
```

`push_to_hub` 函数还可以用于向模型仓库添加其他文件。例如，向模型仓库中添加一个 `tokenizer`：

```
tokenizer.push_to_hub("my-awesome-model")
```

或者，你可能想要将你的微调后的 PyTorch 模型的 TensorFlow 版本添加进去：

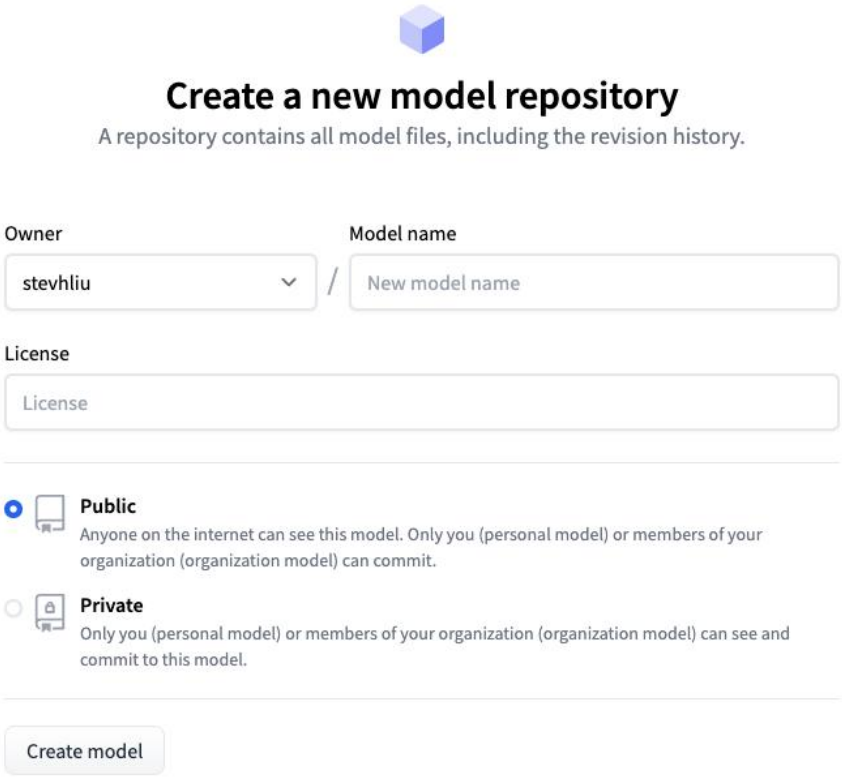
```
tf_model.push_to_hub("my-awesome-model")
```

现在，当你导航到你的 Hugging Face 个人资料时，你应该看到你新创建的模型仓库。点击文件选项卡将显示你已上传到仓库的所有文件。

有关如何创建和上传文件到仓库的更多详细信息，请参考 Hub 文档。

使用 Web 界面上传

喜欢无代码方法的用户可以通过 Hugging Face 的 Web 界面上传模型。访问 `huggingface.co/new` 创建一个新的仓库：



new_model_repo

从这里开始，添加一些关于你的模型的信息：

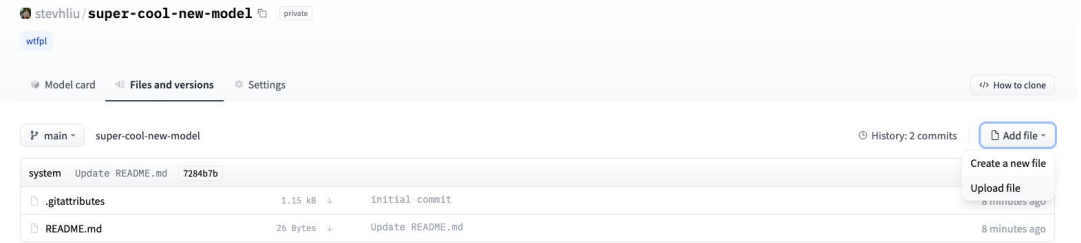
68. 选择仓库的所有者。这可以是你本人或者你所属的任何组织。

69. 为你的项目选择一个名称，该名称也将成为仓库的名称。

70. 选择你的模型是公开还是私有。

71. 指定你的模型的许可证使用情况。

现在点击文件选项卡，然后点击添加文件按钮将一个新文件上传到你的仓库。接着拖放一个文件进行上传，并添加提交信息。



upload_file

添加模型名片

为了确保用户了解你的模型的能力、限制、潜在偏差和伦理考虑，请在仓库中添加一个模型名片。

模型名片在 `README.md` 文件中定义。你可以通过以下方式添加模型名片：

- 手动创建并上传一个 `README.md` 文件。
- 在你的模型仓库中点击编辑模型名片按钮。

可以参考 DistilBert 的模型名片来了解模型名片应该包含的信息类型。有关你可以在 `README.md` 文件中控制的更多选项的细节，例如模型的碳足迹或小部件示例，请参考文档。

Transformers Agents 快速上手

注意！`Transformers Agents`是一个实验性的随时可能发生变化的 API。由于 API 或底层模型可能发生变化，agents 返回的结果也会有所不同。

Transformers 版本 `v4.29.0` 基于 `tools` 和 `agents` 概念构建。你可以在此 Cola 链接中进行测试。

简而言之，它在 Transformers 之上提供了一个自然语言 API：

- 定义了一组经过筛选的 `tools`，并设计了一个 `agents` 来解读自然语言并使用这些 `tools` 。
- 具有很强的可扩展性。
- 筛选了一些相关的 `tools`，但我们将向你展示如何通过社区开发的 `tool` 轻松地扩展系统。

Agent & Tool

Agents

这里的 Agents 本质是一个大型语言模型，我们可以通过提示让它访问特定的工具集。

大型语言模型在生成小代码示例方面表现出色，因此这个 API 利用这一特点，通过提示 LLM 生成一个使用 tools 集合的小代码示例。然后，根据你给 Agents 的任务和 tools 的描述来完成此提示。

Tools

Tools 是有名称和描述的单个函数。然后，我们使用这些 tools 的描述来提示代理。通过提示，让 agent 理解并学会如何使用 tool 来执行查询语言中请求的操作。

注意，**tools 不是 pipelines**，因为 agent 能够通过提示编写更具原子化的 tools，旨在专注于一个非常简单的任务。而 pipelines 经常被重构，且通常是将多个任务合并为一个。

实例化 agent

要使用 `agent.run`，你需要实例化一个 agent。

请注意，实例化的 agent 本质上是一个大型语言模型（LLM），因此你略有变化的提示可能会产生完全不同的结果。所以请尽可能清晰地解释你要执行的任务。

我们支持 `OpenAI` 模型以及来自 BigCode 和 OpenAssistant 的开源替代方案。OpenAI 模型性能更好（但需要你拥有 OpenAI API 密钥，因此无法免费使用），Hugging Face 为 BigCode 和 OpenAssistant 模型提供了免费访问端点。

一开始请安装 `agents` 附加模块，以安装所有默认依赖项。

```
pip install transformers[agents]
```

OpenAI

要使用 OpenAI 模型，可以在安装 `openai` 库后实例化一个 `OpenAiAgent`：

```
pip install openai

from transformers import OpenAiAgent

agent = OpenAiAgent(model="text-davinci-003", api_key="<your_api_key>")
```

BigCode or OpenAssistant

要使用 `BigCode` 或 `OpenAssistant`，需要先登录以访问 `Inference API` 再实例化 `agent`：

```
from huggingface_hub import login

login("<YOUR_TOKEN>")

from transformers import HfAgent

# Starcoder
agent = HfAgent("https://api-inference.huggingface.co/models/bigcode/starcoder")
# StarcoderBase
# agent = HfAgent("https://api-inference.huggingface.co/models/bigcode/starcoderbase")
# OpenAssistant
# agent = HfAgent(url_endpoint="https://api-
inference.huggingface.co/models/OpenAssistant/oasst-sft-4-pythia-12b-epoch-3.5")
```

此示例使用了目前 Hugging Face 免费提供的推理 API。如果你有自己的推理端点用于此模型（或其他模型），你可以用你的 URL 替换上面的 URL。

`StarCoder` 和 `OpenAssistant` 可以免费使用，并且在简单任务上表现出色。然而，当处理更复杂的提示时就不再有效。如果你遇到这样的问题，我们建议尝试使用 `OpenAI` 模型，尽管遗憾的是它不是开源的，但它在目前情况下表现更好。

我们可以从这个新 API 实现的示例开始。在处理多模态任务时它尤其强大，因此让我们快速试着生成图像并大声朗读文本。

```
# 说明以下图片
agent.run("Caption the following image", image=image)

# 将以下文本大声朗读
agent.run("Read the following text out loud", text=text)

# 外部文档回答
agent.run(
    "In the following `document`, where will the TRRF Scientific Advisory Council Meeting take place?",
    document=document,
)
```

单次执行 (run)

单次执行方法是使用 agent 的 `Agent.run`：

```
agent.run("Draw me a picture of rivers and lakes.")
```

它会自动选择适合你要执行的任务的 tool（或 tools），并以适当的方式运行它们。它可以在同一指令中执行一个或多个任务（当你的指令越复杂时，agent 失败的可能性就越大）。

```
agent.run("Draw me a picture of the sea then transform the picture to add an island")
```

每个 `run()` 操作都是**独立的**，因此你可以通过多次连续运行 run() 执行不同的任务。

如果你想在多次执行之间保持同一状态或向 agent 传递非文本对象，**可以通过指定 agent 要使用的变量来实现**。例如，你可以生成有关河流和湖泊的第一幅图像，并要求模型通过执行以下操作向该图片添加一个岛屿：

```
picture = agent.run("Generate a picture of rivers and lakes.")
updated_picture = agent.run("Transform the image in `picture` to add an island to it.", picture=picture)
```

当模型无法准确地理解你的请求和库中的工具时，例如：

```
agent.run("Draw me the picture of a capybara swimming in the sea")
```

在这种情况下，模型会通过两种方式理解你的请求：

- 使用 `text-to-image` 生成在大海中游泳的大水獭
- 使用 `text-to-image` 生成大水獭，然后使用 `image-transformation` 工具使其在大海中游泳

如果你想强制使用第一种方式来生成图片，可以通过将提示作为参数传递给它来实现，提供更为清晰明确的提示让模型理解你的请求：

```
agent.run("Draw me a picture of the `prompt`", prompt="a capybara swimming in the sea")
```

基于交流的执行 (chat)

这种方法可以在不同指令之间保持同一状态，也可以接受参数，以便你可以传递非文本类型或特定的提示。基于交流的执行（chat）方式是使用 `Agent.chat`：

```
agent.chat("Generate a picture of rivers and lakes")

agent.chat("Transform the picture so that there is a rock in there")
```

Transformers 中集成的 tools

我们确定了一组可以赋予这些 agent 强大能力的 tools。以下是我们在 transformers 中集成的 tools：

- 文档问答：给定一个图像格式的文档（例如 PDF），回答该文档上的问题（Donut）
- 文本问答：给定一段长文本和一个问题，回答文本中的问题（Flan-T5）
- 无条件图像字幕：为图像添加字幕！（BLIP）
- 图像问答：给定一张图像，回答该图像上的问题（VILT）
- 图像分割：给定一张图像和一个提示，输出该提示的分割掩模（CLIPSeg）
- 语音转文本：给定一个人说话的音频录音，将演讲内容转录为文本（Whisper）
- 文本转语音：将文本转换为语音（SpeechT5）
- Zero-Shot 文本分类：给定一个文本和一个标签列表，确定文本最符合哪个标签（BART）
- 文本摘要：总结长文本为一两句话（BART）
- 翻译：将文本翻译为指定语言（NLLB）

这些 tools 也可以手动使用，例如：

```
from transformers import load_tool

tool = load_tool("text-to-speech")
audio = tool("This is a text to speech tool")
```

自定义工具

通过将工具的代码上传到 Hugging Face 空间或模型 repository，你可以直接通过 agent 使用 tools。我们已经添加了一些与 transformers 无关的 tools 到 huggingface-tools 组织中，这一实现能够帮助用户快速创建和共享自定义 tool：

- 文本下载器：从 Web URL 下载文本
- 文本到图像：根据提示生成图像，利用 stable diffusion
- 图像转换：根据初始图像和提示修改图像，利用 instruct pix2pix stable diffusion
- 文本到视频：根据提示生成小视频，利用 damo-vilab

从一开始就一直在使用的文生图 tool 是一个远程 tool，位于 huggingface-tools/text-to-image！我们将继续在此组织和其他组织上发布此类 tool，以进一步增强此实现。

agents 默认可以访问存储在 huggingface-tools 上的 tools。我们将在后续的指南中解释如何编写和共享自定义 tools，以及如何利用 Hub 上存在的任何自定义 tools。

代码生成

到目前为止，我们已经展示了如何使用 agents 来为你执行操作。但是，agents 仅使用 Python 解释器执行的代码。如果你想要在不同的环境中使用生成的代码，可以提示 agents 返回可以执行的程序代码，以及 tools 的定义和准确的导入信息。例如：

```
agent.run("Draw me a picture of rivers and lakes", return_code=True)
```

使用 LLMs 进行生成

大语言模型(LLMs)是文本生成背后的关键组成部分。简单来说，它们是经过大规模预训练的 transformer 模型，用于**根据给定的输入文本预测下一个词（下一个 token）。**由于它们**一次只预测一个 token**，因此除了调用模型之外，还需要执行自回归生成来生成新的句子。

`自回归生成`是在给定一些初始输入，通过迭代调用模型及其自身的生成输出来生成文本的推理过程。在🤗 Transformers 中，这由 generate() 方法处理，所有具有生成能力的模型都可以使用该方法。

本教程将向你展示如何：

- 使用 LLM 生成文本
- 避免常见的陷阱
- 帮助你充分利用 LLM 进行下一步指导

在开始之前，请确保已安装所有必要的库：

```
pip install transformers bitsandbytes>=0.39.0 -q
```

生成文本

一个用于因果语言建模训练的语言模型，将文本的 tokens 序列作为输入，并返回下一个 token 的概率分布。

LLM 的前向传递：

【演示媒体资源：../../resources/show/gif/gif_1_1080p.mov（离线电子书未嵌入视频/音频，详见原仓库）】【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

使用 LLM 进行自回归生成的一个关键方面是**如何从这个概率分布中选择下一个 token。**这个步骤是取决于你的需求的，可以是简单地从概率分布中选择最可能的 token，也可以是复杂地对结果分布应用多种变换后再作选择，只要最终能够得到下一个迭代的 token。

自回归生成迭代地从概率分布中选择下一个 token 以生成文本：

【演示媒体资源：../../resources/show/gif/gif_2_1080p.mov（离线电子书未嵌入视频/音频，详见原仓库）】【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

上述过程是可以迭代重复的，直到达到某个停止条件。在理想情况下，停止条件是由模型决定的，该模型需要学会在何时输出一个结束序列（EOS）标记。如果不是这种情况，生成将在达到某个预定义的最大长度时停止。

正确设置下一个 token 的选择步骤和停止条件对于让你的模型按照预期的方式执行任务至关重要。这就是为什么每个模型都要有一个 `generation.GenerationConfig` 文件，它默认配置了一个效果不错的生成参数，会和你的模型一起加载。

如果你对基本的 LLM 使用感兴趣，我们高级的 `Pipeline` 接口是一个很好的起点。然而，LLMs 通常需要像 `quantization` 和对 `token` 选择步骤的精细控制等高级功能，所以最好通过 `generate()`来完成。使用 LLM 进行自回归生成也是资源密集型的操作，需要在 GPU 上执行才能获得足够的吞吐量。

首先，你需要加载模型。

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.1", device_map="auto", load_in_4bit=True
)
```

在 `from_pretrained` 调用中的有两个参数：

- `device_map="auto"`能够确保模型被移动到你的 GPU(s)上。
- `load_in_4bit` 表示应用 4 位动态量化来极大地减少资源需求。

接下来，你需要使用一个 `tokenizer` 来预处理你的文本输入。

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1", padding_side="left")
model_inputs = tokenizer(["A list of colors: red, blue"], return_tensors="pt").to("cuda")
```

`model_inputs` 变量保存着分词后的文本输入以及注意力掩码。`注意力掩码`能够确保模型只关注有效的输入 tokens，而忽略填充 tokens，从而保证生成的准确性和质量。

尽管 `generate()`在未传递注意力掩码时会尽其所能推断出注意力掩码，但建议尽可能地传递它以获得最佳结果。

在对输入进行分词后，可以调用 `generate()`方法来返回生成的 tokens。生成的 tokens 需要在打印之前转换为我们能够理解的文本。

```
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

在最后，你不要一次处理只一个序列！你可以批量输入，这会在小延迟和低内存成本下显著提高吞吐量，你只需要确保能够正确地填充你的输入（详见下文）。

```
tokenizer.pad_token = tokenizer.eos_token  # 大多数 LLM 默认情况下没有设置填充标记
model_inputs = tokenizer(
    ["A list of colors: red, blue", "Portugal is"], return_tensors="pt", padding=True
).to("cuda")
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)
```

常见陷阱

对于许多不同的生成策略，有时默认值并不适合你的用例。如果模型的输出与你期望的结果不匹配，我们收集了一个最常见的陷阱列表来介绍如何避免它们。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1")
tokenizer.pad_token = tokenizer.eos_token  # 大多数 LLM 默认情况下没有填充标记
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.1", device_map="auto", load_in_4bit=True
)
```

生成的输出太短/太长

如果在 `GenerationConfig` 文件中没有指定返回 tokens 的最大数量，`generate()`默认只返回 20 个 tokens。

建议在你的 `generate` 调用中手动设置 `max_new_tokens` 以控制它可以返回的新 `tokens` 的最大数量。**请注意，对于仅解码器架构的 LLMs(https://huggingface.co/learn/nlp-course/chapter1/6?fw=pt)会将输入提示作为输出的一部分返回。**

```
model_inputs = tokenizer(["A sequence of numbers: 1, 2"], return_tensors="pt").to("cuda")

# 默认情况下，输出将最多包含 20 个标记。
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

```
# 设置`max_new_tokens`来控制输出的新 tokens 的最大长度
generated_ids = model.generate(**model_inputs, max_new_tokens=50)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

错误的生成模式(控制模型选择 Token 的步骤)

默认情况下，除非在`[GenerationConfig](https://huggingface.co/docs/transformers/main/zh/main_classes/text_generation#transformers.GenerationConfig)`文件中指定，否则`generate()`会在每个迭代中选择最可能的 token（贪婪解码）。

例如，像聊天机器人或写作文章这样的创造性任务，贪婪解码并不是最理想的生成模式；像音频转录或翻译这样的基于输入的任务，贪婪解码则是相对理想的生成模式。你可以在这篇博客文章中了解更多关于这个话题的信息。

在深度学习模型中，尤其是在自然语言处理（NLP）任务中，`do_sample` 参数通常用于控制生成文本时的采样策略。具体来说，`do_sample=True` 的作用如下：

- 启用随机采样：

  当 `do_sample=True` 时，模型在生成文本时会采用**随机采样**策略。这意味着模型会根据输出的概率分布随机选择下一个词或标记，而不是始终选择概率最高的词。

  这种方法可以增加生成文本的多样性和创造性，使其看起来更自然，减少重复和单调性。

- 对比贪婪解码：

  当 `do_sample=False` 时（或默认情况下），模型通常会使用**贪婪解码**策略，即始终选择概率最高的词作为下一个词。

  贪婪解码虽然简单且速度快，但可能导致生成的文本过于保守和重复。

- 温度控制：

  在随机采样中，常常会结合一个称为“温度”（`temperature`）的参数来调整概率分布的平滑程度。较高的温度会使概率分布更平缓，增加随机性；较低的温度会使分布更尖锐，减少随机性。

  当 `do_sample=True` 时，可以通过设置温度参数来进一步控制生成的多样性。

```
# set_seed 方法在 transformers 库中用于设置随机数生成器的种子。其作用是确保实验的可重复性(reproducibility)，即在相同的条件下运行代码时，可以得到相同的结果。
```

```
from transformers import set_seed
set_seed(42)

model_inputs = tokenizer(["I am a cat."], return_tensors="pt").to("cuda")

# LLM + 贪婪解码 = 重复、乏味的输出
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]

# 通过设置采样`do_sample=True`，输出变得更有创意！
generated_ids = model.generate(**model_inputs, do_sample=True)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

错误的填充位置

对于仅解码器架构的 LLMs，它们会持续迭代你的输入提示。如果你的输入长度不相同，则需要对它们进行填充。

由于某些 LLMs（如 GPT 系列）在预训练过程中可能没有使用 `pad tokens`，或者没有对 pad tokens 进行特别的处理。**如果模型没有经过 pad tokens 的训练，直接使用右填充可能会导致模型在处理输入时出现错误，因为模型可能会将填充标记误认为是有效信息。**

因此在实际应用中，**输入序列应采用左填充方式。**在生成输出时，**必须传递注意力掩码**，以确保模型正确地忽略填充标记，只关注有效的输入信息。

```
# 上面初始化的分词器在默认情况下启用了右填充：
# 第一个序列较短，在序列右侧进行填充，模型将填充标记误认为是有效信息，导致输出的结果不佳。
model_inputs = tokenizer(
    ["1, 2, 3", "A, B, C, D, E"], padding=True, return_tensors="pt"
).to("cuda")
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]

# 使用左填充，模型生成了预期的结果！
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1", padding_side="left")
tokenizer.pad_token = tokenizer.eos_token  # 大多数 LLM 默认情况下没有填充标记
model_inputs = tokenizer(
    ["1, 2, 3", "A, B, C, D, E"], padding=True, return_tensors="pt"
).to("cuda")
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

错误的提示

某些模型和任务需要输入特定的提示格式才能正常工作。当未使用标准的输入格式时，模型虽能正常工作，但是性能将会下降，输出效果也不如预期。

有关提示的更多信息，包括哪些模型和任务需要注意输入的提示格式，可在指南中找到。让我们看一个使用了聊天模板的 LLM 示例：

```
tokenizer = AutoTokenizer.from_pretrained("HuggingFaceH4/zephyr-7b-alpha")
model = AutoModelForCausalLM.from_pretrained(
    "HuggingFaceH4/zephyr-7b-alpha", device_map="auto", load_in_4bit=True
)
set_seed(0)
prompt = """How many helicopters can a human eat in one sitting? Reply as a thug."""
model_inputs = tokenizer([prompt], return_tensors="pt").to("cuda")
input_length = model_inputs.input_ids.shape[1]
generated_ids = model.generate(**model_inputs, max_new_tokens=20)
print(tokenizer.batch_decode(generated_ids[:, input_length:], skip_special_tokens=True)[0])
# 哦，它没有按照我们的指示以暴徒的风格进行回复!
# 让我们看看当我们写一个更好的提示并为这个模型使用正确的模板时会发生什么（通过
`tokenizer.apply_chat_template`)

set_seed(0)
messages = [
    {
        "role": "system",
        "content": "You are a friendly chatbot who always responds in the style of a thug",
    },
    {"role": "user", "content": "How many helicopters can a human eat in one sitting?"},
]
model_inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True,
return_tensors="pt").to("cuda")
input_length = model_inputs.shape[1]
generated_ids = model.generate(model_inputs, do_sample=True, max_new_tokens=20)
print(tokenizer.batch_decode(generated_ids[:, input_length:], skip_special_tokens=True)[0])
# 正如我们所看到的，它的回复遵循了适当的暴徒风格 😎
```

更多资源

虽然自回归生成过程相对简单，但要充分利用 LLM 的能力可能是一个具有挑战性的任务，因为很多组件之间都有着复杂且密切的关联。以下的资源可以帮助你深入了解和使用 LLM：

高级生成用法

72. 介绍如何控制不同的生成方法、如何设置生成配置文件以及如何进行输出流式传输。

73. 介绍聊天 LLMs 的提示模板。

74. 介绍如何充分利用提示设计。

75. API 参考文档，包括 GenerationConfig 、generate()和与生成相关的类。

LLM 排行榜

76. Open LLM Leaderboard：侧重于比较开源模型的质量。

77. Open LLM-Perf Leaderboard：侧重于比较 LLM 的吞吐量。

延迟、吞吐量和内存利用率

78. 如何优化 LLMs 以提高速度和内存利用。

79. 关于 quantization，如 bitsandbytes 和 autogptq 的指南，教你如何大幅降低内存需求。

相关库

80. text-generation-inference：一个面向生产的 LLM 服务器。

81. optimum：一个🤗 Transformers 的扩展，优化特定硬件设备的性能。

Agents and Tools 介绍和指南

什么是 Agent (代理)？

大型语言模型（LLMs）可以处理各种各样的任务，但它们通常难以处理逻辑、计算和搜索等基本任务。当在提示他们在表现不佳的领域进行生成时，往往无法直接生成我们期望的答案。

克服这个缺点的方法之一是创建一个 `Agent`。`Agent` 是一个使用 LLM 作为引擎的系统，它可以访问称为 `Tools` 的功能。这些 `Tools` 是用于执行任务的功能，同时也包含使 Agent 能够正确使用它们的描述信息。

Agents 的程序可编程为：

- 设计一系列动作(Actions)和工具(Tools)并一次性运行它们，例如 `CodeAgent`。
- 逐个计划并执行操作或工具，等待每个操作的结果，根据结果和计划再启动下一个操作，例如 `ReactJsonAgent`。

Agent 的类型

CodeAgent

该 Agent 有一个确定的规划和执行步骤，会生成 Python 代码来一次性执行所有的操作。它能够为其工具处理不同的输入和输出类型，因此它是多模态任务的推荐选择。

ReactJsonAgent

这是解决推理型任务的首选 Agent，因为`ReAct 框架`（Yao 等人，2022 年）使 Agent 在执行后会观察之前的结果再决定下一步，使得每一步都建立在有效思考的基础上。

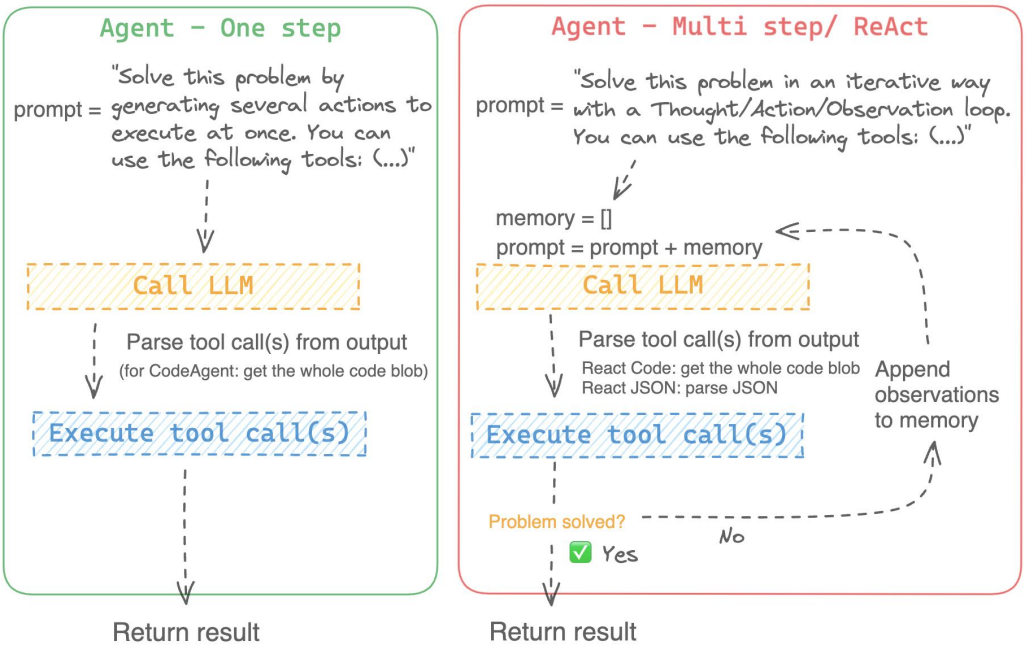
我们实现了两个版本的 `ReactJsonAgent`：

- `ReactJsonAgent` 在其输出中将工具调用生成为 JSON。
- `ReactCodeAgent` 是一种新型的 ReactJsonAgent，它能够通过工具调用生成代码的二进制表示形式(Blob)，这对于具有强大编码能力的 LLMs 非常有效。

阅读将开源 LLMs 作为 LangChain Agents 博客文章，了解更多关于 ReAct Agent 的信息。



Agent_ManimCE



ReAct

下面展示的是 ReactCodeAgent 如何解决问题的过程和输出内容：

```
agent.run(
    "How many more blocks (also denoted as layers) in BERT base encoder than the encoder from the architecture proposed in Attention is All You Need?",
)
```

如何构建 Agent?

要初始化 Agent，你需要以下参数：

- LLM（大型语言模型）：作为 Agent 的核心动力源，LLM 负责生成 Agent 所需的输出。

  Agent 本身并非纯粹的 LLM，而是依托 LLM 进行运作的程序。
- 提示符：这是提供给 LLM 引擎的指令，用以指导其生成特定的输出。
- 工具箱：包含 Agent 可调用的各种工具， Agent 将从中选择合适的工具来执行任务。
- 解析器：用于从 LLM 的输出中提取信息，确定需要调用哪些工具以及相应的参数。

在初始化过程中，工具的属性会被用来生成工具描述，这些描述随后会被嵌入到 Agent 的提示符中，以确保 Agent 清楚了解可用的工具及其适用场景。

首先，需要安装 Agent 附加程序，它将自动安装所有的默认依赖项，为后续的 Agent 系统配置奠定基础。

```
pip install transformers[agents]
```

通过定义一个接受消息列表并返回文本的 llm_engine 方法来构建 LLM 引擎。此调用还需要接受一个停止参数，该参数提示模型何时停止生成。

```
from huggingface_hub import login, InferenceClient

login("<YOUR_HUGGINGFACEHUB_API_TOKEN>")

client = InferenceClient(model="meta-llama/Meta-Llama-3-70B-Instruct")

def llm_engine(messages, stop_sequences=["Task"]) -> str:
    response = client.chat_completion(messages, stop=stop_sequences, max_tokens=1000)
    answer = response.choices[0].message.content
    return answer
```

你可以使用 `llm_engine` 方法，但需要遵守以下几点：

- 输入格式：输入消息必须是列表，里面包含字典，字典的键和值都是字符串(`List[Dict[str, str]]`)。
- 输出格式：返回的结果是一个字符串(`str`)。
- 停止生成：当遇到指定的停止序列(`stop_sequences`)时，输出会停止。

此外，`llm_engine` 还支持一个叫“语法”的参数。如果你在创建 Agent 时设置了语法，这个参数会一起传给 llm_engine，确保生成的输出格式正确。

你还需要提供一个叫 `tools` 的参数，它接受一个工具列表，这个列表可以是空的。如果你想要添加一些默认工具，可以设置 `add_base_tools=True`。

现在，你可以创建一个 Agent 并运行它（比如 `CodeAgent`）。为了方便使用，我们还提供了两种引擎：

- TransformersEngine：可以创建一个具有预初始 pipeline 的 `TransformersEngine`，在你的电脑上使用 transformers 库进行推理。
- HfApiEngine：`HfApiEngine` 是一个专门设计的工具或类，它封装了 `huggingface_hub.InferenceClient`，简化了与 Hugging Face API 的交互过程。由于 Agent 行为通常需要更强大的模型，目前很难在本地运行，通过 Hugging Face 的在线服务，可以直

接使用强大的预训练模型，如 Llama-3.1-70B-Instruct，这些模型在本地运行可能需要很高的计算资源。

```
from transformers import CodeAgent, HfApiEngine

llm_engine = HfApiEngine(model="meta-llama/Meta-Llama-3-70B-Instruct")
agent = CodeAgent(tools=[], llm_engine=llm_engine, add_base_tools=True)

agent.run(
    "Could you translate this sentence from French, say it out loud and return the audio.",
    sentence="Où est la boulangerie la plus proche?",
)
```

你可以不定义参数 `llm_engine`，默认情况下将创建 `HfApiEngine`。

```
from transformers import CodeAgent

agent = CodeAgent(tools=[], add_base_tools=True)

agent.run(
    "Could you translate this sentence from French, say it out loud and give me the audio.",
    sentence="Où est la boulangerie la plus proche?",
)
```

请注意，我们这里用了一个特别的参数叫 `sentence`。这个参数的作用是让你可以把需要处理的文本直接传给模型。

不仅如此，这个 `sentence` 参数还可以用来告诉模型去使用某个本地或远程文件的路径。这样一来，你可以更灵活地处理各种文件，非常方便！

- 文本处理：直接传文本给模型。
- 文件处理：传文件路径给模型。

```
from transformers import ReactCodeAgent

agent = ReactCodeAgent(tools=[], llm_engine=llm_engine, add_base_tools=True)

agent.run("Why does Mike not know many people in New York?",
audio="../../resources/record/transformers_recording.mp3")
```

提示符和输出解析器是自动定义的，但是你可以通过输出 Agent 中的 `system_prompt_template` 来查看它们。

```
print(agent.system_prompt_template)
```

清楚地描述你要执行的任务非常重要。每次调用 `run()` 方法时，它都是独立的操作。因为 Agent 是由大型语言模型（LLM）支持的，所以提示中的微小变化可能会导致结果大不相同。

此外，你还可以连续运行 Agent 来处理不同的任务。每次运行时， Agent 的 `agent.task` 和 `agent.logs` 属性都会重新初始化，确保每个任务都是从干净的状态开始。

代码执行

Python 解释器会在你提供的工具和输入上执行代码。这样做是安全的，因为解释器只能调用你提供的工具（比如 Hugging Face 的工具）和打印函数。这意味着你能控制它能做什么，避免了潜在的风险。

另外，Python 解释器默认不允许导入不在安全列表中的模块，所以大部分常见的攻击手段都不会构成威胁。

不过，如果你需要在 `ReactCodeAgent` 或 `CodeAgent` 中使用额外的模块，你可以在初始化时，通过 `additional_authorized_imports` 参数来授权。这个参数接受一个字符串列表，列出你允许导入的额外模块。

```
from transformers import ReactCodeAgent

agent = ReactCodeAgent(tools=[], additional_authorized_imports=['requests', 'bs4'])
agent.run("Could you get me the title of the page at url 'https://huggingface.co/blog'?")
```

执行将在任何试图执行非法操作的代码处停止，或者 Agent 生成的代码存在常规的 Python 错误。

注意！LLM 可以生成任意代码，然后执行，所以请不要添加任何不安全的导入！

系统提示

Agent ，或者说驱动 Agent 的 LLM，基于系统提示生成输出。系统提示可以根据预期的任务进行自定义和定制。例如，查看 ReactCodeAgent 的系统提示（以下版本略有简化）。

系统提示包含以下几个部分：

82. 介绍：说明 Agent 应该怎么工作和使用工具。
83. 工具描述：所有工具的描述由 <<Tool_Exclusions>> 标记定义。在运行时，这个标记会被用户定义或选择的工具动态替换。
    - 工具描述信息包括工具的属性、名称、描述、输入和输出类型。
    - 这些描述使用一个简单的 `Jinja 2` 模板生成，你可以根据需要优化这个模板。

84. 预期输出格式：说明期望的输出格式是什么样的。

你可以通过以下方式改进系统提示：

85. 添加更多关于输出格式的说明，让 Agent 更清楚应该怎么生成结果。

86. 为了获得最大的灵活性，你可以自定义整个系统提示模板。只需将你的自定义提示作为参数传递给 `system_prompt` 参数，就可以覆盖默认的提示模板。

```
from transformers import ReactJsonAgent
from transformers.agents import PythonInterpreterTool

agent = ReactJsonAgent(tools=[PythonInterpreterTool()], system_prompt="{your_custom_prompt}")
```

请确保在模板的某个地方定义一个叫 <<tool_dations>> 的字符串。这样做的目的是让 Agent 知道有哪些工具可以使用。

检查 Agent 运行

运行 Agent 后，你可以通过以下属性来检查发生了什么：

87. `logs` 属性：
    - 这个属性存储了 Agent 的详细日志。每次 Agent 执行一步操作时，所有相关信息都会被记录在一个字典中，并添加到 `agent.logs` 里。

88. `agent.write_inner_memory_from_logs()` 方法：
    - 这个方法会将 Agent 的日志转换成内部的内存格式，供大型语言模型（LLM）查看，就像一系列聊天消息一样。
    - 它会遍历日志的每一步，只挑选出感兴趣的内容保存为消息。比如，系统提示和任务会被单独保存，每一步的 LLM 输出和工具调用输出也会分别保存为不同的消息。
    - 如果你希望从更高层次了解发生了什么，可以使用这个方法。但请注意，并不是每个日志细节都会被转换成消息。

简单来说：

- 查看详细日志：通过 `agent.logs` 属性。
- 转换日志为内存格式：用 `agent.write_inner_memory_from_logs()` 方法，方便 LLM 查看，但不是所有细节都会被转换。

工具

工具是 Agent 用来执行任务的基本功能模块。

举个例子，查看 `PythonInterpreterTool`：

- 它有名称、描述、输入描述、输出类型，还有一个 __call__ 方法来实际执行操作。

当你初始化 Agent 时，这些工具的属性会被用来生成工具描述。这个描述会被嵌入到 Agent 的系统提示中。这样， Agent 就能知道它有哪些工具可以用，以及这些工具是干什么的。

默认工具箱

Transformers 库自带一个默认的工具箱，可以帮助 Agent 完成各种任务。你可以在初始化 Agent 时，通过设置 `add_base_tools=True` 来添加这个工具箱。

这个工具箱包含以下功能：

- 文档问题回答：给定一个 PDF 等格式的文档，可以回答关于这个文档的问题（比如关于“甜甜圈”的问题）。
- 图像问题回答：给定一张图片，可以回答关于这张图片的问题（使用 VILT 模型）。
- 语音到文本：给定一段语音录音，可以将其转换成文字（使用耳语模型）。
- 文本转语音：将文字转换成语音（使用 SpeechT5 模型）。
- 翻译：将句子从一种语言翻译成另一种语言。
- DuckDuckGo 搜索：使用 DuckDuckGo 浏览器进行网络搜索。
- Python 代码解释器：在安全环境中运行 LLM 生成的 Python 代码。注意，如果你使用 `add_base_tools=True` 初始化，这个工具只会添加到 `ReactJsonAgent`，因为基于代码的 Agent 已经自带 Python 代码执行功能。

你也可以手动使用这些工具，只需调用 `load_tool()` 函数并指定要执行的任务即可。

```
from transformers import load_tool

tool = load_tool("text-to-speech")
audio = tool("This is a text to speech tool")
```

创建新的工具

如果你发现 Hugging Face 提供的默认工具不能满足你的需求，也可以自己创建工具。

比如，我们可以创建一个工具，用来查找并返回在 Hugging Face Hub 上下载次数最多的模型，针对某个特定的任务。你可以从下面这段代码开始动手实现。

```
from huggingface_hub import list_models

task = "text-classification"

model = next(iter(list_models(filter=task, sort="downloads", direction=-1)))
print(model.id)
```

下面这段代码可以快速转换为工具，只需将其包装在函数中并添加工具装饰器：

```
from transformers import tool

@tool
def model_download_counter(task: str) -> str:
    """
    这是一个工具，它会返回 Hugging Face Hub 上给定任务中下载量最高的模型的 Checkpoint 名称。

    Args:
        task: 任务
    """
    model = next(iter(list_models(filter="text-classification", sort="downloads", direction=-
1)))
    return model.id
```

这个功能需要以下几个要素：

- 明确的名字：这个名字要能清楚地描述工具的功能。由于这个工具是用来返回某个任务下载量最多的模型，我们可以给它起名叫 model_download_counter。
- 输入和输出的类型提示：需要明确指出输入和输出数据的类型。
- 详细的描述：包括一个 `Args` 部分，用来描述每个参数的具体作用。所有这些信息在初始化 Agent 时，会自动嵌入到系统提示中。

此定义的格式与`apply_chat_template`中使用的工具模式相同，唯一的区别是添加了工具装饰器：[在这里](https://huggingface.co/blog/unified-tool-use#passing-tools-to-a-chat-template)阅读有关我们的工具使用 API 的更多信息。

初始化 Agent 并运行它：

```
from transformers import CodeAgent
agent = CodeAgent(tools=[model_download_tool], llm_engine=llm_engine)
agent.run(
    "Can you give me the name of the model that has the most downloads in the 'text-to-video' task on the Hugging Face Hub?"
)
```

管理 Agent 的工具箱

如果你已经创建了一个 Agent ，再重新初始化它来添加新工具会比较麻烦。不过，使用 Transformers 库，你可以轻松地添加或替换 Agent 的工具箱中的工具。

比如，我们可以把 `model_download_tool` 添加到一个已经用默认工具箱初始化好的 Agent 中。

```
from transformers import CodeAgent

agent = CodeAgent(tools=[], llm_engine=llm_engine, add_base_tools=True)
agent.toolbox.add_tool(model_download_tool)
```

现在我们可以利用新的工具和之前的文本到语音的工具：

```
agent.run(
    "Can you read out loud the name of the model that has the most downloads in the 'text-to-video' task on the Hugging Face Hub and return the audio?"
)
```

注意！需要将工具添加到运行良好的 Agent 时要小心，因为它可能会使选择偏向于你的工具或选择其他工具，而不是此前已经定义的工具。

你可以用 `agent.toolbox.update_tool()` 方法来替换 Agent 工具箱中的现有工具。这个方法特别有用，因为 Agent 已经知道怎么完成那个任务，若你的新工具是现有工具的直接替代品，那么只要确保你的新工具和被替换的工具使用相同的接口；或者更新系统提示模板，确保所有包含被替换工具的提示都更新了

使用一系列工具

你可以通过 `ToolCollection` 对象并指定工具集合的参数 `collection_slug` 来使用工具集合。只需把这些工具集合转换成一个列表，然后在初始化 Agent 时传递进去，就可以开始使用它们了！

为了加快启动速度，所以只有在 Agent 调用时才会加载工具。

```
from transformers import ToolCollection, ReactCodeAgent

image_tool_collection = ToolCollection(collection_slug="huggingface-tools/diffusion-tools-6630bb19a942c2306a2cdb6f")
```

```
agent = ReactCodeAgent(tools=[*image_tool_collection.tools], add_base_tools=True)

agent.run("Please draw me a picture of rivers and lakes.")
```

第三部分 任务指南

自然语言处理

文本分类

文本分类 是一种常见的自然语言处理（NLP）任务，它的作用是为文本分配标签或类别。很多大公司都在实际应用中使用文本分类，比如判断用户评论是好评还是差评。其中，情感分析 是最流行的文本分类形式之一，它会给文本打上“积极”、“消极”或“中立”的标签。

这本指南会教你怎么做两件事：

89. 微调 DistilBERT：在 IMDb 数据集上训练模型，让它能判断电影评论是好评还是差评。

90. 使用微调模型进行推理：用训练好的模型来分析新的评论。

要查看与此任务兼容的所有架构和 checkpoints，可查阅。

在开始之前，请确保安装了所有必要的库：

```
pip install transformers datasets evaluate accelerate
```

在这一步，需要登录你的 Hugging Face 帐户，出现相关的登陆提示时，输入你的令牌以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 IMDB 数据集

首先，从 IMDB 数据库中加载 IMDB 数据集：

```
from datasets import load_dataset

imdb = load_dataset("imdb")
```

再看一个例子：

```
imdb["test"][0]
```

此数据集中有两个字段：

- `text`：电影评论的文本。
- `label`：一个整数值，0 表示负面评价，1 表示正面评价。

预处理数据

下一步是加载一个 `DistilBERT Tokenizer` 来预处理 `text` 字段：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

创建一个预处理函数来标记文本并截断序列，使其不超过 DistilBERT 的最大输入长度：

```
def preprocess_function(examples):
    return tokenizer(examples["text"], truncation=True)
```

若要对整个数据集应用预处理函数，请使用函数 Datasets.map()。你可以通过设置 `batched=True` 来一次性处理数据集的多个元素来加速 `map`：

```
tokenized_imdb = imdb.map(preprocess_function, batched=True)
```

现在使用 DataCollatorWithPadding 创建一批示例。

在排序过程中，会动态地将句子填充到与这一批数据中最长序列相同的长度，而不是将整个数据集都填充到最大长度，这样会更有效率。

```
# PyTorch
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
# TensorFlow
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer, return_tensors="tf")
```

评估模型

在训练模型时，选择一个合适的指标来评估模型的表现非常重要。

你可以使用“Evaluate”这个工具库，轻松地加载和使用各种评估方法。对于这个任务，我们需要加载一个叫做“accuracy 准确度”的度量指标。

你可以参考 Evaluate 快速指南，那里有详细的步骤教你如何加载和计算这些指标。

```
import evaluate

accuracy = evaluate.load("accuracy")
```

然后，创建一个函数，将你的预测和标签传递给 computing 来计算准确度：

```
import numpy as np


def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return accuracy.compute(predictions=predictions, references=labels)
```

`compute_metrics` 这个函数现在已经准备好了。当你开始设置训练过程时，你会再次用到它。

训练模型

在开始训练模型之前，我们需要用 `id2label` 和 `label2id` 来建立一个映射关系，这样就能把模型的输出编号转换成我们看得懂的标签了。

```
id2label = {0: "NEGATIVE", 1: "POSITIVE"}
label2id = {"NEGATIVE": 0, "POSITIVE": 1}
```

PyTorch

如果你不熟悉如何使用 Trainer 对模型进行微调，请查看此处的基本教程！

现在你可以开始训练你的模型啦！使用 AutoModelForSequenceClassification 来加载 DistilBERT 模型，同时指定你需要的标签数量和标签映射关系：

```
from transformers import AutoModelForSequenceClassification, TrainingArguments, Trainer

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert/distilbert-base-uncased", num_labels=2, id2label=id2label, label2id=label2id
)
```

此时，只剩下三个步骤：

91. 设置训练参数：在 `TrainingArguments` 中定义训练的超参数。这里最重要的是 `output_dir` 参数，它用来指定保存模型的位置。如果你想把模型上传到 Hugging Face，需要设置 `push_to_hub=True`（记得先登录 Hugging Face 账号）。在每个训练周期（epoch）结束时，`Trainer` 会自动评估模型的准确性，并保存训练的检查点。

92. 配置`Trainer`：把训练参数、模型、数据集、分词器（tokenizer）、数据整理器（data collator）以及`compute_metrics`函数一起传递给 Trainer。

93. 开始训练：调用`train()`方法来微调你的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_imdb["train"],
    eval_dataset=tokenized_imdb["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

当你传递`tokenizer`给`Trainer`的时候，它会默认使用动态填充功能。在这种情况下，你不需要手动地指定一个数据整理器。

训练完成后，可以使用 push_to_hub() 方法将你的模型共享到 Hub，以便每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果你不熟悉使用 Keras 微调模型，请查看此处的基本教程！

要在 TensorFlow 中微调模型，首先要设置优化器函数、学习率计划和一些训练超参数：

```
from transformers import create_optimizer
import tensorflow as tf

batch_size = 16
```

```
num_epochs = 5
batches_per_epoch = len(tokenized_imdb["train"]) // batch_size
total_train_steps = int(batches_per_epoch * num_epochs)
optimizer, schedule = create_optimizer(init_lr=2e-5, num_warmup_steps=0,
num_train_steps=total_train_steps)
```

接下来，你可以使用 `TFAutoModelForSequenceClassification` 来加载 `DistilBERT` 模型，同时指定预期的标签数量和标签映射关系：

```
from transformers import TFAutoModelForSequenceClassification

model = TFAutoModelForSequenceClassification.from_pretrained(
    "distilbert/distilbert-base-uncased", num_labels=2, id2label=id2label, label2id=label2id
)
```

使用 prepare_tf_dataset()将数据集转换为 `tf.data.Dataset` 格式：

```
tf_train_set = model.prepare_tf_dataset(
    tokenized_imdb["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)

tf_validation_set = model.prepare_tf_dataset(
    tokenized_imdb["test"],
    shuffle=False,
    batch_size=16,
    collate_fn=data_collator,
)
```

使用 compile 方法来配置模型进行训练。需要注意的是，Transformers 库中的模型已经内置了与任务相关的默认损失函数，所以通常情况下你不需要手动指定损失函数，除非你有特别的需要：

```
import tensorflow as tf

model.compile(optimizer=optimizer)  # 没有指定损失函数
```

在开始训练之前，还需要完成最后两个步骤：

94. 计算预测的准确性：我们需要评估模型预测的准确性。

95. 提供模型上传到 Hub 的方法：这样可以将训练好的模型分享到 Hugging Face 的 Hub。

这两个步骤都可以通过使用 Keras 回调来实现。

具体来说，你可以将 `compute_metrics` 函数传递给 KerasMetricCallback：

```
from transformers.keras_callbacks import KerasMetricCallback

metric_callback = KerasMetricCallback(metric_fn=compute_metrics, eval_dataset=tf_validation_set)
```

在 PushToHubCallback 中指定将模型和 tokenizer 推送到何处：

```
from transformers.keras_callbacks import PushToHubCallback

push_to_hub_callback = PushToHubCallback(
    output_dir="my_awesome_model",
    tokenizer=tokenizer,
)
```

接下来，把你的回调函数组合在一起：

```
callbacks = [metric_callback, push_to_hub_callback]
```

最后，可以开始训练你的模型啦！调用 fit 方法，传入你的训练和验证数据集、训练周期（epoch）的数量以及你设置的回调函数，这样就能微调你的模型了：

```
model.fit(x=tf_train_set, validation_data=tf_validation_set, epochs=3, callbacks=callbacks)
```

训练完成后，你的模型将自动上传到 Hub，以便每个人都可以使用它！

模型推理

当你已经微调了一个模型，你就可以用它来推理了！

抓取一些你想运行推理的文本：

```
text = "This was a masterpiece. Not completely faithful to the books, but enthralling from beginning to end. Might be my favorite of the three."
```

要尝试使用微调后的模型进行推理，最简单的方法是通过 pipeline 来操作。你可以用你的模型创建一个专门用于情感分析的 pipeline，然后把你的文本输入进去：

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis", model="stevhliu/my_awesome_model")
classifier(text)
```

如果有需要，你还可以手动复制 pipeline 的结果：

PyTorch

96. 标记文本并返回 PyTorch 张量：对输入文本进行分词处理，并将其转换为 PyTorch 张量格式。

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("stevhliu/my_awesome_model")
inputs = tokenizer(text, return_tensors="pt")
```

97. 将输入传递给模型并返回 logits：把处理好的张量输入到模型中，获取模型的输出 `logits`。

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("stevhliu/my_awesome_model")
with torch.no_grad():
    logits = model(**inputs).logits
```

98. 获取概率最高的类别，并使用模型的 `id2label` 映射将其转换为文本标签：从 `logits` 中找出概率最高的类别，然后利用模型的 `id2label` 映射关系，将其转换成对应的文本标签。

```
predicted_class_id = logits.argmax().item()
model.config.id2label[predicted_class_id]
```

TensorFlow

99. 标记文本并返回 PyTorch 张量：对输入文本进行分词处理，并将其转换为 PyTorch 张量格式。

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("stevhliu/my_awesome_model")
inputs = tokenizer(text, return_tensors="tf")
```

100. 将输入传递给模型并返回 logits：把处理好的张量输入到模型中，获取模型的输出 `logits`。

```
from transformers import TFAutoModelForSequenceClassification

model = TFAutoModelForSequenceClassification.from_pretrained("stevhliu/my_awesome_model")
logits = model(**inputs).logits
```

101. 获取概率最高的类别，并使用模型的 `id2label` 映射将其转换为文本标签：从 `logits` 中找出概率最高的类别，然后利用模型的 `id2label` 映射关系，将其转换成对应的文本标签。

```
predicted_class_id = int(tf.math.argmax(logits, axis=-1)[0])
model.config.id2label[predicted_class_id]
```

标记分类（实体分类）

标记分类（Token Classification）是指为句子中的每个标记（token）分配一个标签。最常见的标记分类任务之一是命名实体识别（Named Entity Recognition, NER）。NER 的目标是为句子中的每个实体（例如人名、地名或组织名）找到相应的标签。

本指南将向你展示如何：

102. **微调 DistilBERT 模型**：使用 WNUT 17 数据集对 DistilBERT 模型进行微调，以便检测新的实体。

103. **使用微调模型进行推理**：利用微调后的模型进行实际应用中的推理操作。

要查看与此任务兼容的所有架构和检查点，我们建议检查任务页面。

在开始之前，请确保安装了所有必要的库：

```
pip install transformers datasets evaluate seqeval
```

登录你的 Hugging Face 帐户，以便你可以上传并与社区分享你的模型。出现提示时，输入你的令牌来登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 WNUT 17 数据集

首先从 WNUTDatasets 库中加载 WNUT 17 数据集：

```
from datasets import load_dataset

wnut = load_dataset("wnut_17")
```

我们来看一个示例数据：

```
wnut["train"][0]
```

`ner_tags` 中的每个数字代表一个实体。为了找出这些数字对应的实体标签，我们可以将数字转换为它们的标签名称：

```
label_list = wnut["train"].features[f"ner_tags"].feature.names
label_list
```

每个 `ner_tag` 前面的字母表示实体的标记位置：

- `B-` 表示实体的开始。
- `I-` 表示令牌包含在同一实体中（例如，`State` 令牌是 `Empire State Building` 实体的一部分）。
- `0` 表示令牌不对应于任何实体。

预处理数据

下一步是加载 DistilBERT tokenizer 来预处理 `tokens` 字段：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

尽管示例中的 `tokens` 字段看起来像是已经经过标记化的，但实际上并非如此。

为了正确处理这些单词，你需要使用 `is_split_into_words=True` 参数来告诉 tokenizer 将单词拆分为子单词。例如：

```
example = wnut["train"][0]
tokenized_input = tokenizer(example["tokens"], is_split_into_words=True)
tokens = tokenizer.convert_ids_to_tokens(tokenized_input["input_ids"])
tokens
```

当你使用 tokenizer 处理文本时，会引入特殊的标记`[CLS]`和`[SEP]`，并且单词可能会被拆分为多个子词标记。这会导致标记和标签之间的不匹配，因为一个标签可能需要对应多个子词标记。为了解决这个问题，你需要重新对齐标记和标签：

104. 使用 word_ids 方法将所有标记映射到它们对应的单词。
105. 将标签`-100`添加到特殊标记`[CLS]`和`[SEP]`，以便 PyTorch 损失函数忽略它们（参见 `CrossEntropyLoss`）。
106. 仅标记给定单词的第一个标记。将`-100`分配给同一单词的其他子标记。

通过这些步骤，你可以确保每个子词标记都有一个正确的标签，同时特殊标记不会影响模型的训练。

```
def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(examples["tokens"], truncation=True, is_split_into_words=True)

    labels = []
    for i, label in enumerate(examples[f"ner_tags"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)  # Map tokens to their respective
word.
        previous_word_idx = None
        label_ids = []
        for word_idx in word_ids:  # Set the special tokens to -100.
            if word_idx is None:
                label_ids.append(-100)
            elif word_idx != previous_word_idx:  # Only label the first token of a given word.
                label_ids.append(label[word_idx])
            else:
                label_ids.append(-100)
            previous_word_idx = word_idx
        labels.append(label_ids)
```

```
    tokenized_inputs["labels"] = labels
    return tokenized_inputs
```

为了在整个数据集上应用预处理函数，你需要使用 `datasets` 库中的 map 函数。这个函数允许你对数据集中的每个元素应用一个函数。为了提高效率，你可以通过设置 `batched=True` 来让 `map` 函数一次处理多个元素：

```
tokenized_wnut = wnut.map(tokenize_and_align_labels, batched=True)
```

接下来，使用 DataCollatorWithPadding 来创建一批示例。这个数据整理器会在处理批次时动态地将句子填充到该批次中最长句子的长度。这种方法比将整个数据集填充到最大长度更有效率，因为它减少了不必要的填充，从而节省了计算资源：

```
# PyTorch
from transformers import DataCollatorForTokenClassification

data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)

# TensorFlow
from transformers import DataCollatorForTokenClassification

data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer, return_tensors="tf")
```

评估模型

在训练模型时，引入指标是非常有帮助的，因为它能让我们更好地了解模型的性能。你可以使用 Evaluate 库来快速加载和使用评估方法。

具体步骤如下：

107. **加载评估方法**：
    - 对于这个任务，我们推荐使用 `seqeval` 框架。
    - 你可以在 Evaluate 快速浏览部分找到如何加载和计算这些指标的详细说明。
108. **Seqeval 提供的指标**：
    - **精确度**：衡量模型预测正确的比例。
    - **召回率**：衡量模型正确识别的比例。
    - **F1 分数**：精确度和召回率的调和平均值，综合评估模型性能。
    - **准确度**：衡量模型整体预测正确的比例。

通过这些指标，你可以全面了解模型的性能，从而进行更有针对性的优化。

```
import evaluate

seqeval = evaluate.load("seqeval")
```

首先，我们需要获取命名实体识别（NER）的标签。接下来，我们可以创建一个函数，将模型的预测结果和真实的标签传递给 computing，以评估模型的性能。

```
import numpy as np

labels = [label_list[i] for i in example[f"ner_tags"]]


def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    true_predictions = [
        [label_list[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    true_labels = [
        [label_list[l] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]

    results = seqeval.compute(predictions=true_predictions, references=true_labels)
    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"],
    }
```

现在，`compute_metrics` 函数已经准备好了。当你在设置训练过程时，可以随时调用这个函数来评估模型的性能。

训练模型

在开始训练模型之前，使用 `id 2 label` 和 `label 2 id` 创建一个期望 id 到标签的映射：

```
id2label = {
    0: "O",
    1: "B-corporation",
    2: "I-corporation",
    3: "B-creative-work",
    4: "I-creative-work",
    5: "B-group",
    6: "I-group",
    7: "B-location",
```

```
    8: "I-location",
    9: "B-person",
    10: "I-person",
    11: "B-product",
    12: "I-product",
}
label2id = {
    "O": 0,
    "B-corporation": 1,
    "I-corporation": 2,
    "B-creative-work": 3,
    "I-creative-work": 4,
    "B-group": 5,
    "I-group": 6,
    "B-location": 7,
    "I-location": 8,
    "B-person": 9,
    "I-person": 10,
    "B-product": 11,
    "I-product": 12,
}
```

PyTorch

如果你不熟悉使用 `Trainer` 对模型进行微调，请查看此处的基本教程！

现在，你可以开始训练你的模型了！我们将使用 AutoModelForTokenClassification 来加载 DistilBERT，并设置预期标签的数量和标签映射。

```
from transformers import AutoModelForTokenClassification, TrainingArguments, Trainer

model = AutoModelForTokenClassification.from_pretrained(
    "distilbert/distilbert-base-uncased", num_labels=13, id2label=id2label, label2id=label2id
)
```

此时，只剩下三个步骤：

109. **定义训练超参数**：

- 使用 `[TrainingArguments](https://huggingface.co/docs/transformers/main/en/main_classes/trainer#transformers.TrainingArguments)` 来定义训练的超参数。
- **关键参数**：
  - `output_dir`：指定保存模型的位置。

- `push_to_hub=True`：将模型推送到 Hugging Face Hub（需要先登录 Hugging Face 账号）。
- 在每个 epoch 结束时，Trainer 会自动评估 `seqeval` 分数并保存训练检查点。

110. **初始化 Trainer**：
    - 将训练参数、模型、数据集、标记器、数据整理器以及 `compute_metrics` 函数一起传递给 Trainer。

111. **开始训练**：
    - 调用 train() 方法来微调你的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_wnut_model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_wnut["train"],
    eval_dataset=tokenized_wnut["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

完成训练后，你可以通过使用 push_to_hub()方法，把你的模型上传到 Hub。这样一来，任何人都能轻松地访问和使用你的模型。

```
trainer.push_to_hub()
```

TensorFlow

如果你不熟悉使用 `Keras` 微调模型，请查看此处的基本教程！

要在 TensorFlow 中微调模型，首先要设置优化器函数、学习率计划和一些训练超参数：

```
from transformers import create_optimizer

batch_size = 16
num_train_epochs = 3
num_train_steps = (len(tokenized_wnut["train"]) // batch_size) * num_train_epochs
optimizer, lr_schedule = create_optimizer(
    init_lr=2e-5,
    num_train_steps=num_train_steps,
    weight_decay_rate=0.01,
    num_warmup_steps=0,
)
```

接下来，你可以利用 TFAutoModelForTokenClassification 来加载 DistilBERT，同时指定预期标签的数量和标签映射：

```
from transformers import TFAutoModelForTokenClassification

model = TFAutoModelForTokenClassification.from_pretrained(
    "distilbert/distilbert-base-uncased", num_labels=13, id2label=id2label, label2id=label2id
)
```

使用 prepare_tf_dataset()将数据集转换为 tf.data.Dataset 格式：

```
tf_train_set = model.prepare_tf_dataset(
    tokenized_wnut["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)

tf_validation_set = model.prepare_tf_dataset(
    tokenized_wnut["validation"],
    shuffle=False,
    batch_size=16,
    collate_fn=data_collator,
)
```

通过 compile 方法来配置模型，准备进行训练。需要注意的是，Transformers 模型已经内置了与任务相关的默认损失函数，所以通常情况下你不需要手动指定损失函数，除非你有特别的需要：

```
import tensorflow as tf

model.compile(optimizer=optimizer)  # No loss argument!
```

在开始训练之前要做的最后两件事是从预测中计算 seqeval 分数，并提供一种将模型推送到 Hub 的方法。两者都是通过使用 Keras 回调来完成的。

将 `compute_metrics` 函数传递给 KerasMetricCallback：

```
from transformers.keras_callbacks import KerasMetricCallback

metric_callback = KerasMetricCallback(metric_fn=compute_metrics, eval_dataset=tf_validation_set)
```

在 PushToHubCallback 中，指定你想要将模型和标记器上传到的位置：

```
from transformers.keras_callbacks import PushToHubCallback

push_to_hub_callback = PushToHubCallback(
    output_dir="my_awesome_wnut_model",
    tokenizer=tokenizer,
)
```

接下来，把你的回调函数组合在一起：

```
callbacks = [metric_callback, push_to_hub_callback]
```

最后一步，你已经准备好开始训练模型了！使用 fit 方法，传入你的训练和验证数据集、训练轮次（epoch）的数量以及之前设置的回调函数，来对模型进行微调：

```
model.fit(x=tf_train_set, validation_data=tf_validation_set, epochs=3, callbacks=callbacks)
```

训练完成后，你的模型会自动上传到 Hub，这样大家都能方便地使用它了！

模型推理

现在你已经成功微调了一个模型，接下来可以用它来进行推理了！

首先，准备一些你想要进行推理的文本：

```
text = "The Golden State Warriors are an American professional basketball team based in San Francisco."
```

最简单的方法是使用 pipeline()来进行推理。你可以用模型实例化一个 NER（命名实体识别）的管道，然后把文本传给它：

```
from transformers import pipeline

classifier = pipeline("ner", model="stevhliu/my_awesome_wnut_model")
classifier(text)
```

如果你需要的话，还可以手动获取 `pipeline` 的输出结果：

PyTorch

对文本进行标记，并返回一个 PyTorch 张量：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("stevhliu/my_awesome_wnut_model")
inputs = tokenizer(text, return_tensors="pt")
```

将输入数据传递给模型，并获取输出的 `logits`：

```
from transformers import AutoModelForTokenClassification

model = AutoModelForTokenClassification.from_pretrained("stevhliu/my_awesome_wnut_model")
with torch.no_grad():
    logits = model(**inputs).logits
```

找出概率最高的类别，然后利用模型的 `id2label` 映射将其转换成对应的文本标签：

```
predictions = torch.argmax(logits, dim=2)
predicted_token_class = [model.config.id2label[t.item()] for t in predictions[0]]
predicted_token_class
```

TensorFlow

对文本进行标记化处理，并返回一个 TensorFlow 张量：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("stevhliu/my_awesome_wnut_model")
inputs = tokenizer(text, return_tensors="tf")
```

将输入数据传递给模型，并获取输出的 `logits`：

```
from transformers import TFAutoModelForTokenClassification

model = TFAutoModelForTokenClassification.from_pretrained("stevhliu/my_awesome_wnut_model")
logits = model(**inputs).logits
```

找出概率最高的类别，然后利用模型的 `id2label` 映射将其转换成对应的文本标签：

```
predicted_token_class_ids = tf.math.argmax(logits, axis=-1)
predicted_token_class = [model.config.id2label[t] for t in
predicted_token_class_ids[0].numpy().tolist()]
predicted_token_class
```

因果语言模型（CLM）

因果语言模型是语言模型的一种类型，它用于预测序列中下一个标记，并且模型只能关注左侧的标记。这意味着模型无法看到未来的标记。GPT-2 就是一个因果语言模型的例子。

本指南将向您展示如何：

112. 在 ELI5 数据集的 r/askscience 子集上微调 DistilGPT2。

113. 使用您微调的模型进行推理。

在开始之前，请确保您已安装所有必要的库：

```
pip install transformers datasets evaluate
```

我们建议您登录到您的 Hugging Face 账户，这样您就可以上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 ELI5 数据集

首先，使用 🤗 Datasets 库加载 ELI5-Category 数据集的前 5000 个示例。这将让您有机会进行实验并确保一切正常工作，然后再在完整数据集上花费更多时间进行训练。

```
from datasets import load_dataset
eli5 = load_dataset("eli5_category", split="train[:5000]")
```

使用 train_test_split 方法将数据集的 `train` 分割成训练集和测试集：

```
eli5 = eli5.train_test_split(test_size=0.2)
```

然后查看一个示例：

```
eli5["train"][0]
```

虽然这可能看起来很多，但您真正感兴趣的是 `text` 字段。语言建模任务酷的地方在于您不需要标签（也称为无监督任务），因为下一个词就是标签。

预处理

下一步是加载 DistilGPT2 分词器来处理 `text` 子字段：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")
```

您会注意到上面的示例中，`text` 字段实际上嵌套在 `answers` 中。这意味着您需要使用 `flatten` 方法从其嵌套结构中提取 `text` 子字段：

```
eli5 = eli5.flatten()
eli5["train"][0]
```

每个子字段现在都是一个单独的列，如 `answers` 前缀所示，`text` 字段现在是一个列表。不要分别对每个句子进行分词，而是将列表转换为字符串，这样您就可以联合对它们进行分词。

这是一个预处理函数，用于连接每个示例的字符串列表并分词结果：

```
def preprocess_function(examples):
    return tokenizer([" ".join(x) for x in examples["answers.text"]])
```

要使用此预处理函数处理整个数据集，请使用 🤗 Datasets 的 map 方法。您可以通过设置 `batched=True` 来加速 `map` 函数，以便一次处理数据集中的多个元素，并通过 `num_proc` 增加进程数。删除您不需要的任何列：

```
tokenized_eli5 = eli5.map(
    preprocess_function,
    batched=True,
    num_proc=4,
    remove_columns=eli5["train"].column_names,
)
```

此数据集包含标记序列，但其中一些序列的长度超过了模型的最大输入长度。

您现在可以使用第二个预处理函数来：

- 连接所有序列
- 将连接的序列分割成由 `block_size` 定义的较短块，`block_size` 应该既小于最大输入长度，又短到足以适合您的 GPU RAM。

```
block_size = 128


def group_texts(examples):
    # 连接所有文本。
    concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
    total_length = len(concatenated_examples[list(examples.keys())[0]])
    # 我们丢弃小的余数，如果模型支持，我们可以添加填充而不是丢弃，您可以自定义这部分以满足您的需求。
    if total_length >= block_size:
        total_length = (total_length // block_size) * block_size
    # 按 block_size 分块。
    result = {
        k: [t[i:i + block_size] for i in range(0, total_length, block_size)]
        for k, t in concatenated_examples.items()
    }
    result["labels"] = result["input_ids"].copy()
    return result
```

在整个数据集上应用 `group_texts` 函数：

```
lm_dataset = tokenized_eli5.map(group_texts, batched=True, num_proc=4)
```

现在使用 DataCollatorForLanguageModeling 创建一个示例批次。在整理过程中，将句子动态填充到批次中最长的长度比将整个数据集填充到最大长度更有效。

Pytorch

使用序列结束标记作为填充标记，并将 `mlm=False`。这将使用输入作为向右移动一个元素作为标签：

```
from transformers import DataCollatorForLanguageModeling
tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
```

TensorFlow

使用序列结束标记作为填充标记，并将 `mlm=False`。这将使用输入作为向右移动一个元素作为标签：

```
from transformers import DataCollatorForLanguageModeling
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False,
return_tensors="tf")
```

训练

Pytorch

如果您不熟悉使用 Trainer 微调模型，请查看 基本教程！

您现在可以开始训练您的模型了！使用 AutoModelForCausalLM 加载 DistilGPT2：

```
from transformers import AutoModelForCausalLM, TrainingArguments, Trainer
model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
```

在这一点上，只剩下三个步骤：

114. 在 TrainingArguments 中定义您的训练超参数。唯一需要的参数是 `output_dir`，它指定保存模型的位置。您可以通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传模型）。
115. 将训练参数传递给 Trainer，以及模型、数据集和数据整理器。
116. 调用 train() 来微调您的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_eli5_clm-model",
```

```
    eval_strategy="epoch",
    learning_rate=2e-5,
    weight_decay=0.01,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=lm_dataset["train"],
    eval_dataset=lm_dataset["test"],
    data_collator=data_collator,
    tokenizer=tokenizer,
)

trainer.train()
```

训练完成后，使用 evaluate() 方法评估您的模型并获取其困惑度：

```
import math
eval_results = trainer.evaluate()
print(f"Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

然后使用 push_to_hub() 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果您不熟悉使用 Keras 微调模型，请查看 基本教程！

要在 TensorFlow 中微调模型，首先设置优化器函数、学习率计划和一些训练超参数：

```
from transformers import create_optimizer, AdamWeightDecay
optimizer = AdamWeightDecay(learning_rate=2e-5, weight_decay_rate=0.01)
```

然后您可以使用 TFAutoModelForCausalLM 加载 DistilGPT2：

```
from transformers import TFAutoModelForCausalLM
model = TFAutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
```

使用 prepare_tf_dataset() 将您的数据集转换为 `tf.data.Dataset` 格式：

```
tf_train_set = model.prepare_tf_dataset(
    lm_dataset["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)

tf_test_set = model.prepare_tf_dataset(
    lm_dataset["test"],
```

```
    shuffle=False,
    batch_size=16,
    collate_fn=data_collator,
)
```

使用 `compile` 配置模型进行训练。请注意，Transformers 模型都有一个默认的与任务相关的损失函数，因此除非您想要指定一个，否则不需要指定损失函数：

```
import tensorflow as tf
model.compile(optimizer=optimizer)  # 无需指定损失函数
```

您可以通过在 PushToHubCallback 中指定模型和分词器的推送位置来实现：

```
from transformers.keras_callbacks import PushToHubCallback
callback = PushToHubCallback(
    output_dir="my_awesome_eli5_clm-model",
    tokenizer=tokenizer,
)
```

最后，您准备好开始训练您的模型了！使用您的训练和验证数据集、纪元数和回调来微调模型：

```
model.fit(x=tf_train_set, validation_data=tf_test_set, epochs=3, callbacks=[callback])
```

训练完成后，您的模型将自动上传到 Hub，以便每个人都可以使用它！

有关如何微调模型进行因果语言建模的更深入示例，请查看相应的 PyTorch 笔记本 或 TensorFlow 笔记本。

推理

太好了，现在您已经微调了一个模型，您可以使用它进行推理！

想出一个您想从中生成文本的提示：

```
prompt = "Somatic hypermutation allows the immune system to"
```

尝试使用您微调的模型进行推理的最简单方法是将其用于 pipeline()。使用您的模型实例化一个用于文本生成的 `pipeline`，并将您的文本传递给它：

```
from transformers import pipeline
generator = pipeline("text-generation", model="username/my_awesome_eli5_clm-model")
generator(prompt)
```

Pytorch

对文本进行分词并将 `input_ids` 作为 PyTorch 张量返回：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_eli5_clm-model")
inputs = tokenizer(prompt, return_tensors="pt").input_ids
```

使用 generate() 方法生成文本。有关控制生成的不同文本生成策略和参数的更多详细信息，请查看文本生成策略 页面。

```
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained("username/my_awesome_eli5_clm-model")
outputs = model.generate(inputs, max_new_tokens=100, do_sample=True, top_k=50, top_p=0.95)
```

将生成的标记 ID 解码回文本：

```
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

TensorFlow

对文本进行分词并将 `input_ids` 作为 TensorFlow 张量返回：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_eli5_clm-model")
inputs = tokenizer(prompt, return_tensors="tf").input_ids
```

使用 `~transformers.generation_tf_utils.TFGenerationMixin.generate` 方法创建摘要。有关控制生成的不同文本生成策略和参数的更多详细信息，请查看 文本生成策略 页面。

```
from transformers import TFAutoModelForCausalLM
model = TFAutoModelForCausalLM.from_pretrained("username/my_awesome_eli5_clm-model")
outputs = model.generate(input_ids=inputs, max_new_tokens=100, do_sample=True, top_k=50,
top_p=0.95)
```

将生成的标记 ID 解码回文本：

```
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

遮蔽语言模型（MLM）

遮蔽语言模型预测序列中的遮蔽标记，并且模型可以双向关注标记。这意味着模型可以完全访问左右两边的标记。遮蔽语言模型非常适合需要对整个序列有良好上下文理解的任务。BERT 就是一个遮蔽语言模型的例子。

本指南将向你展示如何：

117. 在 ELI5 数据集的 r/askscience 子集上微调 DistilRoBERTa。

118. 使用你的微调模型进行推理。

要查看与此任务兼容的所有架构和检查点，我们推荐查看任务页面。

开始之前，请确保你已安装所有必要的库：

```
pip install transformers datasets evaluate
```

我们鼓励你登录你的 Hugging Face 账户，这样你就可以上传并与社区分享你的模型。当提示时，输入你的 token 以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 ELI5 数据集

首先使用 🤗 Datasets 库加载 ELI5-Category 数据集的前 5000 个示例。这将给你一个机会进行实验并确保一切工作正常，然后再花更多时间在完整数据集上训练。

```
from datasets import load_dataset

eli5 = load_dataset("eli5_category", split="train[:5000]")
```

使用 train_test_split 方法将数据集的 `train` 分割成训练集和测试集：

```
eli5 = eli5.train_test_split(test_size=0.2)
```

然后查看一个示例：

```
eli5["train"][0]
```

虽然这看起来很多，但你真正感兴趣的是 `text` 字段。语言建模任务的酷之处在于你不需要标签（也称为无监督任务），因为下一个词就是标签。

预处理

对于遮蔽语言建模，下一步是加载一个 DistilRoBERTa 分词器来处理 `text` 子字段：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilroberta-base")
```

你会从上面的示例中注意到，`text` 字段实际上嵌套在 `answers` 里面。这意味着你需要使用 `flatten` 方法从其嵌套结构中提取 `text` 子字段：

```
eli5 = eli5.flatten()
eli5["train"][0]
```

现在每个子字段都是一个单独的列，如 `answers` 前缀所示，`text` 字段现在是一个列表。不要单独对每个句子进行分词，将列表转换为字符串，以便你可以一起对它们进行分词。

这是一个预处理函数，用于将每个示例的字符串列表连接起来并分词结果：

```
def preprocess_function(examples):
    return tokenizer([" ".join(x) for x in examples["answers.text"]])
```

要在整个数据集上应用这个预处理函数，使用 🤗 Datasets 的 map 方法。你可以通过设置 `batched=True` 来加速 `map` 函数，以便一次处理数据集中的多个元素，并使用 `num_proc` 增加进程数。删除你不需要的任何列：

```
tokenized_eli5 = eli5.map(
    preprocess_function,
    batched=True,
    num_proc=4,
    remove_columns=eli5["train"].column_names,
)
```

这个数据集包含了标记序列，但其中一些序列比模型的最大输入长度还要长。

现在你可以使用第二个预处理函数来：

- 连接所有的序列
- 将连接的序列分割成由 `block_size` 定义的更短的块，它应该比模型的最大输入长度短，并且足够短以适应你的 GPU 内存。

```
block_size = 128


def group_texts(examples):
    # 连接所有文本。
    concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
    total_length = len(concatenated_examples[list(examples.keys())[0]])
    # 我们可以添加填充而不是这种丢弃，如果模型支持的话，你可以根据自己的需要定制这部分。
    if total_length >= block_size:
        total_length = (total_length // block_size) * block_size
    # 按 block_size 分割块。
    result = {
        k: [t[i : i + block_size] for i in range(0, total_length, block_size)]
        for k, t in concatenated_examples.items()
    }
    return result

lm_dataset = tokenized_eli5.map(group_texts, batched=True, num_proc=4)
```

现在使用 DataCollatorForLanguageModeling 创建一批示例。在整理过程中动态填充句子到批次中的最长长度，而不是将整个数据集填充到最大长度，这样做更有效。

Pytorch

使用序列结束标记作为填充标记，并指定 `mlm_probability` 以在每次遍历数据时随机遮蔽标记：

```
from transformers import DataCollatorForLanguageModeling

tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15)
```

TensorFlow

使用序列结束标记作为填充标记，并指定 `mlm_probability` 以在每次遍历数据时随机遮蔽标记：

```
from transformers import DataCollatorForLanguageModeling

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm_probability=0.15,
return_tensors="tf")
```

训练

Pytorch

如果你不熟悉如何使用 Trainer 微调模型，请查看这里的基本教程！

现在你可以开始训练你的模型了！使用 AutoModelForMaskedLM 加载 DistilRoBERTa：

```
from transformers import AutoModelForMaskedLM

model = AutoModelForMaskedLM.from_pretrained("distilbert/distilroberta-base")
```

此时，只剩下三个步骤：

119. 在 TrainingArguments 中定义你的训练超参数。唯一必需的参数是 `output_dir`，它指定了保存模型的位置。你可以通过设置 `push_to_hub=True`（需要登录 Hugging Face 才能上传你的模型）将此模型推送到 Hub。
120. 将训练参数传递给 Trainer 以及模型、数据集和数据整理器。
121. 调用 train() 微调你的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_eli5_mlm_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
```

```
    num_train_epochs=3,
    weight_decay=0.01,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=lm_dataset["train"],
    eval_dataset=lm_dataset["test"],
    data_collator=data_collator,
    tokenizer=tokenizer,
)

trainer.train()
```

训练完成后，使用 evaluate() 方法评估你的模型并获取它的困惑度：

```
import math

eval_results = trainer.evaluate()
print(f"Perplexity: {math.exp(eval_results['eval_loss']):.2f}")
```

然后使用 push_to_hub() 方法将你的模型分享到 Hub，这样每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果你不熟悉如何使用 Keras 微调模型，请查看这里的基本教程！

要微调 TensorFlow 中的模型，首先设置一个优化器函数、学习率计划和一些训练超参数：

```
from transformers import create_optimizer, AdamWeightDecay

optimizer = AdamWeightDecay(learning_rate=2e-5, weight_decay_rate=0.01)
```

然后你可以使用 TFAutoModelForMaskedLM 加载 DistilRoBERTa：

```
from transformers import TFAutoModelForMaskedLM

model = TFAutoModelForMaskedLM.from_pretrained("distilbert/distilroberta-base")
```

使用 prepare_tf_dataset() 将你的数据集转换为 `tf.data.Dataset` 格式：

```
tf_train_set = model.prepare_tf_dataset(
    lm_dataset["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)
```

```
tf_test_set = model.prepare_tf_dataset(
    lm_dataset["test"],
    shuffle=False,
    batch_size=16,
    collate_fn=data_collator,
)
```

使用 `compile` 配置模型进行训练。注意，Transformer 模型都有一个默认的任务相关损失函数，所以你不需要指定一个，除非你想：

```
import tensorflow as tf

model.compile(optimizer=optimizer)  # No loss argument!
```

这可以通过指定在哪里推送你的模型和分词器来完成 PushToHubCallback：

```
from transformers.keras_callbacks import PushToHubCallback

callback = PushToHubCallback(
    output_dir="my_awesome_eli5_mlm_model",
    tokenizer=tokenizer,
)
```

最后，你准备好开始训练你的模型了！用你的训练和验证数据集、训练周期数和你的回调来调用 `fit` 微调模型：

```
model.fit(x=tf_train_set, validation_data=tf_test_set, epochs=3, callbacks=[callback])
```

一旦训练完成，你的模型会自动上传到 Hub，这样每个人都可以使用它！

有关如何微调遮蔽语言模型的更深入示例，请查看相应的 PyTorch 笔记本 或 TensorFlow 笔记本。

推理

太好了，现在你已经微调了一个模型，你可以用它进行推理了！

想出一些你想让模型填补空白的文本，并使用特殊的 `<mask>` 标记来表示空白：

```
text = "The Milky Way is a <mask> galaxy."
```

尝试你的微调模型进行推理的最简单方法是使用它在 pipeline() 中。为你的模型实例化一个 `pipeline` 进行填充掩码，并传递你的文本给它。如果喜欢，你可以使用 `top_k` 参数来指定返回多少个预测：

```
from transformers import pipeline
```

```
mask_filler = pipeline("fill-mask", "username/my_awesome_eli5_mlm_model")
mask_filler(text, top_k=3)
```

Pytorch

将文本分词并返回 PyTorch 张量的 `input_ids`。你还需要指定 `<mask>` 标记的位置：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_eli5_mlm_model")
inputs = tokenizer(text, return_tensors="pt")
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]

from transformers import AutoModelForMaskedLM

model = AutoModelForMaskedLM.from_pretrained("username/my_awesome_eli5_mlm_model")
logits = model(**inputs).logits
mask_token_logits = logits[0, mask_token_index, :]
```

然后返回三个可能性最高的遮蔽标记并打印它们：

```
top_3_tokens = torch.topk(mask_token_logits, 3, dim=1).indices[0].tolist()

for token in top_3_tokens:
    print(text.replace(tokenizer.mask_token, tokenizer.decode([token])))
```

TensorFlow

将文本分词并返回 TensorFlow 张量的 `input_ids`。你还需要指定 `<mask>` 标记的位置：

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_eli5_mlm_model")
inputs = tokenizer(text, return_tensors="tf")
mask_token_index = tf.where(inputs["input_ids"] == tokenizer.mask_token_id)[0, 1]

from transformers import TFAutoModelForMaskedLM

model = TFAutoModelForMaskedLM.from_pretrained("username/my_awesome_eli5_mlm_model")
logits = model(**inputs).logits
mask_token_logits = logits[0, mask_token_index, :]
```

然后返回三个可能性最高的遮蔽标记并打印它们：

```
top_3_tokens = tf.math.top_k(mask_token_logits, 3).indices.numpy()

for token in top_3_tokens:
    print(text.replace(tokenizer.mask_token, tokenizer.decode([token])))
```

这样，你就可以使用微调后的模型来预测遮蔽词，并得到最有可能的三个填充词。这种方法可以应用于任何需要上下文理解的任务，例如文本生成、问答系统等。

文本翻译

翻译是将一段文本从一种语言转换为另一种语言的过程。它是可以表述为序列到序列（sequence-to-sequence）问题的几种任务之一，这是一个强大的框架，用于从输入返回某些输出，如翻译或总结。翻译系统通常用于不同语言文本之间的翻译，但也可以用于语音或两者之间的某种组合，如文本到语音或语音到文本。

本指南将向您展示如何：

122. 在 OPUS Books 数据集的英语-法语子集上微调 T5，以将英语文本翻译成法语。
123. 使用您微调的模型进行推理。

要查看与该任务兼容的所有架构和检查点，建议查看 任务页面。

在开始之前，请确保已安装所有必要的库：

```
pip install transformers datasets evaluate sacrebleu
```

我们鼓励您登录您的 Hugging Face 账户，以便您可以上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 OPUS Books 数据集

首先从 🤗 Datasets 库中加载 OPUS Books 数据集的英语-法语子集：

```
from datasets import load_dataset
books = load_dataset("opus_books", "en-fr")
```

使用 train_test_split 方法将数据集分为训练集和测试集：

```
books = books["train"].train_test_split(test_size=0.2)
```

然后查看一个示例：

```
books["train"][0]
```

`translation`：文本的英语和法语翻译。

预处理

下一步是加载一个 T5 分词器来处理英语-法语语言对：

```
from transformers import AutoTokenizer
checkpoint = "google-t5/t5-small"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

您要创建的预处理函数需要：

124. 在输入前添加提示，以便 T5 知道这是翻译任务。一些能够执行多种 NLP 任务的模型需要特定任务的提示。

125. 在 `text_target` 参数中设置目标语言（法语），以确保分词器正确处理目标文本。如果不设置 `text_target`，分词器会将目标文本作为英语处理。

126. 截断序列，使其不超过 `max_length` 参数设置的最大长度。

```
source_lang = "en"
target_lang = "fr"
prefix = "translate English to French: "

def preprocess_function(examples):
    inputs = [prefix + example[source_lang] for example in examples["translation"]]
    targets = [example[target_lang] for example in examples["translation"]]
    model_inputs = tokenizer(inputs, text_target=targets, max_length=128, truncation=True)
    return model_inputs
```

要应用预处理函数到整个数据集，使用 🤗 Datasets 的 map 方法。您可以通过设置 `batched=True` 来加速 `map` 函数，以便一次处理数据集的多个元素：

```
tokenized_books = books.map(preprocess_function, batched=True)
```

现在使用 DataCollatorForSeq2Seq 创建一个示例批次。在整理过程中，_动态填充_ 句子到批次中最长的长度更高效，而不是将整个数据集填充到最大长度。

Pytorch

```
from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=checkpoint)
```

TensorFlow

```
from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=checkpoint,
return_tensors="tf")
```

评估

在训练过程中包含一个指标通常有助于评估模型的表现。您可以使用 🤗 Evaluate 库快速加载评估方法。对于此任务，加载 SacreBLEU 指标（参见 🤗 Evaluate 快速教程 以了解更多关于如何加载和计算指标的信息）：

```
import evaluate
metric = evaluate.load("sacrebleu")
```

然后创建一个函数，将您的预测和标签传递给 compute 以计算 SacreBLEU 分数：

```
import numpy as np

def postprocess_text(preds, labels):
    preds = [pred.strip() for pred in preds]
    labels = [[label.strip()] for label in labels]
    return preds, labels

def compute_metrics(eval_preds):
    preds, labels = eval_preds
    if isinstance(preds, tuple):
        preds = preds[0]
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    decoded_preds, decoded_labels = postprocess_text(decoded_preds, decoded_labels)
    result = metric.compute(predictions=decoded_preds, references=decoded_labels)
    result = {"bleu": result["score"]}
    prediction_lens = [np.count_nonzero(pred != tokenizer.pad_token_id) for pred in preds]
    result["gen_len"] = np.mean(prediction_lens)
    result = {k: round(v, 4) for k, v in result.items()}
    return result
```

您的 `compute_metrics` 函数现在准备好了，您将在设置训练时返回到它。

训练

Pytorch

如果您不熟悉使用 Trainer 微调模型，请查看这里的 basic tutorial here!

您现在可以开始训练您的模型了！加载 T5 与 AutoModelForSeq2SeqLM：

```
from transformers import AutoModelForSeq2SeqLM, Seq2SeqTrainingArguments, Seq2SeqTrainer
model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)
```

此时，只剩下三个步骤：

127. 在 Seq2SeqTrainingArguments 中定义您的训练超参数。唯一必需的参数是 `output_dir`，它指定了保存模型的位置。您可以通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传您的模型）。在每个 epoch 结束时，Trainer 将评估 SacreBLEU 指标并保存训练检查点。

128. 将训练参数传递给 Seq2SeqTrainer，同时传递模型、数据集、分词器、数据整理器和 `compute_metrics` 函数。

129. 调用 train() 以微调您的模型。

```
training_args = Seq2SeqTrainingArguments(
    output_dir="my_awesome_opus_books_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=2,
    predict_with_generate=True,
    fp16=True,  # change to bf16=True for XPU
    push_to_hub=True,
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_books["train"],
    eval_dataset=tokenized_books["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

训练完成后，使用 push_to_hub() 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果您不熟悉使用 Keras 微调模型，请查看这里的 basic tutorial here！

要在 TensorFlow 中微调模型，首先设置一个优化器函数、学习率调度和一些训练超参数：

```
from transformers import AdamWeightDecay
optimizer = AdamWeightDecay(learning_rate=2e-5, weight_decay_rate=0.01)
```

然后您可以加载 T5 与 TFAutoModelForSeq2SeqLM：

```
from transformers import TFAutoModelForSeq2SeqLM
model = TFAutoModelForSeq2SeqLM.from_pretrained(checkpoint)
```

使用 prepare_tf_dataset() 将您的数据集转换为 `tf.data.Dataset` 格式：

```
tf_train_set = model.prepare_tf_dataset(
    tokenized_books["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)

tf_test_set = model.prepare_tf_dataset(
    tokenized_books["test"],
    shuffle=False,
    batch_size=16,
    collate_fn=data_collator,
)
```

使用 `compile` 配置模型以进行训练。请注意，Transformers 模型都有一个默认的任务相关损失函数，所以您不需要指定一个，除非您想这样做：

```
import tensorflow as tf
model.compile(optimizer=optimizer)  # No loss argument!
```

在开始训练之前，最后两件事是计算 SacreBLEU 指标和提供一种将您的模型推送到 Hub 的方法。这两者都通过使用 Keras callbacks 完成。

将您的 `compute_metrics` 函数传递给 KerasMetricCallback：

```
from transformers.keras_callbacks import KerasMetricCallback
metric_callback = KerasMetricCallback(metric_fn=compute_metrics, eval_dataset=tf_validation_set)
```

在 PushToHubCallback 中指定要推送模型和分词器的位置：

```
from transformers.keras_callbacks import PushToHubCallback
push_to_hub_callback = PushToHubCallback(
    output_dir="my_awesome_opus_books_model",
    tokenizer=tokenizer,
)
```

然后将您的回调捆绑在一起：

```
callbacks = [metric_callback, push_to_hub_callback]
```

最后，您已经准备好开始训练您的模型了！调用 `fit` 并传入您的训练和验证数据集、epoch 数量和回调以微调模型：

```
model.fit(x=tf_train_set, validation_data=tf_test_set, epochs=3, callbacks=callbacks)
```

训练完成后，您的模型将自动上传到 Hub，以便每个人都可以使用它！

要深入了解如何微调翻译模型，请查看相应的 PyTorch notebook 或 TensorFlow notebook。

推理

太好了，现在您已经微调了一个模型，可以使用它进行推理了！

想出一些您想要翻译成另一种语言的文本。对于 T5，您需要根据您正在处理的任务在输入前添加前缀。对于从英语到法语的翻译，您应该像下面这样添加前缀：

```
text = "translate English to French: Legumes share resources with nitrogen-fixing bacteria."
```

尝试您微调的模型进行推理的最简单方法是使用 pipeline()。使用您的模型实例化一个 `pipeline` 用于翻译，并将您的文本传递给它：

```
from transformers import pipeline
translator = pipeline("translation_en_to_fr", model="username/my_awesome_opus_books_model")
translator(text)
```

您也可以手动复制 `pipeline` 的结果，如果您愿意的话：

Pytorch

对文本进行分词并返回 `input_ids` 作为 PyTorch 张量：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_opus_books_model")
inputs = tokenizer(text, return_tensors="pt").input_ids
```

使用 generate() 方法创建翻译。有关不同文本生成策略和生成控制的更多信息，请查看 Text Generation API。

```
from transformers import AutoModelForSeq2SeqLM
model = AutoModelForSeq2SeqLM.from_pretrained("username/my_awesome_opus_books_model")
outputs = model.generate(inputs, max_new_tokens=40, do_sample=True, top_k=30, top_p=0.95)
```

将生成的 token ids 解码回文本：

```
tokenizer.decode(outputs[0], skip_special_tokens=True)
```

TensorFlow

对文本进行分词并返回 `input_ids` 作为 TensorFlow 张量：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_opus_books_model")
inputs = tokenizer(text, return_tensors="tf").input_ids
```

使用 `~transformers.generation_tf_utils.TFGenerationMixin.generate` 方法创建翻译。有关不同文本生成策略和生成控制的更多信息，请查看 Text Generation API。

```
from transformers import TFAutoModelForSeq2SeqLM
model = TFAutoModelForSeq2SeqLM.from_pretrained("username/my_awesome_opus_books_model")
outputs = model.generate(inputs, max_new_tokens=40, do_sample=True, top_k=30, top_p=0.95)
```

将生成的 token ids 解码回文本：

```
tokenizer.decode(outputs[0], skip_special_tokens=True)
```

文本摘要（文本总结）

文本摘要是将文档或文章的重要信息压缩成更短版本的任务。与翻译一样，它也是可以表述为序列到序列任务的另一个例子。摘要可以是：

- **提取式**：从文档中提取最相关的信息。
- **抽象式**：生成新的文本，捕捉最相关的信息。

本指南将向您展示如何：

130. 在 BillSum 数据集的加利福尼亚州法案子集上微调 T5 模型，用于抽象式摘要。
131. 使用微调后的模型进行推理。

要查看所有与该任务兼容的架构和检查点，我们建议您查看任务页面。

在开始之前，请确保您已安装所有必要的库：

```
pip install transformers datasets evaluate rouge_score
```

我们鼓励您登录您的 Hugging Face 账户，这样您就可以上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 BillSum 数据集

首先，从 🤗 Datasets 库中加载 BillSum 数据集的较小的加利福尼亚州法案子集：

```
from datasets import load_dataset
billsum = load_dataset("billsum", split="ca_test")
```

使用 `train_test_split` 方法将数据集分成训练集和测试集：

```
billsum = billsum.train_test_split(test_size=0.2)
```

然后查看一个示例：

```
billsum["train"][0]
```

您需要使用两个字段：

- `text`：法案的文本，将作为模型的输入。
- `summary`：`text` 的浓缩版本，将作为模型的目标。

预处理

下一步是加载 T5 分词器来处理 `text` 和 `summary`：

```
from transformers import AutoTokenizer
checkpoint = "google-t5/t5-small"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

您需要创建的预处理函数需要执行以下操作：

132. 在输入前加上提示，以便 T5 知道这是一个摘要任务。一些能够处理多种 NLP 任务的模型需要针对特定任务进行提示。
133. 在对标签进行分词时使用 `text_target` 参数。
134. 截断序列，使其长度不超过 `max_length` 参数设置的最大长度。

```
prefix = "summarize: "
def preprocess_function(examples):
    inputs = [prefix + doc for doc in examples["text"]]
    model_inputs = tokenizer(inputs, max_length=1024, truncation=True)
    labels = tokenizer(text_target=examples["summary"], max_length=128, truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

要使用 🤗 Datasets 的 `map` 方法在整个数据集上应用预处理函数，请设置 `batched=True` 以同时处理数据集中的多个元素，从而加快 `map` 函数的速度：

```
tokenized_billsum = billsum.map(preprocess_function, batched=True)
```

现在使用 `DataCollatorForSeq2Seq` 创建一个示例批次。在整理过程中，动态地将句子填充到批次中最长的长度，而不是将整个数据集填充到最大长度，这样做更有效率。

```
from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=checkpoint)
```

评估

在训练过程中包含一个指标通常有助于评估模型的性能。您可以使用 🤗 Evaluate 库快速加载评估方法。对于此任务，加载 ROUGE 指标（请参阅 🤗 Evaluate 快速入门，以了解更多关于如何加载和计算指标的信息）：

```
import evaluate
rouge = evaluate.load("rouge")
```

然后创建一个函数，将您的预测和标签传递给 `compute` 以计算 ROUGE 指标：

```
import numpy as np
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    result = rouge.compute(predictions=decoded_preds, references=decoded_labels,
use_stemmer=True)
    prediction_lens = [np.count_nonzero(pred != tokenizer.pad_token_id) for pred in predictions]
    result["gen_len"] = np.mean(prediction_lens)
    return {k: round(v, 4) for k, v in result.items()}
```

您的 `compute_metrics` 函数现在准备好了，当您设置训练时，您将返回到它。

训练

如果您不熟悉使用 Trainer 进行模型微调，请查看这里的基本教程！

您现在可以开始训练您的模型了！使用 `AutoModelForSeq2SeqLM` 加载 T5：

```
from transformers import AutoModelForSeq2SeqLM, Seq2SeqTrainingArguments, Seq2SeqTrainer
model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)
```

在这一点上，只剩下三个步骤：

135. 在 `Seq2SeqTrainingArguments` 中定义您的训练超参数。唯一需要的参数是 `output_dir`，它指定保存模型的位置。您可以通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传模型）。在每个 epoch 结束时，Trainer 将评估 ROUGE 指标并保存训练检查点。

136. 将训练参数传递给 `Seq2SeqTrainer`，以及模型、数据集、分词器、数据整理器、`compute_metrics` 函数。

137. 调用 `train()` 以微调您的模型。

```
training_args = Seq2SeqTrainingArguments(
    output_dir="my_awesome_billsum_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=4,
    predict_with_generate=True,
    fp16=True, # 对于 XPU，更改为 bf16=True
    push_to_hub=True,
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_billsum["train"],
    eval_dataset=tokenized_billsum["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

一旦训练完成，使用 `push_to_hub()` 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

推理

太好了，现在您已经微调了一个模型，您可以使用它进行推理！

想出一些您想要摘要的文本。对于 T5，您需要根据您正在处理的任务来前缀您的输入。对于摘要，您应该像下面这样前缀您的输入：

```
text = "summarize: The Inflation Reduction Act lowers prescription drug costs, health care costs, and energy costs. It's the most aggressive action on tackling the climate crisis in American history, which will lift up American workers and create good-paying, union jobs across the country. It'll lower the deficit and ask the ultra-wealthy and corporations to pay their fair share. And no one making under $400,000 per year will pay a penny more in taxes."
```

尝试使用微调后的模型进行推理的最简单方法是将其用于 `pipeline()`。为摘要实例化一个 `pipeline`，并将您的文本传递给它：

```
from transformers import pipeline
summarizer = pipeline("summarization", model="username/my_awesome_billsum_model")
summarizer(text)
```

如果您愿意，您也可以手动复制 `pipeline` 的结果：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_billsum_model")
inputs = tokenizer(text, return_tensors="pt").input_ids

from transformers import AutoModelForSeq2SeqLM
model = AutoModelForSeq2SeqLM.from_pretrained("username/my_awesome_billsum_model")
outputs = model.generate(inputs, max_new_tokens=100, do_sample=False)

tokenizer.decode(outputs[0], skip_special_tokens=True)
```

问题解答任务（问答任务）

问题解答任务是给定一个问题，返回一个答案。如果你曾经问过像 Alexa、Siri 或 Google 这样的虚拟助手天气如何，那么你已经使用过问题解答模型了。问题解答任务主要有两种类型：

- **抽取式**：从给定的上下文中提取答案。
- **抽象式**：根据上下文生成一个正确回答问题的答案。

本指南将向你展示如何：

138. 在 SQuAD 数据集上微调 DistilBERT 模型，用于抽取式问题解答。

139. 使用微调后的模型进行推理。

要查看所有与该任务兼容的架构和检查点，我们建议查看 任务页面。

在开始之前，请确保你已经安装了所有必要的库：

```
pip install transformers datasets evaluate
```

我们鼓励你登录你的 Hugging Face 账户，这样你就可以上传并与社区分享你的模型。当提示时，输入你的令牌登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 SQuAD 数据集

首先从 🤗 Datasets 库中加载 SQuAD 数据集的一个较小的子集。这将让你有机会进行实验，并确保一切正常工作，然后再在完整数据集上花费更多时间进行训练。

```
from datasets import load_dataset
squad = load_dataset("squad", split="train[:5000]")
```

使用 train_test_split 方法将数据集的 `train` 分割成一个训练集和测试集：

```
squad = squad.train_test_split(test_size=0.2)
```

然后查看一个示例：

```
squad["train"][0]
```

这里有几个重要的字段：

- `answers`：答案的起始位置和答案文本。
- `context`：模型需要从中提取答案的背景信息。
- `question`：模型应该回答的问题。

预处理

下一步是加载 DistilBERT 分词器来处理 `question` 和 `context` 字段：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

问题解答任务有一些特殊的预处理步骤你应该注意：

140. 数据集中的某些示例可能有一个非常长的 `context`，超过了模型的输入长度限制。为了处理更长的序列，只截断 `context`，设置 `truncation="only_second"`。
141. 接下来，通过设置 `return_offset_mapping=True`，将答案的起始和结束位置映射到原始 `context`。
142. 有了映射，现在你可以找到答案的起始和结束标记。使用 sequence_ids 方法来找到偏移量中哪部分对应于 `question`，哪部分对应于 `context`。

以下是如何创建一个函数来截断并将 `answer` 的起始和结束标记映射到 `context`：

```
def preprocess_function(examples):
    questions = [q.strip() for q in examples["question"]]
    inputs = tokenizer(
```

```
        questions,
        examples["context"],
        max_length=384,
        truncation="only_second",
        return_offsets_mapping=True,
        padding="max_length",
    )

    offset_mapping = inputs.pop("offset_mapping")
    answers = examples["answers"]
    start_positions = []
    end_positions = []

    for i, offset in enumerate(offset_mapping):
        answer = answers[i]
        start_char = answer["answer_start"][0]
        end_char = answer["answer_start"][0] + len(answer["text"][0])
        sequence_ids = inputs.sequence_ids(i)

        # 找到上下文的开始和结束
        idx = 0
        while sequence_ids[idx] != 1:
            idx += 1
        context_start = idx
        while sequence_ids[idx] == 1:
            idx += 1
        context_end = idx - 1

        # 如果答案不完全在上下文中，标记为 (0, 0)
        if offset[context_start][0] > end_char or offset[context_end][1] < start_char:
            start_positions.append(0)
            end_positions.append(0)
        else:
            # 否则它是起始和结束标记的位置
            idx = context_start
            while idx <= context_end and offset[idx][0] <= start_char:
                idx += 1
            start_positions.append(idx - 1)

            idx = context_end
            while idx >= context_start and offset[idx][1] >= end_char:
                idx -= 1
            end_positions.append(idx + 1)

    inputs["start_positions"] = start_positions
    inputs["end_positions"] = end_positions
    return inputs
```

要在整个数据集上应用预处理函数，使用 🤗 Datasets 的 map 函数。你可以通过设置 `batched=True` 来加速 `map` 函数，以便一次处理数据集中的多个元素。删除你不需要的任何列：

```
tokenized_squad = squad.map(preprocess_function, batched=True,
remove_columns=squad["train"].column_names)
```

现在使用 DefaultDataCollator 创建一个示例批次。与其他 🤗 Transformers 中的数据整理器不同，DefaultDataCollator 不会应用任何额外的预处理，如填充。

Pytorch

```
from transformers import DefaultDataCollator
data_collator = DefaultDataCollator()
```

TensorFlow

```
from transformers import DefaultDataCollator
data_collator = DefaultDataCollator(return_tensors="tf")
```

训练

Pytorch

如果你不熟悉使用 Trainer 微调模型，请查看这里的基本教程 这里！

现在你准备好开始训练你的模型了！使用 AutoModelForQuestionAnswering 加载 DistilBERT：

```
from transformers import AutoModelForQuestionAnswering, TrainingArguments, Trainer
model = AutoModelForQuestionAnswering.from_pretrained("distilbert/distilbert-base-uncased")
```

在这一点上，只剩下三个步骤：

143. 在 TrainingArguments 中定义你的训练超参数。唯一需要的参数是 `output_dir`，它指定了保存模型的位置。通过设置 `push_to_hub=True` 将模型推送到 Hub（你需要登录 Hugging Face 才能上传你的模型）。
144. 将训练参数传递给 Trainer，以及模型、数据集、分词器和数据整理器。
145. 调用 train() 来微调你的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_qa_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
```

```
    weight_decay=0.01,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_squad["train"],
    eval_dataset=tokenized_squad["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
)

trainer.train()
```

训练完成后，使用 push_to_hub() 方法将你的模型分享到 Hub，以便每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果你不熟悉使用 Keras 微调模型，请查看这里的基本教程 这里！

要在 TensorFlow 中微调模型，首先设置一个优化器函数、学习率计划和一些训练超参数：

```
from transformers import create_optimizer
batch_size = 16
num_epochs = 2
total_train_steps = (len(tokenized_squad["train"]) // batch_size) * num_epochs
optimizer, schedule = create_optimizer(
    init_lr=2e-5,
    num_warmup_steps=0,
    num_train_steps=total_train_steps,
)
```

然后使用 TFAutoModelForQuestionAnswering 加载 DistilBERT：

```
from transformers import TFAutoModelForQuestionAnswering
model = TFAutoModelForQuestionAnswering.from_pretrained("distilbert/distilbert-base-uncased")
```

使用 prepare_tf_dataset() 将你的数据集转换为 `tf.data.Dataset` 格式：

```
tf_train_set = model.prepare_tf_dataset(
    tokenized_squad["train"],
    shuffle=True,
    batch_size=16,
    collate_fn=data_collator,
)

tf_validation_set = model.prepare_tf_dataset(
    tokenized_squad["test"],
    shuffle=False,
```

```
    batch_size=16,
    collate_fn=data_collator,
)
```

使用 `compile` 配置模型进行训练：

```
import tensorflow as tf
model.compile(optimizer=optimizer)
```

在开始训练之前要设置的最后一件事是提供一种将模型推送到 Hub 的方法。这可以通过在 PushToHubCallback 中指定将模型和分词器推送到哪里来实现：

```
from transformers.keras_callbacks import PushToHubCallback
callback = PushToHubCallback(
    output_dir="my_awesome_qa_model",
    tokenizer=tokenizer,
)
```

最后，你准备好开始训练你的模型了！调用 `fit`，使用你的训练和验证数据集、epoch 数量和你的回调来微调模型：

```
model.fit(x=tf_train_set, validation_data=tf_validation_set, epochs=3, callbacks=[callback])
```

训练完成后，你的模型会自动上传到 Hub，以便每个人都可以使用它！

如果你想要更深入地了解如何为问题解答微调模型，请查看相应的 PyTorch 笔记本 或 TensorFlow 笔记本。

评估

问题解答的评估需要大量的后处理。为了不占用你太多时间，本指南跳过了评估步骤。Trainer 仍然在训练过程中计算评估损失，所以你不会完全不知道你的模型的性能。

如果你有更多时间，并且对如何评估你的问题解答模型感兴趣，请查看 🤗 Hugging Face 课程中的 问题解答 章节！

推理

太好了，现在你已经微调了一个模型，你可以使用它进行推理了！

想出一个问题和一些你想要模型预测的上下文：

```
question = "How many programming languages does BLOOM support?"
context = "BLOOM has 176 billion parameters and can generate text in 46 natural languages and
13 programming languages."
```

尝试你的微调模型进行推理的最简单方法是使用它在一个 pipeline() 中。为问题解答实例化一个 `pipeline`，并将你的文本传递给它：

```
from transformers import pipeline
question_answerer = pipeline("question-answering", model="my_awesome_qa_model")
question_answerer(question=question, context=context)
```

如果你愿意，你也可以手动复制 `pipeline` 的结果：

Pytorch

对文本进行分词并返回 PyTorch 张量：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("my_awesome_qa_model")
inputs = tokenizer(question, context, return_tensors="pt")
```

将你的输入传递给模型并返回 `logits`：

```
import torch
from transformers import AutoModelForQuestionAnswering
model = AutoModelForQuestionAnswering.from_pretrained("my_awesome_qa_model")
with torch.no_grad():
    outputs = model(**inputs)
```

从模型输出中获取起始和结束位置的最高概率：

```
answer_start_index = outputs.start_logits.argmax()
answer_end_index = outputs.end_logits.argmax()
```

解码预测的标记以获得答案：

```
predict_answer_tokens = inputs.input_ids[0, answer_start_index : answer_end_index + 1]
tokenizer.decode(predict_answer_tokens)
```

TensorFlow

对文本进行分词并返回 TensorFlow 张量：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("my_awesome_qa_model")
inputs = tokenizer(question, text, return_tensors="tf")
```

将你的输入传递给模型并返回 `logits`：

```
from transformers import TFAutoModelForQuestionAnswering
model = TFAutoModelForQuestionAnswering.from_pretrained("my_awesome_qa_model")
outputs = model(**inputs)
```

从模型输出中获取起始和结束位置的最高概率：

```
answer_start_index = int(tf.math.argmax(outputs.start_logits, axis=-1)[0])
answer_end_index = int(tf.math.argmax(outputs.end_logits, axis=-1)[0])
```

解码预测的标记以获得答案：

```
predict_answer_tokens = inputs.input_ids[0, answer_start_index : answer_end_index + 1]
tokenizer.decode(predict_answer_tokens)
```

多项选择任务

多项选择任务类似于问答任务，不同之处在于提供了多个候选答案以及一些上下文，模型需要训练以选择正确的答案。

本指南将向您展示如何：

146. 在 SWAG 数据集的 `regular` 配置上微调 BERT，以在给定的多个选项和一些上下文中选择最佳答案。
147. 使用您微调的模型进行推理。

在开始之前，请确保您已安装所有必要的库：

```
pip install transformers datasets evaluate
```

我们鼓励您登录您的 Hugging Face 账户，这样您就可以上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 SWAG 数据集

首先从 🤗 Datasets 库中加载 SWAG 数据集的 `regular` 配置：

```
from datasets import load_dataset
swag = load_dataset("swag", "regular")
```

然后查看一个示例：

```
swag["train"][0]
```

虽然这里看起来有很多字段，但实际上非常简单：

- `sent1` 和 `sent2`：这些字段显示了句子是如何开始的，如果您将这两个字段放在一起，就会得到 `startphrase` 字段。
- `ending`：建议一个可能的句子结束方式，但只有一个是正确的。

- `label`：标识正确的句子结束。

预处理

下一步是加载 BERT 分词器来处理句子开头和四个可能的结尾：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
```

您需要创建的预处理函数需要执行以下操作：

148. 制作 `sent1` 字段的四份副本，并将每个副本与 `sent2` 结合以重新创建句子开头的方式。
149. 将 `sent2` 与每个可能的句子结尾结合。
150. 扁平化这两个列表以便进行分词，然后在分词后进行反扁平化，以便每个示例都有相应的 `input_ids`、`attention_mask` 和 `labels` 字段。

```
ending_names = ["ending0", "ending1", "ending2", "ending3"]

def preprocess_function(examples):
    first_sentences = [[context] * 4 for context in examples["sent1"]]
    question_headers = examples["sent2"]
    second_sentences = [
        [f"{header} {examples[end][i]}" for end in ending_names] for i, header in
enumerate(question_headers)
    ]

    first_sentences = sum(first_sentences, [])
    second_sentences = sum(second_sentences, [])

    tokenized_examples = tokenizer(first_sentences, second_sentences, truncation=True)
    return {k: [v[i:i + 4] for i in range(0, len(v), 4)] for k, v in tokenized_examples.items()}
```

要在整个数据集上应用预处理函数，请使用 🤗 Datasets 的 `map` 方法。您可以通过设置 `batched=True` 来加速 `map` 函数，以便一次处理数据集中的多个元素：

```
tokenized_swag = swag.map(preprocess_function, batched=True)
```

🤗 Transformers 没有为多项选择提供数据整理器，因此您需要调整 `DataCollatorWithPadding` 来创建一个示例批次。在整理过程中，动态地将句子填充到批次中最长的长度，而不是将整个数据集填充到最大长度，这样做更有效率。

`DataCollatorForMultipleChoice` 扁平化所有模型输入，应用填充，然后反扁平化结果：

```
from dataclasses import dataclass
from transformers.tokenization_utils_base import PreTrainedTokenizerBase, PaddingStrategy
```

```
from typing import Optional, Union
import torch

@dataclass
class DataCollatorForMultipleChoice:
    """
    Data collator that will dynamically pad the inputs for multiple choice received.
    """

    tokenizer: PreTrainedTokenizerBase
    padding: Union[bool, str, PaddingStrategy] = True
    max_length: Optional[int] = None
    pad_to_multiple_of: Optional[int] = None

    def __call__(self, features):
        label_name = "label" if "label" in features[0].keys() else "labels"
        labels = [feature.pop(label_name) for feature in features]
        batch_size = len(features)
        num_choices = len(features[0]["input_ids"])
        flattened_features = [
            [{k: v[i] for k, v in feature.items()} for i in range(num_choices)] for feature in
features
        ]
        flattened_features = sum(flattened_features, [])

        batch = self.tokenizer.pad(
            flattened_features,
            padding=self.padding,
            max_length=self.max_length,
            pad_to_multiple_of=self.pad_to_multiple_of,
            return_tensors="pt",
        )

        batch = {k: v.view(batch_size, num_choices, -1) for k, v in batch.items()}
        batch["labels"] = torch.tensor(labels, dtype=torch.int64)
        return batch
```

评估

在训练过程中包含一个指标通常有助于评估您的模型性能。您可以使用 🤗 Evaluate 库快速加载一个评估方法。对于此任务，加载准确性指标（请参阅 🤗 Evaluate 快速入门以了解更多关于如何加载和计算指标的信息）：

```
import evaluate
accuracy = evaluate.load("accuracy")
```

然后创建一个函数，将您的预测和标签传递给 `compute` 以计算准确性：

```
import numpy as np

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return accuracy.compute(predictions=predictions, references=labels)
```

您的 `compute_metrics` 函数现在准备好了，当您设置训练时，您将返回到它。

训练

如果您不熟悉使用 `Trainer` 微调模型，请查看基本教程！

您现在可以开始训练您的模型了！使用 `AutoModelForMultipleChoice` 加载 BERT：

```
from transformers import AutoModelForMultipleChoice, TrainingArguments, Trainer
model = AutoModelForMultipleChoice.from_pretrained("google-bert/bert-base-uncased")
```

在这一点上，只剩下三个步骤：

151. 在 `TrainingArguments` 中定义您的训练超参数。唯一需要的参数是 `output_dir`，它指定了保存模型的位置。您可以通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传模型）。在每个 epoch 结束时，`Trainer` 将评估准确性并保存训练检查点。
152. 将训练参数传递给 `Trainer`，以及模型、数据集、分词器、数据整理器和 `compute_metrics` 函数。
153. 调用 `train()` 以微调您的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_swag_model",
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    learning_rate=5e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_swag["train"],
```

```
    eval_dataset=tokenized_swag["validation"],
    processing_class=tokenizer,
    data_collator=DataCollatorForMultipleChoice(tokenizer=tokenizer),
    compute_metrics=compute_metrics,
)

trainer.train()
```

训练完成后，使用 `push_to_hub()` 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

推理

太好了，现在您已经微调了一个模型，您可以使用它进行推理！

想出一些文本和两个候选答案：

```
prompt = "France has a bread law, Le Décret Pain, with strict rules on what is allowed in a traditional baguette."
candidate1 = "The law does not apply to croissants and brioche."
candidate2 = "The law applies to baguettes."
```

对每个提示和候选答案对进行分词，并返回 PyTorch 张量。您还应该创建一些 `labels`：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("username/my_awesome_swag_model")
inputs = tokenizer([[prompt, candidate1], [prompt, candidate2]], return_tensors="pt", padding=True)
labels = torch.tensor(0).unsqueeze(0)
```

将您的输入和标签传递给模型，并返回 `logits`：

```
from transformers import AutoModelForMultipleChoice
model = AutoModelForMultipleChoice.from_pretrained("username/my_awesome_swag_model")
outputs = model(**{k: v.unsqueeze(0) for k, v in inputs.items()}, labels=labels)
logits = outputs.logits
```

获取概率最高的类别：

```
predicted_class = logits.argmax().item()
predicted_class
```

音频处理

音频分类

音频分类与文本分类类似，都是从输入数据中分配一个类别标签。不同之处在于，音频分类处理的是原始音频波形，而不是文本。音频分类的一些实际应用包括识别说话人的意图、语言分类，甚至是通过声音识别动物种类。

本指南将向你展示如何：

154. 在 MInDS-14 数据集上微调 Wav2Vec2 模型，以分类说话人的意图。

155. 使用微调后的模型进行推理。

要查看与此任务兼容的所有架构和检查点，建议查阅 任务页面。

在开始之前，请确保你已经安装了所有必要的库：

```
pip install transformers datasets evaluate
```

我们鼓励你登录你的 Hugging Face 账户，以便你可以上传并分享你的模型。当提示你时，输入你的令牌来登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 MInDS-14 数据集

首先从 🤗 Datasets 库加载 MInDS-14 数据集：

```
from datasets import load_dataset, Audio

minds = load_dataset("PolyAI/minds14", name="en-US", split="train")
```

使用 train_test_split 方法将数据集的 `train` 分割成较小的训练集和测试集。这可以让你在实验和确保一切正常工作之前，不必花费太多时间在完整数据集上。

```
minds = minds.train_test_split(test_size=0.2)
```

然后查看数据集：

```
minds
```

输出：

虽然数据集中包含了很多有用的信息，如 `lang_id` 和 `english_transcription`，但在本指南中，我们将重点关注 `audio` 和 `intent_class`。使用 remove_columns 方法移除其他列：

```
minds = minds.remove_columns(["path", "transcription", "english_transcription", "lang_id"])
```

现在查看一个示例：

```
minds["train"][0]
```

输出：

这里有两个字段：

- `audio`：一维 `array` 的语音信号，需要调用来加载和重采样音频文件。
- `intent_class`：表示说话人意图的类别 ID。

为了方便模型从标签 ID 获取标签名称，创建一个字典，将标签名称映射到整数，反之亦然：

```
labels = minds["train"].features["intent_class"].names
label2id, id2label = dict(), dict()
for i, label in enumerate(labels):
    label2id[label] = str(i)
    id2label[str(i)] = label
```

现在你可以将标签 ID 转换为标签名称：

```
id2label[str(2)]
```

输出：

预处理

下一步是加载一个 Wav2Vec2 特征提取器来处理音频信号：

```
from transformers import AutoFeatureExtractor

feature_extractor = AutoFeatureExtractor.from_pretrained("facebook/wav2vec2-base")
```

MInDS-14 数据集的采样率为 8000Hz（你可以在其 数据集卡片 中找到这些信息），这意味着你需要将数据集重采样到 16000Hz 以使用预训练的 Wav2Vec2 模型：

```
minds = minds.cast_column("audio", Audio(sampling_rate=16000))
minds["train"][0]
```

输出：

现在创建一个预处理函数，该函数：

156. 调用 `audio` 列来加载音频文件，并在必要时重采样。

157. 检查音频文件的采样率是否与模型预训练时的采样率匹配。你可以在 Wav2Vec2 的 模型卡片 中找到这些信息。

158. 设置最大输入长度，以便在不截断的情况下批量处理较长的输入。

```
def preprocess_function(examples):
    audio_arrays = [x["array"] for x in examples["audio"]]
    inputs = feature_extractor(
        audio_arrays, sampling_rate=feature_extractor.sampling_rate, max_length=16000,
truncation=True
    )
    return inputs
```

为了在整个数据集上应用预处理函数，使用 🤗 Datasets 的 map 函数。通过设置 `batched=True` 可以加快 `map` 的速度，以便一次处理多个数据集元素。移除不需要的列，并将 `intent_class` 重命名为 `label`，因为这是模型期望的名称：

```
encoded_minds = minds.map(preprocess_function, remove_columns="audio", batched=True)
encoded_minds = encoded_minds.rename_column("intent_class", "label")
```

评估

在训练过程中包含一个评估指标通常有助于评估模型的性能。你可以使用 🤗 Evaluate 库快速加载评估方法。对于此任务，加载 准确率 指标（有关如何加载和计算指标的更多信息，请参阅 🤗 Evaluate 的 快速入门）：

```
import evaluate

accuracy = evaluate.load("accuracy")
```

然后创建一个函数，将预测值和标签传递给 `compute` 以计算准确率：

```
import numpy as np

def compute_metrics(eval_pred):
    predictions = np.argmax(eval_pred.predictions, axis=1)
    return accuracy.compute(predictions=predictions, references=eval_pred.label_ids)
```

你现在已经准备好了 `compute_metrics` 函数，稍后在设置训练时会用到它。

训练

如果你不熟悉使用 Trainer 微调模型，请参考 这里 的基本教程！

你现在可以开始训练模型了！使用 AutoModelForAudioClassification 加载 Wav2Vec2，并指定预期的标签数量和标签映射：

```
from transformers import AutoModelForAudioClassification, TrainingArguments, Trainer

num_labels = len(id2label)
model = AutoModelForAudioClassification.from_pretrained(
    "facebook/wav2vec2-base", num_labels=num_labels, label2id=label2id, id2label=id2label
)
```

此时只剩下三个步骤：

159. 在 TrainingArguments 中定义训练超参数。唯一必需的参数是 `output_dir`，它指定了保存模型的位置。通过设置 `push_to_hub=True` 可以将模型推送到 Hub（需要登录 Hugging Face 才能上传模型）。每个 epoch 结束时，Trainer 将评估准确率并保存训练检查点。
160. 将训练参数传递给 Trainer，并传入模型、数据集、分词器、数据收集器和 `compute_metrics` 函数。
161. 调用 train() 来微调模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_mind_model",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=32,
    gradient_accumulation_steps=4,
    per_device_eval_batch_size=32,
    num_train_epochs=10,
    warmup_ratio=0.1,
    logging_steps=10,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=encoded_minds["train"],
    eval_dataset=encoded_minds["test"],
    processing_class=feature_extractor,
```

```
    compute_metrics=compute_metrics,
)

trainer.train()
```

训练完成后，使用 push_to_hub() 方法将模型推送到 Hub，以便每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

有关如何微调模型进行音频分类的更详细示例，请参阅相应的 PyTorch 笔记本。

推理

很好，现在你已经微调了一个模型，可以使用它进行推理了！

加载一个你想进行推理的音频文件。如果需要，记得将音频文件的采样率调整为模型的采样率！

```
from datasets import load_dataset, Audio

dataset = load_dataset("PolyAI/minds14", name="en-US", split="train")
dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))
sampling_rate = dataset.features["audio"].sampling_rate
audio_file = dataset[0]["audio"]["path"]
```

最简单的方法是使用 pipeline() 进行推理。实例化一个用于音频分类的 `pipeline`，并将你的音频文件传递给它：

```
from transformers import pipeline

classifier = pipeline("audio-classification", model="stevhliu/my_awesome_minds_model")
classifier(audio_file)
```

输出：

你也可以手动复制 `pipeline` 的结果：

加载一个特征提取器来预处理音频文件，并返回 `input` 作为 PyTorch 张量：

```
from transformers import AutoFeatureExtractor

feature_extractor = AutoFeatureExtractor.from_pretrained("stevhliu/my_awesome_minds_model")
inputs = feature_extractor(dataset[0]["audio"]["array"], sampling_rate=sampling_rate,
return_tensors="pt")
```

将输入传递给模型并返回 logits：

```
from transformers import AutoModelForAudioClassification

model = AutoModelForAudioClassification.from_pretrained("stevhliu/my_awesome_minds_model")
```

```
with torch.no_grad():
    logits = model(**inputs).logits
```

获取概率最高的类别，并使用模型的 `id2label` 映射将其转换为标签：

```
import torch

predicted_class_ids = torch.argmax(logits).item()
predicted_label = model.config.id2label[predicted_class_ids]
predicted_label
```

输出：

自动语音识别（ASR）

自动语音识别（ASR）是将语音信号转换为文本的技术，它将音频输入序列映射到文本输出。

像 Siri 和 Alexa 这样的虚拟助手每天都使用 ASR 模型来帮助用户，还有许多其他有用的面向用户的应用程序，例如会议期间的实时字幕和笔记。

本指南将向你展示如何：

162. **微调 Wav 2 Vec 2**：在 MInDS-14 数据集上微调 Wav 2 Vec 2 模型，将音频转录为文本。
163. **使用微调模型进行推理**：利用微调后的模型进行语音识别推理。

要查看与此任务兼容的所有体系结构和 checkpoints，我们建议你访问任务页面进行查看。

在开始之前，请确保你已经安装了所有必要的库：

```
pip install transformers datasets evaluate jiwer
```

我们鼓励你登录你的 Hugging Face 账户，这样你就可以上传并与社区分享你的模型。当提示时，输入你的令牌进行登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 MInDS-14 数据集

首先，从🤗 Datasets 库中加载 MInDS-14 数据集的一个较小子集。这将给你一个机会进行实验，确保一切正常工作，然后再花更多时间在完整数据集上进行训练。

```
from datasets import load_dataset, Audio

minds = load_dataset("PolyAI/minds14", name="en-US", split="train[:100]")
```

使用 `Dataset.train_test_split` 方法将数据集的训练部分划分为训练集和测试集：

```
minds = minds.train_test_split(test_size=0.2)
```

然后，查看数据集的内容：

```
minds
```

虽然数据集包含许多有用信息，如 lang_id 和 english_transcription，但本指南中你将专注于音频和转录文本。使用 remove_columns 方法移除其他列：

```
minds = minds.remove_columns(["english_transcription", "intent_class", "lang_id"])
```

数据集中有两个主要字段：

164. **audio**：一个 1 维数组，包含语音信号，必须调用以加载和重采样音频文件。

165. **transcription**：目标文本。

预处理数据

下一步是加载一个 Wav2Vec2 预训练的处理器来处理音频信号：

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("facebook/wav2vec2-base")
```

MInDS-14 数据集的采样率为 8000kHz（你可以在其数据集的 Readme 中找到这一信息），这意味着你需要将数据集重采样到 16000kHz，以便使用预训练的 Wav2Vec2 模型：

```
minds = minds.cast_column("audio", Audio(sampling_rate=16_000))
minds["train"][0]
```

如上转录文本所示，文本包含大小写混合的字符。

Wav2Vec2 分词器只在大写字符上进行了训练，因此你需要确保文本中的内容都能够与分词器的词汇表相匹配：

```
def uppercase(example):
    return {"transcription": example["transcription"].upper()}


minds = minds.map(uppercase)
```

现在，创建一个预处理函数，该函数将：

166. 调用音频列以加载和重采样音频文件。

167. 从音频文件中提取输入值，并使用处理器对转录列进行分词。

```
def prepare_dataset(batch):
    audio = batch["audio"]
    batch = processor(audio["array"], sampling_rate=audio["sampling_rate"],
text=batch["transcription"])
    batch["input_length"] = len(batch["input_values"][0])
    return batch
```

要将对整个数据集应用预处理函数，使用🤗 Datasets 的 map 函数。**你可以通过增加`num_proc`参数中的进程数来加速 map 操作。**使用 remove_columns 方法移除不需要的列：

```
encoded_minds = minds.map(prepare_dataset, remove_columns=minds.column_names["train"],
num_proc=4)
```

🤗 Transformers 库中没有为自动语音识别（ASR）提供现成的数据整理器，因此你需要调整 DataCollatorWithPadding 来创建一个示例批次。

它还将动态地填充你的文本和标签，使其与批次中最长元素的长度一致（而不是整个数据集的长度），从而使它们具有统一的长度。虽然在分词器函数中通过设置 `padding=True` 来填充文本是可能的，但使用动态填充会更加更高效。

与其他数据整理器不同，这个特定的数据整理器需要对 `input_values` 和 `labels` 应用不同的填充方法：

```
import torch

from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Union


@dataclass
class DataCollatorCTCWithPadding:
    processor: AutoProcessor
    padding: Union[bool, str] = "longest"

    def __call__(self, features: List[Dict[str, Union[List[int], torch.Tensor]]]) -> Dict[str,
torch.Tensor]:
        # 将输入和标签分开，因为它们必须具有不同的长度并且需要不同的填充方法。
        input_features = [{"input_values": feature["input_values"][0]} for feature in features]
        label_features = [{"input_ids": feature["labels"]} for feature in features]

        batch = self.processor.pad(input_features, padding=self.padding, return_tensors="pt")

        labels_batch = self.processor.pad(labels=label_features, padding=self.padding,
```

```
return_tensors="pt")

        # 用 -100（表示无声音）替换填充，以正确忽略损失。
        labels = labels_batch["input_ids"].masked_fill(labels_batch.attention_mask.ne(1), -100)

        batch["labels"] = labels

        return batch
```

现在，实例化你的 `DataCollatorForCTCWithPadding`：

```
data_collator = DataCollatorCTCWithPadding(processor=processor, padding="longest")
```

评估模型

在训练过程中包含一个评估指标，通常有助于评估模型的性能。

你可以使用🤗 Evaluate 库快速加载一个评估方法。对于这个任务，加载词错误率（WER）指标（查看🤗 Evaluate 快速教程，了解更多关于如何加载和计算指标的信息）：

```
import evaluate

wer = evaluate.load("wer")
```

然后，创建一个函数，将你的预测和标签传递给 compute 函数来计算 WER：

```
import numpy as np


def compute_metrics(pred):
    pred_logits = pred.predictions
    pred_ids = np.argmax(pred_logits, axis=-1)

    pred.label_ids[pred.label_ids == -100] = processor.tokenizer.pad_token_id

    pred_str = processor.batch_decode(pred_ids)
    label_str = processor.batch_decode(pred.label_ids, group_tokens=False)

    wer = wer.compute(predictions=pred_str, references=label_str)

    return {"wer": wer}
```

你的 `compute_metrics` 函数现在已经准备好了，在设置训练时你将再次使用它。

训练模型

如果你不熟悉如何使用 Trainer 进行模型微调，可以查看这里的基本教程！

现在你已经准备好开始训练模型了！使用 AutoModelForCTC 加载 `Wav2Vec2`。通过 `ctc_loss_reduction` 参数指定要应用的缩减方法。通常使用平均值比默认的求和更好：

```
from transformers import AutoModelForCTC, TrainingArguments, Trainer

model = AutoModelForCTC.from_pretrained(
    "facebook/wav2vec2-base",
    ctc_loss_reduction="mean",
    pad_token_id=processor.tokenizer.pad_token_id,
)
```

`ctc_loss_reduction`参数说明

`ctc_loss_reduction` 参数并不直接指定要应用的损失函数，而是用于确定在计算连接时序分类（CTC）损失时如何处理多个时间步的损失值。具体来说，它决定了是将这些损失值求和（sum）还是取平均值（mean）。默认情况下，CTC 损失是求和的，但有时使用平均值可以更好地平衡不同长度的输入序列对总损失的贡献。

- **sum**：将所有时间步的损失值相加。
- **mean**：将所有时间步的损失值取平均。

通过设置 `ctc_loss_reduction` 参数，你可以选择更适合你任务的处理方式。

至此，只剩下三个步骤：

168. 在 TrainingArguments 中定义你的训练超参数。唯一必需的参数是 `output_dir`，它指定了保存模型的位置。通过设置 `push_to_hub=True`，你将把这个模型推送到 Hub（你需要登录 Hugging Face 才能上传模型）。在每个 epoch 结束时，Trainer 将评估 WER 并保存训练检查点。
169. 将训练参数传递给 Trainer，同时包括模型、数据集、分词器、数据整理器和 compute_metrics 函数。
170. 调用 train()方法来开始微调你的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_asr_mind_model",
    per_device_train_batch_size=8,
    gradient_accumulation_steps=2,
    learning_rate=1e-5,
    warmup_steps=500,
    max_steps=2000,
    gradient_checkpointing=True,
    fp16=True,
```

```
    group_by_length=True,
    eval_strategy="steps",
    per_device_eval_batch_size=8,
    save_steps=1000,
    eval_steps=1000,
    logging_steps=25,
    load_best_model_at_end=True,
    metric_for_best_model="wer",
    greater_is_better=False,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=encoded_minds["train"],
    eval_dataset=encoded_minds["test"],
    processing_class=processor,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

训练完成后，使用 `push_to_hub()`方法将你的模型分享到 Hub，这样每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

如果你想了解更多关于如何微调自动语音识别（ASR）模型的详细示例，可以查看以下博客文章：

- 英文 ASR 微调示例
- 多语言 ASR 微调示例

模型推理

当你已经微调好了一个模型，就可以用它来进行推理了！

加载你想要进行推理的音频文件。如果需要重新采样的话，记得设置音频文件的采样率与模型的采样率一致！

```
from datasets import load_dataset, Audio

dataset = load_dataset("PolyAI/minds14", "en-US", split="train")
dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))
sampling_rate = dataset.features["audio"].sampling_rate
audio_file = dataset[0]["audio"]["path"]
```

尝试使用你的微调模型进行推理的最简单方法是使用 `pipeline()`。实例化一个用于自动语音识别的 pipeline，并将你的音频文件传递给它：

```
from transformers import pipeline

transcriber = pipeline("automatic-speech-recognition",
model="stevhliu/my_awesome_asr_minds_model")
transcriber(audio_file)
```

转录结果应该还可以更好！可以尝试在更多示例上微调你的模型，以获得更佳的结果！

你也可以手动复现 `pipeline` 的结果：

加载一个处理器来预处理音频文件和转录文本，并将输入转换为 PyTorch 张量：

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained("stevhliu/my_awesome_asr_mind_model")
inputs = processor(dataset[0]["audio"]["array"], sampling_rate=sampling_rate,
return_tensors="pt")
```

将你的输入传递给模型，并获取输出日志，查看概率分布（logits）：

```
from transformers import AutoModelForCTC

model = AutoModelForCTC.from_pretrained("stevhliu/my_awesome_asr_mind_model")
with torch.no_grad():
    logits = model(**inputs).logits
```

获取具有最高概率的预测输入 ID，并使用处理器将这些预测输入 ID 解码回文本：

```
import torch

predicted_ids = torch.argmax(logits, dim=-1)
transcription = processor.batch_decode(predicted_ids)
transcription
```

计算机视觉

图像分类

图像分类是为图像分配标签或类别的任务。与文本或音频分类不同，图像分类的输入是构成图像的像素值。图像分类有许多应用，例如在自然灾害后检测损坏情况、监测作物健康状况或帮助筛选医学图像以寻找疾病的迹象。

本指南将说明如何：

171. 在 Food-101 数据集上微调 ViT 模型，以对图像中的食物项进行分类。

172. 使用微调后的模型进行推理。

要查看所有与该任务兼容的架构和检查点，我们建议查看 任务页面。

在开始之前，请确保您已安装所有必要的库：

```
pip install transformers datasets evaluate accelerate pillow torchvision scikit-learn
```

我们鼓励您登录您的 Hugging Face 账户，上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

加载 Food-101 数据集

首先，从 🤗 Datasets 库中加载 Food-101 数据集的一个较小的子集。这将让您有机会进行实验，并确保一切正常工作，然后再在完整数据集上花费更多时间进行训练。

```
from datasets import load_dataset
food = load_dataset("food101", split="train[:5000]")
```

使用 train_test_split 方法将数据集的 `train` 分割成训练集和测试集：

```
food = food.train_test_split(test_size=0.2)
```

然后查看一个示例：

```
food["train"][0]
```

数据集中的每个示例都有两个字段：

- `image`：食物项的 PIL 图像
- `label`：食物项的标签类别

为了使模型能够从标签 ID 获取标签名称，创建一个将标签名称映射到整数和反之亦然的字典：

```
labels = food["train"].features["label"].names
label2id, id2label = dict(), dict()
for i, label in enumerate(labels):
    label2id[label] = str(i)
    id2label[str(i)] = label
```

现在，您可以将标签 ID 转换为标签名称：

```
id2label[str(79)]
```

预处理

下一步是加载 ViT 图像处理器，将图像处理成张量：

```
from transformers import AutoImageProcessor
checkpoint = "google/vit-base-patch16-224-in21k"
image_processor = AutoImageProcessor.from_pretrained(checkpoint)
```

Pytorch

对图像应用一些图像变换，使模型更能抵抗过拟合。这里您将使用 torchvision 的 `transforms` 模块，但您也可以使用您喜欢的任何图像库。

裁剪图像的随机部分，调整其大小，并使用图像均值和标准差进行归一化：

```
from torchvision.transforms import RandomResizedCrop, Compose, Normalize, ToTensor
normalize = Normalize(mean=image_processor.image_mean, std=image_processor.image_std)
size = (
    image_processor.size["shortest_edge"]
    if "shortest_edge" in image_processor.size
    else (image_processor.size["height"], image_processor.size["width"])
)
_transforms = Compose([RandomResizedCrop(size), ToTensor(), normalize])
```

然后创建一个预处理函数，应用变换并返回图像的 `pixel_values` - 模型的输入：

```
def transforms(examples):
    examples["pixel_values"] = [_transforms(img.convert("RGB")) for img in examples["image"]]
    del examples["image"]
    return examples
```

要使用 🤗 Datasets 的 with_transform 方法在整个数据集上应用预处理函数。当您加载数据集的元素时，变换将动态应用：

```
food = food.with_transform(transforms)
```

现在，使用 DefaultDataCollator 创建一个示例批次。与 🤗 Transformers 中的其他数据整理器不同，`DefaultDataCollator` 不会应用额外的预处理，例如填充。

```
from transformers import DefaultDataCollator
data_collator = DefaultDataCollator()
```

TensorFlow

为了避免过拟合并使模型更强大，请在数据集的训练部分添加一些数据增强。这里我们使用 Keras 预处理层来定义训练数据的变换（包括数据增强），以及验证数据的变换（仅中心裁剪、调整大小和归一化）。您可以使用 `tf.image` 或您喜欢的任何其他库。

```
from tensorflow import keras
from tensorflow.keras import layers
size = (image_processor.size["height"], image_processor.size["width"])
train_data_augmentation = keras.Sequential(
    [
        layers.RandomCrop(size[0], size[1]),
        layers.Rescaling(scale=1.0 / 127.5, offset=-1),
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(factor=0.02),
        layers.RandomZoom(height_factor=0.2, width_factor=0.2),
    ],
    name="train_data_augmentation",
)
val_data_augmentation = keras.Sequential(
    [
        layers.CenterCrop(size[0], size[1]),
        layers.Rescaling(scale=1.0 / 127.5, offset=-1),
    ],
    name="val_data_augmentation",
)
```

接下来，创建函数以对一批图像而不是单个图像应用适当的变换。

```
import numpy as np
import tensorflow as tf
from PIL import Image
def convert_to_tf_tensor(image: Image):
    np_image = np.array(image)
    tf_image = tf.convert_to_tensor(np_image)
    # `expand_dims()` 用于添加一个批次维度，因为
    # TF 增强层对批处理输入进行操作。
    return tf.expand_dims(tf_image, 0)
def preprocess_train(example_batch):
    """Apply train_transforms across a batch."""
    images = [
        train_data_augmentation(convert_to_tf_tensor(image.convert("RGB"))) for image in
example_batch["image"]
    ]
    example_batch["pixel_values"] = [tf.transpose(tf.squeeze(image)) for image in images]
    return example_batch
def preprocess_val(example_batch):
    """Apply val_transforms across a batch."""
    images = [
        val_data_augmentation(convert_to_tf_tensor(image.convert("RGB"))) for image in
example_batch["image"]
```

```
    ]
    example_batch["pixel_values"] = [tf.transpose(tf.squeeze(image)) for image in images]
    return example_batch
```

使用 🤗 Datasets 的 set_transform 方法在加载时应用变换：

```
food["train"].set_transform(preprocess_train)
food["test"].set_transform(preprocess_val)
```

作为预处理步骤的最后一步，使用 `DefaultDataCollator` 创建一个示例批次。与 🤗 Transformers 中的其他数据整理器不同，`DefaultDataCollator` 不会应用额外的预处理，例如填充。

```
from transformers import DefaultDataCollator
data_collator = DefaultDataCollator(return_tensors="tf")
```

评估

在训练过程中包含一个指标通常有助于评估您的模型性能。您可以使用 🤗 Evaluate 库快速加载评估方法。对于此任务，加载 accuracy 指标（请参阅 🤗 Evaluate 的 快速入门 以了解更多关于如何加载和计算指标的信息）：

```
import evaluate
accuracy = evaluate.load("accuracy")
```

然后创建一个函数，将您的预测和标签传递给 compute 以计算准确率：

```
import numpy as np
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return accuracy.compute(predictions=predictions, references=labels)
```

您的 `compute_metrics` 函数现在准备好了，当您设置训练时，您将返回到它。

训练

Pytorch

如果您不熟悉使用 Trainer 微调模型，请查看 这里 的基本教程！

您现在可以开始训练您的模型了！使用 AutoModelForImageClassification 加载 ViT。指定标签数量以及预期的标签数量和标签映射：

```
from transformers import AutoModelForImageClassification, TrainingArguments, Trainer
model = AutoModelForImageClassification.from_pretrained(
    checkpoint,
```

```
    num_labels=len(labels),
    id2label=id2label,
    label2id=label2id,
)
```

在这一点上，只剩下三个步骤：

173. 在 TrainingArguments 中定义您的训练超参数。重要的是您不要删除未使用的列，因为这会删除 `image` 列。如果没有 `image` 列，您就无法创建 `pixel_values`。设置 `remove_unused_columns=False` 以防止这种行为！唯一其他必需的参数是 `output_dir`，它指定保存模型的位置。您将通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传模型）。在每个 epoch 结束时，Trainer 将评估准确率并保存训练检查点。
174. 将训练参数传递给 Trainer，以及模型、数据集、标记器、数据整理器和 `compute_metrics` 函数。
175. 调用 train() 以微调您的模型。

```
training_args = TrainingArguments(
    output_dir="my_awesome_food_model",
    remove_unused_columns=False,
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=16,
    gradient_accumulation_steps=4,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    warmup_ratio=0.1,
    logging_steps=10,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    push_to_hub=True,
)
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=food["train"],
    eval_dataset=food["test"],
    processing_class=image_processor,
    compute_metrics=compute_metrics,
)
trainer.train()
```

训练完成后，使用 push_to_hub() 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

TensorFlow

如果您不熟悉使用 Keras 微调模型，请先查看 基本教程！

要在 TensorFlow 中微调模型，请按照以下步骤操作：

176. 定义训练超参数，并设置优化器和学习率计划。

177. 实例化预训练模型。

178. 将 🤗 Dataset 转换为 `tf.data.Dataset`。

179. 编译您的模型。

180. 添加回调并使用 `fit()` 方法运行训练。

181. 将您的模型上传到 🤗 Hub 以与社区分享。

首先定义超参数、优化器和学习率计划：

```
from transformers import create_optimizer
batch_size = 16
num_epochs = 5
num_train_steps = len(food["train"]) * num_epochs
learning_rate = 3e-5
weight_decay_rate = 0.01
optimizer, lr_schedule = create_optimizer(
    init_lr=learning_rate,
    num_train_steps=num_train_steps,
    weight_decay_rate=weight_decay_rate,
    num_warmup_steps=0,
)
```

然后，使用 TFAutoModelForImageClassification 加载 ViT 以及标签映射：

```
from transformers import TFAutoModelForImageClassification
model = TFAutoModelForImageClassification.from_pretrained(
    checkpoint,
    id2label=id2label,
    label2id=label2id,
)
```

使用 to_tf_dataset 和您的 `data_collator` 将您的数据集转换为 `tf.data.Dataset` 格式：

```
# 将我们的训练数据集转换为 tf.data.Dataset
tf_train_dataset = food["train"].to_tf_dataset(
    columns="pixel_values", label_cols="label", shuffle=True, batch_size=batch_size,
collate_fn=data_collator
)
# 将我们的测试数据集转换为 tf.data.Dataset
tf_eval_dataset = food["test"].to_tf_dataset(
```

```
    columns="pixel_values", label_cols="label", shuffle=True, batch_size=batch_size,
collate_fn=data_collator
)
```

使用 `compile()` 配置模型进行训练：

```
from tensorflow.keras.losses import SparseCategoricalCrossentropy
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(optimizer=optimizer, loss=loss)
```

要计算准确率并将模型推送到 🤗 Hub，请使用 Keras 回调。将您的 `compute_metrics` 函数传递给 KerasMetricCallback，并使用 PushToHubCallback 上传模型：

```
from transformers.keras_callbacks import KerasMetricCallback, PushToHubCallback
metric_callback = KerasMetricCallback(metric_fn=compute_metrics, eval_dataset=tf_eval_dataset)
push_to_hub_callback = PushToHubCallback(
    output_dir="food_classifier",
    tokenizer=image_processor,
    save_strategy="no",
)
callbacks = [metric_callback, push_to_hub_callback]
```

最后，您现在可以训练您的模型了！调用 `fit()` 并传入您的训练和验证数据集、epoch 数量以及回调以微调模型：

```
model.fit(tf_train_dataset, validation_data=tf_eval_dataset, epochs=num_epochs,
callbacks=callbacks)
```

恭喜！您已经微调了您的模型并将其分享在 🤗 Hub 上。现在，您可以使用它进行推理了！

有关如何为图像分类微调模型的更深入示例，请查看相应的 PyTorch 笔记本。

推理

太好了，现在您已经微调了一个模型，您可以使用它进行推理了！

加载您想要运行推理的图像：

```
ds = load_dataset("food101", split="validation[:10]")
image = ds["image"][0]
```



image of beignets

尝试使用微调模型进行推理的最简单方法是将其用于 `pipeline()`。为图像分类实例化一个 `pipeline` 并传入您的图像：

```
from transformers import pipeline
classifier = pipeline("image-classification", model="my_awesome_food_model")
classifier(image)
```

如果您愿意，您也可以手动复制 `pipeline` 的结果：

Pytorch

加载图像处理器以预处理图像并返回 PyTorch 张量的 `input`：

```
from transformers import AutoImageProcessor
import torch
image_processor = AutoImageProcessor.from_pretrained("my_awesome_food_model")
inputs = image_processor(image, return_tensors="pt")
```

将您的输入传递给模型并返回 logits：

```
from transformers import AutoModelForImageClassification
model = AutoModelForImageClassification.from_pretrained("my_awesome_food_model")
with torch.no_grad():
    logits = model(**inputs).logits
```

获取概率最高的预测标签，并使用模型的 `id2label` 映射将其转换为标签：

```
predicted_label = logits.argmax(-1).item()
model.config.id2label[predicted_label]
```

TensorFlow

加载图像处理器以预处理图像并返回 TensorFlow 张量的 `input`：

```
from transformers import AutoImageProcessor
image_processor = AutoImageProcessor.from_pretrained("MariaK/food_classifier")
inputs = image_processor(image, return_tensors="tf")
```

将您的输入传递给模型并返回 logits：

```
from transformers import TFAutoModelForImageClassification
model = TFAutoModelForImageClassification.from_pretrained("MariaK/food_classifier")
logits = model(**inputs).logits
```

获取概率最高的预测标签，并使用模型的 `id2label` 映射将其转换为标签：

```
predicted_class_id = int(tf.math.argmax(logits, axis=-1)[0])
model.config.id2label[predicted_class_id]
```

图像分割

图像分割模型用于将图像中对应不同兴趣区域的部分进行分离。这些模型通过为每个像素分配一个标签来实现。分割类型包括语义分割、实例分割和全景分割。

在本指南中，我们将：

182. 查看不同类型的分割。

183. 提供一个端到端的语义分割微调示例。

在开始之前，请确保您已安装所有必要的库：

```
pip install -q datasets transformers evaluate accelerate
```

我们鼓励您登录您的 Hugging Face 账户，这样您就可以上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

分割类型

语义分割为图像中的每个像素分配一个标签或类别。让我们看一下语义分割模型的输出。它将为图像中遇到的对象的每个实例分配相同的类别，例如，所有猫都将被标记为“猫”，而不是“猫-1”、“猫-2”。我们可以使用 transformers 的图像分割管道快速推断语义分割模型。让我们看一下示例图像。

```
from transformers import pipeline
from PIL import Image
import requests

url = "https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/transformers/tasks/segmentation_input.jpg"
image = Image.open(requests.get(url, stream=True).raw)
image
```

我们将使用 nvidia/segformer-b1-finetuned-cityscapes-1024-1024。

```
semantic_segmentation = pipeline("image-segmentation", "nvidia/segformer-b1-finetuned-
cityscapes-1024-1024")
results = semantic_segmentation(image)
results
```

分割管道输出包括每个预测类别的掩码。

查看汽车类别的掩码，我们可以看到每辆汽车都被分类为相同的掩码。

```
results[-1]["mask"]
```



语义分割输出

在实例分割中，目标不是对每个像素进行分类，而是预测给定图像中对象的每个实例的掩码。它的工作原理与目标检测非常相似，目标检测中每个实例都有一个边界框，而实例分割中则是一个分割掩码。我们将使用 facebook/mask2former-swin-large-cityscapes-instance 进行此操作。

```
instance_segmentation = pipeline("image-segmentation", "facebook/mask2former-swin-large-cityscapes-instance")
results = instance_segmentation(image)
results
```

如您所见，这里有多辆汽车被分类，并且除了属于汽车和人实例的像素外，没有对其他像素进行分类。

查看其中一个汽车掩码如下。

```
results[2]["mask"]
```

〔图：实例分割输出〕

（原始媒体：../../resources/images/instance_segmentation_output.png，离线电子书未嵌入，详见原仓库）

全景分割结合了语义分割和实例分割，每个像素都被分类为一个类别和该类别的实例，并且每个类别的每个实例都有多个掩码。我们可以使用 facebook/mask2former-swin-large-cityscapes-panoptic 进行此操作。

```
panoptic_segmentation = pipeline("image-segmentation", "facebook/mask2former-swin-large-cityscapes-panoptic")
results = panoptic_segmentation(image)
results
```

如您所见，我们有更多的类别。稍后我们将说明每个像素都被分类为其中一个类别。

让我们对所有类型的分割进行并排比较。



分割图比较

看到所有类型的分割后，让我们深入了解为语义分割微调模型。

语义分割的常见现实应用包括训练自动驾驶汽车以识别行人和重要交通信息、在医学图像中识别细胞和异常情况，以及从卫星图像中监测环境变化。

为分割微调模型

我们现在将：

184. 在 SceneParse150 数据集上微调 SegFormer。

185. 使用您微调的模型进行推理。

要查看与此任务兼容的所有架构和检查点，我们建议查看 任务页面。

加载 SceneParse150 数据集

首先从 🤗 Datasets 库中加载 SceneParse150 数据集的一个较小的子集。这将让您有机会进行实验并确保一切正常工作，然后再在完整数据集上花费更多时间进行训练。

```
from datasets import load_dataset

ds = load_dataset("scene_parse_150", split="train[:50]")
```

使用 train_test_split 方法将数据集的 `train` 分割成训练集和测试集：

```
ds = ds.train_test_split(test_size=0.2)
train_ds = ds["train"]
test_ds = ds["test"]
```

然后查看一个示例：

```
train_ds[0]

# 查看图像
train_ds[0]["image"]
```

- `image`：场景的 PIL 图像。
- `annotation`：分割图的 PIL 图像，这也是模型的目标。
- `scene_category`：描述图像场景的类别 ID，如“厨房”或“办公室”。在本指南中，您只需要 `image` 和 `annotation`，它们都是 PIL 图像。

您还需要创建一个字典，将标签 ID 映射到标签类，这在稍后设置模型时将非常有用。从 Hub 下载映射并创建 `id2label` 和 `label2id` 字典：

```
import json
from pathlib import Path
from huggingface_hub import hf_hub_download

repo_id = "huggingface/label-files"
filename = "ade20k-id2label.json"
id2label = json.loads(Path(hf_hub_download(repo_id, filename, repo_type="dataset")).read_text())
id2label = {int(k): v for k, v in id2label.items()}
label2id = {v: k for k, v in id2label.items()}
num_labels = len(id2label)
```

自定义数据集

如果您愿意，也可以创建并使用您自己的数据集。如果您想使用 run_semantic_segmentation.py 脚本而不是笔记本实例进行训练，可以按照以下步骤操作。该脚本需要：

186. 一个包含两个 Image 列的 DatasetDict，即“image”和“label”。

```
from datasets import Dataset, DatasetDict, Image

image_paths_train = ["path/to/image_1.jpg", "path/to/image_2.jpg", ..., "path/to/image_n.jpg"]
label_paths_train = ["path/to/annotation_1.png", "path/to/annotation_2.png", ...,
"path/to/annotation_n.png"]

image_paths_validation = [...]
label_paths_validation = [...]

def create_dataset(image_paths, label_paths):
    dataset = Dataset.from_dict({"image": sorted(image_paths),
                                 "label": sorted(label_paths)})
    dataset = dataset.cast_column("image", Image())
    dataset = dataset.cast_column("label", Image())
    return dataset

# 步骤 1: 创建 Dataset 对象
train_dataset = create_dataset(image_paths_train, label_paths_train)
validation_dataset = create_dataset(image_paths_validation, label_paths_validation)

# 步骤 2: 创建 DatasetDict
dataset = DatasetDict({
    "train": train_dataset,
    "validation": validation_dataset,
    }
)

# 步骤 3: 推送到 Hub (假设您已在终端/笔记本中运行了 huggingface-cli login 命令)
dataset.push_to_hub("your-name/dataset-repo")
```

```
# 可选地，您可以将模型推送到 Hub 上的私有仓库
# dataset.push_to_hub("name of repo on the hub", private=True)
```

187. 一个将类整数映射到类名的 id2label 字典。

```
import json
# 简单示例
id2label = {0: 'cat', 1: 'dog'}
with open('id2label.json', 'w') as fp:
    json.dump(id2label, fp)
```

例如，查看这个使用上述步骤创建的 示例数据集，它包含了创建步骤。

预处理

下一步是加载 SegFormer 图像处理器来准备模型所需的图像和注释。一些数据集，如本例中的数据集，使用零索引作为背景类。然而，背景类实际上并不包含在 150 个类中，因此您需要设置 `do_reduce_labels=True` 以从所有标签中减去一。零索引被替换为 `255`，这样 SegFormer 的损失函数就会忽略它：

```
from transformers import AutoImageProcessor

checkpoint = "nvidia/mit-b0"
image_processor = AutoImageProcessor.from_pretrained(checkpoint, do_reduce_labels=True)
```

PyTorch

通常对图像数据集应用一些数据增强，以使模型更能抵抗过拟合。在本指南中，您将使用 torchvision 的 `ColorJitter` 函数随机更改图像的颜色属性，但您也可以使用您喜欢的任何图像库。

```
from torchvision.transforms import ColorJitter

jitter = ColorJitter(brightness=0.25, contrast=0.25, saturation=0.25, hue=0.1)
```

现在创建两个预处理函数来准备模型所需的图像和注释。这些函数将图像转换为 `pixel_values`，并将注释转换为 `labels`。对于训练集，`jitter` 在提供图像给图像处理器之前应用。对于测试集，图像处理器只裁剪和规范化 `images`，并且只裁剪 `labels`，因为在测试期间不应用数据增强。

```
def train_transforms(example_batch):
    images = [jitter(x) for x in example_batch["image"]]
    labels = [x for x in example_batch["annotation"]]
    inputs = image_processor(images, labels)
    return inputs

def val_transforms(example_batch):
    images = [x for x in example_batch["image"]]
```

```
    labels = [x for x in example_batch["annotation"]]
    inputs = image_processor(images, labels)
    return inputs
```

要在整个数据集上应用 `jitter`，使用 🤗 Datasets 的 set_transform 函数。转换是即时应用的，这更快，并且消耗更少的磁盘空间：

```
train_ds.set_transform(train_transforms)
test_ds.set_transform(val_transforms)
```

评估

在训练过程中包含一个指标通常有助于评估您的模型性能。您可以使用 🤗 Evaluate 库快速加载一个评估方法。对于此任务，加载 平均交并比 (IoU) 指标（参阅 🤗 Evaluate 快速入门 以了解更多关于如何加载和计算指标的信息）：

```
import evaluate

metric = evaluate.load("mean_iou")
```

然后创建一个函数来 计算 指标。您的预测需要先转换为 logits，然后重塑以匹配标签的大小，然后您才能调用 compute：

```
import numpy as np
import torch
from torch import nn

def compute_metrics(eval_pred):
    with torch.no_grad():
        logits, labels = eval_pred
        logits_tensor = torch.from_numpy(logits)
        logits_tensor = nn.functional.interpolate(
            logits_tensor,
            size=labels.shape[-2:],
            mode="bilinear",
            align_corners=False,
        ).argmax(dim=1)

        pred_labels = logits_tensor.detach().cpu().numpy()
        metrics = metric.compute(
            predictions=pred_labels,
            references=labels,
            num_labels=num_labels,
            ignore_index=255,
            reduce_labels=False,
        )
        for key, value in metrics.items():
            if isinstance(value, np.ndarray):
```

```
                metrics[key] = value.tolist()
        return metrics
```

训练

PyTorch

如果您不熟悉使用 Trainer 微调模型，请查看 这里 的基本教程！

您现在可以开始训练您的模型了！使用 AutoModelForSemanticSegmentation 加载 SegFormer，并将模型传递给标签 ID 和标签类之间的映射：

```
from transformers import AutoModelForSemanticSegmentation, TrainingArguments, Trainer

model = AutoModelForSemanticSegmentation.from_pretrained(checkpoint, id2label=id2label, label2id=label2id)
```

在这一点上，只剩下三个步骤：

188. 在 TrainingArguments 中定义您的训练超参数。重要的是您不要删除未使用的列，因为这会删除 `image` 列。如果没有 `image` 列，您就不能创建 `pixel_values`。设置 `remove_unused_columns=False` 以防止这种行为！唯一其他必需的参数是 `output_dir`，它指定保存模型的位置。您将通过设置 `push_to_hub=True` 将模型推送到 Hub（您需要登录 Hugging Face 才能上传您的模型）。在每个 epoch 结束时，Trainer 将评估 IoU 指标并保存训练检查点。
189. 将训练参数传递给 Trainer，以及模型、数据集、tokenizer、数据整理器和 `compute_metrics` 函数。
190. 调用 train() 来微调您的模型。

```
training_args = TrainingArguments(
    output_dir="segformer-b0-scene-parse-150",
    learning_rate=6e-5,
    num_train_epochs=50,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    save_total_limit=3,
    eval_strategy="steps",
    save_strategy="steps",
    save_steps=20,
    eval_steps=20,
    logging_steps=1,
    eval_accumulation_steps=5,
    remove_unused_columns=False,
```

```
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=test_ds,
    compute_metrics=compute_metrics,
)

trainer.train()
```

训练完成后，使用 push_to_hub() 方法将您的模型分享到 Hub，以便每个人都可以使用您的模型：

```
trainer.push_to_hub()
```

推理

太好了，现在您已经微调了一个模型，您可以使用它进行推理了！

重新加载数据集并加载一个图像进行推理。

```
from datasets import load_dataset

ds = load_dataset("scene_parse_150", split="train[:50]")
ds = ds.train_test_split(test_size=0.2)
test_ds = ds["test"]
image = ds["test"][0]["image"]
image
```



卧室图像

PyTorch

我们现在将看到如何在没有管道的情况下进行推理。使用图像处理器处理图像，并将 `pixel_values` 放在 GPU 上：

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  # 如果可用，使用 GPU，否则使用 CPU
encoding = image_processor(image, return_tensors="pt")
pixel_values = encoding.pixel_values.to(device)
```

将您的输入传递给模型并返回 `logits`：

```
outputs = model(pixel_values=pixel_values)
logits = outputs.logits.cpu()
```

接下来，将 logits 缩放到原始图像大小：

```
upsampled_logits = nn.functional.interpolate(
    logits,
    size=image.size[::-1],
    mode="bilinear",
    align_corners=False,
)

pred_seg = upsampled_logits.argmax(dim=1)[0]
```

为了可视化结果，加载 数据集颜色调色板 作为 `ade_palette()`，它将每个类映射到它们的 RGB 值。

```
def ade_palette():
    return np.asarray([
        [0, 0, 0],
        [120, 120, 120],
        [180, 120, 120],
        [6, 230, 230],
        [80, 50, 50],
        [4, 200, 3],
        [120, 120, 80],
        [140, 140, 140],
        [204, 5, 255],
        [230, 230, 230],
        [4, 250, 7],
        [224, 5, 255],
        [235, 255, 7],
        [150, 5, 61],
        [120, 120, 70],
        [8, 255, 51],
```

```
        [255, 6, 82],
        [143, 255, 140],
        [204, 255, 4],
        [255, 51, 7],
        [204, 70, 3],
        [0, 102, 200],
        [61, 230, 250],
        [255, 6, 51],
        [11, 102, 255],
        [255, 7, 71],
        [255, 9, 224],
        [9, 7, 230],
        [220, 220, 220],
        [255, 9, 92],
        [112, 9, 255],
        [8, 255, 214],
        [7, 255, 224],
        [255, 184, 6],
        [10, 255, 71],
        [255, 41, 10],
        [7, 255, 255],
        [224, 255, 8],
        [102, 8, 255],
        [255, 61, 6],
        [255, 194, 7],
        [255, 122, 8],
        [0, 255, 20],
        [255, 8, 41],
        [255, 5, 153],
        [6, 51, 255],
        [235, 12, 255],
        [160, 150, 20],
        [0, 163, 255],
        [140, 140, 140],
        [250, 10, 15],
        [20, 255, 0],
        [31, 255, 0],
        [255, 31, 0],
        [255, 224, 0],
        [153, 255, 0],
        [0, 0, 255],
        [255, 71, 0],
        [0, 235, 255],
        [0, 173, 255],
        [31, 0, 255],
        [11, 200, 200],
        [255, 82, 0],
        [0, 255, 245],
        [0, 61, 255],
        [0, 255, 112],
        [0, 255, 133],
```

```
        [255, 0, 0],
        [255, 163, 0],
        [255, 102, 0],
        [194, 255, 0],
        [0, 143, 255],
        [51, 255, 0],
        [0, 82, 255],
        [0, 255, 41],
        [0, 255, 173],
        [10, 0, 255],
        [173, 255, 0],
        [0, 255, 153],
        [255, 92, 0],
        [255, 0, 255],
        [255, 0, 245],
        [255, 0, 102],
        [255, 173, 0],
        [255, 0, 20],
        [255, 184, 184],
        [0, 31, 255],
        [0, 255, 61],
        [0, 71, 255],
        [255, 0, 204],
        [0, 255, 194],
        [0, 255, 82],
        [0, 10, 255],
        [0, 112, 255],
        [51, 0, 255],
        [0, 194, 255],
        [0, 122, 255],
        [0, 255, 163],
        [255, 153, 0],
        [0, 255, 10],
        [255, 112, 0],
        [143, 255, 0],
        [82, 0, 255],
        [163, 255, 0],
        [255, 235, 0],
        [8, 184, 170],
        [133, 0, 255],
        [0, 255, 92],
        [184, 0, 255],
        [255, 0, 31],
        [0, 184, 255],
        [0, 214, 255],
        [255, 0, 112],
        [92, 255, 0],
        [0, 224, 255],
        [112, 224, 255],
        [70, 184, 160],
        [163, 0, 255],
```

```
        [153, 0, 255],
        [71, 255, 0],
        [255, 0, 163],
        [255, 204, 0],
        [255, 0, 143],
        [0, 255, 235],
        [133, 255, 0],
        [255, 0, 235],
        [245, 0, 255],
        [255, 0, 122],
        [255, 245, 0],
        [10, 190, 212],
        [214, 255, 0],
        [0, 204, 255],
        [20, 0, 255],
        [255, 255, 0],
        [0, 153, 255],
        [0, 41, 255],
        [0, 255, 204],
        [41, 0, 255],
        [41, 255, 0],
        [173, 0, 255],
        [0, 245, 255],
        [71, 0, 255],
        [122, 0, 255],
        [0, 255, 184],
        [0, 92, 255],
        [184, 255, 0],
        [0, 133, 255],
        [255, 214, 0],
        [25, 194, 194],
        [102, 255, 0],
        [92, 0, 255],
    ])
```

然后，您可以组合并绘制您的图像和预测的分割图：

```
import matplotlib.pyplot as plt
import numpy as np

color_seg = np.zeros((pred_seg.shape[0], pred_seg.shape[1], 3), dtype=np.uint8)
palette = np.array(ade_palette())
for label, color in enumerate(palette):
    color_seg[pred_seg == label, :] = color
color_seg = color_seg[..., ::-1]  # 转换为 BGR

img = np.array(image) * 0.5 + color_seg * 0.5  # 绘制带有分割图的图像
img = img.astype(np.uint8)

plt.figure(figsize=(15, 10))
```

```
plt.imshow(img)
plt.show()
```



带有分割图的卧室图像

恭喜！您已经成功地微调了一个图像分割模型，并使用它进行了推理。您现在可以将这个模型用于各种图像分割任务，如自动驾驶、医学图像分析和环境监测。

视频分类

视频分类是指为整个视频分配一个标签或类别。每个视频通常只属于一个类别。视频分类模型接收视频作为输入，并返回该视频所属类别的预测结果。这些模型可以用于对视频内容进行分类。视频分类的一个实际应用是动作/活动识别，这在健身应用程序中非常有用。对于视障人士，尤其是在出行时，视频分类也是很有帮助的。

本指南将向你展示如何：

191. 在 UCF101 数据集的子集上微调 VideoMAE。
192. 使用微调后的模型进行推理。

要查看与本任务兼容的所有架构和检查点，我们推荐查看 任务页面。

在开始之前，请确保你已经安装了所有必要的库：

```
pip install -q pytorchvideo transformers evaluate
```

你将使用 PyTorchVideo（简称 `pytorchvideo`）来处理和准备视频。

我们建议你登录到你的 Hugging Face 账户，以便可以上传和分享你的模型。当提示输入令牌时，请输入你的令牌以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 UCF101 数据集

首先加载 UCF-101 数据集的子集。这将让你有机会进行实验并确保一切正常运行，然后再花费更多时间在完整数据集上训练。

```
from huggingface_hub import hf_hub_download

hf_dataset_identifier = "sayakpaul/ucf101-subset"
filename = "UCF101_subset.tar.gz"
file_path = hf_hub_download(repo_id=hf_dataset_identifier, filename=filename,
repo_type="dataset")
```

下载子集后，需要解压压缩包：

```
import tarfile

with tarfile.open(file_path) as t:
    t.extractall(".")
```

数据集的组织结构如下：

然后你可以统计总视频数量：

```
import pathlib

dataset_root_path = "UCF101_subset"
dataset_root_path = pathlib.Path(dataset_root_path)

video_count_train = len(list(dataset_root_path.glob("train/*/ *.avi")))
video_count_val = len(list(dataset_root_path.glob("val/*/ *.avi")))
video_count_test = len(list(dataset_root_path.glob("test/*/ *.avi")))
video_total = video_count_train + video_count_val + video_count_test
print(f"总视频数: {video_total}")

all_video_file_paths = (
    list(dataset_root_path.glob("train/*/ *.avi"))
    + list(dataset_root_path.glob("val/*/ *.avi"))
    + list(dataset_root_path.glob("test/*/ *.avi"))
)
all_video_file_paths[:5]
```

（排序后的）视频路径如下所示：

你会注意到，有些视频片段属于同一个组/场景，组由视频文件路径中的 `g` 表示。例如，`v_ApplyEyeMakeup_g07_c04.avi` 和 `v_ApplyEyeMakeup_g07_c06.avi` 属于同一组。

对于验证和评估拆分，你不希望有来自同一组/场景的视频片段，以防止数据泄露。本教程中使用的子集考虑了这一点。

接下来，你将从数据集中提取标签集。同时，创建两个字典，有助于初始化模型：

- `label2id`：将类别名称映射到整数。
- `id2label`：将整数映射到类别名称。

```
class_labels = sorted({str(path).split("/")[2] for path in all_video_file_paths})
label2id = {label: i for i, label in enumerate(class_labels)}
id2label = {i: label for label, i in label2id.items()}

print(f"唯一类别: {list(label2id.keys())}.")
# 唯一类别: ['ApplyEyeMakeup', 'ApplyLipstick', 'Archery', 'BabyCrawling', 'BalanceBeam',
'BandMarching', 'BaseballPitch', 'Basketball', 'BasketballDunk', 'BenchPress'].
```

共有 10 个唯一类别。每个类别在训练集中有 30 个视频。

加载模型进行微调

从预训练检查点实例化视频分类模型及其关联的图像处理器。模型的编码器带有预训练参数，而分类头则是随机初始化的。图像处理器将在编写数据集的预处理管道时派上用场。

```
from transformers import VideoMAEImageProcessor, VideoMAEForVideoClassification

model_ckpt = "MCG-NJU/videomae-base"
image_processor = VideoMAEImageProcessor.from_pretrained(model_ckpt)
model = VideoMAEForVideoClassification.from_pretrained(
    model_ckpt,
    label2id=label2id,
    id2label=id2label,
    ignore_mismatched_sizes=True,  # 如果你计划微调一个已经微调过的检查点，提供此参数
)
```

在加载模型时，你可能会注意到以下警告：

警告告诉我们，我们在丢弃一些权重（例如 `classifier` 层的权重和偏置），并随机初始化一些其他权重（新的 `classifier` 层的权重和偏置）。这是预期的行为，因为我们在添加一个新的头部，而没

有预训练的权重，因此库提醒我们需要在使用模型进行推理之前对其进行微调，而这正是我们将要做的。

注意：此检查点在类似下游任务上进行了微调，具有相当大的领域重叠，因此在本任务上的表现更好。你可以查看 此检查点，它是通过对 `MCG-NJU/videomae-base-finetuned-kinetics` 进行微调获得的。

准备数据集进行训练

为了预处理视频，你将利用 PyTorchVideo 库。首先导入所需的依赖项。

```
import pytorchvideo.data

from pytorchvideo.transforms import (
    ApplyTransformToKey,
    Normalize,
    RandomShortSideScale,
    RemoveKey,
    ShortSideScale,
    UniformTemporalSubsample,
)

from torchvision.transforms import (
    Compose,
    Lambda,
    RandomCrop,
    RandomHorizontalFlip,
    Resize,
)
```

对于训练数据集的变换，使用均匀时间采样、像素归一化、随机裁剪和随机水平翻转的组合。对于验证和评估数据集的变换，使用相同的变换链，但不包括随机裁剪和水平翻转。有关这些变换的详细信息，请参阅 PyTorchVideo 官方文档。

使用与预训练模型关联的 `image_processor` 获取以下信息：

- 用于归一化视频帧像素的图像均值和标准差。
- 视频帧将调整大小的空间分辨率。

首先定义一些常量。

```
mean = image_processor.image_mean
std = image_processor.image_std
if "shortest_edge" in image_processor.size:
```

```
    height = width = image_processor.size["shortest_edge"]
else:
    height = image_processor.size["height"]
    width = image_processor.size["width"]
resize_to = (height, width)

num_frames_to_sample = model.config.num_frames
sample_rate = 4
fps = 30
clip_duration = num_frames_to_sample * sample_rate / fps
```

现在，分别定义数据集特定的变换和数据集。从训练集开始：

```
train_transform = Compose([
    ApplyTransformToKey(
        key="video",
        transform=Compose([
            UniformTemporalSubsample(num_frames_to_sample),
            Lambda(lambda x: x / 255.0),
            Normalize(mean, std),
            RandomShortSideScale(min_size=256, max_size=320),
            RandomCrop(resize_to),
            RandomHorizontalFlip(p=0.5),
        ]),
    ),
])

train_dataset = pytorchvideo.data.Ucf101(
    data_path=os.path.join(dataset_root_path, "train"),
    clip_sampler=pytorchvideo.data.make_clip_sampler("random", clip_duration),
    decode_audio=False,
    transform=train_transform,
)
```

同样的工作流程可以应用于验证和评估集：

```
val_transform = Compose([
    ApplyTransformToKey(
        key="video",
        transform=Compose([
            UniformTemporalSubsample(num_frames_to_sample),
            Lambda(lambda x: x / 255.0),
            Normalize(mean, std),
            Resize(resize_to),
        ]),
    ),
])

val_dataset = pytorchvideo.data.Ucf101(
    data_path=os.path.join(dataset_root_path, "val"),
    clip_sampler=pytorchvideo.data.make_clip_sampler("uniform", clip_duration),
    decode_audio=False,
```

```
    transform=val_transform,
)

test_dataset = pytorchvideo.data.Ucf101(
    data_path=os.path.join(dataset_root_path, "test"),
    clip_sampler=pytorchvideo.data.make_clip_sampler("uniform", clip_duration),
    decode_audio=False,
    transform=val_transform,
)
```

注意：上述数据集管道取自 PyTorchVideo 官方示例。我们使用的是 `[pytorchvideo.data.Ucf101()](https://pytorchvideo.readthedocs.io/en/latest/api/data/data.html#pytorchvideo.data.Ucf101)` 函数，因为它专为 UCF-101 数据集定制。在底层，它返回一个 `[pytorchvideo.data.labeled_video_dataset.LabeledVideoDataset](https://pytorchvideo.readthedocs.io/en/latest/api/data/data.html#pytorchvideo.data.LabeledVideoDataset)` 对象。`LabeledVideoDataset` 类是 PyTorchVideo 数据集中所有视频的基础类。因此，如果你想使用不支持的自定义数据集，可以扩展 `LabeledVideoDataset` 类。更多详情请参考 `data` API 文档。如果你的数据集结构与此类似（如上所示），那么使用 `pytorchvideo.data.Ucf101()` 应该没问题。

你可以通过访问 `num_videos` 参数来了解数据集中的视频数量。

```
print(train_dataset.num_videos, val_dataset.num_videos, test_dataset.num_videos)
# (300, 30, 75)
```

可视化预处理后的视频以便更好地调试

```
import imageio
import numpy as np
from IPython.display import Image

def unnormalize_img(img):
    """反归一化图像像素。"""
    img = (img * std) + mean
    img = (img * 255).astype("uint8")
    return img.clip(0, 255)

def create_gif(video_tensor, filename="sample.gif"):
    """从视频张量中生成 GIF。

    期望视频张量的形状为：
    (num_frames, num_channels, height, width)。
    """
    frames = []
```

```
    for video_frame in video_tensor:
        frame_unnormalized = unnormalize_img(video_frame.permute(1, 2, 0).numpy())
        frames.append(frame_unnormalized)
    kargs = {"duration": 0.25}
    imageio.mimsave(filename, frames, "GIF", **kargs)
    return filename

def display_gif(video_tensor, gif_name="sample.gif"):
    """从视频张量中生成并显示 GIF。"""
    video_tensor = video_tensor.permute(1, 0, 2, 3)
    gif_filename = create_gif(video_tensor, gif_name)
    return Image(filename=gif_filename)

sample_video = next(iter(train_dataset))
video_tensor = sample_video["video"]
display_gif(video_tensor)
```

〔图：打篮球的人〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/sample_gif.gif，离线电子书未嵌入，详见原仓库）

训练模型

使用 🤗 Transformers 的 `Trainer` 来训练模型。要实例化 `Trainer`，你需要定义训练配置和评估指标。最重要的是 `TrainingArguments`，这是一个包含所有属性以配置训练的类。它需要一个输出文件夹名称，该名称将用于保存模型的检查点。它还有助于在 🤗 Hub 上同步模型仓库中的所有信息。

大多数训练参数都是自解释的，但其中一个相当重要的是 `remove_unused_columns=False`。这个参数会删除模型调用函数未使用的所有特征。默认情况下，它为 `True`，因为在大多数情况下，删除未使用的特征列是理想的，使其更容易将输入解包到模型的调用函数中。但在这种情况下，你需要未使用的特征（特别是 `video`）来创建 `pixel_values`（这是模型输入中必需的键）。

```
from transformers import TrainingArguments, Trainer

model_name = model_ckpt.split("/")[-1]
new_model_name = f"{model_name}-finetuned-ucf101-subset"
num_epochs = 4

args = TrainingArguments(
    new_model_name,
    remove_unused_columns=False,
    eval_strategy="epoch",
    save_strategy="epoch",
```

```
    learning_rate=5e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    warmup_ratio=0.1,
    logging_steps=10,
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    push_to_hub=True,
    max_steps=(train_dataset.num_videos // batch_size) * num_epochs,
)
```

`pytorchvideo.data.Ucf101()` 返回的数据集没有实现 `__len__` 方法。因此，我们在实例化 `TrainingArguments` 时必须定义 `max_steps`。

接下来，你需要定义一个函数来计算预测的指标，这将使用你现在加载的 `metric`。唯一的预处理是你需要对预测的 logits 进行 argmax：

```
import evaluate

metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions = np.argmax(eval_pred.predictions, axis=1)
    return metric.compute(predictions=predictions, references=eval_pred.label_ids)
```

关于评估的说明：

在 VideoMAE 论文中，作者使用了以下评估策略。他们在测试视频的多个片段上评估模型，并对这些片段应用不同的裁剪，报告聚合分数。然而，为了简单和简洁，本教程中不考虑这一点。

你还需定义一个 `collate_fn`，用于将样本批处理在一起。每个批次包含两个键，即 `pixel_values` 和 `labels`。

```
def collate_fn(examples):
    # 调整为 (num_frames, num_channels, height, width)
    pixel_values = torch.stack(
        [example["video"].permute(1, 0, 2, 3) for example in examples]
    )
    labels = torch.tensor([example["label"] for example in examples])
    return {"pixel_values": pixel_values, "labels": labels}
```

然后，你只需将所有这些内容以及数据集传递给 `Trainer`：

```
trainer = Trainer(
    model,
    args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
```

```
    processing_class=image_processor,
    compute_metrics=compute_metrics,
    data_collator=collate_fn,
)
```

你可能会想知道为什么在预处理数据时已经传入了 `image_processor`。这只是为了确保图像处理器的配置文件（存储为 JSON）也会上传到 Hub 仓库中。

现在通过调用 `train` 方法微调模型：

```
train_results = trainer.train()
```

训练完成后，使用 `push_to_hub()` 方法将模型分享到 Hub，这样每个人都可以使用你的模型：

```
trainer.push_to_hub()
```

推理

太好了，现在你已经微调了一个模型，可以使用它进行推理！

加载一个视频用于推理：

```
sample_test_video = next(iter(test_dataset))
```

〔图：队伍进行篮球比赛〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/sample_gif_two.gif，离线电子书未嵌入，详见原仓库）

最简单的方法是使用模型进行推理，就是将其放在一个 `pipeline` 中。实例化一个用于视频分类的 `pipeline` 并传递你的视频：

```
from transformers import pipeline

video_cls = pipeline(model="my_awesome_video_cls_model")
video_cls("https://huggingface.co/datasets/sayakpaul/ucf101-subset/resolve/main/v_BasketballDunk_g14_c06.avi")
```

你也可以手动复制 `pipeline` 的结果，如果你愿意的话：

```
def run_inference(model, video):
    # (num_frames, num_channels, height, width)
    perumuted_sample_test_video = video.permute(1, 0, 2, 3)
    inputs = {
        "pixel_values": perumuted_sample_test_video.unsqueeze(0),
        "labels": torch.tensor(
            [sample_test_video["label"]]
        ),  # 如果没有标签，可以跳过
    }
```

```
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    inputs = {k: v.to(device) for k, v in inputs.items()}
    model = model.to(device)

    # 前向传播
    with torch.no_grad():
        outputs = model(**inputs)
        logits = outputs.logits

    return logits

logits = run_inference(trained_model, sample_test_video["video"])
```

解码 `logits`，我们得到：

```
predicted_class_idx = logits.argmax(-1).item()
print("预测类别:", model.config.id2label[predicted_class_idx])
# 预测类别: BasketballDunk
```

目标检测

目标检测是计算机视觉任务中的一种，用于在图像中检测实例（如人、建筑物或汽车）。目标检测模型接收图像作为输入，并输出检测到的目标的边界框坐标和关联标签。一张图像可以包含多个目标，每个目标有自己的边界框和标签（例如，图像中可能同时包含汽车和建筑物），并且每个目标可以在图像的不同位置出现（例如，图像中可能有几辆汽车）。这项任务常用于自动驾驶中，用于检测行人、路标和交通信号灯等。其他应用场景包括图像中的目标计数、图像搜索等。

在这个指南中，你将学习如何：

193. 在 CPPE-5 数据集上微调 DETR，这是一种结合了卷积主干和编码器-解码器 Transformer 的模型。
194. 使用微调后的模型进行推理。

要查看与该任务兼容的所有架构和检查点，建议查阅 任务页面。

在开始之前，请确保已安装所有必要的库：

```
pip install -q datasets transformers accelerate timm
pip install -q -U albumentations>=1.4.5 torchmetrics pycocotools
```

你将使用 🤗 Datasets 加载来自 Hugging Face Hub 的数据集，使用 🤗 Transformers 训练模型，并使用 `albumentations` 增强数据。

我们鼓励你与社区分享你的模型。登录 Hugging Face 账户以将其上传到 Hub。当提示时，输入你的令牌以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

首先，我们定义全局常量，即模型名称和图像大小。在这个教程中，我们将使用条件 DETR 模型，因为它收敛速度更快。你可以选择 `transformers` 库中可用的任何目标检测模型。

```
MODEL_NAME = "microsoft/conditional-detr-resnet-50"  # 或 "facebook/detr-resnet-50"
IMAGE_SIZE = 480
```

加载 CPPE-5 数据集

CPPE-5 数据集包含注释了医疗个人防护装备（PPE）的图像，这些注释是在 COVID-19 大流行期间生成的。

首先加载数据集并从训练集中创建一个验证集：

```
from datasets import load_dataset

cppe5 = load_dataset("cppe-5")

if "validation" not in cppe5:
    split = cppe5["train"].train_test_split(0.15, seed=1337)
    cppe5["train"] = split["train"]
    cppe5["validation"] = split["test"]

cppe5
```

你会发现这个数据集有 1000 张图像用于训练和验证集，以及一个包含 29 张图像的测试集。

为了熟悉数据，探索一下示例的样子。

```
cppe5["train"][0]
```

数据集中的示例包含以下字段：

- `image_id`：示例图像的 ID
- `image`：包含图像的 `PIL.Image.Image` 对象
- `width`：图像的宽度
- `height`：图像的高度
- `objects`：包含图像中目标的边界框元数据的字典：

- `id`：注释 ID
- `area`：边界框的面积
- `bbox`：目标的边界框（采用 COCO 格式）
- `category`：目标的类别，可能的值包括 `Coverall (0)`、`Face_Shield (1)`、`Gloves (2)`、`Goggles (3)` 和 `Mask (4)`

你可能会注意到 `bbox` 字段遵循 COCO 格式，这是 DETR 模型期望的格式。然而，字段在 `objects` 内部的分组与 DETR 要求的注释格式不同。在使用此数据进行训练之前，需要应用一些预处理转换。

为了更好地理解数据，可视化数据集中的一个示例。

```
import numpy as np
import os
from PIL import Image, ImageDraw

image = cppe5["train"][2]["image"]
annotations = cppe5["train"][2]["objects"]
draw = ImageDraw.Draw(image)

categories = cppe5["train"].features["objects"].feature["category"].names

id2label = {index: x for index, x in enumerate(categories, start=0)}
label2id = {v: k for k, v in id2label.items()}

for i in range(len(annotations["id"])):
    box = annotations["bbox"][i]
    class_idx = annotations["category"][i]
    x, y, w, h = tuple(box)
    # 检查坐标是否已归一化
    if max(box) > 1.0:
        # 坐标未归一化，无需重新缩放
        x1, y1 = int(x), int(y)
        x2, y2 = int(x + w), int(y + h)
    else:
        # 坐标已归一化，重新缩放
        x1 = int(x * width)
        y1 = int(y * height)
        x2 = int((x + w) * width)
        y2 = int((y + h) * height)
    draw.rectangle((x, y, x + w, y + h), outline="red", width=1)
    draw.text((x, y), id2label[class_idx], fill="white")

image
```

为了可视化带有关联标签的边界框，你可以从数据集的元数据中获取标签，特别是 `category` 字段。你还希望创建将标签 ID 映射到标签类的字典（`id2label`）和反向映射的字典（`label2id`）。稍后设置模型时可以使用这些映射。包括这些映射可以使你的模型在上传到 Hugging Face Hub 后更具可重用性。请注意，上述代码中绘制边界框的部分假设它们是 COCO 格式 `(x_min, y_min, width, height)`。如果要支持其他格式（如 `(x_min, y_min, x_max, y_max)`），需要进行调整。

作为熟悉数据的最后一步，探索数据中的潜在问题。目标检测数据集中常见的问题是边界框“超出”图像边缘。这种“越界”的边界框在训练过程中可能会引发错误，应该解决。此数据集中有几个这样的示例。为了简化本指南，我们将在下面的转换中设置 `clip=True`。

预处理数据

为了微调模型，必须对计划使用的数据进行预处理，使其完全符合预训练模型的方法。AutoImageProcessor 会处理图像数据，生成 DETR 模型可以训练的 `pixel_values`、`pixel_mask` 和 `labels`。图像处理器有一些属性你不需要担心：

- `image_mean = [0.485, 0.456, 0.406 ]`
- `image_std = [0.229, 0.224, 0.225]`

这些是在模型预训练期间用于归一化图像的均值和标准差。这些值在进行推理或微调预训练图像模型时至关重要。

从与要微调的模型相同的检查点实例化图像处理器。

```
from transformers import AutoImageProcessor

MAX_SIZE = IMAGE_SIZE

image_processor = AutoImageProcessor.from_pretrained(
    MODEL_NAME,
    do_resize=True,
    size={"max_height": MAX_SIZE, "max_width": MAX_SIZE},
    do_pad=True,
    pad_size={"height": MAX_SIZE, "width": MAX_SIZE},
)
```

在将图像传递给 `image_processor` 之前，对数据集应用两个预处理转换：

195. 增强图像

196. 调整注释以满足 DETR 的期望

首先，为了避免模型在训练数据上过拟合，可以使用任何数据增强库对图像进行增强。这里我们使用 Albumentations。这个库确保转换会影响图像并相应地更新边界框。🤗 Datasets 文档中有详细指南介绍如何为对象检测增强图像，并使用了相同的示例数据集。对图像应用一些几何和颜色变换。有关更多增强选项，可以探索 Albumentations 示例空间。

```
import albumentations as A

train_augment_and_transform = A.Compose(
    [
        A.Perspective(p=0.1),
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(p=0.5),
        A.HueSaturationValue(p=0.1),
    ],
    bbox_params=A.BboxParams(format="coco", label_fields=["category"], clip=True, min_area=25),
)

validation_transform = A.Compose(
    [A.NoOp()],
    bbox_params=A.BboxParams(format="coco", label_fields=["category"], clip=True),
)
```

`image_processor` 期望注释的格式如下：`{'image_id': int, 'annotations': List[Dict]}`，其中每个字典是一个 COCO 对象注释。添加一个函数来重新格式化单个示例的注释：

```
def format_image_annotations_as_coco(image_id, categories, areas, bboxes):
    """将一组图像注释格式化为 COCO 格式

    参数:
        image_id (str): 图像 ID。例如: "0001"
        categories (List[int]): 与提供的边界框对应的类别/类标签列表
        areas (List[float]): 与提供的边界框对应的面积列表
        bboxes (List[Tuple[float]]): 以 COCO 格式提供的边界框列表
            ([center_x, center_y, width, height] 为绝对坐标)

    返回:
        dict: {
            "image_id": 图像 ID,
            "annotations": 格式化的注释列表
        }
    """
    annotations = []
    for category, area, bbox in zip(categories, areas, bboxes):
```

```
        formatted_annotation = {
            "image_id": image_id,
            "category_id": category,
            "iscrowd": 0,
            "area": area,
            "bbox": list(bbox),
        }
        annotations.append(formatted_annotation)

    return {
        "image_id": image_id,
        "annotations": annotations,
    }
```

现在可以组合图像和注释变换，以便在一批示例上使用：

```
def augment_and_transform_batch(examples, transform, image_processor, return_pixel_mask=False):
    """对对象检测任务应用增强并格式化注释为 COCO 格式"""

    images = []
    annotations = []
    for image_id, image, objects in zip(examples["image_id"], examples["image"],
examples["objects"]):
        image = np.array(image.convert("RGB"))

        # 应用增强
        output = transform(image=image, bboxes=objects["bbox"], category=objects["category"])
        images.append(output["image"])

        # 将注释格式化为 COCO 格式
        formatted_annotations = format_image_annotations_as_coco(
            image_id, output["category"], objects["area"], output["bboxes"]
        )
        annotations.append(formatted_annotations)

    # 应用图像处理器变换：调整大小、重新缩放、归一化
    result = image_processor(images=images, annotations=annotations, return_tensors="pt")

    if not return_pixel_mask:
        result.pop("pixel_mask", None)

    return result
```

使用 🤗 Datasets with_transform 方法将此预处理函数应用于整个数据集。此方法会在加载数据集元素时动态应用转换。

此时，可以检查转换后的数据集示例。你应该看到一个包含 `pixel_values` 的张量、一个包含 `pixel_mask` 的张量和 `labels`。

```
from functools import partial

# 为批处理创建转换函数并应用于数据集切片
train_transform_batch = partial(
    augment_and_transform_batch, transform=train_augment_and_transform,
image_processor=image_processor
)
validation_transform_batch = partial(
    augment_and_transform_batch, transform=validation_transform,
image_processor=image_processor
)

cppe5["train"] = cppe5["train"].with_transform(train_transform_batch)
cppe5["validation"] = cppe5["validation"].with_transform(validation_transform_batch)
cppe5["test"] = cppe5["test"].with_transform(validation_transform_batch)

cppe5["train"][15]
```

你已经成功增强了单个图像并准备了它们的注释。但是，预处理尚未完成。在最后一步中，创建一个自定义 `collate_fn` 来批量处理图像。将图像（现为 `pixel_values`）填充到批次中最大的图像，并创建相应的 `pixel_mask` 以指示哪些像素是真实的（1），哪些是填充的（0）。

```
import torch

def collate_fn(batch):
    data = {}
    data["pixel_values"] = torch.stack([x["pixel_values"] for x in batch])
    data["labels"] = [x["labels"] for x in batch]
    if "pixel_mask" in batch[0]:
        data["pixel_mask"] = torch.stack([x["pixel_mask"] for x in batch])
    return data
```

准备计算 mAP 的函数

对象检测模型通常使用一组 COCO 风格的指标 进行评估。我们将使用 `torchmetrics` 计算 `mAP`（平均精度）和 `mAR`（平均召回率）指标，并将其包装到 `compute_metrics` 函数中，以便在 Trainer 中进行评估。

用于训练的中间格式的框是 `YOLO`（归一化），但我们将计算 `Pascal VOC`（绝对坐标）格式的框，以正确处理框的面积。定义一个函数将边界框转换为 `Pascal VOC` 格式：

```
from transformers.image_transforms import center_to_corners_format

def convert_bbox_yolo_to_pascal(boxes, image_size):
    """
    将边界框从 YOLO 格式 (x_center, y_center, width, height) 转换为 Pascal VOC 格式 (x_min, y_min,
```

```
x_max, y_max)，并在绝对坐标中表示。

    参数:

        boxes (torch.Tensor): 以 YOLO 格式的边界框

        image_size (Tuple[int, int]): 图像尺寸，格式为 (高度, 宽度)

    返回:

        torch.Tensor: 以 Pascal VOC 格式 (x_min, y_min, x_max, y_max) 的边界框
    """
    # 转换为中心到角点格式
    boxes = center_to_corners_format(boxes)

    # 转换为绝对坐标
    height, width = image_size
    boxes = boxes * torch.tensor([[width, height, width, height]])

    return boxes
```

然后，在 `compute_metrics` 函数中收集评估循环结果中的预测和目标边界框、得分和标签，并传递给评分函数。

```
import numpy as np
from dataclasses import dataclass
from torchmetrics.detection.mean_ap import MeanAveragePrecision

@dataclass
class ModelOutput:
    logits: torch.Tensor
    pred_boxes: torch.Tensor

@torch.no_grad()
def compute_metrics(evaluation_results, image_processor, threshold=0.0, id2label=None):
    """
    计算对象检测任务的平均 mAP、mAR 及其变体。

    参数:

        evaluation_results (EvalPrediction): 评估的预测和目标。

        threshold (float, optional): 通过置信度筛选预测框的阈值。默认为 0.0。

        id2label (Optional[dict], optional): 类别 ID 到类别名称的映射。默认为 None。

    返回:

        Mapping[str, float]: 以字典形式表示的指标 {<metric_name>: <metric_value>}
    """
```

```
    predictions, targets = evaluation_results.predictions, evaluation_results.label_ids

    # 为指标计算提供：
    #  - 目标：列表中的字典，键为 "boxes" 和 "labels"
    #  - 预测：列表中的字典，键为 "boxes"、"scores" 和 "labels"

    image_sizes = []
    post_processed_targets = []
    post_processed_predictions = []

    # 收集用于指标计算的目标
    for batch in targets:
        # 收集图像尺寸，用于预测后处理
        batch_image_sizes = torch.tensor(np.array([x["orig_size"] for x in batch]))
        image_sizes.append(batch_image_sizes)
        # 收集目标，用于指标计算
        # 框已被转换为 YOLO 格式，用于模型训练
        # 这里将它们转换为 Pascal VOC 格式 (x_min, y_min, x_max, y_max)
        for image_target in batch:
            boxes = torch.tensor(image_target["boxes"])
            boxes = convert_bbox_yolo_to_pascal(boxes, image_target["orig_size"])
            labels = torch.tensor(image_target["class_labels"])
            post_processed_targets.append({"boxes": boxes, "labels": labels})

    # 收集用于指标计算的预测，
    # 模型生成的框为 YOLO 格式，然后图像处理器将其转换为 Pascal VOC 格式
    for batch, target_sizes in zip(predictions, image_sizes):
        batch_logits, batch_boxes = batch[1], batch[2]
        output = ModelOutput(logits=torch.tensor(batch_logits),
pred_boxes=torch.tensor(batch_boxes))
        post_processed_output = image_processor.post_process_object_detection(
            output, threshold=threshold, target_sizes=target_sizes
        )
        post_processed_predictions.extend(post_processed_output)

    # 计算指标
    metric = MeanAveragePrecision(box_format="xyxy", class_metrics=True)
    metric.update(post_processed_predictions, post_processed_targets)
    metrics = metric.compute()

    # 将每类指标的列表替换为每类的单独指标
    classes = metrics.pop("classes")
    map_per_class = metrics.pop("map_per_class")
    mar_100_per_class = metrics.pop("mar_100_per_class")
    for class_id, class_map, class_mar in zip(classes, map_per_class, mar_100_per_class):
        class_name = id2label[class_id.item()] if id2label is not None else class_id.item()
```

```
        metrics[f"map_{class_name}"] = class_map
        metrics[f"mar_100_{class_name}"] = class_mar

    metrics = {k: round(v.item(), 4) for k, v in metrics.items()}

    return metrics

eval_compute_metrics_fn = partial(
    compute_metrics, image_processor=image_processor, id2label=id2label, threshold=0.0
)
```

训练检测模型

在前几节中，你已经完成了大部分繁重的工作，现在可以开始训练模型了！即使经过调整大小后，该数据集中的图像仍然相当大。这意味着微调此模型至少需要一个 GPU。

训练涉及以下步骤：

197. 使用 AutoModelForObjectDetection 从预处理中使用的相同检查点加载模型。
198. 在 TrainingArguments 中定义训练超参数。
199. 将训练参数传递给 Trainer，并附带模型、数据集、图像处理器和数据组合器。
200. 调用 train() 以微调模型。

从用于预处理的相同检查点加载模型时，记得传递从数据集元数据中创建的 `label2id` 和 `id2label` 映射。此外，我们指定 `ignore_mismatched_sizes=True` 以替换现有的分类头。

```
from transformers import AutoModelForObjectDetection

model = AutoModelForObjectDetection.from_pretrained(
    MODEL_NAME,
    id2label=id2label,
    label2id=label2id,
    ignore_mismatched_sizes=True,
)
```

在 TrainingArguments 中使用 `output_dir` 指定保存模型的位置，然后根据需要配置超参数。对于 `num_train_epochs=30`，在 Google Colab T4 GPU 上训练大约需要 35 分钟，增加训练轮数可以获得更好的结果。

重要说明：

- 不要删除未使用的列，因为这将删除图像列。没有图像列，你无法创建 `pixel_values`。因此，将 `remove_unused_columns` 设置为 `False`。

- 设置 `eval_do_concat_batches=False` 以获得正确的评估结果。图像具有不同数量的目标框，如果批次被连接起来，我们将无法确定哪个框属于特定图像。

如果你希望将模型推送到 Hub，将 `push_to_hub` 设置为 `True`（你必须登录 Hugging Face 才能上传模型）。

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="detr_finetuned_cppe5",
    num_train_epochs=30,
    fp16=False,
    per_device_train_batch_size=8,
    dataloader_num_workers=4,
    learning_rate=5e-5,
    lr_scheduler_type="cosine",
    weight_decay=1e-4,
    max_grad_norm=0.01,
    metric_for_best_model="eval_map",
    greater_is_better=True,
    load_best_model_at_end=True,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    remove_unused_columns=False,
    eval_do_concat_batches=False,
    push_to_hub=True,
)
```

最后，将所有内容整合在一起，并调用 train()：

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=cppe5["train"],
    eval_dataset=cppe5["validation"],
    processing_class=image_processor,
    data_collator=collate_fn,
    compute_metrics=eval_compute_metrics_fn,
)

trainer.train()
```

如果你在 `training_args` 中设置了 `push_to_hub` 为 `True`，训练检查点将推送到 Hugging Face Hub。训练完成后，通过调用 push_to_hub() 方法推送最终模型到 Hub。

```
trainer.push_to_hub()
```

评估

```
from pprint import pprint

metrics = trainer.evaluate(eval_dataset=cppe5["test"], metric_key_prefix="test")
pprint(metrics)
```

这些结果可以通过调整 TrainingArguments 中的超参数进一步改进。试试看！

推理

现在你已经微调了一个模型，进行了评估，并将其上传到了 Hugging Face Hub，你可以使用它进行推理。

```
import torch
import requests

from PIL import Image, ImageDraw
from transformers import AutoImageProcessor, AutoModelForObjectDetection

url = "https://images.pexels.com/photos/8413299/pexels-photo-
8413299.jpeg?auto=compress&cs=tinysrgb&w=630&h=375&dpr=2"
image = Image.open(requests.get(url, stream=True).raw)
```

从 Hugging Face Hub 加载模型和图像处理器（如果要在当前会话中使用已训练的模型，可以跳过这一步）：

```
device = "cuda"
model_repo = "qubvel-hf/detr_finetuned_cppe5"

image_processor = AutoImageProcessor.from_pretrained(model_repo)
model = AutoModelForObjectDetection.from_pretrained(model_repo)
model = model.to(device)
```

检测边界框：

```
with torch.no_grad():
    inputs = image_processor(images=[image], return_tensors="pt")
    outputs = model(**inputs.to(device))
    target_sizes = torch.tensor([[image.size[1], image.size[0]]])
    results = image_processor.post_process_object_detection(outputs, threshold=0.3,
target_sizes=target_sizes)[0]

for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    print(
        f"Detected {model.config.id2label[label.item()]} with confidence "
```

```
        f"{round(score.item(), 3)} at location {box}"
    )
```

让我们绘制结果：

```
draw = ImageDraw.Draw(image)

for score, label, box in zip(results["scores"], results["labels"], results["boxes"]):
    box = [round(i, 2) for i in box.tolist()]
    x, y, x2, y2 = tuple(box)
    draw.rectangle((x, y, x2, y2), outline="red", width=1)
    draw.text((x, y), model.config.id2label[label.item()], fill="white")

image
```

零样本目标检测

在传统方式中，用于目标检测的模型需要标注的图像数据集进行训练，并且仅限于检测训练数据中的类。

零样本目标检测由 OWL-ViT 模型支持，该模型采用了一种不同的方法。OWL-ViT 是一种开放词汇目标检测器，它可以根据自由文本查询检测图像中的对象，而无需对模型进行微调以适应标注数据集。

OWL-ViT 利用多模态表示来执行开放词汇检测。它结合了 CLIP 与轻量级的对象分类和定位头。通过使用 CLIP 的文本编码器嵌入自由文本查询，并将其作为对象分类和定位头的输入，这些头将图像与其相应的文本描述关联起来，而 ViT 处理图像补丁作为输入。OWL-ViT 的作者首先从头开始训练 CLIP，然后在标准目标检测数据集上使用二部匹配损失对 OWL-ViT 进行端到端微调。

通过这种方法，模型可以根据文本描述检测对象，而无需预先在标注数据集上进行训练。

在这个指南中，你将学习如何使用 OWL-ViT：

- 基于文本提示检测对象
- 批量目标检测
- 图像引导的目标检测

在开始之前，请确保你已经安装了所有必要的库：

```
pip install -q transformers
```

零样本目标检测管道

使用 OWL-ViT 进行推理的最简单方法是通过 pipeline()。从 Hugging Face Hub 上的一个检查点实例化一个零样本目标检测管道：

```
from transformers import pipeline

checkpoint = "google/owlv2-base-patch16-ensemble"
detector = pipeline(model=checkpoint, task="zero-shot-object-detection")
```

接下来，选择一张你想要检测对象的图像。这里我们将使用 NASA Great Images 数据集中的一张宇航员 Eileen Collins 的图片。

```
import skimage
import numpy as np
from PIL import Image

image = skimage.data.astronaut()
image = Image.fromarray(np.uint8(image)).convert("RGB")
image
```

〔图：宇航员 Eileen Collins〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_1.png，离线电子书未嵌入，详见原仓库）

将图像和候选对象标签传递给管道。这里我们直接传递图像；其他合适的选项包括本地图像路径或图像 URL。我们还传递了要查询的所有项目的文本描述。

```
predictions = detector(
    image,
    candidate_labels=["human face", "rocket", "nasa badge", "star-spangled banner"],
)
predictions
```

让我们可视化预测结果：

```
from PIL import ImageDraw

draw = ImageDraw.Draw(image)

for prediction in predictions:
    box = prediction["box"]
    label = prediction["label"]
    score = prediction["score"]

    xmin, ymin, xmax, ymax = box.values()
    draw.rectangle((xmin, ymin, xmax, ymax), outline="red", width=1)
    draw.text((xmin, ymin), f"{label}: {round(score,2)}", fill="white")
```

```
image
```

〔图：NASA 图像上的预测结果〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_2.png，离线电子书未嵌入，详见原仓库）

手动进行基于文本提示的零样本目标检测

现在你已经看到了如何使用零样本目标检测管道，让我们手动复制相同的结果。

首先从 Hugging Face Hub 加载模型和相关处理器。这里我们将使用与之前相同的检查点：

```
from transformers import AutoProcessor, AutoModelForZeroShotObjectDetection

model = AutoModelForZeroShotObjectDetection.from_pretrained(checkpoint)
processor = AutoProcessor.from_pretrained(checkpoint)
```

为了换换口味，我们选择一张不同的图像。

```
import requests

url =
"https://unsplash.com/photos/oj0zeY2Ltk4/download?ixid=MnwxMjA3fDB8MXxzZWFyY2h8MTR8fHBpY25pY3xlbnwwfHx8fDE2Nzc0OTE1NDk&force=true&w=640"
im = Image.open(requests.get(url, stream=True).raw)
im
```

〔图：海滩照片〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_3.png，离线电子书未嵌入，详见原仓库）

使用处理器准备模型的输入。处理器结合了一个图像处理器，该处理器通过调整大小和归一化来准备图像，以及一个 CLIPTokenizer，它负责处理文本输入。

```
text_queries = ["hat", "book", "sunglasses", "camera"]
inputs = processor(text=text_queries, images=im, return_tensors="pt")
```

将输入传递给模型，进行后处理并可视化结果。由于图像处理器在将图像输入模型之前进行了调整大小，因此需要使用 post_process_object_detection() 方法，以确保预测的边界框相对于原始图像具有正确的坐标：

```
import torch

with torch.no_grad():
    outputs = model(**inputs)
    target_sizes = torch.tensor([im.size[::-1]])
```

```
    results = processor.post_process_object_detection(outputs, threshold=0.1,
target_sizes=target_sizes)[0]

draw = ImageDraw.Draw(im)

scores = results["scores"].tolist()
labels = results["labels"].tolist()
boxes = results["boxes"].tolist()

for box, score, label in zip(boxes, scores, labels):
    xmin, ymin, xmax, ymax = box
    draw.rectangle((xmin, ymin, xmax, ymax), outline="red", width=1)
    draw.text((xmin, ymin), f"{text_queries[label]}: {round(score,2)}", fill="white")

im
```

〔图：带有检测对象的海滩照片〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_4.png，离线电子书未嵌入，详见原仓库）

批量处理

你可以传递多组图像和文本查询，以在多个图像中搜索不同的（或相同的）对象。让我们同时使用宇航员图像和海滩图像。对于批量处理，你应该将文本查询作为嵌套列表传递给处理器，将图像作为 PIL 图像、PyTorch 张量或 NumPy 数组的列表传递。

```
images = [image, im]
text_queries = [
    ["human face", "rocket", "nasa badge", "star-spangled banner"],
    ["hat", "book", "sunglasses", "camera"],
]
inputs = processor(text=text_queries, images=images, return_tensors="pt")
```

在后处理中，你之前传递了单个图像的大小作为张量，但你也可以传递一个元组，或者在处理多个图像时传递一个元组列表。让我们为两个示例创建预测，并可视化第二个示例（`image_idx = 1`）。

```
with torch.no_grad():
    outputs = model(**inputs)
    target_sizes = [x.size[::-1] for x in images]
    results = processor.post_process_object_detection(outputs, threshold=0.1,
target_sizes=target_sizes)

image_idx = 1
draw = ImageDraw.Draw(images[image_idx])

scores = results[image_idx]["scores"].tolist()
labels = results[image_idx]["labels"].tolist()
```

```
boxes = results[image_idx]["boxes"].tolist()

for box, score, label in zip(boxes, scores, labels):
    xmin, ymin, xmax, ymax = box
    draw.rectangle((xmin, ymin, xmax, ymax), outline="red", width=1)
    draw.text((xmin, ymin), f"{text_queries[image_idx][label]}: {round(score,2)}", fill="white")

images[image_idx]
```

〔图：带有检测对象的海滩照片〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_4.png，离线电子书未嵌入，详见原仓库）

图像引导的目标检测

除了使用文本查询进行零样本目标检测外，OWL-ViT 还提供图像引导的目标检测。这意味着你可以使用图像查询在目标图像中找到类似的对象。与文本查询不同，只允许使用一个示例图像。

让我们选择一张沙发上两只猫的图像作为目标图像，一张单猫的图像作为查询图像：

```
url = "http://images.cocodataset.org/val2017/000000039769.jpg"
image_target = Image.open(requests.get(url, stream=True).raw)

query_url = "http://images.cocodataset.org/val2017/000000524280.jpg"
query_image = Image.open(requests.get(query_url, stream=True).raw)
```

让我们快速查看一下图像：

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 2)
ax[0].imshow(image_target)
ax[1].imshow(query_image)
```

〔图：猫〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_5.png，离线电子书未嵌入，详见原仓库）

在预处理步骤中，你现在需要使用 `query_images` 而不是文本查询：

```
inputs = processor(images=image_target, query_images=query_image, return_tensors="pt")
```

对于预测，而不是将输入传递给模型，而是将它们传递给 image_guided_detection()。与之前一样绘制预测结果，但这次没有标签。

```
with torch.no_grad():
    outputs = model.image_guided_detection(**inputs)
    target_sizes = torch.tensor([image_target.size[::-1]])
```

```
    results = processor.post_process_image_guided_detection(outputs=outputs,
target_sizes=target_sizes)[0]

draw = ImageDraw.Draw(image_target)

scores = results["scores"].tolist()
boxes = results["boxes"].tolist()

for box, score in zip(boxes, scores):
    xmin, ymin, xmax, ymax = box
    draw.rectangle((xmin, ymin, xmax, ymax), outline="white", width=4)

image_target
```

〔图：带有边界框的猫〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/zero-sh-obj-detection_6.png，离线电子书未嵌入，详见原仓库）

零样本图像分类

零样本图像分类是一种任务，涉及使用未显式训练过特定类别标注数据的模型对图像进行分类。

在传统方式中，图像分类需要在一个特定的标注图像集上训练模型，该模型学习“映射”某些图像特征到标签。当需要使用这样的模型进行引入新标签的分类任务时，通常需要微调来“重新校准”模型。

相比之下，零样本或开放词汇图像分类模型通常是多模态模型，它们在大量图像及其相关描述的数据集上进行了训练。这些模型学习了对齐的视觉-语言表示，可以用于许多下游任务，包括零样本图像分类。

这是一种更灵活的图像分类方法，允许模型推广到新的和未见过的类别，而无需额外的训练数据，并且用户可以用自由形式的文本描述目标对象来查询图像。

在本指南中，你将学习如何：

- 创建一个零样本图像分类管道
- 手动运行零样本图像分类推理

在开始之前，请确保安装了所有必要的库：

```
pip install -q "transformers[torch]" pillow
```

零样本图像分类管道

尝试支持零样本图像分类的模型推理的最简单方法是使用相应的 pipeline()。从 Hugging Face Hub 上的检查点实例化一个管道：

```
from transformers import pipeline

checkpoint = "openai/clip-vit-large-patch14"
detector = pipeline(model=checkpoint, task="zero-shot-image-classification")
```

接下来，选择一幅你想要分类的图像。

```
from PIL import Image
import requests

url = "https://unsplash.com/photos/g8oS8-
82DxI/download?ixid=MnwxMjA3fDB8MXx0b3BpY3x8SnBnNktpZGwtSGt8fHx8fDJ8fDE2NzgxMDYwODc&force=true&
w=640"
image = Image.open(requests.get(url, stream=True).raw)

image
```

〔图：猫头鹰的照片〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/owl.jpg，离线电子书未嵌入，详见原仓库）

将图像和候选对象标签传递给管道。这里我们直接传递图像；其他合适的选项包括本地路径的图像或图像 URL。候选标签可以是简单的单词，如本例所示，也可以是更详细的描述。

```
predictions = detector(image, candidate_labels=["fox", "bear", "seagull", "owl"])
predictions
```

手动进行零样本图像分类

现在你已经看到了如何使用零样本图像分类管道，让我们看看如何手动运行零样本图像分类。

首先从 Hugging Face Hub 上的检查点加载模型及相关处理器。这里我们将使用与前面相同的检查点：

```
from transformers import AutoProcessor, AutoModelForZeroShotImageClassification

model = AutoModelForZeroShotImageClassification.from_pretrained(checkpoint)
processor = AutoProcessor.from_pretrained(checkpoint)
```

为了换换口味，我们使用一张不同的图像。

```
from PIL import Image
import requests

url =
"https://unsplash.com/photos/xBRQfR2bqNI/download?ixid=MnwxMjA3fDB8MXxhbGx8fHx8fHx8fHwxNjc4Mzg4ODEx&force=true&w=640"
image = Image.open(requests.get(url, stream=True).raw)

image
```

〔图：汽车的照片〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/car.jpg，离线电子书未嵌入，详见原仓库）

使用处理器准备输入数据。处理器结合了一个图像处理器，它通过调整大小和归一化来准备图像，以及一个分词器，它处理文本输入。

```
candidate_labels = ["tree", "car", "bike", "cat"]
# 使用管道提示模板以获得相同的结果
candidate_labels = [f'This is a photo of {label}.' for label in candidate_labels]
inputs = processor(images=image, text=candidate_labels, return_tensors="pt", padding=True)
```

将输入数据传递给模型，并处理结果：

```
import torch

with torch.no_grad():
    outputs = model(**inputs)

logits = outputs.logits_per_image[0]
probs = logits.softmax(dim=-1).numpy()
scores = probs.tolist()

result = [
    {"score": score, "label": candidate_label}
    for score, candidate_label in sorted(zip(probs, candidate_labels), key=lambda x: -x[0])
]

result
```

单目深度估计（单图像深度估计）

“单目”指的是使用单张图像（即只有一个视角或镜头）来获取深度信息。与之相对的是双目或多目深度估计，后者使用两个或多个摄像头来获取深度信息，这个表格能帮助你更好地理解和对比单目深度估计与双目或多目深度估计的特点和应用场景。

特征	单目深度估计	双目或多目深度估计
输入	单张图像	多张图像（通常从不同视角拍摄）
挑战	- 尺度模糊性：无法直接提供绝对深度信息 - 光照影响：光照变化影响估计准确性 - 遮挡和反射：遮挡物和反射表面干扰深度信息提取	- 设备复杂性：需要多个摄像头 - 计算复杂性：处理多张图像需要更多的计算资源
优势	- 设备简单：只需一个摄像头 - 适用范围广：适合资源受限的设备，如智能手机、无人机等	- 直接计算深度：通过视差直接计算深度，准确性高
工作原理	- 利用纹理、颜色、阴影、几何结构和上下文等视觉线索 - 通过学习和模型训练来估计深度	- 通过视差（不同视角下的位置差异）直接计算深度
应用	- 3D 重建 - 增强现实 - 自动驾驶辅助系统 - 机器人导航（资源受限环境）	- 自动驾驶汽车 - 机器人导航（高精度要求） - 工业检测和测量
典型模型	- Depth Anything V2 - ZoeDepth	- Stereo Matching - Structure from Motion (SfM)

单目深度估计

单目深度估计是一项计算机视觉任务，涉及从单个图像中预测场景的深度信息。换句话说，它是从单个摄像头视角估计场景中物体距离的过程。

单目深度估计有多种应用，包括 3D 重建、增强现实、自动驾驶和机器人技术。这是一项具有挑战性的任务，因为它需要模型理解场景中物体之间的复杂关系以及相应的深度信息，这些信息可能会受到光照条件、遮挡和纹理等因素的影响。

深度估计主要分为两类：

- **绝对深度估计**：这种任务变体旨在提供从摄像头到物体的确切深度测量值。这个术语通常与度量深度估计互换使用，其中深度以米或英尺等精确单位表示。绝对深度估计模型输出的深度图包含代表真实距离的数值。
- **相对深度估计**：相对深度估计旨在预测场景中物体或点的深度顺序，而不提供精确的测量值。这些模型输出的深度图显示哪些部分更靠近或远离，但不提供实际的距离。

在本指南中，我们将介绍如何使用 Depth Anything V2（一种最先进的零样本相对深度估计模型）和 ZoeDepth（一种绝对深度估计模型）进行推断。

查看 深度估计任务页面，以查看所有兼容的架构和检查点。

在开始之前，我们需要安装最新版本的 Transformers：

```
pip install -q -U transformers
```

深度估计管道

使用支持深度估计的模型进行推断的最简单方法是使用相应的 pipeline()。从 Hugging Face Hub 上的检查点实例化一个管道：

```
from transformers import pipeline
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
checkpoint = "depth-anything/Depth-Anything-V2-base-hf"
pipe = pipeline("depth-estimation", model=checkpoint, device=device)
```

接下来，选择一张图像进行分析：

```
from PIL import Image
import requests

url = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee.jpg"
image = Image.open(requests.get(url, stream=True).raw)
image
```

〔图：bee〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee.jpg，离线电子书未嵌入，详见原仓库）

将图像传递给管道：

```
predictions = pipe(image)
```

管道返回一个包含两个条目的字典。第一个条目称为 `predicted_depth`，是一个张量，其中每个像素的值表示深度（以米为单位）。第二个条目 `depth` 是一个 PIL 图像，可视化了深度估计结果。

让我们看一下可视化的结果：

```
predictions["depth"]
```

〔图：depth-visualization〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/depth-visualization.png，离线电子书未嵌入，详见原仓库）

手动进行深度估计推断

现在你已经看到了如何使用深度估计管道，接下来我们来看看如何手动复制相同的结果。

首先，从 Hugging Face Hub 上的检查点加载模型及其相关处理器。这里我们使用与之前相同的检查点：

```
from transformers import AutoImageProcessor, AutoModelForDepthEstimation

checkpoint = "Intel/zoedepth-nyu-kitti"

image_processor = AutoImageProcessor.from_pretrained(checkpoint)
model = AutoModelForDepthEstimation.from_pretrained(checkpoint).to(device)
```

使用 `image_processor` 准备图像输入，它将处理必要的图像变换，如调整大小和归一化：

```
pixel_values = image_processor(image, return_tensors="pt").pixel_values.to(device)
```

将准备好的输入传递给模型：

```
with torch.no_grad():
    outputs = model(pixel_values)
```

接下来，对结果进行后处理，以去除任何填充并调整深度图的大小以匹配原始图像的尺寸。`post_process_depth_estimation` 函数输出一个包含 `"predicted_depth"` 的字典列表。

```
# ZoeDepth 动态填充输入图像。因此，我们在调用 `post_process_depth_estimation` 时传递原始图像的尺寸，以去除填充并调整到原始尺寸。
post_processed_output = image_processor.post_process_depth_estimation(
    outputs,
    source_sizes=[(image.height, image.width)],
)

predicted_depth = post_processed_output[0]["predicted_depth"]
depth = (predicted_depth - predicted_depth.min()) / (predicted_depth.max() -
```

```
predicted_depth.min())
depth = depth.detach().cpu().numpy() * 255
depth = Image.fromarray(depth.astype("uint8"))
```

在 原始实现 中，ZoeDepth 模型会对原始图像和翻转后的图像进行推断，并取平均结果。

`post_process_depth_estimation` 函数可以通过传递翻转后的输出到可选参数 `outputs_flipped` 来处理这一点：

```
with torch.no_grad():
    outputs = model(pixel_values)
    outputs_flipped = model(pixel_values=torch.flip(inputs.pixel_values, dims=[3]))

post_processed_output = image_processor.post_process_depth_estimation(
    outputs,
    source_sizes=[(image.height, image.width)],
    outputs_flipped=outputs_flipped,
)
```

〔图：depth-visualization-zoe〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/depth-visualization-zoe.png，离线电子书未嵌入，详见原仓库）

以图生图（图像增强、图像修复等图像处理任务）

图像到图像任务是指应用程序接收一张图像并输出另一张图像的任务。这包括多种子任务，例如图像增强（超分辨率、低光增强、去雨等）、图像修复等。

本指南将向您展示如何：

- 使用图像到图像管道进行超分辨率任务，
- 不使用管道运行图像到图像模型进行相同的任务。

请注意，截至本指南发布时，`image-to-image` 管道仅支持超分辨率任务。

让我们从安装必要的库开始。

```
pip install transformers
```

现在，我们可以使用 Swin2SR 模型 初始化管道。然后，我们可以通过用图像调用管道来进行推理。目前，此管道仅支持 Swin2SR 模型。

```
from transformers import pipeline
import torch
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
pipe = pipeline(task="image-to-image", model="caidas/swin2SR-lightweight-x2-64", device=device)
```

接下来，让我们加载一张图像。

```
from PIL import Image
import requests

url = "https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/transformers/tasks/cat.jpg"
image = Image.open(requests.get(url, stream=True).raw)

print(image.size)
```



猫的照片

现在，我们可以使用管道进行推理。我们将得到猫图像的放大版本。

```
upscaled = pipe(image)
print(upscaled.size)
```

如果您希望不使用管道自行进行推理，可以使用 `Swin2SRForImageSuperResolution` 和 `Swin2SRImageProcessor` 类。我们将使用相同的模型检查点进行此操作。让我们初始化模型和处理器。

```
from transformers import Swin2SRForImageSuperResolution, Swin2SRImageProcessor
```

```
model = Swin2SRForImageSuperResolution.from_pretrained("caidas/swin2SR-lightweight-x2-64").to(device)
processor = Swin2SRImageProcessor("caidas/swin2SR-lightweight-x2-64")
```

`pipeline` 抽象化了我们需要自己做的预处理和后处理步骤，所以让我们预处理图像。我们将图像传递给处理器，然后将像素值移动到 GPU。

```
pixel_values = processor(image, return_tensors="pt").pixel_values
print(pixel_values.shape)

pixel_values = pixel_values.to(device)
```

现在，我们可以通过将像素值传递给模型来进行图像推理。输出是一个 `ImageSuperResolutionOutput` 类型的对象，如下所示：

```
import torch

with torch.no_grad():
  outputs = model(pixel_values)
```

我们需要获取 `reconstruction` 并进行后处理以便可视化。让我们看看它是什么样子的。

```
outputs.reconstruction.data.shape
# torch.Size([1, 3, 880, 1072])
```

我们需要压缩输出并去掉轴 0，裁剪值，然后将其转换为 numpy 浮点数。然后，我们将排列轴以获得形状 `[1072, 880]`，最后，将输出范围恢复到 `[0, 255]`。

```
import numpy as np

# 压缩，移至 CPU 并裁剪值
output = outputs.reconstruction.data.squeeze().cpu().clamp_(0, 1).numpy()
# 重新排列轴
output = np.moveaxis(output, source=0, destination=-1)
# 将值恢复到像素值范围
output = (output * 255.0).round().astype(np.uint8)
Image.fromarray(output)
```



猫的放大照片

图像特征提取

图像特征提取是从给定的图像中提取语义上有意义的特征的任务。这一任务有许多应用场景，包括图像相似性和图像检索。此外，大多数计算机视觉模型都可以用于图像特征提取，其中可以通过移除任务特定的头部（如图像分类、目标检测等）来获取特征。这些特征在高层次上非常有用，例如边缘检测、角点检测等。根据模型的深度，这些特征也可能包含有关现实世界的信息（例如猫的样子）。因此，这些输出可以用于训练特定数据集上的新分类器。

在这个指南中，你将：

- 学习如何基于 `image-feature-extraction` 管道构建一个简单的图像相似性系统。
- 通过裸模型推理实现相同的任务。

使用 image-feature-extraction 管道实现图像相似性

我们有两张猫坐在渔网上的图片，其中一张是生成的。

```
from PIL import Image
import requests

img_urls = ["https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/cats.png", "https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/cats.jpeg"]
image_real = Image.open(requests.get(img_urls[0], stream=True).raw).convert("RGB")
image_gen = Image.open(requests.get(img_urls[1], stream=True).raw).convert("RGB")
```

让我们看看管道的实际应用。首先，初始化管道。如果你不传递任何模型，管道将自动初始化为 `google/vit-base-patch16-224`。如果你想计算相似度，可以将 `pool` 参数设置为 `True`。

```
import torch
from transformers import pipeline

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
pipe = pipeline(task="image-feature-extraction", model_name="google/vit-base-patch16-384",
device=DEVICE, pool=True)
```

要使用 `pipe` 进行推理，可以将两张图片传递给它。

```
outputs = pipe([image_real, image_gen])
```

输出包含这两张图片的池化嵌入。

```
# 获取单个输出的长度
print(len(outputs[0][0]))
# 显示输出
print(outputs)

# 768
# [[-0.03909236937761307, 0.43381670117378235, -0.06913255900144577,
```

为了获得相似度分数，我们需要将它们传递给相似度函数。

```
from torch.nn.functional import cosine_similarity

similarity_score = cosine_similarity(torch.Tensor(outputs[0]),
                                     torch.Tensor(outputs[1]), dim=1)

print(similarity_score)

# tensor([0.6043])
```

如果你希望获取池化之前的最后一个隐藏状态，可以避免传递 `pool` 参数，因为默认值为 `False`。这些隐藏状态对于基于模型特征训练新的分类器或模型非常有用。

```
pipe = pipeline(task="image-feature-extraction", model_name="google/vit-base-patch16-224",
device=DEVICE)
output = pipe(image_real)
```

由于输出未池化，我们得到的是最后一个隐藏状态，其中第一个维度是批量大小，最后两个维度是嵌入形状。

```
import numpy as np
print(np.array(output).shape)
# (1, 197, 768)
```

使用 AutoModel 获取特征和相似性

我们还可以使用 `transformers` 的 `AutoModel` 类来获取特征。`AutoModel` 加载任何没有任务特定头部的变换器模型，我们可以使用它来获取特征。

```
from transformers import AutoImageProcessor, AutoModel

processor = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
model = AutoModel.from_pretrained("google/vit-base-patch16-224").to(DEVICE)
```

让我们编写一个简单的推理函数。我们将首先将输入传递给 `processor`，然后将其输出传递给 `model`。

```
def infer(image):
  inputs = processor(image, return_tensors="pt").to(DEVICE)
  outputs = model(**inputs)
  return outputs.pooler_output
```

我们可以直接将图片传递给这个函数并获取嵌入。

```
embed_real = infer(image_real)
embed_gen = infer(image_gen)
```

再次计算嵌入的相似度。

```
from torch.nn.functional import cosine_similarity

similarity_score = cosine_similarity(embed_real, embed_gen, dim=1)
print(similarity_score)

# tensor([0.6061], device='cuda:0', grad_fn=<SumBackward1>)
```

图像掩码生成

掩码生成是指为图像生成具有语义意义的掩码的任务。这项任务与图像分割非常相似，但有许多不同之处。图像分割模型是在标注数据集上训练的，因此它们仅限于在训练过程中见过的类别；给定一张图像，它们会返回一组掩码及其对应的类别。

掩码生成模型则是在大量数据上训练的，并且以两种模式运行：

- **提示模式**：在这种模式下，模型接收一张图像和一个提示，提示可以是图像中某个对象内的二维点位置（XY 坐标）或围绕对象的边界框。在提示模式下，模型只返回提示指向的对象的掩码。
- **分割一切模式**：在分割一切模式下，给定一张图像，模型会生成图像中的每个掩码。为此，会在图像上生成一个点网格并进行推理。

掩码生成任务由 Segment Anything Model (SAM) 支持。这是一个强大的模型，包括基于视觉 Transformers 的图像编码器、提示编码器和双向 Transformers 掩码解码器。图像和提示被编码，解码器接收这些嵌入并生成有效的掩码。

〔图：SAM 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/sam.png，离线电子书未嵌入，详见原仓库）

SAM 是一个强大的基础模型，因为它覆盖了大量数据。它是在 SA-1B 数据集上训练的，该数据集包含 100 万张图像和 11 亿个掩码。

在本指南中，您将学习如何：

- 使用批量处理进行分割一切模式的推理，
- 进行点提示模式的推理，
- 进行框提示模式的推理。

首先，让我们安装 `transformers`：

```
pip install -q transformers
```

掩码生成管道 (Mask Generation Pipeline)

使用 `mask-generation` 管道是最简单的进行掩码生成模型推理的方法。

```
from transformers import pipeline

checkpoint = "facebook/sam-vit-base"
mask_generator = pipeline(model=checkpoint, task="mask-generation")
```

让我们看一下图像。

```
from PIL import Image
import requests
```

```
img_url = "https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/bee.jpg"
image = Image.open(requests.get(img_url, stream=True).raw).convert("RGB")

![示例图像](https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/bee.jpg)
```

让我们进行分割一切。`points-per-batch` 选项启用在分割一切模式下的并行推理，这可以加快推理解析速度，但会消耗更多内存。此外，SAM 只能在点上进行批处理，而不能在图像上进行批处理。`pred_iou_thresh` 是置信度阈值，只有高于该阈值的掩码才会被返回。

```
masks = mask_generator(image, points_per_batch=128, pred_iou_thresh=0.88)
```

`masks` 的内容如下：

```
{
    'masks': [
        array([
            [False, False, False, ...,  True,  True,  True],
            [False, False, False, ...,  True,  True,  True],
            [False, False, False, ...,  True,  True,  True],
            ...,
            [False, False, False, ..., False, False, False],
            [False, False, False, ..., False, False, False],
            [False, False, False, ..., False, False, False]
        ]),
        array([
            [False, False, False, ..., False, False, False],
            [False, False, False, ..., False, False, False],
            [False, False, False, ..., False, False, False],
            ...
        ),
    'scores': tensor([
        0.9972, 0.9917,
        ...,
    ])
}
```

我们可以这样可视化它们：

```
import matplotlib.pyplot as plt

plt.imshow(image, cmap='gray')

for i, mask in enumerate(masks["masks"]):
    plt.imshow(mask, cmap='viridis', alpha=0.1, vmin=0, vmax=1)

plt.axis('off')
plt.show()
```

下面是灰度显示的原始图像和叠加的彩色地图。非常棒。

〔图：可视化〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee_segmented.png，离线电子书未嵌入，详见原仓库）

模型推理 (Model Inference)

点提示 (Point Prompting)

您也可以不使用管道直接使用模型。为此，需要初始化模型和处理器。

```
from transformers import SamModel, SamProcessor
import torch

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = SamModel.from_pretrained("facebook/sam-vit-base").to(device)
processor = SamProcessor.from_pretrained("facebook/sam-vit-base")
```

要进行点提示，将输入点传递给处理器，然后将处理器的输出传递给模型进行推理。为了处理模型的输出，需要将处理器的初始输出中的 `original_sizes` 和 `reshaped_input_sizes` 传递进来，因为处理器会调整图像大小，输出需要进行外推。

```
input_points = [[[2592, 1728]]]  # 蜜蜂的位置

inputs = processor(image, input_points=input_points, return_tensors="pt").to(device)
with torch.no_grad():
    outputs = model(**inputs)
masks = processor.image_processor.post_process_masks(
    outputs.pred_masks.cpu(), inputs["original_sizes"].cpu(),
inputs["reshaped_input_sizes"].cpu()
)
```

我们可以可视化 `masks` 输出中的三个掩码。

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(1, 4, figsize=(15, 5))

axes[0].imshow(image)
axes[0].set_title('原始图像')
mask_list = [masks[0][0][0].numpy(), masks[0][0][1].numpy(), masks[0][0][2].numpy()]

for i, mask in enumerate(mask_list, start=1):
```

```
    overlayed_image = np.array(image).copy()

    overlayed_image[:,:,0] = np.where(mask == 1, 255, overlayed_image[:,:,0])
    overlayed_image[:,:,1] = np.where(mask == 1, 0, overlayed_image[:,:,1])
    overlayed_image[:,:,2] = np.where(mask == 1, 0, overlayed_image[:,:,2])

    axes[i].imshow(overlayed_image)
    axes[i].set_title(f'掩码 {i}')
for ax in axes:
    ax.axis('off')

plt.show()
```

〔图：可视化〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/masks.png，离线电子书未嵌入，详见原仓库）

框提示 (Box Prompting)

您也可以像点提示一样进行框提示。只需将输入框以 `[x_min, y_min, x_max, y_max]` 格式与图像一起传递给 `processor`。将处理器的输出直接传递给模型，然后再次处理输出。

```
# 围绕蜜蜂的边界框
box = [2350, 1600, 2850, 2100]

inputs = processor(
        image,
        input_boxes=[[[box]]],
        return_tensors="pt"
    ).to("cuda")

with torch.no_grad():
    outputs = model(**inputs)

mask = processor.image_processor.post_process_masks(
    outputs.pred_masks.cpu(),
    inputs["original_sizes"].cpu(),
    inputs["reshaped_input_sizes"].cpu()
)[0][0][0].numpy()
```

您可以像下面这样可视化蜜蜂周围的边界框。

```
import matplotlib.patches as patches

fig, ax = plt.subplots()
ax.imshow(image)

rectangle = patches.Rectangle((2350, 1600), 500, 500, linewidth=2, edgecolor='r',
facecolor='none')
```

```
ax.add_patch(rectangle)
ax.axis("off")
plt.show()
```

〔图：可视化边界框〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/bbox.png，离线电子书未嵌入，详见原仓库）

您可以查看以下推理输出。

```
fig, ax = plt.subplots()
ax.imshow(image)
ax.imshow(mask, cmap='viridis', alpha=0.4)

ax.axis("off")
plt.show()
```

〔图：可视化推理〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/box_inference.png，离线电子书未嵌入，详见原仓库）

关键点检测（图像特征点检测）

关键点检测用于识别和定位图像中的特定兴趣点。这些关键点也称为标志点，代表物体的有意义特征，例如面部特征或物体部分。这些模型输入一张图像，并返回以下输出：

- **关键点和分数**：兴趣点及其置信度分数。
- **描述符**：每个关键点周围图像区域的表示，捕捉其纹理、梯度、方向等属性。

在本指南中，我们将展示如何从图像中提取关键点。

在本教程中，我们将使用 SuperPoint，这是一个用于关键点检测的基础模型。

```
from transformers import AutoImageProcessor, SuperPointForKeypointDetection


# 初始化处理器和模型
processor = AutoImageProcessor.from_pretrained("magic-leap-community/superpoint")
model = SuperPointForKeypointDetection.from_pretrained("magic-leap-community/superpoint")
```

我们将在以下图像上测试模型。

〔图：蜜蜂〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee.jpg，离线电子书未嵌入，详见原仓库）

〔图：猫〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/cats.png，离线电子书未嵌入，详见原仓库）

```
import torch
from PIL import Image
import requests
import cv2

# 下载图像
url_image_1 = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee.jpg"
image_1 = Image.open(requests.get(url_image_1, stream=True).raw)
url_image_2 = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/cats.png"
image_2 = Image.open(requests.get(url_image_2, stream=True).raw)

# 将图像放入列表中
images = [image_1, image_2]

# 处理输入并进行推理
inputs = processor(images, return_tensors="pt").to(model.device, model.dtype)
outputs = model(**inputs)
```

模型输出包括每个图像中的相对关键点、描述符、掩码和分数。掩码突出显示图像中存在关键点的区域。

```
# 输出示例
outputs.keypoints
outputs.scores
outputs.descriptors
outputs.mask
```

为了在图像中标绘实际的关键点，我们需要对输出进行后处理。为此，我们需要将实际的图像尺寸传递给 `post_process_keypoint_detection` 函数，同时传递输出。

```
# 获取图像尺寸
image_sizes = [(image.size[1], image.size[0]) for image in images]
# 进行后处理
outputs = processor.post_process_keypoint_detection(outputs, image_sizes)
```

现在，输出是一个字典列表，每个字典包含处理后的关键点、分数和描述符。

```
# 输出示例
outputs[0]['keypoints']
outputs[0]['scores']
outputs[0]['descriptors']
```

我们可以使用这些数据来标绘关键点。

```
import matplotlib.pyplot as plt
import torch


# 遍历每个图像
for i in range(len(images)):
    keypoints = outputs[i]["keypoints"]
    scores = outputs[i]["scores"]
    descriptors = outputs[i]["descriptors"]

    # 将张量转换为 NumPy 数组
    keypoints = outputs[i]["keypoints"].detach().numpy()
    scores = outputs[i]["scores"].detach().numpy()
    image = images[i]
    image_width, image_height = image.size

    # 绘制图像和关键点
    plt.axis('off')
    plt.imshow(image)
    plt.scatter(
        keypoints[:, 0],
        keypoints[:, 1],
        s=scores * 100,  # 置信度分数影响点的大小
        c='cyan',  # 点的颜色
        alpha=0.4  # 透明度
    )
    plt.show()
```

下面是输出结果。

〔图：蜜蜂〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bee_keypoint.png，离线电子书未嵌入，详见原仓库）

〔图：猫〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/cats_keypoint.png，离线电子书未嵌入，详见原仓库）

知识蒸馏在计算机视觉中的应用

知识蒸馏是一种将大型复杂模型（教师模型）的知识转移到小型简单模型（学生模型）的技术。为了从一个模型中提取知识并传递给另一个模型，我们首先使用一个预训练的教师模型（例如图像分

类任务），然后随机初始化一个学生模型来学习相同的任务。接下来，我们训练学生模型，使其输出与教师模型的输出尽可能接近，从而模拟教师模型的行为。这一技术最早由 Hinton 等人在论文《神经网络中的知识蒸馏》中提出。在本指南中，我们将进行特定任务的知识蒸馏。我们将使用 beans 数据集。

本指南展示了如何使用 🤗 Transformers 的 Trainer API 将一个经过微调的 ViT 模型（教师模型）蒸馏到一个 MobileNet（学生模型）。

让我们安装蒸馏和评估过程中需要的库。

```
pip install transformers datasets accelerate tensorboard evaluate --upgrade
```

在这个例子中，我们将使用 `merve/beans-vit-224` 模型作为教师模型。这是一个基于 `google/vit-base-patch16-224-in21k` 在 beans 数据集上微调的图像分类模型。我们将把这个模型蒸馏到一个随机初始化的 MobileNetV2。

现在我们加载数据集。

```
from datasets import load_dataset

dataset = load_dataset("beans")
```

我们可以使用任一模型的图像处理器，因为在这种情况下它们返回相同分辨率的相同输出。我们将使用 `dataset` 的 `map()` 方法对数据集的每个分片进行预处理。

```
from transformers import AutoImageProcessor
teacher_processor = AutoImageProcessor.from_pretrained("merve/beans-vit-224")

def process(examples):
    processed_inputs = teacher_processor(examples["image"])
    return processed_inputs

processed_datasets = dataset.map(process, batched=True)
```

我们的目标是让学生模型（随机初始化的 MobileNet）模仿教师模型（微调后的视觉变换器）。为了实现这一点，我们首先获取教师模型和学生模型的 logits 输出。然后，我们将每种输出除以参数 `temperature`，该参数控制每个软目标的重要性。参数 `lambda` 用于权衡蒸馏损失的重要性。在本例中，我们将使用 `temperature=5` 和 `lambda=0.5`。我们将使用 Kullback-Leibler 散度损失来计算学生模型和教师模型之间的差异。给定两个数据 P 和 Q，KL 散度解释了用 Q 表示 P 所需的额外信息

量。如果两者完全相同，它们的 KL 散度为零，因为不需要其他信息来解释 P。因此，在知识蒸馏的背景下，KL 散度是有用的。

```
from transformers import TrainingArguments, Trainer
import torch
import torch.nn as nn
import torch.nn.functional as F

class ImageDistilTrainer(Trainer):
    def __init__(self, teacher_model=None, student_model=None, temperature=None,
lambda_param=None, *args, **kwargs):
        super().__init__(model=student_model, *args, **kwargs)
        self.teacher = teacher_model
        self.student = student_model
        self.loss_function = nn.KLDivLoss(reduction="batchmean")
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        self.teacher.to(device)
        self.teacher.eval()
        self.temperature = temperature
        self.lambda_param = lambda_param

    def compute_loss(self, student, inputs, return_outputs=False):
        student_output = self.student(**inputs)

        with torch.no_grad():
            teacher_output = self.teacher(**inputs)

        # 计算教师和学生的软目标
        soft_teacher = F.softmax(teacher_output.logits / self.temperature, dim=-1)
        soft_student = F.log_softmax(student_output.logits / self.temperature, dim=-1)

        # 计算损失
        distillation_loss = self.loss_function(soft_student, soft_teacher) * (self.temperature
** 2)

        # 计算真实标签的损失
        student_target_loss = student_output.loss

        # 计算最终损失
        loss = (1. - self.lambda_param) * student_target_loss + self.lambda_param *
distillation_loss
        return (loss, student_output) if return_outputs else loss
```

现在我们登录 Hugging Face Hub，以便通过 `Trainer` 将模型推送到 Hugging Face Hub。

```
from huggingface_hub import notebook_login

notebook_login()
```

设置 `TrainingArguments`、教师模型和学生模型。

```
from transformers import AutoModelForImageClassification, MobileNetV2Config,
MobileNetV2ForImageClassification

training_args = TrainingArguments(
    output_dir="my-awesome-model",
    num_train_epochs=30,
    fp16=True,
    logging_dir=f"{repo_name}/logs",
    logging_strategy="epoch",
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    report_to="tensorboard",
    push_to_hub=True,
    hub_strategy="every_save",
    hub_model_id=repo_name,
)

num_labels = len(processed_datasets["train"].features["labels"].names)

# 初始化模型
teacher_model = AutoModelForImageClassification.from_pretrained(
    "merve/beans-vit-224",
    num_labels=num_labels,
    ignore_mismatched_sizes=True
)

# 从头开始训练 MobileNetV2
student_config = MobileNetV2Config()
student_config.num_labels = num_labels
student_model = MobileNetV2ForImageClassification(student_config)
```

我们可以使用 `compute_metrics` 函数在测试集上评估模型。此函数将在训练过程中计算模型的 `accuracy` 和 `f1`。

```
import evaluate
import numpy as np

accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    acc = accuracy.compute(references=labels, predictions=np.argmax(predictions, axis=1))
    return {"accuracy": acc["accuracy"]}
```

现在我们使用定义的训练参数初始化 `Trainer`。我们还将初始化数据收集器。

```
from transformers import DefaultDataCollator

data_collator = DefaultDataCollator()
trainer = ImageDistilTrainer(
    student_model=student_model,
    teacher_model=teacher_model,
    training_args=training_args,
    train_dataset=processed_datasets["train"],
    eval_dataset=processed_datasets["validation"],
    data_collator=data_collator,
    processing_class=teacher_processor,
    compute_metrics=compute_metrics,
    temperature=5,
    lambda_param=0.5
)
```

现在我们可以训练模型。

```
trainer.train()
```

我们可以在测试集上评估模型。

```
trainer.evaluate(processed_datasets["test"])
```

在测试集上，我们的模型达到了 72% 的准确率。为了验证蒸馏的有效性，我们还使用相同的超参数从头开始训练了一个 MobileNet，结果在测试集上的准确率为 63%。我们鼓励读者尝试不同的预训练教师模型、学生架构、蒸馏参数，并报告他们的发现。蒸馏模型的训练日志和检查点可以在 这个仓库 中找到，从头开始训练的 MobileNetV2 可以在 这个仓库 中找到。

多模态

图像描述生成

图像描述生成（Image Captioning）是指为给定图像生成描述性文字的任务。它在现实世界中的应用包括帮助视障人士在不同情境中导航。因此，图像描述生成通过为人们描述图像，提高了内容的可访问性。

本指南将展示如何：

- 微调一个图像描述生成模型。
- 使用微调后的模型进行推理。

在开始之前，请确保安装了所有必要的库：

```
pip install transformers datasets evaluate -q
pip install jiwer -q
```

我们鼓励您登录您的 Hugging Face 账户，以便上传并与社区分享您的模型。当提示时，输入您的令牌以登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载 Pokémon BLIP 描述数据集

使用 🤗 Dataset 库加载一个包含 {图像-描述} 对的数据集。要创建您自己的 PyTorch 图像描述生成数据集，可以参考这个笔记本。

```
from datasets import load_dataset

ds = load_dataset("lambdalabs/pokemon-blip-captions")
ds
```

数据集有两个特征：`image` 和 `text`。

许多图像描述生成数据集为每张图像包含多个描述。在这种情况下，常见的策略是在训练过程中从可用的描述中随机采样一个。

使用 train_test_split 方法将数据集的训练部分分为训练集和测试集：

```
ds = ds["train"].train_test_split(test_size=0.1)
train_ds = ds["train"]
test_ds = ds["test"]
```

让我们可视化训练集中的几个样本。

```
from textwrap import wrap
import matplotlib.pyplot as plt
import numpy as np

def plot_images(images, captions):
    plt.figure(figsize=(20, 20))
    for i in range(len(images)):
        ax = plt.subplot(1, len(images), i + 1)
        caption = captions[i]
        caption = "\n".join(wrap(caption, 12))
        plt.title(caption)
        plt.imshow(images[i])
        plt.axis("off")
```

```
sample_images_to_visualize = [np.array(train_ds[i]["image"]) for i in range(5)]
sample_captions = [train_ds[i]["text"] for i in range(5)]
plot_images(sample_images_to_visualize, sample_captions)
```



样本训练图像

预处理数据集

由于数据集有两种模态（图像和文本），预处理流程将分别处理图像和描述。

为此，加载与您即将微调的模型相关联的处理器类。

```
from transformers import AutoProcessor

checkpoint = "microsoft/git-base"
processor = AutoProcessor.from_pretrained(checkpoint)
```

处理器将内部预处理图像（包括调整大小和像素缩放）并对描述进行分词。

```
def transforms(example_batch):
    images = [x for x in example_batch["image"]]
    captions = [x for x in example_batch["text"]]
    inputs = processor(images=images, text=captions, padding="max_length")
    inputs.update({"labels": inputs["input_ids"]})
    return inputs

train_ds.set_transform(transforms)
test_ds.set_transform(transforms)
```

数据集准备好后，您可以设置模型进行微调。

加载基础模型

将 "microsoft/git-base" 加载到 `AutoModelForCausalLM` 对象中。

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(checkpoint)
```

评估

图像描述生成模型通常使用 Rouge Score 或 Word Error Rate 进行评估。在本指南中，您将使用 Word Error Rate (WER)。

我们使用 🤗 Evaluate 库来进行评估。关于 WER 的潜在限制和其他注意事项，请参考这个指南。

```
from evaluate import load
import torch

wer = load("wer")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predicted = logits.argmax(-1)
    decoded_labels = processor.batch_decode(labels, skip_special_tokens=True)
    decoded_predictions = processor.batch_decode(predicted, skip_special_tokens=True)
    wer_score = wer.compute(predictions=decoded_predictions, references=decoded_labels)
    return {"wer_score": wer_score}
```

训练！

现在，您已准备好开始微调模型。您将使用 🤗 Trainer 进行此操作。

首先，使用 TrainingArguments 定义训练参数。

```
from transformers import TrainingArguments, Trainer

model_name = checkpoint.split("/")[1]

training_args = TrainingArguments(
    output_dir=f"{model_name}-pokemon",
    learning_rate=5e-5,
    num_train_epochs=50,
    fp16=True,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    gradient_accumulation_steps=2,
    save_total_limit=3,
    eval_strategy="steps",
    eval_steps=50,
    save_strategy="steps",
    save_steps=50,
    logging_steps=50,
    remove_unused_columns=False,
    push_to_hub=True,
    label_names=["labels"],
```

```
    load_best_model_at_end=True,
)
```

然后将它们与数据集和模型一起传递给 🤗 Trainer。

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=test_ds,
    compute_metrics=compute_metrics,
)
```

要开始训练，只需在 Trainer 对象上调用 train()。

```
trainer.train()
```

您应该看到训练损失随着训练的进行而平稳下降。

训练完成后，使用 push_to_hub() 方法将您的模型分享到 Hub，以便所有人都可以使用您的模型：

```
trainer.push_to_hub()
```

推理

从 `test_ds` 中取一个样本图像来测试模型。

```
from PIL import Image
import requests

url = "https://huggingface.co/datasets/sayakpaul/sample-datasets/resolve/main/pokemon.png"
image = Image.open(requests.get(url, stream=True).raw)
image
```



测试图像

准备图像以供模型使用。

```
device = "cuda" if torch.cuda.is_available() else "cpu"

inputs = processor(images=image, return_tensors="pt").to(device)
pixel_values = inputs.pixel_values
```

调用 `generate` 并解码预测结果。

```
generated_ids = model.generate(pixel_values=pixel_values, max_length=50)
generated_caption = processor.batch_decode(generated_ids, skip_special_tokens=True)[0]
print(generated_caption)
```

从结果看来，微调后的模型生成了一个相当不错的描述！

文本转语音（TTS）

文本转语音（TTS）是将文字转换成自然语音的任务，这种语音可以支持多种语言和多个说话者。目前，🤗 Transformers 中已经提供了多种文本转语音模型，比如 Bark、MMS、VITS 和 SpeechT5。

你可以通过 `text-to-audio` pipeline（或者它的别名——`text-to-speech`）轻松生成音频。

有些模型，比如 Bark 还可以生成非语言语音，比如笑声、叹息和哭泣，甚至可以添加音乐。下面是一个使用 Bark 进行“文本转语音”的示例：

```
from transformers import pipeline

pipe = pipeline("text-to-speech", model="suno/bark-small")
text = "[clears throat] This is a test ... and I just took a long pause."
output = pipe(text)
```

这里有一个代码片段，你可以在 notebook 中运行它来试听生成的音频：

```
from IPython.display import Audio
Audio(output["audio"], rate=output["sampling_rate"])
```

想了解更多关于 Bark 和其他预训练 TTS 模型的示例，请参考我们的音频课程。

如果你想要微调一个 TTS 模型，目前 🤗 Transformers 中可用的文本转语音模型只有 SpeechT5 和 FastSpeech2Conformer，不过未来会添加更多模型。

SpeechT5 是在语音转文本和文本转语音数据的组合上进行预训练的，这使得它能够学习一个统一的隐含表示空间，这个空间由文本和语音共享。这意味着同一个预训练模型可以针对不同任务进行微调。此外，SpeechT5 通过 `x-vector` 说话者嵌入支持多个说话者。

本指南的其余部分将展示如何：

201. 将原本在英语语音上训练的 SpeechT5 微调到 VoxPopuli 数据集的荷兰语（nl）子集上。

202. 使用你微调后的模型进行推理，有两种方法：使用 pipeline 或直接使用。

在开始之前，请确保你已经安装了所有必要的库：

```
pip install datasets soundfile speechbrain accelerate
```

需要从源代码安装 🤗Transformers，因为并非所有 SpeechT5 的功能都已合并到官方版本中：

```
pip install git+https://github.com/huggingface/transformers.git
```

若要按照本指南操作，你还需要一个 GPU。如果你在 notebook 中工作，运行以下命令来检查是否有可用的 GPU：

```
!nvidia-smi
```

对于 AMD GPU，可以使用：

```
!rocm-smi
```

我们鼓励你登录你的 Hugging Face 账户，以便上传并与社区分享你的模型。当提示时，输入你的令牌进行登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

加载数据集

VoxPopuli 是一个大规模的多语种语音语料库，数据来源于 2009-2020 年欧洲议会活动录音。它包含 15 种欧洲语言的标记音频-转录数据。在本指南中，我们使用荷兰语子集，你也可以选择其他子集。

请注意，VoxPopuli 或任何其他自动语音识别（ASR）数据集可能不是训练 TTS 模型的最佳选择。那些对 ASR 有益的特性，比如过多的背景噪音，在 TTS 中通常是不希望出现的。所以，找到高质量、多语种、多说话者的 TTS 数据集确实相当有挑战性。

让我们加载数据：

```
from datasets import load_dataset, Audio

dataset = load_dataset("facebook/voxpopuli", "nl", split="train")
len(dataset)
```

20968 个示例应该足够用于微调。SpeechT5 要求音频数据的采样率为 16 kHz，所以请确保数据集中的示例满足这一要求：

```
dataset = dataset.cast_column("audio", Audio(sampling_rate=16000))
```

预处理数据

首先，定义要使用的模型 checkpoint 并加载相应的处理器：

```
from transformers import SpeechT5Processor

```

```
checkpoint = "microsoft/speecht5_tts"
processor = SpeechT5Processor.from_pretrained(checkpoint)
```

为 SpeechT5 分词进行文本清理

首先，清理文本数据。你需要使用处理器的分词器部分来处理文本：

```
tokenizer = processor.tokenizer
```

数据集示例包含 `raw_text` 和 `normalized_text` 特征。在决定使用哪个特征作为文本输入时，考虑到 `SpeechT5` 的分词器没有用于数字的任何标记。在 `normalized_text` 中，数字被写成文本形式。因此，它更适合作为输入文本，我们建议使用 `normalized_text`。

由于 `SpeechT5` 是在英语上训练的，它可能无法识别荷兰语数据集中的某些字符。如果保留原样，这些字符将被转换为 `<unk>` 标记。然而，在荷兰语中，某些字符如 `à` 用于强调音节。为了保留文本的意义，我们可以将这个字符替换为普通的 `a`。

为了识别不支持的标记，使用 `SpeechT5Tokenizer` 提取数据集中的所有唯一字符，该分词器以字符作为标记。为此，编写 `extract_all_chars` 映射函数，将所有示例的转录文本连接成一个字符串，并将其转换为字符集合。确保在 `dataset.map()` 中设置 `batched=True` 和 `batch_size=-1`，以便所有转录文本一次性提供给映射函数。

```
def extract_all_chars(batch):
    all_text = " ".join(batch["normalized_text"])
    vocab = list(set(all_text))
    return {"vocab": [vocab], "all_text": [all_text]}


vocabs = dataset.map(
    extract_all_chars,
    batched=True,
    batch_size=-1,
    keep_in_memory=True,
    remove_columns=dataset.column_names,
)

dataset_vocab = set(vocabs["vocab"][0])
tokenizer_vocab = {k for k, _ in tokenizer.get_vocab().items()}
```

现在你有两套字符集：一套是数据集中的词汇，另一套是分词器中的词汇。

为了识别数据集中任何不支持的字符，你可以取这两个集合的差集。结果集合将包含那些在数据集中但不在分词器中的字符，即不支持的字符集合。

```
dataset_vocab - tokenizer_vocab
```

为了处理上一步中得到的不支持的字符，需要定义一个函数将这些字符映射到有效的标记。请注意，分词器中空格已经用 _ 替换，不需要单独处理。

```
replacements = [
    ("à", "a"),
    ("ç", "c"),
    ("è", "e"),
    ("ë", "e"),
    ("í", "i"),
    ("ï", "i"),
    ("ö", "o"),
    ("ü", "u"),
]


def cleanup_text(inputs):
    for src, dst in replacements:
        inputs["normalized_text"] = inputs["normalized_text"].replace(src, dst)
    return inputs


dataset = dataset.map(cleanup_text)
```

现在你已经处理了文本中的特殊字符（不支持的字符），接下来该将注意力转向音频数据了。

说话者

VoxPopuli 数据集包含多位说话者的语音，但数据集中有多少位说话者呢?

为了确定这一点，我们可以统计具有独特特征的说话者的数量以及每位说话者为数据集贡献的示例数量。考虑到数据集中一共有 20,968 个示例，这些信息将帮助我们更好地理解说话者和示例在数据中的分布情况。

```
from collections import defaultdict

speaker_counts = defaultdict(int)

for speaker_id in dataset["speaker_id"]:
    speaker_counts[speaker_id] += 1
```

通过绘制直方图，你可以了解每位说话者有多少数据。

```
import matplotlib.pyplot as plt

plt.figure()
plt.hist(speaker_counts.values(), bins=20)
plt.ylabel("Speakers")
```

```
plt.xlabel("Examples")
plt.show()
```

直方图显示，数据集中大约三分之一的说话者拥有少于 100 个示例，而大约 10 位说话者拥有超过 500 个示例。

为了提高训练效率并平衡数据集，我们可以将数据限制在拥有 100 到 400 个示例的说话者范围内。

```
def select_speaker(speaker_id):
    return 100 <= speaker_counts[speaker_id] <= 400


dataset = dataset.filter(select_speaker, input_columns=["speaker_id"])
```

让我们检查一下还剩下多少位说话者：

```
len(set(dataset["speaker_id"]))
```

让我们看看还剩下多少个示例：

```
len(dataset)
```

你现在大约有 10,000 个示例，来自大约 40 位独特的说话者，这应该是足够的。

请注意，如果示例较长，那些只有少量示例的说话者实际上可能会拥有更多的音频数据。然而，确定每位说话者的总音频量需要遍历整个数据集，这是一个比较耗时的过程，涉及加载和解码每个音频文件。因此，我们选择在这里跳过这一步。

说话者嵌入

为了让 TTS 模型能够区分多位说话者，你需要为每个示例创建一个说话者嵌入。说话者嵌入是模型的额外输入，用于捕捉特定说话者的声音特征。

为了生成这些说话者嵌入，这里使用 `SpeechBrain` 的预训练模型 spkrec-xvect-voxceleb。

创建一个函数 `create_speaker_embedding()`，它接受一个输入音频波形，并输出一个包含相应说话者嵌入的 512 元素向量。

```
import os
import torch
from speechbrain.inference.classifiers import EncoderClassifier

spk_model_name = "speechbrain/spkrec-xvect-voxceleb"

device = "cuda" if torch.cuda.is_available() else "cpu"
speaker_model = EncoderClassifier.from_hparams(
    source=spk_model_name,
```

```
    run_opts={"device": device},
    savedir=os.path.join("/tmp", spk_model_name),
)


def create_speaker_embedding(waveform):
    with torch.no_grad():
        speaker_embeddings = speaker_model.encode_batch(torch.tensor(waveform))
        speaker_embeddings = torch.nn.functional.normalize(speaker_embeddings, dim=2)
        speaker_embeddings = speaker_embeddings.squeeze().cpu().numpy()
    return speaker_embeddings
```

需要注意的是，`speechbrain/spkrec-xvect-voxceleb` 模型是在 `VoxCeleb` 数据集的英语语音上训练的，而本指南中的训练示例是荷兰语。尽管我们相信这个模型仍能为我们的荷兰语数据集生成合理的说话者嵌入，但这种假设在所有情况下可能并不成立。

为了获得最佳结果，我们建议首先在目标语音上训练一个 `X-vector` 模型。这将确保模型能够更好地捕捉荷兰语中独特的声音特征。

处理数据集

最后，我们需要把数据转换成模型需要的格式。我们编写一个名为 `prepare_dataset` 的函数，这个函数接收一个样本，然后用 `SpeechT5Processor` 工具对输入的文字进行编码，同时把目标音频转换成对数梅尔频谱图。此外，还要加入 `speaker embeddings` 作为额外的输入信息。

```
def prepare_dataset(example):
    audio = example["audio"]

    example = processor(
        text=example["normalized_text"],
        audio_target=audio["array"],
        sampling_rate=audio["sampling_rate"],
        return_attention_mask=False,
    )

    # strip off the batch dimension
    example["labels"] = example["labels"][0]

    # use SpeechBrain to obtain x-vector
    example["speaker_embeddings"] = create_speaker_embedding(audio["array"])

    return example
```

为了确认数据处理是否正确，我们可以查看一个具体的样本进行验证：

```
processed_example = prepare_dataset(dataset[0])
list(processed_example.keys())
```

说话者的嵌入信息应该是一个包含 512 个元素的向量：

```
processed_example["speaker_embeddings"].shape
```

标记应该是一个包含 80 个梅尔频段的日志梅尔频谱图。

```
import matplotlib.pyplot as plt

plt.figure()
plt.imshow(processed_example["labels"].T)
plt.show()
```

小贴士：如果你觉得这个频谱图看起来很乱，可能是因为你习惯了低频在底部、高频在顶部的显示方式。不过，在使用 `matplotlib` 库绘制频谱图时，y 轴是翻转的，所以频谱图显示的是颠倒的。

接下来，我们将这个处理函数应用到整个数据集上。这个过程大概需要 5 到 10 分钟的时间。

```
dataset = dataset.map(prepare_dataset, remove_columns=dataset.column_names)
```

你可能会看到一个警告，提示数据集中有些样本的输入长度超过了模型能处理的最大限制（600 个标记）。我们需要从数据集中移除这些样本。为了更稳妥地处理，确保批量处理更高效，我们进一步筛选，移除所有超过 200 个标记的样本。

```
def is_not_too_long(input_ids):
    input_length = len(input_ids)
    return input_length < 200


dataset = dataset.filter(is_not_too_long, input_columns=["input_ids"])
len(dataset)
```

接下来，我们需要创建一个基本的训练/测试数据拆分：

```
dataset = dataset.train_test_split(test_size=0.1)
```

数据整理器

为了把多个样本合并成一个批次处理，我们需要定义一个自定义的数据整理器。这个整理器会用填充标记来补齐较短的序列，确保所有样本长度一致。对于频谱图标签，填充部分会用特殊值-100 来代替。这个特殊值告诉模型在计算频谱图损失时，忽略这些填充部分。

```
from dataclasses import dataclass
from typing import Any, Dict, List, Union


@dataclass
class TTSDataCollatorWithPadding:
    processor: Any
```

```
    def __call__(self, features: List[Dict[str, Union[List[int], torch.Tensor]]]) -> Dict[str,
torch.Tensor]:
        input_ids = [{"input_ids": feature["input_ids"]} for feature in features]
        label_features = [{"input_values": feature["labels"]} for feature in features]
        speaker_features = [feature["speaker_embeddings"] for feature in features]

        # collate the inputs and targets into a batch
        batch = processor.pad(input_ids=input_ids, labels=label_features, return_tensors="pt")

        # replace padding with -100 to ignore loss correctly
        batch["labels"] = batch["labels"].masked_fill(batch.decoder_attention_mask.unsqueeze(-
1).ne(1), -100)

        # not used during fine-tuning
        del batch["decoder_attention_mask"]

        # round down target lengths to multiple of reduction factor
        if model.config.reduction_factor > 1:
            target_lengths = torch.tensor([len(feature["input_values"]) for feature in
label_features])
            target_lengths = target_lengths.new(
                [length - length % model.config.reduction_factor for length in target_lengths]
            )
            max_length = max(target_lengths)
            batch["labels"] = batch["labels"][:, :max_length]

        # also add in the speaker embeddings
        batch["speaker_embeddings"] = torch.tensor(speaker_features)

        return batch
```

在 `SpeechT5` 模型中，解码器部分的输入长度会被减半。也就是说，它会从目标序列中每隔一个时间步丢弃一个元素。然后，解码器会预测一个长度是原来两倍的序列。由于原始目标序列的长度可能是奇数，所以数据整理器会确保将批次的最大长度调整为 2 的倍数。

```
data_collator = TTSDataCollatorWithPadding(processor=processor)
```

训练模型

从之前用于加载处理器的同一个 checkpoint 中加载预训练模型：

```
from transformers import SpeechT5ForTextToSpeech

model = SpeechT5ForTextToSpeech.from_pretrained(checkpoint)
```

`use_cache=True` 选项与梯度 checkpointing 功能不兼容。为了顺利进行训练，我们需要关闭这个选项。

```
model.config.use_cache = False
```

接下来，我们定义训练参数。在这个阶段，我们不会在训练过程中计算任何评估指标。我们只关注损失值：

```
from transformers import Seq2SeqTrainingArguments

training_args = Seq2SeqTrainingArguments(
    output_dir="speecht5_finetuned_voxpopuli_nl",  # change to a repo name of your choice
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    learning_rate=1e-5,
    warmup_steps=500,
    max_steps=4000,
    gradient_checkpointing=True,
    fp16=True,
    eval_strategy="steps",
    per_device_eval_batch_size=2,
    save_steps=1000,
    eval_steps=1000,
    logging_steps=25,
    report_to=["tensorboard"],
    load_best_model_at_end=True,
    greater_is_better=False,
    label_names=["labels"],
    push_to_hub=True,
)
```

创建一个 `Trainer` 对象，并将模型、数据集和数据整理器传递给它进行实例化。

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    args=training_args,
    model=model,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    data_collator=data_collator,
    processing_class=processor,
)
```

准备工作完成后，你就可以开始训练了！训练过程可能需要几个小时。根据你的 GPU 性能，启动训练时可能会遇到 CUDA“内存不足”的错误。如果出现这种情况，你可以将 `per_device_train_batch_size` 减半，同时将 `gradient_accumulation_steps` 加倍来补偿。

```
trainer.train()
```

为了确保 checkpoint 能与 pipeline 一起使用，记得把处理器和 checkpoint 一起保存：

```
processor.save_pretrained("YOUR_ACCOUNT_NAME/speecht5_finetuned_voxpopuli_nl")
```

将最终模型上传到🤗 Hub：

```
trainer.push_to_hub()
```

模型推理

使用 pipeline 进行推理

太棒了，现在你已经微调了一个模型，可以开始用它来进行推理了！首先，我们来了解一下如何将它与相应的 pipeline 结合使用。让我们用你的 checkpoint 创建一个 `text-to-speech` 的 pipeline：

```
from transformers import pipeline

pipe = pipeline("text-to-speech", model="YOUR_ACCOUNT_NAME/speecht5_finetuned_voxpopuli_nl")
```

选择一段你想要用荷兰语朗读的文字，比如：

```
text = "hallo allemaal, ik praat nederlands. groetjes aan iedereen!"
```

要将 SpeechT5 与 pipeline 一起使用，你需要一个 `speaker embeddings`。我们可以从测试数据集中的一个示例中获取它：

```
example = dataset["test"][304]
speaker_embeddings = torch.tensor(example["speaker_embeddings"]).unsqueeze(0)
```

现在你可以将文本和说话者嵌入传递给 pipeline，它会自动处理剩下的步骤：

```
forward_params = {"speaker_embeddings": speaker_embeddings}
output = pipe(text, forward_params=forward_params)
output
```

然后你可以聆听生成的结果。

```
from IPython.display import Audio
Audio(output['audio'], rate=output['sampling_rate'])
```

手动运行推理

你也可以在不使用 pipeline 的情况下实现相同的推理结果，但这需要更多步骤。

从 🤗 Hub 加载模型：

```
model = SpeechT5ForTextToSpeech.from_pretrained("YOUR_ACCOUNT/speecht5_finetuned_voxpopuli_nl")
```

从测试数据集中挑选一个例子，获取 `speaker embeddings`：

```
example = dataset["test"][304]
speaker_embeddings = torch.tensor(example["speaker_embeddings"]).unsqueeze(0)
```

定义输入文本并将其标记化(tokenize)：

```
text = "hallo allemaal, ik praat nederlands. groetjes aan iedereen!"
inputs = processor(text=text, return_tensors="pt")
```

使用你的模型创建频谱图：

```
spectrogram = model.generate_speech(inputs["input_ids"], speaker_embeddings)
```

还可以可视化频谱图：

```
plt.figure()
plt.imshow(spectrogram.T)
plt.show()
```

最后，使用声码器（vocoder）将频谱图转换为音频。

```
with torch.no_grad():
    speech = vocoder(spectrogram)

from IPython.display import Audio

Audio(speech.numpy(), rate=16000)
```

根据我们的经验，要从这个模型中获得满意的结果可能有一定难度。`speaker embeddings` 的质量似乎是一个关键因素。由于 SpeechT5 是使用英语 x 向量进行预训练的，所以在使用英语的 `speaker embeddings` 时表现最佳。如果合成的语音效果不佳，建议尝试使用不同的 `speaker embeddings`。

增加训练时间也可能提升结果的质量。尽管如此，合成的语音明显是荷兰语而非英语，并且确实捕捉到了 speaker 的语音特征（与示例中的原始音频相比）。

另一个可以尝试的点是模型的配置。例如，可以尝试设置 `config.reduction_factor = 1`，看看是否会有所改善。

最后，必须考虑到伦理问题。虽然 TTS 技术有许多有益的应用，但也可能被用于恶意目的，比如在未经知情或同意的情况下模仿某人的声音。请务必谨慎和负责任地使用 TTS 技术。

图像视觉多模态理解模型（Image-text-to-text）

图像文本到文本模型，也称为视觉语言模型（VLMs），是接受图像输入的语言模型。这些模型可以处理各种任务，从视觉问答到图像分割。这个任务与图像到文本有许多相似之处，但也有一些重叠

的用例，如图像描述。图像到文本模型只接受图像输入，通常完成一个特定的任务，而 VLMs 接受开放式的文本和图像输入，是更通用的模型。

在本指南中，我们将简要概述 VLMs，并展示如何使用 Transformers 进行推理。

首先，有几种类型的 VLMs：

- 用于微调的基础模型
- 用于对话的聊天微调模型
- 用于指令的微调模型

本指南重点介绍使用指令微调模型进行推理。

让我们开始安装依赖项。

```
pip install -q transformers accelerate flash_attn
```

现在初始化模型和处理器。

```
from transformers import AutoProcessor, AutoModelForImageTextToText
import torch

device = torch.device("cuda")
model = AutoModelForImageTextToText.from_pretrained(
    "HuggingFaceM4/idefics2-8b",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
).to(device)

processor = AutoProcessor.from_pretrained("HuggingFaceM4/idefics2-8b")
```

这个模型有一个聊天模板，帮助用户解析聊天输出。此外，该模型还可以在单个对话或消息中接受多个图像作为输入。我们将准备输入图像，如下所示：



两只猫坐在网上



一只蜜蜂在粉红色的花朵上

下面是聊天模板的示例。我们可以通过在模板末尾附加它来提供对话回合和最后一条消息作为输入。

```
messages = [
    {
        "role": "user",
        "content": [
            {"type": "image"},
            {"type": "text", "text": "What do we see in this image?"}
        ]
    },
```

```
    {
        "role": "assistant",
        "content": [
            {"type": "text", "text": "In this image we can see two cats on the nets."}
        ]
    },
    {
        "role": "user",
        "content": [
            {"type": "image"},
            {"type": "text", "text": "And how about this image?"}
        ]
    },
]
```

我们现在将调用处理器的 `apply_chat_template()`方法来预处理其输出以及图像输入。

```
prompt = processor.apply_chat_template(messages, add_generation_prompt=True)
inputs = processor(text=prompt, images=[images[0], images[1]], return_tensors="pt").to(device)
```

我们现在可以将预处理后的输入传递给模型。

```
with torch.no_grad():
    generated_ids = model.generate(**inputs, max_new_tokens=500)
generated_texts = processor.batch_decode(generated_ids, skip_special_tokens=True)

print(generated_texts)
```

流式传输

我们可以使用文本流式传输来获得更好的生成体验。Transformers 支持使用 `TextStreamer` 或 `TextIteratorStreamer` 类进行流式传输。我们将使用 `TextIteratorStreamer` 与 `IDEFICS-8B`。

假设我们有一个应用程序，它保留聊天历史并接收新的用户输入。我们将像往常一样预处理输入，并初始化 `TextIteratorStreamer` 来在单独的线程中处理生成。这允许你实时流式传输生成的文本标记。任何生成参数都可以传递给 `TextIteratorStreamer`。

```
import time
from transformers import TextIteratorStreamer
from threading import Thread

def model_inference(
    user_prompt,
    chat_history,
    max_new_tokens,
    images
):
    user_prompt = {
```

```
        "role": "user",
        "content": [
            {"type": "image"},
            {"type": "text", "text": user_prompt},
        ]
    }
    chat_history.append(user_prompt)
    streamer = TextIteratorStreamer(
        processor.tokenizer,
        skip_prompt=True,
        timeout=5.0,
    )

    generation_args = {
        "max_new_tokens": max_new_tokens,
        "streamer": streamer,
        "do_sample": False
    }

    # add_generation_prompt=True 使模型生成机器人响应
    prompt = processor.apply_chat_template(chat_history, add_generation_prompt=True)
    inputs = processor(
        text=prompt,
        images=images,
        return_tensors="pt",
    ).to(device)
    generation_args.update(inputs)

    thread = Thread(
        target=model.generate,
        kwargs=generation_args,
    )
    thread.start()

    acc_text = ""
    for text_token in streamer:
        time.sleep(0.04)
        acc_text += text_token
        if acc_text.endswith("<end_of_utterance>"):
            acc_text = acc_text[:-18]
        yield acc_text

    thread.join()
```

现在让我们调用我们创建的 `model_inference` 函数并流式传输值。

```
generator = model_inference(
    user_prompt="And what is in this image?",
    chat_history=messages,
    max_new_tokens=100,
    images=images
```

```
)

for value in generator:
    print(value)
```

在较小硬件上适配模型

VLMs 通常很大，需要优化以适应较小的硬件。Transformers 支持许多模型量化库，这里我们只展示使用 Quanto 的 int8 量化。int8 量化提供了高达 75%的内存改进（如果所有权重都被量化）。然而，它并不是免费的午餐，因为 8 位不是 CUDA 本机精度，权重在运行时被量化来回转换，这增加了延迟。

首先，安装依赖项。

```
pip install -U quanto bitsandbytes
```

要在加载期间量化模型，我们首先需要创建 `QuantoConfig`。然后像往常一样加载模型，但在模型初始化期间传递 `quantization_config`。

```
from transformers import AutoModelForImageTextToText, QuantoConfig

model_id = "HuggingFaceM4/idefics2-8b"
quantization_config = QuantoConfig(weights="int8")
quantized_model = AutoModelForImageTextToText.from_pretrained(
    model_id, device_map="cuda", quantization_config=quantization_config
)
```

就这样，我们可以使用模型，无需任何更改。

进一步阅读

这里有一些关于图像文本到文本任务的更多资源。

- 图像文本到文本任务页面涵盖了模型类型、用例、数据集等。
- 视觉语言模型解释是一篇博客文章，涵盖了关于视觉语言模型和使用 TRL 进行监督微调的所有内容。

视频视觉多模态理解模型（Video-text-to-text）

视频文本到文本模型，也称为视频语言模型或带视频输入的视觉语言模型，是一种接受视频输入的语言模型。这些模型可以处理各种任务，从视频问答到视频字幕生成。

这些模型的架构与图像文本到文本模型几乎相同，只是进行了一些修改以接受视频数据，因为视频数据本质上是带有时间依赖性的图像帧。一些图像文本到文本模型可以接受多张图像，但仅凭这一点还不足以让模型接受视频。此外，视频文本到文本模型通常使用所有视觉模态进行训练。每个示例可能包含视频、多个视频、图像和多个图像。有些模型还可以接受交错输入。

例如，你可以在文本中通过添加视频标记来引用特定视频，如：“这个视频里发生了什么？`【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】`”。

在本指南中，我们简要介绍了视频语言模型，并展示了如何使用 Transformers 进行推理。

首先，视频语言模型有多种类型：

- 用于微调的基模型
- 用于对话的微调模型
- 用于指令的微调模型

本指南重点介绍使用指令微调模型 llava-hf/llava-interleave-qwen-7b-hf 进行推理，该模型可以接受交错数据。如果你的硬件不支持运行 7B 模型，你也可以尝试 llava-interleave-qwen-0.5b-hf。

让我们开始安装依赖项。

```
pip install -q transformers accelerate flash_attn
```

接下来，初始化模型和处理器。

```
from transformers import LlavaProcessor, LlavaForConditionalGeneration
import torch
model_id = "llava-hf/llava-interleave-qwen-0.5b-hf"

processor = LlavaProcessor.from_pretrained(model_id)

model = LlavaForConditionalGeneration.from_pretrained(model_id, torch_dtype=torch.float16)
model.to("cuda")
```

有些模型直接处理 `【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】` 标记，而其他模型接受与采样帧数相等的 `<image>` 标记。这个模型采用后一种方式处理视频。我们将编写一个简单的工具来处理图像标记，另一个工具来从 URL 获取视频并从中采样帧。

```
import uuid
import requests
import cv2
```

```
from PIL import Image

def replace_video_with_images(text, frames):
  return text.replace("<video>", "<image>" * frames)

def sample_frames(url, num_frames):
    response = requests.get(url)
    path_id = str(uuid.uuid4())
    path = f"./{path_id}.mp4"
    with open(path, "wb") as f:
      f.write(response.content)
    video = cv2.VideoCapture(path)
    total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    interval = total_frames // num_frames
    frames = []
    for i in range(total_frames):
        ret, frame = video.read()
        pil_img = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        if not ret:
            continue
        if i % interval == 0:
            frames.append(pil_img)
    video.release()
    return frames
```

获取输入数据。我们将采样帧并将它们连接起来。

```
video_1 = "https://huggingface.co/spaces/merve/llava-interleave/resolve/main/cats_1.mp4"
video_2 = "https://huggingface.co/spaces/merve/llava-interleave/resolve/main/cats_2.mp4"

video_1 = sample_frames(video_1, 6)
video_2 = sample_frames(video_2, 6)

videos = video_1 + video_2

videos
```

【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

【演示媒体资源：../../resources/videos/cats_1.mp4（离线电子书未嵌入视频/音频，详见原仓库）】

【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

【演示媒体资源：../../resources/videos/cats_2.mp4（离线电子书未嵌入视频/音频，详见原仓库）】

【演示媒体资源：（离线电子书未嵌入视频/音频，详见原仓库）】

两个视频都包含猫。

现在我们可以预处理输入数据。

这个模型有一个如下所示的提示模板。首先，我们将所有采样帧放入一个列表中。由于每个视频有八帧，我们将在提示中插入 12 个 `<image>` 标记。在提示末尾添加 `assistant` 以触发模型给出答案。然后我们可以进行预处理。

```
user_prompt = "这两个视频中的猫在做同样的事情吗？"
toks = "<image>" * 12
prompt = "<|im_start|>user" + toks + f"\n{user_prompt}<|im_end|><|im_start|>assistant"
inputs = processor(prompt, images=videos).to(model.device, model.dtype)
```

现在我们可以调用 generate() 进行推理。模型输出我们输入的问题和答案，所以我们只取模型输出中提示和 `assistant` 部分之后的文本。

```
output = model.generate(**inputs, max_new_tokens=100, do_sample=False)
print(processor.decode(output[0][2:], skip_special_tokens=True)[len(user_prompt)+10:])
```

输出结果：

第一个猫处于放松状态，眼睛 closed，表情 content，而第二个猫处于更活跃的状态，嘴巴 wide open，可能是打哈欠或发声。

要了解更多关于视频文本到文本模型的聊天模板和标记流的信息，请参考图像文本到文本任务指南，因为这些模型的工作方式类似。

文档问答（DQA）

文档问答（Document Question Answering，简称 DQA）是一项涉及对文档图像提出的问题提供答案的任务。支持此任务的模型的输入通常是图像和问题的组合，输出是以自然语言表达的答案。这些模型利用了多种模态，包括文本、单词位置（边界框）和图像本身。

本指南将说明如何：

- 在 DocVQA 数据集 上微调 LayoutLMv2。
- 使用微调后的模型进行推理。

要查看与此任务兼容的所有架构和检查点，我们建议查看 任务页面。

LayoutLMv2 通过在标记的最终隐藏状态之上添加一个问答头，来预测答案的起始和结束标记的位置，从而解决文档问答任务。换句话说，这个问题被视为提取式问答：给定上下文，提取哪个信息片段回答了问题。上下文来自 OCR 引擎的输出，这里使用的是谷歌的 Tesseract。

在开始之前，请确保你已安装所有必要的库。LayoutLMv2 依赖于 detectron2、torchvision 和 tesseract。

```
pip install -q transformers datasets

pip install 'git+https://github.com/facebookresearch/detectron2.git'
pip install torchvision

sudo apt install tesseract-ocr
pip install -q pytesseract
```

安装所有依赖项后，重新启动您的运行时。

我们鼓励您与社区分享您的模型。登录您的 Hugging Face 帐户，将其上传到 🤗 Hub。当提示时，输入您的令牌登录：

```
from huggingface_hub import notebook_login

notebook_login()
```

让我们定义一些全局变量。

```
model_checkpoint = "microsoft/layoutlmv2-base-uncased"
batch_size = 4
```

加载数据

在本指南中，我们使用 🤗 Hub 上的预处理的 DocVQA 的一小部分样本。如果您想使用完整的 DocVQA 数据集，您可以在 DocVQA 主页 上注册并下载。如果您这样做，请查看 如何将文件加载到 🤗 数据集中 以继续本指南。

```
from datasets import load_dataset

dataset = load_dataset("nielsr/docvqa_1200_examples")
dataset
```

如您所见，数据集已经分为训练集和测试集。查看一个随机示例，以熟悉特征。

```
dataset["train"].features
```

以下是各个字段表示的含义：

- `id`：示例的 id

- `image`：包含文档图像的 PIL.Image.Image 对象
- `query`：问题字符串 - 自然语言问题，多种语言
- `answers`：人类注释者提供的正确答案列表
- `words` 和 `bounding_boxes`：OCR 的结果，我们这里不使用
- `answer`：另一个模型匹配的答案，我们这里不使用

让我们只保留英语问题，并删除 `answer` 特征，因为它似乎包含另一个模型的预测。我们将从注释者提供的答案集中取第一个答案。或者，您可以随机抽样。

```
updated_dataset = dataset.map(lambda example: {"question": example["query"]["en"]},
remove_columns=["query"])
updated_dataset = updated_dataset.map(
    lambda example: {"answer": example["answers"][0]}, remove_columns=["answer", "answers"]
)
```

请注意，本指南中使用的 LayoutLMv2 检查点已使用 `max_position_embeddings = 512` 进行训练（您可以在 检查点的 `config.json` 文件 中找到此信息）。我们可以截断示例，但为了避免答案可能在大型文档末尾而最终被截断的情况，这里我们将删除一些嵌入长度可能超过 512 的示例。如果您的数据集中的大多数文档都很长，您可以实现滑动窗口策略 - 查看此笔记本了解详细信息。

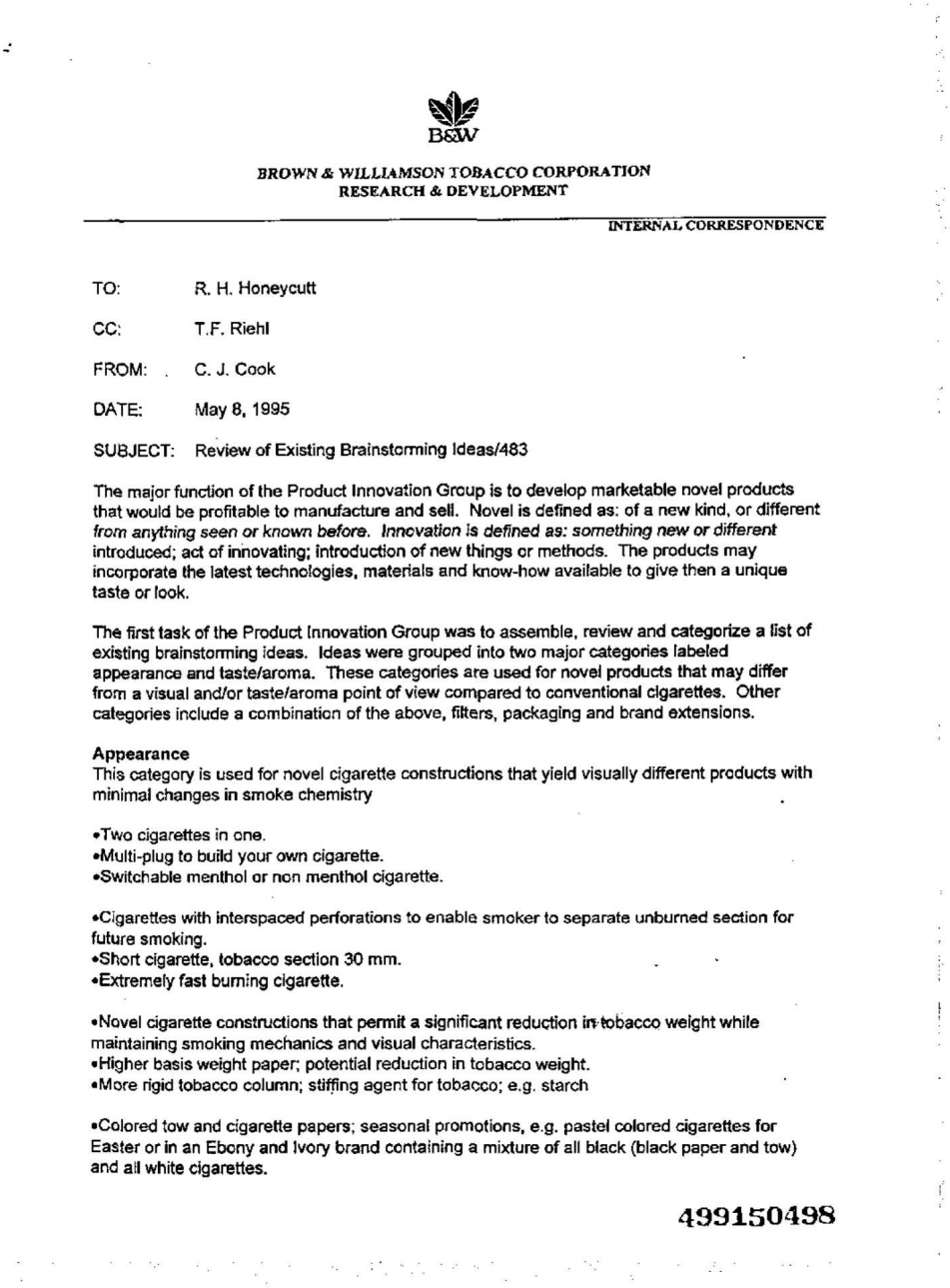
```
updated_dataset = updated_dataset.filter(lambda x: len(x["words"]) + len(x["question"].split())
< 512)
```

此时，让我们也从数据集中删除 OCR 特征。这些是微调另一个模型的结果。如果我们想使用它们，它们仍然需要一些处理，因为它们不符合本指南中使用的模型的输入要求。相反，我们可以在原始数据上使用 LayoutLMv2Processor 进行 OCR 和标记化。这样，我们将得到与模型预期输入相匹配的输入。如果您想手动处理图像，请查看 LayoutLMv2 模型文档 了解模型期望的输入格式。

```
updated_dataset = updated_dataset.remove_columns("words")
updated_dataset = updated_dataset.remove_columns("bounding_boxes")
```

最后，如果我们不查看图像示例，数据探索就不会完整。

```
updated_dataset["train"][11]["image"]
```



B&W

BROWN & WILLIAMSON TOBACCO CORPORATION
RESEARCH & DEVELOPMENT

INTERNAL CORRESPONDENCE

TO: R. H. Honeycutt

CC: T.F. Riehl

FROM: C. J. Cook

DATE: May 8, 1995

SUBJECT: Review of Existing Brainstorming Ideas/483

The major function of the Product Innovation Group is to develop marketable novel products that would be profitable to manufacture and sell. Novel is defined as: of a new kind, or different *from anything seen or known before. Innovation is defined as: something new or different* introduced; act of innovating; introduction of new things or methods. The products may incorporate the latest technologies, materials and know-how available to give then a unique taste or look.

The first task of the Product Innovation Group was to assemble, review and categorize a list of existing brainstorming ideas. Ideas were grouped into two major categories labeled appearance and taste/aroma. These categories are used for novel products that may differ from a visual and/or taste/aroma point of view compared to conventional cigarettes. Other categories include a combination of the above, filters, packaging and brand extensions.

Appearance
This category is used for novel cigarette constructions that yield visually different products with minimal changes in smoke chemistry

•Two cigarettes in one.
•Multi-plug to build your own cigarette.
•Switchable menthol or non menthol cigarette.

•Cigarettes with interspaced perforations to enable smoker to separate unburned section for future smoking.
•Short cigarette, tobacco section 30 mm.
•Extremely fast burning cigarette.

•Novel cigarette constructions that permit a significant reduction in tobacco weight while maintaining smoking mechanics and visual characteristics.
•Higher basis weight paper; potential reduction in tobacco weight.
•More rigid tobacco column; stiffing agent for tobacco; e.g. starch

•Colored tow and cigarette papers; seasonal promotions, e.g. pastel colored cigarettes for Easter or in an Ebony and Ivory brand containing a mixture of all black (black paper and tow) and all white cigarettes.

499150498

Source: https://www.industrydocuments.ucsf.edu/docs/mxcj0037

docvqa_example

预处理数据

文档问答任务是一个多模态任务，您需要确保每个模态的输入都根据模型的期望进行预处理。让我们从加载 LayoutLMv2Processor 开始，它内部结合了一个可以处理图像数据的图像处理器和一个可以编码文本数据的标记器。

```
from transformers import AutoProcessor

processor = AutoProcessor.from_pretrained(model_checkpoint)
```

预处理文档图像

首先，让我们使用处理器中的 `image_processor` 准备文档图像以供模型使用。默认情况下，图像处理器将图像调整大小为 224x224，确保它们具有正确的颜色通道顺序，并应用 tesseract OCR 以获取单词和规范化边界框。在此教程中，所有这些默认设置都是我们需要的。编写一个函数，将默认图像处理应用于一批图像，并返回 OCR 的结果。

```
image_processor = processor.image_processor

def get_ocr_words_and_boxes(examples):
    images = [image.convert("RGB") for image in examples["image"]]
    encoded_inputs = image_processor(images)

    examples["image"] = encoded_inputs.pixel_values
    examples["words"] = encoded_inputs.words
    examples["boxes"] = encoded_inputs.boxes

    return examples
```

要以快速方式将此预处理应用于整个数据集，请使用 map。

```
dataset_with_ocr = updated_dataset.map(get_ocr_words_and_boxes, batched=True, batch_size=2)
```

预处理文本数据

一旦我们对图像应用了 OCR，我们需要对数据集的文本部分进行编码，以准备模型。这涉及将我们在上一步中获得的单词和框转换为标记级别的 `input_ids`、`attention_mask`、`token_type_ids` 和 `bbox`。为了预处理文本，我们将需要处理器中的 `tokenizer`。

除了上面提到的预处理之外，我们还需要添加模型的标签。对于 🤗 Transformers 中的 `xxxForQuestionAnswering` 模型，标签包括 `start_positions` 和 `end_positions`，指示哪个标记是答案的起始，哪个标记是答案的结束。

让我们从这些开始。定义一个辅助函数，它可以在更大的列表（单词列表）中查找一个子列表（答案分割成单词）。

这个函数将接受两个列表作为输入，`words_list` 和 `answer_list`。然后它将在 `words_list` 上迭代，并检查 `words_list` 中的当前单词（`words_list[i]`）是否等于 `answer_list` 中的第一个单词

（`answer_list[0]`），并且 `words_list` 从当前单词开始并具有与 `answer_list` 相同长度的子列表是否等于 `answer_list`。如果此条件为真，则意味着找到了匹配项，并且该函数将记录匹配项、其起始索引（idx）和结束索引（idx + len(answer_list) - 1）。如果找到多个匹配项，该函数将只返回第一个。如果没有找到匹配项，该函数将返回（`None`，0 和 0）。

```
def subfinder(words_list, answer_list):
    matches = []
    start_indices = []
    end_indices = []
    for idx, i in enumerate(range(len(words_list))):
        if words_list[i] == answer_list[0] and words_list[i : i + len(answer_list)] ==
answer_list:
            matches.append(answer_list)
            start_indices.append(idx)
            end_indices.append(idx + len(answer_list) - 1)
    if matches:
        return matches[0], start_indices[0], end_indices[0]
    else:
        return None, 0, 0
```

为了说明这个函数如何在示例中找到答案的位置，让我们在示例上使用它：

```
example = dataset_with_ocr["train"][1]
words = [word.lower() for word in example["words"]]
match, word_idx_start, word_idx_end = subfinder(words, example["answer"].lower().split())
print("Question: ", example["question"])
print("Words:", words)
print("Answer: ", example["answer"])
print("start_index", word_idx_start)
print("end_index", word_idx_end)
```

一旦示例被编码，它们将看起来像这样：

```
encoding = tokenizer(example["question"], example["words"], example["boxes"])
tokenizer.decode(encoding["input_ids"])
```

我们将需要找到答案在编码输入中的位置。

- `token_type_ids` 告诉我们哪些标记是问题的一部分，哪些标记是文档单词的一部分。
- `tokenizer.cls_token_id` 将帮助在输入的开头找到特殊标记。
- `word_ids` 将帮助将原始 `words` 中的答案与编码输入中的相同答案匹配，并确定答案在编码输入中的起始/结束位置。

考虑到这些，让我们创建一个函数来编码数据集中的示例批次：

```
def encode_dataset(examples, max_length=512):
    questions = examples["question"]
```

```
    words = examples["words"]
    boxes = examples["boxes"]
    answers = examples["answer"]

    # 编码示例批次并初始化 start_positions 和 end_positions
    encoding = tokenizer(questions, words, boxes, max_length=max_length, padding="max_length",
truncation=True)
    start_positions = []
    end_positions = []

    # 循环遍历批次中的示例
    for i in range(len(questions)):
        cls_index = encoding["input_ids"][i].index(tokenizer.cls_token_id)

        # 在示例的单词中找到答案的位置
        words_example = [word.lower() for word in words[i]]
        answer = answers[i]
        match, word_idx_start, word_idx_end = subfinder(words_example, answer.lower().split())

        if match:
            # 如果找到匹配项，使用 `token_type_ids` 找到编码中单词的起始位置
            token_type_ids = encoding["token_type_ids"][i]
            token_start_index = 0
            while token_type_ids[token_start_index] != 1:
                token_start_index += 1

            token_end_index = len(encoding["input_ids"][i]) - 1
            while token_type_ids[token_end_index] != 1:
                token_end_index -= 1

            word_ids = encoding.word_ids(i)[token_start_index : token_end_index + 1]
            start_position = cls_index
            end_position = cls_index

            # 循环遍历 word_ids，直到它匹配单词中答案的位置
            # 一旦匹配，将 `token_start_index` 保存为编码中答案的 `start_position`
            for id in word_ids:
                if id == word_idx_start:
                    start_position = token_start_index
                else:
                    token_start_index += 1

            # 类似地，从末尾开始循环遍历 `word_ids` 以找到编码中答案的 `end_position`
            for id in word_ids[::-1]:
                if id == word_idx_end:
                    end_position = token_end_index
                else:
                    token_end_index -= 1
```

```
            start_positions.append(start_position)
            end_positions.append(end_position)

        else:
            start_positions.append(cls_index)
            end_positions.append(cls_index)

    encoding["image"] = examples["image"]
    encoding["start_positions"] = start_positions
    encoding["end_positions"] = end_positions

    return encoding
```

现在我们有了这个预处理函数，我们可以编码整个数据集：

```
encoded_train_dataset = dataset_with_ocr["train"].map(
    encode_dataset, batched=True, batch_size=2,
remove_columns=dataset_with_ocr["train"].column_names
)
encoded_test_dataset = dataset_with_ocr["test"].map(
    encode_dataset, batched=True, batch_size=2,
remove_columns=dataset_with_ocr["test"].column_names
)
```

让我们检查编码数据集的特征是什么样的：

```
encoded_train_dataset.features
```

评估

文档问答的评估需要大量的后处理。为了避免占用您太多时间，本指南跳过了评估步骤。Trainer 仍然在训练过程中计算评估损失，因此您不会完全不了解模型的性能。提取式问答通常使用 F1/精确匹配进行评估。如果您想自己实现，请查看 Hugging Face 课程中的 问答章节 以获取灵感。

训练

恭喜！您已经成功导航了本指南中最困难的部分，现在您已经准备好训练自己的模型了。训练包括以下步骤：

- 使用 `AutoModelForDocumentQuestionAnswering` 加载模型，使用与预处理相同的检查点。
- 在 `TrainingArguments` 中定义您的训练超参数，使用 `output_dir` 指定保存模型的位置，并配置您认为合适的超参数。如果您希望与社区分享您的模型，请将 `push_to_hub` 设置为 `True`（您必

须登录 Hugging Face 才能上传您的模型）。在这种情况下，`output_dir` 也将是存储您的模型检查点的存储库的名称。

- 定义一个简单的数据整理器来批量处理示例，这里 `DefaultDataCollator` 将做得很好。
- 将训练参数传递给 `Trainer`，以及模型、数据集和数据整理器。
- 调用 `train()` 来微调您的模型。

```
from transformers import AutoModelForDocumentQuestionAnswering

model = AutoModelForDocumentQuestionAnswering.from_pretrained(model_checkpoint)

from transformers import TrainingArguments

# 将此替换为你的存储库 ID
repo_id = "MariaK/layoutlmv2-base-uncased_finetuned_docvqa"

training_args = TrainingArguments(
    output_dir=repo_id,
    per_device_train_batch_size=4,
    num_train_epochs=20,
    save_steps=200,
    logging_steps=50,
    eval_strategy="steps",
    learning_rate=5e-5,
    save_total_limit=2,
    remove_unused_columns=False,
    push_to_hub=True,
)

from transformers import DefaultDataCollator

data_collator = DefaultDataCollator()

from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=encoded_train_dataset,
    eval_dataset=encoded_test_dataset,
    processing_class=processor,
)

trainer.train()

trainer.create_model_card()
trainer.push_to_hub()
```

推理

现在您已经微调了 LayoutLMv2 模型，并将其上传到 🤗 Hub，您可以使用它进行推理。尝试您的微调模型进行推理的最简单方法是在 `Pipeline` 中使用它。

让我们举一个例子：

```
example = dataset["test"][2]
question = example["query"]["en"]
image = example["image"]
print(question)
print(example["answers"])
```

接下来，使用您的模型实例化一个文档问答的管道，并将图像 + 问题的组合传递给它。

```
from transformers import pipeline

qa_pipeline = pipeline("document-question-answering", model="MariaK/layoutlmv2-base-uncased_finetuned_docvqa")
qa_pipeline(image, question)
```

如果您愿意，也可以手动复制管道的结果：

203. 取一个图像和一个问题，使用您模型中的处理器为模型准备它们。
204. 将预处理的结果或图像和问题通过模型转发。
205. 模型返回 `start_logits` 和 `end_logits`，它们指示哪个标记是答案的起始，哪个标记是答案的结束。两者都有形状 (batch_size, sequence_length)。
206. 对 `start_logits` 和 `end_logits` 的最后一个维度取 argmax，以获得预测的 `start_idx` 和 `end_idx`。
207. 使用标记器解码答案。

```
import torch
from transformers import AutoProcessor
from transformers import AutoModelForDocumentQuestionAnswering

processor = AutoProcessor.from_pretrained("MariaK/layoutlmv2-base-uncased_finetuned_docvqa")
model = AutoModelForDocumentQuestionAnswering.from_pretrained("MariaK/layoutlmv2-base-uncased_finetuned_docvqa")

with torch.no_grad():
    encoding = processor(image.convert("RGB"), question, return_tensors="pt")
    outputs = model(**encoding)
    start_logits = outputs.start_logits
    end_logits = outputs.end_logits
    predicted_start_idx = start_logits.argmax(-1).item()
```

```
    predicted_end_idx = end_logits.argmax(-1).item()

processor.tokenizer.decode(encoding.input_ids.squeeze()[predicted_start_idx : predicted_end_idx
+ 1])
```

这样，您就可以使用微调后的 LayoutLMv2 模型来回答关于文档图像的问题了。如果您对模型的表现感到满意，可以考虑将其分享到 Hugging Face 的模型库中，以便其他研究人员和开发者可以使用和改进它。

视觉问答（VQA）

视觉问答（Visual Question Answering，简称 VQA）是一项基于图像回答开放式问题的任务。支持此任务的模型的输入通常是图像和问题的组合，输出是以自然语言表达的回答。

VQA 的一些值得注意的用例包括：

- 为视障人士提供无障碍应用。
- 教育：对讲座或教科书中呈现的视觉材料提出问题。VQA 还可用于互动博物馆展览或历史遗址。
- 客户服务和电子商务：VQA 可以通过让用户询问有关产品的问题来提升用户体验。
- 图像检索：VQA 模型可用于检索具有特定特征的图像。例如，用户可以问“有狗吗？”以从一组图像中找到所有包含狗的图像。

在本指南中，你将学习如何：

- 在 `Graphcore/vqa` 数据集上微调分类 VQA 模型，特别是 ViLT。
- 使用微调后的 ViLT 进行推理。
- 使用生成模型（如 BLIP-2）运行零样本 VQA 推理。

微调 ViLT

ViLT 模型将文本嵌入到视觉 Transformer（ViT）中，使其能够具有视觉和语言预训练（VLP）的最小设计。该模型可用于多个下游任务。对于 VQA 任务，在顶部放置一个分类器头（在 `[CLS]` 令牌的最终隐藏状态之上的线性层）并随机初始化。因此，视觉问答被视为一个**分类问题**。

一些较新的模型，如 BLIP、BLIP-2 和 InstructBLIP，将 VQA 视为一个生成任务。在本指南后面，我们将说明如何使用它们进行零样本 VQA 推理。

在开始之前，请确保你已安装所有必要的库。

```
pip install -q transformers datasets
```

我们鼓励你与社区分享你的模型。登录你的 Hugging Face 账户，将其上传到 🤗 Hub。当提示时，输入你的令牌以登录：

```
from huggingface_hub import notebook_login
notebook_login()
```

让我们将模型检查点定义为一个全局变量。

```
model_checkpoint = "dandelin/vilt-b32-mlm"
```

加载数据

为了说明目的，在本指南中，我们使用了一个非常小的注释视觉问答 `Graphcore/vqa` 数据集样本。你可以在 🤗 Hub 上找到完整的数据集。

作为 `Graphcore/vqa` 数据集的替代，你可以手动从官方 VQA 数据集页面下载相同的数据。如果你希望使用自定义数据跟随教程，请查看如何在 🤗 Datasets 文档中创建图像数据集。

让我们从验证拆分中加载前 200 个示例并探索数据集的特征：

```
from datasets import load_dataset
dataset = load_dataset("Graphcore/vqa", split="validation[:200]")
print(dataset)
```

让我们看一下示例，以了解数据集的特征：

```
print(dataset[0])
```

与任务相关的特征包括：

- `question`：从图像中回答的问题
- `image_id`：问题的图像路径
- `label`：注释

我们可以删除其余的特征，因为它们将不再需要：

```
dataset = dataset.remove_columns(['question_type', 'question_id', 'answer_type'])
```

如你所见，`label` 特征包含对同一问题的多个答案（此处称为 `ids`），由不同的注释者收集。这是因为问题的答案可能具有主观性。在这种情况下，问题是“他在看哪里？”。有些人将其注释为“向下”，其他人注释为“在桌子上”，另一个人注释为“滑板”，等等。

看看图像，考虑你会给出什么答案：

```
from PIL import Image
image = Image.open(dataset[0]['image_id'])
image.show()
```

由于问题和答案的模糊性，这样的数据集被视为多标签分类问题（因为多个答案可能有效）。此外，不是简单地创建一个独热编码向量，而是根据某个答案在注释中出现的次数创建软编码。

例如，在上面的示例中，因为答案“向下”被选中的次数远远多于其他答案，所以它的得分（在数据集中称为 `weight`）为 1.0，其余答案的得分小于 1.0。

为了稍后使用适当的分类头实例化模型，让我们创建两个字典：一个将标签名称映射到整数，另一个将整数映射回标签名称：

```
import itertools
labels = [item['ids'] for item in dataset['label']]
flattened_labels = list(itertools.chain(*labels))
unique_labels = list(set(flattened_labels))

label2id = {label: idx for idx, label in enumerate(unique_labels)}
id2label = {idx: label for label, idx in label2id.items()}
```

现在我们有了映射，我们可以将字符串答案替换为它们的 id，并展平数据集以方便进一步预处理。

```
def replace_ids(inputs):
    inputs["label"]["ids"] = [label2id[x] for x in inputs["label"]["ids"]]
    return inputs

dataset = dataset.map(replace_ids)
flat_dataset = dataset.flatten()
print(flat_dataset.features)
```

预处理数据

下一步是加载 ViLT 处理器来准备模型的图像和文本数据。ViltProcessor 将 BERT 令牌器和 ViLT 图像处理器包装成一个方便的单一处理器：

```
from transformers import ViltProcessor
processor = ViltProcessor.from_pretrained(model_checkpoint)
```

为了预处理数据，我们需要使用 ViltProcessor 对图像和问题进行编码。处理器将使用 BertTokenizerFast 对文本进行标记，并为文本数据创建 `input_ids`、`attention_mask` 和 `token_type_ids`。至于图像，处理器将利用 ViltImageProcessor 来调整图像大小和归一化图像，并创建 `pixel_values` 和 `pixel_mask`。

所有这些预处理步骤都在幕后进行，我们只需要调用 `processor`。但是，我们仍然需要准备目标标签。在这种表示中，每个元素对应于一个可能的答案（标签）。对于正确的答案，元素保存它们的相应得分（权重），而其余元素设置为零。

以下函数将 `processor` 应用于图像和问题，并按照上述描述格式化标签：

```
import torch

def preprocess_data(examples):
    image_paths = examples['image_id']
    images = [Image.open(image_path) for image_path in image_paths]
    texts = examples['question']

    encoding = processor(images, texts, padding="max_length", truncation=True,
return_tensors="pt")

    for k, v in encoding.items():
        encoding[k] = v.squeeze()

    targets = []

    for labels, scores in zip(examples['label.ids'], examples['label.weights']):
        target = torch.zeros(len(id2label))

        for label, score in zip(labels, scores):
            target[label] = score

        targets.append(target)

    encoding["labels"] = targets

    return encoding

processed_dataset = flat_dataset.map(preprocess_data, batched=True,
remove_columns=['question','question_type',  'question_id', 'image_id', 'answer_type',
'label.ids', 'label.weights'])
print(processed_dataset)
```

最后一步，使用 DefaultDataCollator 创建一个示例批次：

```
from transformers import DefaultDataCollator
data_collator = DefaultDataCollator()
```

训练模型

现在你已经准备好开始训练你的模型了！使用 ViltForQuestionAnswering 加载 ViLT。指定标签数量以及标签映射：

```
from transformers import ViltForQuestionAnswering
model = ViltForQuestionAnswering.from_pretrained(model_checkpoint, num_labels=len(id2label), id2label=id2label, label2id=label2id)
```

在这一点上，只剩下三个步骤：

208. 在 TrainingArguments 中定义你的训练超参数：

```
from transformers import TrainingArguments
repo_id = "MariaK/vilt_finetuned_200"
training_args = TrainingArguments(
    output_dir=repo_id,
    per_device_train_batch_size=4,
    num_train_epochs=20,
    save_steps=200,
    logging_steps=50,
    learning_rate=5e-5,
    save_total_limit=2,
    remove_unused_columns=False,
    push_to_hub=True,
)
```

209. 将训练参数传递给 Trainer，以及模型、数据集、处理器和数据整理器。

```
from transformers import Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=processed_dataset,
    processing_class=processor,
)
```

210. 调用 train() 来微调你的模型。

```
trainer.train()
```

训练完成后，使用 push_to_hub() 方法将你的模型分享到 Hub，在 🤗 Hub 上分享你的最终模型：

```
trainer.push_to_hub()
```

推理

现在你已经微调了 ViLT 模型，并将其上传到 🤗 Hub，你可以使用它进行推理。尝试你的微调模型进行推理的最简单方法是使用它在一个 Pipeline 中。

```
from transformers import pipeline
pipe = pipeline("visual-question-answering", model="MariaK/vilt_finetuned_200")
```

本指南中的模型仅在 200 个示例上进行训练，所以不要对它期望太高。让我们看看它是否至少从数据中学到了一些东西，并从数据集中取第一个示例来说明推理：

```
example = dataset[0]
image = Image.open(example['image_id'])
question = example['question']
print(question)
print(pipe(image, question, top_k=1))
```

尽管不太自信，但模型确实学到了一些东西。通过更多的示例和更长的训练，你将得到更好的结果！

如果你愿意，你还可以手动复制管道的结果：

211. 取一个图像和问题，使用模型中的处理器为模型准备它们。

212. 将预处理的结果或输入通过模型。

213. 从 logits 中获取最可能的答案的 id，并在 `id2label` 中找到实际的答案。

```
processor = ViltProcessor.from_pretrained("MariaK/vilt_finetuned_200")
image = Image.open(example['image_id'])
question = example['question']
inputs = processor(image, question, return_tensors="pt")
model = ViltForQuestionAnswering.from_pretrained("MariaK/vilt_finetuned_200")
with torch.no_grad():
    outputs = model(**inputs)
logits = outputs.logits
idx = logits.argmax(-1).item()
print("Predicted answer:", model.config.id2label[idx])
```

零样本 VQA

前面的模型将 VQA 视为一个分类任务。一些较新的模型，如 BLIP、BLIP-2 和 InstructBLIP，将 VQA 视为一个生成任务。让我们以 BLIP-2 为例。它引入了一种新的视觉语言预训练范式，其中可以使用任何预训练的视觉编码器和 LLM 的组合（在 BLIP-2 博客文章 中了解更多信息）。这使得在多个视觉语言任务（包括视觉问答）上实现最先进的结果成为可能。

让我们说明如何使用此模型进行 VQA。首先，让我们加载模型。在这里，我们将显式地将模型发送到 GPU（如果可用），在之前的训练中我们不需要这样做，因为 Trainer 会自动处理这个问题：

```
from transformers import AutoProcessor, Blip2ForConditionalGeneration
import torch
processor = AutoProcessor.from_pretrained("Salesforce/blip2-opt-2.7b")
model = Blip2ForConditionalGeneration.from_pretrained("Salesforce/blip2-opt-2.7b",
torch_dtype=torch.float16)
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
```

该模型将图像和文本作为输入，因此让我们使用 VQA 数据集中第一个示例中的确切图像/问题对：

```
example = dataset[0]
image = Image.open(example['image_id'])
question = example['question']
```

要使用 BLIP-2 进行视觉问答任务，文本提示必须遵循特定的格式：`Question: {} Answer:`。

```
prompt = f"Question: {question} Answer:"
```

现在我们需要使用模型的处理器预处理图像/提示，将预处理后的输入通过模型，并解码输出：

```
inputs = processor(image, text=prompt, return_tensors="pt").to(device, torch.float16)
generated_ids = model.generate(**inputs, max_new_tokens=10)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()
print(generated_text)
```

如你所见，该模型识别了人群，以及脸部的方向（向下看），但它似乎忽略了人群在滑板手后面的事实。不过，在无法获得人工注释数据集的情况下，这种方法可以快速产生有用的结果。

生成策略

（自定义）文本生成策略

文本生成是许多自然语言处理任务的核心，例如开放式文本生成、摘要、翻译等。它还在各种混合模式应用中发挥作用，这些应用以文本作为输出，例如语音转文本和视觉转文本。一些可以生成文本的模型包括 GPT2、XLNet、OpenAI GPT、CTRL、TransformerXL、XLM、Bart、T5、GIT 和 Whisper。

查看一些使用 `generate()` 方法为不同任务生成文本输出的示例：

- 文本摘要

- 图像描述
- 音频转录

请注意，`generate` 方法的输入取决于模型的模态。它们由模型的预处理器类返回，例如 AutoTokenizer 或 AutoProcessor。如果一个模型的预处理器创建多种输入，请将所有输入传递给 `generate()`。你可以在相应的模型文档中了解更多关于单个模型预处理器的信息。

选择输出标记以生成文本的过程称为解码，你可以自定义 `generate()` 方法将使用的解码策略。修改解码策略不会改变任何可训练参数的值。然而，它可以对生成输出的质量产生显著影响。它可以帮助减少文本中的重复，并使其更加连贯。

本指南描述了：

- 默认生成配置
- 常见的解码策略及其主要参数
- 将自定义生成配置与你在 🤗 Hub 上微调的模型一起保存和共享

默认文本生成配置

模型的解码策略在其生成配置中定义。当在 pipeline() 内部使用预训练模型进行推理时，模型调用 `PreTrainedModel.generate()` 方法，该方法在幕后应用默认生成配置。当没有与模型一起保存自定义配置时，也使用默认配置。

当你显式加载模型时，可以通过 `model.generation_config` 检查其附带的生成配置：

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
model.generation_config
```

打印 `model.generation_config` 只会显示与默认生成配置不同的值，而不会列出任何默认值。

默认生成配置将输出与输入提示的大小限制为最多 20 个标记，以避免遇到资源限制。默认解码策略是贪婪搜索，这是最简单的解码策略，它选择具有最高概率的标记作为下一个标记。对于许多任务和小输出大小，这工作得很好。然而，当用于生成更长的输出时，贪婪搜索可能会开始产生高度重复的结果。

自定义文本生成

你可以通过将参数及其值直接传递给 `generate` 方法来覆盖任何 `generation_config`：

```
my_model.generate(**inputs, num_beams=4, do_sample=True)
```

即使默认解码策略主要适用于你的任务，你仍然可以调整一些东西。一些常见的调整参数包括：

- `max_new_tokens`：要生成的最大标记数。换句话说，输出序列的大小，不包括提示中的标记。作为使用输出长度作为停止标准的替代方案，你可以选择在完整生成超过一定时间时停止生成。要了解更多信息，请查看 StoppingCriteria。
- `num_beams`：通过指定大于 1 的光束数，你实际上是从贪婪搜索切换到光束搜索。这种策略在每个时间步评估多个假设，并最终选择整个序列具有最高概率的假设。这有助于识别以较低概率初始标记开始的具有较高概率的序列，这些序列会被贪婪搜索忽略。可视化它如何工作 这里。
- `do_sample`：如果设置为 `True`，此参数将启用解码策略，例如多项式采样、光束搜索多项式采样、Top-K 采样和 Top-p 采样。所有这些策略都从整个词汇的概率分布中选择下一个标记，并具有各种策略特定的调整。
- `num_return_sequences`：每个输入要返回的序列候选数。此选项仅适用于支持多个序列候选的解码策略，例如光束搜索和采样的变体。像贪婪搜索和对比搜索这样的解码策略返回单个输出序列。

保存自定义解码策略与你的模型

如果你想要分享你的微调模型以及特定的生成配置，你可以：

- 创建一个 GenerationConfig 类实例
- 指定解码策略参数
- 使用 GenerationConfig.save_pretrained() 保存你的生成配置，确保其 `config_file_name` 参数为空
- 将 `push_to_hub` 设置为 `True` 以将你的配置上传到模型的仓库

```
from transformers import AutoModelForCausalLM, GenerationConfig

model = AutoModelForCausalLM.from_pretrained("my_account/my_model")
generation_config = GenerationConfig(
    max_new_tokens=50, do_sample=True, top_k=50, eos_token_id=model.config.eos_token_id
```

```
)
generation_config.save_pretrained("my_account/my_model", push_to_hub=True)
```

你还可以在单个目录中存储多个生成配置，利用 GenerationConfig.save_pretrained() 中的 `config_file_name` 参数。稍后你可以使用 GenerationConfig.from_pretrained() 实例化它们。如果你想要为单个模型存储多个生成配置（例如，一个用于创意文本生成的采样，一个用于使用光束搜索的摘要），这将非常有用。你必须拥有在模型上添加配置文件的正确 Hub 权限。

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer, GenerationConfig

tokenizer = AutoTokenizer.from_pretrained("google-t5/t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google-t5/t5-small")

translation_generation_config = GenerationConfig(
    num_beams=4,
    early_stopping=True,
    decoder_start_token_id=0,
    eos_token_id=model.config.eos_token_id,
    pad_token=model.config.pad_token_id,
)

# 提示：添加 `push_to_hub=True` 以推送到 Hub
translation_generation_config.save_pretrained("/tmp", "translation_generation_config.json")

# 你可以使用命名生成配置文件来参数化生成
generation_config = GenerationConfig.from_pretrained("/tmp",
"translation_generation_config.json")
inputs = tokenizer("translate English to French: Configuration files are easy to use!",
return_tensors="pt")
outputs = model.generate(**inputs, generation_config=generation_config)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
```

流式传输

`generate()` 支持流式传输，通过其 `streamer` 输入。`streamer` 输入与任何具有以下方法的类实例兼容：`put()` 和 `end()`。在内部，`put()` 用于推送新标记，`end()` 用于标记文本生成的结束。

流式传输类的 API 仍在开发中，将来可能会发生变化。

实际上，你可以为所有目的制作自己的流式传输类！我们也有基本流式传输类可供你使用。例如，你可以使用 TextStreamer 类将 `generate()` 的输出流式传输到你的屏幕，一次一个单词：

```
from transformers import AutoModelForCausalLM, AutoTokenizer, TextStreamer

tok = AutoTokenizer.from_pretrained("openai-community/gpt2")
```

```
model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
inputs = tok(["An increasing sequence: one,"], return_tensors="pt")
streamer = TextStreamer(tok)

# 尽管返回通常的输出，但流式传输器还会将生成的文本打印到 stdout。
_ = model.generate(**inputs, streamer=streamer, max_new_tokens=20)
```

水印

`generate()` 支持通过随机标记一部分标记为“绿色”来对生成的文本进行水印处理。在生成时，“绿色”标记的 logits 将有一个小的“偏差”值添加，从而有更高的生成概率。可以通过计算文本中“绿色”标记的比例并估计人类生成的文本中获得该数量“绿色”标记的统计可能性来检测水印文本。这种水印策略在论文 “On the Reliability of Watermarks for Large Language Models” 中提出。有关水印内部工作原理的更多信息，建议参考该论文。

水印可用于 `transformers` 中的任何生成模型，并且不需要额外的分类模型来检测水印文本。要触发水印，可以直接将带有必要参数的 WatermarkingConfig 传递给 `.generate()` 方法，或者将其添加到 GenerationConfig。稍后可以使用 WatermarkDetector 检测水印文本。

WatermarkDetector 在内部依赖于“绿色”标记的比例，以及生成的文本是否遵循着色模式。这就是为什么建议在提示文本比生成文本长得多时去掉提示文本。当一批中的某个序列比其他行长得多导致其他行被填充时，这也可能产生影响。此外，检测器**必须**使用与生成时相同的 watermark 配置参数进行初始化。

让我们生成一些带水印的文本。在下面的代码片段中，我们将偏差设置为 2.5，这是将添加到“绿色”标记的 logits 的值。生成水印文本后，我们可以直接将其传递给 `WatermarkDetector` 以检查文本是否由机器生成（对于机器生成的输出 `True`，否则输出 `False`）。

```
from transformers import AutoTokenizer, AutoModelForCausalLM, WatermarkDetector,
WatermarkingConfig

model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
tok = AutoTokenizer.from_pretrained("openai-community/gpt2")
tok.pad_token_id = tok.eos_token_id
tok.padding_side = "left"

inputs = tok(["This is the beginning of a long story", "Alice and Bob are"], padding=True,
return_tensors="pt")
input_len = inputs["input_ids"].shape[-1]
```

```
watermarking_config = WatermarkingConfig(bias=2.5, seeding_scheme="selfhash")
out = model.generate(**inputs, watermarking_config=watermarking_config, do_sample=False,
max_length=20)

detector = WatermarkDetector(model_config=model.config, device="cpu",
watermarking_config=watermarking_config)
detection_out = detector(out, return_dict=True)
detection_out.prediction
```

解码策略

`generate()` 参数的某些组合，以及最终的 `generation_config`，可用于启用特定的解码策略。如果你是这方面的新手，我们建议阅读 这篇博客文章，其中说明了常见解码策略的工作原理。

在这里，我们将展示控制解码策略的参数，并说明如何使用它们。

选择给定的解码策略不是你影响 `generate()` 结果的唯一方式。解码策略主要基于 logits，即下一个标记的概率分布，因此选择一个好的 logits 操作策略可以大有帮助！换句话说，除了选择解码策略之外，操纵 logits 是你可以采取的另一个维度。流行的 logits 操作策略包括 `top_p`、`min_p` 和 `repetition_penalty` — 你可以在 GenerationConfig 类中查看完整列表。

贪婪搜索

`generate` 默认使用贪婪搜索解码，因此你不需要传递任何参数来启用它。这意味着参数 `num_beams` 设置为 1 并且 `do_sample=False`。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = "I look forward to"
checkpoint = "distilbert/distilgpt2"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(checkpoint)
outputs = model.generate(**inputs)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

对比搜索

对比搜索解码策略在 2022 年的论文 A Contrastive Framework for Neural Text Generation 中提出。它展示了在生成非重复且连贯的长输出方面的优越结果。要了解对比搜索如何工作，请查看 这篇博客文章。启用和控制对比搜索行为的主要参数是 `penalty_alpha` 和 `top_k`：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

checkpoint = "openai-community/gpt2-large"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint)

prompt = "Hugging Face Company is"
inputs = tokenizer(prompt, return_tensors="pt")

outputs = model.generate(**inputs, penalty_alpha=0.6, top_k=4, max_new_tokens=100)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

多项式采样

与总是选择具有最高概率的标记作为下一个标记的贪婪搜索不同，多项式采样（也称为祖先采样）根据模型给出的整个词汇的概率分布随机选择下一个标记。每个具有非零概率的标记都有被选中的机会，从而减少了重复的风险。

要启用多项式采样，请设置 `do_sample=True` 和 `num_beams=1`。

```
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed
set_seed(0)  # 为了可重复性

checkpoint = "openai-community/gpt2-large"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint)

prompt = "Today was an amazing day because"
inputs = tokenizer(prompt, return_tensors="pt")

outputs = model.generate(**inputs, do_sample=True, num_beams=1, max_new_tokens=100)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

光束搜索解码

与贪婪搜索不同，光束搜索解码在每个时间步保持多个假设，并最终选择整个序列具有最高概率的假设。这有助于识别以较低概率初始标记开始的具有较高概率的序列，这些序列会被贪婪搜索忽略。

![光束搜索解码示意图](https://huggingface.co/spaces/m-ric/beam_search_visualizer)

你可以在 这个交互式演示 中了解光束搜索解码如何工作：输入你的句子，并调整参数以查看解码光束如何变化。

要启用此解码策略，请指定大于 1 的 `num_beams`（即要跟踪的假设数）。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = "It is astonishing how one can"
checkpoint = "openai-community/gpt2-medium"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(checkpoint)

outputs = model.generate(**inputs, num_beams=5, max_new_tokens=50)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

光束搜索多项式采样

顾名思义，这种解码策略结合了光束搜索和多项式采样。你需要指定大于 1 的 `num_beams` 并设置 `do_sample=True` 以使用此解码策略。

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, set_seed
set_seed(0)  # 为了可重复性

prompt = "translate English to German: The house is wonderful."
checkpoint = "google-t5/t5-small"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)

outputs = model.generate(**inputs, num_beams=5, do_sample=True)
tokenizer.decode(outputs[0], skip_special_tokens=True)
```

多样化光束搜索解码

多样化光束搜索解码策略是光束搜索策略的扩展，它允许生成更多样化的光束序列集以供选择。要了解其工作原理，请参阅 Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models。这种方法有三个主要参数：`num_beams`、`num_beam_groups` 和 `diversity_penalty`。多样性惩罚确保输出在组之间是不同的，并且在每个组内使用光束搜索。

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

checkpoint = "google/pegasus-xsum"
prompt = (
    "The Permaculture Design Principles are a set of universal design principles "
    "that can be applied to any location, climate and culture, and they allow us to design "
    "the most efficient and sustainable human habitation and food production systems. "
    "Permaculture is a design system that encompasses a wide variety of disciplines, such "
    "as ecology, landscape design, environmental science and energy conservation, and the "
    "Permaculture design principles are drawn from these various disciplines. Each individual "
    "design principle itself embodies a complete conceptual framework based on sound "
    "scientific principles. When we bring all these separate  principles together, we can "
    "create a design system that both looks at whole systems, the parts that these systems "
    "consist of, and how those parts interact with each other to create a complex, dynamic, "
    "living system. Each design principle serves as a tool that allows us to integrate all "
    "the separate parts of a design, referred to as elements, into a functional, synergistic, "
    "whole system, where the elements harmoniously interact and work together in the most "
    "efficient way possible."
)

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForSeq2SeqLM.from_pretrained(checkpoint)

outputs = model.generate(**inputs, num_beams=5, num_beam_groups=5, max_new_tokens=30,
diversity_penalty=1.0)
tokenizer.decode(outputs[0], skip_special_tokens=True)
```

本指南说明了启用各种解码策略的主要参数。`generate` 方法还有更高级的参数，可以让你对 `generate` 方法的行为有更进一步的掌控。有关可用参数的完整列表，请参阅 API 文档。

投机解码

投机解码（也称为辅助解码）是对上述解码策略的修改，它使用辅助模型（理想情况下是一个更小的模型）来生成几个候选标记。主模型然后在单个前向传递中验证候选标记，从而加快解码过程。如果 `do_sample=True`，则使用 投机解码论文 中引入的带有重新采样的标记验证。辅助解码假设主模型和辅助模型具有相同的分词器，否则，请参阅通用辅助解码。

目前，只有贪婪搜索和采样支持辅助解码，并且辅助解码不支持批量输入。要了解更多关于辅助解码的信息，请查看 这篇博客文章。

通用辅助解码

通用辅助解码（UAD）添加了对主模型和辅助模型具有不同分词器的支持。要使用它，只需使用 `tokenizer` 和 `assistant_tokenizer` 参数传递分词器（见下文）。在内部，主模型输入标记被重新编码为辅助模型标记，然后在辅助编码中生成候选标记，这些候选标记又被重新编码为主模型候选标记。然后按照上述方式继续验证。重新编码步骤涉及将标记 id 解码为文本，然后使用不同的分词器对文本进行编码。由于重新编码标记可能导致分词差异，UAD 找到源编码和目标编码之间的最长公共子序列，以确保新标记包含正确的提示后缀。

要启用辅助解码，请设置 `assistant_model` 参数为模型。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = "Alice and Bob"
checkpoint = "EleutherAI/pythia-1.4b-deduped"
assistant_checkpoint = "EleutherAI/pythia-160m-deduped"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(checkpoint)
assistant_model = AutoModelForCausalLM.from_pretrained(assistant_checkpoint)
outputs = model.generate(**inputs, assistant_model=assistant_model)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

如果主模型和辅助模型具有不同的分词器，请使用通用辅助解码。

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = "Alice and Bob"
checkpoint = "google/gemma-2-9b"
assistant_checkpoint = "double7/vicuna-68m"

assistant_tokenizer = AutoTokenizer.from_pretrained(assistant_checkpoint)
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(checkpoint)
assistant_model = AutoModelForCausalLM.from_pretrained(assistant_checkpoint)
outputs = model.generate(**inputs, assistant_model=assistant_model, tokenizer=tokenizer,
assistant_tokenizer=assistant_tokenizer)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

当使用辅助解码与采样方法时，你可以使用 `temperature` 参数来控制随机性，就像在多项式采样中一样。然而，在辅助解码中，降低温度可能有助于提高延迟。

```
from transformers import AutoModelForCausalLM, AutoTokenizer, set_seed
set_seed(42)  # 为了可重复性

prompt = "Alice and Bob"
checkpoint = "EleutherAI/pythia-1.4b-deduped"
assistant_checkpoint = "EleutherAI/pythia-160m-deduped"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
inputs = tokenizer(prompt, return_tensors="pt")

model = AutoModelForCausalLM.from_pretrained(checkpoint)
assistant_model = AutoModelForCausalLM.from_pretrained(assistant_checkpoint)
outputs = model.generate(**inputs, assistant_model=assistant_model, do_sample=True,
temperature=0.5)
tokenizer.batch_decode(outputs, skip_special_tokens=True)
```

或者，你也可以设置 `prompt_lookup_num_tokens` 以触发基于 n-gram 的辅助解码，而不是基于模型的辅助解码。你可以在这里阅读更多关于它的信息 这里。

DoLa 解码

Decoding by C**o**ntrasting **La**yers (DoLa) 是一种对比解码策略，用于提高 LLM 的事实性和减少幻觉，如 ICLR 2024 年的论文 DoLa: Decoding by Contrasting Layers Improves Factuality in Large Language Models 所述。

DoLa 通过对比从最终层与早期层获得的 logits 之间的差异来实现，从而放大特定于 transformers 层的事实知识。

要在调用 `model.generate` 函数时激活 DoLa 解码，请执行以下两个步骤：

214. 设置 `dola_layers` 参数，它可以是字符串或整数列表。
    - 如果设置为字符串，则可以是 `low` 或 `high` 之一。
    - 如果设置为整数列表，则应该是介于 0 到模型总层数之间的层索引列表。第 0 层是词嵌入，第 1 层是第一个 transformers 层，依此类推。
215. 建议设置 `repetition_penalty = 1.2` 以减少 DoLa 解码中的重复。

以下示例展示了使用 32 层 LLaMA-7B 模型的 DoLa 解码。

```
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed
import torch

tokenizer = AutoTokenizer.from_pretrained("huggyllama/llama-7b")
model = AutoModelForCausalLM.from_pretrained("huggyllama/llama-7b", torch_dtype=torch.float16)
```

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model.to(device)
set_seed(42)

text = "On what date was the Declaration of Independence officially signed?"
inputs = tokenizer(text, return_tensors="pt").to(device)

# Vanilla greddy decoding
vanilla_output = model.generate(**inputs, do_sample=False, max_new_tokens=50)
tokenizer.batch_decode(vanilla_output[:, inputs.input_ids.shape[-1]:], skip_special_tokens=True)

# DoLa decoding with contrasting higher part of layers (layers 16,18,...,30)
dola_high_output = model.generate(**inputs, do_sample=False, max_new_tokens=50,
dola_layers='high')
tokenizer.batch_decode(dola_high_output[:, inputs.input_ids.shape[-1]:],
skip_special_tokens=True)

# DoLa decoding with contrasting specific layers (layers 28 and 30)
dola_custom_output = model.generate(**inputs, do_sample=False, max_new_tokens=50,
dola_layers=[28,30], repetition_penalty=1.2)
tokenizer.batch_decode(dola_custom_output[:, inputs.input_ids.shape[-1]:],
skip_special_tokens=True)
```

理解 dola_layers 参数

`dola_layers` 代表早期层选择中的候选层，如 DoLa 论文中所述。选定的早期层将与最终层进行对比。

将 `dola_layers` 设置为 `'low'` 或 `'high'` 将分别选择较低或较高的层进行对比。

- 对于 `N` 层模型，其中 `N <= 40` 层，`range(0, N // 2, 2)` 和 `range(N // 2, N, 2)` 的层分别用于 `'low'` 和 `'high'` 层。
- 对于 `N > 40` 层的模型，`range(0, 20, 2)` 和 `range(N - 20, N, 2)` 的层分别用于 `'low'` 和 `'high'` 层。
- 如果模型有绑定的词嵌入，我们将跳过词嵌入（第 0 层）并从第 2 层开始，因为从词嵌入中的早期退出将成为恒等函数。
- 将 `dola_layers` 设置为整数列表，用于层索引，以手动指定要对比的层。例如，设置 `dola_layers=[28,30]` 将对比最终层（第 32 层）与第 28 层和第 30 层。

论文建议，对于像 TruthfulQA 这样的简答题任务，对比 `'high'` 层；而对于所有其他长答案推理任务，例如 GSM8K、StrategyQA、FACTOR 和 VicunaQA，对比 `'low'` 层。不建议将 DoLa 应用于像 GPT-2 这样的小模型，因为论文附录 N 中的结果显示效果不佳。

使用缓存优化生成的最佳实践

高效的缓存对于优化各种生成任务（包括文本生成、翻译、摘要和其他基于 Transformer 的应用程序）中模型的性能至关重要。有效的缓存有助于减少计算时间并提高响应速度，特别是在实时或资源密集型应用程序中。

Transformers 支持各种缓存方法，利用“Cache”类来抽象和管理缓存逻辑。本文概述了使用这些类以最大化性能和效率的最佳实践。请查看 API 文档 中所有可用的 `Cache` 类。

什么是缓存以及为什么我们应该关注?

想象一下，你正在与某人交谈，而不是记住之前所说的内容，每次你回应时都必须从头开始。这会很慢且效率低下，对吧？在 Transformer 模型的世界中，有一个类似的概念，那就是键值缓存（KV Cache）的应用。

KV 缓存对于优化自回归模型中的生成是必要的，在这种模型中，模型通过逐个预测文本来生成。这个过程可能很慢，因为模型一次只能生成一个标记，并且每个新的预测都依赖于先前的上下文。这意味着，要在生成中预测第 1000 个标记，你需要来自前 999 个标记的信息，这些信息以对这些标记的表示进行一些矩阵乘法的形式出现。但是，要预测第 1001 个标记，你还需要来自前 999 个标记的相同信息，以及来自第 1000 个标记的额外信息。这就是键值缓存用于优化顺序生成过程的地方，通过存储先前的计算结果以在后续标记中重用，这样它们就不需要再次计算。

更具体地说，键值缓存充当这些生成模型的内存库，模型存储来自先前处理标记的自注意力层的键值对。通过存储这些信息，模型可以避免冗余计算，而是从缓存中检索先前标记的键和值。请注意，缓存只能在推理中使用，在训练时应禁用，否则可能会导致意外的错误。

对于喜欢深入研究的读者

缓存对象如何在注意力机制中工作

当在输入中使用缓存对象时，注意力模块执行几个关键步骤，以无缝地整合过去和现在的信息。

注意力模块将当前键值与存储在缓存中的过去键值连接起来。这导致注意力权重形状为 `(new_tokens_length, past_kv_length + new_tokens_length)`。本质上，过去和当前的键值被组合

起来计算注意力分数，确保模型考虑先前的上下文和新的输入。连接的键值用于计算注意力分数，从而产生形状为 `(new_tokens_length, past_kv_length + new_tokens_length)` 的注意力权重。

因此，当迭代调用 `forward()` 而不是 `generate()` 方法时，确保注意力掩码的形状与过去和当前键值的组合长度相匹配至关重要。注意力掩码的形状应该是 `(batch_size, past_kv_length + new_tokens_length)`。当你调用 `generate()` 方法时，这通常由内部处理。如果你想要使用 Cache 类实现自己的生成循环，请考虑这一点，并为当前和过去标记准备注意力掩码。

编写自己的生成循环时，你需要了解的一个重要概念是 `cache_position`。如果你想要通过调用 `forward()` 来重用已经填充的 Cache 对象，你必须传入一个有效的 `cache_position`，这将指示序列中输入的位置。请注意，`cache_position` 不受填充的影响，并且总是为每个标记添加一个位置。例如，如果键/值缓存包含 10 个标记（无论其中有多少是填充标记），下一个标记的缓存位置应该是 `torch.tensor([10])`。

下面是如何实现自己的生成循环的示例。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, DynamicCache

model_id = "meta-llama/Llama-2-7b-chat-hf"
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.bfloat16,
device_map="cuda:0")
tokenizer = AutoTokenizer.from_pretrained(model_id)

past_key_values = DynamicCache()
messages = [{"role": "user", "content": "Hello, what's your name."}]
inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True,
return_tensors="pt", return_dict=True).to("cuda:0")

generated_ids = inputs.input_ids
cache_position = torch.arange(inputs.input_ids.shape[1], dtype=torch.int64, device="cuda:0")
max_new_tokens = 10

for _ in range(max_new_tokens):
    outputs = model(**inputs, cache_position=cache_position, past_key_values=past_key_values,
use_cache=True)
    # Greedily sample one next token
    next_token_ids = outputs.logits[:, -1:].argmax(-1)
    generated_ids = torch.cat([generated_ids, next_token_ids], dim=-1)

    # Prepare inputs for the next generation step by leaving unprocessed tokens, in our case we
have only one new token
    # and expanding attn mask for the new token, as explained above
    attention_mask = inputs["attention_mask"]
```

```
    attention_mask = torch.cat([attention_mask,
attention_mask.new_ones((attention_mask.shape[0], 1))], dim=-1)
    inputs = {"input_ids": next_token_ids, "attention_mask": attention_mask}
    cache_position = cache_position[-1:] + 1 # add one more position for the next token

print(tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0])
```

使用缓存生成

在 🤗 Transformers 中，我们支持各种缓存类型来优化不同模型和任务中的性能。默认情况下，所有模型都使用缓存生成，其中 ~DynamicCache 类是大多数模型的默认缓存。它允许我们动态地增长缓存大小，通过在生成过程中保存越来越多的键和值。如果你出于某种原因不想使用缓存，你可以在 `generate()` 方法中传入 `use_cache=False`。

请参考下表以了解缓存类型之间的区别，并选择最适合你用例的缓存。建议在调用模型之前初始化的模型应该作为 kwarg 传递给模型。在所有其他情况下，你可以简单地定义所需的 `cache_implementation`，我们负责处理其余的事情。

缓存类型	内存效率	支持 torch.compile()	建议初始化	延迟	长上下文生成
动态缓存	否	否	否	中等	否
静态缓存	否	是	是	高	否
卸载缓存	是	否	否	低	是
卸载静态缓存	否	是	是	高	是
量化缓存	是	否	否	低	是
滑动窗口缓存	否	是	是	高	否
汇缓存	是	否	是	中等	是

这些缓存类可以在生成时通过 `cache_implementation` 参数设置。要了解 `cache_implementation` 标志的可用选项，请参阅 API 文档。现在，让我们详细探讨每种缓存类型，并了解如何使用它们。请注意，下面的示例适用于仅解码器 Transformer 模型。我们也支持特定于模型的缓存类，例如 Mamba 或 Jamba，继续阅读以了解更多细节。

量化缓存

键和值缓存可能占用大量内存，成为 长上下文生成的瓶颈，特别是对于大型语言模型。在使用 `generate()` 时量化缓存可以显著降低内存需求，但代价是速度。

`transformers` 中的 KV 缓存量化很大程度上受到论文 “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache” 的启发，目前支持 ~QuantoQuantizedCache 和 ~HQQQuantizedCache 类。有关内部工作原理的更多信息，请参阅该论文。

要启用键值缓存的量化，需要在 `generation_config` 中指示 `cache_implementation="quantized"`。量化相关的参数应该作为 `dict` 或 ~QuantizedCacheConfig 类的实例传递给 `generation_config`。你必须指示在 ~QuantizedCacheConfig 中使用哪个量化后端，默认是 `quanto`。

如果你使用的是 `quanto` 后端，建议将 `axis-key/axis-value` 参数设置为 `0`；如果你使用的是 `HQQ` 后端，则设置为 `1`。对于其他配置值，请使用默认值，除非你内存不足。在这种情况下，你可能需要考虑减少残差长度。

缓存量化可能在延迟方面有害，如果上下文长度短且 GPU VRAM 足够运行而不需要缓存量化。建议在内存效率和延迟之间寻求平衡。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-chat-hf",
torch_dtype=torch.float16).to("cuda:0")
inputs = tokenizer("I like rock music because", return_tensors="pt").to(model.device)

out = model.generate(**inputs, do_sample=False, max_new_tokens=20,
cache_implementation="quantized", cache_config={"nbits": 4, "backend": "quanto"})
print(tokenizer.batch_decode(out, skip_special_tokens=True)[0])

out = model.generate(**inputs, do_sample=False, max_new_tokens=20)
print(tokenizer.batch_decode(out, skip_special_tokens=True)[0])
```

卸载缓存

与 KV 缓存量化类似，~OffloadedCache 策略旨在减少 GPU VRAM 使用。它通过将大多数层的 KV 缓存移动到 CPU 来实现这一点。当模型的 `forward()` 方法迭代层时，此策略在 GPU 上维护当

前层缓存。同时，它异步地预取下一层缓存，并将前一层缓存发送回 CPU。与 KV 缓存量化不同，此策略总是产生与默认 KV 缓存实现相同的结果。因此，它可以作为替换或备用方案。

根据你的模型和生成任务的特征（上下文大小、生成的标记数、光束数等），你可能会注意到与默认 KV 缓存实现相比，生成吞吐量略有下降。

要启用 KV 缓存卸载，请在 `generation_config` 中传递 `cache_implementation="offloaded"`，或者直接传递给 `generate()` 调用。使用 `cache_implementation="offloaded_static"` 用于卸载静态缓存（也请参阅下面的 卸载静态缓存）。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
ckpt = "microsoft/Phi-3-mini-4k-instruct"

tokenizer = AutoTokenizer.from_pretrained(ckpt)
model = AutoModelForCausalLM.from_pretrained(ckpt, torch_dtype=torch.float16).to("cuda:0")
inputs = tokenizer("Fun fact: The shortest", return_tensors="pt").to(model.device)

out = model.generate(**inputs, do_sample=False, max_new_tokens=23,
cache_implementation="offloaded")
print(tokenizer.batch_decode(out, skip_special_tokens=True)[0])

out = model.generate(**inputs, do_sample=False, max_new_tokens=23)
print(tokenizer.batch_decode(out, skip_special_tokens=True)[0])
```

缓存卸载需要一个 GPU，并且可能比动态 KV 缓存慢。如果你遇到 CUDA 内存不足错误，请使用它。

下面的示例显示了如何将 KV 缓存卸载用作备用策略。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
def resilient_generate(model, *args, **kwargs):
    oom = False
    try:
        return model.generate(*args, **kwargs)
    except torch.cuda.OutOfMemoryError as e:
        print(e)
        print("retrying with cache_implementation='offloaded'")
        oom = True
    if oom:
        torch.cuda.empty_cache()
        kwargs["cache_implementation"] = "offloaded"
        return model.generate(*args, **kwargs)

ckpt = "microsoft/Phi-3-mini-4k-instruct"
tokenizer = AutoTokenizer.from_pretrained(ckpt)
```

```
model = AutoModelForCausalLM.from_pretrained(ckpt, torch_dtype=torch.float16).to("cuda:0")
prompt = ["okay "*1000 + "Fun fact: The most"]
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
beams = { "num_beams": 40, "num_beam_groups": 40, "num_return_sequences": 40,
"diversity_penalty": 1.0, "max_new_tokens": 23, "early_stopping": True, }
out = resilient_generate(model, **inputs, **beams)
responses = tokenizer.batch_decode(out[:,:-28:], skip_special_tokens=True)
```

在拥有 50 GB RAM 的 GPU 上运行此代码将打印:

"""

CUDA out of memory. Tried to allocate 4.83 GiB. GPU

retrying with cache_implementation='offloaded'

"""

然后成功生成 40 个光束。

静态缓存

由于“DynamicCache”会随着每个生成步骤动态增长，它阻止你利用 JIT 优化。~StaticCache 为键和值预先分配一个特定的最大大小，允许你生成到最大长度，而无需修改缓存大小。检查下面的使用示例。

有关带有静态缓存和 JIT 编译的更多示例，请查看 StaticCache & torchcompile

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-chat-hf",
torch_dtype=torch.float16, device_map="auto")
inputs = tokenizer("Hello, my name is", return_tensors="pt").to(model.device)

# simply pass the cache implementation="static"
out = model.generate(**inputs, do_sample=False, max_new_tokens=20,
cache_implementation="static")
tokenizer.batch_decode(out, skip_special_tokens=True)[0]
```

卸载静态缓存

与 ~OffloadedCache 存在用于卸载“DynamicCache”一样，也存在卸载静态缓存。它完全支持 JIT 优化。只需在 `generation_config` 或直接传递给 `generate()` 调用时传递 `cache_implementation="offloaded_static"`。这将使用 ~OffloadedStaticCache 实现。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-chat-hf",
torch_dtype=torch.float16, device_map="auto")
inputs = tokenizer("Hello, my name is", return_tensors="pt").to(model.device)

# simply pass the cache implementation="offloaded_static"
out = model.generate(**inputs, do_sample=False, max_new_tokens=20,
cache_implementation="offloaded_static")
tokenizer.batch_decode(out, skip_special_tokens=True)[0]
```

滑动窗口缓存

顾名思义，这种缓存类型在先前的键和值上实现了一个滑动窗口，只保留最后 `sliding_window` 个标记。它应该与支持滑动窗口注意力的模型（如 Mistral）一起使用。此外，与静态缓存类似，这个缓存也是 JIT 友好的，并且可以使用与静态缓存相同的编译技术。

注意，你只能对支持滑动窗口的模型使用此缓存，例如 Mistral 模型。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, SinkCache

tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.1")
model = AutoModelForCausalLM.from_pretrained("mistralai/Mistral-7B-v0.1",
torch_dtype=torch.float16).to("cuda:0")
inputs = tokenizer("Yesterday I was on a rock concert and.",
return_tensors="pt").to(model.device)

# can be used by passing in cache implementation
out = model.generate(**inputs, do_sample=False, max_new_tokens=30,
cache_implementation="sliding_window")
tokenizer.batch_decode(out, skip_special_tokens=True)[0]
```

汇缓存

汇缓存是在 “Efficient Streaming Language Models with Attention Sinks” 中引入的。它允许你生成长文本序列（根据论文是“无限长度”），而无需任何微调。这是通过智能处理先前的键和值来实现的，具体来说，它保留序列中的一些初始标记，称为“汇标记”。这是基于观察，这些初始标记在生成过程中吸引了大部分注意力分数。在“汇标记”之后出现的标记以滑动窗口的方式被丢弃，只保留最新的 `window_size` 个标记。通过将这些初始标记作为“注意力汇”，模型即使在处理非常长的文本时也能保持稳定的性能，从而丢弃了大部分先前的知识。

与其他缓存类不同，这个类不能通过指示 `cache_implementation` 直接使用。你必须在调用 `generate()` 之前初始化 Cache，如下所示。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, SinkCache

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-chat-hf",
torch_dtype=torch.float16).to("cuda:0")
inputs = tokenizer("This is a long story about unicorns, fairies and magic.",
return_tensors="pt").to(model.device)

# get our cache, specify number of sink tokens and window size
# Note that window size already includes sink tokens, so has to be larger
past_key_values = SinkCache(window_length=256, num_sink_tokens=4)
out = model.generate(**inputs, do_sample=False, max_new_tokens=30,
past_key_values=past_key_values)
tokenizer.batch_decode(out, skip_special_tokens=True)[0]
```

编码器-解码器缓存

~EncoderDecoderCache 是一个包装器，旨在处理编码器-解码器模型的缓存需求。这种缓存类型专门构建来管理自注意力和交叉注意力缓存，确保存储和检索这些复杂模型所需的过去键/值。编码器-解码器缓存的一个很酷的特性是，你可以为编码器和解码器设置不同的缓存类型，具体取决于你的用例。目前这种缓存仅在 Whisper 模型中支持，但我们很快会添加更多模型。

在用法方面，没有什么特别的操作，调用 `generate()` 或 `forward()` 将处理所有事情。

模型特定缓存类

某些模型需要以特定方式存储先前的键、值或状态，上述缓存类无法使用。对于此类情况，我们有几个专门为特定模型设计的缓存类。这些模型只接受它们自己的专用缓存类，并且不支持使用任何其他缓存类型。一些例子包括 ~HybridCache 用于 Gemma2 系列模型或 ~MambaCache 用于 Mamba 架构模型。

使用缓存进行迭代生成

我们已经看到了如何在生成时使用每种缓存类型。如果你想在迭代生成设置中使用缓存，例如在聊天机器人等应用程序中，其中交互涉及多个回合和持续的来回交流，该怎么办？使用缓存进行迭代

生成允许这些系统有效地处理持续的对话，而无需在每一步重新处理整个上下文。但在你开始实现之前，有一些提示你应该知道：

进行迭代生成的一般格式如下。首先，你必须初始化一个你想要的类型的空缓存，然后你可以开始迭代地输入新的提示。对话历史记录和格式的跟踪可以通过聊天模板来完成，更多内容请阅读 chat_templating。

如果你使用的是汇缓存，你必须将输入裁剪到最大长度，因为汇缓存可以生成比其最大窗口长度更长的文本，但它期望第一个输入不超过最大缓存长度。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers.cache_utils import (
    DynamicCache,
    SinkCache,
    StaticCache,
    SlidingWindowCache,
    QuantoQuantizedCache,
    QuantizedCacheConfig,
)

model_id = "meta-llama/Llama-2-7b-chat-hf"
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.bfloat16,
device_map='auto')
tokenizer = AutoTokenizer.from_pretrained(model_id)

user_prompts = ["Hello, what's your name?", "Btw, yesterday I was on a rock concert."]

past_key_values = DynamicCache()
max_cache_length = past_key_values.get_max_length()

messages = []
for prompt in user_prompts:
    messages.append({"role": "user", "content": prompt})
    inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True,
return_tensors="pt", return_dict=True).to(model.device)
    if isinstance(past_key_values, SinkCache):
        inputs = {k: v[:, -max_cache_length:] for k, v in inputs.items()}

    input_length = inputs["input_ids"].shape[1]

    outputs = model.generate(**inputs, do_sample=False, max_new_tokens=256,
past_key_values=past_key_values)
    completion = tokenizer.decode(outputs[0, input_length:], skip_special_tokens=True)
    messages.append({"role": "assistant", "content": completion})

print(messages)
```

重新使用缓存以继续生成

有时你可能想要首先用键/值填充缓存对象以用于某些前缀提示，并多次重新使用它以生成不同的序列。在这种情况下，你可以构建一个 `Cache` 对象来保存指令提示，并多次使用不同的文本序列重新使用它。

```
import copy
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, DynamicCache, StaticCache

model_id = "meta-llama/Llama-2-7b-chat-hf"
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.bfloat16,
device_map="cuda")
tokenizer = AutoTokenizer.from_pretrained(model_id)

# 初始化具有足够大最大长度的 StaticCache（下面的示例中为 1024 个标记）
# 如果更适合你的需求，你也可以初始化一个 DynamicCache
prompt_cache = StaticCache(config=model.config, max_batch_size=1, max_cache_len=1024,
device="cuda", dtype=torch.bfloat16)

INITIAL_PROMPT = "You are a helpful assistant. "
inputs_initial_prompt = tokenizer(INITIAL_PROMPT, return_tensors="pt").to("cuda")
# 这是公共提示缓存，我们需要在没有梯度的情况下运行 forward 来复制
with torch.no_grad():
    prompt_cache = model(**inputs_initial_prompt, past_key_values =
prompt_cache).past_key_values

prompts = ["Help me to write a blogpost about travelling.", "What is the capital of France?"]
responses = []
for prompt in prompts:
    new_inputs = tokenizer(INITIAL_PROMPT + prompt, return_tensors="pt").to("cuda")
    past_key_values = copy.deepcopy(prompt_cache)
    outputs = model.generate(**new_inputs, past_key_values=past_key_values, max_new_tokens=20)
    response = tokenizer.batch_decode(outputs)[0]
    responses.append(response)

print(responses)
```

遗留缓存格式

在引入 `Cache` 对象之前，LLM 的缓存曾经是一个元组的元组，其中包含张量。遗留格式具有动态大小，随着我们生成文本而增长——与 `DynamicCache` 非常相似。如果你的项目依赖于这种遗留格式，你可以无缝地将其转换为 `DynamicCache`，反之亦然。

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, DynamicCache

tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-chat-hf")
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-chat-hf",
torch_dtype=torch.float16, device_map="auto")
inputs = tokenizer("Hello, my name is", return_tensors="pt").to(model.device)

# `return_dict_in_generate=True` 是返回缓存所必需的。`return_legacy_cache` 强制返回的缓存
# 为遗留类型
generation_outputs = model.generate(**inputs, return_dict_in_generate=True,
return_legacy_cache=True, max_new_tokens=5)

# 我们可以将遗留缓存转换为 DynamicCache — 反之亦然。如果你有特定的格式来操作缓存的自定义逻辑，这很有帮助。
cache = DynamicCache.from_legacy_cache(generation_outputs.past_key_values)
legacy_format_cache = cache.to_legacy_cache()
```

提示技术

使用 IDEFICS 大型多模态模型解决图像-文本任务

虽然可以通过微调专门模型来解决特定任务，但最近出现并广受欢迎的一种替代方法是使用大型模型来处理多种任务，而无需微调。例如，大型语言模型可以处理摘要、翻译、分类等自然语言处理任务。这种方法不再局限于单一模态（如文本），在本指南中，我们将展示如何使用一个名为 IDEFICS 的大型多模态模型来解决图像-文本任务。

IDEFICS 是基于 Flamingo 开发的一个开放访问的视觉和语言模型，Flamingo 是由 DeepMind 初始开发的一种最先进的视觉语言模型。该模型接受任意顺序的图像和文本输入，并生成连贯的文本输出。它可以回答有关图像的问题，描述视觉内容，基于多个图像创建故事等。IDEFICS 有两个版本：800 亿参数 和 90 亿参数，两个版本都可在 🤗 Hub 上找到。每个版本还提供了适应对话用例的微调指令版本。

这个模型非常多功能，可以用于各种图像和多模态任务。然而，作为一个大型模型，它需要大量的计算资源和基础设施。你可以根据自己的需求决定这种做法是否比为每个任务微调专门模型更适合。

在本指南中，你将学习如何：

- 加载 IDEFICS 和 加载量化版本的模型
- 使用 IDEFICS 进行：
  - 图像描述
  - 有提示的图像描述
  - 少量样本提示
  - 视觉问答
  - 图像分类
  - 基于图像的文本生成
- 批量模式下运行推理
- 以对话方式使用 IDEFICS

在开始之前，请确保已安装所有必要的库。

```
pip install -q bitsandbytes sentencepiece accelerate transformers
```

要使用非量化版本的模型检查点运行以下示例，你需要至少 20GB 的 GPU 内存。

加载模型

让我们从加载 90 亿参数的模型检查点开始：

```
checkpoint = "HuggingFaceM4/idefics-9b"
```

与其它 Transformers 模型一样，你需要从检查点加载处理器和模型本身。IDEFICS 处理器将 LlamaTokenizer 和 IDEFICS 图像处理器封装到单个处理器中，以便准备文本和图像输入。

```
import torch
from transformers import IdeficsForVisionText2Text, AutoProcessor

processor = AutoProcessor.from_pretrained(checkpoint)
model = IdeficsForVisionText2Text.from_pretrained(checkpoint, torch_dtype=torch.bfloat16,
device_map="auto")
```

将 `device_map` 设置为 `"auto"` 将自动确定如何在现有设备上以最优化的方式加载和存储模型权重。

量化模型

如果高内存 GPU 可用性是一个问题，你可以加载模型的量化版本。要以 4 位精度加载模型和处理器，可以在 `from_pretrained` 方法中传递 `BitsAndBytesConfig`，模型将在加载时动态压缩。

```
import torch
from transformers import IdeficsForVisionText2Text, AutoProcessor, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
)

processor = AutoProcessor.from_pretrained(checkpoint)

model = IdeficsForVisionText2Text.from_pretrained(
    checkpoint,
    quantization_config=quantization_config,
    device_map="auto"
)
```

现在你已经以建议的方式之一加载了模型，让我们继续探索可以使用 IDEFICS 完成的任务。

图像描述

图像描述的任务是为给定图像预测一个描述。一个常见的应用是帮助视障人士导航不同的情境，例如在线浏览图像内容。

为了说明任务，获取一张要描述的图像，例如：

〔图：花丛中的小狗〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-im-captioning.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Hendo Wang。

IDEFICS 接受文本和图像提示。但是，为了描述图像，你不需要向模型提供文本提示，只需提供预处理后的输入图像即可。没有文本提示时，模型将从 BOS（序列开始）标记开始生成文本，从而创建描述。

作为模型的图像输入，你可以使用图像对象（`PIL.Image`）或从其中检索图像的 URL。

```
prompt = [
    "https://images.unsplash.com/photo-1583160247711-2191776b4b91?ixlib=rb-
4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3542&q=8
```

```
0",
]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"],
add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=10, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

在调用 `generate` 时包含 `bad_words_ids` 是一个好主意，以避免在增加 `max_new_tokens` 时出现错误：当模型没有生成图像时，它会尝试生成新的 `<image>` 或 `<fake_token_around_image>` 标记。你可以像本指南中那样即时设置，也可以将其存储在 `GenerationConfig` 中，如 文本生成策略 指南所述。

有提示的图像描述

你可以通过提供文本提示来扩展图像描述，模型将根据图像继续生成文本。让我们再取一张图像来说明：

〔图：夜晚的埃菲尔铁塔〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-prompted-im-captioning.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Denys Nevozhai。

文本和图像提示可以作为单个列表传递给模型的处理器，以创建适当的输入。

```
prompt = [
    "https://images.unsplash.com/photo-1543349689-9a4d426bee8e?ixlib=rb-
4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3501&q=8
0",
    "This is an image of ",
]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"],
add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=10, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

少量样本提示

尽管 IDEFICS 在零样本任务中表现出色，但你的任务可能需要某种特定格式的描述，或者带有其他限制或要求，这增加了任务的复杂性。少量样本提示可以用于上下文学习。通过在提示中提供示例，你可以引导模型生成类似给定示例的结果。

让我们使用前一张埃菲尔铁塔的图像作为模型的示例，并构建一个提示，告诉模型除了学习图像中的对象外，我们还想获得一些有趣的信息。然后，让我们看看是否可以为自由女神像的图像获得相同的响应格式：

〔图：自由女神像〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-few-shot.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Juan Mayobre。

```
prompt = ["User:",
          "https://images.unsplash.com/photo-1543349689-9a4d426bee8e?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3501&q=80",
          "Describe this image.\nAssistant: An image of the Eiffel Tower at night. Fun fact: the Eiffel Tower is the same height as an 81-storey building.\n",
          "User:",
          "https://images.unsplash.com/photo-1524099163253-32b7f0256868?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3387&q=80",
          "Describe this image.\nAssistant:"
          ]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"], add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=30, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

请注意，仅通过一个示例（即 1-shot），模型就学会了如何执行任务。对于更复杂的任务，你可以尝试使用更多的示例（例如 3-shot、5-shot 等）。

视觉问答

视觉问答（VQA）的任务是基于图像回答开放式问题。类似于图像描述，它可以用于辅助技术应用程序，但也可用于教育（对视觉材料进行推理）、客户服务（基于图像回答产品问题）和图像检索。

让我们获取一张新图像用于此任务：

〔图：一对夫妇野餐〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-vqa.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Jarritos Mexican Soda。

你可以通过适当的指令引导模型从图像描述转向视觉问答：

```
prompt = [
    "Instruction: Provide an answer to the question. Use the image to answer.\n",
    "https://images.unsplash.com/photo-1623944889288-cd147dbb517c?ixlib=rb-
4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3540&q=8
0",
    "Question: Where are these people and what's the weather like? Answer:"
]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"],
add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=20, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

图像分类

IDEFICS 能够在未显式训练包含这些类别标签的数据的情况下，将图像分类到不同类别中。给定一个类别列表，利用其图像和文本理解能力，模型可以推断图像可能属于哪个类别。

假设我们有这张蔬菜摊的照片：

〔图：蔬菜摊〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-classification.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Peter Wendt。

我们可以指示模型将图像分类到我们提供的类别之一：

```
categories = ['animals', 'vegetables', 'city landscape', 'cars', 'office']
prompt = [f"Instruction: Classify the following image into a single category from the following list: {categories}.\n",
          "https://images.unsplash.com/photo-1471193945509-9ad0617afabf?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3540&q=80",
          "Category: "
]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"], add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=6, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

在上面的例子中，我们指示模型将图像分类到一个类别中，但你也可以提示模型进行排名分类。

基于图像的文本生成

对于更具创意的应用，你可以使用基于图像的文本生成来根据图像生成文本。这可以用于创建产品描述、广告、场景描述等。

让我们提示 IDEFICS 根据一张简单的红门照片编写一个故事：

〔图：红门前的南瓜〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/tasks/idefics-story-generation.jpg，离线电子书未嵌入，详见原仓库）

照片来源：Craig Tidball。

```
prompt = ["Instruction: Use the image to write a story. \n",
          "https://images.unsplash.com/photo-1517086822157-2b0358e7684a?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=2203&q=80",
          "Story: \n"]

inputs = processor(prompt, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"], add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, num_beams=2, max_new_tokens=200, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
print(generated_text[0])
```

看起来 IDEFICS 注意到了门阶上的南瓜，并讲述了一个万圣节的鬼故事。

对于这种较长的输出，调整文本生成策略会非常有帮助。这可以帮助你显著提高生成输出的质量。请参阅 文本生成策略 以了解更多信息。

批量模式下运行推理

前面的部分展示了 IDEFICS 对单个示例的使用。同样地，你可以通过传递提示列表来为一批示例运行推理：

```
prompts = [
    [
        "https://images.unsplash.com/photo-1543349689-9a4d426bee8e?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3501&q=80",
        "This is an image of ",
    ],
    [
        "https://images.unsplash.com/photo-1623944889288-cd147dbb517c?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3540&q=80",
        "This is an image of ",
    ],
    [
        "https://images.unsplash.com/photo-1471193945509-9ad0617afabf?ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D&auto=format&fit=crop&w=3540&q=80",
        "This is an image of ",
    ],
]

inputs = processor(prompts, return_tensors="pt").to("cuda")
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"], add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, max_new_tokens=10, bad_words_ids=bad_words_ids)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
for i, t in enumerate(generated_text):
    print(f"{i}:\n{t}\n")
```

以对话方式使用 IDEFICS

对于对话用例，你可以在 🤗 Hub 上找到微调的指令版本模型：`HuggingFaceM4/idefics-80b-instruct` 和 `HuggingFaceM4/idefics-9b-instruct`。

这些检查点是在监督和指令微调数据集的混合数据上微调相应基础模型的结果，这提高了下游性能，同时使模型在对话环境中更具可用性。

对话用途的使用和提示与使用基础模型非常相似：

```
import torch
from transformers import IdeficsForVisionText2Text, AutoProcessor

device = "cuda" if torch.cuda.is_available() else "cpu"

checkpoint = "HuggingFaceM4/idefics-9b-instruct"
model = IdeficsForVisionText2Text.from_pretrained(checkpoint,
torch_dtype=torch.bfloat16).to(device)
processor = AutoProcessor.from_pretrained(checkpoint)

prompts = [
    [
        "User: What is in this image?",
        "https://upload.wikimedia.org/wikipedia/commons/8/86/Id%C3%A9fix.JPG",
        "<end_of_utterance>",

        "\nAssistant: This picture depicts Idefix, the dog of Obelix in Asterix and Obelix.
Idefix is running on the ground.<end_of_utterance>",

        "\nUser:",

"https://static.wikia.nocookie.net/asterix/images/2/25/R22b.gif/revision/latest?cb=201108150730
52",
        "And who is that?<end_of_utterance>",

        "\nAssistant:",
    ],
]

# --batched mode
inputs = processor(prompts, add_end_of_utterance_token=False, return_tensors="pt").to(device)
# --single sample mode
# inputs = processor(prompts[0], return_tensors="pt").to(device)

# Generation args
exit_condition = processor.tokenizer("<end_of_utterance>", add_special_tokens=False).input_ids
bad_words_ids = processor.tokenizer(["<image>", "<fake_token_around_image>"],
add_special_tokens=False).input_ids

generated_ids = model.generate(**inputs, eos_token_id=exit_condition,
bad_words_ids=bad_words_ids, max_length=100)
generated_text = processor.batch_decode(generated_ids, skip_special_tokens=True)
for i, t in enumerate(generated_text):
    print(f"{i}:\n{t}\n")
```

LLM 提示指南

大型语言模型，比如 Falcon 和 LLaMA，其实是预先训练好的 Transformer 模型。它们的初衷是预测给定一段文本后，下一个字词会是什么。这些模型拥有数十亿个参数，并且在海量的文本数据上训练了很长时间。正因为如此，它们变得非常强大和多功能。你可以通过用自然语言给出提示，来引导这些模型解决各种自然语言处理（NLP）任务。

为了让这些模型给出最佳结果，设计合适的提示非常重要，这被称为“提示工程”。提示工程是一个不断尝试和改进的过程。自然语言比编程语言更灵活、表达力更强，但也更容易产生歧义。而且，自然语言提示对细节非常敏感，哪怕小小的改动都可能导致完全不同的输出。

虽然没有什么万能的提示方法，但研究人员已经总结出一些最佳实践，帮助大家更稳定地获得好结果。

这本指南会介绍这些提示工程的最佳实践，帮助你制作更有效的 LLM 提示，解决各种 NLP 任务。你将学到：

- 激励的基础：了解如何通过自然语言提示激发大型语言模型的潜力，使其更有效地完成任务。
- LLM 提示的最佳实践：掌握设计和优化提示词的基本原则和技巧，确保模型输出符合预期。
- 高级提示技巧：少样本提示和思维链，深入探讨少样本提示和思维链等高级技巧，提升模型在复杂任务中的表现。
- 何时进行微调而不是提示：明确在什么情况下选择微调模型而不是使用提示词，以获得最佳效果。

提示工程只是优化大型语言模型（LLM）输出的一部分。另一个关键部分是选择最合适的文本生成策略。你可以自定义 LLM 在生成文本时如何选择每个后续的字词，而无需改动任何可训练的参数。通过调整文本生成参数，你可以减少生成文本中的重复内容，使其更加连贯，听起来也更自然。关于文本生成策略和参数的详细内容，本指南不展开讨论，但你可以在以下指南中了解更多信息：

- 使用 LLMs 生成文本
- 文本生成策略

激励的基础

模型的类型

大多数现代大型语言模型（LLMs）都是基于仅解码器的 Transformer 架构。例如：LLaMA、Llama 2、Falcon、GPT-2 等。不过，你也可能会遇到编码器-解码器风格的 Transformer LLMs，比如 Flan-T5 和 BART。

`编码器-解码器`模型通常用于那些输出高度依赖输入的生成任务，比如翻译和摘要。而`仅解码器模型`则适用于其他类型的生成任务。

在使用 pipeline 生成 LLM 文本时，了解你所使用的 LLM 类型非常重要，因为它们使用的 pipeline 不同。

使用文本生成 pipeline 运行仅解码器模型的推理：

```
from transformers import pipeline
import torch

torch.manual_seed(0)
generator = pipeline('text-generation', model = 'openai-community/gpt2')
prompt = "Hello, I'm a language model"

generator(prompt, max_length = 30)
```

要使用`编码器-解码器`模型进行推理，请使用 `text2text-generation`pipeline：

```
text2text_generator = pipeline("text2text-generation", model = 'google/flan-t5-base')
prompt = "Translate from English to French: I'm very happy to see you"

text2text_generator(prompt)
```

基本模型 vs 指令/聊天模型

在🤗 Hub 上提供的最近大型语言模型（LLM）checkpoints 大多有两个版本：**基本版**和**指令版**（或**聊天版**）。例如，tiiuae/falcon-7b 和 tiiuae/falcon-7b-instruct。

基本模型在给定初始提示时，非常擅长完成文本，但它们在需要遵循指令的 NLP 任务或会话使用中并不理想。这时，指令（聊天）版本就发挥作用了。这些 checkpoints 是对预训练的基本版本进行进一步微调，使用了指令和会话数据。这种额外的微调使它们成为许多 NLP 任务的更佳选择。

让我们通过一些简单的提示示例，展示如何使用 tiiuae/falcon-7b-instruct 来解决一些常见的 NLP 任务。

NLP 任务

首先，我们来设置环境：

```
pip install -q transformers accelerate
```

接下来，使用合适的 pipeline（"text-generation"）来加载模型：

```
from transformers import pipeline, AutoTokenizer
import torch

torch.manual_seed(0)
model = "tiiuae/falcon-7b-instruct"

tokenizer = AutoTokenizer.from_pretrained(model)
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
```

Falcon 模型是用 bfloat16 数据类型训练的，所以我们建议你也使用同样的数据类型。这需要较新版本的 CUDA，并且在现代显卡上效果最佳。

现在我们已经通过管道加载了模型，接下来让我们探索如何使用提示来解决 NLP 任务。

文本分类

文本分类中最常见的一种形式是情感分析，它会给一段文本分配一个标签，比如“积极”、“消极”或“中性”。

让我们编写一个提示，指导模型对给定的文本（电影评论）进行分类。我们首先给出指令，然后指定要分类的文本。注意，我们不仅仅停留在这里，还添加了响应的开头——`Sentiment:`：

```
torch.manual_seed(0)
prompt = """Classify the text into neutral, negative or positive.
Text: This movie is definitely one of my favorite movies of its kind. The interaction between respectable and morally strong characters is an ode to chivalry and the honor code amongst thieves and policemen.
Sentiment:
"""
```

```
sequences = pipe(
    prompt,
    max_new_tokens=10,
)

for seq in sequences:
    print(f"Result: {seq['generated_text']}")
```

结果中包含了我们在指令中提供的分类标签列表中的一个，而且这个标签是正确的！

你可能会注意到，除了提示之外，我们还传递了一个`max_new_tokens`参数。这个参数控制模型生成的字词数量，它是许多文本生成参数中的一个，你可以在[《文本生成策略指南》](https://huggingface.co/docs/transformers/main/en/generation_strategies)中了解更多相关信息。

命名实体识别

命名实体识别（NER）是一项在文本中找出命名实体的任务，比如人名、地点或组织。让我们修改提示中的指令，让 LLM 执行这个任务。在这里，我们也设置 `return_full_text = False`，这样输出中就不包含提示内容：

```
torch.manual_seed(1)
prompt = """Return a list of named entities in the text.
Text: The Golden State Warriors are an American professional basketball team based in San Francisco.
Named entities:
"""

sequences = pipe(
    prompt,
    max_new_tokens=15,
    return_full_text = False,
)

for seq in sequences:
    print(f"{seq['generated_text']}")
```

正如你所看到的，模型正确地从给定文本中识别出了两个命名实体。

翻译

另一个 LLM 可以执行的任务是翻译。你可以选择使用编码器-解码器模型来完成这个任务，不过在这里，为了示例的简洁性，我们继续使用 `Falcon-7b-instruct`，它也能做得不错的效果。下面是如何编写一个基本提示，指导模型将一段文本从英语翻译成意大利语：

```
torch.manual_seed(2)
prompt = """Translate the English text to Italian.
Text: Sometimes, I've believed as many as six impossible things before breakfast.
Translation:
"""

sequences = pipe(
    prompt,
    max_new_tokens=20,
    do_sample=True,
    top_k=10,
    return_full_text = False,
)

for seq in sequences:
    print(f"{seq['generated_text']}")
```

在这里，我们添加了 `do_sample=True` 和 `top_k=10`，让模型在生成输出时更加灵活。

文本摘要

与翻译类似，文本摘要也是一个生成任务，输出很大程度上依赖于输入，编码器-解码器模型可能是更好的选择。不过，仅解码器的模型也可以用于这个任务。

之前，我们将指令放在提示的开头。然而，提示的末尾也可以是放置指令的合适位置。通常，将指令放在极端中的某一端会更好。

```
torch.manual_seed(3)
prompt = """Permaculture is a design process mimicking the diversity, functionality and
resilience of natural ecosystems. The principles and practices are drawn from traditional
ecological knowledge of indigenous cultures combined with modern scientific understanding and
technological innovations. Permaculture design provides a framework helping individuals and
communities develop innovative, creative and effective strategies for meeting basic needs while
preparing for and mitigating the projected impacts of climate change.
Write a summary of the above text.
Summary:
"""

sequences = pipe(
    prompt,
```

```
    max_new_tokens=30,
    do_sample=True,
    top_k=10,
    return_full_text = False,
)

for seq in sequences:
    print(f"{seq['generated_text']}")
```

问题回答

对于问题回答任务，我们可以将提示结构化为以下逻辑组件：指令、上下文、问题，以及引导词或短语（`Answer:`），以提示模型开始生成答案：

```
torch.manual_seed(4)
prompt = """Answer the question using the context below.
Context: Gazpacho is a cold soup and drink made of raw, blended vegetables. Most gazpacho
includes stale bread, tomato, cucumbers, onion, bell peppers, garlic, olive oil, wine vinegar,
water, and salt. Northern recipes often include cumin and/or pimentón (smoked sweet paprika).
Traditionally, gazpacho was made by pounding the vegetables in a mortar with a pestle; this
more laborious method is still sometimes used as it helps keep the gazpacho cool and avoids the
foam and silky consistency of smoothie versions made in blenders or food processors.
Question: What modern tool is used to make gazpacho?
Answer:
"""

sequences = pipe(
    prompt,
    max_new_tokens=10,
    do_sample=True,
    top_k=10,
    return_full_text = False,
)

for seq in sequences:
    print(f"Result: {seq['generated_text']}")
```

推理

推理是 LLM 中最困难的任务之一，要取得好结果通常需要应用高级提示技术，比如“思维链”（Chain-of-thought）。

让我们试试看，是否能用一个基本提示让模型对简单的算术任务进行推理：

```
torch.manual_seed(5)
prompt = """There are 5 groups of students in the class. Each group has 4 students. How many
students are there in the class?"""
```

```
sequences = pipe(
    prompt,
    max_new_tokens=30,
    do_sample=True,
    top_k=10,
    return_full_text = False,
)

for seq in sequences:
    print(f"Result: {seq['generated_text']}")
```

结果正确！让我们稍微增加一些复杂性，看看是否还能用基本提示解决问题：

```
torch.manual_seed(6)
prompt = """I baked 15 muffins. I ate 2 muffins and gave 5 muffins to a neighbor. My partner
then bought 6 more muffins and ate 2. How many muffins do we now have?"""

sequences = pipe(
    prompt,
    max_new_tokens=10,
    do_sample=True,
    top_k=10,
    return_full_text = False,
)

for seq in sequences:
    print(f"Result: {seq['generated_text']}")
```

这个答案是错误的，正确答案应该是 12。这种情况可能是由于提示过于简单，也可能是因为模型选择的原因，毕竟我们选择了 Falcon 的最小版本。推理对所有不同大小的模型来说都很困难，但更大的模型可能会表现得更好。

LLM 提示的最佳实践

在本指南的这一部分，我们整理了一些通常能提高提示效果的最佳实践：

216. **选择模型**：选择最新和最强大的模型，它们很可能表现更好。
217. **从简开始**：从一个简单且简短的提示开始，然后逐步迭代。
218. **指令位置**：将指令放在提示的开头或结尾。处理大上下文时，模型会应用各种优化以防止注意力复杂度呈二次方增长。这可能导致模型更关注提示的开头或结尾，而不是中间。
219. **明确分隔**：清晰地将指令与它们应用的文本分开——下一节将详细介绍。
220. **具体描述**：对任务和期望结果进行具体和详细的描述，包括格式、长度、风格、语言等。
221. **避免歧义**：避免使用模糊的描述和指令。

222. **正面指令**：优先使用“做什么”的指令，而不是“不要做什么”的指令。

223. **引导输出**：通过写出第一个词（甚至开始第一句话）来引导输出方向。

224. **高级技术**：使用如少样本提示（Few-shot prompting）和思维链（Chain-of-thought）等高级技术。

225. **测试模型**：用不同模型测试你的提示，以评估其鲁棒性。

226. **版本控制**：对提示进行版本控制和性能跟踪。

高级提示技术

少样本提示（Few-shot prompting）

前面章节中的基本提示是“零样本”提示的例子，也就是说，模型已经得到了指令和上下文，但没有带有解决方案的示例。在指令数据集上微调过的 LLM，通常在这些“零样本”任务上表现不错。然而，你可能会发现你的任务更复杂或更微妙，或许你对输出有一些要求，而模型仅凭指令无法捕捉到。在这种情况下，你可以尝试一种称为少样本提示的技术。

在少样本提示中，我们在提示中提供示例，给模型更多的上下文以提升性能。这些示例使模型能够按照示例中的模式生成输出。

```
torch.manual_seed(0)
prompt = """Text: The first human went into space and orbited the Earth on April 12, 1961.
Date: 04/12/1961
Text: The first-ever televised presidential debate in the United States took place on September
28, 1960, between presidential candidates John F. Kennedy and Richard Nixon.
Date:"""

sequences = pipe(
    prompt,
    max_new_tokens=8,
    do_sample=True,
    top_k=10,
)

for seq in sequences:
    print(f"Result: {seq['generated_text']}")
```

少样本提示的示例

在上面的代码片段中，我们使用了一个示例来向模型展示期望的输出，所以这可以称为“单样本”提示。然而，根据任务的复杂性，你可能需要使用多个示例。

少样本提示的局限性

- **复杂推理任务**：虽然 LLM 可以捕捉示例中的模式，但这种技术在复杂的推理任务上效果不佳。
- **提示长度**：少样本提示需要创建较长的提示。包含大量标记的提示会增加计算和延迟。此外，提示的长度也有上限。
- **意外模式学习**：有时给定多个示例后，模型可能会学习到你并不希望它们学习的模式，例如，第三个电影评论总是负面的。

思维链（Chain-of-thought）

思维链（CoT）提示是一种技术，它促使模型生成中间推理步骤，从而在复杂推理任务上提升结果。

有两种方法引导模型生成推理步骤：

227. **少样本提示**：通过展示带有详细答案的问题示例，向模型展示如何逐步解决问题。
228. **指令引导**：通过添加如“让我们一步步思考”或“深呼吸，一步步解决问题”等短语来指导模型进行推理。

如果我们将 CoT 技术应用到推理部分的示例中，并使用一个更大的模型，比如（tiiuae/falcon-180B-chat），你可以在 HuggingChat 中尝试，我们将在推理结果上看到显著改进：

让我们一步步来看：

229. 你一开始有 15 个松饼。
230. 你吃了 2 个松饼，剩下 13 个。
231. 你给了邻居 5 个松饼，剩下 8 个。
232. 你的伴侣又买了 6 个松饼，总数增加到 14 个。
233. 你的伴侣吃了 2 个松饼，剩下 12 个。

所以，你现在有 12 个松饼。

提示优化与微调对比

通过优化提示，你可以取得很好的结果，但你可能仍在思考是否微调模型对你的情况更有效。以下是一些微调较小模型可能是更佳选择的场景：

234. **领域差异大**：你的领域与 LLM 预训练的内容差异很大，且广泛的提示优化未能带来足够好的结果。

235. **低资源语言**：你需要模型在低资源语言中表现良好。

236. **敏感数据训练**：你需要模型在受严格监管的敏感数据上进行训练。

237. **使用小模型**：由于成本、隐私、基础设施或其他限制，你必须使用小模型。

在上述所有例子中，你需要确保你已经拥有或能够以合理成本轻松获得足够大的领域特定数据集来微调模型。你还需要有足够的时间和资源来微调模型。如果上述情况不符合你的情况，优化提示可能会更有益。

第四部分 开发者指南

使用 Tokenizers 中的分词器

PreTrainedTokenizerFast 依赖于 🤗 Tokenizers 库。从 🤗 Tokenizers 库中获取的分词器可以非常简单地加载到 🤗 Transformers 中。

在详细介绍之前，我们先通过几行代码创建一个简单的分词器：

```
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace

# 创建一个 BPE 分词器，并指定未知令牌 (unk_token) 为 "[UNK]"
tokenizer = Tokenizer(BPE(unk_token="[UNK]"))

# 创建一个 BPE 训练器，并定义特殊令牌
trainer = BpeTrainer(special_tokens=["[UNK]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"])

# 设置预分词器为空白分词器
tokenizer.pre_tokenizer = Whitespace()

# 定义要训练的文件列表
files = [...]
```

```
# 使用训练器训练分词器
tokenizer.train(files, trainer)
```

现在我们已经训练了一个分词器，可以在当前运行时继续使用它，或者将其保存为 JSON 文件以供将来重用。

直接从分词器对象加载

让我们看看如何在 🤗 Transformers 库中利用这个分词器对象。PreTrainedTokenizerFast 类允许通过传递已实例化的分词器对象来轻松实例化：

```
from transformers import PreTrainedTokenizerFast

# 将分词器对象传递给 PreTrainedTokenizerFast
fast_tokenizer = PreTrainedTokenizerFast(tokenizer_object=tokenizer)
```

现在，这个对象可以使用 🤗 Transformers 分词器共享的所有方法！更多详细信息请参阅 分词器页面。

从 JSON 文件加载

为了从 JSON 文件加载分词器，我们首先需要保存分词器：

```
# 将分词器保存为 JSON 文件
tokenizer.save("tokenizer.json")
```

保存文件的路径可以传递给 PreTrainedTokenizerFast 初始化方法，使用 `tokenizer_file` 参数：

```
from transformers import PreTrainedTokenizerFast

# 从 JSON 文件加载分词器
fast_tokenizer = PreTrainedTokenizerFast(tokenizer_file="tokenizer.json")
```

现在，这个对象可以使用 🤗 Transformers 分词器共享的所有方法！更多详细信息请参阅 分词器页面。

使用多语言模型运行推理

在 🤗 Transformers 中有多种多语言模型，它们的推理使用方式与单语言模型有所不同。但并不是所有的多语言模型使用方式都不同。例如，像 google-bert/bert-base-multilingual-uncased 这

样的模型可以像单语言模型一样使用。本指南将向你展示如何使用那些推理使用方式不同的多语言模型。

XLM

XLM 有十个不同的检查点，其中只有一个单语言模型。剩下的九个模型检查点可以分为两类：使用语言嵌入的检查点和不使用语言嵌入的检查点。

XLM 使用语言嵌入

以下 XLM 模型在推理时使用语言嵌入来指定所使用的语言：

- `FacebookAI/xlm-mlm-ende-1024`（掩码语言建模，英德）
- `FacebookAI/xlm-mlm-enfr-1024`（掩码语言建模，英法）
- `FacebookAI/xlm-mlm-enro-1024`（掩码语言建模，英罗）
- `FacebookAI/xlm-mlm-xnli15-1024`（掩码语言建模，XNLI 语言）
- `FacebookAI/xlm-mlm-tlm-xnli15-1024`（掩码语言建模 + 翻译，XNLI 语言）
- `FacebookAI/xlm-clm-enfr-1024`（因果语言建模，英法）
- `FacebookAI/xlm-clm-ende-1024`（因果语言建模，英德）

语言嵌入表示为与传递给模型的 `input_ids` 形状相同的张量。这些张量中的值取决于所使用的语言，并通过分词器的 `lang2id` 和 `id2lang` 属性识别。

在这个例子中，加载 `FacebookAI/xlm-clm-enfr-1024` 检查点（因果语言建模，英法）：

```
import torch
from transformers import XLMTokenizer, XLMWithLMHeadModel

tokenizer = XLMTokenizer.from_pretrained("FacebookAI/xlm-clm-enfr-1024")
model = XLMWithLMHeadModel.from_pretrained("FacebookAI/xlm-clm-enfr-1024")
```

分词器的 `lang2id` 属性显示了该模型支持的语言及其 ID：

```
print(tokenizer.lang2id)
# 输出: {'en': 0, 'fr': 1}
```

接下来，创建一个示例输入：

```
input_ids = torch.tensor([tokenizer.encode("Wikipedia was used to")])  # 批量大小为 1
```

将语言 ID 设置为 `"en"`，并用它定义语言嵌入。语言嵌入是一个填充了 `0` 的张量，因为 `0` 是英语的语言 ID。这个张量的大小应与 `input_ids` 相同。

```
language_id = tokenizer.lang2id["en"]  # 0
langs = torch.tensor([language_id] * input_ids.shape[1])  # torch.tensor([0, 0, 0, ..., 0])

# 将其重塑为（批量大小，序列长度）
langs = langs.view(1, -1)  # 现在形状为 [1，序列长度]（我们有一个批量大小为 1）
```

现在你可以将 `input_ids` 和语言嵌入传递给模型：

```
outputs = model(input_ids, langs=langs)
```

run_generation.py 脚本可以使用 `xlm-clm` 检查点生成带有语言嵌入的文本。

XLM 不使用语言嵌入

以下 XLM 模型在推理时不需要语言嵌入：

- `FacebookAI/xlm-mlm-17-1280`（掩码语言建模，17 种语言）
- `FacebookAI/xlm-mlm-100-1280`（掩码语言建模，100 种语言）

这些模型用于通用句子表示，不同于之前的 XLM 检查点。

BERT

以下 BERT 模型可用于多语言任务：

- `google-bert/bert-base-multilingual-uncased`（掩码语言建模 + 下一句预测，102 种语言）
- `google-bert/bert-base-multilingual-cased`（掩码语言建模 + 下一句预测，104 种语言）

这些模型在推理时不需要语言嵌入。它们应该能够从上下文中识别语言并相应地推理。

XLM-RoBERTa

以下 XLM-RoBERTa 模型可用于多语言任务：

- `FacebookAI/xlm-roberta-base`（掩码语言建模，100 种语言）
- `FacebookAI/xlm-roberta-large`（掩码语言建模，100 种语言）

XLM-RoBERTa 在 100 种语言的 2.5TB 新创建和清理的 CommonCrawl 数据上进行了训练。在分类、序列标注和问答等下游任务上，它相比之前发布的多语言模型如 mBERT 或 XLM 提供了显著的改进。

M2M100

以下 M2M100 模型可用于多语言翻译：

- `facebook/m2m100_418M`（翻译）
- `facebook/m2m100_1.2B`（翻译）

在这个例子中，加载 `facebook/m2m100_418M` 检查点将中文翻译成英文。你可以在分词器中设置源语言：

```
from transformers import M2M100ForConditionalGeneration, M2M100Tokenizer

en_text = "Do not meddle in the affairs of wizards, for they are subtle and quick to anger."
chinese_text = "不要插手巫師的事務，因為他們是微妙的，很快就會發怒."

tokenizer = M2M100Tokenizer.from_pretrained("facebook/m2m100_418M", src_lang="zh")
model = M2M100ForConditionalGeneration.from_pretrained("facebook/m2m100_418M")
```

对文本进行分词：

```
encoded_zh = tokenizer(chinese_text, return_tensors="pt")
```

M2M100 强制目标语言 ID 作为生成的第一个标记以翻译为目标语言。在 `generate` 方法中将 `forced_bos_token_id` 设置为 `en` 以翻译成英文：

```
generated_tokens = model.generate(**encoded_zh, forced_bos_token_id=tokenizer.get_lang_id("en"))
tokenizer.batch_decode(generated_tokens, skip_special_tokens=True)
# 输出: 'Do not interfere with the matters of the witches, because they are delicate and will
soon be angry.'
```

MBart

以下 MBart 模型可用于多语言翻译：

- `facebook/mbart-large-50-one-to-many-mmt`（一到多的多语言机器翻译，50 种语言）
- `facebook/mbart-large-50-many-to-many-mmt`（多到多的多语言机器翻译，50 种语言）
- `facebook/mbart-large-50-many-to-one-mmt`（多到一的多语言机器翻译，50 种语言）

- `facebook/mbart-large-50`（多语言翻译，50 种语言）
- `facebook/mbart-large-cc25`

在这个例子中，加载 `facebook/mbart-large-50-many-to-many-mmt` 检查点将芬兰语翻译成英文。你可以在分词器中设置源语言：

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

en_text = "Do not meddle in the affairs of wizards, for they are subtle and quick to anger."
fi_text = "Älä sekaannu velhojen asioihin, sillä ne ovat hienovaraisia ja nopeasti vihaisia."

tokenizer = AutoTokenizer.from_pretrained("facebook/mbart-large-50-many-to-many-mmt",
src_lang="fi_FI")
model = AutoModelForSeq2SeqLM.from_pretrained("facebook/mbart-large-50-many-to-many-mmt")
```

对文本进行分词：

```
encoded_en = tokenizer(en_text, return_tensors="pt")
```

MBart 强制目标语言 ID 作为生成的第一个标记以翻译为目标语言。在 `generate` 方法中将 `forced_bos_token_id` 设置为 `en` 以翻译成英文：

```
generated_tokens = model.generate(**encoded_en,
forced_bos_token_id=tokenizer.lang_code_to_id["en_XX"])
tokenizer.batch_decode(generated_tokens, skip_special_tokens=True)
# 输出: "Don't interfere with the wizard's affairs, because they are subtle, will soon get
angry."
```

如果你使用的是 `facebook/mbart-large-50-many-to-one-mmt` 检查点，则不需要强制目标语言 ID 作为生成的第一个标记，其他使用方式相同。

创建自定义架构（模型架构）

`AutoClass` 可以自动推断模型架构并下载预训练的配置和权重。通常，我们建议使用 `AutoClass` 来生成与检查点无关的代码。但是，希望对特定模型参数有更多控制的用户可以从一些基础类中创建自定义 🤗 Transformers 模型。这特别有助于那些研究、训练或实验 🤗 Transformers 模型的人。本指南将深入介绍如何在不使用 `AutoClass` 的情况下创建自定义模型。您将学习如何：

- 加载和自定义模型配置。
- 创建模型架构。
- 为文本创建慢速和快速分词器。

- 为视觉任务创建图像处理器。
- 为音频任务创建特征提取器。
- 为多模态任务创建处理器。

配置

配置是指模型的具体属性。每个模型配置都有不同的属性；例如，所有自然语言处理模型都有一些共同的属性，如 `hidden_size`、`num_attention_heads`、`num_hidden_layers` 和 `vocab_size`。这些属性指定了构建模型时要使用的注意力头数量或隐藏层数量。

通过访问 `DistilBertConfig` 来查看 `DistilBERT` 的属性：

```
from transformers import DistilBertConfig

config = DistilBertConfig()
print(config)
```

`DistilBertConfig` 显示了用于构建基础 `DistilBertModel` 的所有默认属性。所有属性都可以自定义，为实验提供了空间。例如，您可以使用 `activation` 参数尝试不同的激活函数，使用 `attention_dropout` 参数调整注意力概率的更高 dropout 比率。

```
my_config = DistilBertConfig(activation="relu", attention_dropout=0.4)
print(my_config)
```

预训练模型的属性可以在 `from_pretrained()` 函数中修改：

```
my_config = DistilBertConfig.from_pretrained("distilbert/distilbert-base-uncased",
activation="relu", attention_dropout=0.4)
```

一旦您对模型配置满意，可以使用 `save_pretrained()` 将其保存。配置文件将以 JSON 文件的形式存储在指定的保存目录中：

```
my_config.save_pretrained(save_directory="./your_model_save_path")
```

要重用配置文件，可以通过 `from_pretrained()` 加载它：

```
my_config = DistilBertConfig.from_pretrained("./your_model_save_path/config.json")
```

您还可以将配置文件保存为字典，甚至只保存自定义配置属性与默认配置属性之间的差异！更多详细信息请参见配置文档。

模型

下一步是创建一个模型。模型（也称为架构）定义了每一层的作用和发生的操作。配置中的属性（如 `num_hidden_layers`）用于定义架构。每个模型都继承自基类 `PreTrainedModel`，并提供了一些常见的方法，如调整输入嵌入的大小和剪枝自注意力头。此外，所有模型都是 `torch.nn.Module`、`tf.keras.Model` 或 `flax.linen.Module` 的子类，这意味着模型与各自框架的使用兼容。

PyTorch

加载自定义配置属性到模型中：

```
from transformers import DistilBertModel

my_config = DistilBertConfig.from_pretrained("./your_model_save_path/config.json")
model = DistilBertModel(my_config)
```

这样创建的模型具有随机值而不是预训练权重。在训练之前，这个模型还不能用于任何有用的任务。训练是一个昂贵且耗时的过程。通常，最好使用预训练模型来更快地获得更好的结果，同时只需要训练所需的一小部分资源。

使用 `from_pretrained()` 创建预训练模型：

```
model = DistilBertModel.from_pretrained("distilbert/distilbert-base-uncased")
```

加载预训练权重时，如果模型由 🤗 Transformers 提供，默认模型配置会自动加载。不过，您仍然可以使用自己的配置属性替换部分或全部默认模型配置属性：

```
model = DistilBertModel.from_pretrained("distilbert/distilbert-base-uncased", config=my_config)
```

TensorFlow

加载自定义配置属性到模型中：

```
from transformers import TFDistilBertModel

my_config = DistilBertConfig.from_pretrained("./your_model_save_path/my_config.json")
tf_model = TFDistilBertModel(my_config)
```

这样创建的模型具有随机值而不是预训练权重。在训练之前，这个模型还不能用于任何有用的任务。训练是一个昂贵且耗时的过程。通常，最好使用预训练模型来更快地获得更好的结果，同时只需要训练所需的一小部分资源。

使用 `from_pretrained()` 创建预训练模型：

```
tf_model = TFDistilBertModel.from_pretrained("distilbert/distilbert-base-uncased")
```

加载预训练权重时，如果模型由 🤗 Transformers 提供，默认模型配置会自动加载。不过，您仍然可以使用自己的配置属性替换部分或全部默认模型配置属性：

```
tf_model = TFDistilBertModel.from_pretrained("distilbert/distilbert-base-uncased",
config=my_config)
```

模型头

此时，您已经有一个输出隐藏状态的基础 DistilBERT 模型。隐藏状态作为输入传递给模型头以产生最终输出。🤗 Transformers 为每个任务提供不同的模型头，前提是模型支持该任务（例如，您不能使用 DistilBERT 进行序列到序列的任务，如翻译）。

PyTorch

例如，`DistilBertForSequenceClassification` 是一个带有序列分类头的基础 DistilBERT 模型。序列分类头是在池化输出顶部的一个线性层。

```
from transformers import DistilBertForSequenceClassification

model = DistilBertForSequenceClassification.from_pretrained("distilbert/distilbert-base-
uncased")
```

通过切换到不同的模型头，可以轻松地将此检查点重用于其他任务。对于问答任务，您可以使用 `DistilBertForQuestionAnswering` 模型头。问答头类似于序列分类头，不同之处在于它是在隐藏状态输出顶部的一个线性层。

```
from transformers import DistilBertForQuestionAnswering

model = DistilBertForQuestionAnswering.from_pretrained("distilbert/distilbert-base-uncased")
```

TensorFlow

例如，`TFDistilBertForSequenceClassification` 是一个带有序列分类头的基础 DistilBERT 模型。序列分类头是在池化输出顶部的一个线性层。

```
from transformers import TFDistilBertForSequenceClassification

tf_model = TFDistilBertForSequenceClassification.from_pretrained("distilbert/distilbert-base-
uncased")
```

通过切换到不同的模型头，可以轻松地将此检查点重用于其他任务。对于问答任务，您可以使用 `TFDistilBertForQuestionAnswering` 模型头。问答头类似于序列分类头，不同之处在于它是在隐藏状态输出顶部的一个线性层。

```
from transformers import TFDistilBertForQuestionAnswering

tf_model = TFDistilBertForQuestionAnswering.from_pretrained("distilbert/distilbert-base-uncased")
```

分词器

在使用模型处理文本数据之前，您需要一个分词器将原始文本转换为张量。🤗 Transformers 支持两种类型的分词器：

- `PreTrainedTokenizer`：Python 实现的分词器。
- `PreTrainedTokenizerFast`：基于 Rust 的 🤗 Tokenizer 库实现的分词器。由于其 Rust 实现，这种分词器在批处理分词时速度更快。快速分词器还提供了额外的方法，如偏移映射，将标记映射回其原始单词或字符。

两种分词器都支持常见方法，如编码和解码、添加新标记和管理特殊标记。

并非每个模型都支持快速分词器。请参见此表格以检查模型是否支持快速分词器。

如果您训练了自己的分词器，可以使用词汇表文件创建一个分词器：

```
from transformers import DistilBertTokenizer

my_tokenizer = DistilBertTokenizer(vocab_file="my_vocab_file.txt", do_lower_case=False, padding_side="left")
```

重要的是要记住，自定义分词器的词汇表与预训练模型的分词器生成的词汇表不同。如果您使用预训练模型，需要使用预训练模型的词汇表，否则输入将没有意义。使用 `DistilBertTokenizer` 类创建一个带有预训练模型词汇表的分词器：

```
from transformers import DistilBertTokenizer

slow_tokenizer = DistilBertTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
```

使用 `DistilBertTokenizerFast` 类创建一个快速分词器：

```
from transformers import DistilBertTokenizerFast

fast_tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert/distilbert-base-uncased")
```

默认情况下，`AutoTokenizer` 会尝试加载快速分词器。您可以通过在 `from_pretrained` 中设置 `use_fast=False` 来禁用此行为。

图像处理器

图像处理器处理视觉输入。它继承自基类 `ImageProcessingMixin`。

要使用图像处理器，请创建一个与所使用模型相关联的图像处理器。例如，如果您使用 `ViT` 进行图像分类，可以创建一个默认的 `ViTImageProcessor`：

```
from transformers import ViTImageProcessor

vit_extractor = ViTImageProcessor()
print(vit_extractor)
```

如果您不需要任何自定义，只需使用 `from_pretrained` 方法加载模型的默认图像处理器参数。

修改 `ViTImageProcessor` 的任何参数以创建自定义图像处理器：

```
from transformers import ViTImageProcessor

my_vit_extractor = ViTImageProcessor(resample="PIL.Image.BOX", do_normalize=False,
image_mean=[0.3, 0.3, 0.3])
print(my_vit_extractor)
```

主干网络

计算机视觉模型由主干网络、颈部和头部组成。主干网络从输入图像中提取特征，颈部组合和增强提取的特征，头部用于主要任务（如对象检测）。首先在模型配置中初始化主干网络，并指定是否要加载预训练权重或随机初始化权重。然后，您可以将模型配置传递给模型头部。

例如，将 `ResNet` 主干网络加载到带有实例分割头部的 `MaskFormer` 模型中：

```
from transformers import MaskFormerConfig, MaskFormerForInstanceSegmentation

# 加载预训练的 ResNet 权重
config = MaskFormerConfig(backbone="microsoft/resnet-50", use_pretrained_backbone=True)  # 主干网络和颈部配置
model = MaskFormerForInstanceSegmentation(config)  # 头部

# 随机初始化 ResNet 主干网络
```

```
config = MaskFormerConfig(backbone="microsoft/resnet-50", use_pretrained_backbone=False)  # 主干网络和颈部配置
model = MaskFormerForInstanceSegmentation(config)  # 头部
```

您也可以单独加载主干网络配置，然后将其传递给模型配置。

```
from transformers import MaskFormerConfig, MaskFormerForInstanceSegmentation, ResNetConfig

backbone_config = ResNetConfig()
config = MaskFormerConfig(backbone_config=backbone_config)
model = MaskFormerForInstanceSegmentation(config)
```

`timm` 模型可以在模型中使用 `use_timm_backbone=True` 或使用 `TimmBackbone` 和 `TimmBackboneConfig` 加载。

使用 `use_timm_backbone=True` 和 `use_pretrained_backbone=True` 加载预训练的 `timm` 主干网络权重：

```
from transformers import MaskFormerConfig, MaskFormerForInstanceSegmentation

config = MaskFormerConfig(backbone="resnet50", use_pretrained_backbone=True, use_timm_backbone=True)  # 主干网络和颈部配置
model = MaskFormerForInstanceSegmentation(config)  # 头部
```

使用 `use_timm_backbone=True` 和 `use_pretrained_backbone=False` 加载随机初始化的 `timm` 主干网络：

```
from transformers import MaskFormerConfig, MaskFormerForInstanceSegmentation

config = MaskFormerConfig(backbone="resnet50", use_pretrained_backbone=False, use_timm_backbone=True)  # 主干网络和颈部配置
model = MaskFormerForInstanceSegmentation(config)  # 头部
```

您还可以单独加载主干网络配置，使用它创建 `TimmBackbone` 或将其传递给模型配置。`timm` 主干网络默认加载预训练权重。使用 `use_pretrained_backbone=False` 加载随机初始化的权重。

```
from transformers import TimmBackboneConfig, TimmBackbone

backbone_config = TimmBackboneConfig("resnet50", use_pretrained_backbone=False)

# 创建主干网络类
backbone = TimmBackbone(config=backbone_config)

# 创建带有 `timm` 主干网络的模型
from transformers import MaskFormerConfig, MaskFormerForInstanceSegmentation
```

```
config = MaskFormerConfig(backbone_config=backbone_config)
model = MaskFormerForInstanceSegmentation(config)
```

特征提取器

特征提取器处理音频输入。它继承自基类 `FeatureExtractionMixin`，并且可能继承自 `SequenceFeatureExtractor` 类以处理音频输入。

要使用特征提取器，请创建一个与所使用模型相关联的特征提取器。例如，如果您使用 `Wav2Vec2` 进行音频分类，可以创建一个默认的 `Wav2Vec2FeatureExtractor`：

```
from transformers import Wav2Vec2FeatureExtractor

w2v2_extractor = Wav2Vec2FeatureExtractor()
print(w2v2_extractor)
```

如果您不需要任何自定义，只需使用 `from_pretrained` 方法加载模型的默认特征提取器参数。

修改 `Wav2Vec2FeatureExtractor` 的任何参数以创建自定义特征提取器：

```
from transformers import Wav2Vec2FeatureExtractor

w2v2_extractor = Wav2Vec2FeatureExtractor(sampling_rate=8000, do_normalize=False)
print(w2v2_extractor)
```

处理器

对于支持多模态任务的模型，🤗 Transformers 提供了一个处理器类，方便地将特征提取器和分词器等处理类封装在一个对象中。例如，让我们使用 `Wav2Vec2Processor` 进行自动语音识别任务（ASR）。ASR 将音频转录为文本，因此您需要一个特征提取器和一个分词器。

创建一个特征提取器来处理音频输入：

```
from transformers import Wav2Vec2FeatureExtractor

feature_extractor = Wav2Vec2FeatureExtractor(padding_value=1.0, do_normalize=True)
```

创建一个分词器来处理文本输入：

```
from transformers import Wav2Vec2CTCTokenizer

tokenizer = Wav2Vec2CTCTokenizer(vocab_file="my_vocab_file.txt")
```

将特征提取器和分词器结合到 `Wav2Vec2Processor` 中：

```
from transformers import Wav2Vec2Processor

processor = Wav2Vec2Processor(feature_extractor=feature_extractor, tokenizer=tokenizer)
```

通过这两个基本类（配置和模型）以及一个额外的预处理类（分词器、图像处理器、特征提取器或处理器），您可以创建 🤗 Transformers 支持的任何模型。这些基类都是可配置的，允许您使用所需的特定属性。您可以轻松地设置模型进行训练或修改现有的预训练模型以进行微调。

创建自定义模型

🤗 Transformers 库设计得非常易于扩展。每个模型都在仓库的一个子文件夹中完全编码，没有任何抽象，因此你可以轻松地复制一个模型文件并根据需要进行调整。

如果你要编写一个全新的模型，从头开始可能会更容易。在这个教程中，我们将展示如何编写一个自定义模型及其配置，以便在 Transformers 中使用，并说明如何与社区分享这个模型（包括它依赖的代码），即使它不在 🤗 Transformers 库中。我们将基于 Transformers 框架扩展功能，加入你自己的钩子和自定义代码。

为了说明这些步骤，我们将以 ResNet 模型为例，通过包装 timm 库 中的 ResNet 类，使其符合 PreTrainedModel 的规范。

编写自定义配置

在深入模型之前，我们先来编写模型的配置。模型的配置对象将包含构建模型所需的所有必要信息。正如我们将在下一节中看到的，模型在初始化时只能接受一个 `config` 对象，因此我们需要确保这个对象尽可能完整。

`transformers` 库中的模型通常遵循这样的约定：它们在 `__init__` 方法中接受一个 `config` 对象，然后将整个 `config` 传递给模型的子层，而不是将 `config` 对象拆分成多个参数分别传递给子层。以这种方式编写模型可以简化代码，确保任何超参数都有一个明确的“唯一来源”，并且也更容易重用 `transformers` 中其他模型的代码。

在我们的示例中，我们将取 ResNet 类的一些参数进行调整。不同的配置将给我们带来不同类型的 ResNets。我们检查一些参数的有效性后，将这些参数存储起来。

```
from transformers import PretrainedConfig
from typing import List

class ResnetConfig(PretrainedConfig):
    model_type = "resnet"

    def __init__(
        self,
        block_type="bottleneck",
        layers: List[int] = [3, 4, 6, 3],
        num_classes: int = 1000,
        input_channels: int = 3,
        cardinality: int = 1,
        base_width: int = 64,
        stem_width: int = 64,
        stem_type: str = "",
        avg_down: bool = False,
        **kwargs,
    ):
        if block_type not in ["basic", "bottleneck"]:
            raise ValueError(f"`block_type` must be 'basic' or 'bottleneck', got {block_type}.")
        if stem_type not in ["", "deep", "deep-tiered"]:
            raise ValueError(f"`stem_type` must be '', 'deep' or 'deep-tiered', got
{stem_type}.")

        self.block_type = block_type
        self.layers = layers
        self.num_classes = num_classes
        self.input_channels = input_channels
        self.cardinality = cardinality
        self.base_width = base_width
        self.stem_width = stem_width
        self.stem_type = stem_type
        self.avg_down = avg_down
        super().__init__(**kwargs)
```

编写自定义配置时有三个重要的点需要注意：

238. 必须继承自 `PretrainedConfig`，

239. 你的 `PretrainedConfig` 的 `__init__` 方法必须接受任何关键字参数（kwargs），

240. 这些 `kwargs` 需要传递给父类的 `__init__` 方法。

继承 `PretrainedConfig` 是为了确保你能获得 🤗 Transformers 库的所有功能，而其他两个约束则是因为 `PretrainedConfig` 有更多的字段，而不仅仅是你设置的那些字段。当使用 `from_pretrained` 方法重新加载配置时，这些字段需要被你的配置接受并传递给父类。

为你的配置定义一个 `model_type`（例如 `model_type="resnet"`）不是强制的，除非你想将你的模型注册到自动类（见最后一节）。

完成这些后，你可以像库中的其他模型配置一样轻松创建和保存你的配置。以下是创建并保存一个 resnet50d 配置的方法：

```
resnet50d_config = ResnetConfig(block_type="bottleneck", stem_width=32, stem_type="deep",
avg_down=True)
resnet50d_config.save_pretrained("custom-resnet")
```

这将在 `custom-resnet` 文件夹中保存一个名为 `config.json` 的文件。然后你可以使用 `from_pretrained` 方法重新加载你的配置：

```
resnet50d_config = ResnetConfig.from_pretrained("custom-resnet")
```

你也可以使用 PretrainedConfig 类的其他方法，比如 push_to_hub() 来直接将配置上传到 Hub。

编写自定义模型

现在我们有了 ResNet 配置，可以继续编写模型了。实际上我们将编写两个模型：一个用于提取一批图像的隐藏特征（类似于 BertModel），另一个适用于图像分类（类似于 BertForSequenceClassification）。

如前所述，为了保持简单，我们只会编写一个松散的模型包装器。在编写这个类之前，我们需要做的是建立一个块类型和实际块类之间的映射。然后，通过将所有内容传递给 `ResNet` 类来定义模型：

```
from transformers import PreTrainedModel
from timm.models.resnet import BasicBlock, Bottleneck, ResNet
from .configuration_resnet import ResnetConfig

BLOCK_MAPPING = {"basic": BasicBlock, "bottleneck": Bottleneck}

class ResnetModel(PreTrainedModel):
    config_class = ResnetConfig

    def __init__(self, config):
        super().__init__(config)
        block_layer = BLOCK_MAPPING[config.block_type]
        self.model = ResNet(
            block_layer,
            config.layers,
            num_classes=config.num_classes,
            in_chans=config.input_channels,
            cardinality=config.cardinality,
```

```
            base_width=config.base_width,
            stem_width=config.stem_width,
            stem_type=config.stem_type,
            avg_down=config.avg_down,
        )

    def forward(self, tensor):
        return self.model.forward_features(tensor)
```

对于将用于图像分类的模型，我们只需更改 `forward` 方法：

```
import torch

class ResnetModelForImageClassification(PreTrainedModel):
    config_class = ResnetConfig

    def __init__(self, config):
        super().__init__(config)
        block_layer = BLOCK_MAPPING[config.block_type]
        self.model = ResNet(
            block_layer,
            config.layers,
            num_classes=config.num_classes,
            in_chans=config.input_channels,
            cardinality=config.cardinality,
            base_width=config.base_width,
            stem_width=config.stem_width,
            stem_type=config.stem_type,
            avg_down=config.avg_down,
        )

    def forward(self, tensor, labels=None):
        logits = self.model(tensor)
        if labels is not None:
            loss = torch.nn.functional.cross_entropy(logits, labels)
            return {"loss": loss, "logits": logits}
        return {"logits": logits}
```

在这两种情况下，注意我们是如何继承自 `PreTrainedModel` 并在初始化时调用父类的 `config`（类似于编写一个普通的 `torch.nn.Module`）。设置 `config_class` 这一行不是强制的，除非你想将你的模型注册到自动类（见最后一节）。

如果你的模型与库中的某个模型非常相似，你可以重用该模型相同的配置。

你的模型可以返回任何你想要的内容，但像我们在 `ResnetModelForImageClassification` 中那样返回一个包含损失（当提供标签时）的字典，可以使你的模型直接在 Trainer 类中使用。使用其他输出格式也是可以的，只要你计划使用自己的训练循环或另一个库进行训练。

现在我们有了模型类，让我们创建一个实例：

```
resnet50d = ResnetModelForImageClassification(resnet50d_config)
```

同样，你可以使用 PreTrainedModel 类的任何方法，比如 save_pretrained() 或 push_to_hub()。我们将在下一节中使用后者，并介绍如何将模型权重和代码一起推送。但首先，让我们加载一些预训练的权重到我们的模型中。

在你自己的用例中，你可能需要在自己的数据上训练自定义模型。为了快速完成本教程，我们将使用预训练的 resnet50d。由于我们的模型只是一个包装器，因此转移这些权重会非常容易：

```
import timm

pretrained_model = timm.create_model("resnet50d", pretrained=True)
resnet50d.model.load_state_dict(pretrained_model.state_dict())
```

现在让我们看看如何确保在调用 save_pretrained() 或 push_to_hub() 时，模型的代码也会被保存。

将带有自定义代码的模型注册到自动类

如果你正在编写一个扩展 😭 Transformers 的库，你可能希望扩展自动类以包含你自己的模型。这与将代码推送到 Hub 不同，因为用户需要导入你的库才能使用自定义模型（而不是从 Hub 自动下载模型代码）。

只要你的配置有一个不同于现有模型类型的 `model_type` 属性，并且你的模型类具有正确的 `config_class` 属性，你就可以将它们添加到自动类中，如下所示：

```
from transformers import AutoConfig, AutoModel, AutoModelForImageClassification

AutoConfig.register("resnet", ResnetConfig)
AutoModel.register(ResnetConfig, ResnetModel)
AutoModelForImageClassification.register(ResnetConfig, ResnetModelForImageClassification)
```

请注意，当你将自定义配置注册到 AutoConfig 时，使用的第一个参数需要与你的自定义配置的 `model_type` 匹配，而在将自定义模型注册到任何自动模型类时，使用的第一个参数需要与这些模型的 `config_class` 匹配。

将代码推送到 Hub

此 API 是实验性的，可能会有一些小幅的破坏性变更。

首先，确保你的模型完全定义在一个 `.py` 文件中。它可以依赖于同一目录中的其他文件的相对导入（目前还不支持子模块）。在我们的示例中，我们将在当前工作目录中的 `resnet_model` 文件夹中定义一个 `modeling_resnet.py` 文件和一个 `configuration_resnet.py` 文件。配置文件包含 `ResnetConfig` 的代码，建模文件包含 `ResnetModel` 和 `ResnetModelForImageClassification` 的代码。

`__init__.py` 可以为空，只是为了确保 Python 检测到 `resnet_model` 可以作为一个模块使用。

如果从库中复制建模文件，你需要将文件顶部的所有相对导入替换为从 `transformers` 包中导入。

请注意，你可以重用（或子类化）现有的配置/模型。

要与社区共享你的模型，请按照以下步骤操作：首先从新创建的文件中导入 ResNet 模型和配置：

```
from resnet_model.configuration_resnet import ResnetConfig
from resnet_model.modeling_resnet import ResnetModel, ResnetModelForImageClassification
```

然后你需要告诉库，当你使用 `save_pretrained` 方法时，希望复制这些对象的代码文件，并正确注册它们到给定的自动类（尤其是对于模型），只需运行：

```
ResnetConfig.register_for_auto_class()
ResnetModel.register_for_auto_class("AutoModel")
ResnetModelForImageClassification.register_for_auto_class("AutoModelForImageClassification")
```

注意，对于配置不需要指定自动类（只有一个自动类 AutoConfig），但对于模型则不同。你的自定义模型可能适合多种不同的任务，因此你需要指定哪个自动类是你的模型的正确类。

如果你希望复制代码文件，请使用 `register_for_auto_class()`。如果你更喜欢从其他仓库使用 Hub 上的代码，则不必调用它。在有多个自动类的情况下，你可以直接修改 `config.json`，使用以下结构：

接下来，我们像之前一样创建配置和模型：

```
resnet50d_config = ResnetConfig(block_type="bottleneck", stem_width=32, stem_type="deep",
avg_down=True)
resnet50d = ResnetModelForImageClassification(resnet50d_config)
```

```
pretrained_model = timm.create_model("resnet50d", pretrained=True)
resnet50d.model.load_state_dict(pretrained_model.state_dict())
```

现在要将模型推送到 Hub，请确保你已登录。可以在终端中运行：

```
huggingface-cli login
```

或者在笔记本中运行：

```
from huggingface_hub import notebook_login

notebook_login()
```

然后你可以将其推送到你自己的命名空间（或你是成员的组织）：

```
resnet50d.push_to_hub("custom-resnet50d")
```

除了模型权重和 JSON 格式的配置外，这还将 `custom-resnet50d` 文件夹中的建模和配置 `.py` 文件复制并上传到 Hub。你可以在 模型库 中查看结果。

有关更多关于推送到 Hub 的方法的信息，请参阅 共享教程。

使用带有自定义代码的模型

你可以使用自动类和 `from_pretrained` 方法来使用任何带有自定义代码文件的配置、模型或分词器。上传到 Hub 的所有文件和代码都会进行恶意软件扫描（更多详细信息请参阅 Hub 安全 文档），但你仍然应该审查模型代码和作者，以避免在机器上执行恶意代码。设置 `trust_remote_code=True` 以使用带有自定义代码的模型：

```
from transformers import AutoModelForImageClassification

model = AutoModelForImageClassification.from_pretrained("sgugger/custom-resnet50d",
trust_remote_code=True)
```

强烈建议传递一个 `revision` 参数并设置一个提交哈希值，以确保模型作者没有更新代码包含恶意的新行（除非你完全信任模型的作者）。

```
commit_hash = "ed94a7c6247d8aedce4647f00f20de6875b5b292"
model = AutoModelForImageClassification.from_pretrained(
    "sgugger/custom-resnet50d", trust_remote_code=True, revision=commit_hash
)
```

请注意，当在 Hub 上浏览模型仓库的提交历史时，有一个按钮可以方便地复制任何提交的哈希值。

聊天模版（Chat Templates）

介绍

大型语言模型（LLMs）的一个越来越常见的应用场景是**聊天**。在聊天上下文中，模型不是继续单个文本字符串（就像标准语言模型那样），而是继续一个由一个或多个**消息**组成的对话，每个消息包括一个**角色**，如“用户”或“助手”，以及消息文本。

与分词化类似，不同的模型对聊天的输入格式有不同的期望。这就是我们添加**聊天模板**功能的原因。聊天模板是分词器的一部分。它们指定了如何将对话（表示为消息列表）转换为模型期望格式的单个可分词字符串。

让我们通过一个使用 `mistralai/Mistral-7B-Instruct-v0.1` 模型的快速示例来具体说明这一点：

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-v0.1")

chat = [
    {"role": "user", "content": "Hello, how are you?"},
    {"role": "assistant", "content": "I'm doing great. How can I help you today?"},
    {"role": "user", "content": "I'd like to show off how chat templating works!"},
]

tokenizer.apply_chat_template(chat, tokenize=False)
```

输出结果：

```
"<s>[INST] Hello, how are you? [/INST]I'm doing great. How can I help you today?</s> [INST] I'd like to show off how chat templating works! [/INST]"
```

注意分词器是如何添加控制标记 `[INST]` 和 `[/INST]` 来表示用户消息的开始和结束（但不包括助手消息！），并将整个聊天内容压缩成一个字符串。如果我们使用 `tokenize=True`（这是默认设置），该字符串也会被分词。

现在，尝试相同的代码，但换成 `HuggingFaceH4/zephyr-7b-beta` 模型，你应该会得到：

Zephyr 和 Mistral-Instruct 都是从同一个基础模型 `Mistral-7B-v0.1` 微调而来的。然而，它们在训练时使用了完全不同的聊天格式。如果没有聊天模板，你将不得不为每个模型编写手动格式化代码，并且很容易犯下影响性能的小错误！聊天模板为你处理了格式化的细节，允许你编写适用于任何模型的通用代码。

如何使用聊天模板?

如上例所示，聊天模板使用起来非常简单。只需构建一个包含 `role` 和 `content` 键的消息列表，然后将其传递给 `apply_chat_template()` 方法。一旦你这样做，你将得到可以直接使用的输出！当使用聊天模板作为模型生成的输入时，使用 `add_generation_prompt=True` 来添加生成提示也是一个好主意。

以下是使用 `Zephyr` 模型准备输入以进行 `model.generate()` 的示例：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

checkpoint = "HuggingFaceH4/zephyr-7b-beta"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint)  # 你可能想在这里使用 bfloat16 并将模型移动到 GPU

messages = [
    {
        "role": "system",
        "content": "You are a friendly chatbot who always responds in the style of a pirate",
    },
    {"role": "user", "content": "How many helicopters can a human eat in one sitting?"},
]
tokenized_chat = tokenizer.apply_chat_template(messages, tokenize=True,
add_generation_prompt=True, return_tensors="pt")
print(tokenizer.decode(tokenized_chat[0]))
```

这将生成 Zephyr 期望的输入格式字符串。

现在我们的输入已经为 Zephyr 正确格式化，我们可以使用模型生成对用户问题的响应：

```
outputs = model.generate(tokenized_chat, max_new_tokens=128)
print(tokenizer.decode(outputs[0]))
```

这将生成：

是否有自动化的聊天管道?

是的，有！我们的文本生成管道支持聊天输入，这使得使用聊天模型变得容易。过去，我们使用专门的“ConversationalPipeline”类，但现在它已被弃用，其功能已合并到 `TextGenerationPipeline` 中。让我们再次尝试 `Zephyr` 示例，但这次使用管道：

```
from transformers import pipeline

pipe = pipeline("text-generation", "HuggingFaceH4/zephyr-7b-beta")
messages = [
    {
        "role": "system",
        "content": "You are a friendly chatbot who always responds in the style of a pirate",
    },
    {"role": "user", "content": "How many helicopters can a human eat in one sitting?"},
]
print(pipe(messages, max_new_tokens=128)[0]['generated_text'][-1])  # 打印助手的响应
```

输出结果：

管道将为你处理所有分词和调用 `apply_chat_template` 的细节——一旦模型有了聊天模板，你只需要初始化管道并传递消息列表！

什么是“生成提示”？

你可能已经注意到 `apply_chat_template` 方法有一个 `add_generation_prompt` 参数。这个参数告诉模板添加标记，指示机器人响应的开始。例如，考虑以下聊天：

```
messages = [
    {"role": "user", "content": "Hi there!"},
    {"role": "assistant", "content": "Nice to meet you!"},
    {"role": "user", "content": "Can I ask a question?"}
]
```

对于使用标准“ChatML”格式的模型，如果没有生成提示，它将如下所示：

```
tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=False)
```

输出结果：

而**有**生成提示时，它将如下所示：

```
tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
```

输出结果：

注意这次我们添加了指示机器人响应开始的标记。这确保了当模型生成文本时，它将写一个机器人响应，而不是做一些意外的事情，比如继续用户的消息。记住，聊天模型仍然是语言模型——它们被训练来继续文本，而聊天只是它们的一种特殊文本！你需要用适当的控制标记来引导它们，以便它们知道它们应该做什么。

并非所有模型都需要生成提示。一些模型，如 LLaMA，在机器人响应之前没有任何特殊标记。在这些情况下，`add_generation_prompt` 参数将没有效果。`add_generation_prompt` 的确切效果将取决于所使用的模板。

“continue_final_message”有什么作用?

当将消息列表传递给 `apply_chat_template` 或 `TextGenerationPipeline` 时，你可以选择格式化聊天，以便模型继续聊天中的最后一条消息，而不是开始一条新消息。这是通过移除指示最后一条消息结束的任何结束标记来实现的，以便模型在开始生成文本时简单地扩展最后一条消息。这对于“预填充”模型的响应很有用。

以下是一个示例：

```
chat = [
    {"role": "user", "content": "Can you format the answer in JSON?"},
    {"role": "assistant", "content": '{"name": "'},
]

formatted_chat = tokenizer.apply_chat_template(chat, tokenize=True, return_dict=True,
continue_final_message=True)
model.generate(**formatted_chat)
```

模型将生成继续 JSON 字符串的文本，而不是开始一条新消息。当你知道你希望模型如何开始其回复时，这种方法可以非常有助于提高模型遵循指令的准确性。

由于 `add_generation_prompt` 添加了开始新消息的标记，而 `continue_final_message` 移除了最后一条消息的结束标记，因此同时使用它们是没有意义的。因此，如果你尝试这样做，将会得到一个错误!

`TextGenerationPipeline` 的默认行为是设置 `add_generation_prompt=True`，以便它开始一条新消息。然而，如果输入聊天中的最后一条消息具有“assistant”角色，它将假设该消息是预填充的，并切换到 `continue_final_message=True`，因为大多数模型不支持连续的助手消息。你可以通过在调用管道时显式传递 `continue_final_message` 参数来覆盖此行为。

我可以在训练中使用聊天模板吗?

是的！这是一个确保聊天模板与模型在训练期间看到的标记匹配的好方法。我们建议你将聊天模板作为数据集的预处理步骤应用。之后，你可以像其他语言模型训练任务一样继续。在训练时，通常应设置 `add_generation_prompt=False`，因为添加的提示助手响应的标记在训练期间没有帮助。让我们看一个示例：

```
from transformers import AutoTokenizer
from datasets import Dataset

tokenizer = AutoTokenizer.from_pretrained("HuggingFaceH4/zephyr-7b-beta")

chat1 = [
    {"role": "user", "content": "Which is bigger, the moon or the sun?"},
    {"role": "assistant", "content": "The sun."}
]
chat2 = [
    {"role": "user", "content": "Which is bigger, a virus or a bacterium?"},
    {"role": "assistant", "content": "A bacterium."}
]

dataset = Dataset.from_dict({"chat": [chat1, chat2]})
dataset = dataset.map(lambda x: {"formatted_chat": tokenizer.apply_chat_template(x["chat"],
tokenize=False, add_generation_prompt=False)})
print(dataset['formatted_chat'][0])
```

输出结果：

从这里开始，只需像使用标准语言建模任务一样继续训练，使用 `formatted_chat` 列。

默认情况下，一些分词器在分词时会添加特殊标记，如 `<bos>` 和 `<eos>`。聊天模板应该已经包含了它们所需的所有特殊标记，因此额外的特殊标记通常是不正确的或重复的，这会损害模型性能。

因此，如果你使用 `apply_chat_template(tokenize=False)` 格式化文本，你应该在稍后分词该文本时设置 `add_special_tokens=False`。如果你使用 `apply_chat_template(tokenize=True)`，则无需担心这一点！

高级：聊天模板的额外输入

`apply_chat_template` 唯一需要的参数是 `messages`。然而，你可以传递任何关键字参数给 `apply_chat_template`，它将在模板内部可访问。这为你提供了很大的自由度，可以使用聊天模板做

很多事情。这些参数的名称或格式没有限制——你可以传递字符串、列表、字典或任何其他你想要的东西。

也就是说，有一些常见的用例，例如传递用于函数调用的工具，或用于检索增强生成的文档。在这些常见情况下，我们有一些关于这些参数名称和格式的建议，这些建议在下面的章节中描述。我们鼓励模型作者使其聊天模板与这种格式兼容，以便在模型之间轻松转移工具调用代码。

高级：工具使用 / 函数调用

“工具使用”LLMs 可以选择在生成答案之前调用外部工具作为函数。当将工具传递给工具使用模型时，你可以简单地将函数列表传递给 `tools` 参数：

```
import datetime

def current_time():
    """获取当前本地时间作为字符串。"""
    return str(datetime.now())

def multiply(a: float, b: float):
    """
    一个乘以两个数字的函数

    参数:
        a: 要乘的第一个数字
        b: 要乘的第二个数字
    """
    return a * b

tools = [current_time, multiply]

model_input = tokenizer.apply_chat_template(
    messages,
    tools=tools
)
```

为了使其正常工作，你应该以如上格式编写你的函数，以便它们可以被正确解析为工具。具体来说，你应该遵循以下规则：

- 函数应该有一个描述性名称
- 每个参数必须有一个类型提示

- 函数必须有一个标准的 Google 风格的文档字符串（即，初始函数描述后跟一个描述参数的 `Args:` 块，除非函数没有任何参数）
- 不要在 `Args:` 块中包含类型。换句话说，写 `a: The first number to multiply`，而不是 `a (int): The first number to multiply`。类型提示应该放在函数头中。
- 函数可以在文档字符串中有一个返回类型和一个 `Returns:` 块。然而，这些是可选的，因为大多数工具使用模型会忽略它们。

将工具结果传递给模型

上面的示例代码足以列出模型可用的工具，但如果模型想要实际使用其中一个工具呢？如果发生这种情况，你应该：

241. 解析模型的输出以获取工具名称和参数。
242. 将模型的工具调用添加到对话中。
243. 使用这些参数调用相应的函数。
244. 将结果添加到对话中。

完整的工具使用示例

让我们逐步完成一个工具使用示例。在这个示例中，我们将使用一个 8B 的 `Hermes-2-Pro` 模型，因为它是撰写本文时在其大小类别中性能最高的工具使用模型之一。如果你有内存，可以考虑使用更大的模型，如 Command-R 或 Mixtral-8x22B，它们也支持工具使用并提供更强的性能。

首先，让我们加载我们的模型和分词器：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

checkpoint = "NousResearch/Hermes-2-Pro-Llama-3-8B"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(checkpoint, torch_dtype=torch.bfloat16,
device_map="auto")
```

接下来，让我们定义一个工具列表：

```
def get_current_temperature(location: str, unit: str) -> float:
    """
    获取指定位置的当前温度。
```

```
    参数:
        location: 要获取温度的位置，格式为 "City, Country"
        unit: 返回温度的单位。 (选项: ["celsius", "fahrenheit"])
    返回:
        指定位置的当前温度，以指定单位表示，为浮点数。
    """
    return 22.0  # 一个真正的函数应该实际获取温度!

def get_current_wind_speed(location: str) -> float:
    """
    获取指定位置的当前风速，单位为 km/h。

    参数:
        location: 要获取风速的位置，格式为 "City, Country"
    返回:
        指定位置的当前风速，单位为 km/h，为浮点数。
    """
    return 6.0  # 一个真正的函数应该实际获取风速!

tools = [get_current_temperature, get_current_wind_speed]
```

现在，让我们为我们的机器人设置一个对话：

```
messages = [
  {"role": "system", "content": "You are a bot that responds to weather queries. You should
reply with the unit used in the queried location."},
  {"role": "user", "content": "Hey, what's the temperature in Paris right now?"}
]
```

现在，让我们应用聊天模板并生成一个响应：

```
inputs = tokenizer.apply_chat_template(messages, tools=tools, add_generation_prompt=True,
return_dict=True, return_tensors="pt")
inputs = {k: v.to(model.device) for k, v in inputs.items()}
out = model.generate(**inputs, max_new_tokens=128)
print(tokenizer.decode(out[0][len(inputs["input_ids"][0]):]))
```

我们得到：

模型已经用有效的参数调用了函数，格式符合函数文档字符串的要求。它推断我们最有可能指的是法国的巴黎，并且它记得，作为国际单位制的故乡，法国的温度应该以摄氏度显示。

输出格式如上所示，是我们在本示例中使用的 `Hermes-2-Pro` 模型的特定格式。其他模型可能会输出不同的工具调用格式，你可能需要在此步骤进行一些手动解析。例如，`Llama-3.1` 模型会输出稍微不同的 JSON，使用 `parameters` 而不是 `arguments`。无论模型输出什么格式，你都应该以如下格式将工具调用添加到对话中，包含 `tool_calls`、`function` 和 `arguments` 键。

接下来，让我们将模型的工具调用添加到对话中。

```
tool_call = {"name": "get_current_temperature", "arguments": {"location": "Paris, France",
"unit": "celsius"}}
messages.append({"role": "assistant", "tool_calls": [{"type": "function", "function":
tool_call}]})
```

如果你熟悉 OpenAI API，你应该注意这里的一个重要区别——`tool_call` 是一个字典，但在 OpenAI API 中它是一个 JSON 字符串。传递字符串可能会导致错误或奇怪的模型行为！

现在我们已经将工具调用添加到对话中，我们可以调用函数并将结果添加到对话中。由于我们只是使用一个总是返回 22.0 的虚拟函数，我们可以直接添加该结果。

```
messages.append({"role": "tool", "name": "get_current_temperature", "content": "22.0"})
```

某些模型架构，特别是 Mistral/Mixtral，还需要在这里使用 `tool_call_id`，它应该是 9 个随机生成的字母数字字符，并分配给工具调用字典的 `id` 键。相同的键也应该分配给工具响应字典的 `tool_call_id` 键，以便工具调用可以与工具响应匹配。因此，对于 Mistral/Mixtral 模型，上面的代码将是：

```
tool_call_id = "9Ae3bDc2F"  # 随机 ID，9 个字母数字字符
tool_call = {"name": "get_current_temperature", "arguments": {"location": "Paris, France",
"unit": "celsius"}}
messages.append({"role": "assistant", "tool_calls": [{"type": "function", "id": tool_call_id,
"function": tool_call}]})

messages.append({"role": "tool", "tool_call_id": tool_call_id, "name":
"get_current_temperature", "content": "22.0"})
```

最后，让我们让助手读取函数输出并继续与用户聊天：

```
inputs = tokenizer.apply_chat_template(messages, tools=tools, add_generation_prompt=True,
return_dict=True, return_tensors="pt")
inputs = {k: v.to(model.device) for k, v in inputs.items()}
out = model.generate(**inputs, max_new_tokens=128)
print(tokenizer.decode(out[0][len(inputs["input_ids"][0]):]))
```

我们得到：

尽管这是一个使用虚拟工具和单次调用的简单演示，但相同的技术适用于多个真实工具和更长的对话。这可以是一种强大的方式，通过实时信息、计算工具（如计算器）或访问大型数据库来扩展对话代理的能力。

理解工具模式

你传递给 `apply_chat_template` 的 `tools` 参数的每个函数都会被转换为 JSON 模式。这些模式随后会被传递给模型聊天模板。换句话说，工具使用模型不会直接看到你的函数，也不会看到函数内部的实际代码。它们关心的是函数的**定义**和它们需要传递给它们的**参数**——它们关心的是工具的功能和如何使用它们，而不是它们的工作原理！你需要读取它们的输出，检测它们是否请求使用工具，将它们的参数传递给工具函数，并将响应返回给聊天。

生成 JSON 模式传递给模板应该是自动且不可见的，只要你的函数遵循上述规范，但如果你遇到问题，或者你只是想更多地控制转换，你可以手动处理转换。以下是一个手动模式转换的示例。

```
from transformers.utils import get_json_schema

def multiply(a: float, b: float):
    """
    一个乘以两个数字的函数

    参数:
        a: 要乘的第一个数字
        b: 要乘的第二个数字
    """
    return a * b

schema = get_json_schema(multiply)
print(schema)
```

这将生成：

```
{
  "type": "function",
  "function": {
    "name": "multiply",
    "description": "A function that multiplies two numbers",
    "parameters": {
      "type": "object",
      "properties": {
        "a": {
          "type": "number",
```

```
          "description": "The first number to multiply"
        },
        "b": {
          "type": "number",
          "description": "The second number to multiply"
        }
      },
      "required": ["a", "b"]
    }
  }
}
```

如果你愿意，你可以编辑这些模式，或者甚至完全不使用 `get_json_schema` 从头开始编写它们。JSON 模式可以直接传递给 `apply_chat_template` 的 `tools` 参数——这为你提供了很大的能力来为更复杂的函数定义精确的模式。但要小心——你的模式越复杂，模型在处理它们时就越容易混淆！我们建议尽可能使用简单的函数签名，将参数（尤其是复杂、嵌套的参数）保持在最低限度。

以下是一个手动定义模式并通过 `apply_chat_template` 直接传递它们的示例：

```
# 一个不带参数的简单函数
current_time = {
  "type": "function",
  "function": {
    "name": "current_time",
    "description": "Get the current local time as a string.",
    "parameters": {
      'type': 'object',
      'properties': {}
    }
  }
}

# 一个带两个数值参数的更完整的函数
multiply = {
  'type': 'function',
  'function': {
    'name': 'multiply',
    'description': 'A function that multiplies two numbers',
    'parameters': {
      'type': 'object',
      'properties': {
        'a': {
          'type': 'number',
          'description': 'The first number to multiply'
        },
        'b': {
          'type': 'number', 'description': 'The second number to multiply'
        }
```

```
      },
      'required': ['a', 'b']
    }
  }
}

model_input = tokenizer.apply_chat_template(
    messages,
    tools = [current_time, multiply]
)
```

高级：检索增强生成

“检索增强生成”或“RAG”LLMs 可以在响应查询之前搜索文档语料库以获取信息。这使得模型能够将其知识库扩展到其有限的上下文大小之外。我们对 RAG 模型的建议是，它们的模板应该接受一个 `documents` 参数。这应该是一个文档列表，其中每个“文档”是一个包含 `title` 和 `contents` 键的单个字典，这两个键都是字符串。由于这种格式比用于工具的 JSON 模式简单得多，因此不需要辅助函数。

以下是一个 RAG 模板实际操作的示例：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

# 加载模型和分词器
model_id = "CohereForAI/c4ai-command-r-v01-4bit"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")
device = model.device  # 获取模型加载的设备

# 定义对话输入
conversation = [
    {"role": "user", "content": "What has Man always dreamed of?"}
]

# 定义用于检索增强生成的文档
documents = [
    {
        "title": "The Moon: Our Age-Old Foe",
        "text": "Man has always dreamed of destroying the moon. In this essay, I shall..."
    },
    {
        "title": "The Sun: Our Age-Old Friend",
        "text": "Although often underappreciated, the sun provides several notable benefits..."
    }
```

```
]

# 使用 RAG 模板对对话和文档进行分词，返回 PyTorch 张量。
input_ids = tokenizer.apply_chat_template(
    conversation=conversation,
    documents=documents,
    chat_template="rag",
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt").to(device)

# 生成响应
gen_tokens = model.generate(
    input_ids,
    max_new_tokens=100,
    do_sample=True,
    temperature=0.3,
)

# 解码并打印生成的文本以及生成提示
gen_text = tokenizer.decode(gen_tokens[0])
print(gen_text)
```

`documents` 输入用于检索增强生成并不广泛支持，许多模型的聊天模板会简单地忽略此输入。

要验证模型是否支持 `documents` 输入，你可以阅读其模型卡，或 `print(tokenizer.chat_template)` 以查看是否使用了 `documents` 键。

支持此功能的一个模型类别是 Cohere 的 Command-R 和 Command-R+，通过其 `rag` 聊天模板。你可以在其模型卡中看到使用此功能的接地生成示例。

高级：聊天模板如何工作?

模型的聊天模板存储在 `tokenizer.chat_template` 属性中。如果没有设置聊天模板，则使用该模型类的默认模板。让我们看一下 `Zephyr` 聊天模板，尽管请注意，这个模板比实际的模板简化了一些!

如果你以前从未见过这个，这是一个 Jinja 模板。Jinja 是一种允许你编写生成文本的简单代码的模板语言。在很多方面，代码和语法类似于 Python。在纯 Python 中，这个模板看起来像这样：

```
for message in messages:
    print(f'<|{message["role"]}|>')
    print(message['content'] + eos_token)
if add_generation_prompt:
    print('<|assistant|>')
```

实际上，模板做了三件事：

245. 对于每条消息，打印角色，用 `<|` 和 `|>` 括起来，如 `<|user|>` 或 `<|assistant|>`。
246. 接下来，打印消息内容，后跟结束标记。
247. 最后，如果设置了 `add_generation_prompt`，打印助手标记，以便模型知道开始生成助手响应。

这是一个相当简单的模板，但 Jinja 为你提供了很大的灵活性来做更复杂的事情！让我们看一个 Jinja 模板，它可以像 LLaMA 那样格式化输入（注意，真正的 LLaMA 模板包括默认系统消息的处理和稍微不同的系统消息处理——不要在你的实际代码中使用这个！）

希望如果你盯着这个看一会儿，你可以看到这个模板在做什么——它根据每条消息的角色添加特定的标记，如 `[INST]` 和 `[/INST]`。用户、助手和系统消息由于它们被包裹的标记而清晰可辨。

高级：添加和编辑聊天模板

如何创建聊天模板?

很简单，只需编写一个 Jinja 模板并设置 `tokenizer.chat_template`。你可能会发现从另一个模型的现有模板开始并根据你的需要进行编辑更容易！例如，我们可以采用上面的 LLaMA 模板，并在助手消息中添加 `[ASST]` 和 `[/ASST]`：

现在，只需设置 `tokenizer.chat_template` 属性。下次你使用 `apply_chat_template()` 时，它将使用你的新模板！此属性将保存在 `tokenizer_config.json` 文件中，因此你可以使用 `push_to_hub()` 将你的新模板上传到 Hub，并确保每个人都在为你的模型使用正确的模板！

```
template = tokenizer.chat_template
template = template.replace("SYS", "SYSTEM")  # 更改系统标记
tokenizer.chat_template = template  # 设置新模板
tokenizer.push_to_hub("model_name")  # 将你的新模板上传到 Hub!
```

使用你的聊天模板的方法 `apply_chat_template()` 被 `TextGenerationPipeline` 类调用，因此一旦你设置了正确的聊天模板，你的模型将自动与 `TextGenerationPipeline` 兼容！

如果你正在为一个聊天模型进行微调，除了设置聊天模板外，你可能还应该将任何新的聊天控制标记作为特殊标记添加到分词器中。特殊标记永远不会被分割，确保你的控制标记总是作为一个整体

处理，而不是被分词成多个部分。你还应该将分词器的 `eos_token` 属性设置为模板中标记助手生成的结束标记。这将确保文本生成工具能够正确判断何时停止生成文本。

为什么有些模型有多个模板?

有些模型为不同的用例使用不同的模板。例如，它们可能为普通聊天使用一个模板，而为工具使用或检索增强生成使用另一个模板。在这些情况下，`tokenizer.chat_template` 是一个字典。这可能会引起一些混淆，因此我们建议尽可能为所有用例使用单个模板。你可以使用 Jinja 语句，如 `if tools is defined` 和 `{% macro %}` 定义，轻松地将多个代码路径包装在一个模板中。

当分词器有多个模板时，`tokenizer.chat_template` 将是一个 `dict`，其中每个键是模板的名称。`apply_chat_template` 方法对某些模板名称有特殊处理：具体来说，它会在大多数情况下查找名为 `default` 的模板，如果找不到则会引发错误。然而，如果用户传递了 `tools` 参数并且存在名为 `tool_use` 的模板，它将使用该模板。要访问其他名称的模板，请将模板名称传递给 `apply_chat_template()` 的 `chat_template` 参数。

我们发现这可能会让用户感到困惑——因此，如果你自己编写模板，我们建议尽可能将所有内容放在一个模板中!

我应该使用什么模板?

当为一个已经训练用于聊天的模型设置模板时，你应该确保模板与模型在训练期间看到的聊天格式完全匹配，否则你可能会遇到性能下降。即使你正在进一步训练模型——如果你保持聊天标记不变，你可能会获得最佳性能。这与分词化非常相似——当你精确匹配训练期间使用的分词化时，通常会获得最佳的推理或微调性能。

如果你从头开始训练一个模型，或者为聊天微调一个基础语言模型，那么你有很大的自由度来选择合适的模板! LLMs 足够聪明，可以处理许多不同的输入格式。一个流行的选择是 `ChatML` 格式，这是许多用例的一个好选择。它看起来像这样:

如果你喜欢这个模板，这里有一个单行代码形式，可以直接复制到你的代码中。单行代码还包括对生成提示的支持，但请注意，它不会添加 BOS 或 EOS 标记! 如果模型需要这些标记，它们不会被 `apply_chat_template` 自动添加——换句话说，文本将以 `add_special_tokens=False` 进行分词。这

是为了避免模板和 `add_special_tokens` 逻辑之间的潜在冲突。如果模型需要特殊标记，请确保将它们添加到模板中！

```
tokenizer.chat_template = "{% if not add_generation_prompt is defined %}{% set
add_generation_prompt = false %}{% endif %}{% for message in messages %}{{'<|im_start|>' +
message['role'] + '\n' + message['content'] + '<|im_end|>' + '\n'}}{% endfor %}{% if
add_generation_prompt %}{{ '<|im_start|>assistant\n' }}{% endif %}"
```

这个模板将每条消息包裹在 `<|im_start|>` 和 `<|im_end|>` 标记中，并将角色作为字符串写入，这允许你在训练时使用灵活的角色。输出看起来像这样：

“user”、“system”和“assistant”角色是聊天的标准角色，我们建议在有意义的情况下使用它们，特别是如果你想让你的模型与 `TextGenerationPipeline` 良好配合。然而，你并不局限于这些角色——模板非常灵活，任何字符串都可以是一个角色。

我想添加一些聊天模板！我应该如何开始？

如果你有任何聊天模型，你应该设置它们的 `tokenizer.chat_template` 属性并使用 `apply_chat_template()` 进行测试，然后将更新后的分词器推送到 Hub。即使你不是模型所有者——如果你使用的模型有一个空的聊天模板，或者仍在使用默认类模板，请打开一个 pull request 到模型仓库，以便正确设置此属性！

一旦设置了属性，就完成了！`tokenizer.apply_chat_template` 现在将正确工作，这意味着它也在 `TextGenerationPipeline` 等地方自动支持！

通过确保模型具有此属性，我们可以确保整个社区能够充分利用开源模型的全部功能。格式不匹配已经困扰该领域太久了，并且默默地损害性能——是时候结束它们了！

高级：模板编写技巧

开始编写 Jinja 模板的最简单方法是查看一些现有的模板。你可以使用 `print(tokenizer.chat_template)` 查看任何聊天模型使用的模板。通常，支持工具使用的模型比其他模型有更复杂的模板——因此，当你刚开始时，它们可能不是一个好的学习例子！你还可以查看 Jinja 文档 以了解一般的 Jinja 格式和语法。

Jinja 模板在 `transformers` 中与在其他地方的 Jinja 模板相同。主要要知道的是，对话历史将在你的模板中作为名为 `messages` 的变量可访问。你可以在模板中像在 Python 中一样访问 `messages`，这意味着你可以使用 `{% for message in messages %}` 循环访问它，或者使用 `{{ messages[0] }}` 访问单个消息。

你还可以使用以下技巧编写干净、高效的 Jinja 模板：

修剪空白

默认情况下，Jinja 会打印块之前或之后的任何空白。这对于聊天模板来说可能是个问题，因为它们通常希望对空白非常精确！为了避免这种情况，我们强烈建议你像这样编写模板：

而不是像这样：

添加 `-` 将去除块之前的任何空白。第二个示例看起来无害，但换行符和缩进可能会出现在输出中，这可能不是你想要的！

特殊变量

在你的模板中，你将可以访问几个特殊变量。最重要的是 `messages`，它包含聊天历史记录作为消息字典的列表。然而，还有其他几个变量。并非每个变量都会在每个模板中使用。最常见的其他变量是：

- `tools` 包含一个 JSON 模式格式的工具列表。如果没有传递工具，则为 `None` 或未定义。
- `documents` 包含一个文档列表，格式为 `{"title": "Title", "contents": "Contents"}`，用于检索增强生成。如果没有传递文档，则为 `None` 或未定义。
- `add_generation_prompt` 是一个布尔值，如果用户请求生成提示，则为 `True`，否则为 `False`。如果设置了此变量，你的模板应该在对话末尾添加助手消息的头部。如果你的模型没有助手消息的特定头部，你可以忽略此标志。
- **特殊标记** 如 `bos_token` 和 `eos_token`。这些从 `tokenizer.special_tokens_map` 中提取。每个模板中可用的确切标记将根据父分词器而有所不同。

你可以将任何 `kwarg` 传递给 `apply_chat_template`，它将在模板中作为变量可访问。通常，我们建议尽量坚持使用上述核心变量，因为如果用户必须编写自定义代码来传递特定于模型的 `kwargs`，模

型的使用将变得更加困难。然而，我们意识到这个领域发展迅速，因此如果你有一个新的用例不适合核心 API，请随意为它使用一个新的 `kwarg`！如果一个新的 `kwarg` 变得普遍，我们可能会将其提升到核心 API 中，并为其创建一个标准、文档化的格式。

可调用函数

在你的模板中，还有一小组可调用函数可用。这些是：

- `raise_exception(msg)`：引发 `TemplateException`。这对于调试很有用，并且可以告诉用户他们正在做一些你的模板不支持的事情。
- `strftime_now(format_str)`：等同于 Python 中的 `datetime.now().strftime(format_str)`。这用于以特定格式获取当前日期/时间，通常包含在系统消息中。

与非 Python Jinja 的兼容性

有多种语言实现了 Jinja。它们通常具有相同的语法，但一个关键区别是，当你在 Python 中编写模板时，你可以使用 Python 方法，例如字符串上的 `.lower()` 或字典上的 `.items()`。如果有人在非 Python 实现中使用你的模板，这将中断。非 Python 实现在部署环境中特别常见，其中 JS 和 Rust 非常流行。

不要惊慌！有一些简单的更改可以确保你的模板在所有 Jinja 实现中兼容：

- 将 Python 方法替换为 Jinja 过滤器。它们通常具有相同的名称，例如 `string.lower()` 变为 `string|lower`，`dict.items()` 变为 `dict|items`。一个值得注意的变化是 `string.strip()` 变为 `string|trim`。查看 Jinja 文档中的内置过滤器列表 了解更多。
- 将 `True`、`False` 和 `None`（这是 Python 特有的）替换为 `true`、`false` 和 `none`。
- 直接渲染字典或列表可能会在其他实现中给出不同的结果（例如，字符串条目可能会从单引号变为双引号）。添加 `tojson` 过滤器可以帮助确保一致性。

编写生成提示

我们在上面提到 `add_generation_prompt` 是一个特殊变量，将在你的模板中可访问，并由用户设置 `add_generation_prompt` 标志控制。如果你的模型期望助手消息的头部，那么你的模板必须支持在 `add_generation_prompt` 设置时添加头部。

以下是一个以 ChatML 风格格式化消息并支持生成提示的模板示例：

助手头部的确切内容将取决于你的特定模型，但它应该始终是**表示助手消息开始的字符串**，以便如果用户使用 `add_generation_prompt=True` 应用你的模板并生成文本，模型将写一个助手响应。还要注意，一些模型不需要生成提示，因为助手消息总是在用户消息之后立即开始。这在 LLaMA 和 Mistral 模型中特别常见，其中助手消息在结束用户消息的 `[/INST]` 标记之后立即开始。在这些情况下，模板可以忽略 `add_generation_prompt` 标志。

生成提示很重要！如果你的模型需要生成提示但在模板中未设置，那么模型生成可能会严重降级，或者模型可能会显示异常行为，如继续最后一条用户消息！

编写和调试更大的模板

当引入此功能时，大多数模板都很小，相当于 Jinja 的“单行脚本”。然而，随着新模型和功能（如工具使用和 RAG）的出现，一些模板可能长达 100 行或更多。在编写这些模板时，最好在文本编辑器中将它们写在一个单独的文件中。你可以轻松地将聊天模板提取到一个文件中：

```
open("template.jinja", "w").write(tokenizer.chat_template)
```

或者将编辑后的模板加载回分词器：

```
tokenizer.chat_template = open("template.jinja").read()
```

额外的奖励是，当你在单独的文件中编写长多行模板时，该文件中的行号将与模板解析或执行错误中的行号完全对应。这将更容易识别问题的来源。

为工具编写模板

尽管聊天模板没有为工具（或任何东西）强制执行特定的 API，但我们建议模板作者尽可能坚持标准 API。聊天模板的重点是允许代码在模型之间可转移，因此偏离标准工具 API 意味着用户将不得不编写自定义代码来使用你的模型的工具。有时这是不可避免的，但通常通过巧妙的模板编写，你可以使标准 API 工作！

下面，我们将列出标准 API 的元素，并给出编写与标准 API 良好配合的模板的技巧。

工具定义

你的模板应该期望 `tools` 变量要么为空（如果没有传递工具），要么是一个 JSON 模式字典的列表。我们的聊天模板方法允许用户将工具作为 JSON 模式或 Python 函数传递，但当传递函数时，我们会自动生成 JSON 模式并将其传递给你的模板。因此，你的模板接收到的 `tools` 变量将始终是一个 JSON 模式列表。以下是一个示例工具 JSON 模式：

```
{
  "type": "function",
  "function": {
    "name": "multiply",
    "description": "A function that multiplies two numbers",
    "parameters": {
      "type": "object",
      "properties": {
        "a": {
          "type": "number",
          "description": "The first number to multiply"
        },
        "b": {
          "type": "number",
          "description": "The second number to multiply"
        }
      },
      "required": ["a", "b"]
    }
  }
}
```

以下是一些在你的聊天模板中处理工具的示例代码。记住，这只是特定格式的示例——你的模型可能需要不同的格式！

你的模板渲染的特定标记和工具描述当然应该与你的模型训练时使用的格式匹配。没有要求你的**模型**理解 JSON 模式输入，只有你的模板可以将 JSON 模式转换为你的模型的格式。例如，Command-R 在训练时使用 Python 函数头定义工具，但 Command-R 工具模板接受 JSON 模式，内部转换类型并将输入工具渲染为 Python 头。你可以用模板做很多事情！

工具调用

如果存在，工具调用将是一个附加到具有“assistant”角色的消息的列表。注意 `tool_calls` 总是一个列表，尽管大多数工具调用模型一次只支持单个工具调用，这意味着列表通常只有一个元素。以下是一个包含工具调用的示例消息字典：

```
{
  "role": "assistant",
  "tool_calls": [
    {
      "type": "function",
      "function": {
        "name": "multiply",
        "arguments": {
          "a": 5,
          "b": 6
        }
      }
    }
  ]
}
```

处理它们的常见模式如下：

再次，你应该使用你的模型期望的格式和特殊标记渲染工具调用。

工具响应

工具响应有一个简单的格式：它们是一个包含“tool”角色的消息字典，一个给出调用函数名称的“name”键，以及一个包含工具调用结果的“content”键。以下是一个示例工具响应：

```
{
  "role": "tool",
  "name": "multiply",
  "content": "30"
}
```

你不需要在工具响应中使用所有键。例如，如果你的模型不期望在工具响应中包含函数名称，那么渲染它可以像这样简单：

再次，记住实际的格式和特殊标记是特定于模型的——你应该非常小心，确保标记、空白和其他一切都与你的模型训练时使用的格式完全匹配！

Trainer 类（完整实现 PyTorch 训练与评估循环）

Trainer 是 Transformers 库中实现的一个完整的 PyTorch 模型训练和评估循环。你只需要传递训练所需的各个部分（模型、分词器、数据集、评估函数、训练超参数等），Trainer 类会处理其余所有事情。这使得你可以更快地开始训练，而无需手动编写自己的训练循环。同时，Trainer 也非常可定制，提供了大量的训练选项，可以根据你的具体训练需求进行调整。

除了 Trainer 类，Transformers 还提供了 Seq2SeqTrainer 类，用于翻译或摘要等序列到序列任务。此外，TRL 库中的 SFTTrainer 类包装了 Trainer 类，并针对使用自回归技术训练像 Llama-2 和 Mistral 这样的语言模型进行了优化。SFTTrainer 还支持序列打包、LoRA、量化和 DeepSpeed 等功能，以便高效地扩展到任何模型大小。

你可以查看 API 参考 了解更多关于这些 Trainer 类型类的信息，了解何时使用哪一个。一般来说，Trainer 是最通用的选项，适用于广泛的多种任务。Seq2SeqTrainer 专为序列到序列任务设计，而 SFTTrainer 则专为训练语言模型设计。

在开始之前，请确保安装了 Accelerate 库，这是一个用于在分布式环境中启用和运行 PyTorch 训练的库。

```
pip install accelerate

# 升级
pip install accelerate --upgrade
```

本指南概述了 Trainer 类。

[](basic-usage)基本用法

Trainer 包含了你在基本训练循环中会找到的所有代码：

248. 执行训练步骤以计算损失
249. 使用 `backward` 方法计算梯度
250. 根据梯度更新权重
251. 重复此过程，直到达到预定的训练轮数

Trainer 类抽象了所有这些代码，因此你不必每次都手动编写训练循环，尤其是当你刚开始使用 PyTorch 进行训练时。你只需要提供训练所需的基本组件，如模型和数据集，Trainer 类会处理其余所有事情。

如果你想指定任何训练选项或超参数，可以在 TrainingArguments 类中找到它们。例如，让我们定义模型保存的位置 `output_dir`，并在训练后将模型推送到 Hub 上，设置 `push_to_hub=True`。

```
from transformers import TrainingArguments

training_args = TrainingArguments(
```

```
    output_dir="your-model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    push_to_hub=True,
)
```

将 `training_args` 传递给 Trainer，同时传递模型、数据集、用于预处理数据集的对象（根据数据类型可能是分词器、特征提取器或图像处理器）、数据整理器，以及在训练期间要跟踪的指标计算函数。

最后，调用 train() 开始训练！

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

[](checkpoints)检查点

Trainer 类会将模型检查点保存到 TrainingArguments 中 `output_dir` 参数指定的目录中。你会在一个名为 `checkpoint-000` 的子文件夹中找到这些检查点，文件夹末尾的数字对应于训练步骤。保存检查点对于稍后恢复训练非常有用。

```
# 从最新的检查点恢复
trainer.train(resume_from_checkpoint=True)

# 从输出目录中保存的特定检查点恢复
trainer.train(resume_from_checkpoint="your-model/checkpoint-1000")
```

你可以通过在 TrainingArguments 中设置 `push_to_hub=True` 将检查点（默认不保存优化器状态）保存到 Hub 上。关于如何保存检查点的其他选项可以在 `hub_strategy` 参数中设置：

- `hub_strategy="checkpoint"` 将最新的检查点推送到名为 “last-checkpoint” 的子文件夹中，你可以从中恢复训练
- `hub_strategy="all_checkpoints"` 将所有检查点推送到 `output_dir` 定义的目录中（你会在模型仓库中看到每个检查点一个文件夹）

当你从检查点恢复训练时，Trainer 会尝试保持 Python、NumPy 和 PyTorch 的随机数生成器状态与保存检查点时相同。但由于 PyTorch 有各种非确定性的默认设置，随机数生成器状态不能保证完全相同。如果你想启用完全确定性，可以查看 控制随机源 指南，了解你可以启用的设置，以使训练完全确定。但请注意，某些设置可能会使训练变慢。

[](customize-the-trainer)自定义 Trainer

虽然 Trainer 类设计为易于使用且易访问，但它也为更有经验的用户提供了大量的可定制性。许多 Trainer 的方法可以被继承和重写，以支持你想要的功能，而无需从头重写整个训练循环。这些方法包括：

- get_train_dataloader() 创建训练 DataLoader
- get_eval_dataloader() 创建评估 DataLoader
- get_test_dataloader() 创建测试 DataLoader
- log() 记录训练过程中观察到的各种对象的信息
- create_optimizer_and_scheduler() 如果没有在 `__init__` 中传递，则创建优化器和学习率调度器；这些也可以分别通过 create_optimizer() 和 create_scheduler() 进行自定义
- compute_loss() 计算一批训练输入的损失
- training_step() 执行训练步骤
- prediction_step() 执行预测和测试步骤
- evaluate() 评估模型并返回评估指标
- predict() 在测试集上进行预测（如果有标签，则计算指标）

例如，如果你想自定义 compute_loss() 方法以使用加权损失。

```
from torch import nn
from transformers import Trainer

class CustomTrainer(Trainer):
```

```
    def compute_loss(self, model, inputs, return_outputs=False):
        labels = inputs.pop("labels")
        # 前向传播
        outputs = model(**inputs)
        logits = outputs.get("logits")
        # 计算自定义损失，为 3 个标签设置不同的权重
        loss_fct = nn.CrossEntropyLoss(weight=torch.tensor([1.0, 2.0, 3.0],
device=model.device))
        loss = loss_fct(logits.view(-1, self.model.config.num_labels), labels.view(-1))
        return (loss, outputs) if return_outputs else loss
```

[](callbacks)回调函数

另一种自定义 Trainer 的方法是使用 回调函数。回调函数不会改变训练循环中的任何内容。它们会检查训练循环的状态，然后根据状态执行某些操作（如提前停止、记录结果等）。换句话说，回调函数不能用于实现自定义损失函数这样的功能，你需要继承并重写 compute_loss() 方法来实现这一点。

例如，如果你想在训练循环中添加一个在 10 步后停止训练的回调函数。

```
from transformers import TrainerCallback

class EarlyStoppingCallback(TrainerCallback):
    def __init__(self, num_steps=10):
        self.num_steps = num_steps

    def on_step_end(self, args, state, control, **kwargs):
        if state.global_step >= self.num_steps:
            return {"should_training_stop": True}
        else:
            return {}
```

然后将其传递给 Trainer 的 `callback` 参数。

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    callback=[EarlyStoppingCallback()],
)
```

[](logging)日志记录

查看 日志记录 API 参考了解更多关于不同日志级别的详细信息。

Trainer 默认设置为 `logging.INFO`，报告错误、警告和其他基本信息。在分布式环境中，Trainer 副本设置为 `logging.WARNING`，仅报告错误和警告。你可以通过 TrainingArguments 中的 `log_level` 和 `log_level_replica` 参数更改日志级别。

要为每个节点配置日志级别设置，可以使用 `log_on_each_node` 参数来确定是否在每个节点上使用日志级别，还是仅在主节点上使用。

Trainer 在 `Trainer.__init__()` 方法中为每个节点单独设置日志级别，因此如果你在创建 Trainer 对象之前使用了其他 Transformers 功能，你可能需要尽早设置此项。

例如，要将主代码和模块设置为根据每个节点使用相同的日志级别：

```
import logging
import sys

logger = logging.getLogger(__name__)

logging.basicConfig(
    format="%(asctime)s - %(levelname)s - %(name)s - %(message)s",
    datefmt="%m/%d/%Y %H:%M:%S",
    handlers=[logging.StreamHandler(sys.stdout)],
)

log_level = training_args.get_process_log_level()
logger.setLevel(log_level)
datasets.utils.logging.set_verbosity(log_level)
transformers.utils.logging.set_verbosity(log_level)

trainer = Trainer(...)
```

使用不同的 `log_level` 和 `log_level_replica` 组合来配置每个节点上记录的内容。

- 单节点

```
my_app.py ... --log_level warning --log_level_replica error
```

- 多节点

```
my_app.py ... --log_level warning --log_level_replica error --log_on_each_node 0

# set to only report errors
my_app.py ... --log_level error --log_level_replica error --log_on_each_node 0
```

NEFTune

NEFTune 是一种通过在训练过程中向嵌入向量添加噪声来提高性能的技术。要在 Trainer 中启用它，可以在 TrainingArguments 中设置 `neftune_noise_alpha` 参数来控制添加的噪声量。

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(..., neftune_noise_alpha=0.1)
trainer = Trainer(..., args=training_args)
```

NEFTune 在训练后会被禁用，以恢复原始的嵌入层，避免任何意外行为。

[](galore)GaLore

梯度低秩投影（GaLore）是一种内存高效的低秩训练策略，允许全参数学习，但比常见的低秩适应方法（如 LoRA）更节省内存。

首先确保安装 GaLore 官方仓库：

```
pip install galore-torch
```

然后只需在 `optim` 中添加 `["galore_adamw", "galore_adafactor", "galore_adamw_8bit"]` 之一，并结合 `optim_target_modules`，它可以是一个字符串列表、正则表达式或对应于你想要适应的目标模块名称的全路径。下面是一个端到端的示例脚本（确保 `pip install trl datasets`）：

```
import torch
import datasets
import trl

from transformers import TrainingArguments, AutoConfig, AutoTokenizer, AutoModelForCausalLM

train_dataset = datasets.load_dataset('imdb', split='train')

args = TrainingArguments(
    output_dir="./test-galore",
    max_steps=100,
    per_device_train_batch_size=2,
    optim="galore_adamw",
    optim_target_modules=[r".*.attn.*", r".*.mlp.*"]
)

model_id = "google/gemma-2b"

config = AutoConfig.from_pretrained(model_id)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_config(config).to(0)

trainer = trl.SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    dataset_text_field='text',
    max_seq_length=512,
)

trainer.train()
```

要传递 GaLore 支持的其他参数，应正确传递 `optim_args`，例如：

```
import torch
import datasets
import trl

from transformers import TrainingArguments, AutoConfig, AutoTokenizer, AutoModelForCausalLM

train_dataset = datasets.load_dataset('imdb', split='train')

args = TrainingArguments(
    output_dir="./test-galore",
    max_steps=100,
    per_device_train_batch_size=2,
    optim="galore_adamw",
    optim_target_modules=[r".*.attn.*", r".*.mlp.*"],
    optim_args="rank=64, update_proj_gap=100, scale=0.10",
)

model_id = "google/gemma-2b"

config = AutoConfig.from_pretrained(model_id)

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_config(config).to(0)

trainer = trl.SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    dataset_text_field='text',
    max_seq_length=512,
)

trainer.train()
```

你可以在 原始仓库 或 论文 中了解更多关于该方法的信息。

目前你只能训练被视为 GaLore 层的线性层，并将使用低秩分解进行训练，而其余层将以传统方式进行优化。

注意，在开始训练之前可能需要一些时间（在 NVIDIA A100 上训练一个 2B 模型大约需要 3 分钟），但训练应该会顺利进行。

你还可以通过在优化器名称后附加 `layerwise` 来执行逐层优化，如下所示：

```
import torch
import datasets
import trl

from transformers import TrainingArguments, AutoConfig, AutoTokenizer, AutoModelForCausalLM

train_dataset = datasets.load_dataset('imdb', split='train')

args = TrainingArguments(
    output_dir="./test-galore",
    max_steps=100,
    per_device_train_batch_size=2,
    optim="galore_adamw_layerwise",
    optim_target_modules=[r".*.attn.*", r".*.mlp.*"]
)

model_id = "google/gemma-2b"

config = AutoConfig.from_pretrained(model_id)

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_config(config).to(0)

trainer = trl.SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    dataset_text_field='text',
    max_seq_length=512,
)

trainer.train()
```

注意，逐层优化是实验性的，不支持 DDP（分布式数据并行），因此你只能在单个 GPU 上运行训练脚本。请参阅 此部分 了解更多详细信息。其他功能，如梯度裁剪、DeepSpeed 等可能不支持开箱即用。如果你遇到此类问题，请在 GitHub 上 提出问题。

[](liger-kernel)Liger Kernel

Liger-Kernel 是由 Linkedin 开发的一组 Triton 内核，专为 LLM 训练设计。我们实现了 Hugging Face 兼容的 RMSNorm、RoPE、SwiGLU、CrossEntropy、FusedLinearCrossEntropy 等，并将继续增加更多功能。它可以有效提高多 GPU 训练吞吐量 20%，并减少 60% 的内存使用。该内核与 flash attention、PyTorch FSDP 和 Microsoft DeepSpeed 开箱即用。

在 LLaMA 3-8B 模型训练中，吞吐量提高 20%，内存使用减少 60%。实现更长的上下文长度和更大的批量大小。如果你想将模型扩展到多头训练或大词汇量，它也非常有用。释放多头训练（medusa）等功能。查看 Liger 中的详细信息和示例。

首先确保安装 Liger 官方仓库：

```
pip install liger-kernel
```

你应该传递 `use_liger_kernel=True` 来在你的模型上应用 Liger 内核，例如：

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="your-model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    push_to_hub=True,
    use_liger_kernel=True
)
```

该内核支持 Llama、Gemma、Mistral 和 Mixtral 模型架构。最新支持的模型列表可以在 这里 找到。当 `use_liger_kernel` 设置为 `True` 时，原始模型中的相应层将被 Liger 的高效实现所替换，因此你不需要做任何额外的事情，只需设置参数值即可。

[](lomo-optimizer)LOMO 优化器

LOMO 优化器在 Full Parameter Fine-Tuning for Large Language Models with Limited Resources 和 AdaLomo: Low-memory Optimization with Adaptive Learning Rate 中被引入。

它们都是一种高效的全参数微调方法，将梯度计算和参数更新融合在一个步骤中，以减少内存使用。支持的 LOMO 优化器有 `"lomo"` 和 `"adalomo"`。首先可以通过 `pip install lomo-optim` 从 pypi 安装 LOMO，或者通过 `pip install git+https://github.com/OpenLMLab/LOMO.git` 从源码安装。

根据作者的建议，使用 `AdaLomo` 时建议不使用 `grad_norm` 以获得更好的性能和更高的吞吐量。

下面是一个简单的脚本，演示如何在全精度下对 google/gemma-2b 模型在 IMDB 数据集上进行微调：

```
import torch
import datasets
from transformers import TrainingArguments, AutoTokenizer, AutoModelForCausalLM
import trl

train_dataset = datasets.load_dataset('imdb', split='train')

args = TrainingArguments(
    output_dir="./test-lomo",
    max_steps=1000,
    per_device_train_batch_size=4,
    optim="adalomo",
    gradient_checkpointing=True,
    logging_strategy="steps",
    logging_steps=1,
    learning_rate=2e-6,
    save_strategy="no",
    run_name="lomo-imdb",
)

model_id = "google/gemma-2b"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, low_cpu_mem_usage=True).to(0)

trainer = trl.SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    dataset_text_field='text',
    max_seq_length=1024,
)

trainer.train()
```

[](grokadamw-optimizer)GrokAdamW 优化器

GrokAdamW 优化器旨在提高训练性能和稳定性，特别是对于那些从 grokking 信号函数中受益的模型。要使用 GrokAdamW，首先通过 `pip install grokadamw` 安装优化器包。

GrokAdamW 对于需要高级优化技术以实现更好性能和稳定性的模型特别有用。

下面是一个简单的脚本，演示如何使用 GrokAdamW 优化器对 google/gemma-2b 模型在 IMDB 数据集上进行微调：

```
import torch
import datasets
from transformers import TrainingArguments, AutoTokenizer, AutoModelForCausalLM, Trainer

# 加载 IMDB 数据集
train_dataset = datasets.load_dataset('imdb', split='train')

# 定义训练参数
args = TrainingArguments(
    output_dir="./test-grokadamw",
    max_steps=1000,
    per_device_train_batch_size=4,
    optim="grokadamw",
    logging_strategy="steps",
    logging_steps=1,
    learning_rate=2e-5,
    save_strategy="no",
    run_name="grokadamw-imdb",
)

# 加载模型和分词器
model_id = "google/gemma-2b"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, low_cpu_mem_usage=True).to(0)

# 初始化 Trainer
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
)

# 训练模型
trainer.train()
```

这个脚本演示了如何使用 GrokAdamW 优化器对 `google/gemma-2b` 模型在 IMDB 数据集上进行微调。`TrainingArguments` 配置为使用 GrokAdamW，数据集传递给 `Trainer` 进行训练。

[](schedule-free-optimizer)Schedule Free 优化器

Schedule Free 优化器在 The Road Less Scheduled 中被引入。Schedule-Free 学习用平均和插值的组合替换基础优化器的动量，完全消除了使用传统学习率调度器的需要。支持的 SFO 优化器有 `"schedule_free_adamw"` 和 `"schedule_free_sgd"`。首先通过 `pip install schedulefree` 从 pypi 安装 schedulefree。

下面是一个简单的脚本，演示如何在全精度下对 google/gemma-2b 模型在 IMDB 数据集上进行微调：

```
import torch
import datasets
from transformers import TrainingArguments, AutoTokenizer, AutoModelForCausalLM
import trl

train_dataset = datasets.load_dataset('imdb', split='train')

args = TrainingArguments(
    output_dir="./test-schedulefree",
    max_steps=1000,
    per_device_train_batch_size=4,
    optim="schedule_free_adamw",
    gradient_checkpointing=True,
    logging_strategy="steps",
    logging_steps=1,
    learning_rate=2e-6,
    save_strategy="no",
    run_name="sfo-imdb",
)

model_id = "google/gemma-2b"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, low_cpu_mem_usage=True).to(0)

trainer = trl.SFTTrainer(
    model=model,
    args=args,
    train_dataset=train_dataset,
    dataset_text_field='text',
    max_seq_length=1024,
)
```

```
trainer.train()
```

[](accelerate-and-trainer)Accelerate 和 Trainer

Trainer 类由 Accelerate 提供支持，这是一个用于在分布式环境中轻松训练 PyTorch 模型的库，支持集成如 FullyShardedDataParallel (FSDP) 和 DeepSpeed。

了解更多关于 FSDP 分片策略、CPU 卸载等的信息，请参阅 Fully Sharded Data Parallel 指南。

要使用 Accelerate 和 Trainer，运行 `accelerate.config` 命令为你的训练环境设置训练配置。此命令会创建一个 `config_file.yaml`，当你启动训练脚本时将使用该文件。例如，你可以设置的一些示例配置包括：

分布式数据并行

FSDP

DeepSpeed

DeepSpeed 与 Accelerate 插件

```
compute_environment: LOCAL_MACHINE
distributed_type: MULTI_GPU
downcast_bf16: 'no'
gpu_ids: all
machine_rank: 0 #根据节点更改 rank
main_process_ip: 192.168.20.1
main_process_port: 9898
main_training_function: main
mixed_precision: fp16
num_machines: 2
num_processes: 8
rdzv_backend: static
same_network: true
tpu_env: []
tpu_use_cluster: false
tpu_use_sudo: false
use_cpu: false
```

`accelerate_launch` 命令是使用 Accelerate 和 Trainer 在分布式系统上启动训练脚本的推荐方式，参数在 `config_file.yaml` 中指定。此文件保存在 Accelerate 缓存文件夹中，当你运行 `accelerate_launch` 时会自动加载。

例如，要使用 FSDP 配置运行 run_glue.py 训练脚本：

```
accelerate launch \
    ./examples/pytorch/text-classification/run_glue.py \
    --model_name_or_path google-bert/bert-base-cased \
    --task_name $TASK_NAME \
    --do_train \
    --do_eval \
    --max_seq_length 128 \
    --per_device_train_batch_size 16 \
    --learning_rate 5e-5 \
    --num_train_epochs 3 \
    --output_dir /tmp/$TASK_NAME/ \
    --overwrite_output_dir
```

你也可以直接在命令行中指定 `config_file.yaml` 文件中的参数：

```
accelerate launch --num_processes=2 \
    --use_fsdp \
    --mixed_precision=bf16 \
    --fsdp_auto_wrap_policy=TRANSFORMER_BASED_WRAP  \
    --fsdp_transformer_layer_cls_to_wrap="BertLayer" \
    --fsdp_sharding_strategy=1 \
    --fsdp_state_dict_type=FULL_STATE_DICT \
    ./examples/pytorch/text-classification/run_glue.py
    --model_name_or_path google-bert/bert-base-cased \
    --task_name $TASK_NAME \
    --do_train \
    --do_eval \
    --max_seq_length 128 \
    --per_device_train_batch_size 16 \
    --learning_rate 5e-5 \
    --num_train_epochs 3 \
    --output_dir /tmp/$TASK_NAME/ \
    --overwrite_output_dir
```

查看 启动你的 Accelerate 脚本 教程，了解更多关于 `accelerate_launch` 和自定义配置的信息。

将模型导出为 ONNX 格式

在生产环境中部署 🤗 Transformers 模型时，通常需要将模型导出为一种序列化格式，以便在专用运行时和硬件上加载和执行。

🤗 Optimum 是 Transformers 的一个扩展，通过其 `exporters` 模块，可以将模型从 PyTorch 或 TensorFlow 导出为序列化格式，如 ONNX 和 TFLite。🤗 Optimum 还提供了一组性能优化工具，以在目标硬件上以最大效率训练和运行模型。

本指南演示了如何使用 🤗 Optimum 将 🤗 Transformers 模型导出为 ONNX 格式。有关将模型导出为 TFLite 的指南，请参阅 导出到 TFLite 页面。

导出到 ONNX

ONNX（开放神经网络交换） 是一个开放标准，定义了一组通用的操作符和一个通用的文件格式，用于在各种框架（包括 PyTorch 和 TensorFlow）中表示深度学习模型。当模型导出为 ONNX 格式时，这些操作符用于构建一个计算图（通常称为中间表示），该图表示数据在神经网络中的流动。

通过公开具有标准化操作符和数据类型的图，ONNX 使得在框架之间切换变得容易。例如，在 PyTorch 中训练的模型可以导出为 ONNX 格式，然后在 TensorFlow 中导入（反之亦然）。

一旦导出为 ONNX 格式，模型可以：

- 通过图优化和量化等技术进行推理优化。
- 通过 `ORTModelForXXX` 类 使用 ONNX Runtime 运行，这些类遵循与 🤗 Transformers 中相同的 `AutoModel` API。
- 通过优化推理管道运行，其 API 与 🤗 Transformers 中的 pipeline() 函数相同。

🤗 Optimum 通过利用配置对象来支持 ONNX 导出。这些配置对象为多种模型架构提供了现成的支持，并且设计为易于扩展到其他架构。

有关现成配置的列表，请参阅 🤗 Optimum 文档。

有两种方法可以将 🤗 Transformers 模型导出为 ONNX，这里我们展示两种方法：

- 通过 CLI 使用 🤗 Optimum 导出。
- 通过 `optimum.onnxruntime` 使用 🤗 Optimum 导出。

通过 CLI 导出 🤗 Transformers 模型到 ONNX

要将 🤗 Transformers 模型导出为 ONNX，首先安装一个额外的依赖项：

```
pip install optimum[exporters]
```

要查看所有可用参数，请参阅 🤗 Optimum 文档，或在命令行中查看帮助：

```
optimum-cli export onnx --help
```

要从 🤗 Hub 导出模型的检查点，例如 `distilbert/distilbert-base-uncased-distilled-squad`，运行以下命令：

```
optimum-cli export onnx --model distilbert/distilbert-base-uncased-distilled-squad
distilbert_base_uncased_squad_onnx/
```

你应该会看到日志指示进度，并显示保存的 `model.onnx` 文件的位置，如下所示：

```
Validating ONNX model distilbert_base_uncased_squad_onnx/model.onnx...
    -[✓] ONNX model output names match reference model (start_logits, end_logits)
    - Validating ONNX Model output "start_logits":
        -[✓] (2, 16) matches (2, 16)
        -[✓] all values close (atol: 0.0001)
    - Validating ONNX Model output "end_logits":
        -[✓] (2, 16) matches (2, 16)
        -[✓] all values close (atol: 0.0001)
The ONNX export succeeded and the exported model was saved at:
distilbert_base_uncased_squad_onnx
```

上面的示例说明了从 🤗 Hub 导出检查点。当导出本地模型时，首先确保你将模型的权重和分词器文件保存在同一目录（`local_path`）中。使用 CLI 时，将 `local_path` 传递给 `model` 参数，而不是 🤗 Hub 上的检查点名称，并提供 `--task` 参数。你可以在 🤗 Optimum 文档 中查看支持的任务列表。如果未提供 `task` 参数，它将默认为模型架构，而不带任何任务特定的头部。

```
optimum-cli export onnx --model local_path --task question-answering
distilbert_base_uncased_squad_onnx/
```

生成的 `model.onnx` 文件可以在支持 ONNX 标准的许多加速器上运行。例如，我们可以使用 ONNX Runtime 加载和运行模型，如下所示：

```
from transformers import AutoTokenizer
from optimum.onnxruntime import ORTModelForQuestionAnswering

tokenizer = AutoTokenizer.from_pretrained("distilbert_base_uncased_squad_onnx")
model = ORTModelForQuestionAnswering.from_pretrained("distilbert_base_uncased_squad_onnx")
inputs = tokenizer("What am I using?", "Using DistilBERT with ONNX Runtime!",
return_tensors="pt")
outputs = model(**inputs)
```

对于 Hub 上的 TensorFlow 检查点，过程是相同的。例如，这里是如何从 Keras 组织 导出纯 TensorFlow 检查点：

```
optimum-cli export onnx --model keras-io/transformers-qa distilbert_base_cased_squad_onnx/
```

通过 optimum.onnxruntime 导出 🤗 Transformers 模型到 ONNX

除了 CLI，你还可以通过编程方式将 🤗 Transformers 模型导出为 ONNX，如下所示：

```
from optimum.onnxruntime import ORTModelForSequenceClassification
from transformers import AutoTokenizer

model_checkpoint = "distilbert_base_uncased_squad"
save_directory = "onnx/"

# 从 transformers 加载模型并导出为 ONNX
ort_model = ORTModelForSequenceClassification.from_pretrained(model_checkpoint, export=True)
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

# 保存 ONNX 模型和分词器
ort_model.save_pretrained(save_directory)
tokenizer.save_pretrained(save_directory)
```

导出不支持架构的模型

如果你想通过添加对当前无法导出的模型的支持来贡献，你应该首先检查它是否在 `optimum.exporters.onnx` 中受支持，如果未受支持，请直接贡献到 😭 Optimum。

通过 transformers.onnx 导出模型

`transformers.onnx` 不再维护，请按照上述方法使用 😭 Optimum 导出模型。此部分将在未来版本中删除。

要将 😭 Transformers 模型通过 `transformers.onnx` 导出为 ONNX，请安装额外的依赖项：

```
pip install transformers[onnx]
```

使用 `transformers.onnx` 包作为 Python 模块，使用现成的配置导出检查点：

```
python -m transformers.onnx --model=distilbert/distilbert-base-uncased onnx/
```

这将导出由 `--model` 参数定义的检查点的 ONNX 图。传递 😭 Hub 上的任何检查点或本地存储的检查点。生成的 `model.onnx` 文件可以在支持 ONNX 标准的许多加速器上运行。例如，使用 ONNX Runtime 加载和运行模型，如下所示：

```
from transformers import AutoTokenizer
from onnxruntime import InferenceSession

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
session = InferenceSession("onnx/model.onnx")
# ONNX Runtime 期望输入为 NumPy 数组
inputs = tokenizer("Using DistilBERT with ONNX Runtime!", return_tensors="np")
outputs = session.run(output_names=["last_hidden_state"], input_feed=dict(inputs))
```

所需的输出名称（如 `["last_hidden_state"]`）可以通过查看每个模型的 ONNX 配置来获得。例如，对于 DistilBERT，我们有：

```
from transformers.models.distilbert import DistilBertConfig, DistilBertOnnxConfig

config = DistilBertConfig()
onnx_config = DistilBertOnnxConfig(config)
print(list(onnx_config.outputs.keys()))
```

对于 Hub 上的 TensorFlow 检查点，过程是相同的。例如，导出纯 TensorFlow 检查点，如下所示：

```
python -m transformers.onnx --model=keras-io/transformers-qa onnx/
```

要导出本地存储的模型，请将模型的权重和分词器文件保存在同一目录（例如 `local-pt-checkpoint`）中，然后通过将 `transformers.onnx` 包的 `--model` 参数指向所需目录来将其导出为 ONNX：

```
python -m transformers.onnx --model=local-pt-checkpoint onnx/
```

将模型导出至 TFLite

TensorFlow Lite 是一个轻量级框架，用于在资源受限的设备（如手机、嵌入式系统和物联网设备）上部署机器学习模型。TFLite 设计的目的是在这些计算能力、内存和功耗有限的设备上高效地优化和运行模型。TensorFlow Lite 模型以一种特殊的高效便携格式表示，文件扩展名为 `.tflite`。

🤗 Optimum 提供了将 🤗 Transformers 模型导出到 TFLite 的功能，通过 `exporters.tflite` 模块实现。支持的模型架构列表请参阅 🤗 Optimum 文档。

要将模型导出到 TFLite，请安装所需的依赖项：

```
pip install optimum[exporters-tf]
```

要查看所有可用的参数，可以参考 🤗 Optimum 文档，或在命令行中查看帮助信息：

```
optimum-cli export tflite --help
```

例如，要从 🤗 Hub 导出 `google-bert/bert-base-uncased` 模型的检查点，可以运行以下命令：

```
optimum-cli export tflite --model google-bert/bert-base-uncased --sequence_length 128
bert_tflite/
```

你应该会看到日志显示进度，并指出生成的 `model.tflite` 文件保存的位置，如下所示：

导出成功，但有警告：参考模型和 TFLite 导出模型的输出之间的最大绝对差异不在设定的容差范围内 1e-05：

- logits: 最大差异 = 5.817413330078125e-05。

导出的模型已保存在：bert_tflite

上述示例展示了如何从 🤗 Hub 导出模型。如果要导出本地模型，首先确保模型的权重和分词器文件都保存在同一目录下（`local_path`）。使用 CLI 时，将 `local_path` 传递给 `model` 参数，而不是 🤗 Hub 上的检查点名称。

导出到 TorchScript

这是我们对 TorchScript 进行实验的起点，我们仍在探索它在处理可变输入大小模型方面的能力。这是我们感兴趣的重点，未来我们将进一步深入分析，并提供更多代码示例、更灵活的实现以及与基于 Python 的代码相比的性能基准测试。

根据 TorchScript 文档：

> TorchScript 是一种从 PyTorch 代码创建可序列化和可优化模型的方法。

有两个 PyTorch 模块，JIT 和 TRACE，允许开发者导出他们的模型以供其他程序（如效率导向的 C++ 程序）重用。

我们提供了一个接口，允许您将 🤗 Transformers 模型导出到 TorchScript，以便在不同的环境中重用这些模型，而不仅仅是基于 PyTorch 的 Python 程序。在这里，我们将解释如何导出和使用这些模型。

导出模型需要两件事：

- 使用 `torchscript` 标志实例化模型
- 使用虚拟输入进行前向传递

这些要求意味着开发者需要注意以下几点。

使用 torchscript 标志和绑定权重

`torchscript` 标志是必需的，因为大多数 🤗 Transformers 语言模型在其 `Embedding` 层和 `Decoding` 层之间有绑定权重。TorchScript 不允许导出具有绑定权重的模型，因此需要预先解开并克隆这些权重。

使用 `torchscript` 标志实例化的模型，其 `Embedding` 层和 `Decoding` 层是分开的，这意味着它们不应再进行训练。继续训练会导致这两个层不同步，从而产生意外结果。

对于没有语言模型头的模型，情况并非如此，因为这些模型没有绑定权重。这些模型可以安全地导出，而无需使用 `torchscript` 标志。

虚拟输入和标准长度

虚拟输入用于模型的前向传递。当输入值通过各层传播时，PyTorch 会记录每个张量上执行的不同操作。这些记录的操作用于创建模型的 _trace_。

trace 是相对于输入维度创建的，因此受虚拟输入维度的限制，对于任何其他序列长度或批量大小都不起作用。尝试使用不同大小的输入时，会引发以下错误：

建议您使用至少与模型推理期间将要输入的最大输入一样大的虚拟输入来跟踪模型。填充可以帮助填补缺失的值。然而，由于模型是使用较大的输入大小跟踪的，矩阵的维度也会较大，导致更多的计算。

请注意每个输入执行的总操作数，并在导出具有变化序列长度的模型时密切关注性能。

在 Python 中使用 TorchScript

本节演示如何保存和加载模型，以及如何使用跟踪进行推理。

保存模型

要将 `BertModel` 导出到 TorchScript，可以从 `BertConfig` 类实例化 `BertModel`，然后将其保存到名为 `traced_bert.pt` 的文件中：

```
from transformers import BertModel, BertTokenizer, BertConfig
import torch

# 加载预训练的分词器
enc = BertTokenizer.from_pretrained("google-bert/bert-base-uncased")

# 对输入文本进行分词
text = "[CLS] Who was Jim Henson ? [SEP] Jim Henson was a puppeteer [SEP]"
tokenized_text = enc.tokenize(text)

# 遮蔽一个输入词
masked_index = 8
tokenized_text[masked_index] = "[MASK]"
indexed_tokens = enc.convert_tokens_to_ids(tokenized_text)
segments_ids = [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1]

# 创建虚拟输入
tokens_tensor = torch.tensor([indexed_tokens])
segments_tensors = torch.tensor([segments_ids])
dummy_input = [tokens_tensor, segments_tensors]

# 使用 torchscript 标志初始化模型
# 尽管这个模型没有 LM 头，但仍然设置为 True
config = BertConfig(
    vocab_size_or_config_json_file=32000,
    hidden_size=768,
    num_hidden_layers=12,
    num_attention_heads=12,
    intermediate_size=3072,
    torchscript=True,
)

# 实例化模型
model = BertModel(config)

# 将模型设置为评估模式
model.eval()

# 如果使用 *from_pretrained* 实例化模型，也可以轻松设置 TorchScript 标志
model = BertModel.from_pretrained("google-bert/bert-base-uncased", torchscript=True)

# 创建跟踪
traced_model = torch.jit.trace(model, [tokens_tensor, segments_tensors])
torch.jit.save(traced_model, "traced_bert.pt")
```

加载模型

现在您可以从磁盘加载之前保存的 `BertModel`（`traced_bert.pt`），并使用之前初始化的 `dummy_input`：

```
loaded_model = torch.jit.load("traced_bert.pt")
loaded_model.eval()

all_encoder_layers, pooled_output = loaded_model(*dummy_input)
```

使用跟踪模型进行推理

使用跟踪模型进行推理时，可以通过调用其 `__call__` 方法：

```
traced_model(tokens_tensor, segments_tensors)
```

使用 Neuron SDK 将 Hugging Face TorchScript 模型部署到 AWS

AWS 引入了 Amazon EC2 Inf1 实例家族，用于低成本、高性能的云中机器学习推理。Inf1 实例由 AWS Inferentia 芯片驱动，这是一种定制的硬件加速器，专门用于深度学习推理工作负载。AWS Neuron 是支持在 Inf1 上部署 Transformer 模型的 SDK。Neuron SDK 提供以下功能：

252. 使用一行代码更改即可轻松跟踪和优化 TorchScript 模型以在云端进行推理的易用 API。
253. 开箱即用的性能优化，以提高 成本效益。
254. 支持使用 PyTorch 或 TensorFlow 构建的 Hugging Face Transformer 模型。

影响

基于 BERT (双向编码器表示来自 Transformer) 架构或其变体（如 distilBERT 和 roBERTa）的 Transformer 模型在 Inf1 上最适合非生成任务，例如提取性问答、序列分类和 token 分类。然而，文本生成任务仍可以根据此 AWS Neuron MarianMT 教程 适应在 Inf1 上运行。更多关于可以直接在 Inferentia 上转换的模型的信息，请参阅 Neuron 文档中的 模型架构适配 部分。

依赖项

使用 AWS Neuron 转换模型需要一个 Neuron SDK 环境，该环境预先配置在 AWS Deep Learning AMI 上。

将模型转换为 AWS Neuron

使用与 在 Python 中使用 TorchScript 相同的代码跟踪 `BertModel`。导入 `torch.neuron` 框架扩展，以通过 Python API 访问 Neuron SDK 的组件：

```
from transformers import BertModel, BertTokenizer, BertConfig
import torch
import torch.neuron
```

您只需要修改以下行：

```
- torch.jit.trace(model, [tokens_tensor, segments_tensors])
+ torch.neuron.trace(model, [tokens_tensor, segments_tensors])
```

这将启用 Neuron SDK 跟踪并优化模型以适用于 Inf1 实例。

要了解有关 AWS Neuron SDK 功能、工具、示例教程和最新更新的更多信息，请参阅 AWS NeuronSDK 文档。

基准测试

Hugging Face 的基准测试工具已弃用，建议使用外部基准测试库来测量 Transformer 模型的速度和内存复杂度。

让我们看看如何对 🤗 Transformers 模型进行基准测试，最佳实践以及已经可用的基准测试。

详细说明如何对 🤗 Transformers 模型进行基准测试的笔记本可以在这里找到：链接。

如何对 🤗 Transformers 模型进行基准测试

类 `PyTorchBenchmark` 和 `TensorFlowBenchmark` 允许灵活地对 🤗 Transformers 模型进行基准测试。这些基准测试类允许我们测量 **峰值内存使用** 和 **所需时间**，包括 **推理** 和 **训练**。

这里，**推理** 定义为一次前向传递，而 **训练** 定义为一次前向传递和一次反向传递。

基准测试类 `PyTorchBenchmark` 和 `TensorFlowBenchmark` 在实例化时分别需要一个 `PyTorchBenchmarkArguments` 或 `TensorFlowBenchmarkArguments` 对象。`PyTorchBenchmarkArguments` 和 `TensorFlowBenchmarkArguments` 是数据类，包含其对应基准测试类的所有相关配置。以下示例展示了如何对类型为 _bert-base-cased_ 的 BERT 模型进行基准测试。

PyTorch

```
from transformers import PyTorchBenchmark, PyTorchBenchmarkArguments

args = PyTorchBenchmarkArguments(
    models=["google-bert/bert-base-uncased"],
    batch_sizes=[8],
    sequence_lengths=[8, 32, 128, 512]
)
benchmark = PyTorchBenchmark(args)
```

TensorFlow

```
from transformers import TensorFlowBenchmark, TensorFlowBenchmarkArguments

args = TensorFlowBenchmarkArguments(
    models=["google-bert/bert-base-uncased"],
    batch_sizes=[8],
    sequence_lengths=[8, 32, 128, 512]
)
benchmark = TensorFlowBenchmark(args)
```

这里，基准测试参数数据类有三个参数：`models`、`batch_sizes` 和 `sequence_lengths`。参数 `models` 是必需的，期望一个包含来自 模型中心 的模型标识符的列表。参数 `batch_sizes` 和 `sequence_lengths` 定义了模型基准测试中 `input_ids` 的大小。通过基准测试参数数据类可以配置许多其他参数。有关这些参数的详细信息，可以直接查阅文件 `src/transformers/benchmark/benchmark_args_utils.py`、`src/transformers/benchmark/benchmark_args.py`（对于 PyTorch）和 `src/transformers/benchmark/benchmark_args_tf.py`（对于 TensorFlow）。或者，从根目录运行以下 shell 命令将分别打印出 PyTorch 和 TensorFlow 的所有可配置参数的描述列表。

PyTorch

```
python examples/pytorch/benchmarking/run_benchmark.py --help
```

实例化的基准测试对象可以通过调用 `benchmark.run()` 简单地运行。

```
results = benchmark.run()
print(results)
```

TensorFlow

```
python examples/tensorflow/benchmarking/run_benchmark_tf.py --help
```

实例化的基准测试对象可以通过调用 `benchmark.run()` 简单地运行。

```
results = benchmark.run()
print(results)
```

默认情况下，会基准测试 **推理** 所需的 **时间** 和 **内存**。在上面的示例输出中，前两部分显示了 **推理时间** 和 **推理内存** 的结果。此外，第三部分 **环境信息** 列出了与计算环境相关的所有相关信息，例如 GPU 类型、系统、库版本等。这些信息可以选保存在一个 `.csv` 文件中，只需在 `PyTorchBenchmarkArguments` 和 `TensorFlowBenchmarkArguments` 中添加参数 `save_to_csv=True`。在这种情况下，每个部分将保存在一个单独的 `.csv` 文件中。可以通过参数数据类定义每个 `.csv` 文件的路径。

除了通过模型标识符（例如 `google-bert/bert-base-uncased`）基准测试预训练模型外，用户还可以基准测试任何可用模型类的任意配置。在这种情况下，必须插入一个配置列表，如下所示。

PyTorch

```
from transformers import PyTorchBenchmark, PyTorchBenchmarkArguments, BertConfig

args = PyTorchBenchmarkArguments(
    models=["bert-base", "bert-384-hid", "bert-6-lay"],
    batch_sizes=[8],
    sequence_lengths=[8, 32, 128, 512]
)
config_base = BertConfig()
config_384_hid = BertConfig(hidden_size=384)
config_6_lay = BertConfig(num_hidden_layers=6)

benchmark = PyTorchBenchmark(args, configs=[config_base, config_384_hid, config_6_lay])
benchmark.run()
```

TensorFlow

```
from transformers import TensorFlowBenchmark, TensorFlowBenchmarkArguments, BertConfig

args = TensorFlowBenchmarkArguments(
    models=["bert-base", "bert-384-hid", "bert-6-lay"],
    batch_sizes=[8],
```

```
    sequence_lengths=[8, 32, 128, 512]
)
config_base = BertConfig()
config_384_hid = BertConfig(hidden_size=384)
config_6_lay = BertConfig(num_hidden_layers=6)

benchmark = TensorFlowBenchmark(args, configs=[config_base, config_384_hid, config_6_lay])
benchmark.run()
```

同样，默认情况下会测量 **推理** 所需的 **时间** 和 **内存**，但这次是针对 `BertModel` 类的自定义配置。此功能在决定模型应以哪种配置进行训练时特别有用。

基准测试最佳实践

本节列出了在基准测试模型时应了解的一些最佳实践。

- 目前仅支持单设备基准测试。当在 GPU 上进行基准测试时，建议用户通过设置 `CUDA_VISIBLE_DEVICES` 环境变量指定代码应在哪个设备上运行，例如在运行代码之前执行 `export CUDA_VISIBLE_DEVICES=0`。
- 选项 `no_multi_processing` 只应在测试和调试时设置为 `True`。为了确保准确的内存测量，建议在单独的进程中运行每个内存基准测试，即确保 `no_multi_processing` 设置为 `True`。
- 分享模型基准测试结果时，应始终说明环境信息。不同的 GPU 设备、库版本等会导致结果有很大差异，因此单独的基准测试结果对社区来说不太有用。

分享你的基准测试

之前所有的核心模型（当时有 10 个）都进行了 **推理时间** 的基准测试，涵盖了多种不同的设置：使用 PyTorch、有无 TorchScript、使用 TensorFlow、有无 XLA。所有这些测试都在 CPU（除 TensorFlow XLA 外）和 GPU 上进行了。

该方法在 这篇博客文章 中有详细介绍，结果可以在 这里 查看。

使用新的 **基准** 工具，分享你的基准测试结果比以往任何时候都更容易。

- PyTorch 基准测试结果
- TensorFlow 基准测试结果

Transformers Notebooks 记事本示例合集

这里列出了 Hugging Face 提供的所有官方笔记本。

我们还希望列出社区创建的有趣内容。如果你编写了利用 🤗 Transformers 的笔记本并希望被列出，请提交一个 Pull Request，以便将其包含在社区笔记本中。

🤗 *Hugging Face 的笔记本*

文档笔记本

你可以将文档的任何页面作为笔记本在 Colab 中打开（页面上有一个直接的按钮），但它们也在这里列出，以备不时之需：

笔记本	描述		
库的快速浏览	展示 Transformers 中的各种 API	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/quicktour.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/en/transformers_doc/quicktour.ipynb)
任务总结	如何按照任务运行 Transformers 库	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/task_summary.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/transformers_doc/en/task_summary.ipynb)

	中的模型		
数据预处理	如何使用分词器预处理数据	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/preprocessing.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/transformers_doc/en/preprocessing.ipynb)
微调预训练模型	如何使用 Trainer 微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/training.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/transformers_doc/en/training.ipynb)
分词器总结	不同分词器算法的区别	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/tokenizer_summary.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/transformers_doc/en/tokenizer_summary.ipynb)
多语言模型	如何使用库中的多语言模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/transformers_doc/en/multilingual.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/transformers_doc/en/multilingual.ipynb)

PyTorch 示例

自然语言处理

笔记本	描述		
训练你自己的分词器	如何训练和使用你自己的分词器	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/tokenizer_training.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/tokenizer_training.ipynb)
训练你的语言模型	如何轻松开始使用 transformers	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/language_modeling_from_scratch.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/language_modeling_from_scratch.ipynb)
如何微调文本分类模	展示如何预处理数据并在任何 GLUE 任	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/text_classification.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/text_classification.ipynb)

型	务上微调预训练模型		
如何微调语言模型	展示如何预处理数据并在因果或掩码 LM 任务上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/language_modeling.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/language_modeling.ipynb)
如何微调标记分类模型	展示如何预处理数据并在 NER 或 PoS 任务上微调预训练	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/token_classification.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/token_classification.ipynb)

	模型		
如何微调问答模型	展示如何预处理数据并在 SQUAD 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/question_answering.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/question_answering.ipynb)
如何微调多项选择模型	展示如何预处理数据并在 SWAG 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/multiple_choice.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/multiple_choice.ipynb)
如何微调翻译	展示如何预处理数据并在	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/translation.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/translation.ipynb)

模型	WMT 上微调预训练模型		
如何微调摘要模型	展示如何预处理数据并在 XSUM 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/summarization.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/summarization.ipynb)
如何从头训练语言模型	高效地在自定义数据上训练 Transformer 模型的所有步骤	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/blog/blob/main/notebooks/01_how_to_train.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/blog/blob/main/notebooks/01_how_to_train.ipynb)

如何生成文本	使用不同的解码方法生成文本	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/blog/blob/main/notebooks/02_how_to_generate.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/blog/blob/main/notebooks/02_how_to_generate.ipynb)
如何生成文本（带约束）	使用用户提供的约束引导语言生成	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/blog/blob/main/notebooks/53_constrained_beam_search.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/blog/blob/main/notebooks/53_constrained_beam_search.ipynb)
Reformer	Reformer 如何推动语言建模的极限	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/blog/blob/main/notebooks/03_reformer.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/patrickvonplaten/blog/blob/main/notebooks/03_reformer.ipynb)

计算机视觉

笔记本	描述		

如何微调图像分类模型（Torchvision）	展示如何使用 Torchvision 预处理数据并微调任何预训练的视觉模型进行图像分类	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_classification.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_classification.ipynb)
如何微调图像分类模型（Albumentations）	展示如何使用 Albumentations 预处理数据并微调任何预训练的视	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_classification_albumentations.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_classification_albumentations.ipynb)

	觉模型进行图像分类		
如何微调图像分类模型（Kornia）	展示如何使用 Kornia 预处理数据并微调任何预训练的视觉模型进行图像分类	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_classification_kornia.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_classification_kornia.ipynb)
如何使用 OWL-ViT 进行零样本目标检测	展示如何使用文本查询在图像上进行零样本目	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/zeroshot_object_detection_with_owlvit.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/zeroshot_object_detection_with_owlvit.ipynb)

	标检测		
如何微调图像描述模型	展示如何在自定义数据集上微调 BLIP 进行图像描述	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_captioning_blip.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_captioning_blip.ipynb)
如何使用 Transformers 构建图像相似性系统	展示如何构建图像相似性系统	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_similarity.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_similarity.ipynb)
如何微调 SegFormer 模型进行语义分割	展示如何预处理数据并微调预训练的 SegFormer 模	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/semantic_segmentation.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/semantic_segmentation.ipynb)

	型进行语义分割		
如何微调 VideoMAE 模型进行视频分类	展示如何预处理数据并微调预训练的 VideoMAE 模型进行视频分类	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/video_classification.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/video_classification.ipynb)

音频

笔记本	描述		
如何微调英语语	展示如何预处理	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/speech_recognition.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/speech_recognition.ipynb)

音识别模型	数据并在 TIMIT 上微调预训练的语音模型		
如何微调多语言语音识别模型	展示如何预处理数据并在 Comm	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/multi_lingual_speech_recognition.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/multi_lingual_speech_recognition.ipynb)

	on Voice 上微调多语言预训练的语音模型		
如何微调音频分类模型	展示如何预处理数据并在关键词识	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/audio_classification.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/audio_classification.ipynb)

	别任务上微调预训练的语音模型		

生物序列

笔记本	描述		
如何微调预训练的蛋白质模	展示如何对蛋白质进行分词	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/protein_language_modeling.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/protein_language_modeling.ipynb)

型	并微调大型预训练的蛋白质“语言”模型		
如何生成蛋白质折叠	展示如何从蛋白质序列生成完整	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/protein_folding.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/protein_folding.ipynb)

	的蛋白质模型和PDB文件		
如何微调核酸Transformer模型	展示如何对DNA进行分词并微调大型预训练	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/nucleotide_transformer_dna_sequence_modelling.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/nucleotide_transformer_dna_sequence_modelling.ipynb)

	的DNA“语言”模型		
如何使用LoRA微调核酸Transformer模型	以内存高效的方式训练更大的DNA模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/nucleotide_transformer_dna_sequence_modelling_with_peft.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/nucleotide_transformer_dna_sequence_modelling_with_peft.ipynb)

其他模态

笔	描述		

记本			
概率时间序列预测	展示如何在自定义数据集上训练时间序列 Transformer	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/time-series-transformers.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/time-series-transformers.ipynb)

工具笔记本

笔记本	描述		
如何导出模型到 ONNX	展示如何通过 ONNX 导出和运行推理工作负载	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/onnx-export.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/onnx-export.ipynb)
如	如何	![在 Colab 中打开]	![在 AWS Studio 中打开]

何使用基准测试	使用 transformers 对模型进行基准测试	开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/benchmark.ipynb)	开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/benchmark.ipynb)

TensorFlow 示例

自然语言处理

笔记本	描述		
训练你自己的分词器	如何训练和使用你自己的分词器	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/tokenizer_training.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/tokenizer_training.ipynb)
训练你的语言	如何轻松开始使用 transfor	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/language_modeling_from_scratch-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/language_modeling_from_scratch-tf.ipynb)

模型	mers		
如何微调文本分类模型	展示如何预处理数据并在任何 GLUE 任务上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/text_classification-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/text_classification-tf.ipynb)
如何微调语言模型	展示如何预处理数据并在因果或掩码 LM 任务上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/language_modeling-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/language_modeling-tf.ipynb)
如何	展示如何	![在 Colab 中打开](https://colab.research.google.com/github	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/git

微调标记分类模型	预处理数据并在 NER 或 PoS 任务上微调预训练模型	/huggingface/notebooks/blob/main/examples/token_classification-tf.ipynb)	hub/huggingface/notebooks/blob/main/examples/token_classification-tf.ipynb)
如何微调问答模型	展示如何预处理数据并在 SQUAD 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/question_answering-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/question_answering-tf.ipynb)
如何微调多项选择	展示如何预处理数据并在 SWAG	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/multiple_choice-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/multiple_choice-tf.ipynb)

模型	上微调预训练模型		
如何微调翻译模型	展示如何预处理数据并在 WMT 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/translation-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/translation-tf.ipynb)
如何微调摘要模型	展示如何预处理数据并在 XSUM 上微调预训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/summarization-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/summarization-tf.ipynb)

计算机视觉

笔记本	描述		
如何微调图像分类模型	展示如何预处理数据并微调任何预训练的视觉模型进行图像分类	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/image_classification-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/image_classification-tf.ipynb)

如何微调 SegFormer 模型进行语义分割	展示如何预处理数据并微调预训练的 SegFormer 模型进行语义分割	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/semantic_segmentation-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/semantic_segmentation-tf.ipynb)

生物序列

笔记	描述		

本			
如何微调预训练的蛋白质模型	展示如何对蛋白质进行分词并微调大型预训练的蛋白质“语言”模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/protein_language_modeling-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/protein_language_modeling-tf.ipynb)

工具笔记本

笔记本	描述		
如何在TPU上训练TF/Keras模型	展示如何在Google的TPU硬件上高速训练模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/tpu_training-tf.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/tpu_training-tf.ipynb)

Optimum 笔记本

🤗 Optimum 是 🤗 Transformers 的扩展，提供了一套性能优化工具，使模型在特定硬件上的训练和运行达到最高效率。

笔记	描		

本	述		
如何使用ONNX Runtime量化文本分类模型	展示如何使用ONNX Runtime在任何GLUE任务上应用静态和动态量化	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/text_classification_quantization_ort.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/text_classification_quantization_ort.ipynb)
如何使用	展示如何	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/text_classification_quantization_inc.ipyn	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/text_classification_quantization_inc.ipyn

Intel Neural Compressor 量化文本分类模型	使用 Intel Neural Compressor (INC) 在任何 GLUE 任务上应用静态、动态和训练感知量	b)	b)

	化		
如何使用 ONNX Runtime 微调文本分类模型	展示如何预处理数据并使用 ONNX Runtime 在任何 GLUE 任务上微调模型	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/examples/text_classification_ort.ipynb)	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/examples/text_classification_ort.ipynb)
如何使	展示如	![在 Colab 中打开](https://colab.research.google.com/github/huggingface/notebooks/blob/main/exam	![在 AWS Studio 中打开](https://studiolab.sagemaker.aws/import/github/huggingface/notebooks/blob/main/exa

用 ONNX Runtime 微调摘要模型	何预处理数据并使用 ONNX Runtime 在 XSUM 上微调模型	ples/summarization_ort.ipynb)	mples/summarization_ort.ipynb)

社区笔记本

更多由社区开发的笔记本可在 这里 查看。

Transformers Community 社区与资源

本页面汇总了社区中关于 🤗 Transformers 的相关资源。

社区资源

资源	描述	作者
Hugging Face Transformers 术语表闪卡	基于 Transformers 文档术语表，通过 Anki 这个开源跨平台应用程序以易于学习/复习的形式制作的闪卡。查看这个 介绍视频了解如何使用闪卡	Darigov Research

社区笔记本

笔记本	描述	作者	
微调预训练 Transformer 生成歌词	如何通过微调 GPT-2 模型来生成你喜欢的艺术家风格的歌词	Aleksey Korshuk	![在 Colab 中打开](https://colab.research.google.com/github/AlekseyKorshuk/huggingartists/blob/master/huggingartists-demo.ipynb)
在 Tensorflow 2 中训练 T5	如何使用 Tensorflow 2 训练 T5 模型。本笔记本展示了使用 SQUAD 数据集实现的问答任务	Muhammad Harris	![在 Colab 中打开](https://colab.research.google.com/github/snapthat/TF-T5-text-to-text/blob/master/snapthatT5/notebooks/TF-T5-Datasets%20Training.ipynb)
在 TPU 上训练 T5	如何使用 Transformers 和 Nlp 在 SQUAD 数据集上训练 T5 模型	Suraj Patil	![在 Colab 中打开](https://colab.research.google.com/github/patil-suraj/exploring-T5/blob/master/T5_on_TPU.ipynb#scrollTo=QLGiFCDqvuil)
微调 T5	如何使用	Suraj	![在 Colab 中打开](https://colab.research.google.com/github/patil-

用于分类和多选任务	PyTorch Lightning 以文本到文本格式微调 T5 模型，用于分类和多选任务	Patil	suraj/exploring-T5/blob/master/t5_fine_tuning.ipynb)
在新数据集和语言上微调 DialoGPT	如何在新数据集上微调 DialoGPT 模型，用于开放式对话聊天机器人	Nathan Cooper	![在 Colab 中打开](https://colab.research.google.com/github/ncoop57/i-am-a-nerd/blob/master/_notebooks/2020-05-12-chatbot-part-1.ipynb)
使用 Reformer 处理长序列建模	如何使用 Reformer 训练长达 500,000 个标记的序列	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/PyTorch_Reformer.ipynb)
微调 BART 用于摘要	如何使用 fastai 和 blurr 微调 BART 模型，用于摘要	Wayde Gilliam	![在 Colab 中打开](https://colab.research.google.com/github/ohmeow/ohmeow_website/blob/master/posts/2021-05-25-mbart-sequence-classification-with-blurr.ipynb)
微调预训练 Transformer 生成推文	如何通过微调 GPT-2 模型生成你喜欢的 Twitter 账户风格的推文	Boris Dayma	![在 Colab 中打开](https://colab.research.google.com/github/borisdayma/huggingtweets/blob/master/huggingtweets-demo.ipynb)
使用 Weights & Biases 优化 😍	完整教程展示如何将 W&B 集成到 Hugging Face 中	Boris Dayma	![在 Colab 中打开](https://colab.research.google.com/github/wandb/examples/blob/master/colabs/huggingface/Optimize_Hugging_Face_models_with_Weights_%26_Biases.ipynb)

Hugging Face 模型			
构建 Longformer 预训练模型	如何构建现有预训练模型的“长”版本	Iz Beltagy	![在 Colab 中打开](https://colab.research.google.com/github/allenai/longformer/blob/master/scripts/convert_model_to_long.ipynb)
微调 Longformer 用于问答任务	如何微调 longformer 模型用于问答任务	Suraj Patil	![在 Colab 中打开](https://colab.research.google.com/github/patil-suraj/Notebooks/blob/master/longformer_qa_training.ipynb)
使用 😭nlp 评估模型	如何使用 `nlp` 评估 longformer 在 TriviaQA 上的性能	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/drive/1m7eTGlPmLRgoPkkA7rkhQdZ9ydpmsdLE?usp=sharing)
微调 T5 用于情感跨度提取	如何使用 PyTorch Lightning 以文本到文本格式微调 T5 模型，用于情感跨度提取	Lorenzo Ampil	![在 Colab 中打开](https://colab.research.google.com/github/enzoampil/t5-intro/blob/master/t5_qa_training_pytorch_span_extraction.ipynb)
微调 DistilBert 用于多类别分类	如何使用 PyTorch 微调 DistilBert 模型，用于多类别分类	Abhishek Kumar Mishra	![在 Colab 中打开](https://colab.research.google.com/github/abhimishra91/transformers-tutorials/blob/master/transformers_multiclass_classification.ipynb)

微调 BERT 用于多标签分类	如何使用 PyTorch 微调 BERT 模型，用于多标签分类	Abhishek Kumar Mishra	![在 Colab 中打开](https://colab.research.google.com/github/abhimishra91/transformers-tutorials/blob/master/transformers_multi_label_classification.ipynb)
微调 T5 用于摘要	如何使用 PyTorch 微调 T5 模型用于摘要，并使用 WandB 跟踪实验	Abhishek Kumar Mishra	![在 Colab 中打开](https://colab.research.google.com/github/abhimishra91/transformers-tutorials/blob/master/transformers_summarization_wandb.ipynb)
使用动态填充/分桶加速 Transformer 微调	如何通过动态填充/分桶将微调速度提高两倍	Michael Benesty	![在 Colab 中打开](https://colab.research.google.com/drive/1CBfRU1zbfu7-ijiOqAAQUA-RJaxfcJoO?usp=sharing)
使用双向自注意力层预训练 Reformer	如何训练具有双向自注意力层的 Reformer 模型	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/drive/1tzzh0i8PgDQGV3SMFUGxM7_gGae3K-uW?usp=sharing)
扩展和微调 Sci-BERT	如何在 CORD 数据集上扩展 AllenAI 预训练的 SciBERT 模型并将其接入管道	Tanmay Thakur	![在 Colab 中打开](https://colab.research.google.com/drive/1rqAR40goxbAfez1xvF3hBJphSCsvXmh8)
使用 Trainer	如何在自定义数据集上使用	Tanmay	![在 Colab 中打开](https://colab.research.google.com/drive/19Wmupuls7mykSGyRN_

API 微调 BlenderBotSmall 用于摘要	Trainer API 微调 BlenderBotSmall 用于摘要	Thakur	Qo6lPQhgp56ymq?usp=sharing)
微调 Electra 并使用集成梯度解释预测	如何微调 Electra 模型用于情感分析，并使用 Captum 集成梯度解释预测	Eliza Szczechla	![在 Colab 中打开](https://colab.research.google.com/github/elsanns/xai-nlp-notebooks/blob/master/electra_fine_tune_interpret_captum_ig.ipynb)
使用 Trainer 类微调非英语 GPT-2 模型	如何使用 Trainer 类微调非英语 GPT-2 模型	Philipp Schmid	![在 Colab 中打开](https://colab.research.google.com/github/philschmid/fine-tune-GPT-2/blob/master/Fine_tune_a_non_English_GPT_2_Model_with_Huggingface.ipynb)
微调 DistilBERT 用于多标签分类任务	如何微调 DistilBERT 模型用于多标签分类任务	Dhaval Taunk	![在 Colab 中打开](https://colab.research.google.com/github/DhavalTaunk08/Transformers_scripts/blob/master/Transformers_multilabel_distilbert.ipynb)
微调 ALBERT 用于句子对分类	如何微调 ALBERT 模型或基于 BERT 的其他模型用于句子对分类任务	Nadir El Manouzi	![在 Colab 中打开](https://colab.research.google.com/github/NadirEM/nlp-notebooks/blob/master/Fine_tune_ALBERT_sentence_pair_classification.ipynb)
微调 Roberta 用于	如何微调 Roberta 模型用	Dhaval Taun	![在 Colab 中打开](https://colab.research.google.com/github/DhavalTaunk08/NLP_scr

情感分析	于情感分析	k	ipts/blob/master/sentiment_analysis_using_roberta.ipynb)
评估生成的问题模型	你的 seq2seq 变换器模型生成的问题答案有多准确?	Pascal Zoleko	![在 Colab 中打开](https://colab.research.google.com/drive/1bpsSqCQU-iw_5nNoRm_crPq6FRuJthq_?usp=sharing)
使用 DistilBERT 和 Tensorflow 分类文本	如何使用 TensorFlow 微调 DistilBERT 模型用于文本分类	Peter Bayerle	![在 Colab 中打开](https://colab.research.google.com/github/peterbayerle/huggingface_notebook/blob/main/distilbert_tf.ipynb)
使用 BERT 进行编码器-解码器摘要 （CNN/Dailymail）	如何使用 _google-bert/bert-base-uncased_ 检查点预热 _EncoderDecoderModel_，用于 CNN/Dailymail 的摘要	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/BERT2BERT_for_CNN_Dailymail.ipynb)
使用 RoBERTa 进行编码器-解码器摘要 （BBC XSum）	如何使用 _FacebookAI/roberta-base_ 检查点预热共享 _EncoderDecoderModel_，用于 BBC/XSum 的摘要	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/RoBERTaShared_for_BBC_XSum.ipynb)
微调 TAPAS	如何使用 _tapas-base_ 检查点微	Niels Rog	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transforme

用于顺序问答（SQA）	调_TapasForQuestionAnswering_，用于顺序问答（SQA）数据集	ge	rs-Tutorials/blob/master/TAPAS/Fine_tuning_TapasForQuestionAnswering_on_SQA.ipynb)
评估 TAPAS 用于表格事实检查（TabFact）	如何使用 🤗 datasets 和 🤗 transformers 库组合评估使用_tapas-base-finetuned-tabfact_ 检查点微调的_TapasForSequenceClassification_	Niels Rogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/TAPAS/Evaluating_TAPAS_on_the_Tabfact_test_set.ipynb)
微调 mBART 用于翻译	如何使用 Seq2SeqTrainer 微调 mBART 用于印地语到英语的翻译	Vasudev Gupta	![在 Colab 中打开](https://colab.research.google.com/github/vasudevgupta7/huggingface-tutorials/blob/main/translation_training.ipynb)
微调 LayoutLM 用于表格理解数据集 FUNSD	如何在 FUNSD 数据集上微调_LayoutLMForTokenClassification_，用于从扫描文档中提取信息	Niels Rogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/LayoutLM/Fine_tuning_LayoutLMForTokenClassification_on_FUNSD.ipynb)
微调 DistilGPT2 并生成文	如何微调 DistilGPT2 并生成文本	Aakash Tripathi	![在 Colab 中打开](https://colab.research.google.com/github/tripathiaakash/DistilGPT2-Tutorial/blob/main/distilgpt2_fine_tuning.ipynb)

本			
微调 LED 用于长达 8K 个标记的摘要	如何在 pubmed 数据集上微调 LED 用于长距离摘要	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/Fine_tune_Longformer_Encoder_Decoder_%28LED%29_for_Summarization_on_pubmed.ipynb)
在 Arxiv 上评估 LED	如何有效地在长距离摘要上评估 LED	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/LED_on_Arxiv.ipynb)
微调 LayoutLM 用于文档图像分类数据集 RVL-CDIP	如何在 RVL-CDIP 数据集上微调 _LayoutLMForSequenceClassification_，用于扫描文档分类	Niels Rogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/LayoutLM/Fine_tuning_LayoutLMForSequenceClassification_on_RVL_CDIP.ipynb)
使用 GPT2 调整 Wav2Vec2 CTC 解码	如何使用语言模型调整 CTC 序列解码	Eric Lam	![在 Colab 中打开](https://colab.research.google.com/drive/1e_z5jQHYbO2YKEaUgzb1ww1WwiAyydAj?usp=sharing)
使用 Trainer 类微调 BART	如何使用 Trainer 类微调 BART 用于双语摘要	Eliza Szczechl	![在 Colab 中打开](https://colab.research.google.com/github/elsanns/xai-nlp-notebooks/blob/master/fine_tune_bart_summarization_two_langs.ipy

用于双语摘要		a	nb)
在 Trivia QA 上评估 Big Bird	如何在 Trivia QA 上评估 BigBird 模型的长文档问答	Patrick von Platen	![在 Colab 中打开](https://colab.research.google.com/github/patrickvonplaten/notebooks/blob/master/Evaluating_Big_Bird_on_TriviaQA.ipynb)
使用 Wav2Vec2 创建视频字幕	如何通过转录音频使用 Wav2Vec 从任何视频创建 YouTube 字幕	Niklas Muennighoff	![在 Colab 中打开](https://colab.research.google.com/github/Muennighoff/ytclipcc/blob/main/wav2vec_youtube_captions.ipynb)
使用 PyTorch Lightning 微调 Vision Transformer 用于 CIFAR-10	如何使用 HuggingFace Transformers、Datasets 和 PyTorch Lightning 微调 Vision Transformer（ViT）用于 CIFAR-10	Niels Rogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/VisionTransformer/Fine_tuning_the_Vision_Transformer_on_CIFAR_10_with_PyTorch_Lightning.ipynb)
使用 🤗 Trainer 微调 Vision Transformer	如何使用 HuggingFace Transformers、Datasets 和 🤗 Trainer 微调 Vision Transformer	Niels Rogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/VisionTransformer/Fine_tuning_the_Vision_Transformer_on_CIFAR_10_with_the_%F0%9F%A4%97_Trainer.ipynb)

用于 CIFAR-10	（ViT）用于 CIFAR-10		
在 Open Entity 数据集上评估 LUKE	如何在 Open Entity 数据集上评估 _LukeForEntityClassification_	Ikuya Yamada	![在 Colab 中打开](https://colab.research.google.com/github/studio-ousia/luke/blob/master/notebooks/huggingface_open_entity.ipynb)
在 TACRED 数据集上评估 LUKE	如何在 TACRED 数据集上评估 _LukeForEntityPairClassification_	Ikuya Yamada	![在 Colab 中打开](https://colab.research.google.com/github/studio-ousia/luke/blob/master/notebooks/huggingface_tacred.ipynb)
在 CoNLL-2003 数据集上评估 LUKE	如何在 CoNLL-2003 数据集上评估 _LukeForEntitySpanClassification_	Ikuya Yamada	![在 Colab 中打开](https://colab.research.google.com/github/studio-ousia/luke/blob/master/notebooks/huggingface_conll_2003.ipynb)
在 PubMed 数据集上评估 BigBird-Pegasus	如何在 PubMed 数据集上评估 _BigBirdPegasusForConditionalGeneration_	Vasudev Gupta	![在 Colab 中打开](https://colab.research.google.com/github/vasudevgupta7/bigbird/blob/main/notebooks/bigbird_pegasus_evaluation.ipynb)
使用 Wav2V	如何使用预训练的 Wav2Vec2 模	Mehrdad	![在 Colab 中打开](https://colab.research.google.com/github/m3hrdadfi/soxan/blob/

ec2 进行语音情感分类	型在 MEGA 数据集上进行情感分类	Farahani	main/notebooks/Emotion_recognition_in_Greek_speech_using_Wav2Vec2.ipynb)
使用 DETR 检测图像中的对象	如何使用训练好的 _DetrForObjectDetection_ 模型检测图像中的对象并可视化注意力	NielsRogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/DETR/DETR_minimal_example_%28with_DetrFeatureExtractor%29.ipynb)
在自定义目标检测数据集上微调 DETR	如何在自定义目标检测数据集上微调 _DetrForObjectDetection_	NielsRogge	![在 Colab 中打开](https://colab.research.google.com/github/NielsRogge/Transformers-Tutorials/blob/master/DETR/Fine_tuning_DetrForObjectDetection_on_custom_dataset_%28balloon%29.ipynb)
微调 T5 用于命名实体识别	如何在命名实体识别任务上微调 _T5_	OgundepoOdunayo	![在 Colab 中打开](https://colab.research.google.com/drive/1obr78FY_cBmWY5ODViCmzdY6O1KB65Vc?usp=sharing)
使用 QLoRA 和 PEFT 微调开源大模型	如何使用 QLoRA 和 PEFT 以节省内存的方式微调大模型，同时使用 MLflow 管理实验跟踪	YukiWatanabe	![在 Colab 中打开](https://colab.research.google.com/github/mlflow/mlflow/blob/master/docs/source/llms/transformers/tutorials/fine-tuning/transformers-peft.ipynb)

Transformers 故障排除

有时会出现错误，但我们在这里帮助您！本指南涵盖了一些最常见的问题及其解决方法。然而，这并不是一个涵盖所有 🤗 Transformers 问题的详尽清单。如果您需要更多帮助，可以尝试以下方法：

255. 在 论坛上寻求帮助。您可以将问题发布到特定的类别中，例如 初学者 或 🤗 Transformers。确保您的帖子描述清楚并包含可复现的代码，以提高解决问题的可能性！
256. 如果是与库相关的 bug，请在 🤗 Transformers 仓库中创建一个 问题。尽量提供尽可能多的信息来描述 bug，以便我们更好地找出问题所在并解决它。
257. 如果您使用的是较旧版本的 🤗 Transformers，请查看 迁移指南，因为不同版本之间引入了一些重要的更改。

有关故障排除和获取帮助的更多详细信息，请参阅 Hugging Face 课程的 第 8 章。

网络隔离环境

某些云上的 GPU 实例或内部网络设置可能被防火墙隔离，导致连接错误。当您的脚本尝试下载模型权重或数据集时，下载会挂起，然后超时，并显示以下消息：

在这种情况下，您应该尝试在离线模式下运行 🤗 Transformers 以避免连接错误。

CUDA 内存不足

没有适当的硬件，训练具有数百万参数的大模型可能会很有挑战性。当 GPU 内存不足时，您可能会遇到以下常见错误：

以下是一些减少内存使用的潜在解决方案：

- 减少 TrainingArguments 中的 `per_device_train_batch_size` 值。
- 尝试在 TrainingArguments 中使用 `gradient_accumulation_steps` 来有效增加整体批次大小。

有关节省内存技巧的更多详细信息，请参阅 性能指南。

无法加载保存的 TensorFlow 模型

TensorFlow 的 model.save 方法会将整个模型（包括架构、权重和训练配置）保存在一个文件中。但是，当您再次加载模型文件时，可能会遇到错误，因为 🤗 Transformers 可能无法加载模型文件中的所有 TensorFlow 相关对象。为了避免保存和加载 TensorFlow 模型时出现问题，我们建议您：

- 使用 `model.save_weights` 将模型权重保存为 `.h5` 文件扩展名，然后使用 `from_pretrained()` 重新加载模型：

```
from transformers import TFPreTrainedModel
from tensorflow import keras

model.save_weights("some_folder/tf_model.h5")
model = TFPreTrainedModel.from_pretrained("some_folder")
```

- 使用 `~TFPretrainedModel.save_pretrained` 保存模型，并使用 `from_pretrained()` 重新加载模型：

```
from transformers import TFPreTrainedModel

model.save_pretrained("path_to/model")
model = TFPreTrainedModel.from_pretrained("path_to/model")
```

导入错误

您可能会遇到的另一个常见错误是 `ImportError`，尤其是对于新发布的模型：

对于这些类型的错误，请确保您安装了最新版本的 🤗 Transformers 以访问最新的模型：

```
pip install transformers --upgrade
```

CUDA 错误：设备端断言触发

有时您可能会遇到一个通用的 CUDA 错误，提示设备代码中有错误：

您应该首先在 CPU 上运行代码以获取更详细的错误消息。在代码开头添加以下环境变量以切换到 CPU：

```
import os

os.environ["CUDA_VISIBLE_DEVICES"] = ""
```

另一种选择是获取更好的 GPU 回溯信息。在代码开头添加以下环境变量以获取指向错误源的回溯：

```
import os

os.environ["CUDA_LAUNCH_BLOCKING"] = "1"
```

当填充标记未被屏蔽时输出不正确

在某些情况下，如果 `input_ids` 包含填充标记，输出 `hidden_state` 可能不正确。为了演示这一点，加载一个模型和分词器。您可以访问模型的 `pad_token_id` 来查看其值。对于某些模型，`pad_token_id` 可能为 `None`，但您可以始终手动设置它。

```
from transformers import AutoModelForSequenceClassification
import torch

model = AutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-uncased")
model.config.pad_token_id
```

以下示例展示了未屏蔽填充标记时的输出：

```
input_ids = torch.tensor([[7592, 2057, 2097, 2393, 9611, 2115], [7592, 0, 0, 0, 0, 0]])
output = model(input_ids)
print(output.logits)
```

以下是第二个序列的实际输出：

```
input_ids = torch.tensor([[7592]])
output = model(input_ids)
print(output.logits)
```

大多数情况下，您应该向模型提供 `attention_mask` 以忽略填充标记，以避免这种无声错误。现在，第二个序列的输出与其实际输出匹配：

默认情况下，分词器会根据您使用的特定分词器的默认设置为您创建 `attention_mask`。

```
attention_mask = torch.tensor([[1, 1, 1, 1, 1, 1], [1, 0, 0, 0, 0, 0]])
output = model(input_ids, attention_mask=attention_mask)
print(output.logits)
```

😭 Transformers 不会在提供填充标记时自动创建 `attention_mask` 以屏蔽填充标记，因为：

- 有些模型没有填充标记。
- 对于某些用途，用户希望模型关注填充标记。

ValueError: 无法识别此种类的 AutoModel 的配置类 XYZ

通常，我们建议使用 AutoModel 类来加载预训练模型。这个类可以根据给定检查点的配置自动推断并加载正确的架构。如果您在从检查点加载模型时看到此 `ValueError`，这意味着 Auto 类无法将给定检查点中的配置映射到您尝试加载的模型类型。最常见的原因是检查点不支持给定任务。例如，您会在以下示例中看到此错误，因为 GPT2 没有用于问答的任务模型：

```
from transformers import AutoProcessor, AutoModelForQuestionAnswering

processor = AutoProcessor.from_pretrained("openai-community/gpt2-medium")
model = AutoModelForQuestionAnswering.from_pretrained("openai-community/gpt2-medium")
```

在 Transformers 中加载 GGUF 文件

GGUF 文件格式用于存储与 GGML 及其依赖库（如非常流行的 llama.cpp 或 whisper.cpp）一起进行推理的模型。

这是一种由 Hugging Face Hub 支持的文件格式，具有快速检查文件中张量和元数据的功能。

这种文件格式设计为“单文件格式”，通常一个文件中包含配置属性、分词器词汇表和其他属性，以及要加载到模型中的所有张量。这些文件根据文件的量化类型有不同的格式。我们在这里简要介绍一些常见的格式：量化类型。

Transformer 中的支持

我们在 `transformers` 中添加了加载 `gguf` 文件的能力，以提供对 gguf 模型进一步训练或微调的能力，并在完成后再将这些模型转换回 `gguf` 格式，以便在 `ggml` 生态系统中使用。加载模型时，我们会先将其去量化为 fp32，然后再加载权重以供 PyTorch 使用。

> **注意**：当前的支持仍处于探索性阶段，我们欢迎贡献以巩固不同量化类型和模型架构的支持。

目前支持的模型架构和量化类型如下：

支持的量化类型

初始支持的量化类型是根据在 Hub 上共享的流行量化文件决定的。

- F32
- F16
- BF16
- Q4_0
- Q4_1
- Q5_0
- Q5_1
- Q8_0
- Q2_K
- Q3_K
- Q4_K
- Q5_K
- Q6_K
- IQ1_S
- IQ1_M
- IQ2_XXS
- IQ2_XS
- IQ2_S
- IQ3_XXS
- IQ3_S
- IQ4_XS
- IQ4_NL

> **注意**：为了支持 gguf 去量化，需要安装 `gguf>=0.10.0`。

支持的模型架构

目前支持的模型架构是在 Hub 上非常流行的架构，包括：

- LLaMa
- Mistral

- Qwen2
- Qwen2Moe
- Phi3
- Bloom
- Falcon
- StableLM
- GPT2
- Starcoder2

示例用法

要在 `transformers` 中加载 `gguf` 文件，您需要在 `from_pretrained` 方法中指定 `gguf_file` 参数，用于加载分词器和模型。以下是如何加载分词器和模型的示例，它们可以从同一个文件中加载：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_id = "TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF"
filename = "tinyllama-1.1b-chat-v1.0.Q6_K.gguf"

tokenizer = AutoTokenizer.from_pretrained(model_id, gguf_file=filename)
model = AutoModelForCausalLM.from_pretrained(model_id, gguf_file=filename)
```

现在，您可以在 PyTorch 生态系统中访问完整的去量化模型，并结合其他多种工具使用。

为了将模型重新转换为 `gguf` 文件，我们推荐使用 llama.cpp 中的 `convert-hf-to-gguf.py` 脚本。

以下是完成上述脚本以保存模型并将其导出为 `gguf` 文件的方法：

```
tokenizer.save_pretrained('directory')
model.save_pretrained('directory')

!python ${path_to_llama_cpp}/convert-hf-to-gguf.py ${directory}
```

在 Transformers 中加载 Tiktoken 文件

在加载模型时，🥹 Transformers 无缝支持 tiktoken 模型文件。当你从 Hub 加载带有 `tokenizer.model` tiktoken 文件的模型（通过 `from_pretrained` 方法）时，该文件会自动转换为我们的 快速分词器。

已知发布带有 tiktoken.model 的模型：

- gpt2
- llama3

示例用法

为了在 `transformers` 中加载 `tiktoken` 文件，请确保 `tokenizer.model` 文件是一个 tiktoken 文件，它会在加载 `from_pretrained` 时自动加载。以下是如何加载分词器和模型的示例，这两个对象可以从同一个文件中加载：

```
from transformers import AutoTokenizer

model_id = "meta-llama/Meta-Llama-3-8B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id, subfolder="original")
```

模块化 Transformers

`transformers` 是一个有明确主张的框架；我们的理念在以下 概念指南 中定义。

这一理念的核心体现在库的“单模型、单文件”方面。这个组件的缺点是限制了从文件到工具包中其他部分的继承和导入。

因此，模型组件在许多文件中重复出现。`transformers` 中定义了与模型数量相同的关注层（attention layers），其中许多是相同的。不幸的后果是，当对特定部分的代码进行修复和更改时，独立实现往往会偏离。

为了平衡这个问题，我们在库中引入了“复制”的概念。通过添加一个注释表明代码是从另一个地方复制的，我们可以通过持续集成（CI）和本地命令来确保复制的代码不会偏离。虽然复杂度较低，但这通常是相当繁琐的。

最后，这增加了贡献模型的显著开销，这是我们希望消除的。这种方法通常需要模型贡献者添加建模代码（约 1000 行）、处理器（约 500 行）、测试、文档等。模型贡献的拉取请求（PR）很少少于 3-5000 行代码，其中大部分是样板代码。

这提高了贡献的门槛，而通过模块化 Transformers（Modular Transformers），我们旨在降低这个门槛到一个更可接受的点。

什么是模块化 Transformers?

模块化 Transformers 在模型文件夹中引入了“模块化”文件的概念。这个模块化文件接受通常不在建模/处理文件中接受的代码，因为它允许从相邻模型导入以及类之间的继承。

这个模块化文件定义了原本在各自模块中定义的模型、处理器和配置类。

最后，这个功能引入了一个新的 `linter`，它会将模块化文件“展开”为“单模型、单文件”目录结构。每次运行脚本时，这些文件都会自动生成；减少了对模块化文件的贡献，因此只需要更改贡献模型与其他模型之间的差异。

模型用户最终会导入并使用单文件接口，因此这里没有变化。这样做，我们希望结合两种方法的优点：简化贡献同时坚持我们的理念。

因此，这是对 `# Copied from` 标记的替代，并且预计以前贡献的模型将在未来的几个月内迁移到新的模块化格式。

细节

“linter”会展开继承关系，并从模块化文件创建所有单文件。在这个过程中，linter 会尽量对 Python 用户透明地展平继承关系。目前，linter 只展平 **单层** 继承。

例如：

- 如果一个配置类继承了另一个类并添加或删除了一个参数，生成的文件将直接引用它（在添加的情况下）或完全删除它（在删除的情况下）。
- 如果一个类继承了另一个类，例如 `class GemmaModel(LlamaModel):`，依赖关系将自动推断。所有子模块将从父类中自动推断。
- 如果你在 `modular` 文件中定义了新函数并在类中使用它们，linter 将自动推断并复制这些函数。

你应该能够在 `modular` 文件中编写一切（分词器、图像处理器、模型、配置），相应的文件将为你自动生成。

强制执行

\[TODO\] 我们引入了一项新的测试，确保生成的内容与 `modular_xxxx.py` 中的内容匹配。

示例

这里是一个快速示例，展示 BERT 和 RoBERTa。这两个模型密切相关：它们的建模实现仅在嵌入层有所不同。

与其重新定义整个模型，以下是 `modular_roberta.py` 文件中建模和配置类的样子（为了示例，暂时忽略了分词器，因为它们非常不同）。

```
from torch import nn
from ..bert.configuration_bert import BertConfig
from ..bert.modeling_bert import (
    BertModel,
    BertEmbeddings,
    BertForMaskedLM
)

# RoBERTa 配置与 BERT 的配置相同
class RobertaConfig(BertConfig):
  model_type = 'roberta'

# 重新定义嵌入层以突出填充 ID 的差异，并重新定义位置嵌入
class RobertaEmbeddings(BertEmbeddings):
    def __init__(self, config):
        super().__init__(config)

        self.padding_idx = config.pad_token_id
        self.position_embeddings = nn.Embedding(
            config.max_position_embeddings, config.hidden_size, padding_idx=self.padding_idx
        )

# RoBERTa 模型与 BERT 模型相同，除了嵌入层。
# 我们在上面重新定义了嵌入层，因此这里不需要额外的工作
class RobertaModel(BertModel):
  def __init__(self, config):
    super().__init__(config)
    self.embeddings = RobertaEmbeddings(config)

# 头部现在只需要将模型内部重新定义为正确的 `RobertaModel`
class RobertaForMaskedLM(BertForMaskedLM):
  def __init__(self, config):
```

```
    super().__init__(config)
    self.model = RobertaModel(config)
```

如果你没有使用你定义的依赖关系，将会出现以下错误：

此外，你可以在这里找到更多示例：

它不是什么

它并不是建模代码的替代品（至少目前还不是），如果您的模型没有基于任何已存在的东西，您可以像往常一样添加一个 `modeling` 文件。

高级用法

删除属性和函数

要删除模块化模型中未使用的属性，并且不希望在展开的建模文件中看到这些属性：

Example 1: GemmaModel 类定义 (LlamaModel)

```
class GemmaModel(LlamaModel):
    def __init__(self, config):
        super().__init__(config)
        del self.embed_tokens
```

Example 2: GemmaModel 类定义 (PreTrainedModel)

```
class GemmaModel(PreTrainedModel):
    def __init__(self, config):
        super().__init__(config)
        self.padding_idx = config.pad_token_id
        self.vocab_size = config.vocab_size

        self.layers = nn.ModuleList(
            [LlamaDecoderLayer(config, layer_idx) for layer_idx in
range(config.num_hidden_layers)]
        )
        self.norm = LlamaRMSNorm(config.hidden_size, eps=config.rms_norm_eps)
        self.rotary_emb = LlamaRotaryEmbedding(config=config)
        self.gradient_checkpointing = False

        # 初始化权重并应用最终处理
        self.post_init()
```

如果你检查原始的 `LlamaModel`，它有一个 `embed_tokens` 属性，这里将会被删除。

删除函数也很类似，只需将其写成包含 `raise ValueError("")` 来模拟在 Python 中删除父类函数的行为。

```
class GemmaTokenizer(LlamaTokenizer):
    # ...

    def get_spm_processor(self):
        raise AttributeError("Gemma 不需要此功能")

    def unk_token_length(self):
        raise AttributeError("Gemma 不需要此功能")
```

定义新函数

如果你在 `modular` 文件中定义了一个新函数并在类中使用它，例如：

```
def my_new_function(*args, **kwargs):
  # 在这里做某事
  pass

class GemmaModel(LlamaModel):
    def forward(*args, **kwargs):
      # 调用函数
      example = my_new_function(*args, **kwargs)
      # 继续操作
```

`my_new_function` 函数（及其递归调用的任何其他新函数）将自动复制到使用它的文件中。

调用 super()

我们最近推出了一些功能，允许你从 Part 1 更新到 Part 2 的调用方式：

Part 1: GemmaTokenizer 类定义

```
class GemmaTokenizer(LlamaTokenizer, PretrainedTokenizerFast):
    def __init__(self, eos_token="</s>"):
        eos_token = AddedToken(eos_token)
        PretrainedTokenizerFast.__init__(self, eos_token)
```

Part 2: GemmaModel 类定义

```
class GemmaModel(nn.Module):
    def __init__(self):
```

```
        eos_token = AddedToken(eos_token)
        super().__init__(eos_token)
```

这在你 **不想** 展开对 `super()` 的调用，并且想区分你在调用哪个父类的初始化方法时非常有用！

特殊命名

我们现在还支持特殊情况，例如：

```
class GemmaVisionModel(CLIPModel):
    pass
```

这里你的类名 `GemmaVision` 与模块化的 `Gemma` 不同。这对于复合模型非常有用。

如何修改 Transformers 模型

🤗 Transformers 库提供了一系列预训练模型和工具，用于自然语言处理、计算机视觉等多领域的应用。虽然这些模型覆盖了广泛的应用场景，但你可能会遇到一些不支持的用例。自定义模型可以解锁新的可能性，例如添加新层、修改架构或优化注意力机制。本指南将向你展示如何修改现有的 Transformer 模型以满足特定需求。好消息是，你不必离开 Transformers 框架就能进行这些修改。你可以直接在 Transformers 中修改模型，并仍然利用 Trainer API、PreTrainedModel 和高效的微调工具如 PEFT 等功能。

在本指南中，我们将逐步介绍如何自定义现有的 Transformer 模型，以满足你的需求，同时不失去生态系统的优势。

你将学习如何：

- 通过修改注意力机制来调整模型架构。
- 将低秩适应（LoRA）等技术应用于特定模型组件。

我们鼓励你贡献自己的“修改”方法，并与社区分享。

示例：修改 Segment Anything Model (SAM)的注意力机制

Segment Anything Model (SAM) 是一个最先进的图像分割模型。在默认实现中，SAM 在注意力机制中使用了一个组合的查询-键-值（`qkv`）投影。然而，你可能希望仅对注意力机制的特定组件进行微调，例如查询（`q`）和值（`v`）投影，以减少可训练参数的数量和计算资源的需求。

动机

通过将组合的 `qkv` 投影拆分为单独的 `q`、`k` 和 `v` 投影，你可以将 **LoRA**（低秩适应）等技术应用于 `q` 和 `v` 投影。这种方法可以让你：

- 减少可训练参数的数量，降低计算开销。
- 通过专注于特定组件，潜在地提高性能。
- 在注意力机制中实验不同的适应策略。

实现

第一步：创建自定义注意力类

接下来，派生原始的 `SamVisionAttention` 类并修改它，使其具有单独的 `q`、`k` 和 `v` 投影。

```
import torch
import torch.nn as nn
from transformers.models.sam.modeling_sam import SamVisionAttention

class SamVisionAttentionSplit(SamVisionAttention, nn.Module):
    def __init__(self, config, window_size):
        super().__init__(config, window_size)
        del self.qkv
        # 创建单独的 q, k, v 投影
        self.q = nn.Linear(config.hidden_size, config.hidden_size, bias=config.qkv_bias)
        self.k = nn.Linear(config.hidden_size, config.hidden_size, bias=config.qkv_bias)
        self.v = nn.Linear(config.hidden_size, config.hidden_size, bias=config.qkv_bias)
        self._register_load_state_dict_pre_hook(self.split_q_k_v_load_hook)

    def split_q_k_v_load_hook(self, state_dict, prefix, *args):
        keys_to_delete = []
        for key in list(state_dict.keys()):
            if "qkv." in key:
                # 将 q, k, v 从组合投影中分离
                q, k, v = state_dict[key].chunk(3, dim=0)
                # 用单独的 q, k, v 投影替换
                state_dict[key.replace("qkv.", "q.")] = q
                state_dict[key.replace("qkv.", "k.")] = k
                state_dict[key.replace("qkv.", "v.")] = v
                # 标记旧的 qkv 键以便删除
                keys_to_delete.append(key)

        # 删除旧的 qkv 键
        for key in keys_to_delete:
```

```
            del state_dict[key]

    def forward(self, hidden_states: torch.Tensor, output_attentions=False) -> torch.Tensor:
        batch_size, height, width, _ = hidden_states.shape
        qkv_shapes = (batch_size * self.num_attention_heads, height * width, -1)
        query = self.q(hidden_states).reshape((batch_size, height * width,
self.num_attention_heads, -1)).permute(0, 2, 1, 3).reshape(qkv_shapes)
        key = self.k(hidden_states).reshape((batch_size, height * width,
self.num_attention_heads, -1)).permute(0, 2, 1, 3).reshape(qkv_shapes)
        value = self.v(hidden_states).reshape((batch_size, height * width,
self.num_attention_heads, -1)).permute(0, 2, 1, 3).reshape(qkv_shapes)

        attn_weights = (query * self.scale) @ key.transpose(-2, -1)

        if self.use_rel_pos:
            attn_weights = self.add_decomposed_rel_pos(
                attn_weights, query, self.rel_pos_h, self.rel_pos_w, (height, width), (height,
width)
            )

        attn_weights = torch.nn.functional.softmax(attn_weights, dtype=torch.float32, dim=-
1).to(query.dtype)
        attn_probs = nn.functional.dropout(attn_weights, p=self.dropout, training=self.training)
        attn_output = (attn_probs @ value).reshape(batch_size, self.num_attention_heads, height,
width, -1)
        attn_output = attn_output.permute(0, 2, 3, 1, 4).reshape(batch_size, height, width, -1)
        attn_output = self.proj(attn_output)

        if output_attentions:
            outputs = (attn_output, attn_weights)
        else:
            outputs = (attn_output, None)
        return outputs
```

解释：

- **单独投影：** 移除了组合的 `qkv` 投影，创建了单独的 `q`、`k` 和 `v` 线性层。
- **权重加载钩子：** `_split_qkv_load_hook` 方法在加载模型时将预训练的 `qkv` 权重拆分为单独的 `q`、`k` 和 `v` 权重，确保与任何预训练模型的兼容性。
- **前向传播：** 分别计算查询、键和值，注意力机制按常规方式进行。

第二步：替换原始的注意力类

将原始的 `SamVisionAttention` 类替换为自定义类，以便模型使用修改后的注意力机制。

```
from transformers import SamModel
from transformers.models.sam import modeling_sam
```

```
# 替换 modeling_sam 模块中的注意力类
modeling_sam.SamVisionAttention = SamVisionAttentionSplit

# 加载预训练的 SAM 模型
model = SamModel.from_pretrained("facebook/sam-vit-base")
```

解释：

- **类替换：** 通过将自定义类赋值给 `modeling_sam.SamVisionAttention`，模型中的所有 `SamVisionAttention` 实例都将使用修改后的版本。因此，当你调用 `SamModel` 时，它将使用新定义的 `SamVisionAttentionSplit`。
- **模型加载：** 使用 `from_pretrained` 加载模型，并集成自定义的注意力机制。

第三步：将 LoRA 应用于特定投影

现在有了单独的 `q`、`k` 和 `v` 投影，你可以将 LoRA 应用于特定组件，例如 `q` 和 `v` 投影。

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q", "v"],  # 将 LoRA 应用于 q 和 v 投影
    lora_dropout=0.1,
    task_type="mask-generation"
)

# 将 LoRA 应用于模型
model = get_peft_model(model, config)
```

解释：

- **LoRA 配置：** `LoraConfig` 指定了秩 `r`、缩放因子 `lora_alpha`、目标模块（`"q"`和`"v"`）、dropout 和任务类型。
- **应用 LoRA：** `get_peft_model` 函数将 LoRA 应用于模型中指定的模块。
- **参数减少：** 通过专注于 `q` 和 `v`，减少了可训练参数的数量，从而加快训练速度并降低内存使用量。

第四步：验证可训练参数的数量

验证可训练参数的数量，看看你的修改带来了什么影响。

```
model.print_trainable_parameters()
```

预期输出：

贡献你自己的“修改”方法

修改预训练模型可以为研究和应用开辟新的途径。通过理解和调整像 SAM 这样的模型的内部机制，你可以根据需要定制它们，优化性能并尝试新的想法。

如果你已经为 Transformers 模型开发了自己的“修改”方法，并希望分享它们，请考虑为本文档贡献代码。

- **发起 Pull Request：** 直接在仓库中分享你的代码更改和改进。
- **编写文档：** 提供清晰的解释和示例，说明你的修改。
- **参与社区：** 通过打开问题与开发者和研究人员讨论你的想法并获得反馈。

第五部分 量化方法

量化技术概述

量化技术旨在用更少的信息表示数据，同时尽量不损失太多的准确性。这通常意味着将数据类型转换为用更少位数表示相同信息的形式。例如，如果你的模型权重存储为 32 位浮点数，并将其量化为 16 位浮点数，模型大小将减半，这样更容易存储并减少内存使用。较低的精度还可以加速推理，因为用更少的位数进行计算所需的时间更短。

想要在 Transformers 中添加新的量化方法吗？请阅读 HfQuantizer 指南以了解如何操作！

如果你是量化领域的初学者，我们建议你查看这些与 DeepLearning.AI 合作的适合初学者的量化课程：

- 使用 Hugging Face 的量化基础
- 深入量化

何时使用哪种方法?

社区已经开发了许多适用于不同场景的量化方法。使用 Transformers 时，可以根据你的具体需求运行任何已集成的方法，因为每种方法都有其优缺点。

例如，一些量化方法需要使用数据集校准模型以实现更准确和“极端”的压缩（最多可压缩到 1-2 位），而其他方法则可以在不使用数据集的情况下即时量化。

另一个需要考虑的参数是与目标设备的兼容性。你想在 CPU、GPU 还是 Apple 芯片上进行量化?

简而言之，支持广泛的量化方法可以让你选择最适合你特定需求的最佳量化方法。

使用下表来帮助你决定使用哪种量化方法。

量化方法	即时量化	CPU	CUDA GPU	RoCm GPU (AMD)	Metal (Apple Silicon)	支持 torch.compile()	位数	支持微调 （通过 PEFT）	可与🤗 transformers 序列化	🤗 transformers 支持	库链接
AQLM	❌	✔	✔	❌	❌	✔	1 / 2	✔	✔	✔	https://github.com/Vahe1994/AQLM
AWQ	❌	❌	✔	✔	❌	?	4	✔	✔	✔	https://github.com/casper-hansen/AutoAWQ
bitsandbytes	✔	⚠ *	✔	⚠ *	❌ |	❌(即将支持！)	4 / 8	✔	✔	✔	https://github.com/bitsandbytes-foundation/bitsandbytes
compre	❌	✔	✔	✔	❌	❌	1	✔	✔	✔	https://github.com/neura

ssed-tensors							-8				lmagic/compressed-tensors
EETQ	✓	✗	✓	✗	✗	?	8	✓	✓	✓	https://github.com/NetEase-FuXi/EETQ
GGUF / GGML (llama.cpp)	✓	✓	✓	✗	✓	✗	1-8	✗	见 GGUF 部分	见 GGUF 部分	https://github.com/ggerganov/llama.cpp
GPTQ	✗	✗	✓	✓	✗	✗	2/3/4/8	✓	✓	✓	https://github.com/AutoGPTQ/AutoGPTQ
HQQ	✓	✓	✓	✗	✗	✓	1-8	✓	✗	✓	https://github.com/mobiusml/hqq/
Quanto	✓	✓	✓	✗	✓	✓	2/4/8	✗	✗	✓	https://github.com/huggingface/quanto
FBGEM	✓	✗	✓	✗	✗	✗	8	✗	✓	✓	https://github.com/pytor

M_FP8											ch/FBGEMM
torchao	✓		✓	✗	部分支持（仅 int4 权重）		4 / 8		✓✗	✓	https://github.com/pytorch/ao

* bitsandbytes 正在重构以支持多个后端，不仅限于 CUDA。目前，ROCm（AMD GPU）和 Intel CPU 的实现已经成熟，Intel XPU 正在开发中，预计在今年第四季度或明年第一季度支持 Apple Silicon。有关安装说明和最新的后端更新，请访问 此链接。

量化模型（8 位与 4 位量化）

bitsandbytes 是量化模型到 8 位和 4 位的最简单选择。8 位量化通过将 fp16 中的异常值与 int8 中的非异常值相乘，然后将非异常值转换回 fp16 并将其相加，从而返回 fp16 格式的权重。这减少了异常值对模型性能的负面影响。4 位量化进一步压缩了模型，通常与 QLoRA 一起用于微调量化的大规模语言模型（LLMs）。

要使用 bitsandbytes，请确保安装了以下库：

8-bit

```
pip install transformers accelerate bitsandbytes>0.37.0
```

4-bit

```
pip install bitsandbytes>=0.39.0
pip install --upgrade accelerate transformers
```

bitsandbytes 正在重构以支持多种后端，而不仅仅是 CUDA。目前，ROCm（AMD GPU）和 Intel CPU 的实现已经成熟，Intel XPU 的实现正在进行中，Apple Silicon 的支持预计在 Q4 或 Q1 完成。有关安装说明和最新的后端更新，请访问 这个链接。

我们欢迎您的反馈，以便在正式发布前发现并修复问题！详情请参阅 这些文档 并提供反馈链接。

现在，您可以通过向 from_pretrained() 方法传递一个 `BitsAndBytesConfig` 来量化模型。只要模型支持使用 Accelerate 加载，并且包含 `torch.nn.Linear` 层，这种方法适用于任何模态的任何模型。

8 位量化

8 位量化可以将模型的内存占用减少一半，对于大型模型，设置 `device_map="auto"` 可以高效地利用可用的 GPU：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_8bit=True)

model_8bit = AutoModelForCausalLM.from_pretrained(
    "bigscience/bloom-1b7",
    quantization_config=quantization_config
)
```

默认情况下，所有其他模块（如 `torch.nn.LayerNorm`）都会被转换为 `torch.float16`。如果您希望更改这些模块的数据类型，可以使用 `torch_dtype` 参数：

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_8bit=True)

model_8bit = AutoModelForCausalLM.from_pretrained(
    "facebook/opt-350m",
    quantization_config=quantization_config,
    torch_dtype=torch.float32
)

# 检查最后一个层的权重数据类型
print(model_8bit.model.decoder.layers[-1].final_layer_norm.weight.dtype)
```

一旦模型被量化为 8 位，除非您使用的是最新版本的 Transformers 和 bitsandbytes，否则无法将量化后的权重推送到 Hub。如果您有最新版本，则可以使用 push_to_hub() 方法将 8 位模型推送到 Hub。首先推送 `config.json` 文件，然后推送量化后的模型权重：

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_8bit=True)

model = AutoModelForCausalLM.from_pretrained(
    "bigscience/bloom-560m",
    quantization_config=quantization_config
)
```

```
tokenizer = AutoTokenizer.from_pretrained("bigscience/bloom-560m")

model.push_to_hub("bloom-560m-8bit")
```

4 位量化

将模型量化为 4 位可以将内存使用量减少 4 倍，对于大型模型，设置 `device_map="auto"` 可以高效地利用可用的 GPU：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

# 配置 4 位量化
quantization_config = BitsAndBytesConfig(load_in_4bit=True)

# 加载预训练模型并应用量化配置
model_4bit = AutoModelForCausalLM.from_pretrained(
    "bigscience/bloom-1b7",
    quantization_config=quantization_config
)
```

默认情况下，所有其他模块（如 `torch.nn.LayerNorm`）会被转换为 `torch.float16`。如果需要，你可以通过 `torch_dtype` 参数更改这些模块的数据类型：

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

# 配置 4 位量化
quantization_config = BitsAndBytesConfig(load_in_4bit=True)

# 加载预训练模型并应用量化配置，同时指定数据类型为 `torch.float32`
model_4bit = AutoModelForCausalLM.from_pretrained(
    "facebook/opt-350m",
    quantization_config=quantization_config,
    torch_dtype=torch.float32
)

# 检查最后一层的权重数据类型
model_4bit.model.decoder.layers[-1].final_layer_norm.weight.dtype
```

如果你安装了 `bitsandbytes>=0.41.3`，你可以将 4 位模型序列化并推送到 Hugging Face Hub。只需在加载 4 位精度模型后调用 `model.push_to_hub()`。你也可以使用 `model.save_pretrained()` 命令将序列化的 4 位模型保存到本地。

使用 8 位和 4 位权重仅支持训练额外的参数。

您可以使用 `get_memory_footprint` 方法检查内存占用情况：

```
print(model.get_memory_footprint())
```

从 from_pretrained() 方法加载量化模型时，无需指定 `load_in_8bit` 或 `load_in_4bit` 参数：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("{your_username}/bloom-560m-8bit",
device_map="auto")
```

8 位量化 (LLM.int8() 算法)

了解更多关于 8 位量化的细节，请阅读这篇 博客文章!

这一部分探讨了 8 位模型的一些特定功能，如卸载、异常值阈值、跳过模块转换和微调。

卸载

8 位模型可以在 CPU 和 GPU 之间卸载权重，以支持将非常大的模型加载到内存中。卸载到 CPU 的权重实际存储为 **float32**，而不是 8 位。例如，要启用 bigscience/bloom-1b7 模型的卸载，首先创建一个 BitsAndBytesConfig：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(llm_int8_enable_fp32_cpu_offload=True)
```

设计一个自定义的设备映射，将大部分内容加载到 GPU 上，但将 `lm_head` 模块卸载到 CPU：

```
device_map = {
    "transformer.word_embeddings": 0,
    "transformer.word_embeddings_layernorm": 0,
    "lm_head": "cpu",
    "transformer.h": 0,
    "transformer.ln_f": 0,
}
```

现在使用自定义的 `device_map` 和 `quantization_config` 加载模型：

```
model_8bit = AutoModelForCausalLM.from_pretrained(
    "bigscience/bloom-1b7",
    device_map=device_map,
    quantization_config=quantization_config,
)
```

异常值阈值

“异常值”是指大于某个阈值的隐藏状态值，这些值是在 fp16 中计算的。通常这些值的分布范围在 \[-3.5, 3.5\] 之间，但对于大型模型，分布可能会非常不同（如 \[-60, 6\] 或 \[6, 60\]）。8 位量化在处理约 5 以内的值时效果很好，但超出这个范围会有显著的性能损失。默认的阈值是 6，但对于不太稳定的模型（小模型或微调），可能需要更低的阈值。

要找到适合您模型的最佳阈值，建议尝试 BitsAndBytesConfig 中的 `llm_int8_threshold` 参数：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

model_id = "bigscience/bloom-1b7"

quantization_config = BitsAndBytesConfig(
    llm_int8_threshold=10,
)

model_8bit = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map=device_map,
    quantization_config=quantization_config,
)
```

跳过模块转换

对于某些模型，如 Jukebox，不需要将每个模块都量化为 8 位，这实际上可能会导致不稳定性。对于 Jukebox，有几个 `lm_head` 模块应使用 `llm_int8_skip_modules` 参数跳过，具体配置如下：

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

model_id = "bigscience/bloom-1b7"

quantization_config = BitsAndBytesConfig(
    llm_int8_skip_modules=["lm_head"],
)

model_8bit = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    quantization_config=quantization_config,
)
```

微调

使用 PEFT 库，您可以使用 8 位量化来微调大型模型，如 flan-t5-large 和 facebook/opt-6.7b。训练时不需要传递 `device_map` 参数，因为它会自动将模型加载到 GPU 上。但是，您仍然可以使用 `device_map` 参数来自定义设备映射（`device_map="auto"` 仅应用于推理）。

4 位量化 (QLoRA 算法)

尝试 4 位量化的方法，请参考这个 笔记本，并了解更多详细信息，可以阅读这篇 博客文章。

这一部分探讨了 4 位模型的一些特定功能，如更改计算数据类型、使用 Normal Float 4 (NF4) 数据类型和使用嵌套量化。

计算数据类型

为了加快计算速度，您可以将数据类型从 float32（默认值）更改为 bf16，具体配置如下：

```
import torch
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_4bit=True,
bnb_4bit_compute_dtype=torch.bfloat16)
```

Normal Float 4 (NF4)

NF4 是 QLoRA 论文中提出的一种 4 位数据类型，适用于从正态分布初始化的权重。训练 4 位基础模型时应使用 NF4。这可以通过 BitsAndBytesConfig 中的 `bnb_4bit_quant_type` 参数来配置：

```
from transformers import BitsAndBytesConfig

nf4_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
)

model_nf4 = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=nf4_config)
```

对于推理，`bnb_4bit_quant_type` 对性能影响不大。但是，为了保持与模型权重的一致性，您应该使用 `bnb_4bit_compute_dtype` 和 `torch_dtype` 值。

嵌套量化

嵌套量化是一种可以在不牺牲性能的情况下节省额外内存的技术。该功能对已经量化的权重进行第二次量化，从而每参数节省 0.4 位。例如，使用嵌套量化，您可以在 16GB 的 NVIDIA T4 GPU 上微调 Llama-13b 模型，序列长度为 1024，批次大小为 1，并启用 4 步梯度累积：

```
from transformers import BitsAndBytesConfig

double_quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
)

model_double_quant = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-13b",
quantization_config=double_quant_config)
```

解量化 bitsandbytes 模型

量化后，您可以将模型解量化回原始精度，但这可能会导致模型质量的轻微损失。确保您的 GPU 内存足够大，以容纳解量化的模型。

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig, AutoTokenizer

model_id = "facebook/opt-125m"

model = AutoModelForCausalLM.from_pretrained(model_id, BitsAndBytesConfig(load_in_4bit=True))
tokenizer = AutoTokenizer.from_pretrained(model_id)

model.dequantize()

text = tokenizer("Hello my name is", return_tensors="pt").to(0)

out = model.generate(**text)
print(tokenizer.decode(out[0]))
```

GPTQ —— 广义剪枝训练量化

你可以在这个 笔记本 中尝试使用 PEFT 进行 GPTQ 量化，并在 这篇博客文章 中了解更多细节！

AutoGPTQ 库实现了 GPTQ 算法，这是一种后训练量化技术，其中权重矩阵的每一行独立量化，以找到一个能够最小化误差的权重版本。这些权重被量化为 int4，但在推理时会动态恢复为 fp16。这可以将内存使用量减少 4 倍，因为 int4 权重是在融合内核中而不是在 GPU 的全局内存中反量化，还可以期待推理速度的提升，因为使用较低的比特宽度通信时间更短。

在开始之前，请确保安装了以下库：

```
pip install auto-gptq
pip install --upgrade accelerate optimum transformers
```

要量化模型（目前仅支持文本模型），需要创建一个 GPTQConfig 类，并设置量化位数、用于校准权重的 数据集和分词器。

```
from transformers import AutoModelForCausalLM, AutoTokenizer, GPTQConfig

model_id = "facebook/opt-125m"
tokenizer = AutoTokenizer.from_pretrained(model_id)
gptq_config = GPTQConfig(bits=4, dataset="c4", tokenizer=tokenizer)
```

你也可以传递自己的数据集作为字符串列表，但强烈建议使用 GPTQ 论文中相同的数据集。

```
dataset = ["auto-gptq 是一个易于使用的模型量化库，具有用户友好的 API，基于 GPTQ 算法。"]
gptq_config = GPTQConfig(bits=4, dataset=dataset, tokenizer=tokenizer)
```

加载要量化的模型，并将 `gptq_config` 传递给 from_pretrained() 方法。设置 `device_map="auto"` 可以自动将模型卸载到 CPU 以帮助模型适应内存，并允许模型模块在 CPU 和 GPU 之间移动以进行量化。

```
quantized_model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto",
quantization_config=gptq_config)
```

如果你因为数据集太大而内存不足，磁盘卸载不被支持。如果出现这种情况，可以通过传递 `max_memory` 参数来分配设备（GPU 和 CPU）上的内存使用量。

```
quantized_model = AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    max_memory={0: "30GiB", 1: "46GiB", "cpu": "30GiB"},
    quantization_config=gptq_config
)
```

根据你的硬件，从头开始量化模型可能需要一些时间。在免费的 Google Colab GPU 上量化 facebook/opt-350m 模型大约需要 5 分钟，但在 NVIDIA A100 上量化一个 175B 参数的模型则需要约 4 小时。在量化模型之前，最好检查一下 Hugging Face Hub 是否已经存在 GPTQ 量化的版本。

一旦模型被量化，你可以将模型和分词器推送到 Hub 以便轻松共享和访问。使用 push_to_hub() 方法保存 GPTQConfig：

```
quantized_model.push_to_hub("opt-125m-gptq")
tokenizer.push_to_hub("opt-125m-gptq")
```

你也可以使用 save_pretrained() 方法将量化后的模型本地保存。如果模型是在设置了 `device_map` 参数的情况下量化的，在保存之前请确保将整个模型移动到 GPU 或 CPU。例如，要在 CPU 上保存模型：

```
quantized_model.save_pretrained("opt-125m-gptq")
tokenizer.save_pretrained("opt-125m-gptq")

# 如果量化时设置了 device_map
quantized_model.to("cpu")
quantized_model.save_pretrained("opt-125m-gptq")
```

使用 from_pretrained() 方法重新加载量化后的模型，并设置 `device_map="auto"` 以自动将模型分布在所有可用的 GPU 上，从而更快地加载模型且不占用过多内存。

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("{your_username}/opt-125m-gptq", device_map="auto")
```

ExLlama

ExLlama 是 Llama 模型的 Python/C++/CUDA 实现，旨在使用 4 位 GPTQ 权重进行更快的推理（参见这些 基准测试）。当创建 GPTQConfig 对象时，默认激活 ExLlama 内核。为了进一步提高推理速度，可以使用 ExLlamaV2 内核，通过配置 `exllama_config` 参数实现：

```
import torch
from transformers import AutoModelForCausalLM, GPTQConfig

gptq_config = GPTQConfig(bits=4, exllama_config={"version":2})
model = AutoModelForCausalLM.from_pretrained("{your_username}/opt-125m-gptq", device_map="auto",
quantization_config=gptq_config)
```

仅支持 4 位模型，如果你正在使用 PEFT 微调量化模型，建议禁用 ExLlama 内核。

只有当整个模型都在 GPU 上时，ExLlama 内核才受支持。如果你在 CPU 上使用 AutoGPTQ（版本 > 0.4.2）进行推理，则需要禁用 ExLlama 内核。这将覆盖 `config.json` 文件中的量化配置中与 ExLlama 内核相关的属性。

```
import torch
from transformers import AutoModelForCausalLM, GPTQConfig

gptq_config = GPTQConfig(bits=4, use_exllama=False)
```

```
model = AutoModelForCausalLM.from_pretrained("{your_username}/opt-125m-gptq", device_map="cpu",
quantization_config=gptq_config)
```

AWQ —— 激活感知权重量化

尝试使用这个 笔记本 来体验 AWQ 量化!

激活感知权重量化（AWQ） 不会量化模型中的所有权重，而是保留一小部分对大语言模型性能至关重要的权重。这显著减少了量化损失，使你可以在 4 位精度下运行模型而不影响性能。

有几个库支持使用 AWQ 算法量化模型，例如 llm-awq、autoawq 或 optimum-intel。Transformers 支持加载使用 llm-awq 和 autoawq 库量化的模型。本指南将向你展示如何加载使用 autoawq 量化的模型，但加载 llm-awq 量化的模型过程类似。

确保你已经安装了 autoawq：

```
pip install autoawq
```

可以通过检查模型的 config.json 文件中的 `quantization_config` 属性来识别 AWQ 量化的模型：

使用 `[from_pretrained()](/docs/transformers/v4.46.3/en/main_classes/model#transformers.PreTrainedModel.from_pretrained)` 方法加载量化的模型。如果你在 CPU 上加载了模型，请确保将其移动到 GPU 设备上。使用 `device_map` 参数指定模型放置的位置：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "TheBloke/zephyr-7B-alpha-AWQ"
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="cuda:0")
```

加载 AWQ 量化的模型时，默认会将其他权重设置为 fp16 以提高性能。如果你希望以不同的格式加载这些权重，请使用 `torch_dtype` 参数：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "TheBloke/zephyr-7B-alpha-AWQ"
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float32)
```

AWQ 量化还可以与 FlashAttention-2 结合使用，以进一步加速推理：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("TheBloke/zephyr-7B-alpha-AWQ",
attn_implementation="flash_attention_2", device_map="cuda:0")
```

融合模块

融合模块可以提高准确性和性能，并且在 Llama 和 Mistral 架构中默认支持 AWQ 模块，但你也可以为不支持的架构融合 AWQ 模块。

融合模块不能与其他优化技术（如 FlashAttention-2）结合使用。

支持的架构

不支持的架构

要为支持的架构启用融合模块，创建一个 AwqConfig，并设置参数 `fuse_max_seq_len` 和 `do_fuse=True`。`fuse_max_seq_len` 参数是总序列长度，应包括上下文长度和预期生成长度。为了安全起见，你可以将其设置为较大的值。

例如，要融合 TheBloke/Mistral-7B-OpenOrca-AWQ 模型的 AWQ 模块：

```
import torch
from transformers import AwqConfig, AutoModelForCausalLM

model_id = "TheBloke/Mistral-7B-OpenOrca-AWQ"

quantization_config = AwqConfig(
    bits=4,
    fuse_max_seq_len=512,
    do_fuse=True,
)

model = AutoModelForCausalLM.from_pretrained(model_id,
quantization_config=quantization_config).to(0)
```

TheBloke/Mistral-7B-OpenOrca-AWQ 模型使用 `batch_size=1` 进行了基准测试，对比了有无融合模块的情况。

未融合的模块

批量大小	预填充长度	解码长度	预填充 token/s	解码 token/s	内存 (VRAM)
1	32	32	60.0984	38.4537	4.50 GB (5.68%)
1	64	64	1333.67	31.6604	4.50 GB

					(5.68%)
1	128	128	2434.06	31.6272	4.50 GB (5.68%)
1	256	256	3072.26	38.1731	4.50 GB (5.68%)
1	512	512	3184.74	31.6819	4.59 GB (5.80%)
1	1024	1024	3148.18	36.8031	4.81 GB (6.07%)
1	2048	2048	2927.33	35.2676	5.73 GB (7.23%)

融合的模块

批量大小	预填充长度	解码长度	预填充 token/s	解码 token/s	内存 (VRAM)
1	32	32	81.4899	80.2569	4.00 GB (5.05%)
1	64	64	1756.1	106.26	4.00 GB (5.05%)
1	128	128	2479.32	105.631	4.00 GB (5.06%)
1	256	256	1813.6	85.7485	4.01 GB (5.06%)
1	512	512	2848.9	97.701	4.11 GB (5.19%)
1	1024	1024	3044.35	87.7323	4.41 GB (5.57%)
1	2048	2048	2715.11	89.4709	5.57 GB (7.04%)

融合和未融合模块的速度和吞吐量也通过 optimum-benchmark 库进行了测试。

〔图：每批生成吞吐量〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/quantization/fused_forward_memory_plot.png，离线电子书未嵌入，详见原仓库）

前向峰值内存/批量大小

〔图：每批前向延迟〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/quantization/fused_generate_throughput_plot.png，离线电子书未嵌入，详见原仓库）

生成吞吐量/批量大小

ExLlama-v2 支持

最近版本的 `autoawq` 支持 ExLlama-v2 内核，以加快预填充和解码速度。要开始使用，请首先安装最新版本的 `autoawq`：

```
pip install git+https://github.com/casper-hansen/AutoAWQ.git
```

通过传递带有 `version="exllama"` 的 `AwqConfig()` 开始使用：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, AwqConfig

quantization_config = AwqConfig(version="exllama")

model = AutoModelForCausalLM.from_pretrained(
    "TheBloke/Mistral-7B-Instruct-v0.1-AWQ",
    quantization_config=quantization_config,
    device_map="auto",
)

input_ids = torch.randint(0, 100, (1, 128), dtype=torch.long, device="cuda")
output = model(input_ids)
print(output.logits)

tokenizer = AutoTokenizer.from_pretrained("TheBloke/Mistral-7B-Instruct-v0.1-AWQ")
input_ids = tokenizer.encode("How to make a cake", return_tensors="pt").to(model.device)
output = model.generate(input_ids, do_sample=True, max_length=50, pad_token_id=50256)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

请注意，此功能支持 AMD GPU。

CPU 支持

最近版本的 `autoawq` 支持使用 ipex 操作优化的 CPU。要开始使用，请首先安装最新版本的 `autoawq`：

```
pip install intel-extension-for-pytorch
pip install git+https://github.com/casper-hansen/AutoAWQ.git
```

通过传递带有 `version="ipex"` 的 `AwqConfig()` 开始使用：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, AwqConfig

quantization_config = AwqConfig(version="ipex")

model = AutoModelForCausalLM.from_pretrained(
    "TheBloke/TinyLlama-1.1B-Chat-v0.3-AWQ",
    quantization_config=quantization_config,
    device_map="cpu",
)

input_ids = torch.randint(0, 100, (1, 128), dtype=torch.long, device="cpu")
output = model(input_ids)
print(output.logits)

tokenizer = AutoTokenizer.from_pretrained("TheBloke/TinyLlama-1.1B-Chat-v0.3-AWQ")
input_ids = tokenizer.encode("How to make a cake", return_tensors="pt")
pad_token_id = tokenizer.eos_token_id
output = model.generate(input_ids, do_sample=True, max_length=50, pad_token_id=pad_token_id)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

请注意，此功能支持 Intel CPU。

AQLM —— 加性量化语言模型

尝试在 Google Colab 上使用 AQLM！

加性量化语言模型（AQLM）是一种大型语言模型压缩方法。它通过量化多个权重并利用它们之间的相互依赖关系来实现压缩。AQLM 将 8-16 个权重表示为多个向量代码的总和。

AQLM 的推理支持由 `aqlm` 库实现。确保安装该库以运行模型（注意：aqlm 只支持 python>=3.10）：

```
pip install aqlm[gpu,cpu]
```

该库提供了高效的 GPU 和 CPU 推理和训练内核。

如何自己量化模型以及所有相关代码的说明可以在相应的 GitHub 仓库 中找到。要运行 AQLM 模型，只需加载已使用 AQLM 量化的模型：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

# 加载量化的模型
quantized_model = AutoModelForCausalLM.from_pretrained(
    "ISTA-DASLab/Mixtral-8x7b-AQLM-2Bit-1x16-hf",
    torch_dtype="auto",
    device_map="auto"
)

# 加载分词器
tokenizer = AutoTokenizer.from_pretrained("ISTA-DASLab/Mixtral-8x7b-AQLM-2Bit-1x16-hf")
```

PEFT

从版本 `aqlm 1.0.2` 开始，AQLM 支持参数高效的微调（PEFT），具体形式为集成到 PEFT 库中的 LoRA。

AQLM 配置

AQLM 量化设置主要取决于使用的码本数量以及码本大小（以位为单位）。最常见的设置及其支持的推理内核如下表所示：

内核	码本数量	码本大小，位	表示法	准确度	加速比	快速 GPU 推理	快速 CPU 推理
Triton	K	N	KxN	-	最高约 0.7 倍	☑	✗
CUDA	1	16	1x16	最佳	最高约 1.3 倍	☑	✗
CUDA	2	8	2x8	较好	最高约 3.0 倍	☑	✗
Numba	K	8	Kx8	良好	最高约 4.0 倍	✗	☑

Optimum-quanto —— PyTorch 多功能量化工具包

你可以通过这个 笔记本 来尝试使用 optimum-quanto + transformers!

🤗 optimum-quanto 是一个多功能的 PyTorch 量化工具包。它使用的量化方法是线性量化。

Optimum-quanto 提供了以下几个独特功能:

- 权重量化（`float8`, `int8`, `int4`, `int2`）
- 激活量化（`float8`, `int8`）
- 模态无关（例如 CV, LLM）
- 设备无关（例如 CUDA, XPU, MPS, CPU）
- 与 `torch.compile` 兼容
- 容易添加特定设备的自定义内核
- 支持量化感知训练

在开始之前，请确保安装了以下库:

```
pip install optimum-quanto accelerate transformers
```

现在，你可以通过在 from_pretrained() 方法中传递 QuantoConfig 对象来量化模型。只要模型包含 `torch.nn.Linear` 层，这适用于任何模态的任何模型。

Transformers 集成仅支持权重量化。对于更复杂的情况，如激活量化、校准和量化感知训练，你应该使用 optimum-quanto 库。

默认情况下，无论权重实际存储的数据类型（如 `torch.float16`），权重都会以全精度（`torch.float32`）加载。设置 `torch_dtype="auto"` 可以根据模型的 `config.json` 文件中定义的数据类型自动加载最节省内存的数据类型。

```
from transformers import AutoModelForCausalLM, AutoTokenizer, QuantoConfig

model_id = "facebook/opt-125m"
tokenizer = AutoTokenizer.from_pretrained(model_id)
quantization_config = QuantoConfig(weights="int8")
quantized_model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype="auto",
device_map="cuda:0", quantization_config=quantization_config)
```

请注意，目前 Transformers 尚不支持序列化，但这即将实现！如果你想要保存模型，可以使用 quanto 库。

Optimum-quanto 库使用线性量化算法进行量化。尽管这是一种基本的量化技术，但我们仍然获得了非常好的结果！请查看以下基准测试（llama-2-7b 在困惑度指标上的表现）。你可以在这里找到更多基准测试：这里

〔图：llama-2-7b-quanto-perplexity〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/quantization/NousResearch-Llama-2-7b-hf_Perplexity.png，离线电子书未嵌入，详见原仓库）

该库足够灵活，可以与大多数 PTQ 优化算法兼容。未来计划以最无缝的方式集成最受欢迎的算法（AWQ, Smoothquant）。

EETQ —— 高效的 Tensor 量化方法

EETQ 是一个支持在 NVIDIA GPU 上进行 int8 每通道权重量化（weight-only quantization）的库。其高性能的 GEMM 和 GEMV 内核来自 FasterTransformer 和 TensorRT-LLM。使用该库时，无需校准数据集，也不需要预先量化模型。此外，由于使用了每通道量化，模型的精度下降几乎可以忽略不计。

确保你已经从 发布页面 安装了 EETQ。

```
pip install --no-cache-dir https://github.com/NetEase-FuXi/EETQ/releases/download/v1.0.0/EETQ-1.0.0+cu121+torch2.1.2-cp310-cp310-linux_x86_64.whl
```

或者通过源代码安装：https://github.com/NetEase-FuXi/EETQ。EETQ 需要 CUDA 能力 <= 8.9 且 >= 7.0。

```
git clone https://github.com/NetEase-FuXi/EETQ.git
cd EETQ/
git submodule update --init --recursive
pip install .
```

未量化的模型可以通过 `from_pretrained` 方法进行量化。

```
from transformers import AutoModelForCausalLM, EetqConfig
path = "/path/to/model"  # 模型路径
quantization_config = EetqConfig("int8")  # 量化配置
model = AutoModelForCausalLM.from_pretrained(path, device_map="auto", quantization_config=quantization_config)
```

量化的模型可以通过 `save_pretrained` 方法保存，并通过 `from_pretrained` 方法再次加载使用。

```
quant_path = "/path/to/save/quantized/model"  # 保存的路径
model.save_pretrained(quant_path)
model = AutoModelForCausalLM.from_pretrained(quant_path, device_map="auto")
```

HQQ —— 半二次量化

半二次量化（Half-Quadratic Quantization, HQQ）通过快速稳健的优化实现了即时量化。它不需要校准数据，并且可以用于量化任何模型。更多详细信息请参阅 官方包。

安装

我们建议您使用以下方法安装最新版本并构建相应的 CUDA 内核：

```
pip install hqq
```

量化模型

要量化一个模型，您需要创建一个 HqqConfig。有以下两种方法：

方法 1：所有线性层使用相同的量化配置

```
from transformers import AutoModelForCausalLM, AutoTokenizer, HqqConfig

# 所有线性层使用相同的量化配置
quant_config = HqqConfig(nbits=8, group_size=64)
```

方法 2：具有相同标签的每个线性层使用专门的量化配置

```
# 不同线性层使用不同的量化配置
q4_config = {'nbits': 4, 'group_size': 64}
q3_config = {'nbits': 3, 'group_size': 32}
quant_config = HqqConfig(dynamic_config={
  'self_attn.q_proj': q4_config,
  'self_attn.k_proj': q4_config,
  'self_attn.v_proj': q4_config,
  'self_attn.o_proj': q4_config,

  'mlp.gate_proj': q3_config,
  'mlp.up_proj': q3_config,
  'mlp.down_proj': q3_config,
})
```

第二种方法特别适用于量化混合专家模型（Mixture-of-Experts, MoEs），因为专家在较低的量化设置下受影响较小。

然后，您可以按以下方式量化模型：

```
model = transformers.AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    device_map="cuda",
    quantization_config=quant_config
)
```

优化运行时

HQQ 支持多种后端，包括纯 PyTorch 和自定义反量化 CUDA 内核。这些后端适合旧 GPU 和 peft/QLoRA 训练。为了更快的推理，HQQ 支持 4 位融合内核（TorchAO 和 Marlin），在单个 4090 GPU 上可以达到每秒 200 个 token 的速度。有关如何使用这些后端的更多详细信息，请参阅 HQQ 文档。

FBGEMM FP8（权重 8 位、激活 8 位的模型量化）

通过使用 FBGEMM FP8 量化方法，你可以将模型量化为 FP8（权重为 8 位，激活为 8 位）：

- 权重将按通道量化为 8 位（FP8）
- 激活将按 token 量化为 8 位（FP8）

该方法依赖于 FBGEMM 库，该库提供了高效的小批量低精度通用矩阵乘法，并支持行级量化和异常值感知量化等减少精度损失的技术。

你需要一个计算能力 >= 9 的 GPU（例如 H100）。

在开始之前，请确保安装了以下库的最新版本：

```
pip install --upgrade accelerate fbgemm-gpu torch
```

如果安装 `fbgemm-gpu` 和 `torch` 库时遇到问题，可能需要安装夜间版。你可以按照 这里的说明 进行操作。

默认情况下，无论权重实际存储的数据类型（如 `torch.float16`），权重都会以全精度（`torch.float32`）加载。设置 `torch_dtype="auto"` 可以根据模型的 `config.json` 文件中定义的数据类型自动加载最节省内存的数据类型。

```
from transformers import FbgemmFp8Config, AutoModelForCausalLM, AutoTokenizer

model_name = "meta-llama/Meta-Llama-3-8B"
quantization_config = FbgemmFp8Config()
quantized_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto",
    quantization_config=quantization_config
)

tokenizer = AutoTokenizer.from_pretrained(model_name)
input_text = "我们今晚吃什么？"
input_ids = tokenizer(input_text, return_tensors="pt").to("cuda")

output = quantized_model.generate(**input_ids, max_new_tokens=10)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

量化后的模型可以通过 `save_pretrained` 保存，并通过 `from_pretrained` 重新加载使用。

```
quant_path = "/path/to/save/quantized/model"
model.save_pretrained(quant_path)
model = AutoModelForCausalLM.from_pretrained(quant_path, device_map="auto")
```

Optimum

Optimum 库支持针对 Intel、Furiosa、ONNX Runtime、GPTQ 以及低级别的 PyTorch 量化函数的量化。如果你使用的是特定优化的硬件，如 Intel CPU、Furiosa NPU 或模型加速器如 ONNX Runtime，建议使用 Optimum 来进行量化。

TorchAO —— 用于 PyTorch 的架构优化库

TorchAO 是一个用于 PyTorch 的架构优化库，它提供了高性能的数据类型、优化技术和内核，适用于推理和训练，并且与原生的 PyTorch 特性（如 `torch.compile`、FSDP 等）无缝集成。一些基准测试数据可以在 这里 查看。

在开始之前，请确保安装了以下库的最新版本：

```
pip install --upgrade torch torchao
```

默认情况下，权重会以全精度（torch.float32）加载，无论实际存储的权重数据类型是什么（例如torch.float16）。设置 `torch_dtype="auto"` 可以根据模型的 `config.json` 文件中定义的数据类型自动加载最节省内存的数据类型。

```
import torch
from transformers import TorchAoConfig, AutoModelForCausalLM, AutoTokenizer

model_name = "meta-llama/Meta-Llama-3-8B"

# 支持的量化配置有 int4_weight_only、int8_weight_only 和 int8_dynamic_activation_int8_weight
# 更多参数示例和文档可以在这里找到：
https://github.com/pytorch/ao/tree/main/torchao/quantization#other-available-quantization-techniques
quantization_config = TorchAoConfig("int4_weight_only", group_size=128)

# 加载量化模型
quantized_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto",
    quantization_config=quantization_config
)

# 加载分词器
tokenizer = AutoTokenizer.from_pretrained(model_name)
input_text = "我们晚餐吃什么？"
input_ids = tokenizer(input_text, return_tensors="pt").to("cuda")

# 编译量化模型以加速
import torchao
torchao.quantization.utils.recommended_inductor_config_setter()
quantized_model = torch.compile(quantized_model, mode="max-autotune")

# 生成输出
output = quantized_model.generate(**input_ids, max_new_tokens=10)
print(tokenizer.decode(output[0], skip_special_tokens=True))

# 测试性能
import torch.utils.benchmark as benchmark

def benchmark_fn(f, *args, **kwargs):
    # 预热
    for _ in range(5):
        f(*args, **kwargs)
```

```
    t0 = benchmark.Timer(
        stmt="f(*args, **kwargs)",
        globals={"args": args, "kwargs": kwargs, "f": f},
        num_threads=torch.get_num_threads(),
    )
    return f"{(t0.blocked_autorange().mean):.3f}"

MAX_NEW_TOKENS = 1000
print("int4wo-128 模型:", benchmark_fn(quantized_model.generate, **input_ids,
max_new_tokens=MAX_NEW_TOKENS))

# 使用 bfloat16 模型进行对比
bf16_model = AutoModelForCausalLM.from_pretrained(model_name, device_map="cuda",
torch_dtype=torch.bfloat16)
bf16_model = torch.compile(bf16_model, mode="max-autotune")
print("bf16 模型:", benchmark_fn(bf16_model.generate, **input_ids,
max_new_tokens=MAX_NEW_TOKENS))
```

序列化和反序列化

TorchAO 的量化是通过 张量子类 实现的，因此仅支持 Hugging Face 的非安全张量序列化和反序列化。它依赖于 `torch.load(..., weights_only=True)` 来避免加载时执行任意用户代码，并使用 add_safe_globals 来白名单一些已知的用户函数。

不支持安全张量序列化的原因是，张量子类允许最大的灵活性，因此我们希望支持新的量化张量格式的工作量较低。而安全张量则优化了最大安全性（不执行用户代码），这意味着我们必须手动支持新的量化格式。

```
# 将量化模型保存到本地
output_dir = "llama3-8b-int4wo-128"
quantized_model.save_pretrained(output_dir, safe_serialization=False)

# 推送到 Hugging Face Hub
# save_to = "{user_id}/llama3-8b-int4wo-128"
# quantized_model.push_to_hub(save_to, safe_serialization=False)

# 加载量化模型
ckpt_id = "llama3-8b-int4wo-128"  # 或者 Hugging Face Hub 模型 ID
loaded_quantized_model = AutoModelForCausalLM.from_pretrained(ckpt_id, device_map="cuda")

# 确认加速效果
loaded_quantized_model = torch.compile(loaded_quantized_model, mode="max-autotune")
```

```
print("加载的 int4wo-128 模型:", benchmark_fn(loaded_quantized_model.generate, **input_ids, max_new_tokens=MAX_NEW_TOKENS))
```

BitNet —— 新型的神经网络架构

BitNet 是一种新型的神经网络架构，它用特殊的 BitLinear 层替换了传统多头注意力机制和前馈网络中的线性层。BitLinear 层使用三值（或二值在旧版本中）精度量化权重，并将激活值量化为 8 位精度。

〔图：BitLinear 层的 BitNet 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/blog/1.58llm_extreme_quantization/bitlinear.png，离线电子书未嵌入，详见原仓库）

训练过程

在训练过程中，我们首先使用对称张量量化方法将权重量化为三值（-1, 0, 1）。具体步骤如下：

258. **计算权重的平均绝对值**：

$$
\text{scale}_w = \frac{1}{\frac{1}{nm} \sum_{ij} |W_{ij}|}
$$

其中 W 是权重矩阵， n 和 m 分别是矩阵的行数和列数。

259. **量化权重**：

$$
W_q = \text{clamp}_{[-1,1]}(\text{round}(W \times \text{scale}))
$$

这里，我们先将权重乘以缩放因子，然后四舍五入并限制在 -1 到 1 之间。

260. **反量化权重**：

$$
W_{\text{dequantized}} = W_q \times \text{scale}_w
$$

接下来，我们将激活值量化为指定的位宽（例如，8 位），使用 absmax 量化方法（对称通道量化）。具体步骤如下：

261. **计算激活值的最大绝对值**：

$$
\text{scale}_x = \frac{127}{|X|_{\text{max}, \, \text{dim}=-1}}
$$

262. **量化激活值**：

$$
X_q = \text{clamp}_{[-128,127]}(\text{round}(X \times \text{scale}))
$$

263. **反量化激活值**：

$$
X_{\text{dequantized}} = X_q \times \text{scale}_x
$$

更多关于如何训练和微调 BitNet 模型的信息，请查看这篇 博客文章。

从 Hub 加载 BitNet 模型

BitNet 模型不能动态量化，它们需要预先训练或微调，应用量化技术（即量化感知训练）。训练完成后，这些模型已经量化完成，并以打包的形式在 Hub 上提供。

加载量化模型的方法如下：

```
from transformers import AutoModelForCausalLM
path = "/path/to/model"
model = AutoModelForCausalLM.from_pretrained(path, device_map="auto")
```

预训练和微调 BitNet 模型

如果你希望使用 Nanotron 预训练或微调自己的 1.58 位模型，可以参考这个 PR，所有需要的信息都在那里！

对于微调，你需要将模型从 Hugging Face 格式转换为 Nanotron 格式（两者有一些差异）。转换步骤可以在这个 PR 中找到。

内核

在我们的初始版本中，我们选择使用 `@torch.compile` 来解包权重并执行前向传播。这种方法实现非常简单，并且带来了显著的速度提升。我们计划在未来的版本中集成更多的优化内核。

压缩张量

`compressed-tensors` 库提供了一种灵活且高效的方法来存储和管理压缩的模型检查点。该库支持多种量化和稀疏性方案，使其成为处理不同模型优化（如 GPTQ、AWQ、SmoothQuant、INT8、FP8、SparseGPT 等）的统一格式。

支持的一些格式包括：

264. `dense`
265. `int-quantized` (示例)：INT8 量化模型
266. `float-quantized` (示例)：FP8 量化模型；目前支持 E4M3
267. `pack-quantized` (示例)：INT4 或 INT8 权重量化模型，打包成 INT32。对于 INT4，权重有 INT4 的范围，但以 INT8 存储并打包成 INT32。

压缩模型可以使用 llm-compressor 轻松创建。或者，也可以独立创建模型，并使用压缩张量配置进行序列化。

要在 Hugging Face Model Hub 上查找现有模型，请搜索带有 `compressed-tensors` 标签的模型。

特性：

- 权重和激活精度：FP8、INT4、INT8（对于 Q/DQ，INT 允许任意精度）
- 量化尺度和零点策略：张量、通道、组、块、令牌
- 动态按令牌激活量化（或任何静态策略）
- 权重中的稀疏性（无结构或半结构化如 2:4）可以与量化组合以实现极端压缩

- 支持任意模块的量化，而不仅仅是线性模块
- 按名称或类目标定位或忽略模块

安装

建议从 PyPI 安装 `compressed-tensors` 的稳定版本：

```
pip install compressed-tensors
```

希望尝试最新功能的开发者可以从源代码安装该包：

```
git clone https://github.com/neuralmagic/compressed-tensors
cd compressed-tensors
pip install -e .
```

快速开始：加载模型

以下示例展示了如何轻松加载量化模型以进行推理。目前只能加载已量化的模型。要将模型量化解压成 `compressed-tensors` 格式，请参阅 llm-compressor。

```
from transformers import AutoModelForCausalLM

# 以压缩张量格式加载模型
ct_model = AutoModelForCausalLM.from_pretrained("nm-testing/Meta-Llama-3.1-8B-Instruct-FP8-hf")

# 测量内存使用情况
mem_params = sum([param.nelement() * param.element_size() for param in ct_model.parameters()])
print(f"{mem_params / 2**30:.4f} GB")
# 8.4575 GB
```

从上面可以看出，压缩张量 FP8 检查点的 Llama 3.1 8B 可以在推理时使用未量化参考检查点一半的内存。

示例用例：加载和运行 FP8 模型

```
from transformers import AutoModelForCausalLM, AutoTokenizer

prompt = [
    "Hello, my name is",
    "The capital of France is",
    "The future of AI is"
]
```

```
model_name = "nm-testing/Meta-Llama-3-8B-Instruct-fp8-hf_compat"

quantized_model = AutoModelForCausalLM.from_pretrained(model_name, device_map="auto")
tokenizer = AutoTokenizer.from_pretrained(model_name)

inputs = tokenizer(prompt, return_tensors="pt")
generated_ids = quantized_model.generate(**inputs, max_length=50, do_sample=False)
outputs = tokenizer.batch_decode(generated_ids)

print(outputs)

"""
['<|begin_of_text|>Hello, my name is [Name]. I am a [Your Profession/Student] and I am here to
learn about the [Course/Program] at [University/Institution]. I am excited to be here and I am
looking forward to', '<|begin_of_text|>The capital of France is Paris, which is located in the
north-central part of the country. Paris is the most populous city in France and is known for
its stunning architecture, art museums, fashion, and romantic atmosphere. The city is home to',
"<|begin_of_text|>The future of AI is here, and it's already changing the way we live and work.
From virtual assistants to self-driving cars, AI is transforming industries and revolutionizing
the way we interact with technology. But what does the future of AI hold"]
"""
```

上述示例展示了如何使用 `compressed-tensors` 模型进行生成。目前，一旦加载模型后，无法保存。

深入了解压缩张量模型检查点

在这个示例中，我们将通过模型配置条目来查看 `compressed-tensors` 模型 `nm-testing/Meta-Llama-3.1-8B-Instruct-FP8-hf` 如何定义，并了解这如何转换为加载后的模型表示。

首先，让我们来看看模型的 `quantization_config` 配置。虽然看起来条目很多，但这是因为 `compressed-tensors` 是一种允许在模型压缩期间和之后灵活表达的格式。

为了简化配置以便更专注于实际的压缩表示，我们可以删除所有默认或空的条目。

从上述配置中可以看出，它指定了一个配置组，其中包括权重和激活的 FP8 量化，采用静态每张量策略。值得注意的是，在 `ignore` 列表中有一个条目用于跳过 `lm_head` 模块的量化，因此该模块在检查点中应保持不变。

为了在实践中看到配置的结果，我们可以使用 safetensors 查看器 查看模型卡上的量化权重、输入比例和权重比例，例如第一个模型层中的所有 Linear 模块（其余层也是如此）。

张量	形状	精度
model.layers.0.input_layernorm.weight	[4096]	BF16

model.layers.0.mlp.down_proj.input_scale	[1]	BF16
model.layers.0.mlp.down_proj.weight	[4096, 14336]	F8_E4M3
model.layers.0.mlp.down_proj.weight_scale	[1]	BF16
model.layers.0.mlp.gate_proj.input_scale	[1]	BF16
model.layers.0.mlp.gate_proj.weight	[14336, 4096]	F8_E4M3
model.layers.0.mlp.gate_proj.weight_scale	[1]	BF16
model.layers.0.mlp.up_proj.input_scale	[1]	BF16
model.layers.0.mlp.up_proj.weight	[14336, 4096]	F8_E4M3
model.layers.0.mlp.up_proj.weight_scale	[1]	BF16
model.layers.0.post_attention_layernorm.weight	[4096]	BF16
model.layers.0.self_attn.k_proj.input_scale	[1]	BF16
model.layers.0.self_attn.k_proj.weight	[1024, 4096]	F8_E4M3
model.layers.0.self_attn.k_proj.weight_scale	[1]	BF16
model.layers.0.self_attn.o_proj.input_scale	[1]	BF16
model.layers.0.self_attn.o_proj.weight	[4096, 4096]	F8_E4M3
model.layers.0.self_attn.o_proj.weight_scale	[1]	BF16
model.layers.0.self_attn.q_proj.input_scale	[1]	BF16
model.layers.0.self_attn.q_proj.weight	[4096, 4096]	F8_E4M3
model.layers.0.self_attn.q_proj.weight_scale	[1]	BF16
model.layers.0.self_attn.v_proj.input_scale	[1]	BF16
model.layers.0.self_attn.v_proj.weight	[1024, 4096]	F8_E4M3
model.layers.0.self_attn.v_proj.weight_scale	[1]	BF16

当我们使用压缩张量 HFQuantizer 集成加载模型时，可以看到 `quantization_config` 中指定的所有 Linear 模块都被替换为 `CompressedLinear` 模块，这些模块管理压缩权重和前向传递以进行推理。注意之前在 `ignore` 列表中提到的 `lm_head` 仍然保留为未量化的 Linear 模块。

```
from transformers import AutoModelForCausalLM
```

```
ct_model = AutoModelForCausalLM.from_pretrained("nm-testing/Meta-Llama-3.1-8B-Instruct-FP8-hf")
print(ct_model)
"""
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(128256, 4096)
    (layers): ModuleList(
      (0-31): 32 x LlamaDecoderLayer(
        (self_attn): LlamaSdpaAttention(
          (q_proj): CompressedLinear(
            in_features=4096, out_features=4096, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (k_proj): CompressedLinear(
            in_features=4096, out_features=1024, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (v_proj): CompressedLinear(
            in_features=4096, out_features=1024, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (o_proj): CompressedLinear(
            in_features=4096, out_features=4096, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (rotary_emb): LlamaRotaryEmbedding()
        )
        (mlp): LlamaMLP(
          (gate_proj): CompressedLinear(
            in_features=4096, out_features=14336, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (up_proj): CompressedLinear(
            in_features=4096, out_features=14336, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (down_proj): CompressedLinear(
            in_features=14336, out_features=4096, bias=False
            (input_observer): MovingAverageMinMaxObserver()
            (weight_observer): MovingAverageMinMaxObserver()
          )
          (act_fn): SiLU()
        )
        (input_layernorm): LlamaRMSNorm((4096,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((4096,), eps=1e-05)
```

```
      )
    )
    (norm): LlamaRMSNorm((4096,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=4096, out_features=128256, bias=False)
)
"""
```

添加新的量化方法

Transformers 支持并集成了许多量化方法，如 QLoRA、GPTQ、LLM.int8 和 AWQ。然而，还有一些其他量化方法尚未集成。为了更轻松地添加和使用这些量化方法，你应该使用 `HfQuantizer` 类。`HfQuantizer` 是一个内部辅助类，用于添加量化方法，而不是应用于每个 PyTorch 模块。

本指南将向你展示如何使用 `HfQuantizer` 类集成新的量化方法。

前提条件

在将新的量化方法集成到 Transformers 中之前，请确保你要添加的方法满足以下前提条件。目前，仅支持可以在 PyTorch 模块上运行的量化方法。

- 量化方法可以通过一个任何人都可以 pip 安装的 Python 包获得（如果你只能从源代码安装该包也是可以的）。理想情况下，pip 包中包含预编译的内核。
- 该方法可以在常用硬件（CPU、GPU 等）上运行。
- 该方法封装在一个 `nn.Module` 中（例如 `Linear8bitLt`、`Linear4bit`），并且量化线性层应具有以下定义：

这样，Transformers 模型可以通过将某些 `nn.Linear` 实例替换为目标类来轻松量化。

- 量化方法应该是可序列化的。你可以将量化后的权重保存到本地或推送到 Hub。
- 确保包含量化内核/原语的包是稳定的（没有频繁的破坏性更改）。

对于一些量化方法，它们可能需要通过数据校准（例如 AWQ）预先量化模型。在这种情况下，我们更倾向于仅在 Transformers 中支持推理，并让第三方库处理模型的量化过程。

构建新的 HfQuantizer 类

268. 在 src/transformers/utils/quantization_config.py 中创建一个新的量化配置类，并确保在 Transformers 主 `init` 文件中公开新的量化配置，即将其添加到 src/transformers/**init**.py 的 `[_import_structure]` 对象中。

269. 在 src/transformers/quantizers/ 目录下创建一个名为 `quantizer_your_method.py` 的新文件，并使其继承自 src/transformers/quantizers/base.py::HfQuantizer。确保在 src/transformers/quantizers/auto.py 中添加新的量化器和量化配置。

270. 为你的量化方法定义以下类属性/属性方法：

    - `requires_calibration`: 量化方法是否需要数据校准过程。如果设置为 `True`，则只能支持推理（带有量化权重）而不能支持推理和量化。
    - `required_packages`: 一个字符串列表，包含使用量化权重所需的包。你可能需要定义一些新的实用方法，例如 transformers/src/utils/import_utils.py 中的 `is_auto_awq_available`。
    - `requires_parameters_quantization`: 仅在量化方法需要对底层 `nn.Parameter` 对象特别关注时才需要。例如，bitsandbytes 使用 `Params4bit` 和 `Int8Param`，这在量化模型时需要特别关注。大多数最近的量化方法将 int2/int4 权重打包到 `torch.uint8` 权重中，因此这个标志通常不需要（默认设置为 `False`）。
    - `is_serializable`: 一个属性方法，用于确定方法是否可序列化。
    - `is_trainable`: 一个属性方法，用于确定你是否可以在量化方法的基础上微调模型（无论是否使用 PEFT 方法）。

271. 编写 `validate_environment` 和 `update_torch_dtype` 方法。这些方法在创建量化模型之前被调用，以确保用户使用的配置正确。你可以参考其他量化器的实现方式。

272. 编写 `_process_model_before_weight_loading` 方法。在 Transformers 中，量化模型首先在 `"meta"` 设备上初始化，然后加载权重。这意味着 `_process_model_before_weight_loading` 方法负责操作模型骨架，以将某些模块（例如 `nn.Linear`）替换为目标模块（量化模块）。你可以在 transformers/src/integrations/ 目录下创建一个新文件，并在该目录的 `__init__.py` 文件中公开相关方法。最好的起点是查看其他量化方法，如 quantizer_awq.py。

273. 编写 `_process_model_after_weight_loading` 方法。此方法允许在加载权重后操作模型以实现额外功能。

274. 文档化一切！确保你的量化方法有文档记录，方法是在 `docs/source/en/quantization` 下添加一个新文件，并在 `docs/source/en/quantization/overview.md` 表格中添加一行。

275. 添加测试！首先在 `docker/transformers-quantization-latest-gpu` 中的夜间构建 Dockerfile 中添加包，然后在 `tests/quantization/xxx` 中添加一个新的测试文件。你可以参考其他量化方法的实现方式。

第六部分 性能优化与可扩展性

性能优化概述

训练大型变压器模型并将其部署到生产环境中面临着各种挑战。在训练过程中，模型可能需要比可用的更多的 GPU 内存，或者表现出训练速度缓慢的问题。在部署阶段，模型可能难以处理生产环境中所需的吞吐量。

本文档旨在帮助您克服这些挑战，并找到适用于您场景的最佳设置。指南分为训练和推理两部分，因为每部分都伴随着不同的挑战和解决方案。每个部分中，您会找到针对不同硬件配置的单独指南，例如单个 GPU 与多个 GPU 的训练或 CPU 与 GPU 的推理。

请将本文档作为您的起点，进一步导航至符合您情况的方法。

训练

高效地训练大型变压器模型通常需要加速器，如 GPU 或 TPU。最常见的场景是您有一个单个 GPU。您可以应用的方法来提高单个 GPU 上的训练效率，也可以扩展到其他设置，如多 GPU。然而，也有一些特定于多 GPU 或 CPU 训练的技术。我们在单独的部分中进行了介绍。

- 在单个 GPU 上高效训练的方法和工具：从这里开始，了解常见的方法，这些方法可以帮助优化 GPU 内存利用率、加快训练速度，或两者兼顾。

- 多 GPU 训练部分：探索这一部分，了解适用于多 GPU 设置的进一步优化方法，如数据并行、张量并行和管道并行。
- CPU 训练部分：了解 CPU 上的混合精度训练。
- 在多个 CPU 上高效训练：了解分布式 CPU 训练。
- 使用 TensorFlow 在 TPU 上训练：如果您是 TPU 新手，请参考这一部分，了解有关在 TPU 上训练和使用 XLA 的意见介绍。
- 用于训练的自定义硬件：在构建自己的深度学习设备时，查找技巧和窍门。
- 使用 Trainer API 进行超参数搜索

推理

在生产环境中使用大型模型进行高效推理可能与训练它们一样具有挑战性。在以下部分中，我们将介绍如何在 CPU 和单个/多个 GPU 设置上运行推理。

- 在单个 CPU 上进行推理
- 在单个 GPU 上进行推理
- 多 GPU 推理
- TensorFlow 模型的 XLA 集成

训练与推理

在这里，您将找到适用于训练模型或运行推理的技术、技巧和窍门。

- 实例化大型模型
- 解决性能问题

LLM 推理优化

大型语言模型（LLMs）通过生成具有高度理解和流畅性的文本，将聊天和代码补全等文本生成应用提升到了新的水平。然而，使 LLMs 如此强大的一个因素——它们的体积——也为推理带来了挑战。

基本的推理速度较慢，因为每次生成下一个标记时都需要反复调用 LLM。随着生成的进行，输入序列会逐渐变长，这使得 LLM 的处理时间越来越长。此外，LLMs 拥有数十亿个参数，这些参数在内存中存储和处理非常困难。

本指南将展示如何使用 Transformers 中的优化技术来加速 LLM 推理。

Hugging Face 还提供了 文本生成推理 (TGI)，这是一个专门用于部署和提供高度优化的 LLM 推理的库。它包括一些 Transformers 不包含的面向部署的优化功能，例如连续批处理以提高吞吐量和多 GPU 推理的张量并行性。

静态键值缓存和 torch.compile

在解码过程中，LLM 会计算每个输入标记的键值对（kv 值）。由于它是自回归的，每次生成的输出都会成为新的输入，因此会重复计算相同的 kv 值。这效率不高，因为它每次都在重新计算相同的 kv 值。

为了优化这一点，可以使用 kv 缓存来存储过去的键和值，而不是每次都重新计算。然而，由于 kv 缓存在每次生成步骤中会增长，并且是动态的，这阻碍了你利用 `torch.compile`，这是一个强大的优化工具，可以将 PyTorch 代码融合成快速且优化的内核。我们有一整篇关于 kv 缓存的指南 在这里。

静态 kv 缓存解决了这个问题，通过预先分配最大值来预分配 kv 缓存的大小，允许你将其与 `torch.compile` 结合使用，从而实现最高 4 倍的速度提升。具体速度提升取决于模型大小（较大的模型速度提升较小）和硬件。

目前，只有 Llama 和少数其他模型支持静态 kv 缓存和 `torch.compile`。请查看 这个 issues 获取实时的模型兼容列表。

根据任务的复杂性，静态 kv 缓存有三种使用方式：

276. 基本使用：只需在 `generation_config` 中设置一个标志（推荐）；
277. 高级使用：处理一个多轮生成或自定义生成循环的缓存对象；
278. 高级使用：如果单图对你有意义，可以将整个 `generate` 函数编译成单个图。

选择下面的正确标签以获取每种方式的详细说明。

无论使用哪种策略与 `torch.compile` 结合，如果你将 LLM 输入左填充到有限的几个值，可以避免与形状相关的重新编译。`pad_to_multiple_of` 分词器标志 是你的朋友！

基本使用：generation_config

高级使用：控制静态缓存

高级使用：端到端生成编译

以下示例使用 Gemma 模型。所有需要做的就是：

279. 访问模型的 `generation_config` 属性并将 `cache_implementation` 设置为“static”；

280. 调用 `torch.compile` 对模型进行编译，以便在静态 kv 缓存下运行前向传播。

就是这样！

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
import os
os.environ["TOKENIZERS_PARALLELISM"] = "false"  # 防止长时间警告

tokenizer = AutoTokenizer.from_pretrained("google/gemma-2b")
model = AutoModelForCausalLM.from_pretrained("google/gemma-2b", torch_dtype="auto",
device_map="auto")

model.generation_config.cache_implementation = "static"

model.forward = torch.compile(model.forward, mode="reduce-overhead", fullgraph=True)
input_text = "The theory of special relativity states "
input_ids = tokenizer(input_text, return_tensors="pt").to(model.device.type)

outputs = model.generate(**input_ids)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
# 输出: ['The theory of special relativity states 1. The speed of light is constant in all
inertial reference']
```

在内部，`generate` 将尝试重用同一个缓存对象，从而避免每次调用时的重新编译。避免重新编译对于充分利用 `torch.compile` 至关重要，你应该注意以下几点：

281. 如果调用之间批量大小发生变化或最大输出长度增加，则缓存需要重新初始化，触发新的编译；

282. 编译函数的前几次调用较慢，因为函数正在被编译。

对于更高级的静态缓存使用，例如多轮对话，我们建议在 generate() 外部实例化和操作缓存对象。请参见高级使用标签。

投机解码

有关更深入的解释，请参阅 辅助生成：通向低延迟文本生成的新方向 博客文章！

自回归的一个问题是，对于每个输入标记，你需要在前向传播期间每次加载模型权重。这对拥有数十亿参数的 LLM 来说既慢又繁琐。投机解码通过使用第二个较小且更快的辅助模型来生成候选标记，这些候选标记由较大的 LLM 在单次前向传播中验证来缓解这种缓慢。如果验证的标记是正确的，LLM 实质上可以“免费”获得这些标记，而不需要自己生成。没有准确性下降，因为验证前向传播确保生成的输出与 LLM 自己生成的输出相同。

为了获得最大的速度提升，辅助模型应该比 LLM 小得多，这样它可以快速生成标记。辅助模型和 LLM 模型还必须共享相同的分词器，以避免重新编码和解码标记。

启用投机解码的方法是加载一个辅助模型并将其传递给 generate() 方法。

贪婪搜索

采样

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
from accelerate.test_utils.testing import get_backend

device, _, _ = get_backend()  # 自动检测底层设备类型 (CUDA, CPU, XPU, MPS 等)

tokenizer = AutoTokenizer.from_pretrained("facebook/opt-1.3b")
inputs = tokenizer("Einstein's theory of relativity states", return_tensors="pt").to(device)

model = AutoModelForCausalLM.from_pretrained("facebook/opt-1.3b", torch_dtype="auto").to(device)
assistant_model = AutoModelForCausalLM.from_pretrained("facebook/opt-125m").to(device)
outputs = model.generate(**inputs, assistant_model=assistant_model)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
# 输出: ["Einstein's theory of relativity states that the speed of light is constant."]
```

提示查找解码

提示查找解码是一种投机解码的变体，也适用于贪婪搜索和采样。提示查找特别适合于输入依赖的任务，例如摘要，其中提示和输出之间经常有重叠的单词。这些重叠的 n-gram 用作 LLM 的候选标记。

要启用提示查找解码，指定 `prompt_lookup_num_tokens` 参数中的应重叠的标记数量。然后你可以将此参数传递给 generate() 方法。

贪婪解码

采样

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
from accelerate.test_utils.testing import get_backend

device, _, _ = get_backend()  # 自动检测底层设备类型 (CUDA, CPU, XPU, MPS 等)

tokenizer = AutoTokenizer.from_pretrained("facebook/opt-1.3b")
inputs = tokenizer("The second law of thermodynamics states", return_tensors="pt").to(device)

model = AutoModelForCausalLM.from_pretrained("facebook/opt-1.3b", torch_dtype="auto").to(device)
assistant_model = AutoModelForCausalLM.from_pretrained("facebook/opt-125m").to(device)
outputs = model.generate(**inputs, prompt_lookup_num_tokens=3)
print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
# 输出: ['The second law of thermodynamics states that entropy increases with temperature.']
```

注意力优化

变压器模型的一个已知问题是，自注意力机制的计算和内存需求随着输入标记数量的增加而呈二次增长。这一限制在处理更长序列的 LLM 中尤为明显。为了解决这个问题，可以尝试 FlashAttention2 或 PyTorch 的缩放点积注意力（SDPA），这两种方法都是更高效的注意力实现，可以加速推理。

FlashAttention-2

FlashAttention 和 FlashAttention-2 将注意力计算分解成更小的部分，并减少读写 GPU 内存的中间操作次数，从而加速推理。FlashAttention-2 在原始 FlashAttention 算法的基础上进行了改进，不仅在序列长度维度上并行化，还更好地划分了硬件上的工作，减少了同步和通信开销。

要使用 FlashAttention-2，可以在 from_pretrained() 方法中设置 `attn_implementation="flash_attention_2"`。

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quant_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-2b",
    quantization_config=quant_config,
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
```

使用 torch.compile 和无填充数据收集器进行微调

除了优化推理外，还可以通过在微调过程中利用 torch.compile 和无填充数据收集器来提高大型语言模型的训练效率。这种方法可以显著加快训练速度并减少计算开销。

以下是如何使用 TRL 库中的 SFTTrainer 微调 Llama 模型，同时启用 torch_compile 并使用无填充数据收集器：

```
#################### 导入 ###################
import math
import datasets
import dataclasses
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments
)
from trl import SFTConfig, SFTTrainer, DataCollatorForCompletionOnlyLM

#################### 加载模型并启用 Flash Attention ###################
model_name = "meta-llama/Llama-3.2-1B"
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    attn_implementation="flash_attention_2"  # 启用 FlashAttention-2
)
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)

#################### 数据预处理（无填充） ###################
response_template = "\n### Label:"
response_template_ids = tokenizer.encode(
    response_template, add_special_tokens=False
)[2:]  # 排除特殊标记

data_collator = DataCollatorForCompletionOnlyLM(
    response_template_ids=response_template_ids,
    tokenizer=tokenizer,
    ignore_index=-100,
    padding_free=True  # 启用无填充收集
)

def format_dataset(example):
    return {
        "output": example["output"] + tokenizer.eos_token
    }

data_files = {"train": "path/to/dataset"}  # 替换为你的数据集路径
json_dataset = datasets.load_dataset("json", data_files=data_files)
formatted_train_dataset = json_dataset["train"].map(format_dataset)

################## 训练配置 ############################
train_args = TrainingArguments(
    num_train_epochs=5,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=1e-5,
    weight_decay=0.0,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
    logging_steps=1,
    include_tokens_per_second=True,
    save_strategy="epoch",
    output_dir="output",
    torch_compile=True,  # 启用 torch.compile
    torch_compile_backend="inductor",
    torch_compile_mode="default"
)

# 将 TrainingArguments 转换为 SFTConfig
transformer_train_arg_fields = [x.name for x in dataclasses.fields(SFTConfig)]
transformer_kwargs = {
    k: v
    for k, v in train_args.to_dict().items()
```

```
    if k in transformer_train_arg_fields
}
training_args = SFTConfig(**transformer_kwargs)

######################## 微调 ####################
trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=formatted_train_dataset,
    data_collator=data_collator,
    dataset_text_field="output",
    args=training_args,
)
trainer.train()
```

PyTorch 缩放点积注意力

缩放点积注意力（SDPA）在 PyTorch 2.0 中自动启用，并且支持 FlashAttention、xFormers 和 PyTorch 的 C++ 实现。SDPA 会在使用 CUDA 后端时选择性能最高的注意力算法。对于其他后端，SDPA 默认使用 PyTorch 的 C++ 实现。

SDPA 支持 FlashAttention-2，只要安装了最新版本的 PyTorch。

使用 torch.backends.cuda.sdp_kernel 上下文管理器来显式启用或禁用三个注意力算法中的任何一个。例如，设置 `enable_flash=True` 以启用 FlashAttention。

```
import torch
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "google/gemma-2b",
    torch_dtype=torch.bfloat16,
)

with torch.backends.cuda.sdp_kernel(enable_flash=True, enable_math=False,
enable_mem_efficient=False):
    outputs = model.generate(**inputs)
```

量化

量化通过以较低精度存储 LLM 权重来减小其大小。这降低了内存使用量，使得在受限于 GPU 内存的情况下更容易加载 LLM 进行推理。如果你不受 GPU 限制，不一定需要量化模型，因为量化和反量化权重所需的额外步骤可能会带来轻微的延迟成本（除非使用 AWQ 和融合 AWQ 模块）。

有许多量化库可用（参见 量化 指南以获取更多详细信息），如 Quanto、AQLM、AWQ 和 AutoGPTQ。你可以尝试这些库，看看哪个最适合你的使用场景。我们也推荐阅读 🤗 Transformers 中本机支持的量化方案概述 博客文章，该文章比较了 AutoGPTQ 和 bitsandbytes。

使用下面的模型内存计算器来估算和比较加载模型所需的内存。例如，尝试估算加载 Mistral-7B-v0.1 所需的内存。

要在半精度下加载 Mistral-7B-v0.1，可以在 from_pretrained() 方法中将 `torch_dtype` 参数设置为 `torch.bfloat16`。这需要 13.74GB 的内存。

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.1", torch_dtype=torch.bfloat16, device_map="auto",
)
```

要加载量化模型（8 位或 4 位）进行推理，可以尝试 bitsandbytes，并将 `load_in_4bit` 或 `load_in_8bit` 参数设置为 `True`。在 8 位下加载模型只需要 6.87 GB 的内存。

```
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
import torch

quant_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.1", quantization_config=quant_config, device_map="auto"
)
```

创建大型模型实例（如何加载大型模型）

大型预训练模型的访问障碍之一是其所需的内存量。加载预训练的 PyTorch 模型通常需要以下步骤：

283. 创建一个带有随机权重的模型。
284. 加载预训练权重。
285. 将这些预训练权重放到模型中。

前两个步骤都需要完整的模型副本在内存中，如果模型有几 GB 大小，那么你可能没有足够的内存来存储两个副本。在分布式训练环境中，这个问题更加严重，因为每个进程都会加载预训练模型并在内存中存储两个副本。

为了解决这个问题，`_fast_init` 参数默认设置为 `True`，以跳过随机初始化，从而提高加载速度。接下来，本文将介绍如何使用 Transformers 加载大型预训练模型，即使这些模型有较高的内存需求。

分区检查点（Sharded Checkpoints）

从 Transformers v4.18.0 开始，大于 10GB 的检查点会由 `save_pretrained()` 方法自动分区。模型会拆分成多个较小的检查点，并创建一个索引文件，将参数名映射到它们所在的文件。

最大分区大小可以通过 `max_shard_size` 参数控制，默认为 5GB，这样可以更轻松地在免费 GPU 实例上运行而不会耗尽内存。

例如，我们将 BioMistral/BioMistral-7B 模型分区：

```
with tempfile.TemporaryDirectory() as tmp_dir:
    model.save_pretrained(tmp_dir, max_shard_size="5GB")
    print(sorted(os.listdir(tmp_dir)))
```

输出结果为：

```
['config.json', 'generation_config.json', 'model-00001-of-00006.safetensors', 'model-00002-of-00006.safetensors', 'model-00003-of-00006.safetensors', 'model-00004-of-00006.safetensors', 'model-00005-of-00006.safetensors', 'model-00006-of-00006.safetensors', 'model.safetensors.index.json']
```

这些分片检查点可以通过 `from_pretrained()` 方法重新加载。

```
with tempfile.TemporaryDirectory() as tmp_dir:
    model.save_pretrained(tmp_dir, max_shard_size="5GB")
    new_model = AutoModel.from_pretrained(tmp_dir)
```

分片检查点的主要优势在于，每次只加载一个分片，从而将内存使用量限制在模型大小和最大分片大小之间。

你也可以使用 `load_sharded_checkpoint()` 方法直接加载分片检查点。

```
from transformers.modeling_utils import load_sharded_checkpoint

with tempfile.TemporaryDirectory() as tmp_dir:
```

```
    model.save_pretrained(tmp_dir, max_shard_size="5GB")
    load_sharded_checkpoint(model, tmp_dir)
```

分片元数据（Shard Metadata）

索引文件决定了哪些键在检查点中以及相应的权重存储在哪里。你可以像加载其他 JSON 文件一样加载这个文件并从中获取字典。

```
import json

with tempfile.TemporaryDirectory() as tmp_dir:
    model.save_pretrained(tmp_dir, max_shard_size="5GB")
    with open(os.path.join(tmp_dir, "model.safetensors.index.json"), "r") as f:
        index = json.load(f)

print(index.keys())
```

`metadata` 键提供了模型的总大小。

```
index["metadata"]
```

`weight_map` 键将每个参数名（通常是 PyTorch 模型中的 `state_dict`）映射到它所在的分片。

```
index["weight_map"]
```

使用 Accelerate 加速大型模型推理

确保你已经安装了 Accelerate v0.9.0 或更高版本以及 PyTorch v1.9.0 或更高版本。

从 Transformers v4.20.0 开始，`from_pretrained()` 方法与 Accelerate 的 Big Model Inference 功能结合使用，可以高效处理非常大的模型！Big Model Inference 在 PyTorch 的 **meta** 设备上创建一个“模型骨架”，随机初始化的参数仅在加载预训练权重时创建。因此，你不会同时在内存中保留两个模型副本，并且最大内存消耗仅限于完整的模型大小。

要启用 Big Model Inference，可以在 `from_pretrained()` 方法中设置 `low_cpu_mem_usage=True`。

```
from transformers import AutoModelForCausalLM

gemma = AutoModelForCausalLM.from_pretrained("google/gemma-7b", low_cpu_mem_usage=True)
```

Accelerate 会自动将模型权重分配到所有可用设备上，优先使用最快的设备（GPU），然后是较慢的设备（CPU 甚至硬盘）。你可以通过在 `from_pretrained()` 方法中设置 `device_map="auto"` 来启用此功能。当你传递 `device_map` 参数时，`low_cpu_mem_usage` 会自动设置为 `True`。

```
from transformers import AutoModelForCausalLM

# 以下加载方法等价
gemma = AutoModelForCausalLM.from_pretrained("google/gemma-7b", device_map="auto")
gemma = AutoModelForCausalLM.from_pretrained("google/gemma-7b", device_map="auto",
low_cpu_mem_usage=True)
```

你还可以自己编写 `device_map`，将每个层映射到一个设备。它应该将所有模型参数映射到一个设备，但如果整个层在同一个设备上，你不需要详细说明该层的所有子模块的位置。

```
device_map = {"model.layers.1": 0, "model.layers.14": 1, "model.layers.31": "cpu", "lm_head":
"disk"}

gemma.hf_device_map
```

输出结果为：

```
{'model.embed_tokens': 0,
 'model.layers.0': 0,
 'model.layers.1': 0,
 'model.layers.2': 0,
 'model.layers.3': 0,
 'model.layers.4': 0,
 'model.layers.5': 0,
 'model.layers.6': 0,
 'model.layers.7': 0,
 'model.layers.8': 0,
 'model.layers.9': 0,
 'model.layers.10': 0,
 'model.layers.11': 0,
 'model.layers.12': 0,
 'model.layers.13': 0,
 'model.layers.14': 'cpu',
 'model.layers.15': 'cpu',
 'model.layers.16': 'cpu',
 'model.layers.17': 'cpu',
 'model.layers.18': 'cpu',
 'model.layers.19': 'cpu',
 'model.layers.20': 'cpu',
 'model.layers.21': 'cpu',
 'model.layers.22': 'cpu',
 'model.layers.23': 'cpu',
 'model.layers.24': 'cpu',
 'model.layers.25': 'cpu',
 'model.layers.26': 'cpu',
 'model.layers.27': 'cpu',
 'model.layers.28': 'cpu',
 'model.layers.29': 'cpu',
 'model.layers.30': 'cpu',
 'model.layers.31': 'cpu',
```

```
'model.norm': 'cpu',
'lm_head': 'cpu'}
```

模型数据类型

PyTorch 模型权重通常以 `torch.float32` 类型实例化，如果你尝试使用不同的数据类型加载模型，可能会有问题。例如，如果你以 `torch.float32` 加载权重再以 `torch.float16` 加载，则需要两倍的内存。

为了节省内存，你可以显式设置 `torch_dtype` 参数为所需的数据类型或设置 `torch_dtype="auto"` 以使用最优化的内存模式（数据类型会自动从模型权重中推导）。

```
from transformers import AutoModelForCausalLM

gemma = AutoModelForCausalLM.from_pretrained("google/gemma-7b", torch_dtype=torch.float16)
```

你还可以为从头开始实例化的模型设置数据类型。

```
import torch
from transformers import AutoConfig, AutoModel

my_config = AutoConfig.from_pretrained("google/gemma-2b", torch_dtype=torch.float16)
model = AutoModel.from_config(my_config)
```

调试（解决训练过程中的问题）

在多个 GPU 上进行训练可能会遇到很多问题，无论是安装问题还是 GPU 之间的通信问题。本调试指南涵盖了可能出现的一些问题以及如何解决这些问题。

DeepSpeed CUDA 安装

如果你正在使用 DeepSpeed，可能已经使用以下命令安装了它。

```
pip install deepspeed
```

DeepSpeed 需要编译 CUDA C++ 代码，因此在构建需要 CUDA 的 PyTorch 扩展时可能会出现错误。这些错误取决于你的系统上如何安装 CUDA。本节主要关注 PyTorch 使用 CUDA 10.2 的情况。

对于其他安装问题，请直接在 DeepSpeed GitHub 仓库上提交问题。

不同版本的 CUDA 工具包

PyTorch 自带 CUDA 工具包，但要使用 DeepSpeed 和 PyTorch，你需要在系统范围内安装相同版本的 CUDA。例如，如果你在 Python 环境中安装了 `cudatoolkit==10.2`，那么你也需要在系统范围内安装 CUDA 10.2。如果你没有安装系统范围的 CUDA，应该先安装。

CUDA 的安装路径可能因系统而异，但常见的路径是 `usr/local/cuda-10.2`。当 CUDA 正确设置并添加到 `PATH` 环境变量后，你可以使用以下命令找到安装路径：

```
which nvcc
```

多个 CUDA 工具包

你可能在系统范围内安装了多个 CUDA 工具包。

通常，包安装程序会设置为最新安装的版本的路径。如果包构建失败，因为它找不到正确的 CUDA 版本（尽管它已经在系统范围内安装），你需要配置 `PATH` 和 `LD_LIBRARY_PATH` 环境变量，指向正确的路径。

首先查看这些环境变量的内容：

```
echo $PATH
echo $LD_LIBRARY_PATH
```

`PATH` 列出可执行文件的位置，而 `LD_LIBRARY_PATH` 列出查找共享库的位置。较早的条目优先于较晚的条目，用 `:` 分隔多个条目。要告诉构建程序在哪里找到特定的 CUDA 工具包，可以将正确的路径插入到列表首位。此命令是前置而不是覆盖现有值。

```
export PATH=/usr/local/cuda-10.2/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-10.2/lib64:$LD_LIBRARY_PATH
```

此外，你应该检查指定的目录是否实际存在。`lib64` 子目录包含各种 CUDA `.so` 对象（例如 `libcudart.so`），虽然你的系统不太可能命名不同，但你应该检查实际名称并相应调整。

旧版 CUDA

有时，旧版 CUDA 可能无法与较新版本的编译器（例如 `gcc-9`）一起构建，而 CUDA 需要 `gcc-7`。通常，安装最新的 CUDA 工具包会启用对新编译器的支持。

你还可以安装一个旧版本的编译器（与当前使用的编译器并存，或者它可能已安装但未默认使用且构建系统无法看到它）。要解决此问题，可以创建一个符号链接，使构建系统能够看到旧版本的编译器。

```
sudo ln -s /usr/bin/gcc-7  /usr/local/cuda-10.2/bin/gcc
sudo ln -s /usr/bin/g++-7  /usr/local/cuda-10.2/bin/g++
```

预构建

如果你在安装 DeepSpeed 时遇到问题，或在运行时构建 DeepSpeed，你可以尝试在安装之前预构建 DeepSpeed 模块。要为 DeepSpeed 进行本地构建：

```
git clone https://github.com/microsoft/DeepSpeed/
cd DeepSpeed
rm -rf build
TORCH_CUDA_ARCH_LIST="8.6" DS_BUILD_CPU_ADAM=1 DS_BUILD_UTILS=1 pip install . \
--global-option="build_ext" --global-option="-j8" --no-cache -v \
--disable-pip-version-check 2>&1 | tee build.log
```

要支持 NVMe 卸载，需在构建命令中添加 `DS_BUILD_AIO=1` 参数，并确保系统范围内安装 `libaio-dev` 包。

接下来，你需要通过编辑 `TORCH_CUDA_ARCH_LIST` 变量来指定 GPU 架构（可以在 此页面 查看 NVIDIA GPU 的架构列表）。要检查与你的架构对应的 PyTorch 版本，运行以下命令：

```
python -c "import torch; print(torch.cuda.get_arch_list())"
```

要查找特定 GPU 的架构，运行以下命令：

```
CUDA_VISIBLE_DEVICES=0 python -c "import torch; print(torch.cuda.get_device_capability())"
```

如果输出为 `8，6`，则可以设置 `TORCH_CUDA_ARCH_LIST="8.6"`。对于具有不同架构的多 GPU，可以列出它们，例如 `TORCH_CUDA_ARCH_LIST="6.1;8.6"`。

虽然也可以不指定 `TORCH_CUDA_ARCH_LIST` 让构建程序自动查询 GPU 架构，但它可能与目标机器上的实际 GPU 不匹配，因此最好明确指定正确的架构。

对于具有相同配置的多台机器上的训练，你需要生成一个二进制 wheel：

```
git clone https://github.com/microsoft/DeepSpeed/
cd DeepSpeed
rm -rf build
TORCH_CUDA_ARCH_LIST="8.6" DS_BUILD_CPU_ADAM=1 DS_BUILD_UTILS=1 \
python setup.py build_ext -j8 bdist_wheel
```

此命令生成一个二进制 wheel 文件，例如 `dist/deepspeed-0.3.13+8cd046f-cp38-cp38-linux_x86_64.whl`。你可以在此机器或另一台机器上安装此 wheel 文件。

```
pip install deepspeed-0.3.13+8cd046f-cp38-cp38-linux_x86_64.whl
```

多 GPU 网络问题调试

当使用 `DistributedDataParallel` 和多个 GPU 进行训练或推理时，如果遇到进程或节点之间的通信问题，可以使用以下脚本诊断网络问题。

```
wget https://raw.githubusercontent.com/huggingface/transformers/main/scripts/distributed/torch-distributed-gpu-test.py
```

例如，测试两个 GPU 之间的交互：

```
python -m torch.distributed.run --nproc_per_node 2 --nnodes 1 torch-distributed-gpu-test.py
```

如果两个进程可以相互通信并分配 GPU 内存，每个进程将输出 OK 状态。

对于更多 GPU 或节点，调整脚本中的参数。

你可以在诊断脚本中找到更多详细信息，甚至有关如何在 SLURM 环境中运行的说明。

添加 `NCCL_DEBUG=INFO` 环境变量来进行更深入的调试：

```
NCCL_DEBUG=INFO python -m torch.distributed.run --nproc_per_node 2 --nnodes 1 torch-distributed-gpu-test.py
```

此命令会输出大量与 NCCL 相关的调试信息，你可以在线搜索是否有问题报告。或者如果你不确定如何解释输出，可以在 Issues 中共享日志文件。

溢出和下溢检测

此功能目前仅适用于 PyTorch。

多 GPU 训练时需要 DDP (`torch.distributed.launch`)。

此功能可以用于任何基于 `nn.Module` 的模型。

如果你开始遇到 `loss=NaN` 或由于激活或权重中的 `inf` 或 `nan` 导致模型出现异常行为，需要发现第一次下溢或溢出以及导致它的原因。幸运的是，你可以通过激活一个特殊模块来自动检测。

如果你使用的是 Trainer，只需在命令行参数中添加：

或者在创建 TrainingArguments 对象时传递 `debug="underflow_overflow"`。

如果你使用自定义训练循环或另一个 Trainer，可以使用以下代码：

```
from transformers.debug_utils import DebugUnderflowOverflow

debug_overflow = DebugUnderflowOverflow(model)
```

`DebugUnderflowOverflow` 会在每次正向调用后插入钩子，以测试输入和输出变量以及相应模块的权重。一旦在激活或权重中至少有一个元素检测到 `inf` 或 `nan`，程序将断言并打印类似于以下内容的报告（这是使用 `google/mt5-small` 和 fp16 混合精度捕获的）：

报告的第一行提供了问题发生在哪个批次（这里是第一个批次）。

每个报告的帧从声明对应的模块的完全限定入口开始。例如，`encoder.block.2.layer.1.layer_norm` 表示这是编码器的第二个块的第一层的层归一化。具体的 `forward` 调用是 `T5LayerNorm`。

要解决问题，需要查看数开始变大的地方，可能是切换到 `fp32` 模式，以防止在乘法或加法时溢出。或者暂时关闭 `amp`，如果它已启用，将原始 `forward` 放入一个辅助包装器中：

```
def _forward(self, hidden_states):
    hidden_gelu = self.gelu_act(self.wi_0(hidden_states))
    hidden_linear = self.wi_1(hidden_states)
    hidden_states = hidden_gelu * hidden_linear
    hidden_states = self.dropout(hidden_states)
    hidden_states = self.wo(hidden_states)
    return hidden_states

import torch

def forward(self, hidden_states):
    if torch.is_autocast_enabled():
        with torch.cuda.amp.autocast(enabled=False):
            return self._forward(hidden_states)
    else:
        return self._forward(hidden_states)
```

调试器默认报告输入和输出的完整帧。如果你需要进一步分析 `forward` 函数中的中间阶段，可以使用 `detect_overflow` 辅助函数在你想调试的地方插入检测器：

```
from debug_utils import detect_overflow

class T5LayerFF(nn.Module):
    [...]
```

```
    def forward(self, hidden_states):
        forwarded_states = self.layer_norm(hidden_states)
        detect_overflow(forwarded_states, "after layer_norm")
        forwarded_states = self.DenseReluDense(forwarded_states)
        detect_overflow(forwarded_states, "after DenseReluDense")
        return hidden_states + self.dropout(forwarded_states)
```

此外，如果你希望自己在代码中实例化调试器，可以调整打印帧的数量（默认为 21）：

```
from transformers.debug_utils import DebugUnderflowOverflow

debug_overflow = DebugUnderflowOverflow(model, max_frames_to_save=100)
```

特定批次绝对最小值和最大值跟踪

调试类也可以用于每批次跟踪，禁用下溢和溢出检测功能。

例如，你想要查看每个 `forward` 调用中所有输入在批次 1 和 3 中的绝对最小值和最大值。可以如下实例化类：

```
debug_overflow = DebugUnderflowOverflow(model, trace_batch_nums=[1, 3])
```

这将对批次 1 和 3 进行完整的跟踪，使用与下溢/溢出检测器相同的格式。

批次从 0 开始编号。

这在你知道程序在某个批次之后开始行为异常时很有帮助。你可以快进到该区域。例如，如果问题在第 150 个批次开始发生，你可以转储批次 149 和 150 的跟踪并比较数值开始偏离的地方。

你还可以指定在某个批次之后停止训练：

```
debug_overflow = DebugUnderflowOverflow(model, trace_batch_nums=[1, 3], abort_after_batch_num=3)
```

TensorFlow 模型中的 XLA 集成

〔图：在 Colab 中打开〕

（原始媒体：https://colab.research.google.com/assets/colab-badge.svg，离线电子书未嵌入，详见原仓库）

〔图：在 Studio Lab 中打开〕

（原始媒体：https://studiolab.sagemaker.aws/studiolab.svg，离线电子书未嵌入，详见原仓库）

加速线性代数（Accelerated Linear Algebra，简称 XLA）是用于加速 TensorFlow 模型运行时的编译器。根据官方文档：

XLA 是一种特定领域的线性代数编译器，可以在不改变源代码的情况下加速 TensorFlow 模型。

在 TensorFlow 中使用 XLA 非常简单——它已经包含在 `tensorflow` 库中，只需通过 `jit_compile` 参数触发即可。例如，在使用像 `tf.function` 这样的图创建函数时，或在使用 Keras 的 `fit()` 和 `predict()` 方法时，可以通过传递 `jit_compile` 参数给 `model.compile()` 来启用 XLA。XLA 并不限于这些方法，还可以加速任意的 `tf.function`。

一些 🤗 Transformers 库中的 TensorFlow 方法已经被重写为 XLA 兼容，包括用于 GPT2、T5 和 OPT 模型的文本生成，以及用于 Whisper 模型的语音处理。

虽然具体的加速效果取决于模型本身，但对于 🤗 Transformers 库中的 TensorFlow 文本生成模型，我们观察到的速度提升大约为 100 倍。本文将介绍如何使用 XLA 提升这些模型的性能，并提供一些额外资源，帮助你了解更多关于基准测试和 XLA 集成的设计理念。

使用 XLA 运行 TensorFlow 函数

让我们以一个简单的 TensorFlow 模型为例：

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, input_shape=(10,), activation="relu"),
    tf.keras.layers.Dense(5, activation="softmax")
])
```

上面的模型接收维度为 `(10, )` 的输入。我们可以像这样运行前向传播：

```
# 生成随机输入
batch_size = 16
input_vector_dim = 10
random_inputs = tf.random.normal((batch_size, input_vector_dim))

# 运行前向传播
_ = model(random_inputs)
```

要使用 XLA 编译的函数运行前向传播，可以这样做：

```
xla_fn = tf.function(model, jit_compile=True)
_ = xla_fn(random_inputs)  # 使用 XLA 编译的函数运行前向传播
```

默认情况下，`model` 的 `call()` 函数用于编译 XLA 图。如果你要编译其他模型函数，也可以这样做：

```
my_xla_fn = tf.function(model.my_xla_fn, jit_compile=True)
```

使用 🤗 Transformers 库中的 XLA 运行文本生成

要启用 🤗 Transformers 库中的 XLA 加速生成，你需要安装最新版本的 `transformers`。可以通过以下命令安装：

```
pip install transformers --upgrade
```

然后可以运行以下代码：

```
import tensorflow as tf
from transformers import AutoTokenizer, TFAutoModelForCausalLM

# 确保安装了最新的 Transformers 版本
from transformers.utils import check_min_version
check_min_version("4.21.0")

tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2", padding_side="left",
pad_token="<</s>>")
model = TFAutoModelForCausalLM.from_pretrained("openai-community/gpt2")
input_string = ["TensorFlow is"]

# 创建一个 XLA 生成函数
xla_generate = tf.function(model.generate, jit_compile=True)

tokenized_input = tokenizer(input_string, return_tensors="tf")
generated_tokens = xla_generate(**tokenized_input, num_beams=2)

decoded_text = tokenizer.decode(generated_tokens[0], skip_special_tokens=True)
print(f"Generated -- {decoded_text}")
```

你会发现，启用 `generate()` 的 XLA 支持只需一行代码。不过，有一些需要注意的地方才能真正实现 XLA 带来的加速效果，我们将在下一节中讨论。

注意事项

当你第一次执行一个 XLA 启用的函数（如上面的 `xla_generate()`），它会尝试推断计算图，这个过程需要时间，被称为“追踪”。

你可能会注意到首次生成时速度并不快。然而，后续调用 `xla_generate()`（或任何其他 XLA 启用的函数）时，如果输入形状与初始构建计算图时一致，则无需重新推断计算图，从而提高生成速度。

对于形状固定的模态（如图像），这不是问题。但对于输入形状变化的模态（如文本），你需要注意这一点。为了确保 `xla_generate()` 始终使用相同的输入形状，你可以在调用分词器时指定 `padding` 参数。

例如：

```
import tensorflow as tf
from transformers import AutoTokenizer, TFAutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2", padding_side="left",
pad_token="<</s>>")
model = TFAutoModelForCausalLM.from_pretrained("openai-community/gpt2")
input_string = ["TensorFlow is"]

xla_generate = tf.function(model.generate, jit_compile=True)

# 调用分词器时指定填充选项
tokenized_input = tokenizer(input_string, pad_to_multiple_of=8, padding=True,
return_tensors="tf")

generated_tokens = xla_generate(**tokenized_input, num_beams=2)
decoded_text = tokenizer.decode(generated_tokens[0], skip_special_tokens=True)
print(f"Generated -- {decoded_text}")
```

这样，你可以确保 `xla_generate()` 始终使用相同的输入形状，从而提高生成速度。你可以通过以下代码验证这一点：

```
import time
import tensorflow as tf
from transformers import AutoTokenizer, TFAutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2", padding_side="left",
pad_token="<</s>>")
model = TFAutoModelForCausalLM.from_pretrained("openai-community/gpt2")

xla_generate = tf.function(model.generate, jit_compile=True)

for input_string in ["TensorFlow is", "TensorFlow is a", "TFLite is a"]:
    tokenized_input = tokenizer(input_string, pad_to_multiple_of=8, padding=True,
return_tensors="tf")
    start = time.time_ns()
    generated_tokens = xla_generate(**tokenized_input, num_beams=2)
    end = time.time_ns()
    print(f"Execution time -- {(end - start) / 1e6:.1f} ms\n")
```

在 Tesla T4 GPU 上，你可能会看到类似以下的输出：

```
Execution time -- 30819.6 ms
Execution time -- 79.0 ms
Execution time -- 78.9 ms
```

首次调用 `xla_generate()` 时由于需要追踪计算图，所以耗时较长，但后续调用则快得多。请注意，任何生成选项的变化都会触发重新追踪，从而导致生成时间变慢。

我们没有涵盖 🤗 Transformers 提供的所有文本生成选项。如果你想了解更多高级用法，请参考官方文档。

进一步资源

如果你想深入了解 🤗 Transformers 和 TensorFlow 中的 XLA，以下是一些额外资源：

- 这个 Colab Notebook 提供了一个交互式演示，展示了 XLA 兼容的编码器-解码器（如 T5）和只解码器（如 GPT2）文本生成模型。
- 这篇博客文章 提供了 XLA 兼容模型的基准测试比较，以及关于 TensorFlow 中 XLA 的友好介绍。
- 这篇博客文章 讨论了我们在 🤗 Transformers 中添加 XLA 支持的设计理念。
- 推荐的学习资源：
    - XLA：机器学习的优化编译器
    - 图形和 tf.function 的介绍
    - 使用 tf.function 提升性能

使用 torch.compile() 优化推理速度

本指南旨在展示使用 `torch.compile()` 在不同硬件和批处理大小下对 🤗 Transformers 中的计算机视觉模型进行推理加速的效果。

使用 torch.compile() 的优势

根据模型和 GPU 的不同，`torch.compile()` 可以使推理速度提高 30%。要使用 `torch.compile()`，只需安装 PyTorch 2.0 以上的版本即可。

编译模型需要一定的时间，因此建议只编译一次，而不是每次推理都重新编译。要编译任何计算机视觉模型，只需在模型上调用 `torch.compile()`，如下所示：

```
from transformers import AutoModelForImageClassification
import torch

model = AutoModelForImageClassification.from_pretrained(MODEL_ID).to(DEVICE)
model = torch.compile(model)  # 编译模型
```

`compile()` 提供了多种编译模式，不同模式的编译时间和推理开销有所不同。`max-autotune` 模式需要更长的编译时间，但推理速度更快。默认模式编译速度最快，但在推理时间上不如 `reduce-overhead` 模式高效。本指南使用的是默认模式。您可以了解更多关于这些模式的内容 这里。

我们在 `torch` 版本 2.0.1 上对不同计算机视觉模型、任务、硬件类型和批处理大小进行了基准测试。

基准测试代码

以下是每个任务的基准测试代码。我们在推理前先预热 GPU，并取 300 次推理的平均时间，每次使用相同的图像。

使用 ViT 进行图像分类

```
import torch
from PIL import Image
import requests
from transformers import AutoImageProcessor, AutoModelForImageClassification
from accelerate.test_utils.testing import get_backend

device, _, _ = get_backend()  # 自动检测底层设备类型（CUDA、CPU、XPU、MPS 等）
url = 'http://images.cocodataset.org/val2017/000000039769.jpg'
image = Image.open(requests.get(url, stream=True).raw)

processor = AutoImageProcessor.from_pretrained("google/vit-base-patch16-224")
model = AutoModelForImageClassification.from_pretrained("google/vit-base-patch16-
224").to(device)
model = torch.compile(model)  # 编译模型

processed_input = processor(image, return_tensors='pt').to(device)

with torch.no_grad():
    _ = model(**processed_input)  # 进行推理
```

使用 DETR 进行目标检测

```
from transformers import AutoImageProcessor, AutoModelForObjectDetection
from accelerate.test_utils.testing import get_backend

device, _, _ = get_backend()  # 自动检测底层设备类型（CUDA、CPU、XPU、MPS 等）
processor = AutoImageProcessor.from_pretrained("facebook/detr-resnet-50")
model = AutoModelForObjectDetection.from_pretrained("facebook/detr-resnet-50").to(device)
model = torch.compile(model)  # 编译模型

texts = ["a photo of a cat", "a photo of a dog"]
inputs = processor(text=texts, images=image, return_tensors="pt").to(device)

with torch.no_grad():
    _ = model(**inputs)  # 进行推理
```

使用 Segformer 进行图像分割

```
from transformers import SegformerImageProcessor, SegformerForSemanticSegmentation
from accelerate.test_utils.testing import get_backend

device, _, _ = get_backend()  # 自动检测底层设备类型（CUDA、CPU、XPU、MPS 等）
processor = SegformerImageProcessor.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")
model = SegformerForSemanticSegmentation.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512").to(device)
model = torch.compile(model)  # 编译模型
seg_inputs = processor(images=image, return_tensors="pt").to(device)

with torch.no_grad():
    _ = model(**seg_inputs)  # 进行推理
```

以下是我们在基准测试中使用的模型列表。

图像分类

- google/vit-base-patch16-224
- microsoft/beit-base-patch16-224-pt22k-ft22k
- facebook/convnext-large-224
- microsoft/resnet-50

图像分割

- nvidia/segformer-b0-finetuned-ade-512-512

- facebook/mask2former-swin-tiny-coco-panoptic
- facebook/maskformer-swin-base-ade
- google/deeplabv3_mobilenet_v2_1.0_513

目标检测

- google/owlvit-base-patch32
- facebook/detr-resnet-101
- microsoft/conditional-detr-resnet-50

以下图表展示了使用和不使用 `torch.compile()` 的推理时间对比以及每个模型在不同硬件和批处理大小下的性能提升百分比。

〔图：A100 批量 1 比较〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/torch_compile/A100_1_duration.png，离线电子书未嵌入，详见原仓库）

〔图：A100 批量 1 提升百分比〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/torch_compile/A100_1_percentage.png，离线电子书未嵌入，详见原仓库）

A100 （批量大小：1）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	9.325	7.584
图像分割/Segformer	11.759	10.500
目标检测/OwlViT	24.978	18.420
图像分类/BeiT	11.282	8.448
目标检测/DETR	34.619	19.040
图像分类/ConvNeXT	10.410	10.208
图像分类/ResNet	6.531	4.124
图像分割/Mask2former	60.188	49.117
图像分割/Maskformer	75.764	59.487
图像分割/MobileNet	8.583	3.974

目标检测/Resnet-101	36.276	18.197
目标检测/Conditional-DETR	31.219	17.993

A100 （批量大小：4）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	14.832	14.499
图像分割/Segformer	18.838	16.476
图像分类/BeiT	13.205	13.048
目标检测/DETR	48.657	32.418
图像分类/ConvNeXT	22.940	21.631
图像分类/ResNet	6.657	4.268
图像分割/Mask2former	74.277	61.781
图像分割/Maskformer	180.700	159.116
图像分割/MobileNet	14.174	8.515
目标检测/Resnet-101	68.101	44.998
目标检测/Conditional-DETR	56.470	35.552

A100 （批量大小：16）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	40.944	40.010
图像分割/Segformer	37.005	31.144
图像分类/BeiT	41.854	41.048
目标检测/DETR	164.382	161.902
图像分类/ConvNeXT	82.258	75.561
图像分类/ResNet	7.018	5.024

图像分割/Mask2former	178.945	154.814
图像分割/Maskformer	638.570	579.826
图像分割/MobileNet	51.693	30.310
目标检测/Resnet-101	232.887	155.021
目标检测/Conditional-DETR	180.491	124.032

V100 （批量大小：1）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	10.495	6.00
图像分割/Segformer	13.321	5.862
目标检测/OwlViT	25.769	22.395
图像分类/BeiT	11.347	7.234
目标检测/DETR	33.951	19.388
图像分类/ConvNeXT	11.623	10.412
图像分类/ResNet	6.484	3.820
图像分割/Mask2former	64.640	49.873
图像分割/Maskformer	95.532	72.207
图像分割/MobileNet	9.217	4.753
目标检测/Resnet-101	52.818	28.367
目标检测/Conditional-DETR	39.512	20.816

V100 （批量大小：4）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	15.181	14.501
图像分割/Segformer	16.787	16.188

图像分类/BeiT	15.171	14.753
目标检测/DETR	88.529	64.195
图像分类/ConvNeXT	29.574	27.085
图像分类/ResNet	6.109	4.731
图像分割/Mask2former	90.402	76.926
图像分割/Maskformer	234.261	205.456
图像分割/MobileNet	24.623	14.816
目标检测/Resnet-101	134.672	101.304
目标检测/Conditional-DETR	97.464	69.739

V100 （批量大小：16）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	52.209	51.633
图像分割/Segformer	61.013	55.499
图像分类/BeiT	53.938	53.581
目标检测/DETR	OOM	OOM
图像分类/ConvNeXT	109.682	100.771
图像分类/ResNet	14.857	12.089
图像分割/Mask2former	249.605	222.801
图像分割/Maskformer	831.142	743.645
图像分割/MobileNet	93.129	55.365
目标检测/Resnet-101	482.425	361.843
目标检测/Conditional-DETR	344.661	255.298

T4 （批量大小：1）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	16.520	15.786
图像分割/Segformer	16.116	14.205
目标检测/OwlViT	53.634	51.105
图像分类/BeiT	16.464	15.710
目标检测/DETR	73.100	53.99
图像分类/ConvNeXT	32.932	30.845
图像分类/ResNet	6.031	4.321
图像分割/Mask2former	79.192	66.815
图像分割/Maskformer	200.026	188.268
图像分割/MobileNet	18.908	11.997
目标检测/Resnet-101	106.622	82.566
目标检测/Conditional-DETR	77.594	56.984

T4 （批量大小：4）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	43.653	43.626
图像分割/Segformer	45.327	42.445
图像分类/BeiT	52.007	51.354
目标检测/DETR	277.850	268.003
图像分类/ConvNeXT	119.259	105.580
图像分类/ResNet	13.039	11.388
图像分割/Mask2former	201.540	184.670
图像分割/Maskformer	764.052	711.280
图像分割/MobileNet	74.289	48.677

目标检测/Resnet-101	421.859	357.614
目标检测/Conditional-DETR	289.002	226.945

T4 （批量大小：16）

任务/模型	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ViT	163.914	160.907
图像分割/Segformer	192.412	163.620
图像分类/BeiT	188.978	187.976
目标检测/DETR	OOM	OOM
图像分类/ConvNeXT	422.886	388.078
图像分类/ResNet	44.114	37.604
图像分割/Mask2former	756.337	695.291
图像分割/Maskformer	2842.940	2656.88
图像分割/MobileNet	299.003	201.942
目标检测/Resnet-101	1619.505	1262.758
目标检测/Conditional-DETR	1137.513	897.390

PyTorch Nightly

我们还在 PyTorch Nightly（2.1.0dev 版本，下载链接）上进行了基准测试，并观察到未编译和编译模型的延迟都有所改进。

A100

任务/模型	**批量大小**	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/BeiT	未批量	12.462	6.954
图像分类/BeiT	4	14.109	12.851

图像分类/BeiT	16	42.179	42.147
目标检测/DETR	未批量	30.484	15.221
目标检测/DETR	4	46.816	30.942
目标检测/DETR	16	163.749	163.706

T4

任务/模型	**批量大小**	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/BeiT	未批量	14.408	14.052
图像分类/BeiT	4	47.381	46.604
图像分类/BeiT	16	42.179	42.147
目标检测/DETR	未批量	68.382	53.481
目标检测/DETR	4	269.615	204.785
目标检测/DETR	16	OOM	OOM

V100

任务/模型	**批量大小**	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/BeiT	未批量	13.477	7.926
图像分类/BeiT	4	15.103	14.378
图像分类/BeiT	16	52.517	51.691
目标检测/DETR	未批量	28.706	19.077
目标检测/DETR	4	88.402	62.949
目标检测/DETR	16	OOM	OOM

减少开销模式基准测试

我们在 A100 和 T4 上使用 PyTorch Nightly 测试了 `reduce-overhead` 模式。

A100

任务/模型	**批量大小**	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ConvNeXT	未批量	11.758	7.335
图像分类/ConvNeXT	4	23.171	21.490
图像分类/ResNet	未批量	7.435	3.801
图像分类/ResNet	4	7.261	2.187
目标检测/Conditional-DETR	未批量	32.823	11.627
目标检测/Conditional-DETR	4	50.622	33.831
图像分割/MobileNet	未批量	9.869	4.244
图像分割/MobileNet	4	14.385	7.946

T4

任务/模型	**批量大小**	**torch 2.0 - 未编译**	**torch 2.0 - 编译**
图像分类/ConvNeXT	未批量	32.137	31.84
图像分类/ConvNeXT	4	120.944	110.209
图像分类/ResNet	未批量	9.761	7.698
图像分类/ResNet	4	15.215	13.871
目标检测/Conditional-DETR	未批量	72.150	57.660
目标检测/Conditional-DETR	4	301.494	247.543

图像分割/MobileNet	未批量	22.266	19.339
图像分割/MobileNet	4	78.311	50.983

高效训练方法

在单个 GPU 上进行高效训练的方法和工具

本指南展示了实用的技术，用于通过优化内存利用、加速训练或两者来提高模型训练的效率。如果您想了解在训练过程中如何利用 GPU，请首先参考 模型训练解剖学 概念指南。本指南侧重于实用技术。

如果您有一台多 GPU 的机器，这些方法仍然有效，此外您还可以利用 多 GPU 部分 中列出的其他方法。

在训练大型模型时，需要同时考虑两个方面：

- 数据吞吐量/训练时间
- 模型性能

最大化吞吐量（每秒样本数）可以降低训练成本。这通常通过尽可能利用 GPU 并填满 GPU 内存来实现。如果所需的批量大小超过了 GPU 内存的限制，可以使用梯度累加等内存优化技术来帮助解决。

然而，如果首选的批量大小适合内存，就没有必要应用内存优化技术，因为它们可能会减慢训练速度。仅仅因为可以使用大批量，并不意味着应该这样做。在超参数调优过程中，您应该确定哪个批量大小效果最好，然后相应地优化资源。

本指南中涵盖的方法和工具可以根据它们对训练过程的影响进行分类：

方法/工具	提高训练速度	优化内存利用
批量大小选择	是	是
梯度累加	否	是
梯度检查点	否	是

混合精度训练	是	可能*
torch_empty_cache_steps	否	是
优化器选择	是	是
数据预加载	是	否
DeepSpeed ZeRO	否	是
torch.compile	是	否
参数高效微调 (PEFT)	否	是

* 注：当使用混合精度与小模型和大批量时，会有一定的内存节省；但对于大模型和小批量，内存使用会更大。

您可以结合上述方法以获得累积效果。无论您是使用 Trainer 进行模型训练，还是编写纯 PyTorch 循环，您都可以 使用 🤗 Accelerate 配置这些优化。

如果这些方法没有带来足够的收益，您可以探索以下选项：

- 构建自定义 Docker 容器，其中包含高效的软件预构建
- 考虑使用专家混合 (MoE) 的模型
- 将模型转换为 BetterTransformer 以利用 PyTorch 原生注意力机制

最后，如果所有方法仍然不够，即使切换到服务器级 GPU（如 A100），也可以考虑转向多 GPU 设置。所有这些方法在多 GPU 设置中仍然有效，并且您可以利用 多 GPU 部分 中概述的额外并行技术。

批量大小选择

要实现最佳性能，首先要确定合适的批量大小。建议使用 2^N 大小的批量和输入/输出神经元数量。通常它是 8 的倍数，但具体取决于所使用的硬件和模型的数据类型。

参考 NVIDIA 推荐的 输入/输出神经元数量 和 批量大小，这些涉及完全连接层（参与 GEMMs（通用矩阵乘法））。

Tensor Core 要求 定义了基于数据类型和硬件的乘数。例如，对于 fp16 数据类型，建议使用 8 的倍数，除非是 A100 GPU，这种情况下使用 64 的倍数。

对于较小的参数，还应考虑 维度量化效应。这里会发生切片，合适的乘数可以显著加速。

梯度累加(Gradient Accumulation)

梯度累加 方法旨在分小批计算梯度，而不是一次性计算整个批量的梯度。这种方法通过多次前向和后向传递模型来迭代计算梯度，并在过程中积累梯度。一旦积累了足够数量的梯度，就执行模型的优化步骤。通过使用梯度累加，可以将 **有效批量大小** 增加到超过 GPU 内存容量的限制。然而，需要注意的是，梯度累加引入的额外前向和后向传递可能会减慢训练过程。

您可以通过在 TrainingArguments 中添加 `gradient_accumulation_steps` 参数来启用梯度累加：

```
training_args = TrainingArguments(per_device_train_batch_size=1, gradient_accumulation_steps=4,
**default_args)
```

在上面的例子中，您的有效批量大小变为 4。

或者，使用 🤗 Accelerate 来获得对训练循环的完全控制。有关 🤗 Accelerate 示例，请参阅 本指南中的进一步说明。

虽然建议尽可能充分利用 GPU，但过多的梯度累加步骤会导致更明显的训练减速。考虑以下示例。假设 `per_device_train_batch_size=4` 不使用梯度累加达到了 GPU 的极限。如果您希望以 64 的批量进行训练，不要将 `per_device_train_batch_size` 设为 1 并将 `gradient_accumulation_steps` 设为 64。相反，保持 `per_device_train_batch_size=4` 并将 `gradient_accumulation_steps` 设为 16。这样可以获得相同的有效批量大小，同时更好地利用可用的 GPU 资源。

有关更多信息，请参阅 RTX-3090 和 A100 的批量大小和梯度累加基准测试。

梯度检查点(Gradient Checkpointing)

即使将批量大小设为 1 并使用梯度累加，某些大型模型仍可能面临内存问题。这是因为在前向传递期间保存所有激活以便在后向传递期间计算梯度会占用大量内存。另一种方法是在需要时丢弃激活并重新计算，但这会引入大量的计算开销并减慢训练过程。

梯度检查点 提供了一种折衷方案，通过在计算图中战略性地保存部分激活，使得只需重新计算一部分激活即可完成梯度计算。有关梯度检查点的深入解释，请参阅 这篇文章。

要在 Trainer 中启用梯度检查点，可以将相应的标志传递给 TrainingArguments：

```
training_args = TrainingArguments(
    per_device_train_batch_size=1, gradient_accumulation_steps=4, gradient_checkpointing=True,
**default_args
)
```

或者，使用 🤗 Accelerate - 请参阅 本指南中的进一步说明。

尽管梯度检查点可以改善内存效率，但它会使训练速度减慢约 20%。

混合精度训练(Mixed precision training)

混合精度训练 是一种通过使用低精度数值格式来优化模型训练计算效率的技术。传统上，大多数模型使用 32 位浮点精度（fp32 或 float32）来表示和处理变量。然而，并非所有变量都需要如此高的精度来获得准确的结果。通过将某些变量的精度降低到更低的数值格式（如 16 位浮点数（fp16 或 float16）），我们可以加快计算速度。在这种方法中，一些计算以半精度进行，而另一些则仍以全精度进行，因此称为混合精度训练。

最常见的混合精度训练是使用 fp16（float16）数据类型，但某些 GPU 架构（如 Ampere 架构）提供 bf16 和 tf32（CUDA 内部数据类型）。有关这些数据类型差异的详细信息，请参阅 NVIDIA 博客。

fp16

混合精度训练的主要优势在于以半精度（fp16）保存激活。尽管梯度也以半精度计算，但在优化步骤中会转换回全精度，因此在这里不会节省内存。虽然混合精度训练可以加快计算速度，但也会导致 GPU 内存使用增加，尤其是对于小批量。这是因为模型现在在 GPU 上同时存在 16 位和 32 位精度（1.5 倍于原始模型）。

要启用混合精度训练，可以设置 `fp16` 标志为 `True`：

```
training_args = TrainingArguments(per_device_train_batch_size=4, fp16=True, **default_args)
```

如果您更喜欢使用 🤗 Accelerate，请参阅 本指南中的进一步说明。

BF16

如果您有访问 Ampere 或更新硬件的权限，可以使用 bf16 进行混合精度训练和评估。虽然 bf16 的精度不如 fp16，但其动态范围更大。在 fp16 中，最大的数字是 `65504`，任何大于该值的数字都会导致溢出。bf16 数字可以达到 `3.39e+38`，与 fp32 相当，因为两者都使用 8 位来表示数值范围。

您可以在 🤗 Trainer 中启用 BF16：

```
training_args = TrainingArguments(bf16=True, **default_args)
```

TF32

Ampere 硬件使用一种名为 tf32 的神奇数据类型。它具有与 fp32 相同的数值范围（8 位），但精度只有 10 位（与 fp16 相同），总共使用 19 位。它之所以被称为“神奇”，是因为您可以使用正常的 fp32 训练和/或推理代码，通过启用 tf32 支持，可以将吞吐量提高多达 3 倍。您只需要在代码中添加以下内容：

```
import torch
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
```

CUDA 将自动切换到使用 tf32 而不是 fp32，前提是使用的 GPU 是 Ampere 系列。

根据 NVIDIA 研究，大多数机器学习训练工作负载显示，使用 tf32 训练与使用 fp32 具有相同的困惑度和收敛性。如果您已经在使用 fp16 或 bf16 混合精度，它也可能有助于吞吐量。

您可以在 🤗 Trainer 中启用此模式：

```
TrainingArguments(tf32=True, **default_args)
```

tf32 不能直接通过 `tensor.to(dtype=torch.tf32)` 访问，因为它是一种内部 CUDA 数据类型。需要 `torch>=1.7` 才能使用 tf32 数据类型。

有关 tf32 与其他精度的更多基准测试，请参阅：RTX-3090 和 A100。

Flash Attention 2

您可以使用 transformers 中的 Flash Attention 2 集成来加速训练吞吐量。有关如何加载带有 Flash Attention 2 模块的模型的详细信息，请参阅 单 GPU 部分。

优化器选择

最常用的优化器是 Adam 或 AdamW（带权重衰减的 Adam）。Adam 通过存储先前梯度的滚动平均值来实现良好的收敛，但会增加与模型参数数量相当的额外内存开销。为了补救这一点，您可以用替代优化器。例如，如果您安装了 NVIDIA GPUs 的 NVIDIA/apex 或 AMD GPUs 的 ROCmSoftwarePlatform/apex，`adamw_apex_fused` 将在所有支持的 AdamW 优化器中提供最快的训练体验。

Trainer 集成了多种优化器，可以直接使用：`adamw_hf`、`adamw_torch`、`adamw_torch_fused`、`adamw_apex_fused`、`adamw_anyprecision`、`adafactor` 或 `adamw_bnb_8bit`。还可以通过第三方实现添加更多优化器。

让我们更详细地看一下两种替代 AdamW 的优化器：

286. `adafactor` 可在 Trainer 中使用

287. `adamw_bnb_8bit` 也在 Trainer 中可用，但下面提供了一个第三方集成示例以供演示。

以一个 3B 参数模型（如 “google-t5/t5-3b”）为例：

- 标准 AdamW 优化器需要 24GB 的 GPU 内存，因为它每个参数使用 8 字节（8 * 3 = 24GB）
- Adafactor 优化器需要超过 12GB 的内存。它每个参数使用略多于 4 字节，所以 4 * 3 加上一些额外的内存。
- 8 位 BNB 量化优化器仅需 (2 * 3) 6GB 的内存，前提是所有优化器状态都被量化。

Adafactor

Adafactor 不为权重矩阵中的每个元素存储滚动平均值，而是保留聚合信息（按行和列汇总的滚动平均值），显著减少其内存占用。然而，与 Adam 相比，Adafactor 在某些情况下可能收敛更慢。

您可以通过在 TrainingArguments 中设置 `optim="adafactor"` 来切换到 Adafactor：

```
training_args = TrainingArguments(per_device_train_batch_size=4, optim="adafactor",
**default_args)
```

结合其他方法（梯度累加、梯度检查点和混合精度训练），可以看到高达 3 倍的改进，同时保持吞吐量！然而，如前所述，Adafactor 的收敛可能不如 Adam。

8 位 Adam

与 Adafactor 不同，8 位 Adam 保留完整的状态并对其进行量化。量化意味着它以较低的精度存储状态，仅在优化时反量化。这类似于混合精度训练的思想。

要使用 `adamw_bnb_8bit`，只需在 TrainingArguments 中设置 `optim="adamw_bnb_8bit"`：

```
training_args = TrainingArguments(per_device_train_batch_size=4, optim="adamw_bnb_8bit",
**default_args)
```

然而，我们也可以使用第三方实现的 8 位优化器进行演示，看看如何集成。

首先，按照 GitHub 仓库 中的安装指南安装实现 8 位 Adam 优化器的 `bitsandbytes` 库。

接下来，需要初始化优化器。这涉及两个步骤：

- 首先，将模型的参数分为两组 - 一组应用权重衰减，另一组不应用。通常，偏差和层归一化参数不进行权重衰减。
- 然后进行一些参数整理，以使用与之前使用的 AdamW 优化器相同的参数。

```
import bitsandbytes as bnb
from torch import nn
from transformers.trainer_pt_utils import get_parameter_names

training_args = TrainingArguments(per_device_train_batch_size=4, **default_args)

decay_parameters = get_parameter_names(model, [nn.LayerNorm])
decay_parameters = [name for name in decay_parameters if "bias" not in name]
optimizer_grouped_parameters = [
    {
        "params": [p for n, p in model.named_parameters() if n in decay_parameters],
        "weight_decay": training_args.weight_decay,
    },
    {
        "params": [p for n, p in model.named_parameters() if n not in decay_parameters],
        "weight_decay": 0.0,
    },
]

optimizer_kwargs = {
    "betas": (training_args.adam_beta1, training_args.adam_beta2),
    "eps": training_args.adam_epsilon,
}
optimizer_kwargs["lr"] = training_args.learning_rate
adam_bnb_optim = bnb.optim.Adam8bit(
    optimizer_grouped_parameters,
    betas=(training_args.adam_beta1, training_args.adam_beta2),
    eps=training_args.adam_epsilon,
```

```
    lr=training_args.learning_rate,
)
```

最后，将自定义优化器作为参数传递给 `Trainer`：

```
trainer = Trainer(model=model, args=training_args, train_dataset=ds, optimizers=(adam_bnb_optim, None))
```

结合其他方法（梯度累加、梯度检查点和混合精度训练），可以预期获得大约 3 倍的内存改进，甚至稍微更高的吞吐量，就像使用 Adafactor 一样。

multi_tensor

pytorch-nightly 引入了 `torch.optim._multi_tensor`，这应该在处理大量小特征张量的情况下显著加快优化器的速度。它最终将成为默认设置，但如果您想提前尝试，可以查看这个 GitHub 问题。

数据预加载

实现快速训练速度的一个重要要求是能够以 GPU 能够处理的最大速度喂数据。默认情况下，所有操作都在主进程中进行，可能无法从磁盘读取数据快到足以创建瓶颈，从而导致 GPU 利用率不足。配置以下参数可以减少瓶颈：

- `DataLoader(pin_memory=True, ...)` - 确保数据预加载到 CPU 的固定内存中，通常可以更快地从 CPU 转移到 GPU 内存。
- `DataLoader(num_workers=4, ...)` - 启动多个工作线程以更快地预加载数据。在训练过程中，观察 GPU 利用率统计信息；如果远未达到 100%，可以尝试增加工作线程的数量。当然，问题可能出在其他地方，因此增加工作线程数量并不一定会提高性能。

当使用 Trainer 时，对应的 TrainingArguments 是：`dataloader_pin_memory`（默认为 `True`），和 `dataloader_num_workers`（默认为 `0`）。

DeepSpeed ZeRO

DeepSpeed 是一个开源深度学习优化库，已集成到 🤗 Transformers 和 🤗 Accelerate 中。它提供了广泛的功能和优化，旨在提高大规模深度学习训练的效率和可扩展性。

如果您的模型能够适应单个 GPU 并有足够的空间容纳小批量，就不需要使用 DeepSpeed，因为它只会减慢速度。然而，如果模型无法适应单个 GPU，或者无法容纳小批量，您可以利用

DeepSpeed ZeRO + CPU 卸载，或 NVMe 卸载来处理更大的模型。在这种情况下，需要单独 安装库，然后按照其中一个指南创建配置文件并启动 DeepSpeed：

- 对于 Trainer 的 DeepSpeed 集成深入指南，请参阅 相关文档，特别是 单 GPU 部分。在笔记本中使用 DeepSpeed 需要进行一些调整，请参阅 相应指南。
- 如果您更喜欢使用 🤗 Accelerate，请参阅 🤗 Accelerate DeepSpeed 指南。

使用 torch.compile

PyTorch 2.0 引入了一个新的编译函数，无需修改现有 PyTorch 代码，只需添加一行代码即可优化代码：`model = torch.compile(model)`。

如果使用 Trainer，只需在 TrainingArguments 中传递 `torch_compile` 选项：

```
training_args = TrainingArguments(torch_compile=True, **default_args)
```

`torch.compile` 使用 Python 的帧评估 API 从现有的 PyTorch 程序中自动创建图。捕获图后，可以部署不同的后端以将图降低到优化引擎。有关更多细节和基准测试，请参阅 PyTorch 文档。

`torch.compile` 有一个不断增长的后端列表，可以通过调用 `torchdynamo.list_backends()` 查看，每个后端都有其可选依赖项。

通过在 TrainingArguments 中指定 `torch_compile_backend` 来选择要使用的后端。一些最常用的后端包括：

调试后端：

- `dynamo.optimize("eager")` - 使用 PyTorch 运行提取的 GraphModule。这在调试 TorchDynamo 问题时非常有用。
- `dynamo.optimize("aot_eager")` - 使用 AotAutograd 但不使用编译器，即仅使用 PyTorch 快速模式运行 AotAutograd 提取的前向和后向图。这在调试时很有用，但不太可能提高速度。

训练和推理后端：

- `dynamo.optimize("inductor")` - 使用 TorchInductor 后端和 AotAutograd 以及代码生成的 Triton 内核。了解更多
- `dynamo.optimize("nvfuser")` - 使用 nvFuser 与 TorchScript。了解更多

- `dynamo.optimize("aot_nvfuser")` - 使用 nvFuser 与 AotAutograd。了解更多
- `dynamo.optimize("aot_cudagraphs")` - 使用 cudagraphs 与 AotAutograd。了解更多

仅推理后端：

- `dynamo.optimize("ofi")` - 使用 TorchScript optimize_for_inference。了解更多
- `dynamo.optimize("fx2trt")` - 使用 NVIDIA TensorRT 进行推理优化。了解更多
- `dynamo.optimize("onnxrt")` - 使用 ONNXRT 进行 CPU/GPU 推理。了解更多
- `dynamo.optimize("ipex")` - 使用 IPEX 进行 CPU 推理。了解更多

有关使用 `torch.compile` 与 🤗 Transformers 的示例，请参阅这篇 博客文章。

使用 🤗 PEFT

参数高效微调 (PEFT) 方法在微调过程中冻结预训练模型参数，并在其顶部添加少量可训练参数（适配器）。

结果是 与优化器状态和梯度相关的内存 大幅减少。

例如，使用标准 AdamW 时，优化器状态的内存需求为：

- fp32 参数副本：4 字节/参数
- 动量：4 字节/参数
- 方差：4 字节/参数

假设一个 7B 参数模型，并使用 低秩适配器 注入 2000 万参数。

普通模型的优化器状态内存需求为 12 * 7 = 84 GB（假设 7B 可训练参数）。

添加 Lora 会略微增加与模型权重相关的内存，但会大幅减少优化器状态的内存需求至 12 * 0.2 = 2.4 GB。

有关 PEFT 及其详细使用的更多信息，请参阅 PEFT 文档 或 PEFT 仓库。

使用 🤗 Accelerate

使用 🤗 Accelerate，您可以在获得对训练循环的完全控制的同时使用上述方法，基本上可以编写纯 PyTorch 循环，只需进行一些小的修改。

假设您已经组合了 TrainingArguments 中的方法，如下所示：

```
training_args = TrainingArguments(
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    gradient_checkpointing=True,
    fp16=True,
    **default_args,
)
```

使用 🤗 Accelerate 的完整训练循环示例只有几行代码：

```
from accelerate import Accelerator
from torch.utils.data.dataloader import DataLoader

dataloader = DataLoader(ds, batch_size=training_args.per_device_train_batch_size)

if training_args.gradient_checkpointing:
    model.gradient_checkpointing_enable()

accelerator = Accelerator(fp16=training_args.fp16)
model, optimizer, dataloader = accelerator.prepare(model, adam_bnb_optim, dataloader)

model.train()
for step, batch in enumerate(dataloader, start=1):
    loss = model(**batch).loss
    loss = loss / training_args.gradient_accumulation_steps
    accelerator.backward(loss)
    if step % training_args.gradient_accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

首先，我们将数据集包装在一个 `DataLoader` 中。然后，可以通过调用模型的 gradient_checkpointing_enable() 方法启用梯度检查点。当初始化 `Accelerator` 时，可以指定是否使用混合精度训练，它将在 `prepare` 调用中为您处理。在 `prepare` 调用中，如果使用多个 GPU，数据加载器也将分布在各个工作线程上。我们使用前面示例中的相同 8 位优化器。

最后，可以添加主要的训练循环。注意，`backward` 调用由 🤗 Accelerate 处理。我们还可以看到梯度累加是如何工作的：我们对损失进行归一化，以便在累积结束时得到平均值，一旦累积了足够的步数，就执行优化。

使用 🤗 Accelerate 实现这些优化技术只需几行代码，而且具有更大的训练循环灵活性。有关所有功能的完整文档，请参阅 Accelerate 文档。

高效软件预构建

PyTorch 的 pip 和 conda 构建 预构建了 cuda 工具包，这足以运行 PyTorch，但不足以构建 cuda 扩展。

有时，可能需要额外的努力来预构建某些组件。例如，如果您使用像 `apex` 这样的库，它们没有预编译。在其他情况下，弄清楚如何系统范围内安装正确的 cuda 工具包可能会很复杂。为了解决这些问题，PyTorch 和 NVIDIA 发布了一个新的 NGC Docker 容器版本，已经预构建了所有组件。您只需在其中安装您的程序，就可以直接运行。

如果要调整 PyTorch 源代码和/或制作新的自定义构建，这种方法也非常有用。要找到所需的 Docker 镜像版本，可以从 PyTorch 发行说明 开始，选择最新的月度发行版之一。进入所需发行版的发行说明，检查环境组件是否符合您的需求（包括 NVIDIA 驱动程序要求！），然后在文档顶部转到相应的 NGC 页面。如果迷失方向，这里有一个 所有 PyTorch NGC 镜像的索引。

接下来，按照说明下载并部署 Docker 镜像。

专家混合(Mixture of Experts)

最近的一些论文报告称，通过在 Transformer 模型中集成专家混合（MoE），训练速度提高了 4-5 倍，推理也更快。

自从发现更多的参数可以提高性能以来，这项技术允许在不增加训练成本的情况下将参数数量增加一个数量级。

在这种方法中，每隔一个 FFN 层被替换为 MoE 层，MoE 层由许多专家组成，通过门控函数根据输入令牌在序列中的位置平衡每个专家的训练。

〔图：MoE Transformer 2x 块〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/perf-moe-transformer.png，离线电子书未嵌入，详见原仓库）

（来源：GLAM）

您可以在本部分末尾列出的论文中找到详尽的细节和比较表。

这种方法的主要缺点是需要巨大的 GPU 内存 - 几乎是其密集型等效模型的十倍。提出了各种蒸馏方法来克服较高的内存要求。

然而，存在直接的权衡。您可以使用少数专家和 2-3 倍更小的基础模型，而不是几十个或几百个专家，从而将模型大小缩小 5 倍，适度提高训练速度，同时适度增加内存需求。

大多数相关论文和实现都围绕 TensorFlow/TPU 构建：

- GShard: 通过条件计算和自动分片扩展巨型模型
- Switch Transformers: 通过简单高效的稀疏性扩展到万亿参数模型
- GLaM: 通用语言模型 (GLaM)

对于 PyTorch，DeepSpeed 也构建了一个：DeepSpeed-MoE: 推动下一代 AI 规模的专家混合推理和训练，专家混合 - 博客文章：1，2 以及特定于大型基于变压器的自然语言生成模型的部署：博客文章，Megatron-DeepSpeed 分支。

使用 PyTorch 原生注意力机制和 Flash Attention

PyTorch 的 `torch.nn.functional.scaled_dot_product_attention`（SDPA） 在底层也可以调用 Flash Attention 和内存高效注意力内核。SDPA 支持目前正被本地添加到 Transformers 中，默认情况下在 `torch>=2.1.1` 时使用，前提是有可用的实现。有关支持的模型列表和更多详细信息，请参阅 PyTorch 缩放点积注意力。

有关使用 SDPA 加速和节省内存的更多信息，请参阅这篇 博客文章。

在多个 GPU 上进行高效训练的方法和工具

如果你发现单个 GPU 训练模型太慢，或者模型的权重无法容纳在单个 GPU 的内存中，那么过渡到多 GPU 配置可能是一个可行的选择。在进行这一转变之前，请务必彻底探索所有在单个 GPU 上提高训练效率的方法（详见单 GPU 高效训练方法和工具），因为这些方法适用于任何数量的 GPU 训练。一旦你应用了这些方法并且发现它们在单个 GPU 上仍然不足，再考虑转向多个 GPU。

从单个 GPU 到多个 GPU 的过渡需要引入某种形式的并行处理，因为工作负载需要分布在资源上。可以采用多种技术来实现并行处理，例如数据并行处理（Data Parallelism）、张量并行处理

（Tensor Parallelism）和流水线并行处理（Pipeline Parallelism）。需要注意的是，并没有一种通用的最佳方案，最佳设置取决于你所使用的具体硬件配置。

本指南深入介绍了各种类型的并行处理，并提供了如何组合技术以及选择合适方法的指导。有关分布式训练的逐步教程，请参阅🤗 Accelerate 文档。

虽然本指南讨论的主要概念可能适用于各种框架，但这里我们重点介绍基于 PyTorch 的实现。

在深入探讨每种技术的具体细节之前，让我们先了解一下在大型基础设施上训练大型模型时的决策过程。

可扩展性策略

首先估算训练模型所需的显存（vRAM）。对于托管在 🤗 Hub 上的模型，可以使用我们的模型内存计算器，该计算器可以提供准确的结果，误差范围在几个百分点内。

单节点/多 GPU 配置的并行化策略

当在一个节点上使用多个 GPU 训练模型时，选择合适的并行化策略对性能有显著影响。以下是你的选项：

情况 1：模型可以容纳在单个 GPU 上

如果模型可以舒适地容纳在一个 GPU 上，你有两个主要选项：

288. DDP - 分布式数据并行（Distributed DataParallel）
289. ZeRO 冗余优化器（ZeRO） - 根据具体情况和配置，这种方法可能更快或更慢，但值得一试。

情况 2：模型无法容纳在单个 GPU 上

如果模型太大无法容纳在一个 GPU 上，你有几种选择：

290. 流水线并行（PipelineParallel，简称 PP）
291. ZeRO
292. 张量并行（TensorParallel，简称 TP）

如果有非常快速的节点间连接（例如 NVLINK 或 NVSwitch），上述三种策略（PP、ZeRO、TP）的性能应该相似。然而，如果没有这些高速连接，PP 的性能会优于 TP 或 ZeRO。TP 的程度也可能有所不同。最好针对你的具体设置进行实验以确定最合适的策略。

TP 几乎总是用于单个节点内，即 TP 大小 ≤ 每个节点的 GPU 数量。

情况 3：模型的最大层无法容纳在单个 GPU 上

293. 如果你不使用 ZeRO，必须使用张量并行（TP），因为仅使用流水线并行（PP）不足以容纳大层。
294. 如果你使用 ZeRO，还需采用单 GPU 高效训练方法和工具中的技术。

多节点/多 GPU 配置的并行化策略

- 当你有快速的节点间连接（例如 NVLINK 或 NVSwitch）时，可以考虑以下选项：
    295. ZeRO - 因为它几乎不需要对模型进行修改。
    296. 流水线并行（PP）与张量并行（TP）和数据并行（DP）的组合 - 这种方法会减少通信次数，但需要对模型进行大量修改。
- 当你有较慢的节点间连接且 GPU 内存较低时：
    297. 使用数据并行（DP）与流水线并行（PP）、张量并行（TP）和 ZeRO 的组合。

在本指南的后续部分，我们将深入探讨这些不同的并行化方法是如何工作的。

数据并行（Data Parallelism）

即使只有 2 个 GPU，你也可以利用 PyTorch 内置功能（如 `DataParallel` (DP) 和 `DistributedDataParallel` (DDP)）提供的加速训练能力。请注意，PyTorch 文档建议在多 GPU 训练时优先使用 `DistributedDataParallel` (DDP) 而不是 `DataParallel` (DP)，因为它适用于所有模型。让我们看看这两种方法的工作原理及其不同之处。

DataParallel 与 DistributedDataParallel

为了理解这两种方法在 GPU 间通信开销上的关键差异，我们来回顾一下每个批次的过程：

DDP：

- 在开始时，主进程从 GPU 0 将模型复制到其他 GPU。
- 然后对于每个批次：

    298. 每个 GPU 直接消耗其小批量数据。

    299. 在 `backward` 阶段，一旦本地梯度准备好，它们会在所有进程中平均。

DP：

对于每个批次：

300. GPU 0 读取一批数据，然后将小批量数据发送给每个 GPU。
301. 最新的模型从 GPU 0 复制到每个 GPU。
302. 执行 `forward`，并将每个 GPU 的输出发送到 GPU 0 以计算损失。
303. 损失从 GPU 0 分发到所有 GPU，然后运行 `backward`。
304. 每个 GPU 的梯度被发送到 GPU 0 并平均。

关键差异包括：

305. DDP 每个批次只进行一次通信 - 发送梯度，而 DP 每个批次进行五次不同的数据交换。

    DDP 使用 torch.distributed 复制数据，而 DP 通过 Python 线程（这会引入与 GIL 相关的限制）在进程内复制数据。因此，除非 GPU 之间的连接速度较慢，否则

    `DistributedDataParallel` (DDP) 通常比 `DataParallel` (DP) 更快。

306. 在 DP 下，GPU 0 的工作量显著大于其他 GPU，导致 GPU 利用率低下。
307. DDP 支持跨多台机器的分布式训练，而 DP 不支持。

这不是 DP 和 DDP 之间差异的详尽列表，但其他细微差别超出了本指南的范围。你可以通过阅读这篇文章来更深入地了解这些方法：使用 PyTorch 在 AWS 上进行分布式数据并行训练。

接下来，我们通过一个实验来说明 DP 和 DDP 之间的差异。我们将基准测试 DP 和 DDP 之间的差异，并添加 NVLink 是否存在的上下文：

- 硬件：2 块 TITAN RTX 24GB 每块 + NVlink，2 条 NVLink（在 `nvidia-smi topo -m` 中显示为 `NV2`）。
- 软件：`pytorch-1.8-to-be` + `cuda-11.0` / `transformers==4.3.0.dev0`。

为了禁用其中一个基准测试的 NVLink 功能，我们使用 `NCCL_P2P_DISABLE=1`。

以下是基准测试代码和输出：

DP

```
rm -r /tmp/test-clm; CUDA_VISIBLE_DEVICES=0,1 \\
python examples/pytorch/language-modeling/run_clm.py \\
--model_name_or_path openai-community/gpt2 --dataset_name wikitext --dataset_config_name
wikitext-2-raw-v1 \\
--do_train --output_dir /tmp/test-clm --per_device_train_batch_size 4 --max_steps 200

{'train_runtime': 110.5948, 'train_samples_per_second': 1.808, 'epoch': 0.69}
```

DDP w/ NVlink

```
rm -r /tmp/test-clm; CUDA_VISIBLE_DEVICES=0,1 \\
torchrun --nproc_per_node 2 examples/pytorch/language-modeling/run_clm.py \\
--model_name_or_path openai-community/gpt2 --dataset_name wikitext --dataset_config_name
wikitext-2-raw-v1 \\
--do_train --output_dir /tmp/test-clm --per_device_train_batch_size 4 --max_steps 200

{'train_runtime': 101.9003, 'train_samples_per_second': 1.963, 'epoch': 0.69}
```

DDP w/o NVlink

```
rm -r /tmp/test-clm; NCCL_P2P_DISABLE=1 CUDA_VISIBLE_DEVICES=0,1 \\
torchrun --nproc_per_node 2 examples/pytorch/language-modeling/run_clm.py \\
--model_name_or_path openai-community/gpt2 --dataset_name wikitext --dataset_config_name
wikitext-2-raw-v1 \\
--do_train --output_dir /tmp/test-clm --per_device_train_batch_size 4 --max_steps 200

{'train_runtime': 131.4367, 'train_samples_per_second': 1.522, 'epoch': 0.69}
```

为了方便起见，我们将上述基准测试结果汇总在下表中：

类型	NVlink	时间
2:DP	Y	110s
2:DDP	Y	101s
2:DDP	N	131s

如你所见，在这种情况下，DP 比带有 NVlink 的 DDP 慢约 10%，但比没有 NVlink 的 DDP 快约 15%。实际差异取决于每个 GPU 需要与其他 GPU 同步的数据量 - 需要同步的数据越多，较慢的连接对整体运行时间的影响就越大。

ZeRO 数据并行（ZeRO Data Parallelism）

由这篇博客文章中的图表展示了 ZeRO 优化的数据并行（ZeRO-DP）。

〔图：DeepSpeed-Image-1〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-zero.png，离线电子书未嵌入，详见原仓库）

尽管看起来很复杂，但它与 `DataParallel` (DP) 的概念非常相似。不同之处在于，ZeRO 不再复制完整的模型参数、梯度和优化器状态，而是每个 GPU 只存储其中的一部分。然后，在运行时需要整个层的参数时，所有 GPU 会同步以互相传递缺失的部分。

为了说明这个概念，假设有一个简单的包含 3 层（La、Lb 和 Lc）的模型，每层有 3 个参数。例如，层 La 有参数 a0、a1 和 a2：

如果我们有 3 个 GPU，ZeRO-DP 会将模型拆分到 3 个 GPU 上，如下所示：

这种方式类似于张量并行（Tensor Parallelism）的水平切片，而不是垂直切片，后者将整个层组放在不同的 GPU 上。现在我们来看看它是如何工作的：

每个 GPU 都会像 DP 一样接收一个小批量数据：

输入数据未经修改，就像它们会被原始模型处理一样。

首先，输入数据到达层 `La`。此时会发生什么？

在 GPU0 上：小批量 x0 需要参数 a0、a1 和 a2 以完成前向传播，但 GPU0 只有 a0。它将从 GPU1 获取 a1，并从 GPU2 获取 a2，从而将模型的所有部分合在一起。

同时，GPU1 接收另一个小批量 x1。GPU1 有参数 a1，但需要 a0 和 a2，所以它从 GPU0 和 GPU2 获取这些参数。同样的，GPU2 接收小批量 x2，并从 GPU0 和 GPU1 获取 a0 和 a1。

这样，每个 GPU 都会重构完整的张量并使用自己的小批量进行前向传播。一旦计算完成，不再需要的数据就会被丢弃 - 它们只在计算过程中使用。重构通过预取高效完成。

然后，整个过程会重复应用于层 Lb，然后是 Lc 的前向传播，再接着是 Lc -> Lb -> La 的反向传播。

这种机制类似于高效的背包策略：A 人携带帐篷，B 人携带炉子，C 人携带斧头。每天晚上，他们会共享自己拥有的物品，并从其他人那里获取自己没有的物品。早上，他们收拾分配给自己的装备并继续前进。这就是 ZeRO DP/Sharded DDP。相比之下，每个人都必须携带自己的帐篷、炉子和斧头（类似于 PyTorch 中的 DataParallel (DP 和 DDP)），这会低效得多。

在阅读相关文献时，你可能会遇到以下同义词：Sharded、Partitioned。如果你仔细观察 ZeRO 如何分割模型的权重 - 它看起来非常类似于张量并行（Tensor Parallelism），这将在后面讨论。这是因为它分割/切片了每一层的权重，而不是垂直模型并行（Vertical Model Parallelism），这将在下一节中讨论。

实现：

- DeepSpeed ZeRO-DP 阶段 1+2+3
- `Accelerate` 集成
- `transformers` 集成

从简单的模型并行到流水线并行

为了解释流水线并行（Pipeline Parallelism），我们先来看看简单的模型并行（Naive Model Parallelism，也称为 Vertical MP）。这种方法涉及将模型层组分布到多个 GPU 上，通过 `.to()` 将特定层分配给特定 GPU。随着数据流经这些层，数据会被移动到与层相同的 GPU 上，而其他层保持不变。

我们称这种模型并行为“垂直”的，因为模型通常是这样可视化的。例如，下图显示了一个 8 层模型垂直切分为两部分，将第 0 到第 3 层放在 GPU0 上，第 4 到第 7 层放在 GPU1 上：

在这个例子中，当数据从第 0 层传到第 3 层时，与常规前向传播没有区别。然而，将数据从第 3 层传到第 4 层需要将数据从 GPU0 移动到 GPU1，这会引入通信开销。如果参与的 GPU 在同一个计算节点上（例如同一台物理机），这种复制很快，但如果 GPU 分布在不同的计算节点上（例如多台机器），通信开销可能会大幅增加。

之后，第 4 到第 7 层的工作方式与原始模型相同。在第 7 层完成后，通常需要将数据发送回第 0 层（或反之，将标签发送到最后一层）。现在可以计算损失，优化器可以进行工作。

简单模型并行存在以下几个缺点：

- **任何时候只有一个 GPU 处于活跃状态**：如果使用 4 个 GPU，这几乎等同于将单个 GPU 的内存容量增加四倍，而忽略其他硬件。
- **设备间的数据传输开销**：例如，4 张 6GB 显卡可以容纳与一张 24GB 显卡相同大小的模型，但单张 24GB 显卡会更快完成训练，因为它没有数据复制开销。但是，如果你有 40GB 的显卡，需要容纳 45GB 的模型，可以使用 4 张 40GB 的显卡（但几乎不可能，因为还要考虑梯度和优化器状态）。
- **共享嵌入层的复制**：共享嵌入层可能需要在 GPU 之间来回复制。

现在你已经熟悉了简单的模型并行如何工作及其缺点，我们来看流水线并行（Pipeline Parallelism，简称 PP）。PP 与简单的模型并行几乎相同，但它通过将传入批次切分为微批次并人工创建流水线，解决了 GPU 闲置问题，允许不同的 GPU 并行参与计算过程。

下图来自 GPipe 论文，展示了顶部的简单模型并行和底部的流水线并行：

〔图：MP vs PP〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-gpipe-bubble.png，离线电子书未嵌入，详见原仓库）

在图的底部，可以看到流水线并行（PP）方法最小化了被称为“气泡”的空闲 GPU 区域。两个部分都显示了并行度为 4，这意味着有 4 个 GPU 参与流水线。你可以看到，有一个前向路径的 4 个管道阶段（F0、F1、F2 和 F3），然后是反向路径按逆序（B3、B2、B1 和 B0）。

PP 引入了一个新的超参数来调整 - `chunks`，这决定了通过同一管道阶段发送的数据块的数量。例如，在底部的图中，你可以看到 `chunks=4`。GPU0 对数据块 0、1、2 和 3 进行相同的前向传播（F0,0、F0,1、F0,2、F0,3），然后等待其他 GPU 完成工作。只有当其他 GPU 开始完成它们的工作时，GPU0 才会再次开始工作，对数据块 3、2、1 和 0 进行反向传播（B0,3、B0,2、B0,1、B0,0）。

注意，这与梯度累积步骤的概念相同。PyTorch 使用 `chunks`，而 DeepSpeed 称之为梯度累积步骤。

由于 `chunks`，PP 引入了微批次（Micro-Batches，简称 MBS）的概念。DP 将全局数据批次大小切分为小批次，所以如果你的 DP 程度为 4，全局批次大小为 1024，将会被切分为 4 个小批次，每个小批次大小为 256（1024/4）。如果 `chunks`（或 GAS）为 32，我们最终得到的微批次大小为 8

（256/32）。每个管道阶段一次处理一个微批次。要计算 DP + PP 设置的全局批次大小，使用公式：`mbs * chunks * dp_degree`（8 *32* 4 = 1024）。当 `chunks=1` 时，你会得到简单的 MP，这是低效的。当 `chunks` 值很大时，你会得到非常小的微批次，这也是低效的。因此，我们鼓励你尝试不同的 `chunks` 值，以找到最高效的 GPU 利用率。

你可能会注意到图中的一个“死区”，这部分不能并行化，因为最后一个 `forward` 阶段必须等待 `backward` 完成流水线。寻找最佳 `chunks` 值的目的是实现在所有参与的 GPU 上的高并发利用率，从而最小化“气泡”的大小。

流水线 API 解决方案已实现在：

- PyTorch
- DeepSpeed
- Megatron-LM

这些解决方案存在一些缺点：

- 它们需要对模型进行大量修改，因为流水线要求将正常模块流程重写为 `nn.Sequential` 序列，这可能需要对模型设计进行更改。
- 当前的流水线 API 非常受限。如果你在流水线的第一阶段传递了一组 Python 变量，你必须找到变通方法。目前，流水线接口要求输入和输出只能是一个 Tensor 或一组 Tensors。这些 Tensors 必须在第一个维度上有批次大小，因为流水线会将小批次切分为微批次。相关改进正在讨论中：https://github.com/pytorch/pytorch/pull/50693
- 无法在管道阶段的条件控制流 - 例如，T5 编码器-解码器模型需要特殊的工作 around 来处理条件编码阶段。
- 它们需要安排每个层，使得一个层的输出成为另一个层的输入。

更近期的解决方案包括：

- Varuna
- Sagemaker

我们还没有尝试过 Varuna 和 SageMaker，但它们的论文报告称，它们克服了上述问题，并且需要对用户模型进行较小的修改。

实现：

- PyTorch（初始支持在 pytorch-1.8，1.9 和 1.10 版本中逐步改进）。一些 示例
- DeepSpeed
- Megatron-LM 有一个内部实现 - 没有 API。
- Varuna
- SageMaker - 这是一个专有解决方案，只能在 AWS 上使用。
- OSLO - 这是基于 Hugging Face Transformers 实现的。

😭 Transformers 状态：截至本文写作时，没有任何模型支持全 PP。GPT2 和 T5 模型支持简单的 MP。主要障碍是无法将模型转换为 `nn.Sequential` 并确保所有输入都是 Tensors。这是因为在当前的模型中包含了许多特性，使得转换非常复杂，需要移除这些特性才能实现。

DeepSpeed 和 Megatron-LM 集成可用在 🤗 Accelerate 中。

其他方法：

DeepSpeed、Varuna 和 SageMaker 使用了 交错流水线 的概念。

〔图：交错流水线执行〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-sagemaker-interleaved-pipeline.png，离线电子书未嵌入，详见原仓库）

这里通过优先进行反向传播进一步最小化了“气泡”（空闲时间）。Varuna 进一步尝试通过模拟来发现最有效的调度。

OSLO 采用了基于 Transformers 的流水线并行实现，无需进行 `nn.Sequential` 转换。

张量并行（Tensor Parallelism）

在张量并行（Tensor Parallelism）中，每个 GPU 处理张量的一个切片，并在需要时聚合完整的张量。为了描述这种方法，本指南的这一部分依赖于 Megatron-LM 论文 Efficient Large-Scale Language Model Training on GPU Clusters 中的概念和图表。

任何变换器的核心组成部分是一个全连接的 `nn.Linear` 层，后跟一个非线性激活函数 `GeLU`。根据 Megatron 论文的记号，它的点积部分可以表示为 `Y = GeLU(XA)`，其中 `X` 是输入向量，`Y` 是输出向量，`A` 是权重矩阵。

如果我们以矩阵形式查看计算过程，可以看到矩阵乘法可以如何在多个 GPU 之间拆分：

〔图：并行 GEMM〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-tp-parallel_gemm.png，离线电子书未嵌入，详见原仓库）

如果我们按列拆分权重矩阵 `A` 并在 `N` 个 GPU 上并行执行矩阵乘法 `XA_1` 到 `XA_n`，我们将得到 `N` 个输出向量 `Y_1，Y_2，...，Y_n`，这些向量可以独立地输入 `GeLU` 函数：

〔图：独立 GeLU〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-tp-independent-gelu.png，离线电子书未嵌入，详见原仓库）

基于这一原则，我们可以更新任意深度的多层感知器（MLP），而无需在 GPU 之间进行同步，直到最后一步，需要从分片中重建输出向量。Megatron-LM 论文的作者提供了一个有用的插图：

〔图：并行分片处理〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-tp-parallel_shard_processing.png，离线电子书未嵌入，详见原仓库）

多头自注意力层的并行化更简单，因为它们本身就是并行的，具有多个独立的头！

〔图：并行自注意力〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-tp-parallel_self_attention.png，离线电子书未嵌入，详见原仓库）

特别注意事项：TP 需要非常快的网络，因此不建议跨多个节点进行 TP。实际上，如果一个节点有 4 个 GPU，则最高 TP 程度为 4。如果你需要 TP 程度为 8，需要使用至少有 8 个 GPU 的节点。

本节基于 @anton-l 的 详细 TP 概述。

其他名称：

- DeepSpeed 称之为 张量切片

实现：

- Megatron-LM 有一个内部实现，因为它非常特定于模型。
- parallelformers（目前仅支持推理）
- SageMaker - 这是一个专有解决方案，只能在 AWS 上使用。
- OSLO 有一个基于 Transformers 的张量并行实现。

SageMaker 结合了 TP 和 DP 以实现更高效的处理。

😭 Transformers 状态：

- 核心部分：尚未实现在核心部分
- 但如果你需要推理，parallelformers 为大多数模型提供了支持。因此，你可以在核心实现之前使用它。希望训练模式也会得到支持。
- Deepspeed-Inference 还支持我们的 BERT、GPT-2 和 GPT-Neo 模型的超快速 CUDA 内核推理模式，更多详情请参阅 这里

😭 Accelerate 集成了 Megatron-LM 的 TP。

数据并行 + 流水线并行

下图来自 DeepSpeed 流水线教程，展示了如何结合 DP 和 PP。

〔图：DP + PP-2d〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-zero-dp-pp.png，离线电子书未嵌入，详见原仓库）

这里重要的是看到 DP 排名 0 不能看到 GPU2，而 DP 排名 1 不能看到 GPU3。对于 DP 来说，只有 GPU0 和 GPU1，就像只有 2 个 GPU 一样，数据被送入这两个 GPU。GPU0 会“秘密地”将部分负载卸载到 GPU2，使用 PP。同样，GPU1 也会利用 GPU3 的帮助。

由于每个维度至少需要 2 个 GPU，这里你需要至少 4 个 GPU。

实现：

- DeepSpeed
- Megatron-LM
- Varuna
- SageMaker
- OSLO

😭 Transformers 状态：尚未实现在核心部分

数据并行 + 流水线并行 + 张量并行

为了实现更高效的训练，3D 并行化将 PP 与 TP 和 DP 结合起来。这可以从下图中看出。

〔图：dp-pp-tp-3d〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-deepspeed-3d.png，离线电子书未嵌入，详见原仓库）

此图来自一篇博客文章 3D 并行化：扩展到万亿参数模型，这篇文章也值得阅读。

由于每个维度至少需要 2 个 GPU，这里你需要至少 8 个 GPU。

实现：

- DeepSpeed - DeepSpeed 还包括一个更高效的 DP，称为 ZeRO-DP。
- Megatron-LM
- Varuna
- SageMaker
- OSLO

😫 Transformers 状态：尚未实现在核心部分，因为我们没有 PP 和 TP。

ZeRO 数据并行 + 流水线并行 + 张量并行

DeepSpeed 的一个主要特点是 ZeRO，这是一个超级可扩展的 DP 扩展。已经在 ZeRO 数据并行部分讨论过。通常它是一个独立的功能，不需要 PP 或 TP。但可以与 PP 和 TP 结合使用。

当 ZeRO-DP 与 PP（可选 TP）结合时，通常只启用 ZeRO 第 1 阶段（优化器切片）。

理论上可以在流水线并行中使用 ZeRO 第 2 阶段（梯度切片），但这会对性能产生负面影响。每个微批次都需要一个额外的 reduce-scatter 集体操作来聚合梯度，然后再切片，这会增加潜在的通信开销。由于流水线并行的性质，使用较小的微批次，重点在于平衡算术强度（微批次大小）与最小化流水线气泡（微批次数量）。因此，这些通信成本会影响性能。

此外，由于 PP 已经减少了层数，因此内存节省不会很大。PP 已经将梯度大小减少了 `1/PP`，因此在纯 DP 上的梯度切片节省不太显著。

ZeRO 第 3 阶段也不是一个好的选择，原因相同 - 需要更多的节点间通信。

由于我们有 ZeRO，另一个好处是 ZeRO-Offload。在第 1 阶段，优化器状态可以卸载到 CPU。

实现：

- Megatron-DeepSpeed 和 BigScience 的 Megatron-DeepSpeed，这是前者的分支。
- OSLO

重要论文：

- 使用 DeepSpeed 和 Megatron 训练 Megatron-Turing NLG 530B，一个大规模生成语言模型

😭 Transformers 状态：尚未实现在核心部分，因为我们没有 PP 和 TP。

FlexFlow

FlexFlow 以稍微不同的方法解决了并行化问题。

论文：“超越深度神经网络的数据并行和模型并行” by Zhihao Jia, Matei Zaharia, Alex Aiken

它在样本、算子、属性和参数四个维度上进行 4D 并行化。

308. 样本 = 数据并行（样本并行）
309. 算子 = 将单个操作并行化为多个子操作
310. 属性 = 数据并行（长度并行）
311. 参数 = 模型并行（无论维度 - 水平或垂直）

示例：

- 样本

假设我们有 10 批序列长度为 512。如果在样本维度上并行化为 2 个设备，我们会得到 10 x 512，变为 5 x 2 x 512。

- 算子

如果执行层归一化，我们先计算标准差，再计算均值，然后进行数据归一化。算子并行化允许并行计算标准差和均值。因此，如果我们在算子维度上并行化为 2 个设备（cuda:0, cuda:1），首先将输入数据复制到两个设备，cuda:0 计算标准差，cuda:1 同时计算均值。

- 属性

我们有 10 批长度为 512 的数据。如果在属性维度上并行化为 2 个设备，10 x 512 会变成 10 x 2 x 256。

- 参数

类似于张量模型并行或逐层模型并行。

〔图：flex-flow-soap〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/parallelism-flexflow.jpeg，离线电子书未嵌入，详见原仓库）

该框架的重要意义在于，它会根据资源（1）GPU/TPU/CPU vs. （2）RAM/DRAM vs. （3）快速内部连接/缓慢外部连接，自动优化算法决定在哪里使用哪种并行化。

非常重要的一点是，FlexFlow 适用于静态和固定工作负载的模型并行化优化，因为动态行为的模型可能在不同迭代中需要不同的并行化策略。

因此，该框架的前景非常诱人 - 它可以在选定的集群上运行 30 分钟的模拟，然后提出最佳策略以利用该特定环境。如果你添加、移除或替换任何部分，它会重新优化计划。然后你可以进行训练。不同的设置会有自己的定制优化。

🤗 Transformers 状态：Transformers 模型可以通过 transformers.utils.fx 进行 FX 跟踪，这是 FlexFlow 的先决条件，但需要在 FlexFlow 方面进行更改以使其与 Transformers 模型兼容。

GPU 选择

在多个 GPU 上训练时，你可以指定使用的 GPU 数量和顺序。这在你有不同计算能力的 GPU 且希望先使用较快的 GPU 时非常有用。选择过程适用于 DistributedDataParallel 和 DataParallel，使用它们只需要部分可用的 GPU，而不需要 Accelerate 或 DeepSpeed 集成。

GPU 数量

例如，如果你有 4 个 GPU 且只想使用前 2 个：

torchrun

Accelerate

DeepSpeed

使用 `--nproc_per_node` 选择要使用的 GPU 数量。

```
torchrun --nproc_per_node=2  trainer-program.py ...
```

GPU 顺序

现在，要选择要使用的 GPU 及其顺序，你将使用 `CUDA_VISIBLE_DEVICES` 环境变量。最简单的方法是在 `~/bashrc` 或其他启动配置文件中设置环境变量。`CUDA_VISIBLE_DEVICES` 用于映射哪些 GPU 被使用。例如，如果你有 4 个 GPU（0、1、2、3）且只想要运行 GPU 0 和 2：

```
CUDA_VISIBLE_DEVICES=0,2 torchrun trainer-program.py ...
```

只有 2 个物理 GPU（0 和 2）对 PyTorch 是“可见”的，它们分别被映射到 `cuda:0` 和 `cuda:1`。你也可以反转 GPU 的顺序，先使用 GPU 2。现在，映射关系是 `cuda:1` 对应 GPU 0，`cuda:0` 对应 GPU 2。

```
CUDA_VISIBLE_DEVICES=2,0 torchrun trainer-program.py ...
```

你还可以将 `CUDA_VISIBLE_DEVICES` 环境变量设置为空值，以创建一个没有 GPU 的环境。

```
CUDA_VISIBLE_DEVICES= python trainer-program.py ...
```

像任何环境变量一样，它们可以导出而不是添加到命令行中。然而，这不推荐，因为如果你忘记环境变量是如何设置的，可能会使用错误的 GPU。相反，常见做法是在特定训练运行的同一命令行中设置环境变量。

`CUDA_DEVICE_ORDER` 是另一个可以用来控制 GPU 顺序的环境变量。你可以按照以下方式排序：

312. 使用 PCIe 总线 ID，这与 NVIDIA 和 AMD GPU 的 `nvidia-smi` 和 `rocm-smi` 的顺序匹配：

```
export CUDA_DEVICE_ORDER=PCI_BUS_ID
```

313. 按 GPU 计算能力排序：

```
export CUDA_DEVICE_ORDER=FASTEST_FIRST
```

`CUDA_DEVICE_ORDER` 尤其有用，如果你的训练设置包含旧 GPU 和新 GPU，且旧 GPU 优先出现，但你无法物理更换显卡以使新 GPU 优先出现。在这种情况下，设置 `CUDA_DEVICE_ORDER=FASTEST_FIRST` 以始终优先使用更新和更快的 GPU（`nvidia-smi` 或 `rocm-smi` 仍然按 PCIe 顺序报告 GPU）。或者你也可以设置 `export CUDA_VISIBLE_DEVICES=1,0`。

FSDP —— 完全分片数据并行方法

完全分片数据并行 (FSDP) 是一种数据并行方法，它将模型的参数、梯度和优化器状态分布在可用的 GPU（也称为工作进程或 _rank_）上。与 DistributedDataParallel (DDP) 不同，FSDP 通过在

每个 GPU 上复制模型来减少内存使用。这提高了 GPU 的内存效率，并允许你在更少的 GPU 上训练更大的模型。FSDP 集成了 Accelerate 库，这是一个用于轻松管理分布式环境中的训练的库，这意味着你可以在 Trainer 类中使用它。

在开始之前，请确保已安装 Accelerate 并且至少使用 PyTorch 2.1.0 或更新版本。

```
pip install accelerate
```

FSDP 配置

首先，运行以下命令以创建一个配置文件，用于设置你的训练环境。Accelerate 使用这个配置文件根据你在 `accelerate config` 中选择的训练选项自动设置正确的训练环境。

```
accelerate config
```

当你运行 `accelerate config` 时，系统会提示你选择一系列选项来配置你的训练环境。本节介绍一些最重要的 FSDP 选项。要了解其他可用的 FSDP 选项，请查看 fsdp_config 参数。

分片策略

FSDP 提供了多种分片策略供你选择：

- `FULL_SHARD` - 在工作进程中分片模型参数、梯度和优化器状态；选择 `1` 表示此选项
- `SHARD_GRAD_OP` - 在工作进程中分片梯度和优化器状态；选择 `2` 表示此选项
- `NO_SHARD` - 不分片任何内容（相当于 DDP）；选择 `3` 表示此选项
- `HYBRID_SHARD` - 在每个工作进程中分片模型参数、梯度和优化器状态，每个工作进程也有完整副本；选择 `4` 表示此选项
- `HYBRID_SHARD_ZERO2` - 在每个工作进程中分片梯度和优化器状态，每个工作进程也有完整副本；选择 `5` 表示此选项

这是通过 `fsdp_sharding_strategy` 标志启用的。

CPU 卸载

你还可以在不使用参数和梯度时将其卸载到 CPU，以节省更多 GPU 内存，帮助你适应大模型，即使 FSDP 也不足以支持。这是通过在运行 `accelerate config` 时设置 `fsdp_offload_params: true` 来启用的。

包装策略

FSDP 通过包装网络中的每一层来应用。通常情况下，包装是以嵌套方式应用的，每次前向传递后丢弃完整的权重，以便为下一层节省内存。最简单的实现方法是使用 _自动包装_ 策略，你不需要更改任何代码。你应该选择 `fsdp_auto_wrap_policy: TRANSFORMER_BASED_WRAP` 以包装 Transformer 层，并选择 `fsdp_transformer_layer_cls_to_wrap` 以指定要包装的层（例如 `BertLayer`）。

否则，你可以选择基于大小的包装策略，当某层的参数数量超过一定阈值时，FSDP 将应用于该层。这是通过设置 `fsdp_wrap_policy: SIZE_BASED_WRAP` 和 `min_num_param` 为所需的大小阈值来启用的。

检查点

应使用 `fsdp_state_dict_type: SHARDED_STATE_DICT` 保存中间检查点，因为当 CPU 卸载在 rank 0 上启用时，保存完整的状态字典会耗费大量时间，并且经常由于无限期挂起导致 `NCCL Timeout` 错误。你可以使用 load_state 方法从分片状态字典中恢复训练。

```
# 包含检查点的目录
accelerator.load_state("ckpt")
```

然而，当训练结束时，你希望保存完整的状态字典，因为分片状态字典仅适用于 FSDP。

```
if trainer.is_fsdp_enabled:
    trainer.accelerator.state.fsdp_plugin.set_state_dict_type("FULL_STATE_DICT")

trainer.save_model(script_args.output_dir)
```

TPU

PyTorch XLA 支持 TPU 的 FSDP 训练，可以通过修改 `accelerate config` 生成的 FSDP 配置文件来启用。除了上面指定的分片策略和包装选项，你还可以在文件中添加以下参数。

`xla_fsdp_settings` 允许你为 FSDP 配置更多的 XLA 特定参数。

启动训练

一个示例 FSDP 配置文件可能如下所示：

要启动训练，运行以下命令，它将自动使用你之前使用 `accelerate config` 创建的配置文件。

```
accelerate launch my-trainer-script.py
```

你也可以显式指定 FSDP 配置：

```
accelerate launch --fsdp="full shard" --fsdp_config="path/to/fsdp_config/" my-trainer-script.py
```

下一步

FSDP 是训练大规模模型的强大工具，特别是在你有多个 GPU 或 TPU 的情况下。通过分片模型参数、优化器和梯度状态，甚至在不活跃时将它们卸载到 CPU，FSDP 可以降低大规模训练的高成本。如果你希望了解更多，以下资源可能会有所帮助：

- 跟随 Accelerate 的 FSDP 详细指南。
- 阅读 介绍 PyTorch 完全分片数据并行 (FSDP) API 博客文章。
- 阅读 使用 FSDP 在云 TPU 上扩展 PyTorch 模型 博客文章。

DeepSpeed —— 分布式高效训练 PyTorch 模型优化库

DeepSpeed 是一个 PyTorch 优化库，它可以让你在分布式环境中高效地训练大型模型。DeepSpeed 的核心功能是 ZeRO Redundancy Optimizer (ZeRO)，它能够实现在大规模上训练大型模型。ZeRO 分为几个阶段：

- **ZeRO-1**：将优化器状态分布在多个 GPU 上
- **ZeRO-2**：将梯度分布在多个 GPU 上
- **ZeRO-3**：将参数分布在多个 GPU 上

在 GPU 资源有限的环境下，ZeRO 还可以将优化器的内存和计算从 GPU 卸载到 CPU，以便在单个 GPU 上训练非常大的模型。DeepSpeed 已经与 Transformers 的 Trainer 类集成，支持所有 ZeRO 阶段和卸载功能。你只需要提供一个配置文件，或者使用提供的模板。对于推理，Transformers 支持 ZeRO-3 和卸载，因为这允许加载巨大的模型。

本指南将引导你如何部署 DeepSpeed 训练，介绍你可以启用的功能，如何为不同的 ZeRO 阶段和卸载设置配置文件，以及如何在不使用 Trainer 的情况下使用 DeepSpeed。

安装

DeepSpeed 可以通过 PyPI 或 Transformers 安装。更多详细的安装选项，请参阅 DeepSpeed 的安装详情 或 GitHub 的 README。

如果你在安装 DeepSpeed 时遇到困难，可以查看 DeepSpeed CUDA 安装 指南。尽管 DeepSpeed 有一个可通过 pip 安装的 PyPI 包，但强烈建议 从源码安装，以更好地匹配你的硬件并支持某些特性（如 1-bit Adam），这些特性在 PyPI 分发中不可用。

通过 PyPI 安装

```
pip install deepspeed
```

通过 Transformers 安装

```
pip install transformers[deepspeed]
```

内存需求

在开始之前，最好检查一下你的 GPU 和 CPU 是否有足够的内存来容纳你的模型。DeepSpeed 提供了一个工具来估算所需的 CPU/GPU 内存。例如，要估计 bigscience/T0_3B 模型在一个 GPU 上的内存需求：

```
python -c 'from transformers import AutoModel; \
from deepspeed.runtime.zero.stage3 import estimate_zero3_model_states_mem_needs_all_live; \
model = AutoModel.from_pretrained("bigscience/T0_3B"); \
estimate_zero3_model_states_mem_needs_all_live(model, num_gpus_per_node=1, num_nodes=1)'
```

输出可能如下所示：

这意味着你需要一个 80GB 的 GPU（不使用 CPU 卸载）或一个 8GB 的 GPU 和大约 60GB 的 CPU 内存来进行卸载（这些只是参数、优化器状态和梯度的内存需求，你还需要一些额外的内存用于 CUDA 内核和激活）。你还应该考虑成本和速度之间的权衡，因为租用或购买较小的 GPU 会更便宜，但训练模型的时间会更长。

如果 GPU 内存足够，确保禁用 CPU/NVMe 卸载以加快速度。

选择 ZeRO 阶段

安装了 DeepSpeed 并了解了内存需求后，下一步是选择一个 ZeRO 阶段。按速度和内存效率排序：

最快	内存最省
ZeRO-1	ZeRO-3 + 卸载
ZeRO-2	ZeRO-3
ZeRO-2 + 卸载	ZeRO-2 + 卸载
ZeRO-3	ZeRO-2
ZeRO-3 + 卸载	ZeRO-1

为了找到最适合你的方案，可以从最快的方式开始尝试，如果内存不足，再尝试下一级别，虽然速度会稍慢但内存效率更高。你可以根据自己的偏好从最节省内存的方案开始尝试，直到找到速度和内存使用的最佳平衡点。

一个通用的过程是（从批量大小为 1 开始）：

314. 启用梯度检查点
315. 尝试 ZeRO-2
316. 尝试 ZeRO-2 并卸载优化器
317. 尝试 ZeRO-3
318. 尝试 ZeRO-3 并将参数卸载到 CPU
319. 尝试 ZeRO-3 并将参数和优化器都卸载到 CPU
320. 尝试降低默认值，如在使用 generate() 方法时使用更窄的搜索束
321. 尝试混合精度（在较旧的 GPU 架构上使用 fp16，在 Ampere GPU 上使用 bf16）
322. 如果可能，添加更多硬件或启用 Infinity 以将参数和优化器卸载到 NVMe
323. 一旦不再出现内存不足，测量有效吞吐量，然后尝试尽可能增加批量大小以最大化 GPU 效率
324. 最后，尝试优化训练配置，禁用某些卸载功能或使用更快的 ZeRO 阶段，并调整批量大小以找到速度和内存使用之间的最佳权衡

DeepSpeed 配置文件

DeepSpeed 通过包含所有配置参数的配置文件与 Trainer 类配合使用。当你执行训练脚本时，DeepSpeed 会将从 Trainer 接收到的配置日志记录到控制台，以便你可以看到确切的配置。

完整的 DeepSpeed 配置选项列表可以在 DeepSpeed 配置 JSON 参考中找到。你还可以在 DeepSpeedExamples 存储库或主 DeepSpeed 存储库中找到更多实用的 DeepSpeed 配置示例。要快速找到特定示例，可以：

```
git clone https://github.com/microsoft/DeepSpeedExamples
cd DeepSpeedExamples
find . -name '*json'
# 查找使用 Lamb 优化器的示例
grep -i Lamb $(find . -name '*json')
```

DeepSpeed 配置文件可以通过命令行接口传递为 JSON 文件的路径，也可以在笔记本环境中作为嵌套的 `dict` 对象传递。

配置文件路径

嵌套字典

DeepSpeed 和 Trainer 参数

有三种类型的配置参数：

325. **共享参数**：有些配置参数由 Trainer 和 DeepSpeed 共享，当定义冲突时识别错误可能会很困难。为了简化问题，这些共享配置参数应从 Trainer 命令行参数中配置。
326. **自动派生参数**：有些配置参数是从模型配置中自动派生的，因此你不需要手动调整这些值。Trainer 使用配置值 `auto` 来确定最正确或高效的值。你可以显式设置自己的配置参数，但必须确保 Trainer 参数和 DeepSpeed 配置参数一致。不一致可能导致训练失败，而且很难检测到！
327. **仅限 DeepSpeed 的参数**：有些配置参数仅适用于 DeepSpeed，需要根据你的训练需求手动设置。

你还可以修改 DeepSpeed 配置并编辑 TrainingArguments：

328. 创建或加载一个 DeepSpeed 配置作为主要配置

329. 基于这些 DeepSpeed 配置值创建一个 TrainingArguments 对象

一些值，如 `scheduler.params.total_num_steps`，是在训练过程中由 Trainer 计算的。

ZeRO 配置

有三个配置，每个对应一个不同的 ZeRO 阶段。阶段 1 对于可扩展性来说并不是很有吸引力，本指南关注的是阶段 2 和 3。`zero_optimization` 配置包含了所有启用和配置的选项。有关每个参数的详细说明，请参阅 DeepSpeed 配置 JSON 参考。

DeepSpeed 不会验证参数名称，任何拼写错误都会回退到参数的默认设置。你可以观看 DeepSpeed 引擎启动日志消息，以了解它将使用哪些值。

以下配置必须通过 DeepSpeed 设置，因为 Trainer 没有提供相应的命令行参数。

ZeRO-1

ZeRO-1 将优化器状态分片分布在多个 GPU 上，你可以期望有轻微的速度提升。ZeRO-1 配置可以这样设置：

NVMe 配置

ZeRO-Infinity 允许将模型状态卸载到 CPU 和/或 NVMe 以节省更多内存。智能分区和切片算法允许每个 GPU 在卸载期间发送和接收非常少量的数据，使得现代 NVMe 可以提供比你可用的训练进程更大的总内存池。ZeRO-Infinity 需要 ZeRO-3。

根据可用的 CPU 和/或 NVMe 内存，你可以卸载 优化器状态 和 参数，只卸载其中一个，或者都不卸载。你还需要确保 `nvme_path` 指向 NVMe 设备，因为尽管它仍然可以使用普通硬盘或固态硬盘，但速度会显著变慢。使用现代 NVMe，你可以期望读取峰值速度约为 3.5GB/s，写入峰值速度约为 3GB/s。最后，运行基准测试 以确定你的训练设置的最佳 `aio` 配置。

以下示例 ZeRO-3/Infinity 配置文件将大多数参数值设置为 `auto`，但你也可以手动添加这些值。

DeepSpeed 功能

在 DeepSpeed 配置文件中指定许多重要的参数，本节简要介绍了这些参数。

激活/梯度检查点

激活和梯度检查点交换了速度以换取更多的 GPU 内存，这可以帮助你在 GPU 内存不足或增加批量大小以提高性能的情况下解决问题。要启用此功能：

330. 对于 Hugging Face 模型，设置 `model.gradient_checkpointing_enable()` 或在 Trainer 中使用 `--gradient_checkpointing`。
331. 对于非 Hugging Face 模型，使用 DeepSpeed 的 激活检查点 API。你还可以替换 Transformers 的建模代码，将 `torch.utils.checkpoint` 替换为 DeepSpeed API。这种方法更灵活，因为你可以将前向激活卸载到 CPU 内存而不是重新计算它们。

优化器和调度器

DeepSpeed 和 Transformers 的优化器和调度器可以混合使用，只要你没有启用 `offload_optimizer`。当 `offload_optimizer` 被启用时，你可以使用非 DeepSpeed 优化器（除了 LAMB），但前提是它同时具有 CPU 和 GPU 实现。

配置文件中的优化器和调度器参数可以从命令行设置，以避免难以发现的错误。例如，如果学习率在其他地方设置了不同的值，你可以从命令行覆盖它。除了优化器和调度器参数外，你还需要确保 Trainer 命令行参数与 DeepSpeed 配置一致。

优化器

DeepSpeed 提供了几种 优化器（Adam, AdamW, OneBitAdam, 和 LAMB），但你也可以从 PyTorch 中导入其他优化器。如果你没有在配置中配置优化器，Trainer 会自动选择 AdamW，并使用命令行提供的值或默认值设置以下参数：`lr`，`adam_beta1`，`adam_beta2`，`adam_epsilon`，`weight_decay`。

你可以将参数设置为 `"auto"` 或手动输入你希望的值。

你还可以使用不受支持的优化器，只需在顶级配置中添加以下内容：

从 DeepSpeed==0.8.3 开始，如果你想使用卸载，还需要在顶级配置中添加以下内容，因为卸载与 DeepSpeed 的 CPU Adam 优化器配合效果最佳。

精度

Deepspeed 支持 fp32、fp16 和 bf16 混合精度。

- **fp32**：全精度浮点数
- **fp16**：半精度浮点数
- **bf16**：脑浮点数（BFloat16）

如果模型不支持混合精度，例如没有在混合精度下预训练，你可能会遇到溢出或下溢问题，导致损失为 NaN。在这种情况下，你应该显式禁用默认的 fp16 模式，使用全 fp32 精度。

对于 Ampere GPU 和 PyTorch > 1.7，它会自动将某些操作切换为更高效的 tf32 格式，但结果仍然是 fp32。你可以在 Trainer 中通过设置 `--tf32` 来启用它，通过 `--tf32 0` 或 `--no_tf32` 来禁用它。

批量大小

批量大小可以自动配置或显式设置。如果你选择使用 `"auto"` 选项，Trainer 会将 `train_micro_batch_size_per_gpu` 设置为 `args.per_device_train_batch_size` 的值，将 `train_batch_size` 设置为 `args.world_size * args.per_device_train_batch_size * args.gradient_accumulation_steps`。

梯度累积

梯度累积可以自动配置或显式设置。如果你选择使用 `"auto"` 选项，Trainer 会将其设置为 `args.gradient_accumulation_steps` 的值。

梯度裁剪

梯度裁剪可以自动配置或显式设置。如果你选择使用 `"auto"` 选项，Trainer 会将其设置为 `args.max_grad_norm` 的值。

通信数据类型

对于诸如归约、聚集和散射等通信集体操作，使用不同的数据类型。

所有聚集和散射操作都在相同的数据类型下进行。例如，如果你使用 bf16 训练，数据也会以 bf16 形式聚集，因为聚集是非损耗操作。

归约操作是损耗的，例如当梯度在多个 GPU 上平均时。当通信以 fp16 或 bf16 进行时，更容易发生损耗，因为在低精度下加法运算不是精确的。特别是 bf16，其精度低于 fp16。因此，默认情况下，归约操作使用 fp16，因为在平均梯度时损失最小。

你可以在配置文件中通过设置 `communication_data_type` 参数来选择通信数据类型。例如，选择 fp32 会增加少量开销，但确保归约操作在 fp32 下累积，当准备就绪时，再降精度到你正在训练的半精度数据类型。

部署

DeepSpeed 可以通过不同的启动器部署，如 torchrun、`deepspeed` 启动器或 Accelerate。要部署，可以在 Trainer 命令行中添加 `--deepspeed ds_config.json`。推荐使用 DeepSpeed 的 `add_config_arguments` 实用程序来添加任何必要的命令行参数到你的代码中。

本指南将展示如何使用 `deepspeed` 启动器在不同的训练设置中部署 DeepSpeed。你可以查看这个帖子 获取更多实际使用示例。

多 GPU 部署

要在多个 GPU 上部署 DeepSpeed，添加 `--num_gpus` 参数。如果你想要使用所有可用的 GPU，不需要添加 `--num_gpus`。以下示例使用 2 个 GPU。

```
deepspeed --num_gpus=2 examples/pytorch/translation/run_translation.py \
--deepspeed tests/deepspeed/ds_config_zero3.json \
--model_name_or_path google-t5/t5-small --per_device_train_batch_size 1 \
--output_dir output_dir --overwrite_output_dir --fp16 \
--do_train --max_train_samples 500 --num_train_epochs 1 \
--dataset_name wmt16 --dataset_config "ro-en" \
--source_lang en --target_lang ro
```

多节点部署

一个节点是用于运行工作负载的一个或多个 GPU。更强大的设置是多节点设置，可以使用 `deepspeed` 启动器启动。假设有两个节点，每个节点有 8 个 GPU。第一个节点可以通过 `ssh`

`hostname1` 访问，第二个节点可以通过 `ssh hostname2` 访问。两个节点必须能够在本地通过 ssh 无密码通信。

默认情况下，DeepSpeed 期望你的多节点环境使用共享存储。如果这不是情况，每个节点只能看到本地文件系统，你需要调整配置文件，包括一个 `checkpoint`，以允许在没有共享文件系统访问的情况下加载：

你也可以使用 Trainer 的 `--save_on_each_node` 参数自动将上述 `checkpoint` 添加到你的配置中。

使用 torchrun

对于 torchrun，你需要通过 ssh 登录到每个节点，并在两个节点上运行以下命令。启动器会等待两个节点同步后再启动训练。

```
torchrun --nproc_per_node=8 --nnode=2 --node_rank=0 --master_addr=hostname1 \
--master_port=9901 your_program.py <normal cl args> --deepspeed ds_config.json
```

使用 SLURM

在 SLURM 环境中，你需要根据你的具体 SLURM 环境调整 SLURM 脚本。一个示例 SLURM 脚本可能如下所示：

```
#SBATCH --job-name=test-nodes        # job name
#SBATCH --nodes=2                    # number of nodes
#SBATCH --ntasks-per-node=1          # crucial - only 1 task per dist per node!
#SBATCH --cpus-per-task=10           # number of cores per task
#SBATCH --gres=gpu:8                 # number of GPUs
#SBATCH --time 20:00:00              # maximum execution time (HH:MM:SS)
#SBATCH --output=%x-%j.out           # output file name

export GPUS_PER_NODE=8
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=9901

srun --jobid $SLURM_JOBID bash -c 'python -m torch.distributed.run \
 --nproc_per_node $GPUS_PER_NODE --nnodes $SLURM_NNODES --node_rank $SLURM_PROCID \
 --master_addr $MASTER_ADDR --master_port $MASTER_PORT \
your_program.py <normal cl args> --deepspeed ds_config.json'
```

然后你可以使用以下命令安排多节点部署，该命令在同一时间在所有节点上启动训练。

```
sbatch launch.slurm
```

笔记本

`deepspeed` 启动器不支持从笔记本部署，因此你需要模拟分布式环境。然而，这仅适用于 1 个 GPU。如果你想要使用多个 GPU，必须使用多进程环境，这意味着你必须使用 `deepspeed` 启动器，而不能像这里所示那样模拟。

```
# DeepSpeed 需要在只有一个进程的情况下也需要分布式环境
# 这在笔记本中模拟启动器
import os

os.environ["MASTER_ADDR"] = "localhost"
os.environ["MASTER_PORT"] = "9994"  # 如果出现 RuntimeError: Address already in use，修改此端口
os.environ["RANK"] = "0"
os.environ["LOCAL_RANK"] = "0"
os.environ["WORLD_SIZE"] = "1"

# 现在正常进行，加上 DeepSpeed 配置文件
training_args = TrainingArguments(..., deepspeed="ds_config_zero3.json")
trainer = Trainer(...)
trainer.train()
```

如果你想要在当前目录中的笔记本中动态创建配置文件，可以在一个专用的单元格中这样做。

如果训练脚本在一个文件中而不是笔记本单元格中，你可以从笔记本单元格中的 shell 正常启动 `deepspeed`。例如，启动 `run_translation.py`：

```
!git clone https://github.com/huggingface/transformers
!cd transformers; deepspeed examples/pytorch/translation/run_translation.py ...
```

你也可以使用 `%%bash` 魔法命令编写多行代码来运行 shell 程序，但你将无法在训练完成前查看日志。使用 `%%bash` 魔法命令时，你不需要模拟分布式环境。

```
%%bash

git clone https://github.com/huggingface/transformers
cd transformers
deepspeed examples/pytorch/translation/run_translation.py ...
```

保存模型权重

DeepSpeed 将主要的全精度 fp32 权重存储在自定义检查点优化器文件中（路径模式类似于 `global_step*/*optim_states.pt`），并保存在常规检查点下。

- **fp16**：使用 ZeRO-2 训练的模型将 `pytorch_model.bin` 权重保存为 fp16。要在使用 ZeRO-3 训练的模型中以 fp16 保存模型权重，你需要设置 `"stage3_gather_16bit_weights_on_model_save": true`，因为模型权重被分片分布在多个 GPU 上。否则，Trainer 不会以 fp16 保存权重，也不会创建 `pytorch_model.bin` 文件。这是因为 DeepSpeed 的 `state_dict` 包含占位符而不是实际权重，你将无法加载它们。

ZeRO 推理

ZeRO 推理 将模型权重放置在 CPU 或 NVMe 内存中，以减轻 GPU 的负担，从而在 GPU 上运行巨大模型的推理成为可能。推理不需要大量的额外内存来存储优化器状态和梯度，因此你可以在相同的硬件上处理更大的批量或序列长度。

ZeRO 推理使用与 ZeRO-3 相同的配置文件，ZeRO-2 和 ZeRO-1 的配置文件不会起作用，因为它们对推理没有好处。

要运行 ZeRO 推理，将常用的训练参数传递给 TrainingArguments 类，并添加 `--do_eval` 参数。

```
deepspeed --num_gpus=2 your_program.py <normal cl args> --do_eval --deepspeed ds_config.json
```

无 Trainer 的 DeepSpeed 集成

DeepSpeed 也可以在不使用 Trainer 类的情况下与 Transformers 一起使用。这由 `HfDeepSpeedConfig` 处理，它负责在调用 from_pretrained() 时收集 ZeRO-3 参数并将模型拆分到多个 GPU 上。

如果你希望一切都能自动处理，尝试使用 DeepSpeed 与 Trainer！你需要遵循 DeepSpeed 文档，并在配置文件中手动配置参数值（不能使用 `"auto"` 值）。

要高效地部署 ZeRO-3，必须在实例化模型之前实例化 `HfDeepSpeedConfig` 对象，并保持该对象存活：

预训练模型

```
from transformers.integrations import HfDeepSpeedConfig
from transformers import AutoModel
import deepspeed

ds_config = {...}  # deepspeed 配置对象或文件路径
```

```
# 必须在实例化模型之前运行以检测零 3
dschf = HfDeepSpeedConfig(ds_config)  # 保持此对象存活
model = AutoModel.from_pretrained("openai-community/gpt2")
engine = deepspeed.initialize(model=model, config_params=ds_config, ...)
```

非预训练模型

```
from transformers.integrations import HfDeepSpeedConfig
from transformers import AutoModel
import deepspeed

ds_config = {...}  # deepspeed 配置对象或文件路径
# 必须在实例化模型之前运行以检测零 3
dschf = HfDeepSpeedConfig(ds_config)  # 保持此对象存活
model = AutoModel.from_config(config=your_custom_config)
engine = deepspeed.initialize(model=model, config_params=ds_config, ...)
```

无 Trainer 的 ZeRO 推理

要在无法将模型放入单个 GPU 的情况下运行 ZeRO 推理，可以尝试使用额外的 GPU 或/和卸载到 CPU 内存。理解的重要细节是 ZeRO 的设计方式，你可以在不同的 GPU 上并行处理不同的输入。

确保：

- 如果你有足够的 GPU 内存，禁用 CPU 卸载（因为它会减慢速度）。
- 如果你有 Ampere 或更新的 GPU，启用 bf16 以加快速度。如果没有这些 GPU，你可以启用 fp16，只要你不使用在 bf16 上预训练的模型（如 T5 模型），因为这可能导致溢出错误。

查看以下脚本，以更好地了解如何在无法将模型放入单个 GPU 的情况下运行 ZeRO 推理。

```
#!/usr/bin/env python

# 本脚本演示了如何在无法将模型放入单个 GPU 的情况下使用 Deepspeed ZeRO 进行推理。
#
# 1. 使用 1 个 GPU 和 CPU 卸载
# 2. 或者使用多个 GPU
#
# 首先需要安装 deepspeed: pip install deepspeed
#
# 这里我们使用 3B 的 "bigscience/T0_3B" 模型，需要大约 15GB GPU 内存 - 因此 1 个大 GPU 或 2
# 个小 GPU 可以处理它。或者 1 个小 GPU 和大量 CPU 内存。
#
```

```
# 要使用更大的模型如 "bigscience/T0"，需要大约 50GB，除非你有一个 80GB GPU -
# 你需要 2-4 个 GPU。然后你可以调整脚本以处理更多 GPU，如果你想一次处理多个输入。
#
# 提供的 deepspeed 配置还启用了 CPU 内存卸载，所以如果有大量可用的 CPU 内存并且不介意速度变慢，你应该能够加载一个通常无法放入单个 GPU 的模型。如果你有足够的 GPU 内存，程序会在不卸载到 CPU 的情况下运行得更快 - 因此禁用该部分。
#
# 在 1 个 GPU 上部署：
#
# deepspeed --num_gpus 1 t0.py
# 或者：
# python -m torch.distributed.run --nproc_per_node=1 t0.py
#
# 在 2 个 GPU 上部署：
#
# deepspeed --num_gpus 2 t0.py
# 或者：
# python -m torch.distributed.run --nproc_per_node=2 t0.py

from transformers import AutoTokenizer, AutoConfig, AutoModelForSeq2SeqLM
from transformers.integrations import HfDeepSpeedConfig
import deepspeed
import os
import torch

os.environ["TOKENIZERS_PARALLELISM"] = "false"  # 避免 tokenizers 并行警告

# 分布式设置
local_rank = int(os.getenv("LOCAL_RANK", "0"))
world_size = int(os.getenv("WORLD_SIZE", "1"))
torch.cuda.set_device(local_rank)
deepspeed.init_distributed()

model_name = "bigscience/T0_3B"

config = AutoConfig.from_pretrained(model_name)
model_hidden_size = config.d_model

# 批量大小必须能被 world_size 整除，但可以大于 world_size
train_batch_size = 1 * world_size

# ds_config 注释
#
# - 如果你使用 Ampere 或更高版本的 GPU，启用 bf16 - 这将以混合精度运行并更快。
#
```

```
# - 对于较旧的 GPU，你可以启用 fp16，但它只适用于非 bf16 预训练的模型 - 例如，所有官方的 t5 模型都是
bf16 预训练的
#
# - 如果不想使用 CPU 卸载，将 offload_param.device 设置为 "none" 或完全删除 offload_param 部分
#
# - 如果使用 offload_param，你可以手动微调 stage3_param_persistence_threshold 以控制哪些参数保留在
GPU 上 - 值越大，卸载的大小越小
#
# 更多关于 Deepspeed 配置的详细信息请参阅
# https://huggingface.co/docs/transformers/main/main_classes/deepspeed

# 保持与 json 一致的格式，只是将 true/false 用小写表示
# fmt: off
ds_config = {
    "fp16": {
        "enabled": False
    },
    "bf16": {
        "enabled": False
    },
    "zero_optimization": {
        "stage": 3,
        "offload_param": {
            "device": "cpu",
            "pin_memory": True
        },
        "overlap_comm": True,
        "contiguous_gradients": True,
        "reduce_bucket_size": model_hidden_size * model_hidden_size,
        "stage3_prefetch_bucket_size": 0.9 * model_hidden_size * model_hidden_size,
        "stage3_param_persistence_threshold": 10 * model_hidden_size
    },
    "steps_per_print": 2000,
    "train_batch_size": train_batch_size,
    "train_micro_batch_size_per_gpu": 1,
    "wall_clock_breakdown": False
}
# fmt: on

# 下一行指示 transformers 在调用模型的 from_pretrained 方法时直接将模型拆分到多个 GPU 上。
#
# **必须在加载模型 AutoModelForSeq2SeqLM.from_pretrained(model_name) 之前运行**
#
# 否则模型将首先正常加载，仅在前向传播时拆分，这效率较低，且当 CPU 内存较少时可能会失败
dschf = HfDeepSpeedConfig(ds_config)  # 保持此对象存活
```

```
# 现在可以加载模型。
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# 初始化 Deepspeed ZeRO 并仅保留引擎对象
ds_engine = deepspeed.initialize(model=model, config_params=ds_config)[0]
ds_engine.module.eval()  # 推理

# Deepspeed ZeRO 可以在每个 GPU 上处理不相关的输入。所以对于 2 个 GPU，你可以一次处理 2 个输入。
# 如果使用更多 GPU，相应地调整输入数量。
# 当然，如果只有一个输入要处理，你需要将相同的字符串传递给所有 GPU
# 如果只使用一个 GPU，则只有 rank 0
rank = torch.distributed.get_rank()
if rank == 0:
    text_in = "Is this review positive or negative? Review: this is the best cast iron skillet
you will ever buy"
elif rank == 1:
    text_in = "Is this review positive or negative? Review: this is the worst restaurant ever"

tokenizer = AutoTokenizer.from_pretrained(model_name)
inputs = tokenizer.encode(text_in, return_tensors="pt").to(device=local_rank)
with torch.no_grad():
    outputs = ds_engine.module.generate(inputs, synced_gpus=True)
text_out = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(f"rank{rank}:\n   in={text_in}\n  out={text_out}")
```

将脚本保存为 `t0.py` 并启动：

```
$ deepspeed --num_gpus 2 t0.py
rank0:
   in=Is this review positive or negative? Review: this is the best cast iron skillet you will
ever buy
  out=Positive
rank1:
   in=Is this review positive or negative? Review: this is the worst restaurant ever
  out=negative
```

这是一个非常基本的示例，你需要根据自己的用途进行调整。

生成

使用多个 GPU 和 ZeRO-3 进行生成时，需要通过在 generate() 方法中设置 `synced_gpus=True` 来同步 GPU。否则，如果一个 GPU 比另一个 GPU 先完成生成，整个系统会挂起，因为剩余的 GPU 尚未从第一个完成的 GPU 收到权重分片。

对于 Transformers >= 4.28，如果在生成过程中检测到多个 GPU，`synced_gpus` 会自动设置为 `True`。

故障排除

当你遇到问题时，应该考虑是否是 DeepSpeed 导致的问题，因为往往并非如此（除非你在异常中看到了 DeepSpeed 模块）！第一步应该是不使用 DeepSpeed 重试你的设置，如果问题仍然存在，那么你可以报告问题。如果问题是 DeepSpeed 核心问题且与 Transformers 集成无关，请在 DeepSpeed 仓库 中打开一个问题。

对于与 Transformers 集成相关的问题，请提供以下信息：

- 完整的 DeepSpeed 配置文件
- Trainer 的命令行参数，或如果你自己编写 Trainer 设置的 TrainingArguments 参数（不要转储 TrainingArguments，因为它包含数十个无关条目）
- 以下命令的输出：

```
python -c 'import torch; print(f"torch: {torch.__version__}")'
python -c 'import transformers; print(f"transformers: {transformers.__version__}")'
python -c 'import deepspeed; print(f"deepspeed: {deepspeed.__version__}")'
```

- 一个指向 Google Colab 笔记本的链接以重现问题
- 如果不可能，提供一个标准的非自定义数据集，我们也尝试使用现有的示例来重现问题

以下部分提供了解决最常见的两个问题的指南。

DeepSpeed 进程在启动时被杀死

当 DeepSpeed 进程在启动时被杀死且没有堆栈跟踪时，通常意味着程序尝试分配的 CPU 内存超过了系统的可用内存，或者你的进程尝试分配的 CPU 内存超过了允许的内存，导致操作系统内核终止进程。这种情况下，检查你的配置文件是否有 `offload_optimizer`、`offload_param` 或两者都设置为卸载到 CPU。

如果你有 NVMe 和 ZeRO-3 设置，尝试将数据卸载到 NVMe (估算 你的模型的内存需求)。

NaN 损失

NaN 损失通常发生在模型在 bf16 上预训练，然后你尝试使用 fp16 时（特别是在 TPU 上训练的模型）。要解决这个问题，使用 fp32 或 bf16（如果硬件支持，如 TPU、Ampere GPU 或更新版本）。

另一个可能是与使用 fp16 相关。例如，如果你的 fp16 配置如下：

你可能会在日志中看到以下 `OVERFLOW!` 消息：

这意味着 DeepSpeed 损失缩放器无法找到一个缩放系数来克服损失溢出。要解决这个问题，尝试更高的 `initial_scale_power` 值（32 通常有效）。

资源

DeepSpeed ZeRO 是一项强大的技术，可以在有限的 GPU 资源下训练和加载非常大的模型，使其更具可访问性。要了解更多关于 DeepSpeed 的信息，可以阅读 博客文章、文档 和 GitHub 仓库。

以下论文也是深入了解 ZeRO 的绝佳资源：

- ZeRO: Memory Optimizations Toward Training Trillion Parameter Models
- ZeRO-Offload: Democratizing Billion-Scale Model Training
- ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning

在单个 CPU 上高效训练

本指南重点介绍如何在 CPU 上高效地训练大型模型。

使用 IPEX 实现混合精度

混合精度是指在模型中同时使用单精度 (fp32) 和半精度 (bf16/fp16) 数据类型来加速训练或推理，同时仍保持大部分单精度的准确性。现代 CPU 如第 3 代、第 4 代和第 5 代 Intel® Xeon® Scalable 处理器原生支持 bf16。第 6 代 Intel® Xeon® Scalable 处理器则支持 bf16 和 fp16。通过启用混合精度训练，你可以直接获得更好的性能提升。

为了进一步提高训练性能，你可以使用 Intel® Extension for PyTorch (IPEX)。IPEX 是基于 PyTorch 构建的一个库，它增加了对 Intel® 高级向量扩展 512 矢量神经网络指令 (Intel® AVX512-VNNI) 和 Intel® 高级矩阵扩展 (Intel® AMX) 的支持，从而在 Intel CPU 上提供额外的性能提升。然而，仅支持 AVX2 的 CPU（例如，AMD 或较旧的 Intel CPU）在 IPEX 下不一定有更好性能。

自 PyTorch 1.10 版本起，CPU 后端已经启用了自动混合精度 (AMP)。IPEX 也支持 CPU 上的 bf16/fp16 操作优化，并部分合并到主 PyTorch 分支。使用 IPEX AMP 可以获得更好的性能和用户体验。

了解更多详细信息，请参阅 自动混合精度。

IPEX 安装

IPEX 的发布版本跟随 PyTorch。通过 pip 安装：

PyTorch 版本	**IPEX 版本**
2.5.0	2.5.0+cpu
2.4.0	2.4.0+cpu
2.3.0	2.3.0+cpu
2.2.0	2.2.0+cpu

请运行 `pip list | grep torch` 获取你的 `pytorch_version`，以便获取相应的 `IPEX version_name`。

```
pip install intel_extension_for_pytorch==<version_name> -f https://developer.intel.com/ipex-whl-stable-cpu
```

如果需要，你可以在 ipex-whl-stable-cpu 查看最新版本。

了解更多的安装方法，请参阅 IPEX 安装指南。

在 Trainer 中使用

要在 Trainer 中启用 IPEX 自动混合精度，用户应在训练命令参数中添加 `use_ipex`、`bf16` 或 `fp16` 以及 `no_cuda`。

以下是在 Transformers 问答任务中使用的示例：

- 使用 IPEX 和 BF16 自动混合精度在 CPU 上训练：

```
python examples/pytorch/question-answering/run_qa.py \
--model_name_or_path google-bert/bert-base-uncased \
--dataset_name squad \
--do_train \
--do_eval \
--per_device_train_batch_size 12 \
--learning_rate 3e-5 \
--num_train_epochs 2 \
--max_seq_length 384 \
--doc_stride 128 \
--output_dir /tmp/debug_squad/ \
--use_ipex \
--bf16 \
--use_cpu
```

如果你想在脚本中启用 `use_ipex` 和 `bf16`，可以像这样在 `TrainingArguments` 中添加这些参数：

```
training_args = TrainingArguments(
    output_dir=args.output_path,
    bf16=True,
    use_ipex=True,
    use_cpu=True,
    **kwargs
)
```

实践示例

博客：使用 Intel Sapphire Rapids 加速 PyTorch Transformers

在多个 CPU 上高效训练

当在单个 CPU 上进行训练太慢时，我们可以使用多个 CPU。本指南重点介绍如何在裸机和 Kubernetes 上使用基于 PyTorch 的分布式数据并行（DDP）进行高效的 CPU 分布式训练。

Intel® oneCCL 绑定库 for PyTorch

Intel® oneCCL（集体通信库）是一个用于高效分布式深度学习训练的库，实现了诸如 allreduce、allgather、alltoall 等集体操作。有关 oneCCL 的更多信息，请参阅 oneCCL 文档 和 oneCCL 规范。

模块 `oneccl_bindings_for_pytorch`（1.12 版本之前称为 `torch_ccl`）实现了 PyTorch C10D ProcessGroup API，并且可以作为外部 ProcessGroup 动态加载，目前仅支持 Linux 平台。

更多详细信息请参阅 oneccl_bind_pt。

安装 Intel® oneCCL 绑定库 for PyTorch

以下 Python 版本提供了 wheel 文件：

扩展版本	Python 3.7	Python 3.8	Python 3.9	Python 3.10	Python 3.11
2.5.0		√	√	√	√
2.4.0		√	√	√	√
2.3.0		√	√	√	√
2.2.0		√	√	√	√

请运行 `pip list | grep torch` 来获取您的 `pytorch_version`。

```
pip install oneccl_bind_pt=={pytorch_version} -f https://developer.intel.com/ipex-whl-stable-cpu
```

其中 `{pytorch_version}` 是您的 PyTorch 版本，例如 2.4.0。更多安装方法请参阅 oneccl_bind_pt 安装文档。oneCCL 和 PyTorch 的版本必须匹配。

Intel® MPI 库

使用基于标准的 MPI 实现，在 Intel® 架构上提供灵活、高效的可扩展集群消息传递。该组件是 Intel® oneAPI HPC 工具包的一部分。

`oneccl_bindings_for_pytorch` 随 MPI 工具集一起安装。在使用前需要设置环境变量。

```
oneccl_bindings_for_pytorch_path=$(python -c "from oneccl_bindings_for_pytorch import cwd; print(cwd)")
source $oneccl_bindings_for_pytorch_path/env/setvars.sh
```

安装 Intel® 扩展 for PyTorch

Intel 扩展 for PyTorch (IPEX) 提供了 CPU 训练性能优化，支持 Float32 和 BFloat16（更多内容请参阅 单 CPU 部分）。

以下“在 Trainer 中使用”的部分以 Intel® MPI 库中的 `mpirun` 为例。

在 Trainer 中使用

要在 Trainer 中启用多 CPU 分布式训练（使用 ccl 后端），用户需要在命令参数中添加 `**--ddp_backend ccl**`。

让我们来看一个 问答示例 的例子。

以下命令在一台 Xeon 节点上启用 2 个进程进行训练，每个进程在一个插槽上运行。可以调整 OMP_NUM_THREADS/CCL_WORKER_COUNT 变量以获得最佳性能。

```
export CCL_WORKER_COUNT=1
export MASTER_ADDR=127.0.0.1
mpirun -n 2 -genv OMP_NUM_THREADS=23 \
python3 examples/pytorch/question-answering/run_qa.py \
--model_name_or_path google-bert/bert-large-uncased \
--dataset_name squad \
--do_train \
--do_eval \
--per_device_train_batch_size 12 \
--learning_rate 3e-5 \
--num_train_epochs 2 \
--max_seq_length 384 \
--doc_stride 128 \
--output_dir /tmp/debug_squad/ \
--no_cuda \
--ddp_backend ccl \
--use_ipex
```

以下命令在两台 Xeon 节点（node0 和 node1，以 node0 为主节点）上启用总共 4 个进程进行训练，每台节点上的进程数（ppn）设置为 2，每个进程在一个插槽上运行。可以调整 OMP_NUM_THREADS/CCL_WORKER_COUNT 变量以获得最佳性能。

在 node0 中，您需要创建一个包含每个节点 IP 地址的配置文件（例如 hostfile），并将该配置文件路径作为参数传递。

```
cat hostfile
xxx.xxx.xxx.xxx # node0 ip
xxx.xxx.xxx.xxx # node1 ip
```

现在，在 node0 中运行以下命令，并在 node0 和 node1 中启用 4DDP 和 BF16 自动混合精度：

```
export CCL_WORKER_COUNT=1
export MASTER_ADDR=xxx.xxx.xxx.xxx # node0 ip
mpirun -f hostfile -n 4 -ppn 2 \
-genv OMP_NUM_THREADS=23 \
python3 examples/pytorch/question-answering/run_qa.py \
```

```
--model_name_or_path google-bert/bert-large-uncased \
--dataset_name squad \
--do_train \
--do_eval \
--per_device_train_batch_size 12 \
--learning_rate 3e-5 \
--num_train_epochs 2 \
--max_seq_length 384 \
--doc_stride 128 \
--output_dir /tmp/debug_squad/ \
--no_cuda \
--ddp_backend ccl \
--use_ipex \
--bf16
```

在 Kubernetes 上使用

上一节中的分布式训练作业可以部署到 Kubernetes 集群中，使用 Kubeflow PyTorchJob 训练操作符。

准备工作

本示例假定您已经具备以下条件：

- 访问安装了 Kubeflow 的 Kubernetes 集群
- 安装并配置了 `kubectl` 以访问 Kubernetes 集群
- 有一个 持久卷声明 (PVC) 可用于存储数据集和模型文件。设置 PVC 的选项包括使用 NFS 存储类 或云存储桶。
- 一个包含模型训练脚本及其所有依赖项的 Docker 容器。对于分布式 CPU 训练作业，这通常包括 PyTorch、Transformers、Intel 扩展 for PyTorch、Intel oneCCL 绑定库 for PyTorch 和 OpenSSH 以在容器之间通信。

以下是一个示例 Dockerfile，使用支持分布式 CPU 训练的基础镜像，然后将 Transformers 发行版提取到 `/workspace` 目录中，以便示例脚本包含在镜像中：

```
FROM intel/intel-optimized-pytorch:2.4.0-pip-multinode

RUN apt-get update -y && \
    apt-get install -y --no-install-recommends --fix-missing \
    google-perftools \
    libomp-dev

WORKDIR /workspace
```

```
# 下载并解压 Transformers 代码
ARG HF_TRANSFORMERS_VER="4.46.0"
RUN pip install --no-cache-dir \
    transformers==${HF_TRANSFORMERS_VER} && \
    mkdir transformers && \
    curl -sSL --retry 5
https://github.com/huggingface/transformers/archive/refs/tags/v${HF_TRANSFORMERS_VER}.tar.gz |
tar -C transformers --strip-components=1 -xzf -
```

此镜像需要构建并复制到集群的节点上，或者推送到容器注册表，然后再将 PyTorchJob 部署到集群中。

PyTorchJob 规范文件

Kubeflow PyTorchJob 用于在集群上运行分布式训练作业。PyTorchJob 的 YAML 文件定义了以下参数：

- PyTorchJob 的名称
- 副本数量（工作节点）
- 将用于运行训练作业的 Python 脚本及其参数
- 每个工作节点所需的资源类型（节点选择器、内存和 CPU）
- 要使用的 Docker 镜像和标签
- 环境变量
- 卷挂载点用于 PVC

卷挂载点定义了 PVC 将在每个工作节点 Pod 的容器中挂载的路径。这个位置可以用于数据集、检查点文件和训练完成后保存的模型。

以下是一个带有 4 个工作节点的 PyTorchJob YAML 文件示例，运行 问答示例：

要运行此示例，请根据您的训练脚本和集群节点更新 YAML 文件。

YAML 文件中的 CPU 资源限制/请求定义在 CPU 单位中，其中 1 个 CPU 单位相当于 1 个物理 CPU 核心或 1 个虚拟核心（取决于节点是物理主机还是虚拟机）。YAML 文件中定义的 CPU 和内存限制/请求应小于单个机器上的可用 CPU/内存容量。通常，最好不要使用整个机器的全部容量，以留出一些资源给 kubelet 和操作系统。为了获得 “保证” 服务质量，请将相同的 CPU 和内存数量设置为资源限制和请求。

部署

更新 PyTorchJob 规范文件中的值，使其适合您的集群和训练作业，然后使用以下命令将其部署到集群中：

```
export NAMESPACE=<指定您的命名空间>

kubectl create -f pytorchjob.yaml -n ${NAMESPACE}
```

然后可以使用 `kubectl get pods -n ${NAMESPACE}` 命令列出命名空间中的 Pod。您应该看到刚刚部署的 PyTorchJob 的工作节点 Pod。初始状态可能是“Pending”，因为容器正在被拉取和创建，然后状态应变为“Running”。

可以使用 `kubectl logs <pod name> -n ${NAMESPACE}` 查看工作节点的日志。添加 `-f` 以实时流式传输日志，例如：

```
kubectl logs transformers-pytorchjob-worker-0 -n ${NAMESPACE} -f
```

训练作业完成后，可以从 PVC 或存储位置复制训练好的模型。完成作业后，可以使用 `kubectl delete -f pytorchjob.yaml -n ${NAMESPACE}` 从集群中删除 PyTorchJob 资源。

总结

本指南介绍了如何在裸机和 Kubernetes 集群上使用多个 CPU 进行分布式 PyTorch 训练。两种情况都使用了 Intel 扩展 for PyTorch 和 Intel oneCCL 绑定库 for PyTorch 以实现最佳训练性能，可以作为模板用于在多个节点上运行自己的工作负载。

使用 TensorFlow 在 TPU 上进行训练

如果你不想阅读长篇大论，只想直接查看 TPU 的代码示例，可以查看我们的 TPU 示例笔记本！

什么是 TPU?

TPU 是 **张量处理单元（Tensor Processing Unit）**。它们是由谷歌设计的硬件，用于大幅加速神经网络中的张量计算，类似于 GPU。TPU 可以用于网络训练和推理。它们通常通过谷歌的云服务访问，但在 Google Colab 和 Kaggle Kernels 中也可以免费访问小型 TPU。

因为 所有 🤗 Transformers 中的 TensorFlow 模型都是 Keras 模型，本文档中的大多数方法也适用于任何 Keras 模型的 TPU 训练！不过，有些内容是特定于 HuggingFace 的 Transformers 和 Datasets 生态系统的，我们会在需要时特别指出这些内容。

可以使用哪些类型的 TPU?

新用户经常对不同类型的 TPU 和访问方式感到困惑。首先需要了解的关键区别是 **TPU 节点（TPU Node）** 和 **TPU 虚拟机（TPU VM）** 之间的区别。

当你使用 **TPU 节点** 时，实际上是间接访问远程 TPU。你需要一个单独的虚拟机来初始化网络和数据管道，然后将它们转发到远程节点。在 Google Colab 中使用 TPU 时，实际上是在使用 **TPU 节点**。

使用 TPU 节点可能会有一些令人意外的行为，特别是对于不熟悉它的人！具体来说，由于 TPU 位于与运行 Python 代码的机器物理上不同的系统中，因此你的数据不能保存在本地机器上——从本地机器内部存储加载数据的数据管道将完全失败！相反，数据必须存储在 Google Cloud Storage 中，以便数据管道在远程 TPU 节点上运行时仍能访问这些数据。

如果你可以将所有数据存储在内存中作为 `np.ndarray` 或 `tf.Tensor`，则可以在使用 Colab 或 TPU 节点时直接调用 `fit()` 而不需要上传数据到 Google Cloud Storage。

🤗 **Hugging Face 小贴士 🤗：** 方法 `Dataset.to_tf_dataset()` 及其高级包装器 `model.prepare_tf_dataset()` 在 TPU 节点上会失败。原因是虽然它们创建了一个 `tf.data.Dataset`，但它不是一个“纯” `tf.data` 管道，而是使用 `tf.numpy_function` 或 `Dataset.from_generator()` 从底层的 HuggingFace `Dataset` 流式传输数据。这个 HuggingFace `Dataset` 存储在本地磁盘上，远程 TPU 节点无法读取。

第二种访问 TPU 的方式是通过 **TPU 虚拟机**。使用 TPU 虚拟机时，你可以直接连接到 TPU 附带的机器，就像在 GPU 虚拟机上训练一样。TPU 虚拟机通常更容易使用，尤其是在处理数据管道方面。上述所有警告都不适用于 TPU 虚拟机！

这是一个有偏见的文档，所以这里是我们的一点意见：**如果可能，尽量避免使用 TPU 节点。** 它比 TPU 虚拟机更令人困惑且更难调试。未来 TPU 节点也很可能不再被支持——谷歌最新的 TPU，TPUv4，只能作为 TPU 虚拟机访问，这表明 TPU 节点越来越成为一个“遗留”访问方法。然而，

我们理解唯一免费的 TPU 访问是在 Colab 和 Kaggle Kernels 上，这些平台使用 TPU 节点——所以我们将会解释如何处理这个问题！请查看 TPU 示例笔记本 获取更详细的代码示例。

有哪些大小的 TPU 可供使用?

单个 TPU（如 v2-8/v3-8/v4-8）运行 8 个副本。TPU 以 **吊舱（pod）** 形式存在，可以同时运行数百或数千个副本。当使用的 TPU 数量超过单个 TPU 但少于整个吊舱时（例如 v3-32），你的 TPU 集群称为 **吊舱切片（pod slice）**。

通过 Colab 免费访问 TPU 时，通常会获得一个单个 v2-8 TPU。

我经常听说 XLA 这个东西。什么是 XLA? 它与 TPU 有何关系?

XLA 是一种优化编译器，由 TensorFlow 和 JAX 使用。在 JAX 中它是唯一的编译器，而在 TensorFlow 中是可选的（但在 TPU 上是强制的）。最简单的方式来启用 XLA 是在调用 `model.compile()` 时传递参数 `jit_compile=True`。如果你没有遇到错误且性能良好，那就说明你已经准备好在 TPU 上运行了！

在 TPU 上调试通常比在 CPU/GPU 上更困难，因此我们建议先在 CPU/GPU 上使用 XLA 运行代码，然后再尝试在 TPU 上运行。当然，你不必训练很长时间——只需跑几步以确保模型和数据管道按预期工作即可。

XLA 编译后的代码通常更快——即使你不打算在 TPU 上运行，添加 `jit_compile=True` 也可以提高性能。不过，请注意以下关于 XLA 兼容性的注意事项！

经验之谈： 尽管使用 `jit_compile=True` 是提高速度并测试 CPU/GPU 代码是否 XLA 兼容的好方法，但实际在 TPU 上训练时将其保留可能会导致很多问题。XLA 编译将在 TPU 上隐式进行，因此记得在实际运行代码时删除这一行！

如何使我的模型 XLA 兼容?

在许多情况下，你的代码可能已经是 XLA 兼容的！但是，有些在普通 TensorFlow 中有效的方法在 XLA 中无效。我们将其归纳为以下三条核心规则：

😭 **Hugging Face 小贴士** 😭： 我们投入了很多努力重写了 TensorFlow 模型和损失函数，使其 XLA 兼容。我们的模型和损失函数默认遵循规则 #1 和 #2，因此如果你使用的是 `transformers` 模型，可以跳过这两条规则。但在编写自己的模型和损失函数时不要忘记这些规则！

XLA 规则 1：代码中不能有“数据依赖的条件语句”

这意味着任何 `if` 语句都不能依赖于 `tf.Tensor` 内部的值。例如，以下代码块无法用 XLA 编译：

```
if tf.reduce_sum(tensor) > 10:
    tensor = tensor / 2.0
```

这可能一开始看起来非常限制，但大多数神经网络代码并不需要这样做。你可以通过使用 `tf.cond`（参见文档 这里）或移除条件并找到一个巧妙的数学技巧来绕过这个限制，如下所示：

```
sum_over_10 = tf.cast(tf.reduce_sum(tensor) > 10, tf.float32)
tensor = tensor / (1.0 + sum_over_10)
```

这段代码与上面的效果相同，但由于避免了条件语句，确保它可以无问题地用 XLA 编译！

XLA 规则 2：代码中不能有“数据依赖的形状”

这意味着代码中所有 `tf.Tensor` 对象的形状不能依赖于它们的值。例如，函数 `tf.unique` 无法用 XLA 编译，因为它返回一个包含输入中每个唯一值的 `tensor`。这个输出的形状显然会根据输入 `Tensor` 的重复程度而不同，因此 XLA 拒绝处理它！

通常情况下，大多数神经网络代码默认遵循规则 #2。但有一些常见的情况会导致问题。一个非常常见的情况是使用 **标签掩码**，将标签设置为负值以表示在计算损失时应忽略这些位置。如果你查看支持标签掩码的 NumPy 或 PyTorch 损失函数，通常会看到如下代码，使用 布尔索引：

```
label_mask = labels >= 0
masked_outputs = outputs[label_mask]
masked_labels = labels[label_mask]
loss = compute_loss(masked_outputs, masked_labels)
mean_loss = torch.mean(loss)
```

这段代码在 NumPy 或 PyTorch 中没有问题，但在 XLA 中会出错！为什么？因为 `masked_outputs` 和 `masked_labels` 的形状取决于有多少位置被掩码——这是 **数据依赖的形状**。然而，就像规则 #1 一样，我们可以重写这段代码以在没有数据依赖形状的情况下产生相同的输出。

```
label_mask = tf.cast(labels >= 0, tf.float32)
loss = compute_loss(outputs, labels)
```

```
loss = loss * label_mask  # 将负标签位置设为 0
mean_loss = tf.reduce_sum(loss) / tf.reduce_sum(label_mask)
```

通过计算每个位置的损失，但在计算平均值时将掩码位置在分子和分母中都设为 0，我们得到了与第一段代码相同的结果，同时保持了 XLA 兼容性。请注意，我们使用了与规则 #1 相同的技巧——将 `tf.bool` 转换为 `tf.float32` 并将其用作指示变量。这是一个非常有用的技巧，记住它如果需要将代码转换为 XLA！

XLA 规则 3：XLA 需要为每个不同的输入形状重新编译模型

这是最重要的一个规则。这意味着如果输入形状变化很大，XLA 将不得不反复重新编译模型，从而导致严重的性能问题。这在 NLP 模型中很常见，因为文本经过分词后的长度各不相同。在其他模态中，静态形状更为常见，这个规则的问题要少得多。

如何解决规则 #3？关键是 **填充**——如果你将所有输入填充到相同的长度，并使用 `attention_mask`，你就可以得到与可变形状相同的结果，而不会出现 XLA 问题。然而，过度填充也会导致严重的减速——如果将所有样本填充到整个数据集的最大长度，可能会得到大量填充标记的批次，浪费大量的计算和内存！

没有完美的解决方案。但你可以尝试一些技巧。一个非常有用的技巧是 **将样本批次填充到某个数字（如 32 或 64）的倍数**。这通常只会增加少量标记，但大大减少了独特的输入形状数量，因为每个输入形状现在必须是 32 或 64 的倍数。较少的独特输入形状意味着较少的 XLA 编译！

🤗 **Hugging Face 小贴士** 🤗：我们的分词器和数据整理器有一些方法可以帮助你。你可以在调用分词器时使用 `padding="max_length"` 或 `padding="longest"` 来获取填充后的数据。我们的分词器和数据整理器还有一个 `pad_to_multiple_of` 参数，可以用来减少你看到的独特输入形状数量！

如何在我的 TPU 上实际训练模型？

一旦你的训练 XLA 兼容并且（如果你使用 TPU 节点/Colab）数据集已适当准备，运行 TPU 实际上非常简单！你只需要在代码中添加几行来初始化 TPU，并确保模型和数据集在 `TPUStrategy` 范围内创建。查看 我们的 TPU 示例笔记本 了解实际操作！

总结

这里涉及的内容很多，让我们总结一个快速检查清单，帮助你为 TPU 训练做好准备：

- 确保你的代码遵循 XLA 的三条规则
- 在 CPU/GPU 上使用 `jit_compile=True` 编译模型并确认可以使用 XLA 进行训练
- 要么将数据集加载到内存中，要么使用 TPU 兼容的数据加载方法（参见 笔记本）
- 将代码迁移到 Colab（将加速器设置为“TPU”）或 Google Cloud 上的 TPU 虚拟机
- 添加 TPU 初始化代码（参见 笔记本）
- 创建 `TPUStrategy` 并确保数据加载和模型创建都在 `strategy.scope()` 内（参见 笔记本）
- 不要忘记在迁移到 TPU 时移除 `jit_compile=True`！
- 🙏🙏🙏😳😳😳
- 调用 `model.fit()`
- 你成功了！

使用 PyTorch 在 Apple 芯片上训练模型

之前，在 Mac 上训练模型只能使用 CPU。随着 PyTorch v1.12 的发布，你现在可以利用 Apple 的硅基 GPU 进行模型训练，从而获得显著更快的性能和训练速度。这是通过在 PyTorch 中集成 Apple 的 Metal Performance Shaders (MPS) 后端实现的。MPS 后端 将 PyTorch 操作实现为自定义 Metal 着色器，并将这些模块放置在 `mps` 设备上。

目前，一些 PyTorch 操作尚未在 MPS 中实现，可能会抛出错误。为了避免这种情况，你可以设置环境变量 `PYTORCH_ENABLE_MPS_FALLBACK=1`，这样当遇到不支持的操作时会自动回退到 CPU 内核（你仍然会看到一个 `UserWarning`）。

如果你遇到其他错误，请在 PyTorch 仓库中提交问题，因为 Trainer 仅集成了 MPS 后端。

设置 `mps` 设备后，你可以：

- 在本地训练更大的网络或批量大小
- 减少数据检索延迟，因为 GPU 的统一内存架构允许直接访问完整的内存存储
- 节省成本，因为不需要在基于云的 GPU 或添加本地 GPU 上进行训练

首先确保你已经安装了 PyTorch。MPS 加速支持 macOS 12.3+。

```
pip install torch torchvision torchaudio
```

TrainingArguments 默认情况下会使用 `mps` 设备，如果它可用的话，这意味着你不需要显式设置设备。例如，你可以运行 run_glue.py 脚本，而无需进行任何更改即可自动启用 MPS 后端。

```
export TASK_NAME=mrpc

python examples/pytorch/text-classification/run_glue.py \
  --model_name_or_path google-bert/bert-base-cased \
  --task_name $TASK_NAME \
  --use_mps_device \
  --do_train \
  --do_eval \
  --max_seq_length 128 \
  --per_device_train_batch_size 32 \
  --learning_rate 2e-5 \
  --num_train_epochs 3 \
  --output_dir /tmp/$TASK_NAME/ \
  --overwrite_output_dir
```

对于 分布式设置的后端如 `gloo` 和 `nccl`，`mps` 设备不支持这些后端，这意味着你只能在单个 GPU 上使用 MPS 后端进行训练。

了解更多关于 MPS 后端的信息，可以阅读 加速 PyTorch 在 Mac 上的训练 博客文章。

使用自定义的硬件进行训练

运行模型训练和推理所使用的硬件对性能有很大影响。关于 GPU 的深入探讨，请务必查看 Tim Dettmer 的优秀 博客文章。

让我们来看一些关于 GPU 设置的实际建议。

GPU

当你训练更大的模型时，基本上有三种选择：

- 更大的 GPU
- 更多的 GPU
- 更多的 CPU 和 NVMe（通过 DeepSpeed-Infinity 卸载）

我们从单个 GPU 的情况开始谈起。

电源和冷却

如果你购买了昂贵的高端 GPU，请确保为其提供正确的电源和充分的冷却。

电源：

一些高端消费级 GPU 卡有 2 个甚至 3 个 PCI-E 8 针电源接口。确保每个接口都插有独立的 12V PCI-E 8 针电缆。不要使用同一根电缆两端的两个分叉头（通常称为猪尾巴电缆）。例如，如果你的 GPU 有两个电源接口，那么你应该从电源供应器 (PSU) 连接两根 PCI-E 8 针电缆到 GPU，而不是一根带有两个 PCI-E 8 针连接头的电缆。否则，你无法充分发挥卡的性能。

每个 PCI-E 8 针电源电缆需要插入 PSU 的 12V 电源线，可以提供高达 150W 的功率。

有些其他卡可能使用 PCI-E 12 针连接器，这些连接器可以提供高达 500-600W 的功率。

低端卡可能使用 6 针连接器，这些连接器可以提供高达 75W 的功率。

此外，你还需要一个高质量的 PSU，它能提供稳定的电压。一些低质量的 PSU 可能无法为卡提供所需的稳定电压，以使其在峰值状态下运行。

当然，PSU 必须有足够的剩余瓦数来为卡供电。

冷却：

当 GPU 过热时，它会开始降低性能，甚至在过热时关闭。很难确定 GPU 在高负载下最佳的目标温度是多少，但通常只要低于 +80°C 就可以了，而 70-75°C 是一个非常好的范围。通常在温度达到约 84-90°C 时，性能开始下降。除了降低性能外，长时间的高温还可能缩短 GPU 的使用寿命。

接下来，我们来看看使用多个 GPU 时最重要的方面之一：连接性。

多 GPU 连接性

如果你使用多个 GPU，卡之间的互连方式可能会对总训练时间产生巨大影响。如果 GPU 在同一物理节点上，你可以运行：

```
nvidia-smi topo -m
```

这将告诉你 GPU 如何互连。在具有双 GPU 并且通过 NVLink 连接的机器上，你可能会看到类似以下内容：

而在没有 NVLink 的不同机器上，你可能会看到：

报告包括以下图例：

所以第一个报告中的 `NV2` 告诉我们 GPU 之间通过 2 条 NVLink 连接，而第二个报告中的 `PHB` 表明我们有一个典型的消费级 PCIe+桥接设置。

检查你的设置中使用的是哪种连接类型。其中一些可以使卡之间的通信更快（例如 NVLink），而另一些则较慢（例如 PHB）。

根据使用的可扩展性解决方案的不同，连接速度的影响可能是重大或轻微的。如果 GPU 需要很少同步（如 DDP），较慢的连接影响较小。如果 GPU 需要频繁互相发送消息（如 ZeRO-DP），那么更快的连接变得非常关键，以实现更快的训练。

NVLink

NVLink 是由 Nvidia 开发的一种基于线缆的串行多通道近程通信链路。

每一代新产品的带宽都会更快。例如，根据 Nvidia Ampere GA102 GPU 架构白皮书 中的引用：

> 第三代 NVLink® GA102 GPU 使用 NVIDIA 的第三代 NVLink 接口，该接口包括四条 x4 链路，每条链路在两个 GPU 之间提供 14.0625 GB/s 的双向带宽。四条链路总共提供 56.25 GB/s 的双向带宽，以及 112.5 GB/s 的总带宽。两块 RTX 3090 GPU 可以通过 NVLink 连接以实现 SLI。（注意，不支持 3-Way 和 4-Way SLI 配置。）

因此，在 `nvidia-smi topo -m` 的输出报告中，`NVX` 的 `X` 越高越好。这取决于你的 GPU 架构。

让我们比较一下在 `openai-community/gpt2` 语言模型上使用小样本 wikitext 进行训练的执行情况。

结果如下：

NVLink	**时间 (秒)**
Y	101
N	131

可以看到，使用 NVLink 完成训练大约快 23%。在第二个基准测试中，我们使用 `NCCL_P2P_DISABLE=1` 告诉 GPU 不使用 NVLink。

以下是完整的基准测试代码和输出：

```
# DDP 使用 NVLink

rm -r /tmp/test-clm; CUDA_VISIBLE_DEVICES=0,1 torchrun \
--nproc_per_node 2 examples/pytorch/language-modeling/run_clm.py --model_name_or_path openai-
community/gpt2 \
--dataset_name wikitext --dataset_config_name wikitext-2-raw-v1 --do_train \
--output_dir /tmp/test-clm --per_device_train_batch_size 4 --max_steps 200

{'train_runtime': 101.9003, 'train_samples_per_second': 1.963, 'epoch': 0.69}

# DDP 不使用 NVLink

rm -r /tmp/test-clm; CUDA_VISIBLE_DEVICES=0,1 NCCL_P2P_DISABLE=1 torchrun \
--nproc_per_node 2 examples/pytorch/language-modeling/run_clm.py --model_name_or_path openai-
community/gpt2 \
--dataset_name wikitext --dataset_config_name wikitext-2-raw-v1 --do_train \
--output_dir /tmp/test-clm --per_device_train_batch_size 4 --max_steps 200

{'train_runtime': 131.4367, 'train_samples_per_second': 1.522, 'epoch': 0.69}
```

硬件：2 块 TITAN RTX，每块 24GB 内存 + 2 条 NVLink (`NV2` 在 `nvidia-smi topo -m` 中)

软件：`pytorch-1.8-to-be` + `cuda-11.0` / `transformers==4.3.0.dev0`

使用 Trainer API 进行超参数搜索

🤗 Transformers 提供了一个优化的 Trainer 类，用于训练 🤗 Transformers 模型，使得开始训练变得更加容易，而无需手动编写自己的训练循环。Trainer 提供了超参数搜索的 API。本文档展示了如何在示例中启用它。

超参数搜索后端

Trainer 目前支持四个超参数搜索后端：Optuna、SigOpt、Ray Tune 和 Weights & Biases (W&B)。

在使用这些超参数搜索后端之前，需要先安装它们：

```
pip install optuna/sigopt/wandb/ray[tune]
```

如何在示例中启用超参数搜索

定义超参数搜索空间，不同的后端需要不同的格式。

SigOpt

参考 SigOpt 的 object_parameter，如下所示：

```
def sigopt_hp_space(trial):
    return [
        {"bounds": {"min": 1e-6, "max": 1e-4}, "name": "learning_rate", "type": "double"},
        {
            "categorical_values": ["16", "32", "64", "128"],
            "name": "per_device_train_batch_size",
            "type": "categorical",
        },
    ]
```

Optuna

参考 Optuna 的 object_parameter，如下所示：

```
def optuna_hp_space(trial):
    return {
        "learning_rate": trial.suggest_float("learning_rate", 1e-6, 1e-4, log=True),
        "per_device_train_batch_size": trial.suggest_categorical("per_device_train_batch_size",
[16, 32, 64, 128]),
    }
```

Optuna 支持多目标超参数优化（HPO）。您可以在 `hyperparameter_search` 中传递 `direction` 参数，并定义自己的 `compute_objective` 函数来返回多个目标值。`hyperparameter_search` 将返回 Pareto 前沿（`List[BestRun]`），您可以参考 test_trainer 中的测试用例 `TrainerHyperParameterMultiObjectOptunaIntegrationTest`。如下所示：

```
best_trials = trainer.hyperparameter_search(
    direction=["minimize", "maximize"],
    backend="optuna",
    hp_space=optuna_hp_space,
    n_trials=20,
    compute_objective=compute_objective,
)
```

Ray Tune

参考 Ray Tune 的 object_parameter，如下所示：

```
def ray_hp_space(trial):
    return {
        "learning_rate": tune.loguniform(1e-6, 1e-4),
        "per_device_train_batch_size": tune.choice([16, 32, 64, 128]),
    }
```

Weights & Biases (W&B)

参考 W&B 的 object_parameter，如下所示：

```
def wandb_hp_space(trial):
    return {
        "method": "random",
        "metric": {"name": "objective", "goal": "minimize"},
        "parameters": {
            "learning_rate": {"distribution": "uniform", "min": 1e-6, "max": 1e-4},
            "per_device_train_batch_size": {"values": [16, 32, 64, 128]},
        },
    }
```

定义 model_init 函数

定义一个 `model_init` 函数并将其传递给 Trainer，例如：

```
def model_init(trial):
    return AutoModelForSequenceClassification.from_pretrained(
        model_args.model_name_or_path,
        from_tf=bool(".ckpt" in model_args.model_name_or_path),
        config=config,
        cache_dir=model_args.cache_dir,
        revision=model_args.model_revision,
        token=True if model_args.use_auth_token else None,
    )
```

创建 Trainer

创建一个 Trainer，包括您的 `model_init` 函数、训练参数、训练和测试数据集以及评估函数：

```
trainer = Trainer(
    model=None,
    args=training_args,
    train_dataset=small_train_dataset,
    eval_dataset=small_eval_dataset,
    compute_metrics=compute_metrics,
    processing_class=tokenizer,
    model_init=model_init,
    data_collator=data_collator,
)
```

调用超参数搜索

调用超参数搜索，获取最佳试验参数。后端可以是 `"optuna"`、`"sigopt"`、`"wandb"` 或 `"ray"`。`direction` 可以是 `"minimize"` 或 `"maximize"`，表示要优化更大的目标还是更小的目标。

您可以定义自己的 `compute_objective` 函数，如果不定义，将调用默认的 `compute_objective` 函数，返回评估指标（如 F1 值）的总和作为目标值。

```
best_trial = trainer.hyperparameter_search(
    direction="maximize",
    backend="optuna",
    hp_space=optuna_hp_space,
    n_trials=20,
    compute_objective=compute_objective,
)
```

适用于 DDP 微调的超参数搜索

目前，适用于 DDP 的超参数搜索仅支持 Optuna 和 SigOpt。只有排名为零的进程会生成搜索试验并将参数传递给其他进程。

优化推理方法（优化生成方法）

通过 CPU 优化推理

通过一些优化技术，可以在 CPU 上高效地运行大型模型的推理。其中一种优化方法是将 PyTorch 代码编译成中间格式，以便在高性能环境中（如 C++）运行。另一种方法是将多个操作合并到一个内核中，以减少每个操作单独运行时的开销。

在这篇文章中，你会学到如何使用 BetterTransformer 提高推理速度，以及如何将 PyTorch 代码转换为 TorchScript。如果你使用的是英特尔 CPU，还可以利用 英特尔扩展库 中的图优化技术进一步提高推理速度。最后，你将学到如何使用 🤗 Optimum 通过 ONNX Runtime 或 OpenVINO（如果你使用的是英特尔 CPU）加速推理。

BetterTransformer

BetterTransformer 通过其快速路径（原生 PyTorch 专门实现的 Transformer 函数）执行来加速推理。快速路径执行中的两种优化技术是：

332. **融合**：将多个连续的操作合并到一个“内核”中，以减少计算步骤的数量。
333. **跳过填充标记的固有稀疏性**：避免对嵌套张量进行不必要的计算。

BetterTransformer 还将所有注意力操作转换为使用更节省内存的 缩放点积注意力。

并不是所有模型都支持 BetterTransformer。请查看这个 列表，看看你的模型是否支持 BetterTransformer。

在开始之前，请确保已经安装了 🤗 Optimum。你可以参考 安装指南 安装。

使用 PreTrainedModel.to_bettertransformer() 方法启用 BetterTransformer：

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("bigcode/starcoder", torch_dtype="auto")
```

TorchScript

TorchScript 是一种 PyTorch 模型表示形式，可以在生产环境中用于性能至关重要的场景。你可以先在 PyTorch 中训练模型，然后将其导出为 TorchScript，以摆脱 Python 性能限制。PyTorch 通过 追踪 模型返回一个优化过的 `ScriptFunction`，该函数通过即时编译（JIT）进行优化。与默认的急切模式相比，PyTorch 的 JIT 模式通常通过操作融合等优化技术在推理时提供更好的性能。

要了解 TorchScript 的基本概念，可以参考 PyTorch TorchScript 入门教程。

使用 Trainer 类时，可以通过设置 `--jit_mode_eval` 标志来启用 CPU 推理的 JIT 模式：

```
python examples/pytorch/question-answering/run_qa.py \
--model_name_or_path csarron/bert-base-uncased-squad-v1 \
--dataset_name squad \
--do_eval \
--max_seq_length 384 \
--doc_stride 128 \
--output_dir /tmp/ \
--no_cuda \
--jit_mode_eval
```

对于 PyTorch >= 1.14.0，JIT 模式可以通过支持字典输入的 `jit.trace` 来受益于任何模型的预测和评估。

对于 PyTorch < 1.14.0，如果模型的前向参数顺序与 `jit.trace` 中的元组输入顺序匹配（例如问答模型），JIT 模式可以受益。如果前向参数顺序不匹配（例如文本分类模型），`jit.trace` 将失败，并通过异常捕获回退。日志用于通知用户。

IPEX 图优化

英特尔® 扩展库（IPEX）在 JIT 模式下为英特尔 CPU 提供了进一步的优化，我们建议将其与 TorchScript 结合使用以获得更快的性能。IPEX 图优化 可以融合多头注意力、拼接线性层、线性+加法、线性+Gelu、加法+层归一化等多种操作。

要利用这些图优化，请确保已安装 IPEX。你可以参考 安装指南 安装：

```
pip install intel_extension_for_pytorch
```

在 Trainer 类中设置 `--use_ipex` 和 `--jit_mode_eval` 标志以启用带有图优化的 JIT 模式：

```
python examples/pytorch/question-answering/run_qa.py \
--model_name_or_path csarron/bert-base-uncased-squad-v1 \
--dataset_name squad \
--do_eval \
--max_seq_length 384 \
--doc_stride 128 \
--output_dir /tmp/ \
--no_cuda \
--use_ipex \
--jit_mode_eval
```

🤗 Optimum

有关如何使用 ORT 与 🤗 Optimum 的详细信息，请参阅 使用 ONNX Runtime 进行 Optimum 推理 指南。本节仅提供一个简短而简单的示例。

ONNX Runtime（ORT）是一种模型加速器，默认情况下在 CPU 上运行推理。ORT 由 🤗 Optimum 支持，可以在 🤗 Transformers 中使用，而无需对代码进行太多更改。你只需要将 🤗 Transformers 的 `AutoClass` 替换为其等效的 ORTModel，并加载一个 ONNX 格式的检查点。

例如，如果你正在运行一个问答任务的推理，可以加载 optimum/roberta-base-squad2 检查点，其中包含一个 `model.onnx` 文件：

```
from transformers import AutoTokenizer, pipeline
from optimum.onnxruntime import ORTModelForQuestionAnswering

model = ORTModelForQuestionAnswering.from_pretrained("optimum/roberta-base-squad2")
tokenizer = AutoTokenizer.from_pretrained("deepset/roberta-base-squad2")

onnx_qa = pipeline("question-answering", model=model, tokenizer=tokenizer)

question = "我的名字是什么？"
context = "我的名字是 Philipp，我住在纽伦堡。"
pred = onnx_qa(question, context)
```

如果你使用的是英特尔 CPU，请查看 🤗 Optimum Intel，它支持多种压缩技术（量化、剪枝、知识蒸馏）和工具，用于将模型转换为 OpenVINO 格式，以实现更高性能的推理。

通过多个 GPU 优化推理

现在，某些模型在使用 PyTorch 时支持内置的张量并行（TP）。张量并行将模型拆分到多个 GPU 上，从而支持更大规模的模型，并行化诸如矩阵乘法等计算。

要启用张量并行，可以在调用 `from_pretrained()` 函数时传递参数 `tp_plan="auto"`：

```
import os
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "meta-llama/Meta-Llama-3-8B-Instruct"

# 初始化分布式环境
rank = int(os.environ["RANK"])  # 获取当前进程的排名
device = torch.device(f"cuda:{rank}")  # 选择对应的 GPU 设备
torch.distributed.init_process_group("nccl", device_id=device)  # 初始化分布式进程组

# 获取张量并行模型
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    tp_plan="auto",  # 启用自动张量并行
)

# 准备输入数据
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id)  # 加载分词器
prompt = "Can I help"  # 输入提示
inputs = tokenizer(prompt, return_tensors="pt").input_ids.to(device)  # 将输入转换为张量并移动到GPU

# 分布式运行
outputs = model(inputs)
```

你可以使用 `torchrun` 来启动上述脚本，每个进程映射到一个 GPU：

```
torchrun --nproc-per-node 4 demo.py
```

PyTorch 张量并行目前支持以下模型：

- Llama

如果你希望为其他模型添加张量并行支持，可以通过在 GitHub 上创建 Issue 或 Pull Request 来提出请求。

期待的加速效果

在推理时，你可以从显著的加速中受益，尤其是在处理大批量输入或长序列时。

对于 Llama 模型，当序列长度为 512 时，不同批量大小下单次前向传播的预期加速效果如下图所示：

〔图：未命名〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/Meta-Llama-3-8B-Instruct,%20seqlen%20=%20512,%20python,%20w_%20compile.png，离线电子书未嵌入，详见原仓库）

通过 GPU 优化推理

GPU 是机器学习硬件的标准选择，与 CPU 相比，它们在内存带宽和并行性方面进行了优化。为了在现有或较旧的硬件上处理现代模型的更大规模，或加速大型模型的推理，可以使用多种优化方法。在本指南中，您将学习如何使用 FlashAttention-2（更高效的注意力机制）、BetterTransformer（PyTorch 本地快速执行路径）和 bitsandbytes 来量化模型以降低精度。最后，您将学习如何使用 🤗 Optimum 在 Nvidia 和 AMD GPU 上通过 ONNX Runtime 加速推理。

大多数优化方法也适用于多 GPU 配置！

FlashAttention-2

FlashAttention-2 是一种实验性的、更快、更高效的注意力机制实现，它可以通过以下方式显著加速推理：

334. 在序列长度上并行计算注意力。

335. 在 GPU 线程之间分配工作，减少线程之间的通信和共享内存的读写。

FlashAttention-2 当前支持以下架构：

- Bark
- Bart
- Chameleon
- CLIP
- Cohere
- GLM
- Dbrx
- DistilBert
- Gemma
- Gemma2
- GPT2
- GPTBigCode
- GPTNeo
- GPTNeoX
- GPT-J
- Falcon
- Llama
- Llava
- 和更多...

您可以在 GitHub 上请求支持其他模型。

在开始之前，请确保已安装 FlashAttention-2：

```
pip install flash-attn --no-build-isolation
```

要启用 FlashAttention-2，请在加载模型时传递参数 `attn_implementation="flash_attention_2"`：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "tiiuae/falcon-7b"
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
```

FlashAttention-2 只能在模型的 `fp16` 或 `bf16` 精度下使用。确保在使用 FlashAttention-2 之前将模型转换为合适的精度并加载到支持的设备上。

您可以结合其他优化技术（如量化）进一步加速推理：

```
# 8-bit 量化
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    load_in_8bit=True,
    attn_implementation="flash_attention_2",
)

# 4-bit 量化
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    load_in_4bit=True,
    attn_implementation="flash_attention_2",
)
```

期望的加速效果

对于较长的序列，FlashAttention-2 可以显著加速推理。但是，FlashAttention-2 不支持带有填充标记的注意力分数计算，因此在批量推理时需要手动填充/去填充注意力分数，这会导致显著的减速。为了克服这一点，应该在训练期间避免填充标记（例如通过打包数据集或连接序列直到达到最大序列长度）。

对于无填充标记的序列，单次前向传递的加速效果如下图所示：

〔图：falcon-7b 加速效果〕

（原始媒体：https://huggingface.co/datasets/ybelkada/documentation-images/resolve/main/falcon-7b-inference-large-seqlen.png，离线电子书未嵌入，详见原仓库）

对于带有填充标记的序列，加速效果会有所不同：

〔图：llama-2 带填充标记的加速效果〕

（原始媒体：https://huggingface.co/datasets/ybelkada/documentation-images/resolve/main/llama-2-large-seqlen-padding.png，离线电子书未嵌入，详见原仓库）

FlashAttention 更加节省内存，意味着您可以训练更长的序列而不出现内存不足的问题。对于更长的序列，内存使用量最多可减少 20 倍。更多详细信息请参阅 flash-attention 仓库。

PyTorch 缩放点积注意力

PyTorch 的 `torch.nn.functional.scaled_dot_product_attention` (SDPA) 可以调用 FlashAttention 和高效的注意力内核。SDPA 支持正在加入 Transformers，并在 `torch>=2.1.1` 中默认使用。您也可以在加载模型时显式设置 `attn_implementation="sdpa"`。

SDPA 支持的架构包括：

- Albert
- Audio Spectrogram Transformer
- Bart
- Bert
- BioGpt
- CamemBERT
- CLIP
- GLM
- Cohere
- data2vec_audio
- 和更多...

FlashAttention 只能用于 `fp16` 或 `bf16` 精度的模型。SDPA 不支持某些注意力参数，如 `head_mask` 和 `output_attentions=True`。在这种情况下，您会看到警告信息，并会回退到较慢的 eager 实现。

默认情况下，SDPA 会选择最高效的内核，但您可以使用 `torch.backends.cuda.sdp_kernel` 来检查特定设置（硬件、问题大小）下是否可用某个内核：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("facebook/opt-350m")
model = AutoModelForCausalLM.from_pretrained("facebook/opt-350m",
torch_dtype=torch.float16).to("cuda")

input_text = "Hello my dog is cute and"
inputs = tokenizer(input_text, return_tensors="pt").to("cuda")

with torch.backends.cuda.sdp_kernel(enable_flash=True, enable_math=False,
enable_mem_efficient=False):
    outputs = model.generate(**inputs)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

如果您遇到错误，可以尝试使用 PyTorch 的 nightly 版本，以获取更广泛的 FlashAttention 支持。

BetterTransformer

BetterTransformer 是一种优化技术，通过 PyTorch 本地实现的快速执行路径（fastpath）加速推理。快速执行路径的两个优化点是：

336. 融合（fusion），将多个连续的操作合并为一个“内核”以减少计算步骤。

337. 跳过填充标记的固有稀疏性，以避免不必要的计算。

BetterTransformer 还将所有注意力操作转换为更节省内存的缩放点积注意力（SDPA），并在后台调用优化内核（如 FlashAttention）。

在使用 BetterTransformer 之前，请确保已安装 🤗 Optimum：

```
pip install optimum
```

然后可以使用 `PreTrainedModel.to_bettertransformer()` 方法启用 BetterTransformer：

```
model = model.to_bettertransformer()
```

如果您想恢复原始的 🤗 Transformers 模型，可以使用 `reverse_bettertransformer()` 方法：

```
model = model.reverse_bettertransformer()
model.save_pretrained("saved_model")
```

bitsandbytes

bitsandbytes 是一个量化库，支持 4 比特和 8 比特量化。量化可以减少模型的大小，使其更容易在内存有限的 GPU 上运行。

安装 bitsandbytes 和 🤗 Accelerate：

```
pip install bitsandbytes>=0.39.0 accelerate>=0.20.0
pip install transformers
```

4-bit

要以 4 比特加载模型进行推理，使用 `load_in_4bit` 参数。`device_map` 参数是可选的，但建议设置为 `"auto"` 以允许 🤗 Accelerate 自动高效分配模型：

```
from transformers import AutoModelForCausalLM

model_name = "bigscience/bloom-2b5"
model_4bit = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype="auto",
device_map="auto", load_in_4bit=True)
```

对于多 GPU 环境，您可以控制每个 GPU 分配的内存：

```
max_memory_mapping = {0: "600MB", 1: "1GB"}
model_name = "bigscience/bloom-3b"
model_4bit = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype="auto", device_map="auto", load_in_4bit=True,
max_memory=max_memory_mapping
)
```

8-bit

要以 8 比特加载模型进行推理，使用 `load_in_8bit` 参数：

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

model_name = "bigscience/bloom-2b5"
model_8bit = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype="auto",
quantization_config=BitsAndBytesConfig(load_in_8bit=True))
```

对于多 GPU 环境，您可以控制每个 GPU 分配的内存：

```
max_memory_mapping = {0: "1GB", 1: "2GB"}
model_name = "bigscience/bloom-3b"
model_8bit = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype="auto", device_map="auto", load_in_8bit=True,
max_memory=max_memory_mapping
)
```

🤗 Optimum

🤗 Optimum 支持使用 ONNX Runtime（ORT）加速 Nvidia 和 AMD GPU 上的推理。ORT 使用诸如将常见操作融合为单个节点和常量折叠等优化技术，以减少计算步骤并加速推理。ORT 还将最密集的计算任务分配给 GPU，其余任务分配给 CPU，以智能分配工作负载。

要使用 ORT，您需要使用一个特定任务的 ORTModel，并指定 `provider` 参数，例如 `CUDAExecutionProvider`、`ROCMExecutionProvider` 或 `TensorrtExecutionProvider`。如果要导出模型为 ONNX 格式，可以设置 `export=True`。

```
from optimum.onnxruntime import ORTModelForSequenceClassification

ort_model = ORTModelForSequenceClassification.from_pretrained(
  "distilbert/distilbert-base-uncased-finetuned-sst-2-english",
  export=True,
  provider="CUDAExecutionProvider",
)

from optimum.pipelines import pipeline
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased-finetuned-sst-2-
english")

pipeline = pipeline(task="text-classification", model=ort_model, tokenizer=tokenizer,
device="cuda:0")
result = pipeline("Both the music and visual were astounding, not to mention the actors
performance.")
```

组合优化

可以组合多种优化技术以获得最佳的推理性能。例如，您可以加载 4 比特量化模型并启用 BetterTransformer 和 FlashAttention：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# 加载 4 比特模型
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16
)

tokenizer = AutoTokenizer.from_pretrained("facebook/opt-350m")
```

```
model = AutoModelForCausalLM.from_pretrained("facebook/opt-350m", torch_dtype="auto",
quantization_config=quantization_config)

# 启用 BetterTransformer
model = model.to_bettertransformer()

input_text = "Hello my dog is cute and"
inputs = tokenizer(input_text, return_tensors="pt").to("cuda")

# 启用 FlashAttention
with torch.backends.cuda.sdp_kernel(enable_flash=True, enable_math=False,
enable_mem_efficient=False):
    outputs = model.generate(**inputs)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

第七部分 概念指南

哲学理念

😭 Transformers 是一个具有明确目标的库，致力于服务以下几类人：

- 机器学习研究人员和教育工作者，他们希望使用、研究或扩展大规模的 Transformer 模型。
- 实践者，他们希望微调这些模型或在生产环境中部署这些模型，或者两者兼而有之。
- 工程师，他们只想下载预训练模型并用它来解决特定的机器学习任务。

该库的设计基于两个核心目标：

338. **尽可能简单易用**：

    - 我们尽量减少用户需要学习的抽象概念。实际上，使用每个模型只需了解三个标准类：
        - 配置类：存储模型的超参数。
        - 模型类：实现具体的模型。
        - 预处理类：根据任务不同，分为文本的分词器、图像的图像处理器、音频的特征提取器以及多模态输入的处理器。

- 所有这些类都可以通过一个通用的 `from_pretrained()` 方法从预训练实例简单且统一地初始化。这个方法会从 Hugging Face Hub 或用户自己的保存点下载、缓存并加载相关的类实例和关联数据（如配置的超参数、分词器的词汇表和模型的权重）。
- 在这三个基础类之上，库提供了两个 API：
  - pipeline()：用于快速在特定任务上进行推理。
  - Trainer：用于快速训练或微调 PyTorch 模型（所有 TensorFlow 模型都兼容 `Keras.fit`）。
- 因此，此库并不是一个用于构建神经网络的模块化工具箱。如果你想扩展或基于此库构建新功能，可以使用常规的 Python、PyTorch、TensorFlow 或 Keras 模块，继承库的基础类并重用模型加载和保存等功能。如果你想了解有关模型编码哲学的更多信息，可以查看我们的 重复你自己 博客文章。

339. **提供尽可能接近原版模型的最先进模型**：
    - 我们为每种架构至少提供一个示例，以重现官方作者提供的结果。
    - 代码尽可能接近原始代码库，这意味着一些 PyTorch 代码可能不如原生 PyTorch 代码“地道”，因为它们是从 TensorFlow 代码转换而来的，反之亦然。

其他目标还包括：

- **尽可能一致地暴露模型的内部结构**：
  - 使用单一 API 提供对完整隐藏状态和注意力权重的访问。
  - 标准化预处理类和基础模型 API，以便轻松切换模型。
- **集成有前景的微调和研究工具**：
  - 提供简单一致的方法来添加新词汇和嵌入以进行微调。
  - 提供简单方法来屏蔽和修剪 Transformer 头。
- **轻松切换 PyTorch、TensorFlow 2.0 和 Flax**，允许使用一个框架进行训练，用另一个框架进行推理。

核心概念

该库围绕每个模型的三类类构建：

- **模型类**：可以是 PyTorch 模型(torch.nn.Module)、Keras 模型(tf.keras.Model) 或 JAX/Flax 模型(flax.linen.Module)，它们使用库中提供的预训练权重。
- **配置类**：存储构建模型所需的超参数（如层数、隐藏单元大小）。如果使用的是预训练模型而没有修改，创建模型时会自动实例化配置，因此你不一定需要手动实例化这些类。
- **预处理类**：将原始数据转换为模型可接受的格式。分词器存储每个模型的词汇表，并提供将字符串编码为标记嵌入索引列表的方法。图像处理器预处理视觉输入，特征提取器预处理音频输入，处理器处理多模态输入。

所有这些类都可以通过以下三种方法实例化、保存本地并共享到 Hub：

- `from_pretrained()`：从预训练版本实例化模型、配置和预处理类。预训练模型可以来自库本身支持的模型（可在模型库中找到），也可以是用户本地保存的模型。
- `save_pretrained()`：将模型、配置和预处理类保存到本地，以便使用 `from_pretrained()` 重新加载。
- `push_to_hub()`：将模型、配置和预处理类共享到 Hub，以便他人轻松访问。

专业术语

此术语表定义了一些通用的机器学习和 🤗 Transformers 术语，帮助您更好地理解文档。

A

注意力掩码 (attention mask)

当批量处理序列时，注意力掩码是一个可选参数。它告诉模型哪些标记（tokens）应该被关注，哪些不应该被关注。

例如，考虑以下两个序列：

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-cased")
sequence_a = "This is a short sequence."
sequence_b = "This is a rather long sequence. It is at least longer than the sequence A."
```

```
encoded_sequence_a = tokenizer(sequence_a)["input_ids"]
encoded_sequence_b = tokenizer(sequence_b)["input_ids"]
```

编码后的版本长度不同：

```
len(encoded_sequence_a), len(encoded_sequence_b)  # 输出：(8, 19)
```

因此，我们无法将它们直接放入同一个张量中。第一序列需要填充到第二序列的长度，或者第二序列需要截断到第一序列的长度。

在第一种情况下，ID 列表将通过填充索引进行扩展。我们可以将一个列表传递给分词器并要求它进行填充：

```
padded_sequences = tokenizer([sequence_a, sequence_b], padding=True)
```

我们看到第一个句子的右侧添加了 0，使其与第二个句子长度相同：

```
padded_sequences["input_ids"]
# 输出：[[101, 1188, 1110, 170, 1603, 4954, 119, 102, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
#        [101, 1188, 1110, 170, 1897, 1263, 4954, 119, 1135, 1110, 1120, 1655, 2039, 1190, 1103,
4954, 138, 119, 102]]
```

然后，这可以转换为 PyTorch 或 TensorFlow 的张量。注意力掩码是一个二进制张量，指示填充索引的位置，以便模型不会关注这些位置。对于 `BertTokenizer`，`1` 表示应被关注的值，而 `0` 表示填充值。注意力掩码在分词器返回的字典中键为 “attention_mask”：

```
padded_sequences["attention_mask"]
# 输出：[[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
#        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]
```

自编码模型 (autoencoding models)

请参阅 编码器模型 和 遮蔽语言模型

自回归模型 (autoregressive models)

请参阅 因果语言模型 和 解码器模型

B

主干 (backbone)

主干是输出原始隐藏状态或特征的网络（嵌入和层）。它通常连接到一个 头部，该头部接受特征作为输入以进行预测。例如，`ViTModel` 是一个没有特定头部的主干。其他模型也可以使用 `ViTModel` 作为主干，例如 DPT。

C

因果语言模型 (causal language modeling)

一种预训练任务，模型按顺序阅读文本，并预测下一个单词。这通常通过阅读整个句子但使用掩码在特定时间步隐藏未来的标记来完成。

通道 (channel)

彩色图像由三个通道的值组合而成：红色、绿色和蓝色（RGB），而灰度图像只有一个通道。在 🤗 Transformers 中，通道可以是图像张量的第一个或最后一个维度：`[n_channels, height, width]` 或 `[height, width, n_channels]`。

时序分类 (connectionist temporal classification, CTC)

一种算法，允许模型在不知道输入和输出之间确切对齐的情况下进行学习。CTC 计算给定输入的所有可能输出的分布，并从中选择最可能的输出。CTC 常用于语音识别任务，因为语音并不总是与转录本整洁地对齐，原因包括说话者的不同语速等。

卷积 (convolution)

神经网络中的一种层，输入矩阵与一个更小的矩阵（内核或过滤器）逐元素相乘，并在新矩阵中汇总值。这个操作称为卷积操作，并在整个输入矩阵上重复进行。每次操作应用于输入矩阵的不同部分。卷积神经网络（CNN）常用于计算机视觉。

D

数据并行 (DataParallel, DP)

一种用于在多个 GPU 上训练的技术，其中相同的设置被复制多次，每个实例接收不同的数据切片。处理过程并行进行，并在每个训练步骤结束时同步所有设置。

了解更多关于 DataParallel 的工作原理 请点击这里。

解码器输入 ID (decoder input IDs)

这是编码器-解码器模型特有的输入，包含将被传递给解码器的输入 ID。这些输入应用于序列到序列任务，如翻译或摘要，通常每种模型有自己的特定构建方式。

大多数编码器-解码器模型（如 BART 和 T5）会从 `labels` 自动生成 `decoder_input_ids`。在这些模型中，传递 `labels` 是处理训练的首选方式。

请查阅每种模型的文档，以了解它们如何处理这些输入 ID 的序列到序列训练。

解码器模型 (decoder models)

也称为自回归模型，解码器模型涉及一个预训练任务（称为因果语言模型），其中模型按顺序阅读文本并预测下一个单词。这通常通过阅读整个句子但使用掩码在特定时间步隐藏未来的标记来完成。

深度学习 (deep learning, DL)

使用多层神经网络的机器学习算法。

E

编码器模型 (encoder models)

也称为自编码模型，编码器模型将输入（如文本或图像）转换为一个更紧凑的数值表示，称为嵌入。通常，编码器模型使用技术如 遮蔽语言模型 进行预训练，该技术通过遮蔽输入序列的部分内容来强迫模型创建更有意义的表示。

F

特征提取 (feature extraction)

将原始数据选择和转换为对机器学习算法更有信息量和有用的一组特征的过程。例如，将原始文本转换为词嵌入，或从图像/视频数据中提取重要的特征如边缘或形状。

前馈分块 (feed forward chunking)

在每个残差注意力块中，自注意力层通常后面跟着两个前馈层。前馈层的中间嵌入大小通常大于模型的隐藏大小（例如对于 `google-bert/bert-base-uncased`）。

对于大小为 `[batch_size, sequence_length]` 的输入，存储中间前馈嵌入 `[batch_size, sequence_length, config.intermediate_size]` 所需的内存可能占总内存使用量的很大一部分。《Reformer: The Efficient Transformer》的作者注意到，由于计算与 `sequence_length` 维度无关，因此可以等效地计算两个前馈层的输出嵌入 `[batch_size, config.hidden_size]_0, ..., [batch_size, config.hidden_size]_n`，然后将其连接成 `[batch_size, sequence_length, config.hidden_size]`，其中 `n = sequence_length`，这以增加计算时间为代价来减少内存使用，但产生数学上 **等价** 的结果。

对于使用函数 apply_chunking_to_forward() 的模型，`chunk_size` 定义了并行计算的输出嵌入数量，从而定义了内存和时间复杂度之间的权衡。如果 `chunk_size` 设置为 0，则不进行前馈分块。

精调模型 (finetuned models)

精调（finetuning）是一种迁移学习的形式，涉及取一个预训练模型，冻结其权重，并替换输出层为新添加的 模型头部。模型头部在目标数据集上进行训练。

有关如何使用 🤗 Transformers 进行精调的更多信息，请参阅 精调预训练模型 教程。

H

头部 (head)

头部是神经网络的最后一层，它接受原始隐藏状态并将其映射到不同维度。每个任务都有不同的头部。例如：

- GPT2ForSequenceClassification 是一个序列分类头部——在基础 GPT2Model 之上添加的一个线性层。
- ViTForImageClassification 是一个图像分类头部——在基础 ViTModel 的 `CLS` 令牌最终隐藏状态之上添加的一个线性层。
- Wav2Vec2ForCTC 是一个带 CTC 的语言建模头部——在基础 Wav2Vec2Model 之上。

I

图像块 (image patch)

基于视觉的 Transformer 模型将图像分割成更小的块，这些块被线性嵌入，然后作为序列传递给模型。您可以在模型配置中找到 `patch_size` 或分辨率。

推理 (inference)

推理是在训练完成后对新数据进行模型评估的过程。有关如何使用 🤗 Transformers 进行推理的更多信息，请参阅 推理管道 教程。

输入 ID (input IDs)

输入 ID 通常是传递给模型的唯一必需参数。它们是标记索引，是构建序列的标记的数值表示，这些序列将被用作模型的输入。

每个分词器的工作方式不同，但底层机制相同。以下是一个使用 BERT 分词器（基于 WordPiece 的分词器）的例子：

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-cased")
```

```
sequence = "A Titan RTX has 24GB of VRAM"

# 分词器将序列拆分为分词器词汇表中的标记
tokenized_sequence = tokenizer.tokenize(sequence)

# 这些标记可以是单词或子单词。例如，“VRAM”不在模型词汇表中，因此它被拆分为“V”，“RA”和“M”
print(tokenized_sequence)  # 输出：['A', 'Titan', 'R', '##T', '##X', 'has', '24', '##GB', 'of',
'V', '##RA', '##M']

# 将标记转换为 ID，这是模型可以理解的
inputs = tokenizer(sequence)

# 分词器返回一个包含所有必需参数的字典，其中标记索引在键 `input_ids` 下
encoded_sequence = inputs["input_ids"]
print(encoded_sequence)  # 输出：[101, 138, 18696, 155, 1942, 3190, 1144, 1572, 13745, 1104, 159,
9664, 2107, 102]

# 分词器会自动添加特殊标记（如果关联模型需要）
decoded_sequence = tokenizer.decode(encoded_sequence)
print(decoded_sequence)  # 输出：[CLS] A Titan RTX has 24GB of VRAM [SEP]
```

L

标签 (labels)

标签是可选参数，用于让模型计算损失。这些标签应该是模型的预期预测值。它将使用标准损失来计算其预测值与预期值（标签）之间的损失。不同模型的标签不同。例如：

- 对于序列分类模型（如 BertForSequenceClassification），模型期望一个维度为 `(batch_size)` 的张量，每个值对应整个序列的预期标签。
- 对于标记分类模型（如 BertForTokenClassification），模型期望一个维度为 `(batch_size, seq_length)` 的张量，每个值对应每个标记的预期标签。
- 对于遮蔽语言模型（如 BertForMaskedLM），模型期望一个维度为 `(batch_size, seq_length)` 的张量，每个值对应每个标记的预期标签：标签是遮蔽标记的标记 ID，其余值通常为 -100。
- 对于序列到序列任务（如 BartForConditionalGeneration 和 MBartForConditionalGeneration），模型期望一个维度为 `(batch_size, tgt_seq_length)` 的张量，每个值对应每个输入序列的目标序列。在训练期间，BART 和 T5 会内部生成适当的

`decoder_input_ids` 和解码器注意力掩码。它们通常不需要提供。这不适用于使用编码器-解码器框架的模型。

- 对于图像分类模型（如 ViTForImageClassification），模型期望一个维度为 `(batch_size)` 的张量，每个值对应每个图像的预期标签。
- 对于语义分割模型（如 SegformerForSemanticSegmentation），模型期望一个维度为 `(batch_size, height, width)` 的张量，每个值对应每个像素的预期标签。
- 对于目标检测模型（如 DetrForObjectDetection），模型期望一个包含 `class_labels` 和 `boxes` 键的字典列表，每个值对应每个图像的预期标签和边界框数量。
- 对于自动语音识别模型（如 Wav2Vec2ForCTC），模型期望一个维度为 `(batch_size, target_length)` 的张量，每个值对应每个标记的预期标签。

每个模型的标签可能不同，因此请确保始终检查每个模型的文档以了解更多关于其特定标签的信息！

基础模型（如 BertModel）不接受标签，因为这些是基础 Transformer 模型，只输出特征。

大型语言模型 (Large Language Models, LLM)

指在大量数据上训练的 Transformer 语言模型（如 GPT-3、BLOOM、OPT）。这些模型通常拥有大量可学习参数（如 GPT-3 拥有 1750 亿个参数）。

M

遮蔽语言建模 (Masked Language Modeling, MLM)

一种预训练任务，其中模型看到的是经过破坏的文本，通常通过随机遮蔽一些标记来完成，然后需要预测原始文本。

多模态 (multimodal)

结合文本和其他输入（如图像）的任务。

N

自然语言生成 (Natural Language Generation, NLG)

所有与生成文本相关的任务（例如，使用 Transformers 生成、翻译）。

自然语言处理 (Natural Language Processing, NLP)

处理文本的通用术语。

自然语言理解 (Natural Language Understanding, NLU)

所有与理解文本内容相关的任务（例如，对整个文本或单个单词进行分类）。

P

管道 (pipeline)

在 🤗 Transformers 中，管道是指一系列按特定顺序执行的步骤，用于预处理和转换数据并返回模型的预测。管道中的一些示例阶段包括数据预处理、特征提取和归一化。

有关更多信息，请参阅 推理管道。

管道并行 (PipelineParallel, PP)

一种并行技术，其中模型按层级别垂直拆分到多个 GPU 上，使得每个 GPU 只包含一个或几个层。每个 GPU 并行处理不同的管道阶段，并处理小批量的数据块。了解更多关于 PipelineParallel 的工作原理 请点击这里。

像素值 (pixel values)

传递给模型的图像的数值表示的张量。像素值的形状为 `[batch_size, num_channels, height, width]`，由图像处理器生成。

池化 (pooling)

将矩阵缩小为更小矩阵的操作，通常通过取池化维度的最大值或平均值来完成。池化层通常位于卷积层之间，用于降采样特征表示。

位置 ID (position IDs)

与 RNN 不同，Transformers 对每个标记的位置没有内置的感知。因此，模型使用位置 ID (`position_ids`) 来标识每个标记在标记列表中的位置。

这是一个可选参数。如果没有传递 `position_ids`，它们会自动创建为绝对位置嵌入。

绝对位置嵌入在范围 `[0, config.max_position_embeddings - 1]` 中选择。一些模型使用其他类型的位置嵌入，如正弦位置嵌入或相对位置嵌入。

预处理 (preprocessing)

将原始数据准备成机器学习模型可以轻松使用的格式的任务。例如，文本通常通过分词进行预处理。有关其他输入类型预处理的更多信息，请参阅 预处理 教程。

预训练模型 (pretrained model)

在某些数据（如维基百科的全部内容）上预训练的模型。预训练方法涉及自监督目标，可以是阅读文本并尝试预测下一个单词（见 因果语言模型），或者遮蔽一些单词并尝试预测它们（见 遮蔽语言建模）。

语音和视觉模型也有自己的预训练目标。例如，Wav2Vec2 是一个预先训练的语音模型，它通过对比任务进行训练，要求模型从一组“错误”的语音表示中识别出“真实”的语音表示。另一方面，BEiT 是一个预先训练的视觉模型，它通过掩码图像建模任务进行训练，该任务遮蔽图像的一些补丁并要求模型预测遮蔽的补丁（类似于遮蔽语言建模目标）。

R

循环神经网络 (Recurrent Neural Network, RNN)

一种通过层上的循环来处理文本的模型。

表征学习 (representation learning)

机器学习的一个子领域，专注于从原始数据中学习有意义的表征。一些表征学习技术的例子包括词嵌入、自动编码器和生成对抗网络（GANs）。

S

采样率 (sampling rate)

每秒采样的次数（以赫兹为单位）。采样率是将连续信号（如语音）离散化后的结果。

自注意力 (self-attention)

输入的每个元素确定它们应该关注输入的哪些其他元素。

自监督学习 (self-supervised learning)

一种机器学习技术，模型从无标签数据中创建自己的学习目标。它与 无监督学习 和 监督学习 不同，因为学习过程是受监督的，但不是由用户显式指定。

一个自监督学习的例子是 遮蔽语言建模，其中模型被传递了部分标记被移除的句子，并学习预测缺失的标记。

半监督学习 (semi-supervised learning)

一种机器学习训练技术，利用少量有标签数据和大量无标签数据来提高模型的准确性。这与 监督学习 和 无监督学习 不同。

例如，“自训练”是一种半监督学习方法，模型首先在有标签数据上进行训练，然后用于对无标签数据进行预测。模型预测最自信的部分无标签数据将被添加到有标签数据集中，以重新训练模型。

序列到序列 (sequence-to-sequence, seq2seq)

生成新序列的模型，如翻译模型或摘要模型（如 Bart 或 T5）。

分片 DDP (Sharded DDP)

另一种实现 ZeRO 概念的方式，由各种 ZeRO 实现使用。

步长 (stride)

在 卷积 或 池化 中，步长是指内核在矩阵上移动的距离。步长为 1 表示内核每次移动一个像素，步长为 2 表示内核每次移动两个像素。

监督学习 (supervised learning)

一种直接使用带标签的数据来纠正和指导模型性能的训练方法。数据被输入到正在训练的模型中，其预测与已知标签进行比较。模型根据预测的错误程度更新权重，并重复该过程以优化模型性能。

T

张量并行 (Tensor Parallelism, TP)

一种在多个 GPU 上训练的并行技术，其中每个张量被拆分为多个块，因此张量的每个分片都驻留在指定的 GPU 上。分片被分别并行处理，然后在处理步骤结束时同步结果。这有时称为水平并行，因为拆分发生在水平层。了解更多关于张量并行的原理 请点击这里。

令牌 (token)

句子的一部分，通常是单词，也可以是子单词（不常见的单词通常被拆分为子单词）或标点符号。

令牌类型 ID (token Type IDs)

有些模型的任务是执行句子对分类或问答。

这些任务需要两个不同的序列连接成一个“input_ids”条目，这通常通过特殊标记如分类器 (`[CLS]`) 和分隔符 (`[SEP]`) 来完成。例如，BERT 模型构建两个序列输入如下：

```
# [CLS] SEQUENCE_A [SEP] SEQUENCE_B [SEP]
```

我们可以使用分词器自动生成这样的句子，将两个序列作为两个参数传递给 `tokenizer`（而不是列表）：

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-cased")
sequence_a = "HuggingFace is based in NYC"
sequence_b = "Where is HuggingFace based?"

encoded_dict = tokenizer(sequence_a, sequence_b)
decoded = tokenizer.decode(encoded_dict["input_ids"])
print(decoded)
# 输出：[CLS] HuggingFace is based in NYC [SEP] Where is HuggingFace based? [SEP]
```

这已经足够让某些模型理解一个序列在哪里结束，另一个序列在哪里开始。然而，其他模型，如 BERT，还使用令牌类型 ID（也称为段 ID）。它们是一个二进制掩码，标识模型中的两种序列类型。

分词器返回该掩码作为 “token_type_ids” 条目：

```
encoded_dict["token_type_ids"]
# 输出：[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

第一个序列（“上下文”）的所有标记用 `0` 表示，而第二个序列（“问题”）的所有标记用 `1` 表示。

一些模型，如 XLNetModel，使用一个额外的标记，用 `2` 表示。

迁移学习 (transfer learning)

一种技术，涉及将预训练模型适应特定于您任务的数据集。与其从头训练模型，不如利用现有模型中获得的知识作为起点。这加速了学习过程，并减少了所需的训练数据量。

Transformer

基于自注意力的深度学习模型架构。

U

无监督学习 (unsupervised learning)

一种模型训练方法，提供给模型的数据没有标签。无监督学习技术利用数据分布的统计信息来发现对任务有用的数据模式。

Z

零冗余优化器 (Zero Redundancy Optimizer, ZeRO)

一种并行技术，与 张量并行 类似，除了在前向或后向计算时重建整个张量，因此模型不需要修改。该方法还支持各种卸载技术以弥补有限的 GPU 内存。了解更多关于 ZeRO 的原理 请点击这里。

Transformers 库能做什么

🤗 Transformers 是一个包含预训练最先进模型的库，可用于自然语言处理（NLP）、计算机视觉和音频及语音处理任务。该库不仅包含 Transformer 模型，还包括用于计算机视觉任务的现代卷积网络。如今，许多流行的产品，如智能手机、应用程序和电视，都依赖于某种深度学习技术。想从手机拍摄的照片中移除背景物体吗？这就是全景分割任务的一个例子（如果您还不了解这指的是什么，别担心，我们将在下面的章节中详细介绍）。

本页面概述了使用 🤗 Transformers 库，只需三行代码即可解决的音频和语音、计算机视觉以及 NLP 任务。

音频

音频和语音处理任务与其他模态任务有所不同，主要是因为音频作为输入是一个连续信号。与文本不同，原始音频波形无法像句子被分成词那样被整齐地分割成离散块。为了解决这个问题，通常会在固定的时间间隔内对原始音频信号进行采样。在相同时间间隔内采样次数越多，采样率越高，音频越接近原始音频源。

以前的方法会预先处理音频以提取有用特征。现在，更常见的是直接将原始音频波形输入特征编码器，以提取音频表示。这简化了预处理步骤，并使模型能够学习最重要的特征。

音频分类

音频分类任务是根据预定义的类别对音频数据进行标注。这是一个广泛的任务，具体应用包括：

- **声景分类**：给音频标注场景标签（例如“办公室”、“海滩”、“体育场”）

- **声学事件检测**：给音频标注声音事件标签（例如“汽车喇叭声”、“鲸鱼叫声”、“玻璃破碎声”）
- **标签**：给包含多个声音的音频标注（例如鸟鸣声、会议中的发言人识别）
- **音乐分类**：根据音乐类型给音乐分类（例如“金属”、“嘻哈”、“乡村”）

```
from transformers import pipeline

classifier = pipeline(task="audio-classification", model="superb/hubert-base-superb-er")
preds = classifier("https://huggingface.co/datasets/Narsil/asr_dummy/resolve/main/mlk.flac")
preds = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in preds]
```

自动语音识别

自动语音识别（ASR）将语音转录为文本。由于语音是人类自然的交流方式，因此 ASR 是最普遍的音频任务之一。如今，ASR 系统已嵌入到智能扬声器、手机和汽车等产品中。我们可以要求虚拟助手播放音乐、设置提醒并告知天气。

Transformer 架构有助于解决低资源语言的问题。通过在大量语音数据上进行预训练，仅使用一个小时内标签化的低资源语言语音数据进行微调，就能获得高质量的结果，而以前的 ASR 系统需要 100 倍以上的标签化数据。

```
from transformers import pipeline

transcriber = pipeline(task="automatic-speech-recognition", model="openai/whisper-small")
transcriber("https://huggingface.co/datasets/Narsil/asr_dummy/resolve/main/mlk.flac")
```

计算机视觉

计算机视觉领域的早期成功任务之一是使用卷积神经网络（CNN）识别邮政编码数字。图像由像素组成，每个像素有一个数值。这使得图像可以表示为像素值的矩阵。每个特定的像素值组合描述了图像的颜色。

解决计算机视觉任务的两种常见方法是：

340. 使用卷积来学习图像的分层特征，从低级特征到高级抽象特征。
341. 将图像分成块，并使用 Transformer 逐步学习每个图像块之间的关系，从而形成图像。与 CNN 偏好的自底向上方法不同，这种方法更像是从模糊图像开始，然后逐渐聚焦。

图像分类

图像分类是对整张图像进行预定义类别标注。像大多数分类任务一样，图像分类有许多实际应用场景，例如：

- **医疗保健**：标注医学图像以检测疾病或监测患者健康
- **环境**：标注卫星图像以监测森林砍伐、指导荒地管理或检测野火
- **农业**：标注作物图像以监测植物健康或卫星图像进行土地利用监测
- **生态学**：标注动植物图像以监测野生动物数量或跟踪濒危物种

```
from transformers import pipeline

classifier = pipeline(task="image-classification")
preds = classifier("https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/pipeline-cat-chonk.jpeg")
preds = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in preds]
print(*preds, sep="\n")
```

目标检测

与图像分类不同，目标检测识别图像中的多个对象及其在图像中的位置（通过边界框定义）。目标检测的应用示例包括：

- **自动驾驶车辆**：检测日常交通物体，如其他车辆、行人和交通灯
- **遥感**：灾难监测、城市规划和天气预报
- **缺陷检测**：检测建筑物中的裂缝或结构损坏以及制造缺陷

```
from transformers import pipeline

detector = pipeline(task="object-detection")
preds = detector("https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/pipeline-cat-chonk.jpeg")
preds = [{"score": round(pred["score"], 4), "label": pred["label"], "box": pred["box"]} for pred in preds]
```

图像分割

图像分割是一项像素级任务，它将图像中的每个像素分配给一个类别。与目标检测使用边界框标注和预测图像中的对象不同，分割更细致，可以检测到像素级别的对象。图像分割有几种类型：

- **实例分割**：除了标注对象类别，还标注每个独立实例（例如“狗-1”、“狗-2”）

- **全景分割**：结合语义分割和实例分割；它为每个像素标注语义类别**和**每个独立实例的对象

分割任务对自动驾驶车辆很有帮助，可以为车辆创建周围世界的像素级地图，以安全地避开行人和其他车辆。在医疗成像中，更细致的分割可以识别异常细胞或器官特征。图像分割也可以用于电子商务，通过覆盖虚拟对象来虚拟试穿衣物或创建增强现实体验。

```
from transformers import pipeline

segmenter = pipeline(task="image-segmentation")
preds = segmenter("https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/pipeline-cat-chonk.jpeg")
preds = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in preds]
print(*preds, sep="\n")
```

深度估计

深度估计预测图像中每个像素到相机的距离。这项计算机视觉任务对于场景理解和重建非常重要。例如，在自动驾驶车辆中，车辆需要理解行人、交通标志和其他车辆的距离，以避免障碍物和碰撞。深度信息对于从 2D 图像构建 3D 表示也非常有用，可以用于创建高质量的生物结构或建筑物的 3D 表示。

深度估计有两种方法：

- **立体方法**：通过比较从不同角度拍摄的两张图像来估计深度。
- **单目方法**：从单张图像估计深度。

```
from transformers import pipeline

depth_estimator = pipeline(task="depth-estimation")
depth_estimator("https://huggingface.co/datasets/huggingface/documentation-
images/resolve/main/pipeline-cat-chonk.jpeg")
```

自然语言处理

NLP 任务是最常见的任务之一，因为文本是人类自然的交流方式之一。为了将文本转换为模型可以识别的格式，需要对其进行分词。这意味着将一段文本分成单独的词语或子词（tokens），然后将这些 tokens 转换为数字。结果，你可以将一段文本表示为数字序列，一旦有了数字序列，就可以将其输入模型以解决各种 NLP 任务！

文本分类

类似于其他模态的分类任务，文本分类根据预定义的类别对文本序列（可以是句子、段落或文档）进行标注。文本分类有许多实际应用，例如：

- **情感分析**：根据一些极性（如“积极”或“消极”）对文本进行标注，这可以帮助政治、金融和营销领域的决策。
- **内容分类**：根据某些主题（如“天气”、“体育”、“金融”等）对文本进行标注，以帮助整理和过滤新闻和社交媒体信息。

```
from transformers import pipeline

classifier = pipeline(task="sentiment-analysis")
preds = classifier("Hugging Face 是自切片面包以来最好的东西！")
preds = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in preds]
```

词分类

在任何 NLP 任务中，文本会通过将文本序列分为单独的词或子词（tokens）进行预处理。词分类将每个 token 分配一个预定义类别中的标签。

常见的词分类任务包括：

- **命名实体识别（NER）**：根据实体类别（如组织、人名、地点或日期）标注 token。NER 在生物医学领域中非常受欢迎，可以标注基因、蛋白质和药物名称。
- **词性标注（POS）**：根据语法类别（如名词、动词或形容词）标注 token。POS 对帮助翻译系统理解两个相同单词之间的语法差异（如名词“银行”与动词“银行”）非常有用。

```
from transformers import pipeline

classifier = pipeline(task="ner")
preds = classifier("Hugging Face 是一家总部位于纽约市的法国公司。")
preds = [
    {
        "entity": pred["entity"],
        "score": round(pred["score"], 4),
        "index": pred["index"],
        "word": pred["word"],
        "start": pred["start"],
        "end": pred["end"],
    }
    for pred in preds
```

```
]
print(*preds, sep="\n")
```

问答

问答是另一个 token 级别的任务，它根据问题返回答案，有时带有上下文（开放域）有时不带上下文（封闭域）。当我们在虚拟助手处询问餐厅是否营业时，就会用到问答任务。它还可以用于提供客户服务或技术支持，并帮助搜索引擎检索相关信息。

常见的问答类型有：

- **抽取式**：给定问题和一些上下文，答案是从上下文中提取的文本片段。
- **生成式**：给定问题和一些上下文，答案是从上下文中生成的。此方法由 Text2TextGenerationPipeline 处理，而不是 QuestionAnsweringPipeline。

```
from transformers import pipeline

question_answerer = pipeline(task="question-answering")
preds = question_answerer(
    question="仓库的名称是什么？",
    context="仓库的名称是 huggingface/transformers",
)
print(
    f"score: {round(preds['score'], 4)}, start: {preds['start']}, end: {preds['end']}, answer:
{preds['answer']}"
)
```

摘要生成

摘要生成创建一个较短版本的文本，同时尽量保留原始文档的大部分含义。摘要是序列到序列任务，其输出文本序列短于输入。许多长文档可以被概括以帮助读者快速了解主要信息，例如立法法案、法律和金融文件、专利和科学论文。

与问答类似，摘要生成也有两种类型：

- **抽取式**：从原始文本中识别并提取最重要的句子。
- **生成式**：从原始文本中生成目标摘要（可能包含输入文档中不存在的新词）。SummarizationPipeline 使用生成式方法。

```
from transformers import pipeline

summarizer = pipeline(task="summarization")
```

```
summarizer(
    "在本文中，我们介绍了 Transformer，这是一种完全基于注意力机制的序列转换模型，取代了编码器-解码器架构中最常用的循环层，使用多头自注意力。对于翻译任务，Transformer 可以显著更快地进行训练，优于基于循环或卷积层的架构。在 WMT 2014 英语到德语和 WMT 2014 英语到法语的翻译任务中，我们达到了新的最先进水平。在前一个任务中，我们的最佳模型优于所有先前报告的集成模型。"
)
```

翻译

翻译将一种语言的文本转换为另一种语言。它在帮助来自不同背景的人交流、帮助翻译内容以到达更广泛的受众以及作为学习工具帮助人们学习新语言方面非常重要。与摘要生成一样，翻译也是序列到序列任务，意味着模型接收输入序列并返回目标输出序列。

早期，翻译模型主要是单语的，但近年来，对多语言模型的兴趣不断增加，这些模型可以在多种语言对之间进行翻译。

```
from transformers import pipeline

text = "将英文翻译成法文：Hugging Face 是一个以社区为基础的开源机器学习平台。"
translator = pipeline(task="translation", model="google-t5/t5-small")
translator(text)
```

语言建模

语言建模任务是预测文本序列中的一个词语。由于预训练语言模型可以通过微调解决许多其他下游任务，因此它已成为非常流行的 NLP 任务。最近，大型语言模型（LLMs）引起了广泛关注，它们可以实现零样本或少样本学习，这意味着模型可以解决它没有专门训练过的任务！语言模型可以生成流畅和有说服力的文本，但需要注意的是，生成的文本并不总是准确的。

语言建模有两种类型：

- **因果式**：模型的目标是预测序列中的下一个 token，未来的 token 被遮盖。

```
from transformers import pipeline

prompt = "Hugging Face 是一个以社区为基础的开源机器学习平台。"
generator = pipeline(task="text-generation")
generator(prompt)  # doctest: +SKIP
```

- **遮盖式**：模型的目标是预测序列中被遮盖的 token，完全访问序列中的所有 token。

```
from transformers import pipeline

text = "Hugging Face 是一个以社区为基础的开源 &lt;mask&gt; 机器学习。"
fill_mask = pipeline(task="fill-mask")
preds = fill_mask(text, top_k=1)
preds = [
    {
        "score": round(pred["score"], 4),
        "token": pred["token"],
        "token_str": pred["token_str"],
        "sequence": pred["sequence"],
    }
    for pred in preds
]
```

多模态

多模态任务需要模型处理多种数据模态（文本、图像、音频、视频）以解决特定问题。图像字幕生成就是一个多模态任务的例子，模型以图像作为输入，输出描述图像或图像属性的文本序列。

尽管多模态模型处理不同类型的数据，但在内部，预处理步骤帮助模型将所有数据类型转换为嵌入（向量或数字列表，包含有关数据的有意义信息）。对于图像字幕生成任务，模型学习图像嵌入和文本嵌入之间的关系。

文档问答

文档问答任务是根据文档回答自然语言问题。与基于文本的问答任务不同，文档问答以文档图像作为输入，并附带关于文档的问题，以返回答案。文档问答可以用于解析结构化文档并从中提取关键信息。例如，可以从收据中提取总金额和找零金额。

```
from transformers import pipeline
from PIL import Image
import requests

url = "https://huggingface.co/datasets/hf-internal-testing/example-
documents/resolve/main/jpeg_images/2.jpg"
image = Image.open(requests.get(url, stream=True).raw)

doc_question_answerer = pipeline("document-question-answering", model="magorshunov/layoutlm-
invoices")
preds = doc_question_answerer(
    question="总金额是多少？",
```

```
    image=image,
)
```

希望本页面为您提供了一些关于各个模态任务类型及其实际重要性的背景信息。在下一章节中，您将了解 🤗 Transformers 是如何解决这些任务的。

Transformers 库解决了哪些问题

在🤗 Transformers 的应用场景一文中，您了解了自然语言处理（NLP）、语音和音频、计算机视觉任务以及它们的一些重要应用。本页面将详细解释这些任务的解决方案，并解释模型内部的工作原理。虽然每种任务的解决方案可能各不相同，但 Transformer 模型的基本思路是相似的。由于其灵活的架构，大多数模型是编码器、解码器或编码器-解码器结构的变体。此外，我们的库中还包含几种卷积神经网络（CNN），这些网络仍然用于计算机视觉任务。我们也将解释现代 CNN 的工作原理。

要解释任务的解决方法，我们将详细说明模型内部的工作流程，以便输出有用的预测结果。

- Wav2Vec2 用于音频分类和自动语音识别（ASR）
- ViT（Vision Transformer） 和 ConvNeXT 用于图像分类
- DETR 用于目标检测
- Mask2Former 用于图像分割
- GLPN 用于深度估计
- BERT 用于使用编码器的任务，如文本分类、标记分类和问答
- GPT2 用于使用解码器的任务，如文本生成
- BART 用于使用编码器-解码器的任务，如摘要和翻译

在继续阅读之前，了解原始 Transformer 架构的一些基本知识是有益的。了解编码器、解码器和注意力机制的工作原理，有助于您理解不同 Transformer 模型的工作原理。如果您是初学者或需要复习，请查看我们的课程以获取更多信息！

语音和音频

Wav2Vec2 是一种自监督模型，预训练于无标签语音数据，并在有标签数据上进行微调，用于音频分类和自动语音识别（ASR）。

〔图：Wav2Vec2 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/wav2vec2_architecture.png，离线电子书未嵌入，详见原仓库）

该模型有四个主要组件：

342. **特征编码器**：它接收原始音频波形，将其归一化为零均值和单位方差，并将其转换为一系列长度为 20ms 的特征向量。

343. 由于波形是连续的，无法像文本那样分割成离散单元，因此特征向量被传递给**量化模块**，该模块试图学习离散的语音单元。语音单元从一组代码字中选择，称为**词汇表**。从词汇表中选择最能代表连续音频输入的向量或语音单元，并将其传递给模型。

344. 约一半的特征向量被随机遮掩，并将遮掩的特征向量传递给**上下文网络**，这是一个 Transformer 编码器，它还添加了相对位置嵌入。

345. 上下文网络的预训练目标是一个**对比任务**。模型须从一组错误的选项中预测遮掩部分的真实量化语音表示，这鼓励模型找到最相似的上下文向量和量化语音单元（目标标签）。

预训练后，您可以微调 Wav2Vec2 以用于音频分类或自动语音识别！

音频分类

要使用预训练模型进行音频分类，请在基础 Wav2Vec2 模型上添加一个序列分类头。分类头是线性层，它接收编码器的隐藏状态。隐藏状态表示每个音频帧中学习到的特征，这些特征长度不同。为了创建一个固定长度的向量，隐藏状态首先进行池化，然后转换为类标签的逻辑值。计算逻辑值和目标之间的交叉熵损失，以找到最有可能的类别。

想尝试音频分类吗？请查看我们的音频分类指南，学习如何微调 Wav2Vec2 并用于推理！

自动语音识别

要使用预训练模型进行自动语音识别，请在基础 Wav2Vec2 模型上添加一个语言模型头，用于连接时序分类（CTC）。语言模型头是线性层，它接收编码器的隐藏状态并将其转换为逻辑值。每个逻辑值代表一个标记类（标记数量来自任务词汇表）。计算 CTC 损失以找到最有可能的标记序列，然后对其进行解码，以生成转录文本。

想尝试自动语音识别吗？请查看我们的自动语音识别指南，学习如何微调 Wav2Vec2 并用于推理！

计算机视觉

处理计算机视觉任务有两种方法：

346. 将图像分割成一系列补丁，并使用 Transformer 并行处理它们。

347. 使用现代 CNN（如 ConvNeXT），其依赖于卷积层，但采用了现代网络设计。

第三种方法是将 Transformer 与卷积混合使用（例如 Convolutional Vision Transformer 或 LeViT）。我们不讨论这些方法，因为它们只是结合了前面两种方法。

ViT 和 ConvNeXT 常用于图像分类，而对于目标检测、分割和深度估计等其他视觉任务，我们将分别查看 DETR、Mask2Former 和 GLPN，这些模型更适合这些任务。

图像分类

ViT 和 ConvNeXT 都可用于图像分类，主要区别在于：ViT 使用注意力机制，而 ConvNeXT 使用卷积。

Transformer

ViT 完全用纯 Transformer 架构替代了卷积。如果您熟悉原始 Transformer，那么您已经掌握了理解 ViT 的大部分内容。

〔图：ViT 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/transformers/model_doc/vit_architecture.jpg，离线电子书未嵌入，详见原仓库）

ViT 引入的主要变化是图像如何输入到 Transformer：

348. 将图像分割为非重叠的方形补丁，每个补丁转换为向量或**补丁嵌入**。补丁嵌入由卷积 2D 层生成，以创建适当的输入维度（对于基础 Transformer 每个补丁嵌入的维度是 768）。例如，如果图像大小为 224x224 像素，可以将其分割为 196 个 16x16 的图像补丁。就像文本被分词为单词一样，图像也被“分词”为一系列补丁。

349. 添加一个**可学习嵌入**——特殊 `[CLS]` 标记——到补丁嵌入的开头，就像 BERT 一样。最终的 `[CLS]` 标记的隐藏状态用作分类头的输入；其他输出被忽略。这个标记帮助模型学习如何编码图像表示。

350. 将位置嵌入添加到补丁和可学习嵌入中，因为模型不知道图像补丁的顺序。位置嵌入也是可学习的，并且与补丁嵌入具有相同的大小。最后，所有嵌入被传递给 Transformer 编码器。

351. 输出，特别是带有 `[CLS]` 标记的输出，被传递给多层感知器头（MLP）。ViT 的预训练目标是简单的分类。像其他分类头一样，MLP 头将输出转换为类标签的逻辑值，并计算交叉熵损失以找到最有可能的类别。

想尝试图像分类吗？请查看我们的图像分类指南，学习如何微调 ViT 并用于推理！

卷积神经网络（CNN）

本节简要解释卷积，但了解它们如何改变图像的形状和大小是有帮助的。如果您不熟悉卷积，请查看 fastai 书籍中的卷积神经网络章节！

ConvNeXT 是一种卷积神经网络架构，采用了新的现代网络设计以提高性能。然而，卷积仍然是模型的核心。从高层次看，卷积是将一个小矩阵（内核）与图像像素的小窗口相乘的操作。它从该窗口中计算一些特征，例如特定纹理或线条的曲率。然后它滑动到下一个像素窗口；卷积的距离称为**步幅**。

〔图：卷积操作〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/convolution.gif，离线电子书未嵌入，详见原仓库）

基本卷积（不使用填充或步幅），取自深度学习卷积算术指南

您可以将此输出传递给另一个卷积层，随着每层的增加，网络学习到更复杂和抽象的东西，如热狗或火箭。卷积层之间通常添加一个池化层，以减少维度并使模型对特征位置的变化更具鲁棒性。

〔图：ConvNeXT 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/convnext_architecture.png，离线电子书未嵌入，详见原仓库）

ConvNeXT 通过五种方式现代化 CNN：

352. 改变每个阶段的块数，并“打补丁”图像，使用更大的步幅和对应的内核大小。非重叠滑动窗口使这种打补丁策略类似于 ViT 如何将图像分割为补丁。

353. **瓶颈层**缩小通道数量并恢复，因为 1x1 卷积更快，您可以增加深度。倒瓶颈层则相反，它通过扩展通道数量并缩小来提高内存效率。

354. 用**深度卷积**代替瓶颈层中的典型 3x3 卷积层，深度卷积对每个输入通道分别应用卷积，然后将它们重新堆叠在一起。这拓宽了网络宽度，提高了性能。

355. ViT 具有全局感受野，这意味着它可以一次性看到更多内容，这要归功于其注意力机制。ConvNeXT 通过将内核大小增加到 7x7 来模仿这种效果。

356. ConvNeXT 还在几个层设计上模仿 Transformer 模型。减少了激活和归一化层的数量，激活函数从 ReLU 切换为 GELU，并使用 LayerNorm 代替 BatchNorm。

卷积块的输出被传递给分类头，将输出转换为逻辑值，并计算交叉熵损失以找到最有可能的标签。

目标检测

DETR（DEtection TRansformer）是一种端到端的目标检测模型，结合了 CNN 和 Transformer 编码器-解码器。

〔图：DETR 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/detr_architecture.png，离线电子书未嵌入，详见原仓库）

357. 预训练的 CNN **骨干**接受图像（由像素值表示），并创建低分辨率的特征图。对特征图应用 1x1 卷积以降低维度，生成具有高级图像表示的新特征图。由于 Transformer 是序列模型，特征图被展平为特征向量的序列，并与位置嵌入结合。

358. 特征向量被传递给编码器，编码器使用其注意力层学习图像表示。然后编码器的隐藏状态与**对象查询**结合在解码器中。对象查询是可学习的嵌入，专注于图像的不同区域，并在通过每个注意力层时进行更新。解码器的隐藏状态被传递给前馈网络，以预测每个对象查询的边界框坐标和类别标签，或预测为“无对象”。

DETR 并行解码每个对象查询，输出 _N_ 个最终预测，其中 _N_ 是查询的数量。与通常一次预测一个元素的自回归模型不同，目标检测是集预测任务（`边界框`，`类别标签`），在单次传递中输出 _N_ 个预测。

359. DETR 在训练期间使用**二分匹配损失**，将其固定数量的预测与固定数量的真实标签进行比较。如果 _N_ 个标签中真实标签较少，则用“无对象”类别填充。此损失函数鼓励 DETR 找到预测与真实标签之间的一对一对应关系。如果边界框或类别标签不正确，会计算损失。同样，如

果 DETR 预测了一个不存在的对象，它也会受到惩罚，这促使 DETR 寻找图像中的其他对象，而不是只关注一个突出对象。

目标检测头添加在 DETR 顶部，以找到类别标签和边界框的坐标。检测头有两个组件：一个线性层，将解码器的隐藏状态转换为类别标签的逻辑值；以及一个多层感知器，用于预测边界框。

想尝试目标检测吗？请查看我们的目标检测指南，学习如何微调 DETR 并用于推理！

图像分割

Mask2Former 是一种通用架构，用于解决各种图像分割任务。传统分割模型通常针对特定的图像分割子任务，如实例、语义或全景分割。Mask2Former 将每个任务视为**掩码分类**问题。掩码分类将像素分组为 _N_ 个片段，并为给定图像预测 _N_ 个掩码及其对应的类别标签。让我们解释 Mask2Former 的工作原理，然后您可以尝试微调 SegFormer。

〔图：Mask2Former 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/mask2former_architecture.png，离线电子书未嵌入，详见原仓库）

Mask2Former 有三个主要组件：

360. Swin 骨干接受图像，并通过三个连续的 3x3 卷积创建低分辨率图像特征图。
361. 特征图被传递给**像素解码器**，像素解码器逐步上采样低分辨率特征，生成高分辨率的像素嵌入。像素解码器实际上生成多尺度特征（包含低分辨率和高分辨率特征），分辨率为原始图像的 1/32、1/16 和 1/8。
362. 这些不同尺度的特征图依次传递给一个 Transformer 解码器层，以捕捉高分辨率特征中的小对象。Mask2Former 的关键是解码器中的**掩码注意力**机制。与可以关注整个图像的交叉注意力不同，掩码注意力仅关注图像的特定区域。这使得速度更快，并且由于局部特征就足够模型学习，因此性能更好。
363. 像 DETR 一样，Mask2Former 也使用可学习的对象查询，并将它们与像素解码器的图像特征结合，进行集预测（`类别标签`，`掩码预测`）。解码器的隐藏状态被传递给线性层，转换为类别标签的逻辑值。计算逻辑值与类别标签之间的交叉熵损失，以找到最有可能的类别标签。

掩码预测是通过将像素嵌入与最终解码器的隐藏状态结合生成的。计算逻辑值与真实掩码之间的 sigmoid 交叉熵和骰子损失，以找到最有可能的掩码。

想尝试图像分割吗？请查看我们的图像分割指南，学习如何微调 SegFormer 并用于推理！

深度估计

GLPN（Global-Local Path Network）是用于深度估计的 Transformer，结合了 SegFormer 编码器和轻量解码器。

〔图：GLPN 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/glpn_architecture.jpg，离线电子书未嵌入，详见原仓库）

364. 类似 ViT，图像被分割成一系列补丁，但这些图像补丁更小，更适合密集预测任务（如分割或深度估计）。图像补丁被转换为补丁嵌入（有关补丁嵌入的详细信息，请参阅图像分类部分），并传递给编码器。

365. 编码器接受补丁嵌入，并通过多个编码器块输出最终的隐藏状态。每个块包含注意力层和 Mix-FFN 层。后者用于提供位置信息。每个编码器块的末尾是一个**补丁合并**层，用于创建分层表示。每个邻近补丁的特征被连接，然后通过线性层应用于连接的特征，以减少补丁数量，降低到 1/4 的分辨率。这成为下一个编码器块的输入，整个过程重复进行，直到生成分辨率为 1/8、1/16 和 1/32 的图像特征。

366. 轻量解码器接受编码器的最后一个特征图（1/32 分辨率），并上采样到 1/16 分辨率。从这里开始，特征被传递给**选择性特征融合（SFF）模块**，该模块为每个特征选择和组合局部和全局特征，并将其上采样到 1/8 分辨率。此过程重复进行，直到解码特征与原始图像大小相同。输出通过两个卷积层，然后应用 sigmoid 激活函数以预测每个像素的深度。

自然语言处理

Transformer 最初是为机器翻译设计的，自那时以来，它几乎已成为解决所有 NLP 任务的默认架构。一些任务更适合使用 Transformer 的编码器结构，而其他任务更适合使用解码器。还有一些任务则结合使用编码器-解码器结构。

文本分类

BERT 是一种仅编码器模型，首次有效地实现了双向性，以学习更丰富的文本表示。

367. BERT 使用 WordPiece 分词生成文本的标记嵌入。为了区分单个句子和句子对，添加了特殊 `[SEP]` 标记。每个文本序列的开头添加一个特殊 `[CLS]` 标记。最终的 `[CLS]` 标记输出用于分类任务的分类头输入。BERT 还添加了段嵌入，以表示标记是属于第一个句子还是第二个句子。

368. BERT 的预训练有两个目标：掩码语言建模和下一句预测。在掩码语言建模中，一定比例的输入标记被随机遮掩，模型需要预测这些。这解决了双向性的问题，模型无法通过看到所有单词来“预测”下一个单词。预测的遮掩标记的最终隐藏状态被传递给具有 softmax 的前馈网络，以预测遮掩的单词。

第二个预训练目标是下一句预测。模型必须预测句子 B 是否跟在句子 A 之后。一半的时间句子 B 是下一个句子，另一半时间句子 B 是随机句子。预测结果（是否为下一句）被传递给具有 softmax 的前馈网络，以预测两个类别（`IsNext` 和 `NotNext`）。

369. 输入嵌入被传递给多个编码器层，以输出最终的隐藏状态。

要使用预训练模型进行文本分类，请在基础 BERT 模型上添加一个序列分类头。序列分类头是一个线性层，它接收最终的隐藏状态并执行线性变换以转换为逻辑值。计算逻辑值与目标之间的交叉熵损失，以找到最有可能的标签。

想尝试文本分类吗？请查看我们的文本分类指南，学习如何微调 DistilBERT 并用于推理！

标记分类

要使用 BERT 进行标记分类任务（如命名实体识别 NER），请在基础 BERT 模型上添加一个标记分类头。标记分类头是一个线性层，它接收最终的隐藏状态并执行线性变换以转换为逻辑值。计算逻辑值与每个标记之间的交叉熵损失，以找到最有可能的标签。

想尝试标记分类吗？请查看我们的标记分类指南，学习如何微调 DistilBERT 并用于推理！

问答

要使用 BERT 进行问答任务，请在基础 BERT 模型上添加一个跨度分类头。这个线性层接收最终的隐藏状态并执行线性变换，以计算答案的跨度起始和结束逻辑值。计算逻辑值与标签位置之间的交叉熵损失，以找到最有可能对应答案的文本跨度。

想尝试问答吗？请查看我们的问答指南，学习如何微调 DistilBERT 并用于推理！

💡 一旦 BERT 进行预训练，使用它进行不同任务是多么容易！您只需要在预训练模型上添加一个特定的头部，以操作隐藏状态，生成所需的输出！

文本生成

GPT-2 是一种仅解码器模型，预训练于大量文本。给定提示，它可以生成令人信服的文本（尽管不总是真实的！），并完成其他 NLP 任务，如问答，尽管它并不是为此专门训练的。

〔图：GPT-2 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/gpt2_architecture.png，离线电子书未嵌入，详见原仓库）

370. GPT-2 使用 字节对编码（BPE） 对词进行分词并生成标记嵌入。位置编码被添加到标记嵌入中，以指示序列中每个标记的位置。输入嵌入被传递给多个解码器块，以输出最终的隐藏状态。在每个解码器块内，GPT-2 使用**掩码自注意力**层，这意味着 GPT-2 不能关注未来的标记。它只能关注左侧的标记。这与 BERT 的 `mask` 标记不同，因为在掩码自注意力中，使用注意力掩码将未来标记的分数设置为 `0`。

371. 来自解码器的输出被传递给语言模型头，执行线性变换以将隐藏状态转换为逻辑值。标签是序列中的下一个标记，创建方式是将逻辑值向右移动一位。计算移位后的逻辑值与标签之间的交叉熵损失，以输出下一个最有可能的标记。

GPT-2 的预训练目标完全基于因果语言建模，即预测序列中的下一个词。这使得 GPT-2 特别适合涉及生成文本的任务。

想尝试文本生成吗？请查看我们的因果语言建模指南，学习如何微调 DistilGPT-2 并用于推理！

有关文本生成的更多信息，请查看文本生成策略指南！

摘要

编码器-解码器模型（如 BART 和 T5）专为摘要任务的序列到序列模式设计。我们将在本节中解释 BART 的工作原理，然后您可以尝试微调 T5。

〔图：BART 架构〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/bart_architecture.png，离线电子书未嵌入，详见原仓库）

372. BART 的编码器架构与 BERT 非常相似，接受文本的标记嵌入和位置嵌入。BART 通过破坏输入并用解码器重建它来进行预训练。与其他具有特定破坏策略的编码器不同，BART 可以应用任何类型的破坏。**文本填充**破坏策略效果最好。在文本填充中，一定数量的文本片段被替换为**单个** `mask` 标记。这很重要，因为模型必须预测被遮掩的标记，它教导模型预测缺失的标记数量。输入嵌入和被遮掩的片段被传递给编码器，输出最终的隐藏状态，但与 BERT 不同，BART 并未在末尾添加一个最终的前馈网络来预测词。

373. 编码器的输出被传递给解码器，解码器必须从编码器的输出中预测被遮掩的标记和未被破坏的标记。这为解码器提供了额外的上下文，以帮助其恢复原始文本。解码器的输出被传递给语言模型头，执行线性变换以将隐藏状态转换为逻辑值。计算逻辑值与标签之间的交叉熵损失，标签只是向右移动一位的标记。

想尝试摘要吗？请查看我们的摘要指南，学习如何微调 T5 并用于推理！

有关文本生成的更多信息，请查看文本生成策略指南！

翻译

翻译是另一个序列到序列任务的示例，这意味着您可以使用编码器-解码器模型（如 BART 或 T5）来实现它。我们将在本节中解释 BART 的工作原理，然后您可以尝试微调 T5。

BART 通过添加一个独立的随机初始化编码器来适应翻译任务。该编码器将源语言映射为输入，可以被解码器解码为目标语言。这个新编码器的嵌入被传递给预训练的编码器，而不是原始的词嵌入。源编码器通过更新源编码器、位置嵌入和输入嵌入，使用模型输出的交叉熵损失进行训练。在第一步中，模型参数被冻结，所有模型参数在第二步中一起训练。

BART 随后推出了多语言版本 mBART，旨在进行翻译，并在许多不同语言上进行了预训练。

想尝试翻译吗？请查看我们的翻译指南，学习如何微调 T5 并用于推理！

有关文本生成的更多信息，请查看文本生成策略指南！

Transformer 模型家族

自 2017 年推出以来，原始的 Transformer 模型（可以通过《注释版 Transformer》这篇博客文章获取更易懂的技术介绍）启发了许多新模型，这些模型的应用范围已经超出了自然语言处理

（NLP）任务。这些模型包括预测蛋白质折叠结构，训练猎豹奔跑，以及时间序列预测。有这么多的 Transformer 变体，人们很容易忽略整体情况。这些模型的共同点是它们都基于原始的 Transformer 架构。有些模型只使用了编码器或解码器，而有些则同时使用了两者。这为我们提供了一个有用的分类方法，可以帮助理解不同模型之间的高级差异，也帮助你理解之前未接触过的 Transformer 模型。

如果你不熟悉原始的 Transformer 模型或需要复习，可以查看 Hugging Face 课程的《Transformer 是如何工作的》章节。

计算机视觉

卷积网络

长期以来，卷积网络（CNN）一直是计算机视觉任务的主流范式，直到 Vision Transformer 展示了它的可扩展性和高效性。即使如此，CNN 的一些优秀特性（如平移不变性）对于某些任务来说仍然非常强大，因此一些 Transformer 采用了卷积结构。相反，ConvNeXt 用 Transformer 的设计理念来改进 CNN。例如，ConvNeXt 使用非重叠滑动窗口将图像划分为补丁，并使用更大的卷积核以增加全局感受野。ConvNeXt 还引入了几种层设计选择，以提高内存效率和性能，使其与 Transformer 竞争。

视觉编码器

Vision Transformer (ViT) 打开了计算机视觉任务中不使用卷积的大门。ViT 使用标准的 Transformer 编码器，但其主要突破在于如何处理图像。它将图像分割为固定大小的补丁，并使用这些补丁创建嵌入，就像将句子分割为标记一样。ViT 利用 Transformer 的高效架构，在训练所需的资源较少的情况下，展示了与当时的 CNN 竞争的结果。随后，其他视觉模型也出现了，可以处理密集视觉任务（如分割和检测）。

其中一个模型是 Swin Transformer。它类似于 CNN，从较小的补丁构建分层特征图，并将其与相邻补丁合并到更深层。注意力机制仅在局部窗口内计算，并且在注意力层之间移动窗口，以帮助模型学习。由于 Swin Transformer 可以生成分层特征图，因此它适用于密集预测任务（如分割和检

测）。SegFormer 也使用 Transformer 编码器来构建分层特征图，但其添加了一个简单的多层感知器（MLP）解码器，用于组合所有特征图并进行预测。

其他视觉模型，如 BeIT 和 ViTMAE，借鉴了 BERT 的预训练目标。BeIT 通过 _masked image modeling (MIM)_ 进行预训练：图像补丁被随机遮蔽，并且图像也被分词化为视觉标记。BeIT 被训练以预测遮蔽补丁对应的视觉标记。ViTMAE 的预训练目标类似，但它必须预测像素而不是视觉标记。特别之处在于，75% 的图像补丁被遮蔽！解码器从遮蔽的标记和编码的补丁中重建像素。预训练后，解码器被丢弃，编码器准备用于下游任务。

视觉解码器

仅使用解码器的视觉模型较为罕见，因为大多数视觉模型依赖编码器来学习图像表示。但在像图像生成这样的用例中，解码器是一个自然的选择，我们从文本生成模型（如 GPT-2）中已经看到了这一点。ImageGPT 与 GPT-2 使用相同的架构，但它预测的是图像中的下一个像素，而不是序列中的下一个标记。除了图像生成，ImageGPT 还可以微调用于图像分类。

视觉编码器-解码器

视觉模型通常使用编码器（也称为主干）提取重要的图像特征，然后将其传递给 Transformer 解码器。DETR 使用预先训练的主干，但还使用完整的 Transformer 编码器-解码器架构进行目标检测。编码器学习图像表示，并在解码器中将其与目标查询（每个目标查询是聚焦于图像中某个区域或目标的嵌入）结合。DETR 预测每个目标查询的边界框坐标和类别标签。

自然语言处理

NLP 编码器

BERT 是一个仅使用编码器的 Transformer，它通过随机遮蔽输入中的某些标记来避免“作弊”。预训练目标是根据上下文预测被遮蔽的标记。这使 BERT 能够充分利用左右上下文，以帮助它学习更深入和丰富的输入表示。然而，BERT 的预训练策略仍有改进的空间。RoBERTa 通过引入新的预训练方法来改进这一点，包括更长时间的训练和更大的批量训练，每个 epoch 随机遮蔽标记（而不仅仅是预处理阶段一次遮蔽），并删除了下一个句子预测目标。

改进性能的主要策略是增加模型规模。但训练大型模型在计算上代价高昂。一种减少计算成本的方法是使用更小的模型，如 DistilBERT。DistilBERT 使用知识蒸馏——一种压缩技术——创建一个更小版本的 BERT，同时保持其绝大部分的语言理解能力。

然而，大多数 Transformer 模型继续增加参数数量，导致出现了新的模型，专注于提高训练效率。ALBERT 通过两种方式减少内存消耗：将较大的词汇表嵌入分为两个较小的矩阵，并允许多层共享参数。DeBERTa 引入了一种解耦的注意力机制，将单词和位置分别编码为两个向量。注意力从这些独立的向量计算，而不是从单个包含单词和位置嵌入的向量计算。Longformer 也专注于使注意力效率更高，特别是处理较长序列的文档。它使用局部窗口注意（注意力仅计算每个标记周围的固定窗口大小）和全局注意力（仅用于特定的任务标记，如分类中的 `[CLS]`）来创建稀疏注意力矩阵而不是完全注意力矩阵。

NLP 解码器

GPT-2 是一个仅使用解码器的 Transformer，它预测序列中的下一个单词。它遮蔽右侧的标记，以防止模型“作弊”。通过预训练大量文本，GPT-2 变得非常擅长生成文本，即使这些文本有时是不准确或不真实的。但 GPT-2 缺乏 BERT 预训练中的双向上下文，这使其不适合某些任务。XLNET 结合 BERT 和 GPT-2 的预训练目标，使用排列语言建模目标（PLM），使之能够学习双向上下文。

在 GPT-2 之后，语言模型变得更大，现在被称为 _大型语言模型（LLMs）_。LLMs 如果在足够大的数据集上预训练，可以展示出少量或零样本学习的能力。GPT-J 是一个具有 6B 参数的 LLM，并在 400B 个标记上进行了训练。GPT-J 之后是 OPT，这是一系列仅使用解码器的模型，最大的模型有 175B 参数，并在 180B 个标记上进行了训练。BLOOM 也在同一时间发布，最大的模型有 176B 参数，并在 46 种语言和 13 种编程语言的 366B 个标记上进行了训练。

NLP 编码器-解码器

BART 保持了原始的 Transformer 架构，但修改了预训练目标：通过 _文本填充_ 腐蚀，将一些文本片段替换为单个 `mask` 标记。解码器预测未腐败的标记（未来标记被遮蔽），并使用编码器的隐状态来帮助它。Pegasus 类似于 BART，但它遮蔽的是整个句子而不是文本片段。除了掩码语言建模，Pegasus 还通过句子间隙生成（GSG）进行预训练。GSG 目标在于屏蔽文书中重要的整个句子，用 `mask` 标记替换它们。解码器必须根据剩余的句子生成输出。T5 是一个更独特的模型，它将

所有 NLP 任务转换为文本到文本的问题，使用特定的前缀。例如，前缀 `Summarize:` 表示摘要任务。T5 通过监督（GLUE 和 SuperGLUE）训练和自我监督训练（随机样本并丢弃 15% 的标记）进行预训练。

音频

音频编码器

Wav2Vec2 使用 Transformer 编码器直接从原始音频波形中学习语音表示。它通过对比任务进行预训练，以确定真实语音表示和其他虚假表示之间的差异。HuBERT 与 Wav2Vec2 类似，但训练过程不同。目标标签是通过聚类步骤创建的，其中相似的音频片段被分配到一个聚类，该聚类成为隐藏单元。隐藏单元被映射到嵌入以进行预测。

音频编码器-解码器

Speech2Text 是一个用于自动语音识别（ASR）和语音翻译的语音模型。该模型接受从音频波形中提取的对数梅尔滤波器组特征，并自回归地预训练以生成转录或翻译。Whisper 也是一个 ASR 模型，但与许多其他语音模型不同，它是在巨大的✨标注✨音频转录数据上预训练的，因此可以在零样本情况下表现良好。数据集中很大一部分是非英语语言，这意味着 Whisper 也可以用于低资源语言。在结构上，Whisper 类似于 Speech2Text。音频信号被转换为由编码器编码的对数梅尔谱图。解码器根据编码器的隐状态和之前的标记自回归地生成转录。

多模态

多模态编码器

VisualBERT 是一个用于视觉-语言任务的多模态模型，发布于 BERT 之后不久。它结合了 BERT 和预训练的对象检测系统，将图像特征提取为视觉嵌入，并与文本嵌入一起传递给 BERT。VisualBERT 根据未遮蔽的文本和视觉嵌入预测遮蔽的文本，并预测文本是否与图像对齐。随着 ViT 的发布，ViLT 在其架构中采用了 ViT，因为这样更容易获得图像嵌入。图像嵌入与文本嵌入共同处理。从那时起，ViLT 通过图像文本匹配、掩码语言建模和整体词汇遮蔽进行预训练。

CLIP 采用不同的方法，对 (`image`, `text`) 对进行预测。图像编码器（ViT）和文本编码器（Transformer）共同在包含 4 亿个 (`image`, `text`) 对的数据集上进行预训练，以最大化图像和文本嵌入之间的相似性。预训练后，可以使用自然语言指导 CLIP 从图像预测文本或反之亦然。OWL-ViT 在 CLIP 的基础上进行了扩展，使用 CLIP 作为其零样本目标检测的主干。预训练后，添加了一个目标检测头，对 (`class`, `bounding box`) 对进行预测。

多模态编码器-解码器

光学字符识别（OCR）是文本识别的长期任务，通常涉及多个组件来理解图像并生成文本。TrOCR 使用端到端的 Transformer 简化了这一过程。编码器是一个 ViT 风格的模型，用于图像理解，并将图像处理为固定大小的补丁。解码器接受编码器的隐状态并自回归地生成文本。Donut 是一个更通用的视觉文档理解模型，不依赖于基于 OCR 的方法。它使用 Swin Transformer 作为编码器和多语言 BART 作为解码器。Donut 通过预测图像和文本注释中的下一个词进行预训练。解码器根据提示生成标记序列。每个下游任务都有一个特殊标记。例如，文档解析有一个特殊的 `parsing` 标记，结合编码器的隐状态将文档解析为结构化的输出格式（JSON）。

强化学习

强化解码器

Decision 和 Trajectory Transformer 将状态、动作和奖励视为序列建模问题。Decision Transformer 生成一系列动作，使未来期望的回报最大化，基于回报到目前为止、过去的状态和动作。对于最后 _K_ 个时间步，每个模态被转换为标记嵌入，并由 GPT 样式模型处理以预测未来的动作标记。Trajectory Transformer 也将状态、动作和奖励标记化，并使用 GPT 架构处理。与 Decision Transformer 不同，它专注于奖励条件，Trajectory Transformer 通过束搜索生成未来的动作。

Tokenizer —— 文本分词器概述

在本页面中，我们将深入探讨文本分词。

在预处理教程中，我们了解到，分词是将文本拆分为单词或子词，然后通过查找表将其转换为 ID。将单词或子词转换为 ID 的过程非常直观，所以在本文中，我们将重点介绍如何将文本拆分为单词或子词（即分词）。具体来说，我们将介绍在 🤗 Transformers 中使用的三种主要分词器：字节对编码（BPE）、WordPiece 和 SentencePiece，并展示不同模型使用哪种分词器类型的示例。

请注意，在每个模型页面上，您可以查看相关分词器的文档以了解预训练模型使用了哪种分词器类型。例如，查看 BertTokenizer，可以看到该模型使用了 WordPiece。

简介

将文本拆分为更小部分的任务比看起来要复杂得多，并且有多种方法可以实现。例如，考虑这句话：`"Don't you love 🤗 Transformers? We sure do."`

一种简单的分词方法是按空格拆分文本，结果会是：

```
["Don't", "you", "love", "🤗", "Transformers?", "We", "sure", "do."]
```

这是一个合理的初步步骤，但如果看分词`"Transformers?"`和`"do."`，我们会发现标点符号与单词`"Transformers"`和`"do"`连接在一起，这是不理想的。我们应该将标点符号与单词分开，以便模型不必为每个单词和可能跟随它的标点符号学习不同的表示法，这将使模型需要学习的表示法数量大幅增加。考虑标点符号后，我们的示例文本可以拆分为：

```
["Don", "'", "t", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."]
```

更好了。然而，分词器处理`"Don't"`的方式仍然存在不足。`"Don't"`代表`"do not"`，所以更合理的分词结果应该是`["Do", "n't"]`。这正是每个模型有自己的分词器类型的原因之一。根据我们为分词文本所应用的规则，相同文本可能会生成不同的分词输出。预训练模型只有在其输入与训练数据所用规则相同的前提下才能正常工作。

spaCy 和 Moses 是两种流行的基于规则的分词器。将它们应用于我们的示例，spaCy 和 Moses 会输出如下：

```
["Do", "n't", "you", "love", "🤗", "Transformers", "?", "We", "sure", "do", "."]
```

可以看到，这里使用了空格和标点分词以及基于规则的分词。空格和标点分词以及基于规则的分词都是词汇分词的示例，即大致定义为将句子拆分为单词。虽然这种方法最直观，但在处理海量文本语料库时可能会出现问题。例如，Transformer XL 使用空格和标点分词，导致词汇量为 267,735！

如此大的词汇量迫使模型需要一个巨大的嵌入矩阵作为输入和输出层，这不仅增加了内存，还增加了时间复杂度。通常，Transformer 模型的词汇量很少超过 50,000，特别是在仅在一个语言上进行预训练的情况下。

因此，如果简单的空格和标点分词不令人满意，为什么不按字符进行分词呢？

虽然字符分词非常简单，并且可以大大减少内存和时间复杂度，但会使得模型更难学习有意义的输入表示。例如，学习`"t"`的有意义上下文无关表示比学习`"today"`的表示要困难得多。因此，字符分词通常伴随着性能损失。为了兼顾两者的优势，Transformer 模型使用介于词汇级和字符级分词之间的混合方法，称为**子词**分词。

子词分词

子词分词算法依赖于以下原则：常用单词不应拆分为更小的子词，而罕见单词应分解为有意义的子词。例如，`"annoyingly"`可能被视为一个罕见单词，可以拆分为`"annoying"`和`"ly"`。`"annoying"`和`"ly"`作为单独的子词会更频繁地出现，同时，`"annoying"`和`"ly"`的组合意义保留了`"annoyingly"`的含义。这对粘着语（如土耳其语）特别有用，因为可以使用子词连接形成几乎任意长的复杂单词。

子词分词使模型能够拥有合理的词汇量大小，同时学习有意义的上下文无关表示。此外，子词分词使模型能够处理从未见过的单词，通过将其分解为已知的子词。例如，BertTokenizer 会将`"I have a new GPU!"`分词为：

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("google-bert/bert-base-uncased")
tokenizer.tokenize("I have a new GPU!")
```

因为我们在使用小写模型，所以句子首先被转换为了小写。可以看到，词典中包含`["i", "have", "a", "new"]`，但不包含`"gpu"`，因此将其拆分为已知的子词`["gp"`和`"##u"]`。`"##"`表示该标记应连接到前一个标记，且中间没有空格（用于解码或逆转分词）。

作为另一个例子，XLNetTokenizer 将我们之前的示例文本分词为：

```
from transformers import XLNetTokenizer
tokenizer = XLNetTokenizer.from_pretrained("xlnet/xlnet-base-cased")
tokenizer.tokenize("Don't you love 🤗 Transformers? We sure do.")
```

我们稍后会解释`"▁"`的含义（参见 SentencePiece）。可以看到，罕见单词`"Transformers"`被拆分为了更常见的子词`"Transform"`和`"ers"`。

现在，让我们看看不同的子词分词算法是如何工作的。请注意，所有这些分词算法都依赖某种形式的训练，通常是在相关模型将要训练的语料库上进行训练。

字节对编码 (BPE)

字节对编码（BPE）首次引入于 Neural Machine Translation of Rare Words with Subword Units (Sennrich et al., 2015)。BPE 依赖于预分词器将训练数据拆分为单词。预分词可以像空格分词一样简单，例如 GPT-2、RoBERTa。更复杂的预分词包括基于规则的分词，例如 XLM、FlauBERT 使用 Moses 处理大多数语言，或 GPT 使用 spaCy 和 ftfy，以统计训练语料库中每个单词的频率。

预分词后，会创建一组唯一的单词，并确定每个单词在训练数据中出现的频率。接下来，BPE 创建一个包含所有唯一单词中出现的符号的基本词典，并学习合并规则，通过将两个基本词典中的符号组合为一个新符号。它会不断进行，直到词典达到所需的大小。请注意，所需的词典大小是需要在训练分词器之前定义的超参数。

例如，假设预分词后的词频如下：

```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
```

因此，基本词典是`["b", "g", "h", "n", "p", "s", "u"]`。将所有单词拆分为基本词典中的符号，我们得到：

```
("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
```

BPE 然后统计每个可能的符号对出现的频率，并选择最常出现的符号对。在上述例子中，`"h"`后面跟着`"u"`出现了 10 + 5 = 15 次（在`"hug"`的 10 次和`"hugs"`的 5 次中）。然而，最常出现的符号对是`"u"`后面跟着`"g"`，总共出现了 10 + 5 + 5 = 20 次。因此，分词器学习的第一条合并规则是将所有`"u"`后面跟着`"g"`的符号组合在一起。接下来，`"ug"`被添加到词典中。单词集变为：

```
("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)
```

BPE 然后识别下一个最常出现的符号对。它是`"u"`后面跟`"n"`，出现了 16 次。`"u"`和`"n"`被合并为`"un"`并添加到词典中。下一个最常出现的符号对是`"h"`后面跟`"ug"`，出现了 15 次。再次合并，`"hug"`可以被添加到词典中。

此时，词典是`["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]`，我们唯一的单词集表示为：

```
("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)
```

假设 BPE 训练在此时停止，学习到的合并规则将应用于新单词（只要这些新单词不包含基本词典中不存在的符号）。例如，单词`"bug"`会被分词为`["b", "ug"]`，但`"mug"`会被分词为`["<unk>", "ug"]`，因为符号`"m"`不在基本词典中。通常，单个字母如`"m"`不会被替换为`"<unk>"`符号，因为训练数据通常至少包含每个字母的一次出现，但对于一些特殊字符（如表情符号），可能会发生这种情况。

如前所述，词典的大小，即基本词典大小加上合并次数，是需要选择的超参数。例如，GPT 的词汇量大小为 40,478，因为它们有 478 个基本字符，并选择在 40,000 次合并后停止训练。

字节级 BPE

如果考虑所有可能的基本字符，如所有 Unicode 字符，则基本词典可能会非常大。为了获得更好的基本词典，GPT-2 使用字节作为基本词典，这是一种巧妙的技巧，强迫基本词典的大小为 256，同时确保每个基本字符都包含在词典中。通过一些额外的规则处理标点符号，GPT2 的分词器可以分词任何文本，而无需使用`<unk>`符号。GPT-2 的词汇量大小为 50,257，对应 256 个字节基础标记、一个特殊结束文本标记以及通过 50,000 次合并学习到的符号。

WordPiece

WordPiece 是用于 BERT、DistilBERT 和 Electra 的子词分词算法。该算法首次出现在 Japanese and Korean Voice Search (Schuster et al., 2012)中，与 BPE 非常相似。WordPiece 首先将词典初始化为包含训练数据中所有字符，并逐步学习指定数量的合并规则。与 BPE 不同的是，WordPiece 不选择出现最频繁的符号对，而是选择能最大化训练数据似然度的符号对。

具体来说，最大化训练数据的似然度相当于找到一个符号对，其概率除以其第一个符号和第二个符号的概率之比在所有符号对中最大。例如，`"u"`后面跟`"g"`只会在`"ug"`的概率除以`"u"`和`"g"`的概率大于任何其他符号对时被合并。直观地说，WordPiece 与 BPE 略有不同，因为它会评估合并两个符号所“损失”的内容，以确保“值得”。

Unigram

Unigram 是一种子词分词算法，首次出现在 Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates (Kudo, 2018)中。与 BPE 或 WordPiece 不同，Unigram 将基本词典初始化为大量符号，并逐步修剪每个符号以获得较小的词典。基本词典可以对应所有预分词的单词和最常见子串。Unigram 未直接用于 transformers 库中的任何模型，而是与 SentencePiece 结合使用。

在每次训练步骤中，Unigram 算法在当前词典和单语模型下定义训练数据的损失（通常定义为对数似然）。然后，对于词典中的每个符号，计算如果将其从词典中移除，整体损失会增加多少。Unigram 会移除 p（通常是 10%或 20%）个损失增加最少的符号，即对训练数据整体损失影响最小的符号。这个过程重复进行，直到词典达到所需的大小。Unigram 算法总是保留基本字符，以便任何单词都可以被分词。

由于 Unigram 不是基于合并规则（与 BPE 和 WordPiece 不同），因此在训练后有多种方式对新文本进行分词。例如，如果训练后的 Unigram 分词器词典为：

```
["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]
```

`"hugs"`可以被分词为`["hug", "s"]`、`["h", "ug", "s"]`或`["h", "u", "g", "s"]`。那么，应该选择哪一个呢？Unigram 会保存训练语料库中每个标记的概率，从而在训练后计算每种可能分词的概率。实际上，算法会选择最可能的分词，但也提供了根据其概率采样可能分词的方法。

这些概率由分词器训练所用的损失定义。假设训练数据由单词 `x1,...,xN` 组成，单词 `xi` 的所有可能分词集定义为 `S(xi)`，则整体损失定义为：

SentencePiece

到目前为止，所有分词算法都有一个问题：假设输入文本使用空格分隔单词。然而，并不是所有语言都使用空格分隔单词。一个可能的解决方案是使用特定语言的预分词器，例如 XLM 使用特定的中文、日文和泰文预分词器。为了解决这一问题，SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing (Kudo et al., 2018)将输入视为原始输入流，从而将空格包括在字符集中。然后使用 BPE 或 Unigram 算法构建适当的词典。

例如，XLNetTokenizer 使用 SentencePiece，这就是为什么在之前的例子中"▁"字符被包含在词典中。使用 SentencePiece 进行解码非常简单，因为所有标记可以简单地连接起来，并用空格替换"▁"。

transformers 库中使用 SentencePiece 的所有模型都将其与 Unigram 结合使用。例如，ALBERT、XLNet、Marian 和 T5 都使用 SentencePiece。

Attention —— 注意力机制

大多数 transformer 模型使用全注意力机制，这意味着注意力矩阵是方形的。对于长文本而言，这可能会成为一个巨大的计算瓶颈。为了更高效，Longformer 和 Reformer 等模型使用了稀疏注意力矩阵来加速训练。

局部敏感哈希 (LSH) 注意力

Reformer 模型采用了 LSH 注意力机制。在 softmax(QK^t) 中，只有矩阵 QK^t 中最大的元素（在 softmax 维度上）会给出有用的贡献。因此，对于查询矩阵 Q 中的每个查询 q，我们只需要考虑与 q 接近的键 k。通过哈希函数来确定 q 和 k 是否接近。为了防止查询和键相等（即非常相似），除了在第一个位置外，注意力掩码会屏蔽当前的 token。由于哈希函数可能是随机的，实际上会使用多个哈希函数（由 n_rounds 参数确定），然后对结果进行平均。

局部注意力

Longformer 模型使用了局部注意力机制：通常情况下，局部上下文（例如，左右两个 token）对于处理给定的 token 已经足够了。通过堆叠具有小窗口的注意力层，最后一层将具有超出窗口 token 的感受野，从而可以构建整个句子的表示。

一些预选的输入 token 可以获得全局注意力：对于这些少数 token，注意力矩阵可以访问所有 token，这个过程是对称的：所有其他 token 也可以访问这些特定 token（除了它们本地窗口中的 token）。如下图所示，这是论文中的一个示例注意力掩码：

〔图：未命名〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/local_attention_mask.png，离线电子书未嵌入，详见原仓库）

使用这些参数较少的注意力矩阵可以让模型处理更长的序列。

其他技巧

轴向位置编码

Reformer 模型使用了轴向位置编码：在传统的 transformer 模型中，位置编码 E 是一个大小为 l × d 的矩阵，其中 l 是序列长度，d 是隐藏状态的维度。如果文本很长，这个矩阵可能会非常大，占用过多的 GPU 空间。为了缓解这个问题，轴向位置编码通过将大矩阵 E 分解成两个较小的矩阵 E1 和 E2 来实现，其尺寸分别为 l1 × d1 和 l2 × d2，满足 l1 × l2 = l 和 d1 + d2 = d（通过长度相乘，最终会小得多）。时间步 j 在 E 中的嵌入是通过连接 E1 中的时间步 j % l1 和 E2 中的时间步 j // l1 的嵌入获得的。

Padding & Truncation —— 填充与截断

在处理批量输入时，输入的长度通常是不同的，因此不能直接转换为固定大小的张量。填充（Padding）和截断（Truncation）是解决这一问题的策略，用于将不同长度的批量输入转换为矩形张量。填充通过添加特殊的**填充标记（padding token）**来确保较短的序列与批次中最长的序列或模型可接受的最大长度一致。截断则是通过截断较长的序列来解决问题。

通常情况下，将批次填充到最长序列的长度，并截断到模型可接受的最大长度，这种方法效果很好。但 API 还支持更多策略。你需要了解的三个参数是：`padding`、`truncation` 和 `max_length`。

padding 参数

`padding` 参数控制填充行为。它可以是布尔值或字符串：

- `True` 或 `'longest'`：将填充到批次中最长的序列长度。如果你只提供单个序列，则不会应用填充。

- `'max_length'`：将填充到由 `max_length` 参数指定的长度，或者如果未提供 `max_length`，则填充到模型可接受的最大长度。即使只提供单个序列，也会应用填充。
- `False` 或 `'do_not_pad'`：不应用填充。这是默认行为。

truncation 参数

`truncation` 参数控制截断行为。它可以是布尔值或字符串：

- `True` 或 `'longest_first'`：将截断到由 `max_length` 参数指定的最大长度，或者如果未提供 `max_length`，则截断到模型可接受的最大长度。这会逐个标记截断，从一对序列中最长的序列中移除标记，直到达到适当的长度。
- `'only_second'`：将截断到由 `max_length` 参数指定的最大长度，或者如果未提供 `max_length`，则截断到模型可接受的最大长度。这对成对序列中的第二个句子进行截断。
- `'only_first'`：将截断到由 `max_length` 参数指定的最大长度，或者如果未提供 `max_length`，则截断到模型可接受的最大长度。这对成对序列中的第一个句子进行截断。
- `False` 或 `'do_not_truncate'`：不应用截断。这是默认行为。

max_length 参数

`max_length` 参数控制填充和截断的长度。它可以是整数或 `None`，在这种情况下，默认为模型可接受的最大长度。如果模型没有指定的最大输入长度，则不激活截断或填充到 `max_length`。

下表总结了建议的填充和截断设置方式。如果你在以下示例中使用成对的输入序列，可以将 `truncation=True` 替换为 `STRATEGY`，选择 `['only_first', 'only_second', 'longest_first']` 中的策略，以控制成对序列中两个序列的截断方式。

截断	填充	指令
无截断	无填充	`tokenizer(batch_sentences)`
	填充到批次中最长序列长度	`tokenizer(batch_sentences, padding=True)` 或 `tokenizer(batch_sentences, padding='longest')`
	填充到模型最大输入长度	`tokenizer(batch_sentences, padding='max_length')`

	填充到指定长度	`tokenizer(batch_sentences, padding='max_length', max_length=42)`
	填充到某个值的倍数	`tokenizer(batch_sentences, padding=True, pad_to_multiple_of=8)`
截断到模型最大输入长度	无填充	`tokenizer(batch_sentences, truncation=True)` 或 `tokenizer(batch_sentences, truncation=STRATEGY)`
	填充到批次中最长序列长度	`tokenizer(batch_sentences, padding=True, truncation=True)` 或 `tokenizer(batch_sentences, padding=True, truncation=STRATEGY)`
	填充到模型最大输入长度	`tokenizer(batch_sentences, padding='max_length', truncation=True)` 或 `tokenizer(batch_sentences, padding='max_length', truncation=STRATEGY)`
	填充到指定长度	不支持
截断到指定长度	无填充	`tokenizer(batch_sentences, truncation=True, max_length=42)` 或 `tokenizer(batch_sentences, truncation=STRATEGY, max_length=42)`
	填充到批次中最长序列长度	`tokenizer(batch_sentences, padding=True, truncation=True, max_length=42)` 或 `tokenizer(batch_sentences, padding=True, truncation=STRATEGY, max_length=42)`
	填充到模型最大输入长度	不支持
	填充到指定长度	`tokenizer(batch_sentences, padding='max_length', truncation=True, max_length=42)` 或 `tokenizer(batch_sentences, padding='max_length', truncation=STRATEGY, max_length=42)`

BERTology —— 基于 BERT 进行的相关研究

有一个新兴的研究领域专注于研究像 BERT 这样的大规模转换器模型的内部工作原理（有些人称之为“BERTology”）。以下是一些这个领域的优秀例子：

- Ian Tenney、Dipanjan Das 和 Ellie Pavlick 的《BERT 重新发现经典 NLP 流水线》：链接
- Paul Michel、Omer Levy 和 Graham Neubig 的《十六个注意力头真的比一个好吗？》：链接
- Kevin Clark、Urvashi Khandelwal、Omer Levy 和 Christopher D. Manning 的《BERT 关注什么？对 BERT 注意力的分析》：链接
- 《CAT 探针：一种基于指标的方法来解释预训练编程语言模型如何关注代码结构》：链接

为了帮助这一新领域的发展，我们在 BERT/GPT/GPT-2 模型中添加了一些额外的功能，主要是从 Paul Michel 的卓越工作中改编而来（论文链接）：

- 访问 BERT/GPT/GPT-2 的所有隐藏状态，
- 访问 BERT/GPT/GPT-2 每个注意力头的所有注意力权重，
- 获取注意力头的输出值和梯度，以便计算头的重要性分数并修剪头（如论文中所述）。

为了帮助您理解和使用这些功能，我们添加了一个特定的示例脚本：bertology.py，这个脚本可以从预训练的 GLUE 模型中提取信息并进行剪枝。

注释：

- **隐藏状态**：在神经网络中，隐藏状态是每一层的输出，这些输出会传递给下一层。通过访问这些隐藏状态，研究人员可以更好地理解模型内部的工作机制。
- **注意力权重**：在注意力机制中，每个位置（或“头”）对其他位置的关注程度用一个权重表示。通过分析这些权重，可以了解模型在处理不同任务时如何分配注意力。
- **修剪头（Pruning Heads）**：通过计算每个注意力头的重要性分数，可以移除对模型性能影响较小的头，从而简化模型结构并提高效率。

这些功能和方法为研究 BERT 等模型的内部工作原理提供了有力的支持。

Perplexity —— 困惑度

什么是困惑度?

困惑度（PPL）是评估语言模型性能最常用的指标之一。需要注意的是，这个指标主要用于传统的自回归（或因果）语言模型，而对于像 BERT 这样的掩码语言模型则不太适用（详见模型总结）。

困惑度的定义是对序列的平均负对数似然取指数。如果有分词后的序列 $X = (x_0, x_1, \dots, x_t)$，那么该序列的困惑度 PPL(X) 通过以下公式计算：

$$
\text{PPL}(X) = \exp \left\{ -\frac{1}{t}\sum_{i=1}^t \log p_\theta (x_i|x_{<i}) \right\}
$$

其中，$\log p_\theta (x_i|x_{<i})$ 是根据模型计算的第 i 个词在其前面的词条件下的对数似然。直观上，困惑度可以评估模型在给定语料库中对指定词汇的预测均匀性。需要注意的是，分词方式会直接影响模型的困惑度，因此在比较不同模型时应始终考虑这一点。

困惑度也等价于数据与模型预测之间的交叉熵的指数。有关困惑度及其与每字符比特数（BPC）和数据压缩关系的更深入理解，可以参考这篇博客文章。

使用固定长度模型计算困惑度

如果我们不受模型上下文大小的限制，可以对整个序列进行自回归分解，每次预测时都以前面所有词为条件。

〔图：无限制上下文长度的全序列分解〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/ppl_full.gif，离线电子书未嵌入，详见原仓库）

但在实际应用中，我们通常会受到模型处理的令牌数量的限制。例如，GPT-2 的最大上下文长度为 1024 个令牌，因此当 $t>1024$ 时，我们无法直接计算 $p_\theta(x_t|x_{<t})$。

为了应对这种情况，通常将序列分割成与模型最大输入长度相等的子序列。如果模型的最大输入长度为 k，那么我们只需考虑每个令牌前面的 $k-1$ 个令牌，而不是整个上下文。计算模型困惑度时，一个简单但次优的方法是将序列分解为不重叠的片段，并独立计算每个片段的对数似然。

〔图：未充分利用上下文的困惑度计算〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/ppl_chunked.gif，离线电子书未嵌入，详见原仓库）

这种方法的优点是计算速度快，但作为完全分解困惑度的近似效果较差，通常会得到更高的（更差的）困惑度，因为模型在大多数预测步骤中会有更少的上下文。

更好的方法是使用滑动窗口策略。通过不断滑动上下文窗口，使模型在每次预测时都能获得更多的上下文。

〔图：充分利用上下文的滑动窗口困惑度计算〕

（原始媒体：https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/ppl_sliding.gif，离线电子书未嵌入，详见原仓库）

这种方法更接近真实序列概率的分解，通常会产生更优的结果。缺点是需要为语料库中的每个令牌进行一次前向计算。一种实用的折中方法是采用步长较大的滑动窗口，而不是每次只滑动一个令牌。这可以在保持较大上下文的同时提高计算速度。

示例：使用 🤗 Transformers 和 GPT-2 计算困惑度

让我们用 GPT-2 来演示这个过程。

```
# 导入必要的库和模型
from transformers import GPT2LMHeadModel, GPT2TokenizerFast
from accelerate.test_utils.testing import get_backend

# 自动检测设备类型（CUDA、CPU、XPU、MPS 等）
device, _, _ = get_backend()
model_id = "openai-community/gpt2-large"
model = GPT2LMHeadModel.from_pretrained(model_id).to(device)
tokenizer = GPT2TokenizerFast.from_pretrained(model_id)
```

我们将加载 WikiText-2 数据集，并使用不同的滑动窗口策略评估困惑度。由于该数据集较小，我们可以一次性将整个数据集加载到内存中。

```
from datasets import load_dataset

# 加载和编码数据集
test = load_dataset("wikitext", "wikitext-2-raw-v1", split="test")
encodings = tokenizer("\n\n".join(test["text"]), return_tensors="pt")
```

在 🤗 Transformers 中，我们只需将 `input_ids` 作为 `labels` 传递给模型，它会返回每个令牌的平均负对数似然作为损失。使用滑动窗口方法时，传递给模型的令牌之间会存在重叠，我们不希望将作为上下文的令牌的对数似然包含在损失中，因此可以将这些目标设置为 `-100`，让它们被忽略。以下是一个使用步长为 512 的示例。

```
import torch
from tqdm import tqdm

# 设置最大长度和步长
max_length = model.config.n_positions
stride = 512
seq_len = encodings.input_ids.size(1)

# 初始化变量
nll_sum = 0.0
n_tokens = 0
prev_end_loc = 0

# 迭代滑动窗口
for begin_loc in tqdm(range(0, seq_len, stride)):
    end_loc = min(begin_loc + max_length, seq_len)
    trg_len = end_loc - prev_end_loc  # 在最后一轮可能与步长不同
    input_ids = encodings.input_ids[:, begin_loc:end_loc].to(device)
    target_ids = input_ids.clone()
    target_ids[:, :-trg_len] = -100  # 忽略作为上下文的令牌

    with torch.no_grad():
        # 前向传播并计算损失
        outputs = model(input_ids, labels=target_ids)
        neg_log_likelihood = outputs.loss

    # 累积总负对数似然和有效令牌数量
    num_valid_tokens = (target_ids != -100).sum().item()  # 有效令牌数量
    batch_size = target_ids.size(0)
    num_loss_tokens = num_valid_tokens - batch_size  # 考虑内部标签移位
    nll_sum += neg_log_likelihood * num_loss_tokens
    n_tokens += num_loss_tokens

    prev_end_loc = end_loc
    if end_loc == seq_len:
        break

# 计算平均负对数似然和困惑度
```

```
avg_nll = nll_sum / n_tokens
ppl = torch.exp(avg_nll)
```

当步长等于最大输入长度（1024）时，相当于我们之前讨论的次优的非滑动窗口策略。步长越小，模型在每次预测时获得的上下文越多，报告的困惑度通常会越好。例如，使用 `stride = 1024`（无重叠）时，困惑度为 19.44，接近 GPT-2 论文中报告的 19.93。而使用 `stride = 512`（滑动窗口）时，困惑度降低到 16.44。这种方法不仅能得到更优的分数，还更接近真实的自回归分解。

使用 pipelines 作为 Web 服务器提供推理服务

创建一个推理引擎是一个复杂的话题，"最佳"解决方案通常取决于你的具体需求。你是在使用 CPU 还是 GPU？你追求的是最低延迟、最高吞吐量、支持多个模型，还是只优化一个特定模型？虽然有多种方法可以解决这个问题，但我们提供的是一种通用的入门方案，可能不是最优化的解决方案，但适合初学者使用。

关键概念

我们可以使用一个迭代器来处理请求，就像处理数据集一样，因为 Web 服务器本质上是一个等待请求并逐个处理它们的系统。通常，Web 服务器是多路复用的（多线程、异步等），以并发处理多个请求。然而，推理 pipelines（尤其是底层模型）不太适合并行处理，它们会占用大量内存，所以最好在运行时给它们提供所有可用资源，尤其是计算密集型任务。

方案设计

我们将通过让 Web 服务器处理接收和发送请求的轻量级任务，同时使用单线程来处理实际的推理任务。这里我们使用 `starlette` 框架作为示例，但如果你使用其他框架，可能需要调整代码以实现相同的效果。

创建 server.py

```
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.routing import Route
from transformers import pipeline
import asyncio
```

```
# 定义主页路由处理函数
async def homepage(request):
    # 获取请求体
    payload = await request.body()
    string = payload.decode("utf-8")

    # 创建一个队列用于存储推理结果
    response_q = asyncio.Queue()

    # 将请求字符串和队列放入模型队列中
    await request.app.model_queue.put((string, response_q))

    # 从队列中获取推理结果并返回
    output = await response_q.get()
    return JSONResponse(output)

# 定义服务器循环处理函数
async def server_loop(q):
    # 初始化推理 pipelines
    pipe = pipeline(model="google-bert/bert-base-uncased")

    while True:
        # 从队列中获取请求字符串和结果队列
        (string, response_q) = await q.get()

        # 执行推理并返回结果
        out = pipe(string)
        await response_q.put(out)

# 创建 Starlette 应用实例
app = Starlette(
    routes=[
        Route("/", homepage, methods=["POST"]),
    ],
)

# 定义启动事件
@app.on_event("startup")
async def startup_event():
    q = asyncio.Queue()  # 创建队列
    app.model_queue = q  # 将队列绑定到应用实例
    asyncio.create_task(server_loop(q))  # 创建推理循环任务
```

启动服务器

```
uvicorn server:app
```

测试服务器

```
curl -X POST -d "test [MASK]" http://localhost:8000/
```

响应示例：

这样，你就可以创建一个简单的 Web 服务器来处理推理请求了！

重要注意事项

模型加载

我们只加载一次模型，以避免在服务器上创建多个模型副本，从而节省内存。通过队列机制，你还可以实现一些高级功能，例如在推理之前积累多个请求来进行动态批处理。

动态批处理

下面的代码是一个伪代码示例，用于说明如何实现动态批处理。请注意，这只是一个示例，实际使用时需要根据你的资源进行调整。

```
(string, rq) = await q.get()
strings = []
queues = []

while True:
    try:
        # 尝试在 1 毫秒内获取队列中的请求
        (string, rq) = await asyncio.wait_for(q.get(), timeout=0.001)  # 1ms
    except asyncio.exceptions.TimeoutError:
        break
    strings.append(string)
    queues.append(rq)

# 批量推理
outs = pipe(strings, batch_size=len(strings))

# 将结果放入各自的队列中
for rq, out in zip(queues, outs):
    await rq.put(out)
```

需要考虑的其他问题

错误检查

生产环境中可能会遇到各种问题，例如内存不足、磁盘空间不足、模型加载失败、请求格式错误、模型配置错误等。通常，服务器应该向用户返回错误信息，因此在代码中添加 `try..except` 语句来捕获并显示错误是一个好主意。但请记住，暴露所有错误信息可能会带来安全风险，具体取决于你的安全要求。

断路器

Web 服务器在过载时返回适当的错误（例如 503 或 504）而不是无休止地等待请求，通常会让服务器表现得更好。可以通过检查队列大小来提前返回错误，以防止服务器在高负载下崩溃。

阻塞主线程

目前，PyTorch 不支持异步处理，计算任务会阻塞主线程。因此，最好将 PyTorch 运行在独立的线程或进程中。这虽然会使代码复杂化，但在单个推理任务耗时较长时非常重要。

动态批处理

批处理并不总是比逐个处理更有效（详细信息见 批处理详情）。但在某些情况下，例如处理非常大的模型（如 BLOOM）时，动态批处理是提高推理效率的关键。

模型训练剖析

要理解可以应用于提高模型训练速度和内存利用率的优化技术，了解 GPU 在训练中的使用情况以及不同操作的计算强度变化是很有帮助的。

我们从一个 GPU 使用情况和模型训练运行的示例开始。为了演示，我们需要安装几个库：

```
pip install transformers datasets accelerate nvidia-ml-py3
```

`nvidia-ml-py3` 库允许我们在 Python 中监控模型的内存使用情况。你可能熟悉终端中的 `nvidia-smi` 命令——这个库可以直接在 Python 中访问相同的信息。

接下来，我们创建一些虚拟数据：输入 ID 在 100 到 30000 之间的随机值，以及用于分类的二进制标签。总共生成 512 个长度为 512 的序列，并将其存储在一个使用 PyTorch 格式的 Dataset 中。

```
import numpy as np
from datasets import Dataset

seq_len, dataset_size = 512, 512
dummy_data = {
    "input_ids": np.random.randint(100, 30000, (dataset_size, seq_len)),
    "labels": np.random.randint(0, 2, (dataset_size)),
}
ds = Dataset.from_dict(dummy_data)
ds.set_format("pt")
```

为了打印 GPU 使用情况的摘要统计信息和训练运行情况，我们定义两个辅助函数：

```
from pynvml import *

def print_gpu_utilization():
    nvmlInit()
    handle = nvmlDeviceGetHandleByIndex(0)
    info = nvmlDeviceGetMemoryInfo(handle)
    print(f"GPU 内存占用: {info.used//1024**2} MB.")

def print_summary(result):
    print(f"时间: {result.metrics['train_runtime']:.2f} 秒")
    print(f"每秒样本数: {result.metrics['train_samples_per_second']:.2f}")
    print_gpu_utilization()
```

我们先确认 GPU 内存是空闲的：

```
print_gpu_utilization()
# 输出:
# GPU 内存占用: 0 MB.
```

一切正常：在加载任何模型之前，GPU 内存应该是空闲的。如果你的机器不是这样，请确保停止所有占用 GPU 内存的进程。然而，并不是所有空闲的 GPU 内存都可以被用户使用。当模型加载到 GPU 上时，内核也一同加载，这会占用 1-2GB 的内存。我们通过将一个小型张量加载到 GPU 来触发内核的加载。

```
import torch

torch.ones((1, 1)).to("cuda")
print_gpu_utilization()
# 输出:
# GPU 内存占用: 1343 MB.
```

我们可以看到内核本身占用了 1.3GB 的 GPU 内存。现在我们来看看模型占用了多少空间。

加载模型

首先，我们加载 `google-bert/bert-large-uncased` 模型。我们将模型权重直接加载到 GPU 上，以检查它们占用的内存空间。

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("google-bert/bert-large-uncased").to("cuda")
print_gpu_utilization()
# 输出:
# GPU 内存占用: 2631 MB.
```

我们看到模型权重本身占用了 1.3GB 的 GPU 内存。具体数量取决于你使用的 GPU。请注意，在较新的 GPU 上，模型可能会占用更多空间，因为权重是以优化的方式加载的，以加快模型的使用速度。现在，我们可以快速检查一下 `nvidia-smi` CLI 是否给出相同的结果：

```
nvidia-smi
```

我们得到了与之前相同的数字，并且可以看到我们正在使用一个有 16GB 内存的 V100 GPU。现在我们可以开始训练模型，并观察 GPU 内存使用的变化。首先，我们设置一些标准的训练参数：

```
default_args = {
    "output_dir": "tmp",
    "eval_strategy": "steps",
    "num_train_epochs": 1,
    "log_level": "error",
    "report_to": "none",
}
```

如果你计划运行多个实验，为了正确清除内存，请在实验之间重启 Python 内核。

原始训练的内存使用情况

我们使用 Trainer 来训练模型，不使用任何 GPU 性能优化技术，并设置批大小为 4：

```
from transformers import TrainingArguments, Trainer, logging

logging.set_verbosity_error()

training_args = TrainingArguments(per_device_train_batch_size=4, **default_args)
trainer = Trainer(model=model, args=training_args, train_dataset=ds)
```

```
result = trainer.train()
print_summary(result)
```

我们可以看到即使是相对较小的批大小也几乎填满了 GPU 的全部内存。然而，更大的批大小通常可以加快模型收敛速度或提高最终性能。理想情况下，我们应该根据模型的需求调整批大小，而不是受 GPU 限制。有趣的是，我们使用的内存比模型本身的内存要多得多。为了更好地理解原因，我们来看看模型的操作和内存需求。

模型操作剖析

Transformer 架构包括以下三组主要操作，按计算强度分组：

374. **张量收缩**
    - 线性层和多头注意力组件中的所有操作都涉及批处理的 **矩阵-矩阵乘法**。这些操作是训练变压器最计算密集的部分。
375. **统计归一化**
    - Softmax 和层归一化操作比张量收缩要轻得多，涉及一个或多个 **归约操作**，其结果通过映射应用。
376. **逐元素操作**
    - 剩下的操作是 **偏置、Dropout、激活和残差连接**。这些是最不计算密集的操作。

了解这些信息有助于分析性能瓶颈。

模型内存剖析

我们已经看到训练模型使用的内存远不止将模型加载到 GPU 上的内存。这是因为训练过程中使用的 GPU 内存有多个组件，包括：

377. 模型权重
378. 优化器状态
379. 梯度
380. 用于梯度计算的前向激活
381. 临时缓冲区

382. 功能特定内存

一个典型的使用混合精度和 AdamW 优化器训练的模型需要每个参数 18 字节，加上激活内存。对于推理，没有优化器状态和梯度，因此可以减去这些部分，最终每个参数需要 6 字节加上激活内存。

我们来看一下这些组件的详细信息：

- **模型权重**：
  - fp32 训练：4 字节 × 参数数量
  - 混合精度训练：6 字节 × 参数数量（同时维护 fp32 和 fp16 模型）
- **优化器状态**：
  - 正常 AdamW：8 字节 × 参数数量（维护两个状态）
  - 8 位 AdamW 优化器（如 bitsandbytes）：2 字节 × 参数数量
  - 带动量的 SGD 等优化器：4 字节 × 参数数量（仅维护一个状态）
- **梯度**：
  - fp32 或混合精度训练：4 字节 × 参数数量（梯度始终保存为 fp32）
- **前向激活**：
  - 大小取决于许多因素，关键因素包括序列长度、隐藏大小和批大小。
- **临时内存**：
  - 在计算过程中会释放各种临时变量，但在计算过程中这些变量可能会占用额外的内存，导致内存不足。因此，在编程时要战略性地考虑这些临时变量，并在不再需要时显式释放。
- **功能特定内存**：
  - 例如，使用束搜索生成文本时，软件需要维护多个输入和输出的副本。
- **前向 vs 后向执行速度**：
  - 对于卷积和线性层，后向传播的浮点运算量是前向传播的两倍，通常导致后向传播的速度大约是前向传播的两倍（有时更多，因为后向传播的大小往往更尴尬）。激活通常受带宽限制，通常在后向传播中需要读取更多数据（例如，激活前向传播读取一次，写入一次，激活后向传播读取两次，gradOutput 和前向传播的输出，写入一次，gradInput）。

优化 LLM 部署的有效技术

大型语言模型（LLM）如 GPT3/4、Falcon 和 Llama 在处理人类任务方面能力日益增强，成为现代知识型产业中的重要工具。然而，在实际任务中部署这些模型仍面临挑战：

- 为了接近人类的文本理解和生成能力，LLM 需要包含数十亿个参数，这增加了推理所需的内存需求。
- 在许多实际任务中，LLM 需要处理大量的背景信息，这对模型处理长输入序列的能力提出了要求。

本文将介绍优化 LLM 部署的有效技术：

383. **降低精度**：研究表明，使用 8 位或 4 位的低精度运算可以在不大幅降低模型性能的情况下获得计算优势。
384. **Flash 注意力**：Flash 注意力是一种更高效的注意力算法，能够有效利用 GPU 内存，提高效率。
385. **架构创新**：提出了一些专门针对长时间输入序列优化的模型架构，如 Alibi、旋转位置嵌入（Rotary embeddings）、多查询注意力（MQA）和分组查询注意力（GQA）。

1. 降低精度

我们可以将 LLM 视为一系列权重矩阵和向量，文本输入为一系列向量。每个参数通常存储为 `float32`、`bfloat16` 或 `float16` 格式。例如，对于拥有 X 亿个参数的模型，在 `float32` 精度下加载权重需要大约 4X GB 的显存。

为了减少显存占用，现在大多数模型都是用 `bfloat16` 或 `float16` 训练的。因此，加载权重的显存需求变为 2X GB。

对于较短的文本输入（少于 1024 个标记），推理所需的显存主要由加载权重占用。以下是几个模型所需的显存量：

- **GPT3**：2 *175 GB = 350 GB**
- **Bloom****：2 *176 GB = 352 GB**
- **Llama-2-70b****：2 *70 GB = 140 GB**

- **Falcon-40b**：2 *40 GB = 80 GB*
- **MPT-30b**：2 *30 GB = 60 GB*
- **bigcode/starcoder**：2 *15.5 = 31 GB*

目前市场上的最大 GPU（如 A100 或 H100）提供 80GB 显存，而大多数模型需要更多显存，因此需要使用张量并行或管道并行技术。

使用 `device_map="auto"`可以自动将不同层分配到可用的 GPU 上。此外，可以使用文本生成推理库来编写支持张量并行的模型。

2. Flash 注意力

Flash 注意力通过在计算注意力时动态量化权重，减少了显存需求并提高了计算效率。对于长输入序列，Flash 注意力显著降低了显存占用。

3. 架构创新

为了更高效地处理长时间输入序列，研究人员提出了一些改进模型架构的方法，如：

- **相对位置嵌入**（如 RoPE 和 ALiBi），可以更好地处理长文本输入。
- **多查询注意力（MQA）**，减少了缓存占用。
- **分组查询注意力（GQA）**，在保持性能的同时进一步减少了内存占用。

3.1 改进 LLM 的位置嵌入

绝对位置嵌入（如正弦位置嵌入和学习位置嵌入）对长文本输入效果较差。相对位置嵌入（如 RoPE 和 ALiBi）通过在注意力层中直接编码句子顺序，使模型更好地理解相对位置关系。例如，RoPE 通过旋转向量角度来编码相对位置，而 ALiBi 通过添加负整数来调整注意力分数。

3.2 缓存键值对

自回归文本生成可以通过缓存先前的键值对来减少计算量。使用缓存后，每次只计算当前查询向量与缓存键值对的乘积，显著提高了推理速度并减少了显存占用。

3.2.1 多轮对话

在多轮对话中，缓存键值对可以避免重复计算，使模型能够快速响应后续问题。例如，在多轮对话中，缓存第一轮对话的键值对，第二轮对话只需处理新输入部分，从而使推理速度更快。

3.2.2 多查询注意力（MQA）

MQA 通过共享所有注意力头的键值投影权重，显著减少了缓存占用。例如，对于 16000 个输入序列，使用 MQA 可以将缓存占用从 15 GB 减少到不到 400 MB。

3.2.3 分组查询注意力（GQA）

GQA 通过使用少量键值投影权重（如 2、4 或 8 个），在保持性能的同时减少了内存占用。GQA 还可以通过少量计算将现有模型转换为 GQA 架构。